Space Exploration & Aerospace Engineering

# Part XIII

# Artificial Intelligence in the Space Ecosystem

Artificial intelligence now runs on satellites, plans rover drives, forecasts the weather, screens collision warnings and designs rocket parts. Part XIII is an in-depth guide in eleven chapters: the foundations of machine learning and deep learning, the data and hardware that make AI work in orbit, its applications in Earth observation, weather, autonomy, operations, security, science, exploration, design and business — and the assurance and governance needed to trust it.

**Chapters 43–53** · pages 797–1070 of the complete manual
First Edition · Current to October 2026 · Written in plain language with deep technical detail

ENGINEERING REFERENCE MANUAL

# Space Exploration & Aerospace Engineering

A complete, illustrated guide to the space ecosystem — from stratospheric balloons to interstellar probes — explaining how every major class of space system works, how it is engineered, and where artificial intelligence is changing it.

First Edition · Information current to October 2026
A4 format · Fourteen Parts · Fifty-three chapters · Ten appendices
Version 1.0 · October 2026

## About this edition

**Space Exploration & Aerospace Engineering — An Engineering Reference Manual.** First Edition, Version 1.0, October 2026.

This manual was written to be read by students, engineers, analysts, managers and curious readers. Each topic starts with plain-language explanations and builds up to the equations, numbers and design trade-offs that practising engineers use. Sentences are kept short and every technical term is explained in plain words when it first appears; the glossary (Appendix H) and the list of acronyms (Appendix G) collect them. Facts, mission status and programme milestones are current to October 2026; space programmes change quickly, so dates of future events should be read as plans, not certainties.

## Figures, photographs and data

Diagrams, charts and graphs were drawn specifically for this manual from published physical relationships and public data. Photographs are from the NASA Image and Video Library and are used under NASA's media usage guidelines (NASA material is generally not subject to copyright in the United States); individual credits appear with each image and in Appendix J. NASA does not endorse this publication. Numerical values (masses, power levels, performance) are representative published figures and may differ slightly between sources and mission phases.

## Safety and responsibility

This is an educational reference. It is not a design standard, a flight rule, or a substitute for the requirements, analyses and verification processes of a qualified engineering organisation. Chapters dealing with military and intelligence systems use only openly published information and describe principles, not operational or classified details.

## How numbers are written

SI units are used throughout, with common aerospace units (nautical miles, feet, pounds-force) shown where the industry uses them. Large numbers use thin spacing (e.g., 35 786 km). "≈" means approximately, "~" means order-of-magnitude. Dates follow the ISO format (YYYY-MM-DD) in tables.

## Typography and production

Set in Source Serif 4, Inter and JetBrains Mono. Equations typeset with MathJax. Produced from Markdown sources with an automated pipeline (pandoc, Chromium, pypdf); the sources and build scripts accompany the PDF.

# Contents of Part XIII

## PART XIII · ARTIFICIAL INTELLIGENCE IN THE SPACE ECOSYSTEM


**43 AI Foundations: Machine Learning for Space Engineers ..... 798**
- 43.1 What AI Is — and Is Not ..... 799
- 43.2 Problems, Data and Ways of Learning ..... 803
- 43.3 The Mathematics of Learning ..... 805
- 43.4 Classical Machine Learning ..... 810
- 43.5 Measuring Performance ..... 816
- 43.6 From Notebook to Operations ..... 819
- 43.7 Road Map of Part XIII ..... 821

**44 Deep Learning, Foundation Models and Generative AI ..... 823**
- 44.1 Building Blocks of Deep Networks ..... 824
- 44.2 Convolutional Networks for Images ..... 826
- 44.3 Sequences, Transformers and Graphs ..... 830
- 44.4 Generative Models ..... 834
- 44.5 Foundation Models and Self-Supervised Learning ..... 837
- 44.6 Large Language Models and Agents ..... 840
- 44.7 Physics, Uncertainty and Explanation ..... 842

**45 Data, Computing Hardware and Edge AI in Orbit ..... 849**
- 45.1 Data for Space AI ..... 850
- 45.2 Benchmarks, Competitions and Synthetic Data ..... 853
- 45.3 MLOps: From Archive to Operations ..... 855
- 45.4 Radiation and AI Hardware ..... 858
- 45.5 Processors for AI in Orbit ..... 861
- 45.6 Making Models Fit: Compression ..... 863
- 45.7 On Board or on the Ground? ..... 865
- 45.8 Updating AI in Orbit ..... 867
- 45.9 Computing in Orbit: Orbital Data Centres ..... 869

**46 AI for Earth Observation ..... 874**
- 46.1 From Pixels to Information ..... 875
- 46.2 Land: Crops, Forests, Cities and Water ..... 882
- 46.3 Oceans and Maritime Awareness ..... 884
- 46.4 Disasters ..... 885
- 46.5 Greenhouse Gases and Human Development ..... 889
- 46.6 AI on Board and Smart Tasking ..... 892
- 46.7 The EO Analytics Business ..... 893
- 46.8 Pitfalls and Good Practice ..... 894

**47 AI for Weather, Climate and Space Weather ..... 898**
- 47.1 From Physics to Learning ..... 899
- 47.2 The Machine-Learning Weather Revolution, 2022–26 ..... 900
- 47.3 How Learned Weather Models Work ..... 903
- 47.4 Measuring Skill Fairly ..... 908
- 47.5 Machine Learning in the Observing Chain ..... 910
- 47.6 Climate: Emulators, Downscaling and Digital Twins ..... 912
- 47.7 Space Weather ..... 915

**48 AI for Autonomy, Navigation and Robotics ..... 922**
- 48.1 Why Spacecraft Need Autonomy ..... 923
- 48.2 Inside an Autonomous Spacecraft ..... 925

48.3 Onboard Planning and Scheduling 927
48.4 Navigation Without the Ground 930
48.5 Pose Estimation for Rendezvous and Servicing 934
48.6 Guidance and Control: Optimisation or Learning? 936
48.7 Rendezvous, Docking and Proximity Operations 939
48.8 Robots and Swarms 940

**49 AI for Mission Operations, Communications and Networks 947**
49.1 Mission Operations Today 948
49.2 Watching Telemetry: Anomaly Detection 950
49.3 Prognostics and Health Management 956
49.4 Automating Operations 958
49.5 Scheduling Contacts and Ground Networks 961
49.6 AI in Space Communications 963
49.7 Networks in Orbit 967
49.8 Securing Ground Segments and Links 969

**50 AI for Space Situational Awareness, Defence and Security 972**
50.1 Space Situational Awareness at Machine Scale 973
50.2 Conjunction Assessment and Space Traffic 979
50.3 Watching Behaviour: Manoeuvres, Characterisation and Re-entry 983
50.4 AI in Defence and Intelligence Space Systems 987
50.5 Law, Ethics and Policy 993
50.6 Lessons from the Experiments 994

**51 AI for Space Science, Planetary Exploration and Human Spaceflight 996**
51.1 Why Science Needs Machine Learning 997
51.2 Exoplanets: Finding Small Signals in Big Data 999
51.3 The Time-Domain Sky 1002
51.4 Galaxies, Lenses and Cosmology 1005
51.5 The Sun: Features and Event Catalogues 1007
51.6 Planetary Surfaces and Science Autonomy 1008
51.7 AI for Human Spaceflight 1013
51.8 Validating an AI Discovery 1018

**52 AI in Spacecraft Design, Manufacturing, Launch and the Space Business 1021**
52.1 AI Across the Life Cycle 1022
52.2 Design: Letting Algorithms Propose Shapes 1023
52.3 Exploring the Trade Space 1028
52.4 AI for Systems Engineers 1030
52.5 Manufacturing and Test 1032
52.6 Launch: Where Learning Stays on the Ground 1036
52.7 The Space Business 1039
52.8 Bringing AI into an Engineering Team 1043

**53 Trustworthy AI for Space: Assurance, Security, Governance and Outlook 1046**
53.1 Why Learned Systems Are Hard to Assure 1047
53.2 Standards and Guidance, 2024–26 1049
53.3 Building the Evidence: Assurance Methods 1051
53.4 Security: Attacks on Learned Systems 1059
53.5 Governance, Law and Ethics 1061
53.6 Outlook to 2035 1066
53.7 Part XIII in Review 1068

This volume is one Part of *Space Exploration & Aerospace Engineering — An Engineering Reference Manual*. Page numbers, chapter numbers and figure numbers are the same as in the complete manual, so cross-references to other chapters (for example "Chapter 24") and to the appendices refer to the complete book.

PART XIII

# Artificial Intelligence in the Space Ecosystem

Artificial intelligence now runs on satellites, plans rover drives, forecasts the weather, screens collision warnings and designs rocket parts. Part XIII is an in-depth guide in eleven chapters: the foundations of machine learning and deep learning, the data and hardware that make AI work in orbit, its applications in Earth observation, weather, autonomy, operations, security, science, exploration, design and business — and the assurance and governance needed to trust it.


43 AI Foundations: Machine Learning for Space Engineers

44 Deep Learning, Foundation Models and Generative AI

45 Data, Computing Hardware and Edge AI in Orbit

46 AI for Earth Observation

47 AI for Weather, Climate and Space Weather

48 AI for Autonomy, Navigation and Robotics

49 AI for Mission Operations, Communications and Networks

50 AI for Space Situational Awareness, Defence and Security

51 AI for Space Science, Planetary Exploration and Human Spaceflight

52 AI in Spacecraft Design, Manufacturing, Launch and the Space Business

53 Trustworthy AI for Space: Assurance, Security, Governance and Outlook


Image: NASA/ARC (Dominic Hart)

CHAPTER

# 43 AI Foundations: Machine Learning for Space Engineers

Image: NASA (BRANDON TORRES)

*Artificial intelligence now screens clouds on board satellites, flags anomalies in telemetry, maps the planet's forests and plans rover drives on Mars. This chapter gives space engineers the foundations needed to judge, specify and build such systems. It separates symbolic AI — rules, search and planning — from machine learning, traces how each wave of AI reached space from the expert systems of the 1980s to the foundation models of the 2020s, and maps the main problem types and learning paradigms onto space examples. It then explains, with equations and computed examples, how models learn by gradient descent, why they overfit, how classical methods work, how to measure performance honestly for rare events, and how spatial and temporal autocorrelation can make a useless model look excellent.*

IN THIS CHAPTER

43.1 What AI Is — and Is Not
43.2 Problems, Data and Ways of Learning
43.3 The Mathematics of Learning
43.4 Classical Machine Learning
43.5 Measuring Performance
43.6 From Notebook to Operations
43.7 Road Map of Part XIII

A modern Earth-observation satellite can collect more data in a day than its operators could inspect in a lifetime. NASA's Earth science archive alone held 178.7 PB at the end of September 2025. It was growing by about 160 TB a day, and a large mission may record thousands of telemetry channels every second for decades. Nobody can write explicit rules for every pattern hidden in such data. **Machine learning** — programs that improve their performance by learning from examples — has therefore become as much a part of the space engineer's toolkit as orbital mechanics or link budgets.

Machine learning is also easy to misuse. A model can score 99 % accuracy and be useless; it can pass every validation test and fail on the first image from a new region. This chapter builds the foundations needed to avoid these traps: what artificial intelligence (AI) is and is not, how machines learn, which classical methods suit which problems, and how to measure performance honestly. The ten chapters that follow apply these ideas across the space ecosystem.

## 43.1 What AI Is — and Is Not

**Artificial intelligence** is the field of building computer systems that perform tasks which normally require human intelligence: perceiving, reasoning, planning, deciding and learning. The term was coined for a workshop at Dartmouth College in 1956. It implies neither consciousness nor understanding, and it is not one technique but a family of them (Figure 43.1).

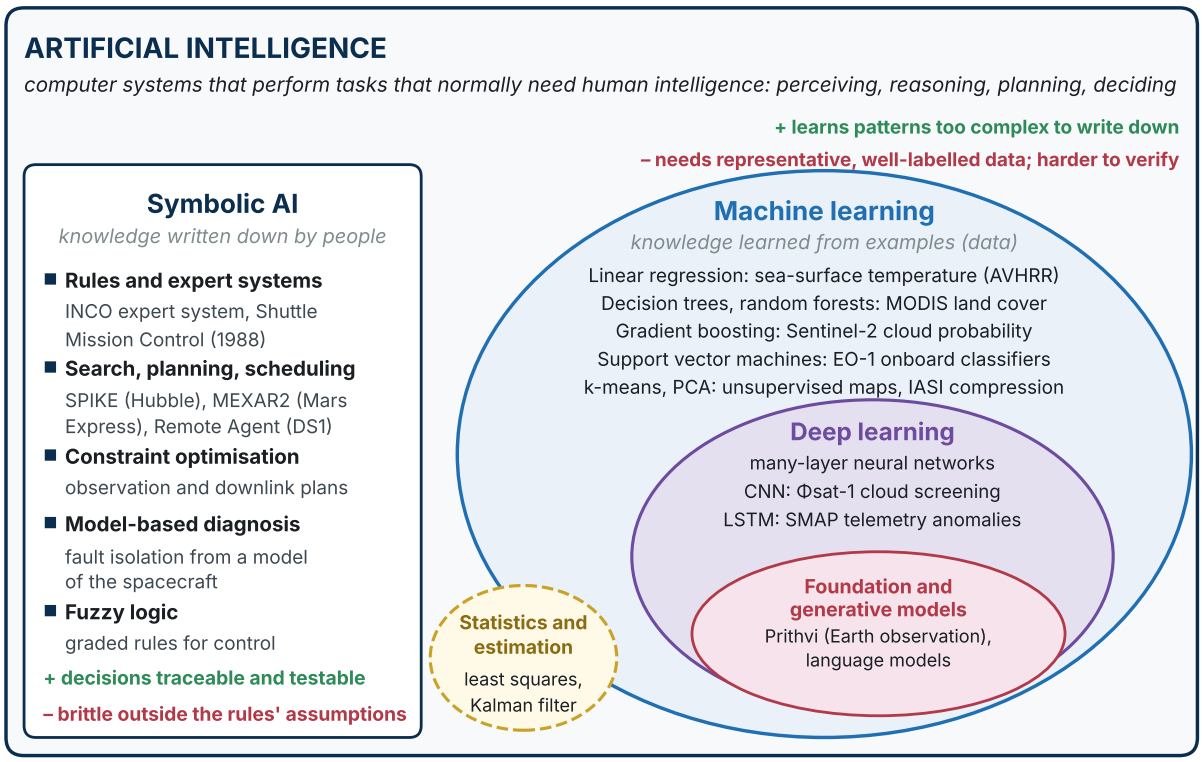



**FIGURE 43.1** How the families of AI relate. Symbolic AI encodes knowledge written down by people; machine learning learns it from data; deep learning uses many-layer neural networks; foundation models are large networks pre-trained on huge unlabelled data sets. Statistics and estimation theory overlap machine learning. All examples are space systems discussed in this chapter.

### 43.1.1 Symbolic and Learned AI

**Symbolic AI** represents knowledge explicitly, in a form that people write and read: rules (“if the bus voltage is below 26 V and the battery is discharging, shed heater loads”), facts, models and search procedures. Its main branches are **expert systems**, which apply hundreds or thousands of if–then rules elicited from specialists, and **search and planning**, which find a sequence of actions that reaches a goal within constraints. **Constraint optimisation** builds schedules that satisfy many conditions at once. **Model-based diagnosis** explains symptoms by reasoning over a model of the hardware. In **fuzzy logic**, rules hold to a degree between true and false.

**Machine learning (ML)** takes the opposite approach: the engineer provides examples and an algorithm finds a function that fits them. A cloud detector is not told that "clouds are bright in the visible and moderately bright at 1.6 μm". Instead, it is shown thousands of pixels labelled "cloud" or "clear" and works out a boundary between them. **Deep learning** uses neural networks with many layers, which learn their own features from raw images, spectra or time series (Chapter 44). **Foundation models** are very large deep networks pre-trained on enormous unlabelled data sets and then adapted to many tasks.

Table 43.1 compares the two traditions. The boundary is not sharp: a rover's planner may call a learned terrain classifier, and a learned model may be wrapped in hand-written safety rules. Many of the strongest space systems combine both.

**TABLE 43.1** Symbolic AI and machine learning compared.

| Aspect | Symbolic AI | Machine learning |
|---|---|---|
| Source of knowledge | Experts write rules, models, constraints | Algorithm infers patterns from examples |
| Typical methods | Expert systems, planners, schedulers, fuzzy logic | Regression, trees, forests, SVMs, clustering, neural networks |
| Space examples | Shuttle INCO expert system, Hubble's SPIKE, Mars Express MEXAR2 | Sea-surface temperature regression, MODIS land cover, onboard cloud screening |
| Strengths | Transparent, predictable, verifiable against requirements | Handles patterns too complex to write down; improves with data |
| Weaknesses | Brittle outside its assumptions; costly to maintain | Needs representative labelled data; can fail silently on new inputs |

### 43.1.2 Why Symbolic Methods Still Matter in Space

Spaceflight puts a premium on **verifiability**. Flight software must be shown, before launch, to do what its requirements say in every mode the mission can enter (Chapter 6). A rule base or planner can be reviewed line by line and sometimes proved correct with formal methods. A neural network's behaviour is known only where it has been tested, and engineers must argue statistically about everything in between (Chapter 53).

Safety-critical layers — fault protection, safe modes, attitude control — are therefore still built from explicit logic and classical control. Learned models are used where their errors are tolerable or can be caught: choosing images to downlink, flagging telemetry for a human, proposing plans that a validated planner checks. On 8 and 10 December 2025 Perseverance drove 210 m and 246 m along waypoints generated by a generative AI model. Every command was first checked on a digital twin of the rover that tracked more than 500 000 telemetry variables.

### 43.1.3 Four Waves of AI in Space

AI reached space in waves, each riding on progress elsewhere in computing (Figure 43.2).

**Expert systems and schedulers (1980s–1990s).** In 1985 NASA's Johnson Space Center began developing **CLIPS**, a rule engine written in C because the LISP-based tools of the day rarely left the laboratory; it is still maintained. In 1988, during STS-26, two expert systems ran in Mission Control for the first time. One covered the Shuttle's communications and instrumentation and was used by the INCO flight controller; the other covered its main engines. Both ran beside

conventional displays so that controllers could compare them (Figure 43.3). Hubble's long-range observing plan has been built since shortly after its launch in April 1990 with **SPIKE**, a scheduler from the Space Telescope Science Institute (Figure 43.4). SPIKE expresses each constraint — Sun and Moon avoidance, target visibility, guide-star availability — as a "suitability" between 0 and 1 over time and searches for the schedule with the highest total suitability. In May 1999 the Remote Agent experiment took control of Deep Space 1 (Chapter 48).

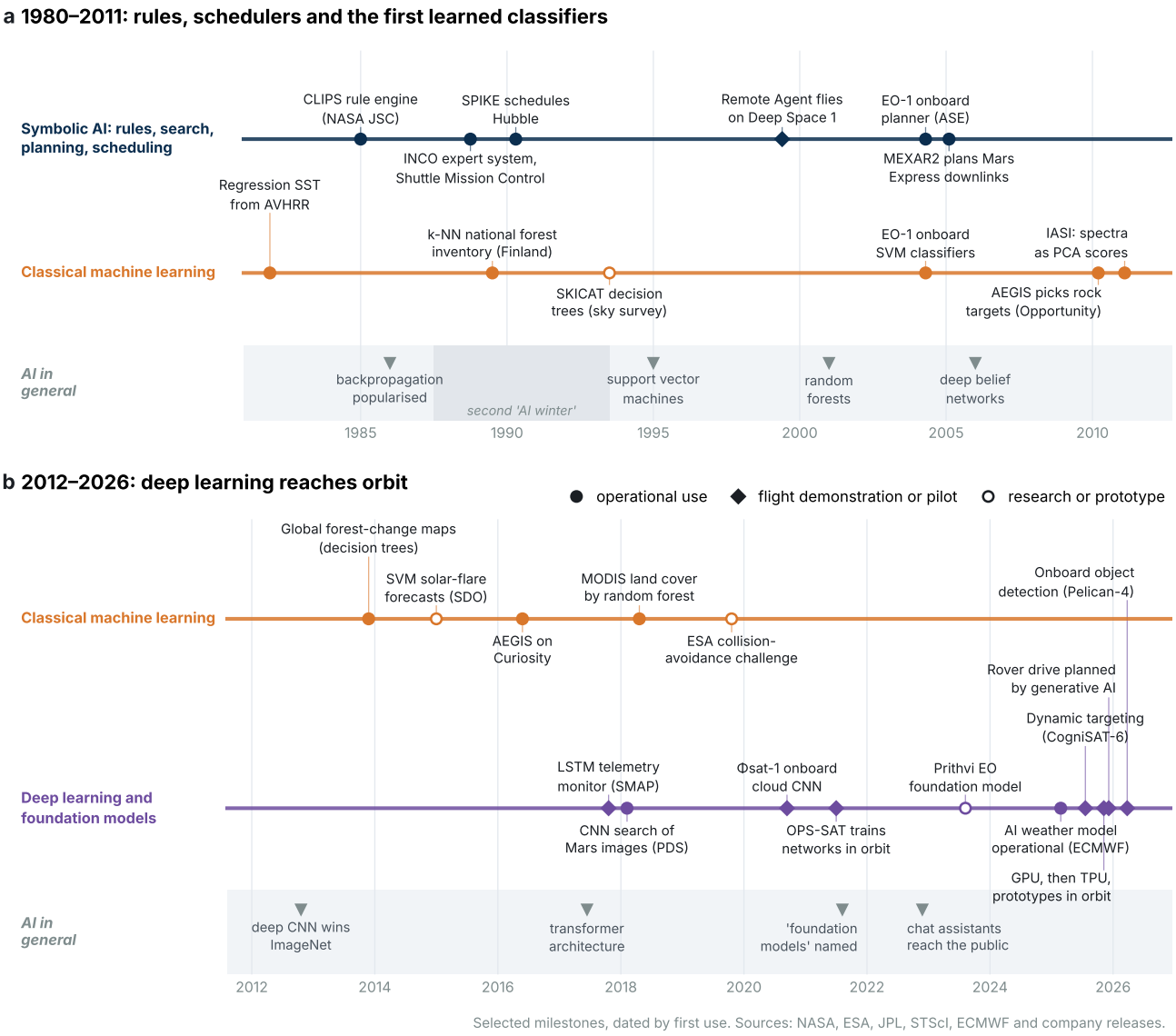



**FIGURE 43.2** Selected milestones of artificial intelligence in space, 1980–2026, sorted into symbolic AI, classical machine learning and deep learning, with general milestones of AI along the bottom. Filled circles mark operational use, diamonds flight demonstrations or pilots, open circles research or prototypes.

**FIGURE 43.3** Flight controller John Muratore at the INCO console in June 1988, during a simulation for STS-26. Beside him an operator works at the Real Time Data System, an expert-system workstation that monitored Shuttle telemetry. Image: NASA/JSC.

**FIGURE 43.4** Hubble above Discovery's payload bay before its release in April 1990 (STS-31). Its long-term observing schedule has been built ever since with SPIKE, a constraint-based AI scheduler. Image: NASA/JSC.

**Classical machine learning (1980s–2010s).** Statistical learning arrived quietly, often without the AI label. From the early 1980s NOAA computed sea-surface temperature from AVHRR's thermal channels with coefficients found by regression against drifting buoys. In the early 1990s JPL and Caltech's SKICAT used decision trees to classify faint stars and galaxies on the digitised Palomar sky survey. It reached about 94 % accuracy on objects too faint to classify by eye. From 2003–04 Earth Observing-1 ran onboard classifiers and a planner that retasked it automatically, and from 2010 the AEGIS software let Opportunity and later Curiosity choose their own rock targets.

**Deep learning (2012–).** In 2012 a deep convolutional network, AlexNet, cut the error rate of the ImageNet image-recognition contest from 26.2 % to 15.3 %. It showed that networks trained on graphics processors (GPUs) could learn better visual features than any designed by hand. In 2017 NASA's SMAP team began piloting a recurrent neural network that flags telemetry anomalies, and in 2018 JPL used networks pre-trained on everyday photographs to make NASA's Mars images searchable by content. Launched in September 2020, ESA's **Φsat-1** demonstrated a deep network screening its images for cloud on board, on a low-power Intel Movidius Myriad 2 vision processor.

**Foundation models and generative AI (2020s).** The **transformer** architecture (2017) made it practical to train networks with billions of parameters on unlabelled data; "foundation model" was coined in 2021. NASA and IBM released the Prithvi geospatial foundation model in 2023; its second version (December 2024) has 300 or 600 million parameters, pre-trained on about 4.2 million samples of harmonised Landsat and Sentinel-2 imagery. In February 2025 ECMWF made a machine-learned global weather model operational (Chapter 47). In December 2025 a satellite carrying a data-centre GPU trained a small language model in orbit. Planet's Pelican-4 detected aircraft in its own images over Alice Springs on 25 March 2026. On 1 October 2026 Google and Planet launched a prototype carrying four tensor processing units for a year-long test (Chapter 45). Most of these are **demonstrations**; operational systems still rest mainly on classical methods and carefully bounded deep networks.

## 43.2 Problems, Data and Ways of Learning

### 43.2.1 Ten Kinds of Problem

Before choosing an algorithm, an engineer must say what output is wanted. Table 43.2 maps ten problem types onto space examples.

**TABLE 43.2** Machine-learning problem types and space examples.

| Problem type | Output | Space example | Typical methods | Typical metric |
|---|---|---|---|---|
| Classification | A class for each input | Cloud or clear pixel; land-cover type | Boosted trees, forests, CNNs | Precision, recall, F1 |
| Regression | A continuous value | Sea-surface temperature; forest biomass | Linear regression, forests, Gaussian processes | RMSE, MAE, bias |
| Semantic segmentation | A class for every pixel, keeping shapes | Flood extent, burn scars, buildings | Convolutional encoder–decoders | Intersection over union |
| Object detection | Boxes and classes | Ships in radar images, aircraft | CNN detectors | Mean average precision |
| Forecasting | Future values or event probabilities | Solar flares, geomagnetic storms, weather | Trees, SVMs, recurrent and graph networks | Skill scores, Brier score |
| Anomaly detection | "Normal" or "unusual" | Telemetry faults, radio interference | Prediction errors, isolation forests | Event precision and recall |
| Clustering | Groups found without labels | Spectral classes, solar-wind types | k-means, Gaussian mixtures | Purity, Rand index |
| Dimensionality reduction | A few variables summarising many | 8 461-channel IASI spectra sent as about 300 numbers | PCA, autoencoders | Variance explained |
| Ranking | An ordering by value | Which rock to zap, which image to downlink first | Scoring models | Precision of top items |
| Decision and control | Actions | Scheduling, rover driving, pointing | Planners, reinforcement learning | Reward, mission return |

Anomaly detection and ranking show ML supporting people (Figure 43.5, Figure 43.6). JPL's telemetry monitor for SMAP and Curiosity, described in 2018, used a recurrent network to predict each of 82 telemetry channels from its recent history. It flagged values that departed from

the prediction by more than an automatically set threshold. On 105 labelled anomaly sequences it reached a precision of 87.5 % and a recall of 80 %. Even so, false positives kept it from becoming central to telemetry review. Curiosity's AEGIS ranks candidate rocks in navigation-camera images by criteria set by scientists and sends the best to the ChemCam laser. Over 2.5 km of driving it picked the preferred material more than 93 % of the time, against about 24 % expected without it.

**FIGURE 43.5** NASA's SMAP satellite being weighed before launch in November 2014. Since 2017 its operations team has piloted a neural-network system that flags anomalies in its telemetry. Image: NASA/KSC.

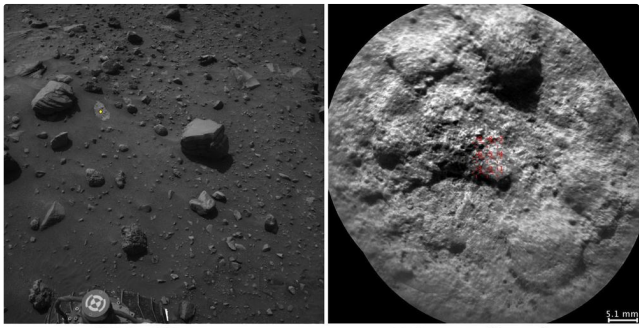



**FIGURE 43.6** Ranking on Mars: after a 13 m drive on 14 July 2016, Curiosity's AEGIS software picked the bright outcrop marked in this navigation-camera image (left); within 30 minutes ChemCam had fired its laser at nine points on it (right). Image: NASA/JPL (NASA/JPL-Caltech/LANL/CNES/IRAP/LPGNantes/CNRS/IAS).

### 43.2.2 Data, Features and Labels

Every ML problem starts with **data**: examples described by **features** $\boldsymbol{x}$ (reflectances, telemetry values, orbital elements) and usually a **label** $\boldsymbol{y}$ (the right answer). Good features encode physics. Clouds and snow are both bright at visible wavelengths, but snow absorbs strongly near 1.6 µm while water clouds do not (Chapter 13). A classifier given only visible bands confuses them (Figure 43.7), while one given a short-wave infrared band can separate them. This chapter's examples use this two-band problem, with synthetic pixels from eight surface and cloud types.

Labels are usually the scarcest ingredient. They come from experts drawing polygons, field campaigns and in-situ sensors, volunteers, simulations and operations logs, and they are never perfect. In the EO-1 study described below, the classifiers correctly found small lakes that the expert had missed — and were scored as wrong. Label errors set a floor on the achievable error and give flexible models something false to memorise.

### 43.2.3 Ways of Learning

The learning paradigms of Table 43.3 differ in the supervision they need, and are often combined: a network may be pre-trained without labels and fine-tuned with a few thousand.

**TABLE 43.3** Learning paradigms.

| Paradigm | Labels needed | How it learns | Space example |
|---|---|---|---|
| Supervised | A label for every example | Fits a mapping from features to labels | EO-1 onboard classifiers; cloud masks |
| Unsupervised | None | Finds clusters or low-dimensional structure | Spectral clustering; PCA of IASI spectra |

| Paradigm | Labels needed | How it learns | Space example |
|---|---|---|---|
| Self-supervised | None — data supply their own targets | Predicts hidden parts of the input | Prithvi, pre-trained on 4.2 million image samples |
| Semi-supervised | A few labels, many unlabelled examples | Extends labels to similar data | Mapping from a handful of field plots |
| Active learning | Chosen by the model | Asks for labels where most uncertain | Galaxy Zoo: 35–60 % fewer labelled galaxies |
| Transfer learning | Few, for the new task | Reuses a model trained on another task | Photo-trained networks fine-tuned on Mars images |
| Reinforcement learning | None; a reward instead | Learns actions by trial and error | Scheduling and guidance research (Chapter 48) |

Galaxy Zoo shows the value of combining people and machines. Launched in July 2007, it attracted about 150 000 volunteers who made some 50 million galaxy classifications in its first year. Later, Bayesian convolutional networks sent the galaxies they were least sure about to volunteers. This **active learning** cut the number of labelled galaxies needed by 35–60 %. **Reinforcement learning (RL)**, in which an agent learns by trial and error to maximise a reward, is covered in Chapter 48.

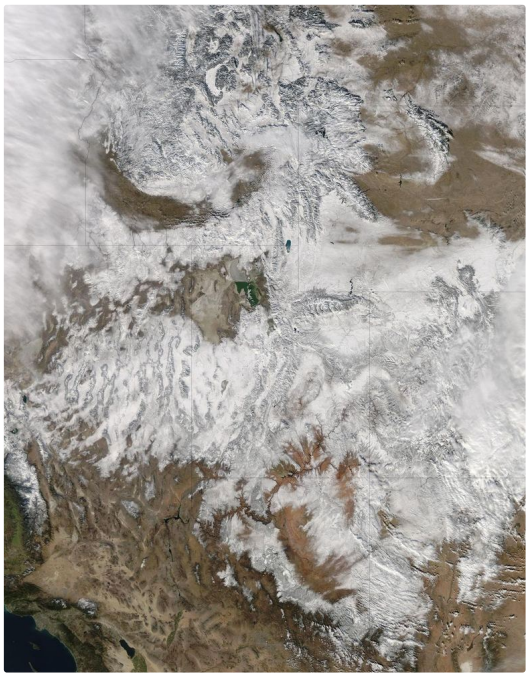


**FIGURE 43.7** Snow and cloud over the Rocky Mountains in a natural-colour MODIS image from Aqua, 19 December 2012. Both are bright in the visible; telling them apart needs a short-wave infrared band, in which snow is dark. Image: NASA/GSFC.

## 43.3 The Mathematics of Learning

### 43.3.1 Models, Losses and Empirical Risk

A **model** is a function $f_\theta(x)$ with adjustable **parameters** $\theta$ — two coefficients in a straight-line fit, millions of weights in a neural network. A **loss function** $\ell(f_\theta(x), y)$ measures how wrong one prediction is. For regression the usual choice is the squared error $(f_\theta(x) - y)^2$. For classification the model outputs a probability $p$ that the label is 1, and the loss is the **cross-entropy**

$$\ell = -\,[\,y \ln p + (1-y)\ln(1-p)\,],$$

which is zero for a confident correct answer and grows without limit for a confident wrong one. Learning means choosing the parameters that minimise the average loss over the $N$ training examples, the **empirical risk**:

$$\hat{R}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\big(f_\theta(x_i), y_i\big).$$

What we really want is a small loss on future data, the **expected risk**; the difference between the two — the **generalisation gap** — is the central difficulty of machine learning.

### 43.3.2 Gradient Descent

For a few models, such as ordinary least squares, the minimum has a closed form; in general it is found iteratively by **gradient descent**. The gradient $\nabla_\theta \hat{R}$ points uphill on the loss surface, so each step moves the parameters a little way downhill (Figure 43.8):

$$\theta_{t+1} = \theta_t - \eta\, \nabla_\theta \hat{R}(\theta_t),$$

where the **learning rate** $\eta$ sets the step size. Computing the full gradient over millions of examples at every step is wasteful, so **stochastic gradient descent (SGD)** estimates it from a small random **mini-batch** $B$:

$$\theta_{t+1} = \theta_t - \frac{\eta}{|B|}\sum_{i \in B} \nabla_\theta\, \ell\big(f_\theta(x_i), y_i\big).$$

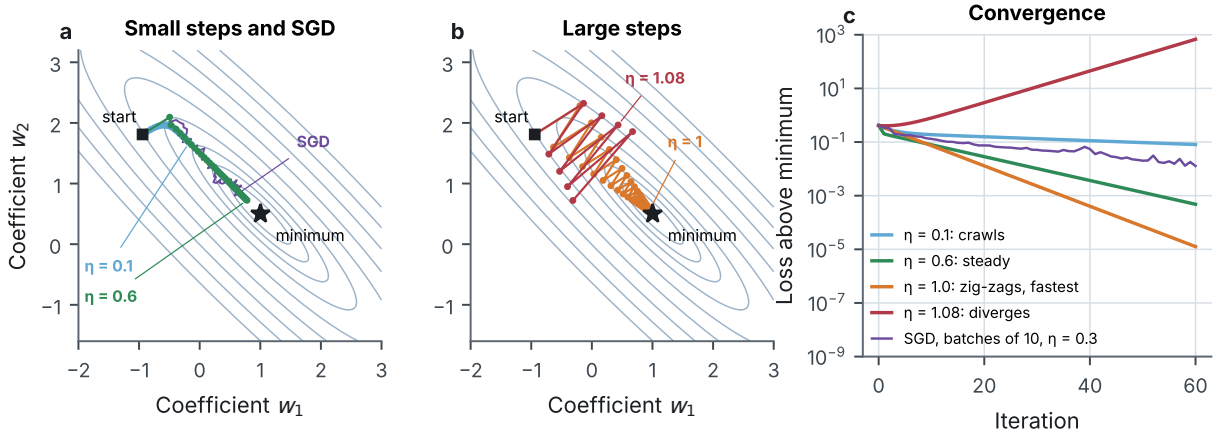



**FIGURE 43.8** Gradient descent on the least-squares loss of a linear model with correlated predictors (r = 0.9). (a) Small steps crawl; stochastic gradient descent (mini-batches of 10) wanders near the minimum. (b) Steps near the stability limit zig-zag; η = 1.08 diverges. (c) Excess loss after 60 iterations: 1.3 × $10^{-5}$ for η = 1.0, 5 × $10^{-4}$ for η = 0.6, 0.08 for η = 0.1.

Each step is noisy but cheap. Practical optimisers such as **Adam** add momentum (an average of recent gradients) and scale each parameter's step by the size of its recent gradients.

The learning rate is the most important setting. Figure 43.8 shows gradient descent on the least-squares loss of a linear model with two strongly correlated predictors, which stretch the loss surface into a narrow valley. Near a minimum any loss surface behaves like a quadratic with curvatures given by the eigenvalues $\lambda$ of its Hessian (here 0.083 and 1.92), and gradient descent is stable only if $\eta < 2/\lambda_{\max}$ = 1.04. A small step crawls along the valley; a step near the limit zig-zags

yet converges fastest; a slightly larger one diverges. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$, here 23, is the **condition number**: the larger it is, the slower the convergence, which is why inputs are routinely **standardised** to zero mean and unit variance.

**EXAMPLE: ONE GRADIENT-DESCENT STEP BY HAND**

A logistic-regression cloud detector computes $p = \sigma(w_1 x_1 + w_2 x_2 + b)$, where $x_1$ and $x_2$ are the visible and 1.6 µm reflectances and $\sigma(z) = 1/(1+e^{-z})$. It starts with all parameters zero and sees one cloudy pixel, $x = (0.70, 0.45)$, $y = 1$, with learning rate $\eta = 0.5$.

1. **Prediction.** $z = 0$, so $p = 0.5$ and the cross-entropy loss is $-\ln 0.5 = \mathbf{0.693}$.
2. **Gradient.** For logistic regression $\partial \ell/\partial w_j = (p - y)\, x_j$ and $\partial \ell/\partial b = p - y$. Here $p - y = -0.5$, so the gradients are $(-0.35, -0.225)$ for the weights and $-0.5$ for the bias.
3. **Update.** $w \leftarrow w - \eta \nabla_w \ell = (0.175, 0.1125)$ and $b \leftarrow 0.25$.
4. **Check.** Now $z = 0.175 \times 0.70 + 0.1125 \times 0.45 + 0.25 = 0.423$, $p = 0.604$ and the loss has fallen to **0.504**. But a clear snow pixel, $x = (0.80, 0.10)$, now also gets $p = 0.599$: the model has learned "bright means cloud". Only more examples — including snow — and a model able to bend its boundary can fix this.

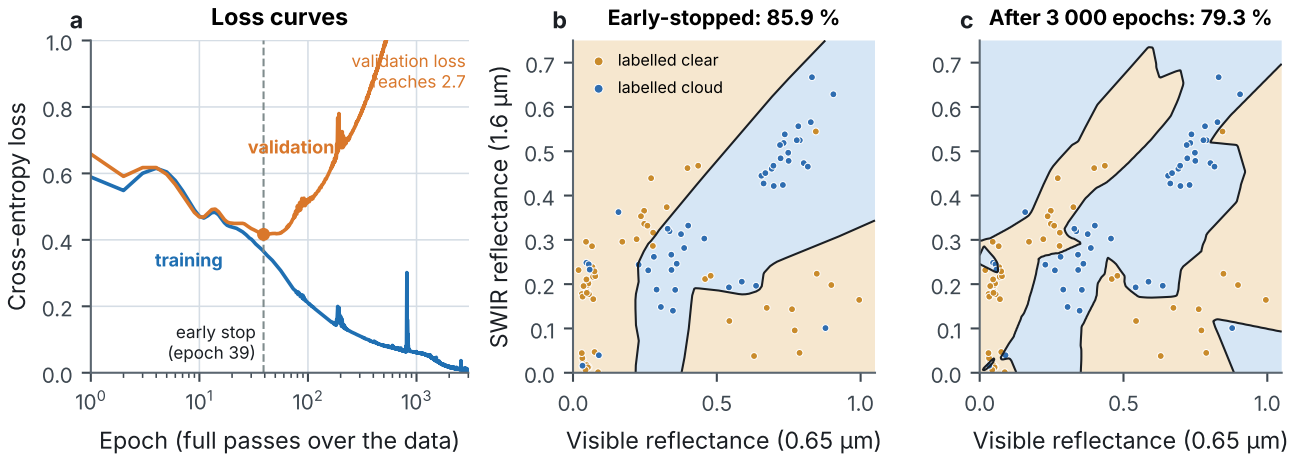
### 43.3.3 Neural Networks and Backpropagation

A **neural network** builds a flexible function by stacking simple layers. Each layer multiplies its input vector by a weight matrix, adds a bias and applies a non-linear **activation function**; the popular **ReLU** (rectified linear unit) replaces negative values by zero. A **multilayer perceptron (MLP)** with two hidden layers computes

$$h_1 = \mathrm{ReLU}(W_1 x + b_1), \qquad h_2 = \mathrm{ReLU}(W_2 h_1 + b_2), \qquad p = \sigma(w_3^{\top} h_2 + b_3).$$

Training needs the gradient of the loss with respect to every weight, computed by **backpropagation**. A forward pass stores each layer's output. A backward pass then applies the chain rule layer by layer, starting from $\partial \ell/\partial z = p - y$ at the output and multiplying by each layer's weights and activation slopes on the way back. The backward pass costs only about twice the forward pass, which is why networks with billions of parameters can be trained (Chapter 44).

Figure 43.9 shows such a network, written from scratch in numpy, learning the cloud-or-clear problem from only 100 pixels, one label in ten deliberately wrong. At first training and validation losses fall together. Then the network starts to fit the mislabelled pixels: training loss heads towards zero, while the loss on 2 000 independent validation pixels climbs from 0.42 to 2.7. This is **overfitting**, and the simplest cure is **early stopping** — keeping the weights from the epoch with the lowest validation loss, here epoch 39. The early-stopped network classifies 85.9 % of 20 000 test pixels correctly; trained to the end, it scores 99 % on its own training data but only 79.3 % on the test pixels.




Numpy network 2–64–64–1 (ReLU, Adam, step 0.01) trained on 100 synthetic pixels with 10 % wrong labels; test set 20 000 pixels.

**FIGURE 43.9** A numpy neural network (two hidden layers of 64 ReLU units, about 4 400 parameters) trained by backpropagation on 100 synthetic cloud and clear pixels, one label in ten wrong. (a) Validation loss is lowest at epoch 39, then rises as the network memorises label errors. (b) The early-stopped network: 85.9 % of 20 000 test pixels correct. (c) After 3 000 epochs the boundary wraps around mislabelled pixels: 79.3 %.

### 43.3.4 Regularisation, Bias and Variance

Any technique that discourages a model from fitting noise is **regularisation**. Early stopping is one; another adds a penalty on large weights, $\hat{R}(\theta) + \lambda\|\theta\|^2$ (**weight decay**, or **ridge** regression for linear models); the penalty $\lambda \sum_j |\theta_j|$ (**lasso**) drives unimportant weights to zero. Networks are also regularised by **dropout**, which randomly switches off units in training, and **data augmentation** with rotated or noisier copies of images; trees, by limiting depth.

The **bias–variance trade-off** explains why regularisation works. For squared loss, the expected error on new data splits into three parts:

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f(x)\big)^2}_{\text{bias}^2} + \underbrace{\operatorname{Var}\big[\hat{f}(x)\big]}_{\text{variance}} + \sigma^2 .$$

**Bias** comes from assumptions too simple for the problem; **variance** is how much the fitted model changes between training sets; $\sigma^2$ is the **irreducible error** from noise in data and labels. Simple models underfit (high bias); flexible models overfit (high variance). Figure 43.10 (a) shows the trade-off for decision trees on 1 000 pixels: the cross-validated error is lowest, 15.1 %, at a depth of 5. Panel (b) shows **learning curves** — test error against training-set size. Logistic regression levels off at about 31 % whatever the amount of data, because a straight line cannot separate snow from cloud. A random forest and a k-nearest-neighbour classifier reach 12.3 % and 12.1 % with 20 000 pixels. That is close to the irreducible 11.9 % set by overlapping classes and 10 % label noise. Learning curves show whether more labels or a better model is needed.

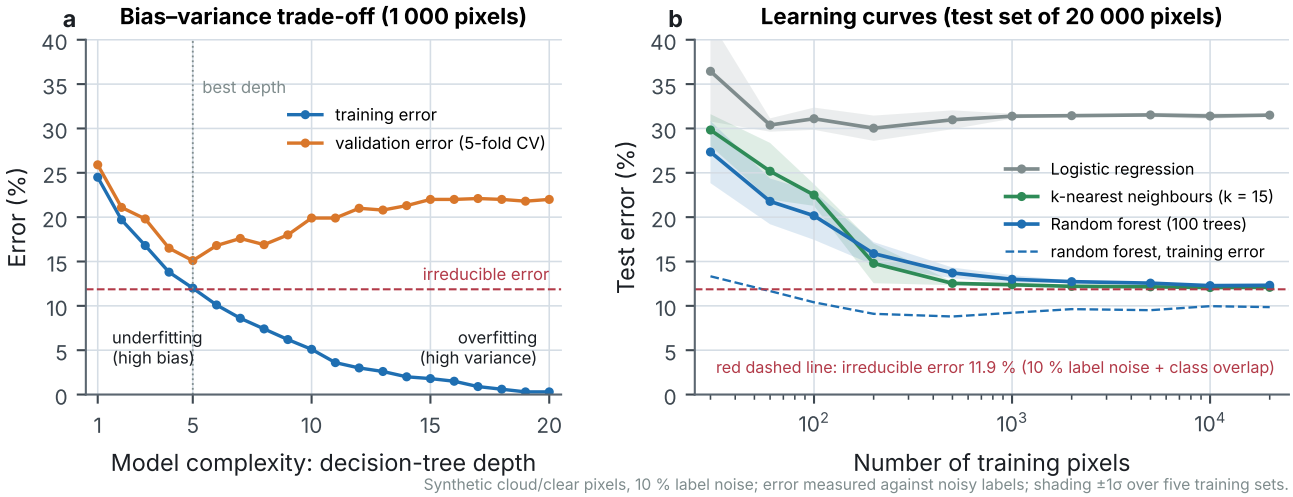



**FIGURE 43.10** (a) The bias–variance trade-off for decision trees fitted to 1 000 synthetic cloud and clear pixels: cross-validated error is lowest (15.1 %) at depth 5 and rises to about 22 % for deep trees. (b) Learning curves: flexible models approach the irreducible error of 11.9 % as training data grow; logistic regression stays near 31 % (high bias).

### 43.3.5 Training, Validation and Test Sets

Labelled data are split three ways. The **training set** fits the parameters. The **validation set** is used to choose **hyper-parameters** — settings that are not learned, such as the learning rate, network size or tree depth — and to decide when to stop. The **test set** is locked away and used once, at the end; every look at it that influences a design decision makes the final estimate optimistic. When data are scarce, **k-fold cross-validation** trains the model $k$ times, each time testing on a different fifth or tenth of the data, and averages the scores, with the test set still untouched.

### 43.3.6 Leakage: When Space Data Cheat

**Data leakage** occurs when information about the test data reaches the model during training, so that the test score no longer measures performance on genuinely new inputs. The most dangerous leak in space applications is **autocorrelation**. Neighbouring pixels are alike, and so are consecutive telemetry samples or successive days of solar activity. With a random split, almost every test sample has a near-twin in the training set. The model can then score well by remembering places or times, rather than by learning physics that would carry it to a new region or season.

Figure 43.11 makes the effect concrete. A random forest learns a synthetic target made of a genuine function of one predictor plus a stronger smooth spatial signal (correlation length 12 km) that no predictor explains. Five further predictors are smooth random fields unrelated to the target — but they let the forest recognise *where* a sample is. Random five-fold cross-validation reports an excellent $R^2$ of 0.80. Cross-validation with **spatial blocks** larger than the correlation length reports about 0.25. Tested in an unseen region, the model scores −0.39 — worse than predicting the regional average — while the true relationship explains only 17 % of the variance there. The forest had learned a map, not a law of nature.

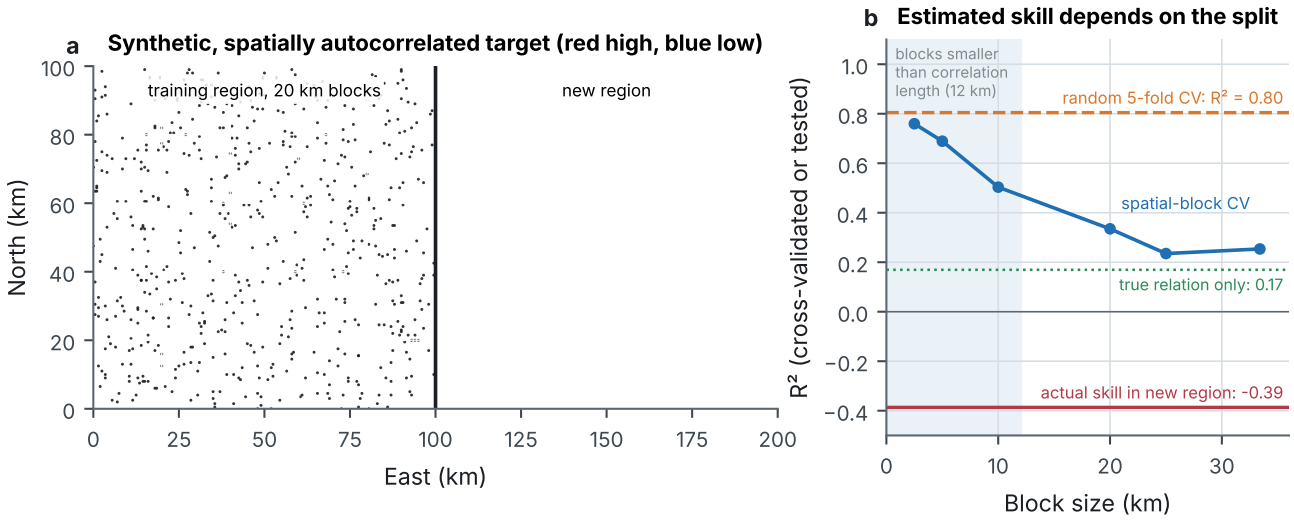



**FIGURE 43.11** Leakage through spatial autocorrelation, computed with a random forest on a synthetic 200 km × 100 km field. (a) The target, with training samples in the western half and test samples in an unseen eastern region. (b) Random five-fold cross-validation reports $R^2$ = 0.80 and spatial-block cross-validation about 0.25; the actual skill in the new region is −0.39, and even the true relationship explains only 17 % of the variance there.

**DATA BOX — SPATIAL VALIDATION OF BIOMASS MAPS**

- In 2020 Ploton and colleagues rebuilt a typical mapping workflow: a random forest predicting forest biomass in central Africa from 9 MODIS and 27 environmental predictors, trained on an inventory of 11.8 million trees.
- With random 10-fold cross-validation the model appeared to explain more than half of the variation ($R^2$ = 0.53); with spatial cross-validation, only 14 % ($R^2$ = 0.14). For test plots more than about 100 km from any training plot, its predictive power was close to zero; biomass was spatially autocorrelated up to about 120 km.
- Predictors with no real relationship to the target, the authors concluded, can appear powerful simply because they vary smoothly in space.

The cure is to split the data the way the model will be used. Maps that must be predicted far from training sites need **spatial blocks**, or **buffers** that remove training samples near each test sample, larger than the range of autocorrelation (estimated from a variogram of the residuals). Forecasts need **forward chaining**: train on the past, test on later periods only. Products from several satellites or instruments need **grouped** splits that keep each orbit, pass, sensor or mission within one fold.

## 43.4 Classical Machine Learning

Deep networks dominate image and language tasks, but most operational space products still rest on the classical methods of Table 43.4. They train quickly on modest data and hardware and are easier to verify.

**TABLE 43.4** Classical machine-learning methods and their space uses.

| Method | Key idea | Strengths | Limitations | Space use (status) |
|---|---|---|---|---|
| Linear regression | Weighted sum fitted by least squares | Fast, interpretable | Linear only | AVHRR sea-surface temperature (operational since 1980s) |
| Logistic regression | Sigmoid of a weighted sum | Probabilities; baseline | Linear boundary | Baselines for flare and fire detection |
| k-nearest neighbours | Vote of the $k$ closest examples | No training | Slow; few features | Finnish forest inventory (operational since about 1990) |
| Decision tree | Sequence of threshold tests | Readable | Unstable; overfits | SKICAT (1990s); MODIS land cover (2000s) |
| Random forest | Average of randomised trees | Robust, little tuning | Large; opaque | MODIS land cover since Collection 6 |
| Gradient boosting | Trees added to correct errors | Top accuracy on tables | Needs tuning | Sentinel-2 cloud probability (operational) |
| Support vector machine | Maximum-margin boundary | Good with few samples | Slow on big data | EO-1 onboard classifiers (2004–17) |
| k-means | Nearest-centroid clustering | Fast, simple | Choice of $k$ | Unsupervised image classification |
| PCA | Main directions of variance | Compression, denoising | Linear only | IASI spectra as PC scores (since 2011) |
| Gaussian process | Prior over functions | Honest error bars | Cost grows as $N^3$ | Stellar-activity models (research) |

### 43.4.1 Linear and Logistic Regression

**Linear regression** predicts $\hat{y} = w^{\top}x + b$ and chooses the weights that minimise the squared error, with the closed-form solution $w = (X^{\top}X)^{-1}X^{\top}y$. NOAA's **multichannel sea-surface temperature** algorithm exploits the fact that water vapour absorbs more at 12 µm than at 11 µm. Temperature is a weighted combination of the 11 µm brightness temperature and the difference between the channels. The coefficients are found by regression against simultaneous readings from drifting buoys. In 1985 its root-mean-square difference from ship and buoy data was about 0.5 °C (Chapter 16).

**Logistic regression** turns the weighted sum into a probability, $p = \sigma(w^{\top}x + b)$, and is trained by minimising cross-entropy, as in the worked example. Its decision boundary, $p = 0.5$, is a straight line (a plane, in more dimensions). That makes it a good **baseline** — every new model should be compared with a simple one. It also explains its failure in Figure 43.12: no straight line can put snow on the clear side and cloud on the other.

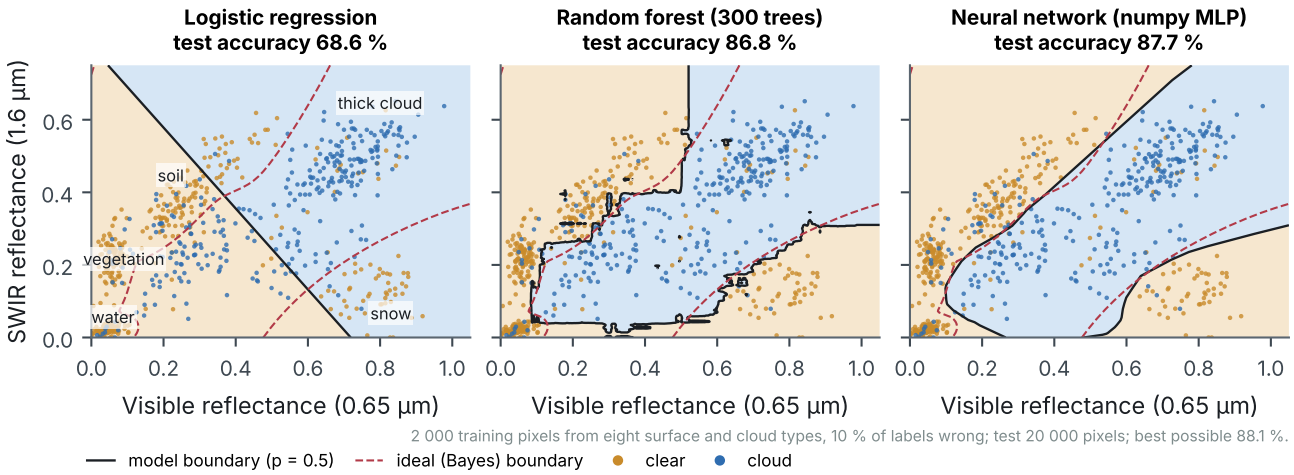



**FIGURE 43.12** Decision boundaries of three classifiers trained on the same 2 000 synthetic pixels from eight surface and cloud types, 10 % of labels wrong. Logistic regression labels 96 % of snow pixels as cloud (test accuracy 68.6 %); a random forest (86.8 %) and the early-stopped numpy network (87.7 %) approach the best possible 88.1 %, whose ideal boundary is dashed.

## 43.4.2 Nearest Neighbours

The **k-nearest-neighbour (k-NN)** method stores the training data and, for each new input, takes a majority vote (classification) or an average (regression) of the $k$ closest examples. It needs no training and adapts to any boundary shape, but prediction is slow for large data sets and distances lose meaning when there are many features — the "curse of dimensionality". Since about 1990 Finland's multi-source national forest inventory has used k-NN to give every satellite pixel the timber volume, species and age of the field plots whose spectra it most resembles.

## 43.4.3 Decision Trees, Random Forests and Gradient Boosting

A **decision tree** asks a sequence of threshold questions, such as "is the 1.6 µm reflectance above 0.3?". Each question is chosen to make the resulting groups as pure as possible, measured for example by the **Gini impurity** $G = 1 - \sum_k p_k^2$, where $p_k$ is the fraction of class $k$ in a group. Trees are readable and fast — the best tree JPL built for EO-1 needed at most 24 comparisons per pixel — but deep trees memorise their training data and are unstable.

A **random forest** averages hundreds of trees, each grown on a random bootstrap sample of the data and choosing each split from a random subset of features. The trees make different errors, so their average has much lower variance. NASA's MODIS land-cover product switched from a C4.5 decision tree in Collection 5 to a random forest in Collection 6, classifying the globe each year since 2001 at 500 m resolution. Hansen and colleagues' 2013 global forest-change maps applied decision trees to about 654 000 Landsat scenes, using one million CPU-core hours in Google Earth Engine.

**Gradient boosting** builds trees in sequence. Each new small tree $h_m$ is fitted to the errors (strictly, the negative gradient of the loss) left by the ensemble so far. It is added with a small weight $\nu$, so $F_m(x) = F_{m-1}(x) + \nu\, h_m(x)$. Boosted trees, in libraries such as XGBoost and LightGBM, are usually the most accurate models for tabular data. Sentinel Hub's **s2cloudless** detector is a LightGBM model working on ten Sentinel-2 bands. It underlies the cloud-probability layer distributed with Sentinel-2 imagery in Google Earth Engine, although it gives high probabilities over very bright surfaces such as snow.

### 43.4.4 Support Vector Machines

A **support vector machine (SVM)** chooses, among all boundaries that separate two classes, the one with the widest **margin** — the largest gap to the nearest training points, the support vectors. For a linear boundary $w^{\top}x + b = 0$ the margin is $2/\|w\|$, so the SVM minimises

$$\tfrac{1}{2}\|w\|^2 + C\sum_i \max\left(0,\; 1 - y_i(w^{\top}x_i + b)\right),$$

with labels coded $y_i = \pm 1$; the second term, the **hinge loss**, penalises points on the wrong side of the margin. A **kernel** such as the Gaussian $K(x, x') = \exp(-\gamma\|x - x'\|^2)$ lets the same mathematics draw curved boundaries. In 2015 Bobra and Couvidat used a Gaussian-kernel SVM to forecast M- and X-class flares within 24 hours (Chapter 47). It worked on magnetic-field features of solar active regions measured by the Solar Dynamics Observatory, and reached a true skill statistic of 0.76 on 303 flaring and 5 000 non-flaring examples.

#### EO-1: LEARNED CLASSIFIERS IN COMMAND

NASA's Earth Observing-1, launched in 2000 as a technology demonstrator, became from 2003–04 the testbed of the **Autonomous Sciencecraft Experiment**. This onboard software analysed images from the Hyperion spectrometer, detected floods, eruptions and changes in snow and ice, and replanned observations without waiting for the ground (Figure 43.13). The payload computer was a Mongoose V of about 8 MIPS with 256 MB of memory, and only 12 of Hyperion's bands could be read on board. JPL compared four pixel classifiers for land, water, snow, ice and cloud: one built by an expert, the best of thousands of thresholded band ratios, a decision tree and a **linear SVM** on 11 bands. The SVM needed just 11 multiplications and 11 additions per class for each pixel. On 50 expert-labelled test images (about 91 million pixels) the SVM classified 81.3 % of pixels correctly, the expert classifier 82.9 % and the tree 80.9 %. The averages hid a weakness: ice made up only 2 % of the test pixels, and the SVM found just 47.8 % of them. The expert and SVM classifiers ran on board — the SVMs for more than a dozen years, until the mission ended in 2017. The team reported a hundred-fold increase in science returned per megabyte downlinked.

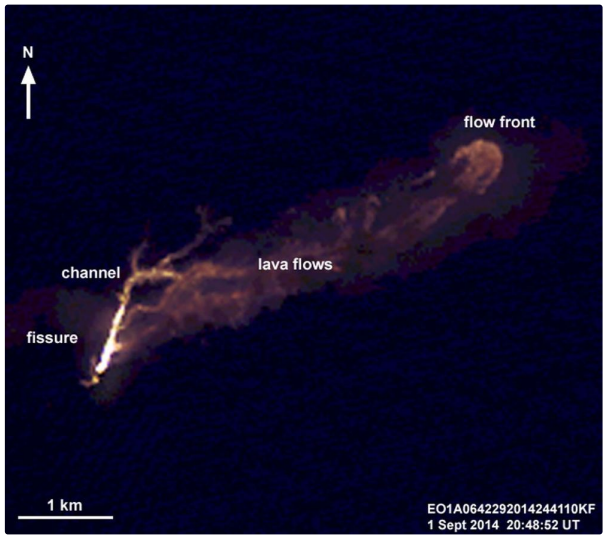
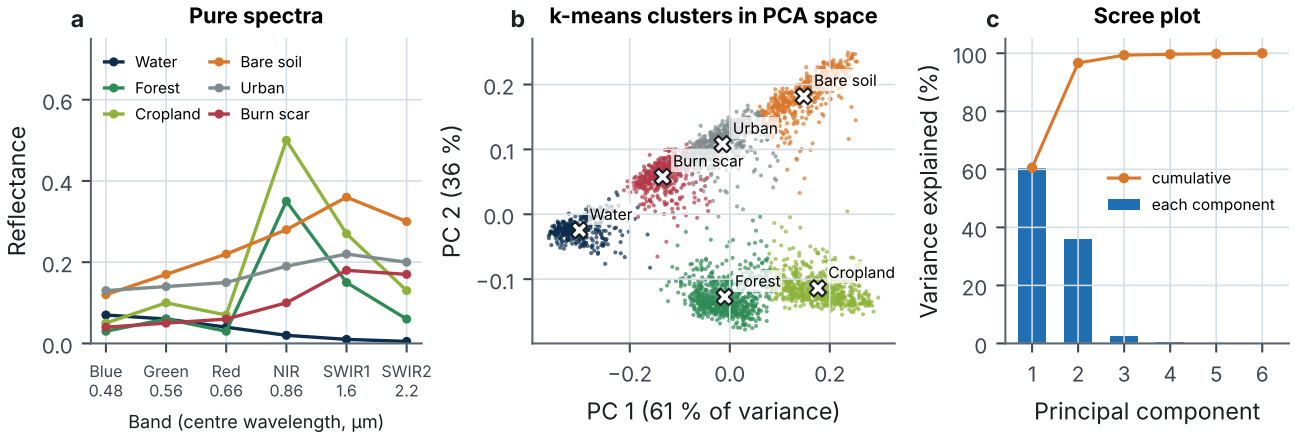



**FIGURE 43.13** The Holuhraun eruption in Iceland on 1 September 2014, imaged by EO-1 in false colour to emphasise the hottest vents and lava flows. Detecting such thermal events on board, and retasking the satellite to observe them again, was one of the jobs of EO-1's autonomy software. Image: NASA/JPL (NASA/JPL/EO-1 Mission/GSFC/Ashley Davies).

### 43.4.5 Clustering and Dimensionality Reduction

**k-means** clustering divides unlabelled data into $k$ groups. It starts from $k$ trial centres, assigns each sample to the nearest centre, moves each centre to the mean of its samples, and repeats, minimising the total squared distance $\sum_i \|x_i - \mu_{c(i)}\|^2$. Its variant ISODATA, which splits and merges clusters, has been used for unsupervised image classification since the 1970s; the analyst names each spectral cluster afterwards (Chapter 13). The number of clusters must be chosen, for example from an "elbow" in the total distance as $k$ grows.

**Principal component analysis (PCA)** finds new axes, the eigenvectors of the covariance matrix $C = \frac{1}{N}\sum_i (x_i - \bar{x})(x_i - \bar{x})^{\top}$, ordered by the variance $\lambda_k$ along each. Keeping only the first few components compresses the data and removes noise. In Figure 43.14 two components hold 96.6 % of the variance of six-band pixels, and k-means, never shown a label, recovers the six cover types well (adjusted Rand index 0.89, where 1 is perfect agreement). The IASI sounder on the Metop satellites measures 8 461 spectral channels. Since 2011 EUMETSAT has also distributed each spectrum as about 300 principal-component scores — a 28-fold reduction. Users reconstruct the radiances from them with much of the instrument noise removed.




3 000 synthetic six-band pixels (shaded mixtures dominated by one cover, plus noise); k-means with k = 6: adjusted Rand index 0.89.

**FIGURE 43.14** Unsupervised learning on 3 000 synthetic six-band pixels, each a shaded, noisy mixture dominated by one cover type. (a) The pure spectra. (b) PCA compresses six bands into two components holding 96.6 % of the variance; k-means with six clusters, run without labels, finds groups (crosses: centres) matching the cover types (adjusted Rand index 0.89). (c) Variance explained by each component.

### 43.4.6 Gaussian Processes

A **Gaussian process (GP)** treats the unknown function itself as random, with a **kernel** $k(x, x')$ saying how strongly the function values at two inputs are correlated — typically more strongly the closer they are. Conditioning on $N$ noisy observations $y$ gives, at a new input $x_*$, a predictive mean and variance

$$\mu_* = k_*^{\top}(K + \sigma_n^2 I)^{-1} y, \qquad \sigma_*^2 = k(x_*, x_*) - k_*^{\top}(K + \sigma_n^2 I)^{-1} k_*,$$

where $K$ holds the kernel values between training inputs, $k_*$ those between them and $x_*$, and $\sigma_n^2$ is the noise variance. The variance widens away from the data, giving honest error bars; the cost of the $N \times N$ matrix inversion grows as $N^3$. In geostatistics the same mathematics is called **kriging**. GPs are a standard tool for separating planetary signals from stellar activity in radial-

velocity planet searches. In 2014 a GP model of activity in the star CoRoT-7 gave masses of 4.73 ± 0.95 and 13.56 ± 1.08 Earth masses for its two planets (Chapter 51). They also underlie **Bayesian optimisation** of hyper-parameters.

### 43.4.7 The Kalman Filter: Estimation Meets Learning

The **Kalman filter** (Chapter 6) estimates the state of a dynamic system — position and velocity, or attitude and gyro bias — from noisy measurements. Its **prediction** step propagates the estimate and its covariance with a dynamic model, $\hat{\boldsymbol{x}}^- = \boldsymbol{F}\hat{\boldsymbol{x}}$ and $\boldsymbol{P}^- = \boldsymbol{F}\boldsymbol{P}\boldsymbol{F}^\top + \boldsymbol{Q}$, where $\boldsymbol{Q}$ is the process noise. Its **update** blends in a measurement $\boldsymbol{z}$:

$$\boldsymbol{K} = \boldsymbol{P}^- \boldsymbol{H}^\top (\boldsymbol{H}\boldsymbol{P}^- \boldsymbol{H}^\top + \boldsymbol{R})^{-1}, \qquad \hat{\boldsymbol{x}} = \hat{\boldsymbol{x}}^- + \boldsymbol{K}(\boldsymbol{z} - \boldsymbol{H}\hat{\boldsymbol{x}}^-), \qquad \boldsymbol{P} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^-,$$

where $\boldsymbol{H}$ maps the state to the measurement, $\boldsymbol{R}$ is the measurement noise and the **gain** $\boldsymbol{K}$ weights new information by the relative confidence in prediction and measurement. This is exact **Bayesian inference** for linear systems with Gaussian noise: prior (prediction) times likelihood (measurement) gives the posterior (update). Stanley Schmidt's group at NASA Ames adapted Kalman's 1960 theory to the non-linear problem of navigating Apollo to the Moon, and the resulting **extended Kalman filter** flew in the Apollo guidance computer (Figure 43.15).

**FIGURE 43.15** Jim Lovell at the guidance and navigation station of Apollo 8 in December 1968, in a frame from 16 mm film. Star sightings made here were combined with a dynamic model in a recursive estimator of the kind adapted for Apollo at NASA Ames. Image: NASA/JSC.

Estimation theory and machine learning are two dialects of one language. Least-squares orbit determination (Chapter 3) is regression, and a Kalman filter can be seen as Gaussian-process regression in time. A network trained with weight decay is a maximum *a posteriori* estimate with a Gaussian prior on its weights. Estimation starts from physics with few parameters; learning starts from data with many. The best systems use physics where it is known and learning where it is not.

# 43.5 Measuring Performance

## 43.5.1 From the Confusion Matrix to F1

For a two-class detector — fire or no fire, anomaly or normal — every prediction falls into one cell of the **confusion matrix**. A **true positive** (TP) is an event correctly flagged and a **false positive** (FP) is a false alarm. A **false negative** (FN) is a miss and a **true negative** (TN) is a correct rejection. In map accuracy assessment (Chapter 13) precision is called user's accuracy and recall producer's accuracy. Table 43.5 lists the scores built from these counts. **Precision** is the fraction of alarms that are real, **recall** (probability of detection) the fraction of events found, and the **F1 score** their harmonic mean, which is high only if both are.

**TABLE 43.5** Evaluation metrics.

| Metric | Definition | Use it when | Caution |
|---|---|---|---|
| Accuracy | (TP + TN) / N | Classes are balanced | Meaningless for rare events |
| Precision | TP / (TP + FP) | False alarms are costly | Falls as events become rarer |
| Recall | TP / (TP + FN) | Misses are costly | Ignores false alarms |
| F1 | 2 × precision × recall / (precision + recall) | One number for a rare class | Depends on threshold |
| ROC AUC | Area under recall versus false-alarm rate | Comparing rankings | Optimistic for rare events |
| Average precision | Area under precision versus recall | Rare events | Depends on prevalence |
| RMSE, MAE | √(mean error²); mean absolute error | Regression | RMSE stresses large errors |
| CSI | TP / (TP + FP + FN) | Event forecasts | Depends on event rate |
| TSS | Recall − false-alarm rate | Flare and storm forecasts | Can be high with poor precision |
| Brier score | Mean of (probability − outcome)² | Probability forecasts | Compare with climatology |

**EXAMPLE: SCORING A WILDFIRE DETECTOR**

A thermal-infrared fire detector is tested on a scene of 1 000 000 land pixels containing 500 fire pixels, checked by an independent high-resolution sensor (Figure 43.16). It raises 620 alarms: 410 at real fires and 210 elsewhere.

1. **Counts.** TP = 410, FP = 210, FN = 500 − 410 = 90 and TN = 1 000 000 − 500 − 210 = 999 290.
2. **Precision** = 410 / 620 = **0.661**: one alarm in three is false. **Recall** = 410 / 500 = **0.820**. **F1** = 2 × 0.661 × 0.820 / (0.661 + 0.820) = **0.732**.
3. **False-alarm rate** = 210 / 999 500 = $2.1 \times 10^{-4}$; CSI = 410 / 710 = 0.577; TSS = 0.820 − 0.0002 = 0.820.
4. **Accuracy** = (410 + 999 290) / 1 000 000 = **99.97 %** — but a "detector" that never raises an alarm scores 99.95 %. Accuracy cannot tell them apart; precision and recall can.
5. **Sensitivity.** If a season of sun glint doubles the false alarms to 420, precision falls to 0.494 and F1 to 0.617, while accuracy barely changes (99.95 %).

**FIGURE 43.16** Smoke from lightning-caused wildfires in northern California on 4 August 2015, in a natural-colour image from MODIS on Terra. Red outlines mark actively burning areas detected by MODIS's thermal bands — a detection problem in which fire pixels are a tiny fraction of the scene. Image: NASA/GSFC.

## 43.5.2 ROC and Precision–Recall Curves

Most classifiers output a score, and an alarm is raised above a chosen **threshold**; lowering it finds more events but raises more false alarms. The **receiver operating characteristic (ROC)** curve plots recall against false-alarm rate for every threshold; its area (AUC) is the probability that a random event scores higher than a random non-event. The **precision–recall (PR)** curve plots precision against recall; its area is the **average precision** (Figure 43.17).

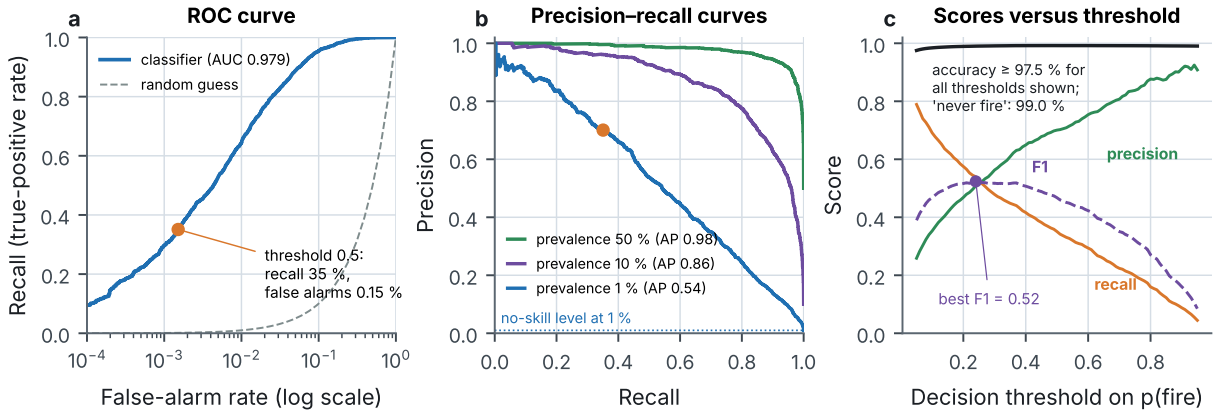



**FIGURE 43.17** Why accuracy misleads for rare events: a logistic-regression detector on a synthetic fire task (2 001 fires among 200 000 test pixels; 5 % of clear pixels fire-like). (a) The ROC area is 0.979, yet at threshold 0.5 only 35 % of fires are found. (b) Average precision is 0.54 at 1 % prevalence, 0.86 at 10 % and 0.98 at 50 %. (c) Accuracy stays above 97.5 % at all thresholds from 0.05 to 0.95; F1 peaks at 0.52.

For rare events the two tell very different stories. A logistic-regression detector for a synthetic fire task with 1 % prevalence has a near-perfect-looking ROC area of 0.979. Yet at the default threshold of 0.5 it finds only 35 % of the fires, and its average precision is just 0.54. On samples in which fires make up 10 % or 50 % of the pixels, the same detector reaches average precisions of 0.86 and 0.98 — its ROC curve does not change at all. Because the false-alarm rate divides by the

huge number of non-events, the ROC curve hides the fact that even a small rate then produces as many false alarms as detections. Accuracy is worse still: it stays above 97.5 % at every threshold from 0.05 to 0.95, and a detector that never fires scores 99.0 % — the **accuracy paradox**. The threshold should be set from the real costs of misses and false alarms, not by a software default.

### 43.5.3 Detection, Segmentation and Regression Metrics

For segmentation and object detection the basic unit is the **intersection over union** $\mathrm{IoU} = |A \cap B| / |A \cup B|$ of predicted and true regions; a detection usually counts as correct if its IoU exceeds 0.5. **Mean average precision (mAP)** averages the average precision over classes, sometimes also over IoU thresholds of 0.5–0.95, so mAP figures are comparable only under the same definition. For regression the usual scores are the **root-mean-square error** $\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i (\hat{y}_i - y_i)^2}$, which emphasises large errors, and the **mean absolute error** (MAE). Others are the mean **bias** and $R^2$, the fraction of variance explained. As Figure 43.11 showed, $R^2$ becomes negative when a model is worse than predicting the mean.

### 43.5.4 Forecast Skill and Calibration

Forecasts of events use further scores. The **critical success index**, CSI = TP / (TP + FP + FN), ignores the many correct "no event" forecasts. The **true skill statistic**, TSS = recall − false-alarm rate, ranges from −1 to 1 and does not depend on how often events occur. That is why it is widely recommended for comparing flare forecasts made on different samples. A **skill score**, $\mathrm{SS} = 1 - S/S_{\mathrm{ref}}$, compares a score $S$ (lower is better) with that of a reference such as climatology or persistence.

Probabilistic forecasts must also be **calibrated**: of all the occasions on which a flare model says "30 % chance", about 30 % should see a flare. The **Brier score**, $\mathrm{BS} = \frac{1}{N}\sum_i (p_i - o_i)^2$ with outcomes $o_i$ of 0 or 1, rewards both calibration and sharpness, and a **reliability diagram** plots observed frequency against forecast probability. Many classifiers are poorly calibrated out of the box. Modern deep networks tend to be over-confident, while boosted trees and SVMs distort probabilities in other ways. They can be recalibrated on validation data, for example by Platt scaling or isotonic regression. Calibration matters whenever probabilities feed decisions, such as collision-avoidance manoeuvres (Chapter 21).

### 43.5.5 Rare Events

Fires, anomalies, high-risk conjunctions and large flares are all rare, and a model that minimises average loss can do well by ignoring the rare class. Remedies include **re-weighting** the loss, **re-sampling** the classes, setting the threshold for the actual costs and evaluating with precision, recall and PR curves rather than accuracy. The EO-1 classifiers showed the problem: overall accuracy above 80 % concealed a recall below 50 % for ice, which made up 2 % of the pixels.

DATA BOX — RARE EVENTS: ESA'S COLLISION-AVOIDANCE CHALLENGE

- In 2019 ESA released 199 082 conjunction data messages for 15 321 close approaches involving its satellites between 2015 and 2019, each with 103 attributes. It asked competitors to predict the final collision risk two days before closest approach.
- Only 515 events (3.4 %) had a final risk of $10^{-6}$ or more, 131 (0.9 %) of $10^{-5}$ or more and 30 of $10^{-4}$ or more.
- 96 teams made 862 submissions over two months. A naive forecast — "the final risk equals the latest known risk" — scored 0.694 on the competition's loss; only 12 teams beat it, and the winners only modestly.
- The organisers concluded that naive forecasts are a surprisingly strong benchmark that machine learning can beat, but that the data's imbalance limited off-the-shelf methods (Chapter 50).

### 43.5.6 How Sure Are We?

A test score must be quoted with its uncertainty. If a detector produces $k$ false alarms among $n$ independent negative samples, the rate $\hat{p} = k/n$ has a standard error of about $\sqrt{\hat{p}(1-\hat{p})/n}$. An approximate 95 % confidence interval is $\hat{p} \pm 1.96$ standard errors. The **Wilson interval** is more accurate when $k$ is small. If no false alarms are seen in $n$ trials, the "rule of three" gives a 95 % upper limit of about $3/n$. Two models tested on the same samples are compared with a paired test such as McNemar's.

EXAMPLE: HOW BIG MUST THE TEST SET BE?

A requirement says that an onboard event detector must have a false-alarm rate of about 1 %, measured to within ±0.2 % at 95 % confidence.

1. **Sample size.** Solving $1.96\sqrt{p(1-p)/n} = E$ for $n$ gives $n = 1.96^2 \times 0.01 \times 0.99 / 0.002^2$ = **9 508 negative samples**.
2. **Check.** With 95 false alarms in 9 508 samples, the Wilson 95 % interval is 0.82–1.22 %, as required.
3. **Correlation.** Pixels are not independent. Suppose they come in clusters of 25 neighbouring pixels with an intra-cluster correlation of 0.2. The **design effect** is then 1 + (25 − 1) × 0.2 = 5.8, and about **55 000 pixels** — some 2 200 independent clusters — are needed.
4. **Rarer requirements.** Halving the interval needs four times the samples; a 0.1 % rate measured to ±0.02 % needs about 96 000 independent negatives.
5. **The positives too.** The 500 fire pixels of the wildfire example measure its recall of 0.82 only to within about ±0.034.

## 43.6 From Notebook to Operations

In operations a model becomes one component of a data system (Figure 43.18). It may run on the ground, in a processing chain or cloud service. After verification on flight-like hardware it may also run on the spacecraft itself, where it can decide what to downlink or whether to retask the sensor (Chapter 45). In JPL's Dynamic Targeting experiment on the CogniSAT-6 cubesat in July–August 2025, the satellite looked about 74–90 seconds ahead along its track. It analysed the image for cloud on board and then pointed its camera at the clearest scenes.

### PRACTICE: BUILDING AN ONBOARD CLOUD SCREEN FOR A NEW IMAGER

1. **Define the decision.** The screen decides which scenes are downlinked. State the cost of each error — a cloudy scene wastes link capacity, a discarded clear scene is lost — and the onboard power, memory and time limits.
2. **Gather data.** Use data processed as on board, not ground products, covering all seasons, latitudes, Sun angles and surfaces — especially snow, ice, deserts, glint and smoke.
3. **Label.** Have experts mark cloud, thin cloud, shadow and clear; label a sample twice to measure the label-error rate.
4. **Set a baseline.** Score a physics-based threshold test on visible and 1.6 µm bands; any learned model must beat it clearly.
5. **Split by space and time.** Assign whole scenes, regions and dates to each set; hold out a region and a season entirely, and lock the test set.
6. **Train and tune.** Start with boosted trees on bands and ratios, then try a small convolutional network; tune on validation data only.
7. **Evaluate.** Report precision and recall per surface type at the chosen threshold, with confidence intervals; inspect the worst errors by eye.
8. **Deploy and verify.** Convert the model to the flight processor's number format, check its outputs against the ground version, and run it in "shadow mode" — deciding but not acting — before it controls the downlink.
9. **Monitor.** Compare flagged fractions with ground cloud masks; retrain after recalibration or for new regions.

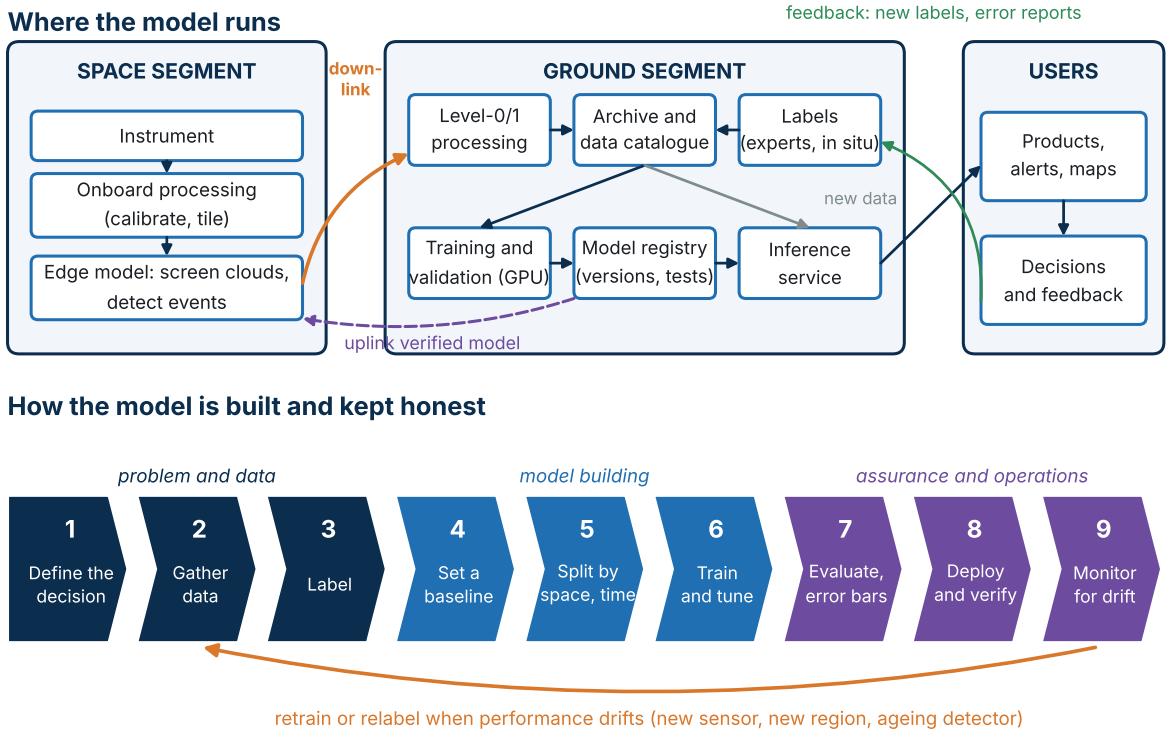



**FIGURE 43.18** Where machine learning sits in a space data system (top) and the nine-step workflow for building and operating a model (bottom). Models run on the ground or, once verified, are uplinked to the spacecraft; new labels and error reports from users feed retraining when performance drifts.

Operations bring **distribution shift**: instruments degrade and are recalibrated, new satellites have slightly different spectral responses, and models meet new regions and seasons. The remedy is monitoring inputs and outputs, comparing predictions with independent reference

data, and retraining when performance falls. A model must also be versioned and documented like any other flight or ground software, with its training data, test results and known limitations recorded (Chapter 53).

## 43.7 Road Map of Part XIII

The rest of Part XIII builds on these foundations (Table 43.6). Chapters 44 and 45 cover deep-learning methods, data and computers, and Chapters 46–52 apply them to each domain of the space ecosystem. Chapter 53 asks how learned systems can be verified, secured and governed well enough to be trusted.

**TABLE 43.6** Road map of Part XIII.

| Chapter | Topic | Main themes |
|---|---|---|
| 44 | Deep learning, foundation models and generative AI | CNNs, transformers, pre-trained and generative models |
| 45 | Data, computing hardware and edge AI in orbit | Data, labels, processors for onboard AI |
| 46 | AI for Earth observation | Mapping, change detection, disasters, image triage |
| 47 | AI for weather, climate and space weather | AI forecasts versus physics; flares and storms |
| 48 | AI for autonomy, navigation and robotics | Planners, AutoNav, learned guidance, RL |
| 49 | AI for mission operations, communications and networks | Anomaly detection, scheduling, cognitive radio |
| 50 | AI for space situational awareness, defence and security | Tracking, conjunctions, intelligence |
| 51 | AI for space science, planetary exploration and human spaceflight | Planet finding, planetary data, crew support |
| 52 | AI in spacecraft design, manufacturing, launch and the space business | Design, inspection, launch operations, markets |
| 53 | Trustworthy AI for space | Verification, security, governance, outlook |

### KEY TAKEAWAYS

- Symbolic AI writes knowledge down as rules and plans; machine learning learns it from examples. Space systems combine them, with verifiable logic guarding learned parts.
- AI reached space in waves — expert systems and schedulers, classical machine learning, deep learning, foundation models; most AI flying in 2026 is still demonstration.
- Learning minimises empirical risk by (stochastic) gradient descent, with learning rates below $2/\lambda_{\max}$; regularisation, early stopping and validation data control overfitting.
- Spatial and temporal autocorrelation leak information between random splits; split by block, region, time or mission, or a useless model can score $R^2 = 0.80$.
- Classical methods — regression, k-NN, trees, forests, boosting, SVMs, k-means, PCA, Gaussian processes, Kalman filters — still underpin most operational products.
- For rare events report precision, recall and skill scores with confidence intervals, not accuracy; a 1 % false-alarm rate needs about 9 500 independent negatives to measure to ±0.2 %.

### REVIEW QUESTIONS

1. Contrast symbolic AI and machine learning, with a space example of each. Why is safe mode still implemented with explicit logic?

2. Describe how Hubble's SPIKE scheduler represents constraints, and why a planner of this kind is easier to verify than a neural network.
3. Name the problem type and a suitable metric for: mapping flood extent from radar images; predicting the Kp index three hours ahead; choosing which of 50 images to downlink first; finding unusual behaviour in battery telemetry.
4. A loss surface has Hessian eigenvalues of 0.5 and 20. What is the largest stable learning rate for gradient descent, and what is the condition number? Why does standardising the inputs help?
5. Repeat the gradient-descent example for a clear vegetation pixel $\boldsymbol{x} = (0.06, 0.22)$, $\boldsymbol{y} = 0$, starting from the updated parameters. How does the prediction change?
6. Sketch training and validation loss against epochs for a network that overfits. What are early stopping, weight decay and data augmentation?
7. Why did random cross-validation give $R^2 = 0.80$ in Figure 43.11 when the model's skill in a new region was negative? How would you choose the block size for spatial cross-validation of a soil-moisture model?
8. How does a random forest reduce the variance of decision trees, and how does gradient boosting differ? Why did EO-1 use a linear SVM on board?
9. A conjunction-screening model raises 40 alerts in a year, of which 4 are real high-risk events, and misses 1. Compute its precision, recall and F1. Why is accuracy not useful here?
10. How many independent negatives are needed to measure a 2 % false-alarm rate to ±0.5 % at 95 % confidence? How does spatial correlation change this?

CHAPTER

# 44 Deep Learning, Foundation Models and Generative AI

Image: NASA/ARC (Dominic Hart)

*Deep neural networks now map floods from radar, find fresh craters on Mars, forecast the weather and help plan rover drives. This chapter explains how they work, from first principles and with small networks written and trained from scratch. It covers the building blocks of deep networks and the scaling laws that govern them; convolutional networks for images; recurrent networks, transformers and graph networks for telemetry, sequences and meshes; and generative models from autoencoders to diffusion. It then describes self-supervised foundation models for Earth, Sun and Moon, large language models and agents, physics-informed learning, uncertainty quantification and explainability. Verified examples show what each method has achieved in space as of October 2026, and where it can fail.*

**IN THIS CHAPTER**

44.1 Building Blocks of Deep Networks
44.2 Convolutional Networks for Images
44.3 Sequences, Transformers and Graphs
44.4 Generative Models
44.5 Foundation Models and Self-Supervised Learning
44.6 Large Language Models and Agents
44.7 Physics, Uncertainty and Explanation

Chapter 43 showed how a small neural network learns by gradient descent. **Deep learning** stacks many such layers — tens to hundreds — and trains them on large data sets. The network then learns its own features from raw pixels, spectra or telemetry, instead of relying on features chosen by an engineer. Since about 2012 this approach has overtaken older methods in image recognition, speech, translation and, since 2023, global weather forecasting.

This chapter explains the main families of deep networks, each in plain words with its key equations, then its space uses. Most figures come from small numpy networks trained for this book, so every number can be checked. Data and onboard processors are covered in Chapter 45, applications in Chapters 46–52 and assurance in Chapter 53.

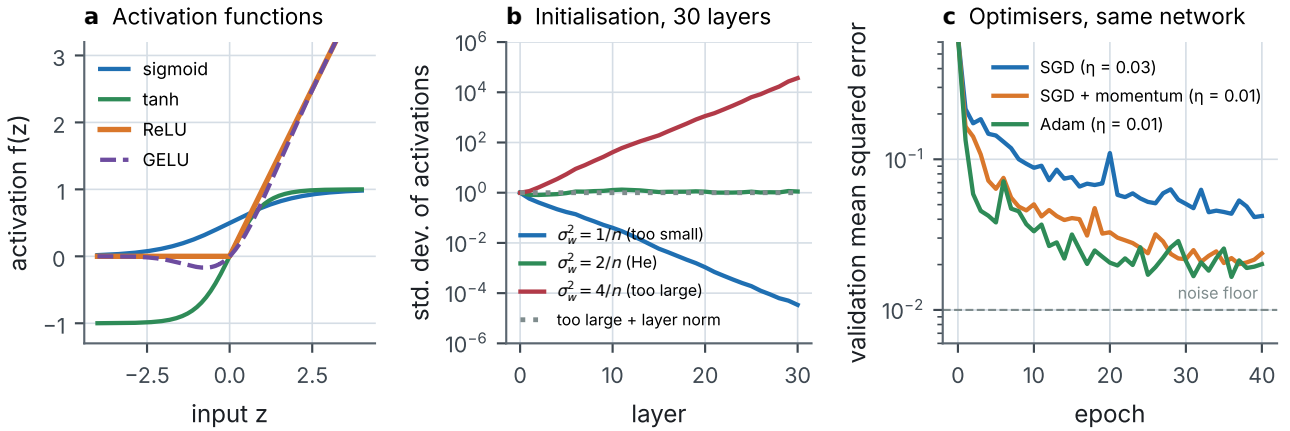
## 44.1 Building Blocks of Deep Networks

A deep network is a chain of simple, differentiable steps. Each step is cheap; the power comes from stacking many of them and fitting millions of weights to data. Almost every modern network uses the same few parts: linear layers, activations, careful initialisation, normalisation, residual connections and an adaptive optimiser.

### 44.1.1 Neurons, Activations and Layers

An artificial **neuron** computes a weighted sum of its inputs, adds a bias and passes the result through an **activation function** $f$: $y = f(\mathbf{w}^{\top}\mathbf{x} + b)$. A **layer** is many neurons side by side, so it computes $\mathbf{h} = f(W\mathbf{x} + \mathbf{b})$. Without the activation, any stack of layers would collapse into a single matrix multiplication and could only draw straight boundaries.

Figure 44.1 (a) shows the common activations. The **sigmoid** and **tanh** squash inputs into a fixed range; their slope is almost zero for large inputs, which slows learning. The **rectified linear unit (ReLU)**, $\max(0, z)$, has slope 1 for every positive input and is cheap to compute. **GELU** and **SiLU** are smooth versions of it used in transformers. For classification the last layer uses the **softmax**, $p_k = e^{z_k} / \sum_j e^{z_j}$, which turns scores into probabilities that sum to one.




(b) 512 random inputs through 30 ReLU layers of width 256. (c) Network 2-64-64-1 (ReLU, He init.) trained on 2 000 noisy samples of a smooth 2-D function, mini-batches of 32; best of four learning rates per optimiser on 2 000 validation samples; one seed.

**FIGURE 44.1** Building blocks, computed in numpy. (a) Common activations. (b) Activation spread through 30 ReLU layers: He initialisation (variance 2/n) keeps it near 1; 1/n and 4/n make it vanish or explode, unless layer normalisation is used. (c) One small network trained by three optimisers, each at its best learning rate.

### 44.1.2 Training: Backpropagation and Optimisers

Training uses **backpropagation**, which applies the chain rule backwards through the layers to find the gradient of the loss with respect to every weight (Chapter 43). Plain **stochastic gradient descent (SGD)** then steps each weight downhill. Two refinements make training much faster.

**Momentum** keeps a running sum of past gradients, $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t$, and steps along it, $\theta_t = \theta_{t-1} - \eta \mathbf{v}_t$, with $\beta \approx 0.9$. Like a heavy ball, it rolls through small bumps and along narrow valleys. **Adam** also tracks the mean square of each weight's gradient and divides by its square root:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \quad \theta_t = \theta_{t-1} - \eta \, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon},$$

where $\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$ are corrected for their start at zero, $\beta_1$ = 0.9 and $\beta_2$ = 0.999. Each weight thus gets its own step size. In Figure 44.1 (c), a network with two hidden layers of 64 units learns a smooth two-dimensional function. After 10 epochs (passes through the data) the validation error is 0.088 with plain SGD, 0.050 with momentum and 0.033 with Adam. After 40 epochs Adam reaches 0.020, against a noise floor of 0.010. Large models also use a **learning-rate schedule**: a short warm-up, then a slow decay.

### 44.1.3 Initialisation, Normalisation and Regularisation

Deep networks are fragile at the start. If the initial weights are slightly too small, the signal shrinks at every layer; if slightly too large, it explodes. Figure 44.1 (b) sends random inputs through 30 ReLU layers. With weight variance $1/n$ ($n$ inputs per neuron) the spread of the activations falls to $3 \times 10^{-5}$ of its starting value. With $4/n$ it grows to $4 \times 10^{4}$. **He initialisation**, with variance $2/n$, keeps it near 1, because ReLU zeroes half of its inputs. Gradients flowing backwards suffer the same fate, which is why badly initialised deep networks do not learn.

**Normalisation layers** fix the scale during training as well. **Layer normalisation** rescales each sample's activations to zero mean and unit variance, then applies a learned scale $\gamma$ and shift $\beta$: $y = \gamma(x - \mu)/\sigma + \beta$. **Batch normalisation** does the same across a mini-batch. **Dropout** randomly zeroes 10–50 % of units during training, so the network cannot rely on any single one.

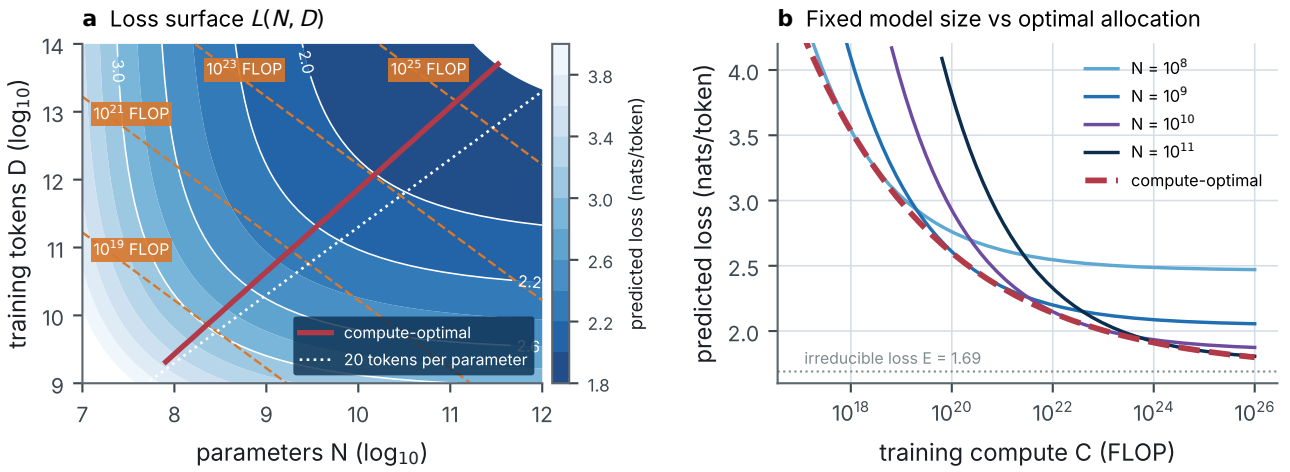
### 44.1.4 Scaling Laws

Since 2020, studies of language models have found that test loss falls smoothly, as a **power law**, as the number of parameters $N$, the number of training tokens $D$ and the compute $C$ grow. Hoffmann and colleagues (2022) fitted

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \qquad C \approx 6ND,$$

with $E$ = 1.69, $A$ = 406.4, $B$ = 410.7, $\alpha$ = 0.34 and $\beta$ = 0.28. $E$ is the loss that no model can beat, set by the randomness of language itself. The cost rule $C \approx 6ND$ counts about six floating-point operations (FLOP) per parameter per training token: two for the forward pass and four for the backward pass.

The formula answers a practical question: for a fixed compute budget, how big should the model be? Figure 44.2 computes it. For $10^{22}$ FLOP the best choice is about 5 billion parameters trained on about 320 billion tokens, giving a loss of 2.14. A 1-billion-parameter model trained with the same compute on five times more data reaches only 2.20. With these constants the optimal model size grows as $C^{0.45}$ and the data as $C^{0.55}$, so data matter as much as parameters.




Computed from the published fit L = E + A/N^α + B/D^β (E = 1.69, A = 406.4, B = 410.7, α = 0.34, β = 0.28; Hoffmann et al. 2022) for language models; training compute C ≈ 6ND floating-point operations (dashed lines in a: equal compute). Constants differ for other data types and architectures.

**FIGURE 44.2** Scaling laws from the published fit of Hoffmann et al. (2022). (a) Predicted loss against model size and data; dashed lines join points of equal compute, and the red line marks the best split. (b) Loss against compute: each fixed model size flattens, while the compute-optimal curve keeps falling.

Two cautions apply. First, for budgets of $10^{20}$–$10^{24}$ FLOP these published constants imply 40–100 tokens per parameter. The paper's other two methods recommend about 20. A 2024 replication traced the difference to errors in this fit. Second, the constants are for mostly English text. Earth-observation and weather models show similar trends, but with their own constants.

## 44.2 Convolutional Networks for Images

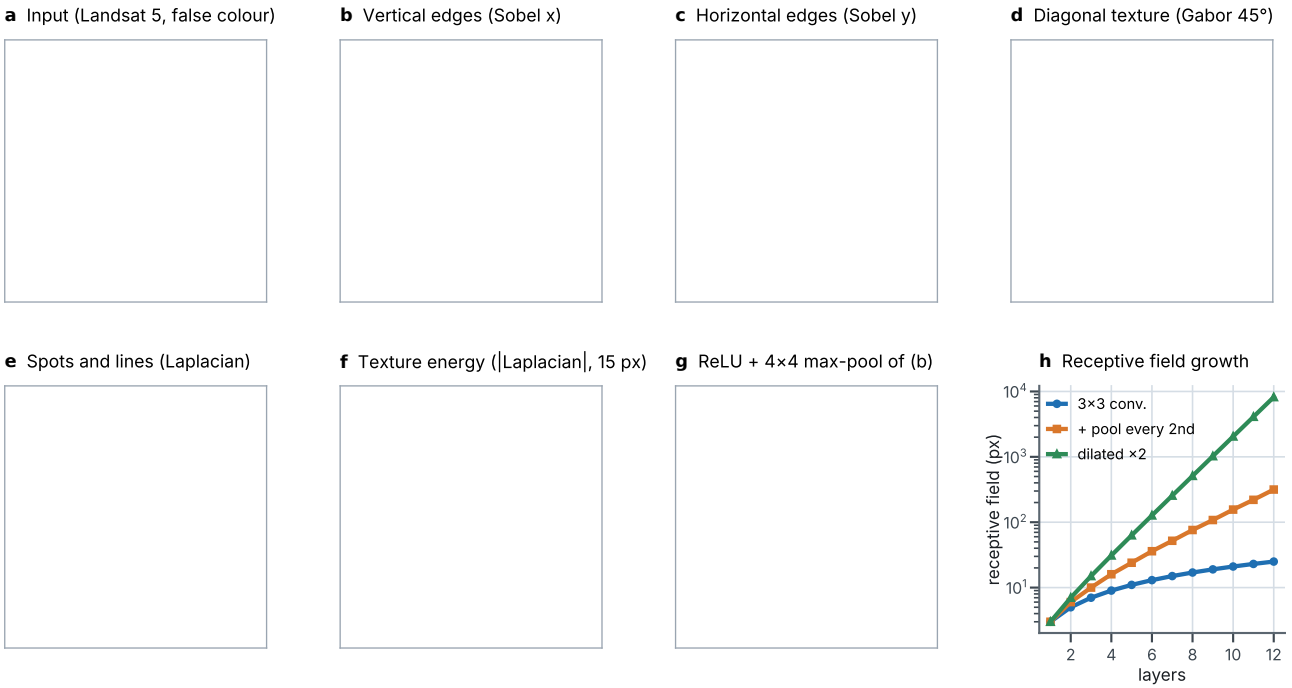
Images have structure: nearby pixels are related, and a coastline or a crater looks the same wherever it appears. A **convolutional neural network (CNN)** builds these facts into its design. It is the workhorse of image analysis in space, from cloud masks to crater catalogues.

### 44.2.1 Convolution, Pooling and Receptive Field

A **convolution layer** slides a small grid of weights, the **kernel**, over the image. At each position it multiplies the kernel by the pixels beneath and sums the result. Output channel $c'$ at pixel $(i, j)$ is

$$y_{i,j,c'} = b_{c'} + \sum_{c} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} w_{u,v,c,c'}\, x_{i+u,\, j+v,\, c}.$$

Here $c$ counts input channels and the kernel is $k \times k$ pixels. The same kernel is used everywhere — **weight sharing** — so a layer needs only $k^2 c\, c'$ weights, whatever the image size. Each output channel is a **feature map** that responds to one pattern. In Figure 44.3, hand-made kernels detect edges, texture and spots in a Landsat 5 image of Las Vegas. **Max-pooling** then keeps the largest value in each 4 × 4 block. A trained CNN learns similar filters in its first layer and combines them into shapes and objects in deeper layers.




Fixed (hand-designed) 3×3 and 9×9 kernels convolved with a 480 × 480-pixel crop (about 40 m per pixel) of a Landsat 5 image of Las Vegas, Nevada, 2 November 2011 (bands 7, 4, 2; NASA/USGS). Red and blue: positive and negative responses. A trained CNN learns similar filters in its first layer.

**FIGURE 44.3** Feature maps of a real satellite image: fixed kernels applied to a 480 × 480-pixel crop of a Landsat 5 false-colour image of Las Vegas (2 November 2011). (b–e) Edge, texture and spot filters; (f) texture energy; (g) ReLU and 4 × 4 max-pooling of (b); (h) receptive-field growth with depth.

The **receptive field** is the patch of input that can influence one output pixel. Twelve stacked 3 × 3 layers see only 25 × 25 pixels. Adding 2 × 2 pooling after every second layer widens this to 316 pixels. **Dilated (atrous) convolutions** space the kernel taps apart; doubling the spacing at each layer gives 8 191 pixels after 12 layers, without losing resolution (Figure 44.3 h). Context matters: telling a lake from a cloud shadow needs a view of the surroundings.

### 44.2.2 Residual Networks

Simply stacking more layers eventually makes networks harder to train. **Residual networks (ResNets)** (2015) fixed this with a shortcut: each block learns a correction $\boldsymbol{F}(\mathbf{x})$ and outputs $\mathbf{y} = \mathrm{ReLU}(\boldsymbol{F}(\mathbf{x}) + \mathbf{x})$ (Figure 44.4). An unneeded block can learn $\boldsymbol{F} \approx \mathbf{0}$ and pass its input straight through, and gradients flow back through the shortcuts undiminished. An ensemble of ResNets, the deepest with 152 layers, won the 2015 ImageNet competition with a top-5 error of 3.57 %. Residual connections are now in almost every deep network.

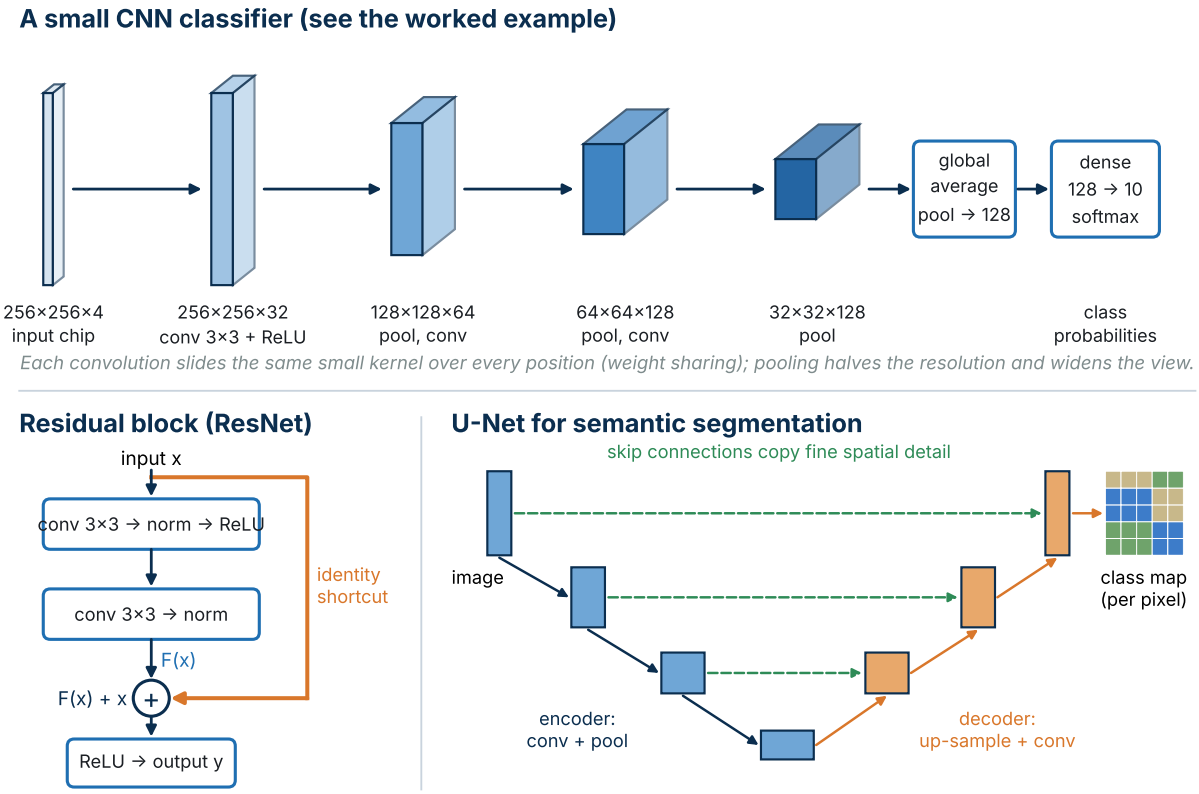



**FIGURE 44.4** Three convolutional architectures. Top: a small classifier for 256 × 256-pixel, four-band chips, with the data shape after each stage (see the worked example). Bottom: a residual block, and a U-Net whose skip connections copy fine detail from encoder to decoder.

### EXAMPLE: PARAMETERS AND OPERATIONS OF A SMALL CNN

The classifier at the top of Figure 44.4 takes 256 × 256-pixel chips with four bands. It has three 3 × 3 convolution layers with 32, 64 and 128 channels, with 2 × 2 pooling before the second and third. Global average pooling and a dense layer for 10 classes follow.

1. **One layer.** A convolution has $k^2 c_{in} c_{out} + c_{out}$ parameters and needs $HWk^2 c_{in} c_{out}$ multiply–accumulate operations (MACs). Layer 1: 9 × 4 × 32 + 32 = **1 184 parameters**, and 256 × 256 × 9 × 4 × 32 = 75.5 million MACs.
2. **Layers 2 and 3.** Layer 2 has 9 × 32 × 64 + 64 = 18 496 parameters at 128 × 128 pixels (302 million MACs). Layer 3 has 9 × 64 × 128 + 128 = 73 856 parameters at 64 × 64 pixels (also 302 million MACs).
3. **Head.** Pooling has no weights; the dense layer has 128 × 10 + 10 = 1 290.
4. **Totals.** The network has **94 826 parameters** and needs 679 million MACs, or **1.36 GFLOP** (two operations per MAC), per chip.
5. **Weight sharing.** A dense layer mapping the same input to the same 256 × 256 × 32 output would need $5.5 \times 10^{11}$ weights — about six million times more than the whole network.
6. **A whole scene.** A 10 000 × 10 000-pixel scene holds about 1 526 chips: about $2.1 \times 10^{12}$ operations, or some 2 seconds on a processor sustaining $10^{12}$ operations per second (1 TOPS; Chapter 45).

## 44.2.3 Object Detection

**Object detection** finds each object in an image and draws a box around it with a class label. **Two-stage detectors**, such as Faster R-CNN (2015), first propose regions that might contain an object, then classify and refine each one. They are accurate but slower. **One-stage detectors**, such as the YOLO ("you only look once", 2016) family, divide the image into a grid and predict boxes and

classes for every cell in a single pass. Overlapping boxes for the same object are then merged by **non-maximum suppression**. Ultralytics' YOLO26, released in January 2026, removes that step by predicting one box per object directly. Detection is scored by mean average precision (mAP, Chapter 43). Ships, aircraft and vehicles in satellite images are detected this way (Chapter 46), as are craters and boulders on planetary surfaces.

**LANDMARK: AI FINDS FRESH CRATERS ON MARS (2020)**

- Fresh impacts on Mars are found by comparing before-and-after images, a slow task for people. JPL researchers trained a classifier on 6 830 Context Camera (CTX) images from Mars Reconnaissance Orbiter.
- Running up to 750 copies in parallel on a JPL cluster, it scanned about 112 000 CTX images, at about 5 seconds per image against about 40 minutes for a human.
- Its first confirmed find was a cluster of craters up to about 4 m across in Noctis Fossae, formed between March 2010 and May 2012. HiRISE confirmed it on 26 August 2020 (Figure 44.5), and more than 20 further candidates were queued for follow-up.
- The network did not replace the scientists: it ranked the archive so that HiRISE looked where a discovery was likely.

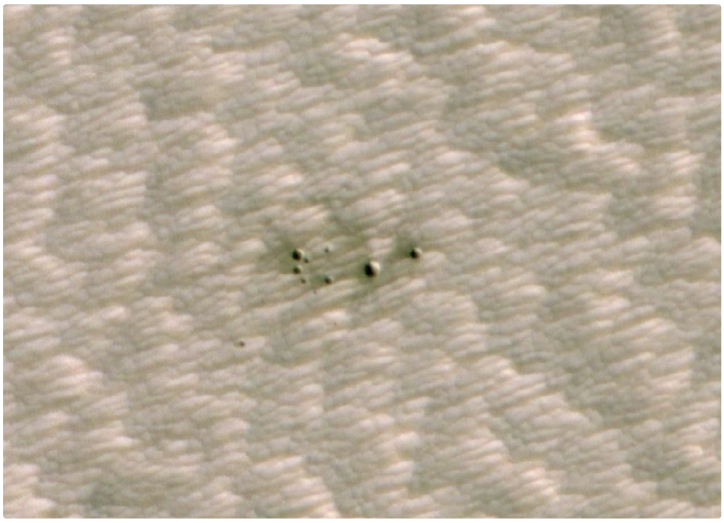


**FIGURE 44.5** The first crater cluster on Mars found by artificial intelligence, seen by HiRISE on Mars Reconnaissance Orbiter. A classifier flagged the dark spots in Context Camera images; HiRISE confirmed fresh craters made by one impactor that broke apart. Image: NASA/JPL (NASA/JPL-Caltech/University of Arizona).

### 44.2.4 Semantic Segmentation

**Semantic segmentation** assigns a class to every pixel: water or land, cloud or clear, sand or bedrock. The **U-Net** (2015) is an encoder–decoder (Figure 44.4): the encoder shrinks the image while widening the view, the decoder up-samples back to full resolution, and **skip connections** copy fine detail across. **DeepLab** (2014–18) instead uses dilated convolutions and a pyramid of dilation rates to capture context at several scales. Table 44.1 summarises the vision architectures and their space uses.

DeepMoon (2019) trained a U-Net on lunar elevation maps. It recovered 92 % of the craters in its test catalogue and nearly doubled the number found, though about 11 % of the new detections checked by hand were false. NASA JPL's **AI4Mars** project collected about 326 000 crowd-sourced

labels on about 35 000 rover images, marking soil, bedrock, sand and big rocks (Figure 44.6). A DeepLabv3 network trained on them classified more than 96 % of test pixels correctly. Terrain classifiers of this kind support Curiosity's drive planning on the ground (Chapter 48).

**TABLE 44.1** Vision architectures and typical space uses.

| Task | Architecture | Key idea | Space example |
|---|---|---|---|
| Scene classification | ResNet, EfficientNet, ViT | Deep features, one label per image | Land-use classes, image triage on board |
| Object detection | Faster R-CNN; YOLO family | Region proposals; single-pass grid | Ships, aircraft, craters, boulders |
| Semantic segmentation | U-Net, DeepLab | Encoder–decoder with skips; dilation | Floods, clouds, burn scars, Mars terrain |
| Instance segmentation | Mask R-CNN | Box plus per-object mask | Building footprints, individual fields |
| Super-resolution | Residual CNNs, GANs, diffusion | Learned up-sampling | Sharpening multispectral bands |

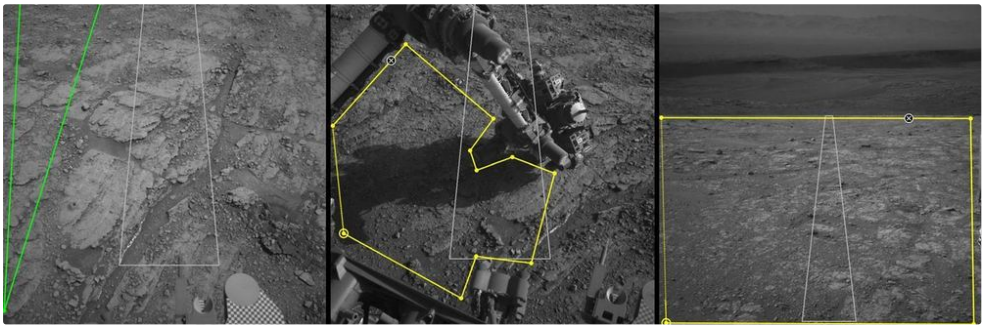


**FIGURE 44.6** The public AI4Mars labelling tool. Volunteers outlined soil, bedrock, sand and large rocks in Curiosity images to train terrain-segmentation networks. Image: NASA/JPL (NASA/JPL-Caltech).

## 44.3 Sequences, Transformers and Graphs

Telemetry, light curves and solar-wind data are **sequences**, in which the past helps predict the future. Weather on a sphere and satellite networks are better described as **graphs**. This section covers networks for both, including the now-dominant transformer.

### 44.3.1 Recurrent Networks, LSTM and Temporal Convolutions

A **recurrent neural network (RNN)** reads a sequence one step at a time and keeps a **hidden state** that summarises what it has seen: $\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$. Gradients are multiplied by one matrix per step backwards through time, so they shrink or explode over long sequences. Plain RNNs forget after a few tens of steps.

The **long short-term memory (LSTM)** network (1997) adds a separate **cell state** $\mathbf{c}_t$ and three gates, each a sigmoid between 0 and 1. The forget gate $\mathbf{f}_t$ decides what to keep, the input gate $\mathbf{i}_t$ what to add and the output gate $\mathbf{o}_t$ what to show:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

where $\odot$ means element-by-element multiplication. Because the cell state is updated by addition, information and gradients survive hundreds of steps. The **gated recurrent unit (GRU)** (2014) is a lighter version with two gates. In 2018 JPL researchers used LSTMs to predict telemetry from the

SMAP satellite and the Curiosity rover; large prediction errors flagged anomalies (Chapter 49).

A **temporal convolutional network (TCN)** applies one-dimensional convolutions that look only at the past (**causal** convolutions), with dilations 1, 2, 4, 8 and so on. With kernel size $k$ and $L$ layers, its receptive field is $1 + (k-1)(2^L - 1)$ samples: 2 047 samples for $k = 3$ and ten layers. Unlike an RNN, a TCN processes all time steps in parallel during training. Table 44.2 compares the options.

**TABLE 44.2** Sequence models compared.

| Model | How it remembers | Parallel training | Cost per sequence | Typical space use |
|---|---|---|---|---|
| RNN | Hidden state | No | Linear in length | Rarely used alone today |
| LSTM / GRU | Gated cell state | No | Linear | Telemetry prediction, anomaly detection |
| TCN | Dilated causal convolutions | Yes | Linear | Fast onboard forecasting, event detection |
| Transformer | Attention over all past steps | Yes | Quadratic in length | Long telemetry, solar time series, language |

### 44.3.2 Attention

**Attention** lets each element of a sequence look directly at every other element and weigh its importance. Each input vector is projected into a **query** $\mathbf{q}$ ("what am I looking for?"), a **key** $\mathbf{k}$ ("what do I contain?") and a **value** $\mathbf{v}$ ("what do I pass on?"). Stacking them into matrices $Q$, $K$ and $V$ gives **scaled dot-product attention**:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V.$$

Each softmax row holds weights that sum to one, so each output is a weighted average of the values. The weights are large where the query matches the key. Dividing by $\sqrt{d_k}$, where $d_k$ is the key length, keeps the softmax in a range with useful gradients. For forecasting, a **causal mask** sets the weights on future positions to zero.

In Figure 44.7, a single attention layer trained in numpy predicts the next sample of noisy periodic "telemetry". The period is 16 steps, but the model was not told this. After 4 000 training steps it looks back 15 and 31 steps, each with a mean weight of 0.21. These are the samples at the same phase one and two cycles before the predicted one. Averaging several noisy copies of the same phase cancels much of the noise. On new sequences its error is 0.331, against 0.551 for repeating the last value and 0.425 for copying the value one period earlier. The noise sets a floor of 0.30.

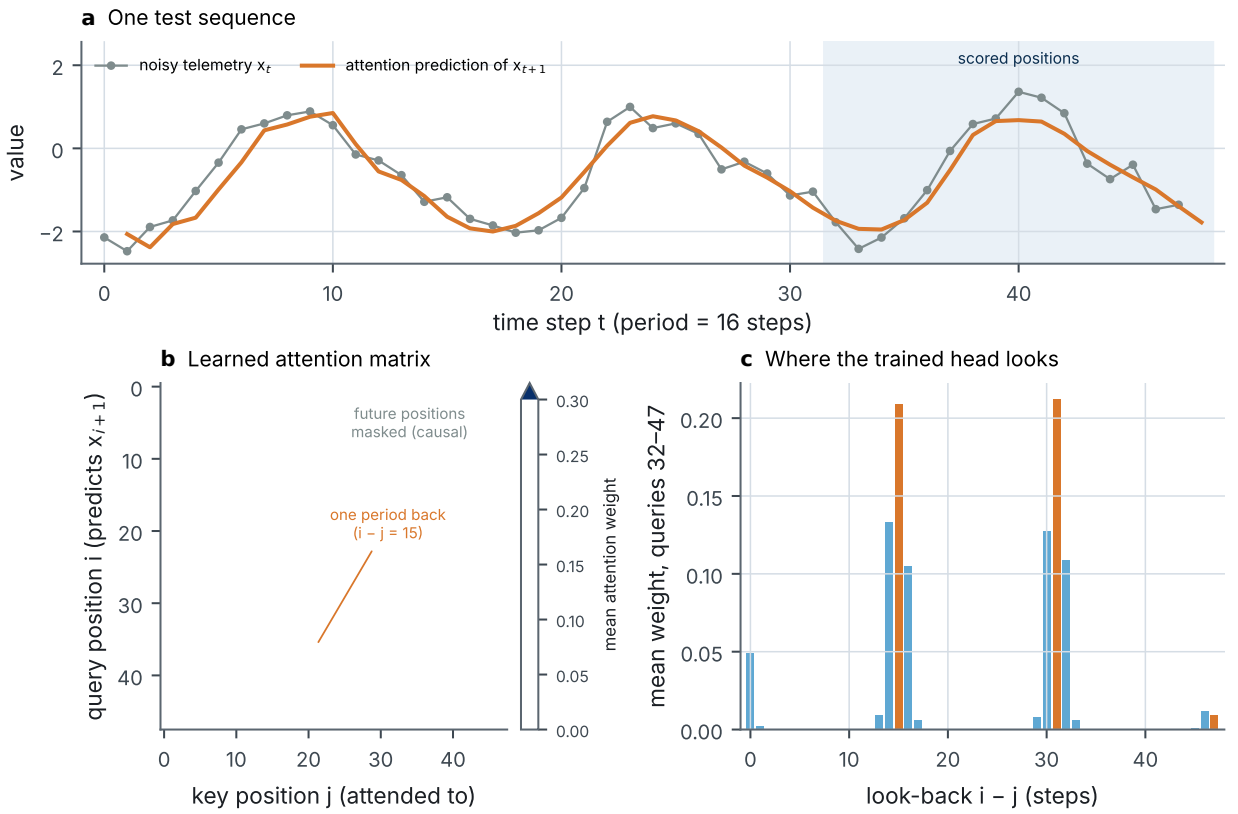



**FIGURE 44.7** A causal self-attention layer trained in numpy on noisy periodic signals. (a) One test sequence and its forecast. (b) Mean learned attention weights: bright diagonals one and two periods back. (c) Mean weight against look-back: 0.21 at 15 and at 31 steps. Test RMSE 0.331, against 0.551 for persistence.

### 44.3.3 The Transformer

The **transformer** (2017) is built from attention. Each layer has a **multi-head attention** block: several attention operations in parallel, so that different heads can track different relationships. A small two-layer network, applied to each position, follows. Both parts have residual connections and layer normalisation. A layer of width $d$ has about $12d^2$ weights: $4d^2$ for the attention projections and $8d^2$ for the position-wise network.

Attention ignores order, so each element's position is added to its input vector as a **positional encoding**. The original transformer used sine waves of different frequencies, $PE_{p,2i} = \sin(p/10\,000^{2i/d})$ and $PE_{p,2i+1} = \cos(p/10\,000^{2i/d})$; many newer models learn them.

The **vision transformer (ViT)** (2020) cuts an image into patches, typically 16 × 16 pixels, and treats them as a sequence of "words". The **Swin transformer** (2021) limits attention to windows of 7 × 7 patches and shifts the windows between layers so that information still spreads. Prithvi uses ViTs; Microsoft's Aurora weather model uses a three-dimensional Swin transformer.

For time series, transformers treat each time step, or a short patch of steps, as a token. They can relate events far apart, such as a solar active region seen again one rotation (27 days) later. But they are not always better: a 2023 study found that a simple linear model beat several transformer forecasters on standard benchmarks. Always compare with persistence, climatology and a linear model.

### 44.3.4 Why Attention Is Expensive

Attention compares every token with every other one, so its cost grows with the **square** of the sequence length $n$. Per layer, the projections and position-wise network cost about $24nd^2$ FLOP and the attention itself $4n^2d$. The first term dominates for short sequences, the second for long ones.

> **EXAMPLE: ATTENTION COST FOR AN IMAGE SPLIT INTO 16 × 16 PATCHES**
>
> Take a ViT-Base layer: width $d = 768$ and 12 heads.
>
> 1. **A 224 × 224 image** gives $(224/16)^2 =$ **196 patches**. The linear parts cost $24 \times 196 \times 768^2 = 2.8$ GFLOP and attention $4 \times 196^2 \times 768 = 0.12$ GFLOP: only 4 % of the layer. The attention matrices of all 12 heads, stored in 16-bit numbers, take about 1 MB.
> 2. **A 1 024 × 1 024 image** gives 4 096 patches. Linear parts: 58 GFLOP; attention: 51.5 GFLOP, now 47 % of the cost. The score matrices take $4\,096^2 \times 12 \times 2$ bytes = **403 MB per layer**.
> 3. **A 4 096 × 4 096 solar image** from SDO gives 65 536 patches. Attention costs 13 200 GFLOP per layer — 93 % of the total — and its score matrices would need **103 GB per layer**, more than most processors hold.
> 4. **Windows.** Restricting attention to 7 × 7-patch windows, as in Swin, cuts the attention cost to $4 \times 65\,536 \times 49 \times 768 = 9.9$ GFLOP, about 1 340 times less.
> 5. **Conclusion.** Full-resolution images and long records need windowed, sparse or memory-efficient attention. The Surya solar model, for example, combines spectral filtering with long- and short-range attention to handle 4 096 × 4 096-pixel SDO images.

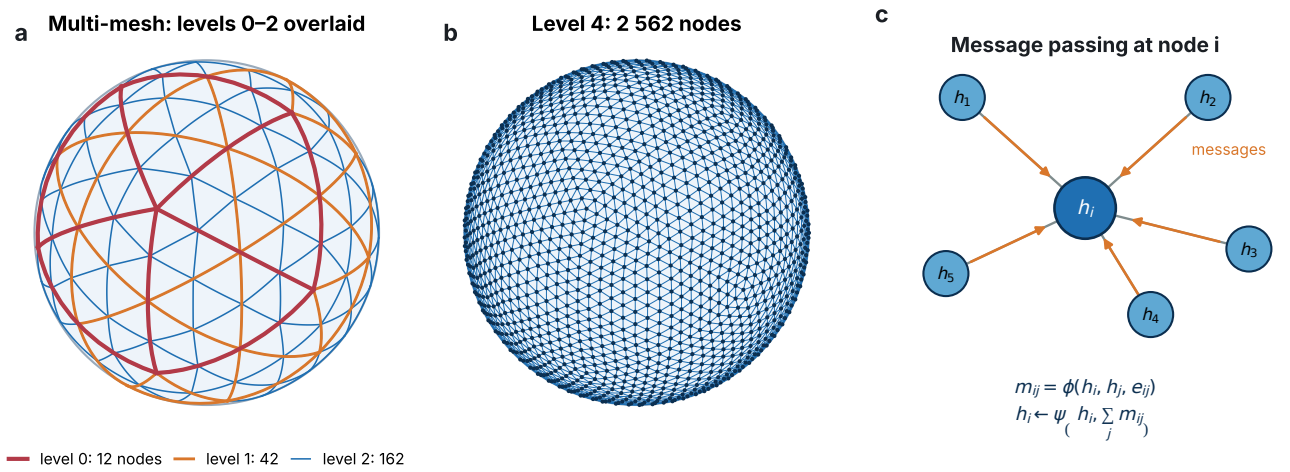
### 44.3.5 Graph Neural Networks

Many space problems are naturally **graphs**: nodes joined by edges. Weather on a sphere can be held on the points of a mesh, and a constellation is satellites joined by inter-satellite links. A **graph neural network (GNN)** learns on such structures by **message passing**. Each node $i$ holds a feature vector $\mathbf{h}_i$. In each layer, every neighbour $j$ sends a message computed by a small network $\phi$. The node then updates itself with another network $\psi$:

$$\mathbf{m}_{ij} = \phi(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij}), \qquad \mathbf{h}_i \leftarrow \psi\Big(\mathbf{h}_i, \sum_j \mathbf{m}_{ij}\Big),$$

where $\mathbf{e}_{ij}$ describes the edge, such as its length and direction. The same small networks are used at every node, so a GNN works on graphs of any size. After $L$ layers each node has heard from nodes up to $L$ edges away.

Google DeepMind's **GraphCast** (2023) maps the atmosphere from a 0.25° grid (about one million points, 37 pressure levels) onto an icosahedral mesh, passes messages 16 times and decodes the result back to the grid. The mesh is made by splitting each triangle of an icosahedron into four, six times over (Figure 44.8). The finest level has 40 962 nodes about 120 km apart. Its "multi-mesh" keeps the edges of all coarser levels too, so its 327 660 directed edges include links about 7 000 km long, and information crosses the globe in a few steps. With 36.7 million parameters, it makes a 10-day forecast in under a minute (Chapter 47).




Computed: a regular icosahedron refined by splitting each triangle into four; level 6 has 40 962 nodes (81 920 faces, mean edge 120 km) — the finest level of GraphCast's mesh. Level-0 edges are about 7 054 km long, so a few message-passing steps on the multi-mesh carry information across the globe.

**FIGURE 44.8** The icosahedral mesh of graph-network weather models, computed by repeated subdivision. (a) Levels 0–2 overlaid. (b) Level 4, 2 562 nodes. (c) Message passing at one node. Level 6 has 40 962 nodes with a mean spacing of 120 km.

ECMWF's operational AIFS model, running since 25 February 2025, also uses graph layers to encode and decode the grid, with a transformer in between. GNNs for routing traffic through constellations and for screening conjunctions are still mostly research (Chapters 49 and 50).

## 44.4 Generative Models

A **generative model** learns the probability distribution of its training data and can draw new samples from it: new images, new weather states, new designs. In space engineering this supports super-resolution, synthetic training data, filling gaps under clouds, ensemble forecasts and design exploration. Table 44.3 compares the main families.

**TABLE 44.3** Families of generative model.

| Family | How it generates | Strengths | Weaknesses | Space uses |
|---|---|---|---|---|
| Autoencoder, VAE | Decodes a point from a compact latent space | Compression, smooth latent space | Blurry samples | Anomaly detection, compression |
| GAN | Generator trained to fool a discriminator | Sharp images, fast sampling | Unstable training, misses modes | Super-resolution, SAR-to-optical translation |
| Diffusion | Removes noise step by step | High quality, covers all modes | Many steps, slow sampling | Ensemble weather (GenCast), downscaling |
| Flow matching | Follows a learned velocity field from noise to data | Straighter paths, fewer steps | Newer, less tested | Fast image and weather generation |
| Autoregressive | Predicts the next token or patch | Flexible, works for text | Errors accumulate | Language models, any-to-any EO models |

### 44.4.1 Autoencoders and Variational Autoencoders

An **autoencoder** squeezes its input through a narrow **latent** layer and then reconstructs it, so it must capture the main structure of the data in a few numbers. Inputs that it reconstructs badly are unusual, which makes autoencoders popular for telemetry anomaly detection (Chapter 49).

A **variational autoencoder (VAE)** (2013) makes the latent space probabilistic. Its encoder outputs a mean and spread for a latent vector $\mathbf{z}$. Training maximises a lower bound on the log-probability of the data.

$$\log p(\mathbf{x}) \;\geq\; \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\big[\log p(\mathbf{x}|\mathbf{z})\big] - \mathrm{KL}\big(q(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big).$$

The first term rewards good reconstructions. The second keeps the latent codes close to a simple Gaussian, so new samples can be made by decoding random $\mathbf{z}$. VAE samples tend to be blurry, but their compact latents are used inside many larger models.

### 44.4.2 Generative Adversarial Networks

A **generative adversarial network (GAN)** (2014) trains two networks against each other. A **generator** $G$ turns random noise $\mathbf{z}$ into a fake sample, and a **discriminator** $D$ tries to spot the fakes. They play the game:

$$\min_{G}\max_{D}\; \mathbb{E}_{\mathbf{x}}\big[\log D(\mathbf{x})\big] + \mathbb{E}_{\mathbf{z}}\big[\log\big(1 - D(G(\mathbf{z}))\big)\big].$$

GANs make sharp images quickly. They have been used to sharpen satellite images and to make radar images look optical. Their training is unstable, and they can suffer **mode collapse**, producing only a few kinds of output. A GAN can also invent plausible detail: a "super-resolved" building or crater must never be mistaken for a measurement.

### 44.4.3 Diffusion Models

**Diffusion models** learn to reverse a gradual noising process. The **forward process** adds a little Gaussian noise at each of $T$ steps, with variances $\beta_t$, until only noise remains. Its useful property is that the noisy sample at any step can be written in one go:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\; (1-\bar{\alpha}_t)\,\mathbf{I}\big), \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

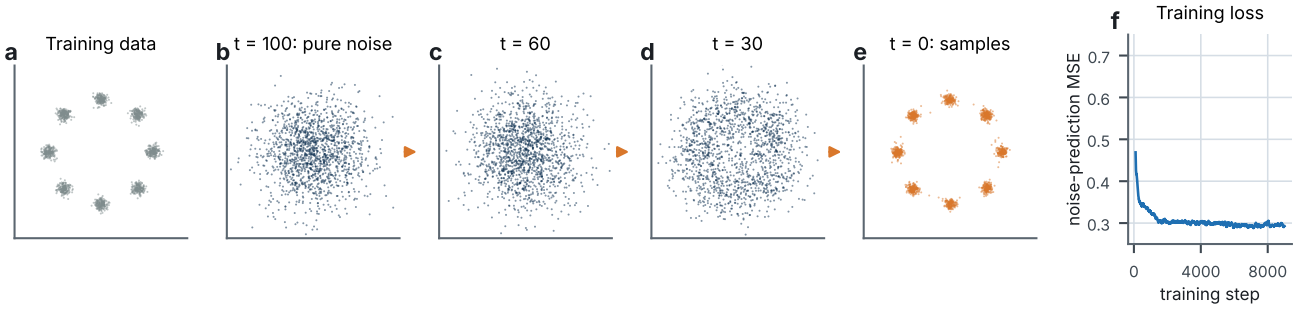
A network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is trained to predict the noise that was added, with the simple loss $\mathbb{E}\,\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$. To generate, the **reverse process** starts from pure noise and removes a little predicted noise at each step:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z},$$

where $\mathbf{z}$ is fresh noise and $\sigma_t$ a set variance. The noise predictor is, in effect, learning the direction towards more probable data at every noise level.

In Figure 44.9, a complete diffusion model written in numpy learns a ring of eight clusters. It uses 100 noise steps and a network with three hidden layers of 128 units, trained for 9 000 steps (a little over a minute on a desktop processor). The reverse process gathers 4 000 points of pure

noise onto the ring. Of the generated points, 98.5 % lie within three standard deviations of a cluster centre, against 98.7 % of real samples. Every cluster gets 10.9–14.0 % of the points, close to the ideal 12.5 %, so no mode has been lost.




Numpy DDPM: noise predictor 18-128-128-128-2 (SiLU), sinusoidal time embedding, T = 100 steps, linear β from $10^{-4}$ to 0.12, 9 000 Adam steps on a ring of eight Gaussians. 98 % of 4 000 generated samples lie within 3σ of a mode; every mode is populated (11–14 % each).

**FIGURE 44.9** A two-dimensional diffusion model trained in numpy. (a) Training data: eight clusters on a ring. (b–e) Samples during the reverse process, from pure noise at step 100 to step 0. (f) Training loss. 98.5 % of samples fall within 3σ of a cluster, and every cluster is populated.

**Flow matching** (2022) is a close relative. It draws a straight line from a noise sample $\mathbf{x}_0$ to a data sample $\mathbf{x}_1$, $\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$, and trains a network to predict the velocity $\mathbf{x}_1 - \mathbf{x}_0$. Sampling integrates $d\mathbf{x}/dt = \mathbf{v}_\theta(\mathbf{x}, t)$, and the straighter paths need fewer steps.

### 44.4.4 Generative Models in Space

The most important space use so far is **ensemble weather forecasting**. Google DeepMind's **GenCast** (December 2024) is a diffusion model on the sphere that generates 15-day, 0.25° forecasts as ensembles of 50 or more members. On 2019 weather it beat ECMWF's ensemble on 97.2 % of 1 320 combinations of variable, level and lead time. One member takes about 8 minutes on one TPU v5 processor. For hurricanes (Figure 44.10), forecasters need the probability of each track, not one best guess (Chapter 47).

**FIGURE 44.10** Hurricane Milton, a Category 5 storm, in the Gulf of Mexico, seen from the International Space Station on 8 October 2024. Diffusion-based ensembles aim to give the probabilities of tracks and intensities of storms like this. Image: NASA/JSC.

Generative models also **downscale** coarse fields: NVIDIA's CorrDiff research model turns 25 km atmospheric data into 2 km detail over Taiwan. They create **synthetic training data**, such as rendered spacecraft images (Chapter 45), and propose structures in **generative design** (Chapter 52). TerraMind can generate a land-cover map or radar-like image from an optical one. In every case the output is a plausible sample, not a measurement.

## 44.5 Foundation Models and Self-Supervised Learning

A **foundation model** is a large network pre-trained on huge amounts of unlabelled data, then adapted to many tasks with few labels. In space, labelled flood maps or flare catalogues take experts months to build, while satellites collect petabytes of unlabelled data. Foundation models exploit that imbalance (Figure 44.11).

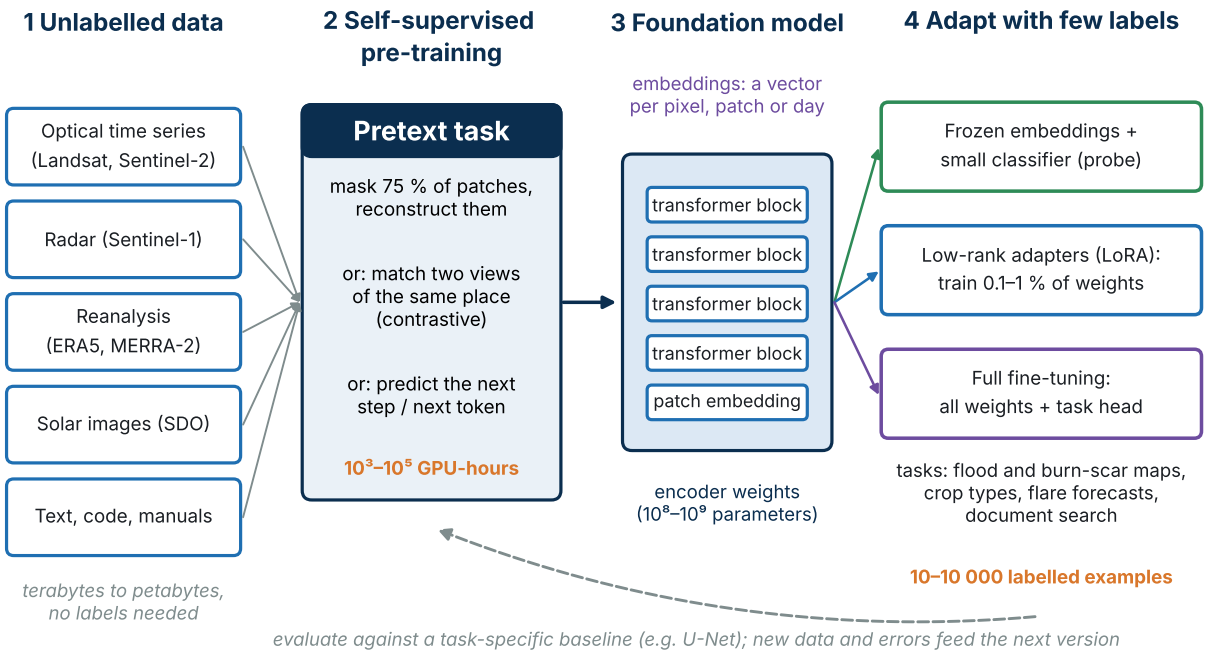



**FIGURE 44.11** How foundation models are built and used. Self-supervised pre-training on unlabelled archives produces an encoder that turns any input into embeddings. Small labelled data sets then adapt it to each task, and results are checked against a task-specific baseline.

### 44.5.1 Learning Without Labels

**Self-supervised learning** invents a training task from the data itself. In **masked autoencoding (MAE)** (2021) the image is cut into patches, typically 75 % of them are hidden, and the network must reconstruct the missing ones. To fill in a hidden field or coastline, it must learn what landscapes, seasons and clouds look like.

**Contrastive learning** teaches the network that two views of the same thing should map to nearby vectors. The views might be two crops of a scene, radar and optical images of the same place, or an image and its caption. Views of different things should map far apart. The common **InfoNCE** loss for a pair of embeddings $\mathbf{z}_i$ and $\mathbf{z}_j$ is

$$\ell_{ij} = -\log \frac{\exp\left(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau\right)}{\sum_{k \neq i} \exp\left(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau\right)},$$

where sim is the cosine similarity and $\tau$ a "temperature" setting. In 2021 an image–text model trained this way on 400 million captioned images learned to recognise objects it had never been labelled for.

The output of a pre-trained encoder is an **embedding**: a vector of tens to thousands of numbers that summarises a pixel, patch, image or day. Similar inputs have similar embeddings. Google DeepMind's **AlphaEarth Foundations** (July 2025) summarises a year of optical, radar, lidar and other data for every 10 m patch of land in 64 numbers. Annual layers for 2017–2025 are open in Google Earth Engine. Users map crops or change with a simple model on top. Google reports about 24 % lower error than the other approaches it tested.

### 44.5.2 Adapting a Foundation Model

There are three main ways to adapt a pre-trained model (Figure 44.11). A **linear probe** freezes the encoder and trains only a small classifier on its embeddings; it is cheap and hard to overfit. **Full fine-tuning** updates all weights with a new task head; it is usually the most accurate but needs memory for every weight's gradients and optimiser state.

**Parameter-efficient fine-tuning** sits in between. Its most popular form, **low-rank adaptation (LoRA)** (2021), freezes each large weight matrix $W_0$. It learns a small correction made of two thin matrices:

$$W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll d, k.$$

Here $r$ is the rank. For a 1 024 × 1 024 matrix with $r$ = 8, the correction has 16 384 trainable numbers instead of 1 048 576 — 1.6 %. For a 175-billion-parameter language model, the LoRA authors cut the trainable parameters about 10 000-fold and the memory about threefold. **Few-shot learning** goes further, adapting from a handful of examples.

### 44.5.3 Earth and Space Foundation Models

Since 2023 many foundation models have been released for Earth observation (Table 44.4) and for the atmosphere, the Sun and the Moon (Table 44.5). Most are open. They differ in data, size and design, so their published scores are hard to compare.

**TABLE 44.4** Earth-observation foundation models (status October 2026). HLS: Harmonized Landsat Sentinel-2.

| Model | Developer, year | Pre-training data | Size (parameters) | Licence |
|---|---|---|---|---|
| SatMAE | Stanford, 2022 | fMoW RGB and Sentinel-2 images, with time | ViT-B, ViT-L | Public code and weights |
| Prithvi-EO-1.0 | NASA, IBM, 2023 | > 1 TB of HLS optical time series, contiguous US | 100 M | Apache-2.0 |
| SkySense | Ant Group, Wuhan Univ., 2024 | 21.5 M optical and SAR time series | 2.06 B | Non-commercial research |
| Clay v1.5 | Clay Foundation, 2024 | ~70 M chips: Sentinel-1/2, Landsat, NAIP, MODIS, others | 632 M (311 M encoder) | Apache-2.0 |
| Prithvi-EO-2.0 | NASA, IBM, Jülich, 2024 | 4.2 M global HLS time series, 2014–23 | 300 M, 600 M | Apache-2.0 |

| Model | Developer, year | Pre-training data | Size (parameters) | Licence |
|---|---|---|---|---|
| TerraMind | IBM, ESA Φ-lab, Jülich, 2025 | 9 M aligned samples, 9 modalities incl. SAR, DEM, text | Base, Large | Apache-2.0 |
| AlphaEarth Foundations | Google DeepMind, 2025 | Optical, radar, lidar, climate | 64-number embeddings | Embeddings CC-BY 4.0 |
| OlmoEarth | Allen Institute for AI, 2025 | ~10 TB optical, radar and map layers | 1.4 M–300 M | OlmoEarth Artifact License |

**TABLE 44.5** Foundation models for the atmosphere, Sun, Moon and science text (status October 2026).

| Model | Developer, year | Pre-training data | Size | Main uses | Licence |
|---|---|---|---|---|---|
| Prithvi WxC | NASA, IBM, 2024 | 160 MERRA-2 variables, 40 years | 2.3 B | Downscaling, gravity waves, forecasts | CDLA-Permissive-2.0 |
| Aurora | Microsoft, 2024 (Nature 2025) | > 1 million hours of geophysical data | 1.3 B | Weather, air quality, waves, cyclones | MIT |
| Surya | NASA, IBM, 2025 | 9 years of SDO images, ~218 TB | 366 M | Flares, solar wind, active regions | Apache-2.0 |
| Lunar Foundation Model | NASA, IBM, 2026 | ~2 M tile bundles from LRO, Kaguya, GRAIL and others | ViT-B size | Craters, ice prospecting | Apache-2.0 |
| INDUS | NASA, IBM, 2024 | 60 B tokens of science text | 125 M encoder, smaller variants | Search, retrieval | Apache-2.0 |
| EVE | ESA Φ-lab, Pi School, 2026 | Earth-science text, open base model | 24 B | Question answering with sources | Open release planned |

DATA BOX — PRITHVI-EO-2.0 IN NUMBERS

- **Data.** About 4.2 million samples from the global Harmonized Landsat Sentinel-2 archive (Figure 44.12), each 256 × 256 pixels at 30 m with four time steps, from 2014–2023.
- **Compute.** 400 epochs on the JUWELS Booster supercomputer in Jülich: about 21 000 GPU-hours on 80 A100 processors for the 300-million-parameter model, and about 58 000 GPU-hours on 240 processors for the 600-million version.
- **Gains.** About 8 % better than Prithvi-EO-1.0 on the GEO-Bench suite. Burn scars: 90.5 % mean intersection-over-union (mIoU), against 86.9 % for version 1.0; floods: 90.3 % against 88.3 %.
- **Few labels.** Landslide mapping with about 1 % of the training data, some 50 images: 67.0 % mIoU, against 59.7 % for a U-Net trained from scratch.

Foundation models do not always win. On the PANGAEA benchmark, the TerraMind authors report a mean mIoU of 59.6 % for their large model and 58.4 % for the base model. A U-Net trained for each task scored 57.6 %, and the first Prithvi model 51.0 %: only TerraMind beat the U-Net. Foundation models are most valuable when labels are scarce, tasks change often or data types must be combined.

The same recipe now reaches beyond Earth. **Surya** (August 2025) was pre-trained on about nine years of full-resolution images from the Solar Dynamics Observatory (Figure 44.13). Fine-tuned with LoRA, it segmented active regions with an intersection-over-union of 0.768 and, in preliminary tests, forecast flares about 16 % better than existing benchmarks. The **NASA-IBM**

**Lunar Foundation Model** (September 2026) cut the error of a polar-ice prospectivity map by about 22 % compared with a general vision model. Its developers state that it is not yet validated for operational decisions.

**FIGURE 44.12** Landsat 9 at Vandenberg Space Force Base on 16 August 2021, before encapsulation. Harmonised Landsat and Sentinel-2 data trained the Prithvi Earth-observation models. Image: NASA/KSC (NASA/Randy Beaudoin).

**FIGURE 44.13** The Solar Dynamics Observatory with its arrays deployed in pre-launch tests, 2009. About nine years of its images, some 218 TB, trained the Surya model. Image: NASA/KSC.

LANDMARK: THE FIRST FOUNDATION MODEL IN ORBIT (2025–26)

- Researchers from Adelaide University, ESA Φ-lab, Thales Alenia Space and the SmartSat Cooperative Research Centre compressed Prithvi-EO-2.0-300M for spacecraft processors.
- They used knowledge distillation: training a small “student” network to copy the large “teacher”. It cut the width from 1 024 to 256 and the file from 1.2 GB to 73 MB, about 16 times smaller. Accuracy on cloud detection stayed within about one percentage point, and biomass error within 3 %.
- The compact model ran on the IMAGIN-e payload on the International Space Station, on an ordinary 16-core ARM processor, using an uploaded test set. For South Australia’s Kanyini cubesat, the December 2025 preprint reported only ground tests on an engineering model. In May 2026 NASA reported that the model had also been uploaded to Kanyini. It called it the first geospatial foundation model deployed in orbit.
- Open weights spared the team from training its own model. Onboard hardware and compression are discussed in Chapter 45.

## 44.6 Large Language Models and Agents

A **large language model (LLM)** is a transformer trained to predict the next piece of text. Scaled to billions of parameters and trillions of words, this simple task yields systems that answer questions, summarise documents, write code and follow instructions. In 2025 such models even helped plan a Mars rover drive.

### 44.6.1 How Language Models Are Trained

Text is first split into **tokens**: common words, parts of words or single characters, typically about three-quarters of an English word each. A model with parameters $\theta$ learns to give a high probability to each actual next token. **Pre-training** minimises the average negative log-probability over a huge text corpus:

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_1, \dots, x_{t-1}).$$

To predict well, the model must absorb grammar, facts and some reasoning patterns, but it only continues text. **Instruction tuning** fine-tunes it on examples of requests and good answers. In **reinforcement learning from human feedback (RLHF)**, people rank alternative answers, a **reward model** learns their preferences, and the language model is tuned to please it. In a 2022 study, raters preferred a 1.3-billion-parameter model tuned this way to an untuned one over 100 times larger.

The **context window** is the amount of text the model can consider at once. In 2026 leading commercial models accepted about a million tokens, well over a thousand pages, though they use long contexts less reliably than advertised. **Multimodal** models also accept images, diagrams or audio, often turning image patches into tokens with a ViT.

### 44.6.2 Retrieval, Tools and Agents

A language model's knowledge is frozen at training time and may be wrong. **Retrieval-augmented generation (RAG)** helps with both. Documents such as requirements, anomaly reports and standards are split into passages and turned into embeddings. For each question, the nearest passages are retrieved and placed in the prompt, and the model answers from them with citations. NASA and IBM's **INDUS** models (2024), trained on 60 billion tokens of science text, support this kind of search. ESA's **Earth Virtual Expert (EVE)** combines a 24-billion-parameter domain-tuned model with retrieval and a hallucination check; it had about 350 pilot users in March 2026.

**Tool use** lets a model call programs: a calculator, a database, an orbit propagator or a code interpreter. An **agent** wraps the model in a loop — plan, call tools, read the results, repeat — until the goal is reached. In December 2025 JPL engineers let commercial vision–language models plan waypoints for two Perseverance drives, of 210 m and 246 m. Every command was first checked on JPL's digital twin of the rover, which simulates over 500 000 telemetry variables (Figure 44.14).

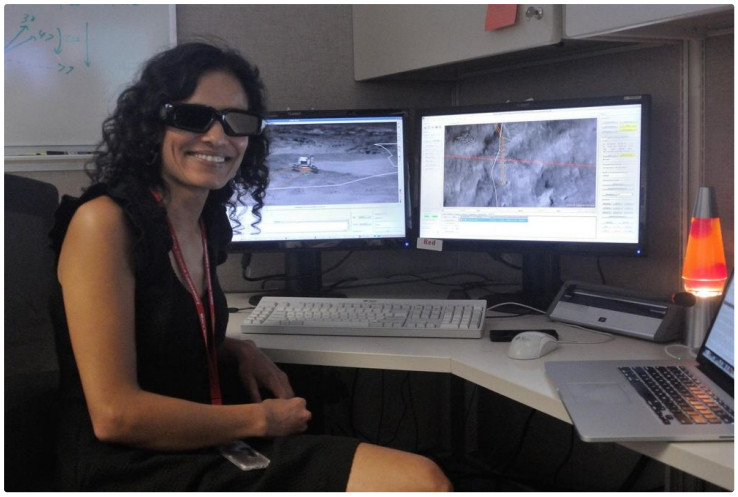


**FIGURE 44.14** A JPL rover driver plans a Curiosity drive, using 3D glasses to see the terrain's shape. In December 2025, vision–language models proposed waypoints for two Perseverance drives; people checked every command on a digital twin. Image: NASA/JPL (NASA/JPL-Caltech).

### 44.6.3 Hallucination

A language model generates fluent text whether or not it knows the answer. A confident, plausible but false statement is called a **hallucination**. Retrieval reduces it but does not remove it. A 2025 study of commercial legal-research tools built on RAG found that they still gave false or unsupported answers to 17–33 % of test questions.

In engineering this is a serious hazard: a wrong torque value or an invented requirement looks exactly like a correct one. Treat language-model output as a draft from an unverified source. Answers should cite sources that people can check, and safety-related changes need normal engineering review. Space uses are discussed in Chapters 49 (operations assistants), 52 (systems engineering) and 53 (assurance).

**EXAMPLE: MEMORY FOR A 7-BILLION-PARAMETER LANGUAGE MODEL**

A typical open model of this size has 32 transformer layers of width 4 096.

1. **Weights.** At 16 bits (2 bytes) per parameter: $7 \times 10^9 \times 2$ = **14 GB**. At 8 bits: **7 GB**. At 4 bits: **3.5 GB**, or about 3.7 GB with one 16-bit scale factor per 64 weights (4.25 bits per weight).
2. **Key–value cache.** During generation each layer stores a key and a value vector for every token: 2 × 32 × 4 096 × 2 bytes = 0.5 MB per token. A 4 096-token context needs **2.1 GB**; 32 768 tokens need **17 GB** — more than the 16-bit weights.
3. **Full fine-tuning.** Adam in mixed precision needs about 16 bytes per parameter for weights, gradients, two moments and a 32-bit master copy: about **112 GB** before activations.
4. **LoRA.** Rank-16 adapters on two projection matrices per layer add $32 \times 2 \times 16 \times (4\,096 + 4\,096)$ = **8.4 million** trainable parameters, 0.12 % of the model. Their optimiser state needs about 0.13 GB.
5. **Conclusion.** With 4-bit weights and LoRA, the model can be fine-tuned on one 24 GB processor and run on a laptop, but it is far too large for most flight computers (Chapter 45).

## 44.7 Physics, Uncertainty and Explanation

Space engineering needs more than a good average score. A model must respect physics, say how sure it is and give reasons that engineers can check.

### 44.7.1 Physics-Informed Networks and Neural Operators

A **physics-informed neural network (PINN)** (2019) adds the governing equations to the loss. If the physics is written as a differential equation $\mathcal{N}[u] = 0$, the network $u_\theta$ is trained on

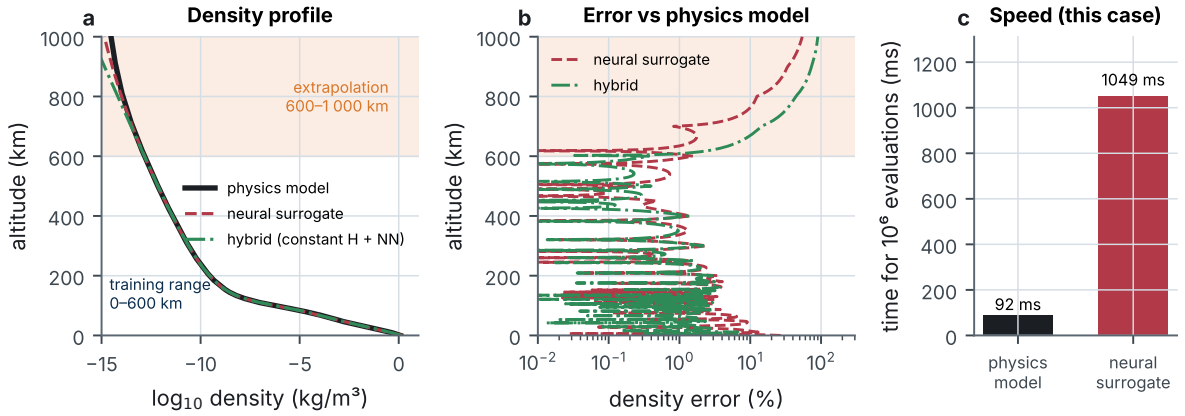
$$\mathcal{L} = \frac{1}{N_d}\sum_{i=1}^{N_d}\left|u_\theta(\mathbf{x}_i) - u_i\right|^2 + \lambda\,\frac{1}{N_r}\sum_{j=1}^{N_r}\left|\mathcal{N}[u_\theta](\mathbf{x}_j)\right|^2.$$

The first term fits the $N_d$ measurements. The second penalises the **residual** of the equations at $N_r$ points, where no data are needed; derivatives come from the same automatic differentiation used in backpropagation. Martin and Schaub's PINN gravity models learn the potential of an irregular asteroid such as Eros and take the acceleration as its gradient, which helps them extrapolate.

A **neural operator** learns a mapping between whole functions, such as from today's weather field to tomorrow's. The **Fourier neural operator (FNO)** (2020) applies a fast Fourier transform, multiplies the lowest frequencies by learned weights $R_\phi$ and transforms back: $v_{l+1} = \sigma\big(W v_l + \mathcal{F}^{-1}[R_\phi \cdot \mathcal{F}(v_l)]\big)$. Working with frequencies, it can be trained at one resolution and applied at another (Chapter 47).

A **surrogate** or **emulator** is a fast network trained to imitate a slow simulation, such as fluid dynamics or a climate model, enabling thousands of design trades (Chapter 52). **Differentiable simulators** compute gradients through the physics itself, so a trajectory or antenna shape can be optimised directly.

In Figure 44.15, a small network learns atmospheric density from 0 to 600 km from a standard-atmosphere model. Within that range its median error is 0.75 %, with a worst case of 27 % at the ground, the edge of the range. Above 600 km, where it has never seen data, the error grows to −12 % at 800 km and −54 % at 1 000 km. A hybrid that adds a learned correction to a constant-scale-height model fits better in range (median 0.57 %, worst case 6.9 %). But it extrapolates even worse, −90 % at 1 000 km, because the real scale height keeps rising as light helium takes over: a hybrid is only as good as its physics. Nor is speed automatic. Here the closed-form physics model is about ten times faster than the network, so surrogates pay off only when the model they replace is expensive.




Physics model: US Standard Atmosphere 1976 below 86 km and exponential segments (Vallado) to 1 000 km. Surrogates: numpy MLP 1-32-32-1 (tanh), 12 000 Adam steps on 601 altitudes, 0–600 km; hybrid = constant-scale-height fit to 500–600 km plus a learned correction. Timings: one CPU core, vectorised.

**FIGURE 44.15** A neural surrogate of atmospheric density, trained in numpy on 0–600 km only. (a) Density from the physics model and two surrogates. (b) Error: below 1 % on average in range, tens of per cent beyond it. (c) Time for a million evaluations: here the physics model is about ten times faster.

### 44.7.2 Uncertainty Quantification

Every prediction has two kinds of uncertainty. **Aleatoric** uncertainty is noise in the data themselves, such as a sensor's random error; more data do not reduce it. **Epistemic** uncertainty is the model's ignorance, largest where training data are sparse or absent; more data do reduce it. A useful model should report both.

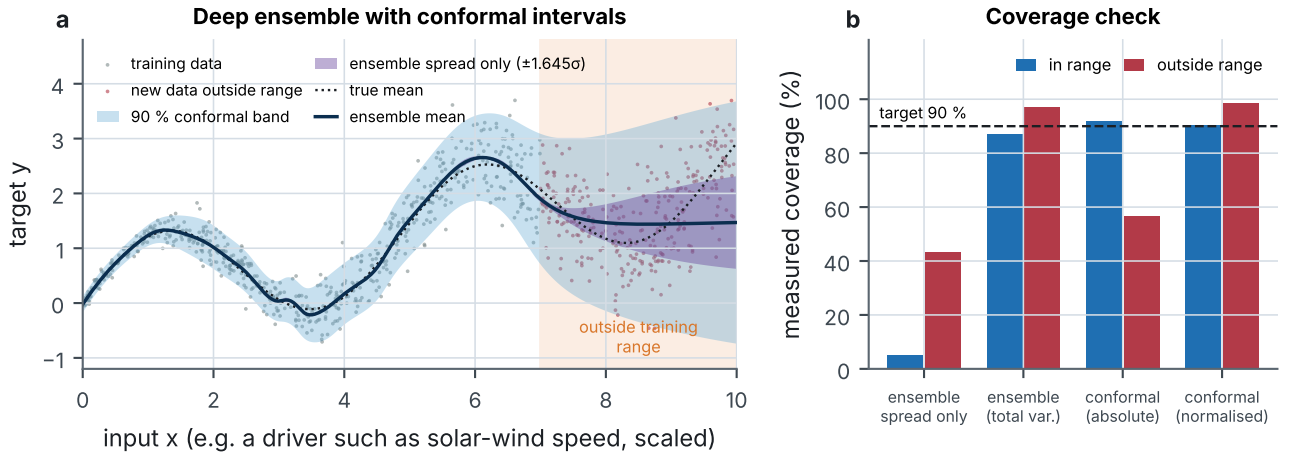
A network can learn aleatoric uncertainty by predicting a mean $\mu(x)$ and a variance $\sigma^2(x)$, trained with the Gaussian negative log-likelihood $\frac{1}{2}\log\sigma^2 + (y-\mu)^2/(2\sigma^2)$. For epistemic uncertainty, the methods in Table 44.6 compare several plausible models. A **deep ensemble** trains several networks from different random starts; their disagreement measures epistemic uncertainty. The total variance is the mean of the members' variances plus the variance of their means.

**TABLE 44.6** Ways to quantify the uncertainty of a neural network.

| Method | Captures | Extra cost | Guarantee | Notes |
|---|---|---|---|---|
| Mean–variance network | Aleatoric | None | None | Overconfident far from data |
| Deep ensemble | Both | 5–10 × training and running | None | Simple, strong baseline |
| Monte Carlo dropout | Mostly epistemic | Many passes at run time | None | Cheap to add; often underestimates |
| Bayesian neural network | Both | High | Only if the prior and approximation are right | Hard to scale |
| Conformal prediction | Total (calibrated) | A calibration set | Coverage if data are exchangeable | Wraps any model |

**Monte Carlo dropout** keeps dropout on at prediction time and treats many passes as an ensemble. **Bayesian neural networks** place probability distributions on the weights. **Conformal prediction** wraps any model in intervals with guaranteed coverage. On a separate calibration set of $n$ points it computes the scores $s_i = |y_i - \hat{y}_i|/\hat{\sigma}_i$. It takes $q$ as the $\lceil (n+1)(1-\alpha) \rceil$-th smallest score and reports $\hat{y} \pm q\hat{\sigma}$. If new data are **exchangeable** with the calibration data, meaning drawn from the same distribution, a fraction of at least $1-\alpha$ of the intervals will contain the truth.

In Figure 44.16, the noise of a regression problem grows across the input range. In range, the spread of five ensemble members alone covers the truth only 5 % of the time: they agree with each other but ignore the noise. The ensemble's total variance gives 87 % coverage at a nominal 90 %; conformal intervals give 91.7 % (absolute) and 90.4 % (normalised). Outside the range, the members disagree, with 40 times more epistemic spread, so the normalised intervals widen and still cover 98 %. The fixed-width band covers only 57 %: its guarantee assumed data like the calibration set.




Synthetic regression y = sin(1.3x) + 0.25x with noise s.d. rising from 0.08 to 0.50. Five numpy MLPs (1-64-64-2, tanh) trained on 600 points with the Gaussian negative log-likelihood; conformal quantiles from 400 calibration points; coverage measured on 4 000 in-range and 2 000 out-of-range test points.

**FIGURE 44.16** A deep ensemble of five numpy networks with conformal prediction. (a) Data, ensemble mean, member spread and a 90 % normalised conformal band. (b) Coverage of nominal 90 % intervals: member spread alone 5 %, total variance 87 %, conformal 91.7 % and 90.4 % in range; outside it, only variance-scaled intervals hold.

In space weather, Licata and Mehta (2022) trained a thermospheric-density model with calibrated uncertainty on US Space Force density data. A 2026 study of flare-intensity forecasts found that conformalised quantile regression gave the best coverage with the narrowest intervals. Calibrated uncertainty lets a conjunction analyst or forecaster decide whether to act (Chapters 47 and 50).

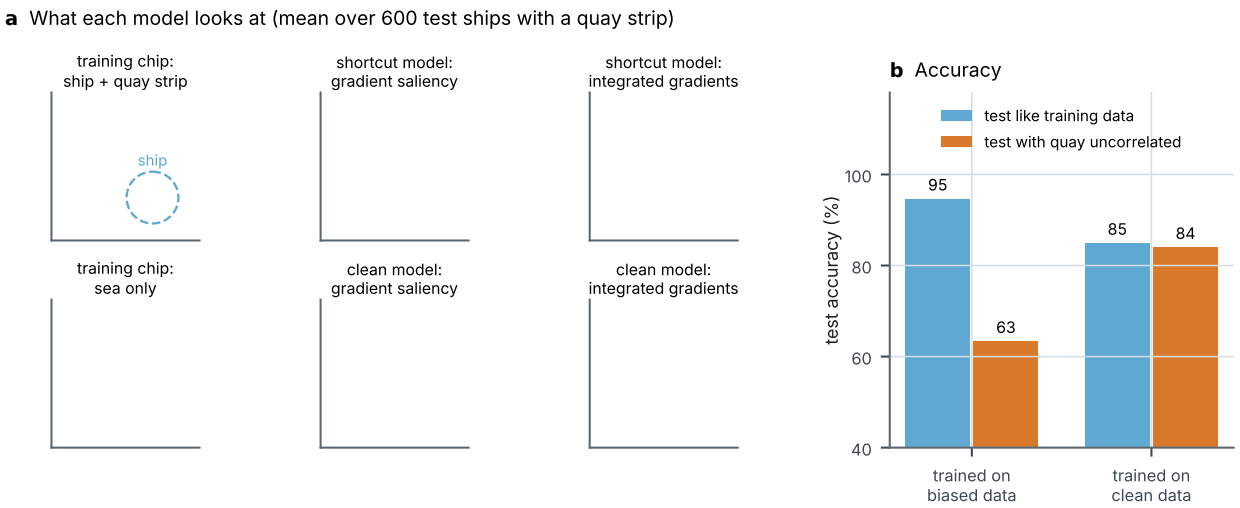
### 44.7.3 Explainability

**Explainable AI** tries to show why a model produced its output. A **saliency map** shows the gradient of the output with respect to each input pixel: where a small change would most alter the answer. **Grad-CAM** (2017) weights the feature maps $A^k$ of the last convolution layer by the average gradient of the class score $y^c$, $\alpha_k^c = \frac{1}{Z}\sum_{i,j} \partial y^c / \partial A_{ij}^k$, and shows $\mathrm{ReLU}\big(\sum_k \alpha_k^c A^k\big)$ as a heat map. **Integrated gradients** (2017) average the gradient along a straight path from a neutral baseline image $\mathbf{x}'$ to the input:

$$\mathrm{IG}_i(\mathbf{x}) = (x_i - x_i') \int_0^1 \frac{\partial F\big(\mathbf{x}' + a(\mathbf{x} - \mathbf{x}')\big)}{\partial x_i}\, da.$$

The attributions add up exactly to $F(\mathbf{x}) - F(\mathbf{x}')$. **SHAP** values (2017) share a prediction among the input features using Shapley values from game theory: each feature's average contribution over all subsets of the others.

Explanations are most useful for catching **shortcuts**, where a model learns an accidental cue instead of the real signal. In Figure 44.17, radar-like chips contain sea clutter and, in half of them, a ship. In the biased training set a bright quay strip along one edge accompanies 95 % of the ships. The trained network scores 95 % on similar test chips but 63 % when the strip appears at random, finding only 28 % of ships without one. Integrated gradients expose the shortcut: the strip, 8 % of the pixels, carries 68 % of the net evidence for "ship". When the strip is replaced by sea, detections fall from 98 % to 26 %. A network trained on clean data scores 84–85 % on both test sets, and its strip carries 1 % of the evidence.




Synthetic 24 × 24-pixel radar-like chips (gamma sea clutter; ships = bright elongated blobs). In the biased training set a bright quay strip on the left edge accompanies 95 % of ships. Numpy MLP 576-64-1 (ReLU), 60 Adam epochs on 6 000 chips. Gradient saliency: mean |∂logit/∂pixel|. Integrated gradients (red = evidence for ship, blue = against): 32 steps from a uniform clutter-mean baseline.

**FIGURE 44.17** Explaining a ship detector that learned a shortcut (synthetic data, numpy networks). (a) Training chips, and what each model uses, averaged over 600 test ships beside a quay strip: gradient saliency looks alike, but integrated gradients show the strip dominating the shortcut model. (b) Its accuracy falls from 95 % to 63 % on unbiased data.

The same figure shows the limits of explanation. The plain gradient maps of the two models look alike, and the strip carries only 7 % of the biased model's gradient: two methods, two answers. A 2018 study found that some popular saliency methods gave almost the same maps for trained and untrained networks. Test explanations, for example by removing the highlighted input and checking that the output changes.

Pandey and colleagues (2023) trained a convolutional network to forecast M-class and larger solar flares from full-disk magnetograms, with a true skill statistic of about 0.51. Guided Grad-CAM showed that it attended to active regions, even near the edge of the disk. Such an explanation supports a model but cannot prove it correct (Chapter 53).

### PRACTICE: FINE-TUNING AN EARTH-OBSERVATION FOUNDATION MODEL WITH FEW LABELS

1. **Define the task and the baseline.** For example, map flood water at 30 m within six hours of a pass. Choose the metric (water-class IoU) and a simple baseline, such as a water-index threshold or a small U-Net.
2. **Choose the model.** Pick an open model pre-trained on data like yours — Prithvi-EO-2.0 for HLS imagery, TerraMind for radar plus optical. Check its licence.
3. **Label a little, well.** Label 50–200 chips from many regions, seasons and event types. Have a second expert relabel 10 % of them to measure label noise.
4. **Split by place and time.** Hold out whole events and regions for validation and testing (Chapter 43). Never let chips from the same scene fall on both sides of a split.
5. **Start with a probe.** Freeze the encoder and train a small segmentation head on its embeddings. This takes minutes and sets a floor.
6. **Then adapt.** Add LoRA adapters (rank 8–16) or fine-tune all weights with a low learning rate ($10^{-5}$–$10^{-4}$), with early stopping on the validation set.
7. **Compare honestly.** Report the foundation model, the probe and the baseline on the locked test set, with confidence intervals. Keep the simplest model that meets the requirement.
8. **Check failure modes.** Inspect errors by surface type: cloud shadow, terrain shadow, urban areas, wet snow. Add labels where errors cluster.
9. **Calibrate and document.** Add conformal intervals, record data, version and limits in a model card, and monitor after deployment (Chapters 45 and 53).

### KEY TAKEAWAYS

- Deep networks stack linear layers and non-linear activations. They train reliably only with careful initialisation, normalisation, residual connections and optimisers such as Adam.
- Scaling laws predict how loss falls with parameters, data and compute; for a fixed budget, model size and data should grow together.
- CNNs exploit weight sharing in images: U-Nets and DeepLab segment, YOLO-type and two-stage networks detect. A small CNN may have under 100 000 weights yet need over a gigaflop per chip.
- Attention cost grows with the square of sequence length, so large images and long records need windowed or sparse attention. Graph networks pass messages on meshes.
- Diffusion models generate by removing noise step by step and underpin ensemble weather forecasts; generated outputs are samples, not measurements.
- Foundation models such as Prithvi, TerraMind, AlphaEarth, Aurora and Surya adapt to new tasks with few labels, but must still beat a simple baseline.
- Language models hallucinate: use retrieval with citations and human review. Check every model for extrapolation, calibration and shortcuts.

### REVIEW QUESTIONS

1. Why can a network without activation functions draw only straight decision boundaries? Why is ReLU preferred to the sigmoid in hidden layers?
2. Using Figure 44.1, explain what goes wrong when the weight variance in a 30-layer ReLU network is 1/n or 4/n, and how He initialisation and layer normalisation fix it.
3. Using $C \approx 6ND$, estimate the training compute of a 1-billion-parameter model trained on 20 billion tokens. Would extra compute be better spent on a bigger model or on more data?
4. Compute the parameters and multiply–accumulate operations of a 3 × 3 convolution with 64 input and 64 output channels on a 128 × 128 feature map. How would they change for a 5 × 5 kernel?

5. Which architecture would you use to count ships, to map flood extent and to outline individual buildings? Explain why.
6. Write the scaled dot-product attention equation and explain the roles of the queries, keys, values and the factor $\sqrt{d_k}$. In Figure 44.7, why does the trained head look back 15 and 31 steps rather than 16 and 32?
7. A transformer must process a 2 048 × 2 048 image in 16 × 16 patches. How many tokens is that, and how much memory do the 16-bit attention scores of 12 heads need per layer? Suggest two ways to reduce it.
8. Write the forward process of a diffusion model and state what the network learns to predict. Why is a super-resolved image not a measurement?
9. Describe masked autoencoding and LoRA. A team has 60 labelled landslide chips: outline how they should use a foundation model and how they would show it beats a U-Net.
10. In Figure 44.16, why does the member spread alone cover only 5 % of test points, and why does the fixed-width conformal band fail outside the training range? What does exchangeability mean for a space-weather model during a record storm?

CHAPTER

# 45 Data, Computing Hardware and Edge AI in Orbit

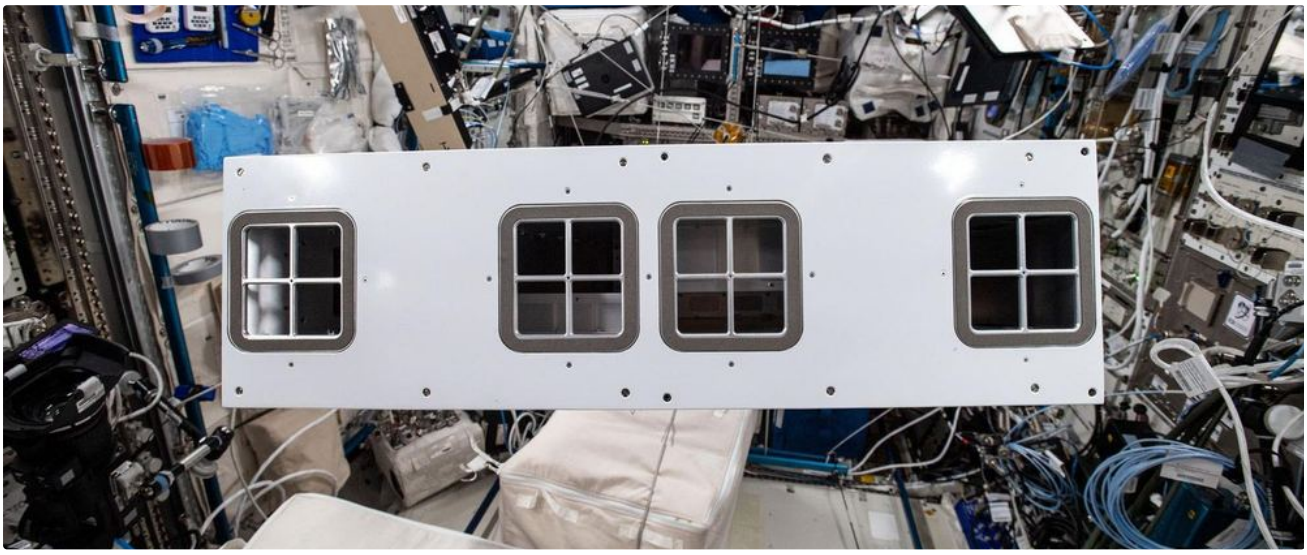


Image: NASA/JSC (Michael Hopkins)

*Every AI system in space rests on two foundations, data and computers. This chapter follows data from sensor to model. It covers petabyte archives, the kinds of space data, how labels are made, and the benchmarks and synthetic data used to train and test models. It explains the machine-learning operations (MLOps) practices that keep models honest in service. It then turns to hardware: why radiation flips bits in neural networks and how to protect them, which processors fly in 2026, and how compression makes models fit. It weighs processing on board against processing on the ground. It ends with updating models in orbit, federated learning across constellations, and the physics and economics of orbital data centres.*

IN THIS CHAPTER

45.1 Data for Space AI
45.2 Benchmarks, Competitions and Synthetic Data
45.3 MLOps: From Archive to Operations
45.4 Radiation and AI Hardware
45.5 Processors for AI in Orbit
45.6 Making Models Fit: Compression
45.7 On Board or on the Ground?
45.8 Updating AI in Orbit
45.9 Computing in Orbit: Orbital Data Centres

A modern Earth-observation satellite collects far more data than its radio link can send home. A model on the ground must wait for the data; a model in orbit must live with little power, intense radiation and a computer that cannot be repaired. Either way, it is only as good as the data it learned from. This chapter is about these two foundations of space AI: **data** and **computing hardware**.

The numbers are large. NASA's Earth science archive held 178.7 PB at the end of September 2025 and grew by about 160 TB a day. Yet in 2020 ESA's Φsat-1 screened its images with a deep neural network running on a vision chip of about 1 W. In 2025 a satellite carrying a data-centre graphics processor (GPU), of a kind that draws several hundred watts, trained a small language model in orbit. The hero image above shows HPE's Spaceborne Computer-2 on the International Space Station (ISS). Launched in 2021, it followed a first HPE unit of 2017 and added a GPU to commercial servers in orbit.

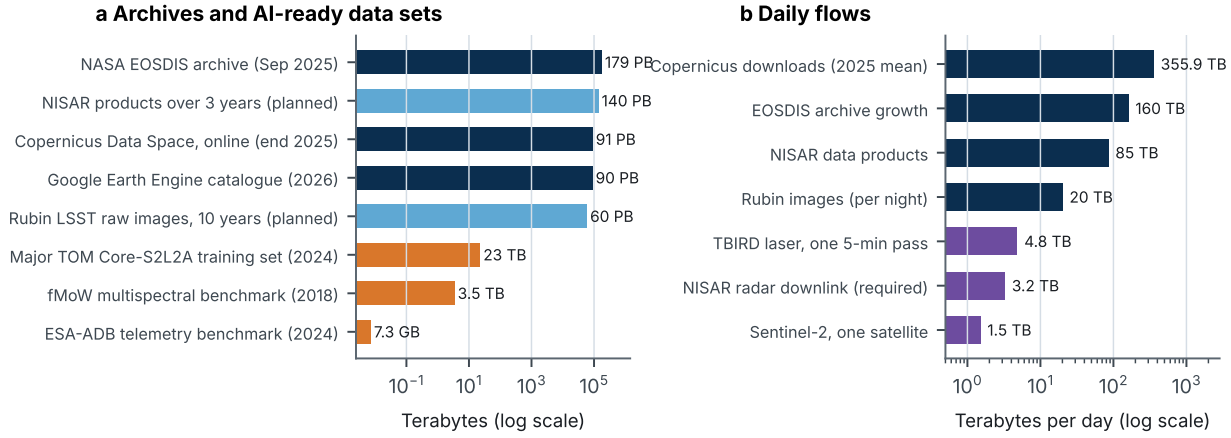
Chapters 43 and 44 explain the learning methods, and Chapters 46–53 apply them and ask how far to trust them. Radiation, flight computers and radio links are introduced in Chapters 2, 6 and 7.

## 45.1 Data for Space AI

The key idea of this section is simple: in space AI, data are plentiful but labels are scarce. Archives hold petabytes, yet a carefully labelled training set is often a few gigabytes.

### 45.1.1 How Much Data?

Archives dwarf the data sets that models are trained on (Figure 45.1). NASA's Earth Observing System Data and Information System (EOSDIS) is the largest, and 116.2 PB of it already sits in commercial cloud storage. The **Copernicus Data Space Ecosystem**, the European Union's archive, held about 91 PB online at the end of 2025, and its users downloaded 129.9 PB directly that year. **NISAR**, the NASA–ISRO radar satellite launched in July 2025, is expected to produce close to 85 TB of products a day and up to about 140 PB in three years. The **Vera C. Rubin Observatory** in Chile, which began its ten-year survey in 2026, records about 20 TB a night. It issues about 10 million alerts a night, each within about 60 seconds.




Sources: NASA Earthdata metrics (FY2025); Copernicus Data Space Ecosystem annual report (2025 data); Google Earth Engine; NASA/JPL NISAR; Rubin Observatory; Major TOM (ESA Phi-lab); fMoW; ESA-ADB (Kotowski et al. 2024); MIT Lincoln Laboratory TBIRD; Airbus (Sentinel-2). In (b) blue bars are ground-side flows, violet bars are what a single spacecraft sends down.

**FIGURE 45.1** Data volumes in the space ecosystem. (a) Archives hold tens to hundreds of petabytes; AI training and benchmark sets are thousands to millions of times smaller. (b) Users download and archives ingest (blue) far more than one spacecraft sends down (violet).

Telemetry — the housekeeping measurements a spacecraft sends about its own health — is smaller but richer in structure. ESA's anomaly-detection benchmark of 2024 holds 1.55 billion data points from two missions, yet only 7.32 GB compressed. Most telemetry has never been labelled.

### 45.1.2 Kinds of Data

Each form of space data needs different models and has different traps (Table 45.1). Optical images resemble photographs, so everyday computer-vision methods transfer well. Radar images do not: they are speckled, complex-valued and taken from the side (Chapter 15). Telemetry, light curves and solar data are time series. Text — anomaly reports, logs and procedures — is the newest data type for space AI (Chapter 49).

**TABLE 45.1** Main types of space data for AI.

| Data type | Examples | Structure | Typical AI tasks | Main traps |
|---|---|---|---|---|
| Optical multispectral | Landsat, Sentinel-2, Planet | 3–13 bands, 0.3–60 m pixels | Land cover, objects, change | Clouds, shadows, haze, sensor differences |
| Hyperspectral | EnMAP, PRISMA, Tanager-1 | 200+ bands per pixel | Minerals, methane, crops | Noisy bands, huge volumes |
| Synthetic aperture radar | Sentinel-1, NISAR, ICEYE | Complex amplitude and phase | Floods, ships, ice, ground motion | Speckle, side-looking geometry |
| Telemetry | Housekeeping channels | Multichannel time series | Anomaly detection, forecasting | Few labelled faults, mode changes |
| Spectra and light curves | Space telescopes, Rubin | 1-D signals over wavelength or time | Classification, transient alerts | Uneven sampling, rare classes |
| Text and logs | Anomaly reports, procedures | Free text | Search, summarising, assistants | Jargon, confidential content |

### 45.1.3 Labels: Where the Truth Comes From

A **label** is the answer a model should learn: “this pixel is water”, “this telemetry interval is an anomaly”. Labels are the most expensive part of most projects, and there are four main ways to obtain them.

**Expert labels** are drawn by trained analysts. They are the most reliable but slow and costly. The So2Sat LCZ42 data set took 15 experts six months, and they judged 85 % of the labels correct. Even experts disagree, so a good project labels a sample twice and measures the disagreement.

**Crowdsourced labels** come from many volunteers or paid workers. They are cheaper but less reliable, so several people label each item and their answers are combined by vote.

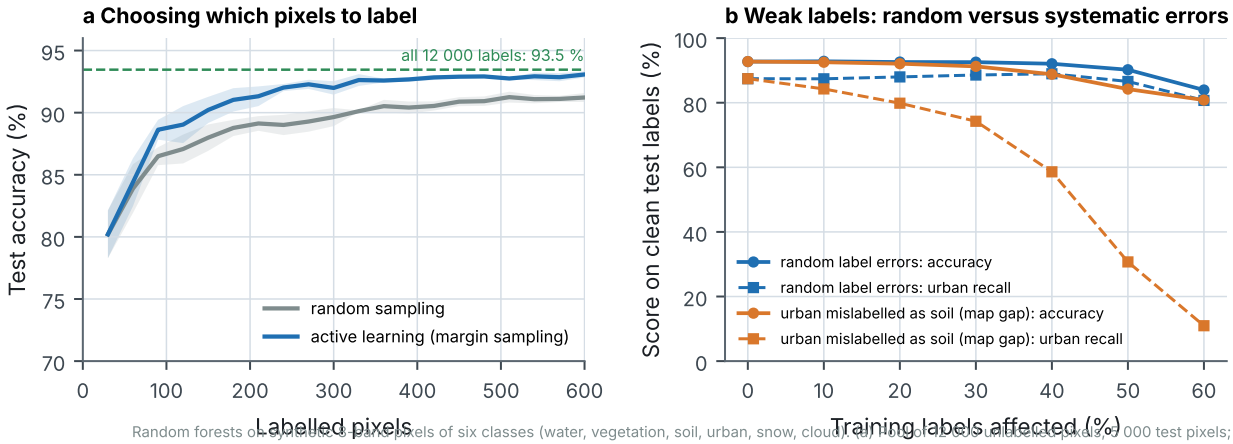
**Weak labels** come from existing sources that were not made for training, such as OpenStreetMap buildings and roads or national land-cover maps. They are cheap and cover huge areas, but they contain errors. Some errors are random. Others are **systematic**: a map may omit new buildings, so the model learns that buildings are bare soil.

**Active learning** lets the model choose which samples experts should label next. The simplest rule, **uncertainty sampling**, labels the samples the current model is least sure about. One measure is the **margin** between the two most likely classes,

$$m(x) = p_{(1)}(x) - p_{(2)}(x),$$

where $p_{(1)}$ and $p_{(2)}$ are the largest and second-largest predicted probabilities. A small margin means the sample lies near a decision boundary, where a new label teaches the most. Another measure is the **entropy** $H(x) = -\sum_k p_k(x) \ln p_k(x)$.

In Figure 45.2 a random forest classifies synthetic eight-band pixels. In panel (a), margin sampling with 210 labels reaches 91.3 % accuracy, while random sampling reaches only 91.2 % with 600 labels. Active learning needed about a third as many labels. In panel (b), random label errors barely matter: the forest still scores 90.2 % with half of all labels wrong, because the errors are spread over five other classes. A systematic map gap is far worse. When 40 % of urban pixels are labelled as soil, urban recall — the share of urban pixels found — falls from 87.4 % to 58.6 %. At 50 % it falls to 30.7 %.




Random forests on synthetic 8-band pixels of six classes (water, vegetation, soil, urban, snow, cloud). (a) Pool of 12 000 unlabelled pixels, 4 000 test pixels; start with 5 labels per class, add 30 per round; margin sampling labels the pixels whose two most likely classes are closest; mean and +/-1 s.d. of 4 runs. (b) 4 000 training pixels; 'random' replaces a label by another class at random; 'map gap' relabels urban pixels as soil, as when buildings are missing from a reference map; mean of 3 runs.

**FIGURE 45.2** Labelling strategies, computed with random forests on synthetic eight-band pixels of six classes. (a) Margin sampling (blue) reaches 91.3 % with 210 labels; random sampling (grey) is still below that with 600. (b) Random label errors (blue) hurt little; turning urban labels into soil (orange), as when a map misses buildings, stops the model finding urban areas.

The lesson is clear. Random noise can be averaged away; systematic omissions cannot. Weak labels should be checked against a small expert-labelled sample before they are trusted.

### 45.1.4 Analysis-Ready, AI-Ready and FAIR Data

**Analysis-ready data (ARD)** are calibrated, geolocated, corrected to surface reflectance and cloud-masked, so that users can analyse them directly (Chapter 13). **AI-ready data** go further. They are cut into consistent patches or time windows, paired with labels, split into training, validation and test sets, and documented. ESA's **Major TOM** collection (2024) is an example. It offers 2 245 886 Sentinel-2 patches of 1 068 × 1 068 pixels covering more than half of Earth's surface; the L2A part alone is about 23 TB.

Good data sets follow the **FAIR principles** of 2016. Data should be **Findable**, with persistent identifiers and searchable metadata, and **Accessible** through open protocols. They should be **Interoperable**, in common formats and vocabularies, and **Reusable**, with clear licences and records of their origin. In Earth observation, the SpatioTemporal Asset Catalog (STAC) and Cloud-Optimized GeoTIFF (COG) formats put these principles into practice.

**Licensing** matters as much as format. Copernicus and Landsat data are free and open for any use. Many benchmark sets allow only non-commercial use; CloudSEN12, for example, carries a CC-BY-NC 4.0 licence. A model may inherit such limits, so the licence of every training set must be recorded before a model is sold or flown.

## 45.2 Benchmarks, Competitions and Synthetic Data

A **benchmark** is a fixed data set with an agreed task and score, so that different methods can be compared fairly. Benchmarks drive progress, but they also mislead when models are tuned to them rather than to the real problem.

### 45.2.1 Benchmark Data Sets

Widely used benchmarks are listed in Table 45.2. Most Earth-observation sets come from the free Sentinel-1 and -2 data or from commercial imagery donated for research. SPEED+ serves spacecraft pose estimation, and the NASA and ESA telemetry sets serve anomaly detection.

**TABLE 45.2** Benchmark data sets for space AI.

| Benchmark | Year | Data | Size | Task |
|---|---|---|---|---|
| EuroSAT | 2019 | Sentinel-2, 13 bands | 27 000 patches | 10 land-cover classes |
| BigEarthNet (v2.0 2024) | 2019 | Sentinel-2 (+ Sentinel-1 in v2.0) | 590 326 patches (v2.0: 549 488 pairs) | Multi-label land cover |
| SEN12MS | 2019 | Sentinel-1, Sentinel-2, MODIS | 180 662 triplets | Land cover |
| So2Sat LCZ42 | 2020 | Sentinel-1 and -2 | 400 673 patch pairs | 17 local climate zones |
| SpaceNet 1–8 | 2016–22 | Commercial optical (0.3–4 m) and radar | SpaceNet 8: 850 km² | Buildings, roads, floods |
| xView | 2018 | WorldView-3, 0.3 m | > 1 million objects, 1 400 km² | 60 object classes |
| fMoW | 2018 | High-resolution multispectral | 1 047 691 images | 63 categories of land use |
| xBD | 2019 | Before and after disasters | 850 736 buildings, 19 disasters | Damage level |
| CloudSEN12 | 2022 | Sentinel-2 and -1 | 49 400 patches | Cloud and shadow masks |
| SPEED+ | 2021 | Rendered and laboratory images | 59 960 + 9 531 images | Spacecraft pose |
| SMAP/MSL (Telemanom) | 2018 | Spacecraft telemetry | 82 channels, 105 anomalies | Anomaly detection |
| ESA-ADB | 2024 | Telemetry of two missions | 1.55 billion points, 844 events | Anomaly detection |

Three traps recur. **Leakage**: neighbouring patches in training and test sets look alike, so random splits overstate accuracy (Chapter 43); BigEarthNet v2.0 therefore splits by geography. **Saturation**: once top scores pass 98 %, a benchmark no longer separates good methods from great ones. **Label quality**: errors in a benchmark's own labels cap the score any honest model can reach.

### 45.2.2 Competitions and Research Sprints

ESA's **Kelvins** platform has run open competitions on real mission problems since 2016. Topics have included Mars Express power, PROBA-V super-resolution, collision avoidance and spacecraft pose. In the **OPS-SAT case** (July–October 2022), teams built a small, quantisation-aware model from few labelled images to run on board OPS-SAT.

NASA's **Frontier Development Lab (FDL)** was created in 2016 by NASA, the SETI Institute and Trillium Technologies. It brings machine-learning researchers and space scientists together for research sprints of eight to ten weeks; a European version has run with ESA in Oxford. Sprints produce prototypes, and turning one into an operational system takes years.

### 45.2.3 Synthetic Data and the Sim-to-Real Gap

Some data cannot be collected in advance: nobody has thousands of labelled photographs of a particular satellite tumbling in orbit. Engineers therefore **render** synthetic images with computer graphics, or generate synthetic telemetry from a **digital twin** — a simulation detailed enough to stand in for the spacecraft.

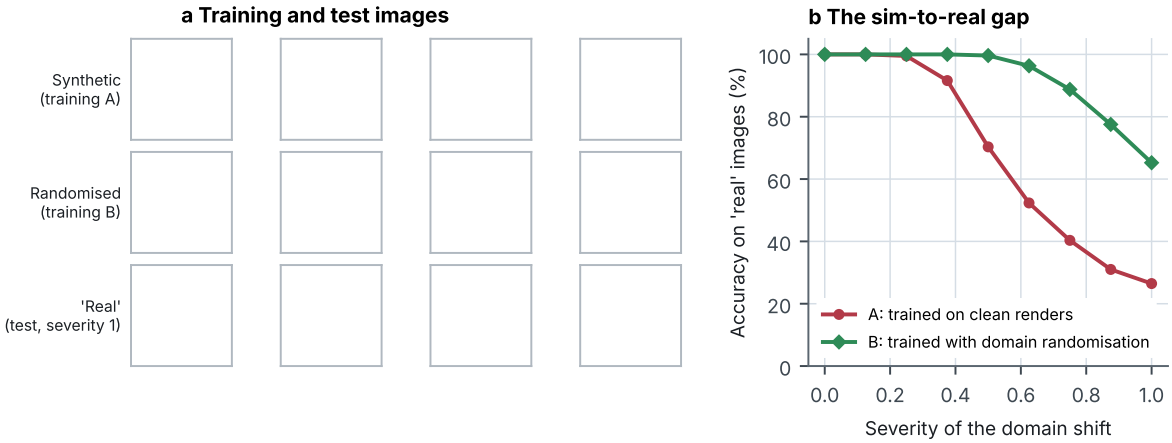
The danger is the **sim-to-real gap**: a model that excels on synthetic data fails on real data, because rendering never captures every detail of lighting, materials, optics and noise. SPEED+ measures this gap. A pose network trained on its rendered images scored 0.16 on synthetic validation images (lower is better). It scored 1.21 on laboratory images lit by a lightbox and 1.73 under a lamp that mimics the Sun — eight to eleven times worse.

**Domain randomisation** is the main defence. Instead of rendering the most realistic images possible, the engineer randomises everything that should not matter — lighting, background, texture, blur, noise, gain. The model then learns the one thing that stays constant, the object itself. Formally, training minimises the expected loss over the randomisation parameters $\xi$:

$$\min_{\theta} \; \mathbb{E}_{\xi \sim p(\xi)} \Big[ L\big( f_\theta(\mathrm{render}(s, \xi)),\, y(s) \big) \Big],$$

where $s$ is the scene (here, the spacecraft's pose), $y(s)$ its label and $f_\theta$ the network. With luck, the real world is just one more random draw.

A toy renderer shows the effect (Figure 45.3). A numpy network trained on clean renders identifies a spacecraft's orientation perfectly on similar images. On "real" images with a different Sun angle, an Earth background, blur and noise, its accuracy falls to 70.3 % at half severity and 26.5 % at full severity. The same network trained on randomised renders keeps 99.6 % and 65.2 %. Randomisation narrows the gap but does not close it. The winners of ESA's 2021 pose challenge also used **domain adaptation**, which trains the network so that its internal features look the same for synthetic and real images.




Toy renderer: 20 × 20-pixel images of a T-shaped spacecraft in 8 orientation classes (45 deg apart, +/-11 deg jitter). Numpy network 400-96-8 trained on 8 000 images per set. A: Sun fixed, black sky, no noise. B: random Sun angle, random gradient and blob backgrounds, blur, gain and noise. 'Real' test images (3 000 per point): Sun angle spread, textured Earth-limb background, blur, gain error and noise all grow with severity (0 = identical to A).

**FIGURE 45.3** The sim-to-real gap, computed with a numpy network on 20 × 20-pixel renders of a toy spacecraft in 8 orientations. (a) Clean training images (A), randomised training images (B) and “real” test images at full severity. (b) Accuracy on 3 000 “real” images as the shift grows; chance is 12.5 %.

## 45.3 MLOps: From Archive to Operations

**MLOps** (machine-learning operations) is the set of practices that take a model into reliable service and keep it there. The key idea is that a model is never finished. It is one stage in a loop of data, training, testing, deployment and monitoring that runs for the life of the mission.

### 45.3.1 The End-to-End Pipeline

The loop for an Earth-observation mission is drawn in Figure 45.4. Data flow from the sensor through the ground stations into processing and the archive. Labels are attached, and a frozen **snapshot** of training data is taken. Models are trained, registered and tested against validation gates. A model that passes is deployed on the ground or, after further checks, uplinked. Monitoring watches for drift and triggers retraining. The data shrink at each step, from terabytes a day to a model of megabytes.

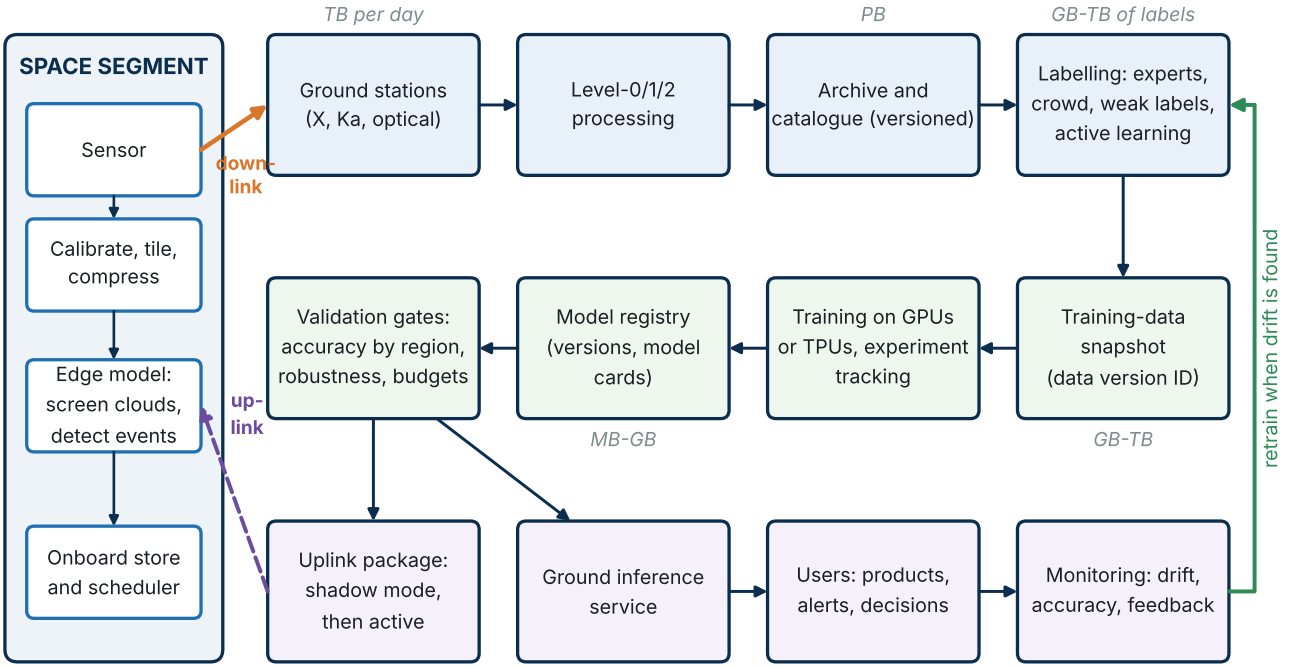



**FIGURE 45.4** An end-to-end MLOps pipeline from satellite to model to user. Volumes shrink from terabytes a day at the downlink to megabytes for the model. Validation gates come before deployment on the ground or in orbit; monitoring triggers retraining when the data drift.

## 45.3.2 Versions, Registries and Validation Gates

Four habits make a pipeline trustworthy.

1. **Version the data.** Each training snapshot gets a unique identifier, often a hash of its contents, so that any model can be traced to the exact data it saw.
2. **Track experiments.** Every training run records its code version, settings, random seeds and results, so that it can be reproduced.
3. **Register models.** A **model registry** stores each released model with its version, data identifier and test results. It adds a **model card**: a short note on what the model is for, how it was tested and where it fails.
4. **Gate releases.** A **validation gate** is a test the model must pass before release. It checks accuracy by region, season and surface type, and robustness to noise and calibration changes. It also checks memory, time and power on the flight processor, and **bit-exact** agreement with the ground reference.

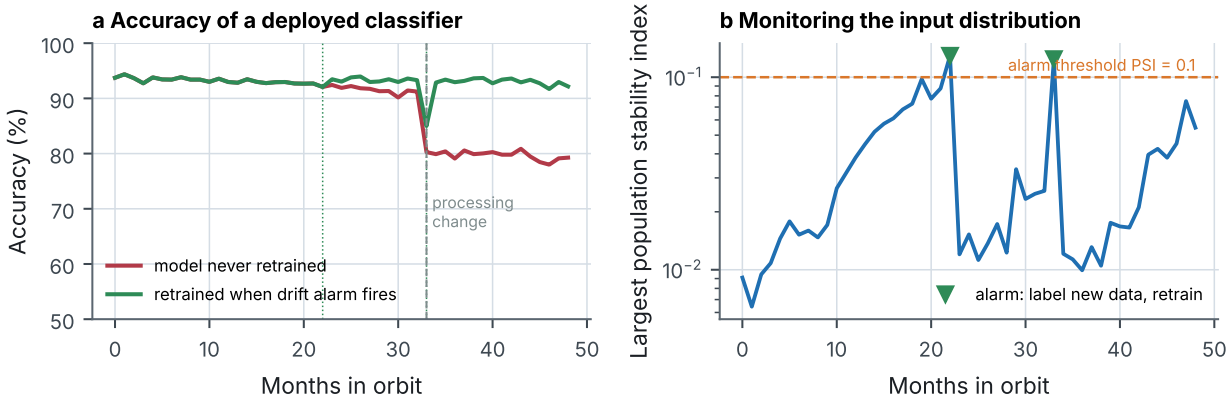
## 45.3.3 Monitoring for Drift

After launch the world changes. Detectors age, calibration is updated, and seasons bring snow and smoke. This is **drift**: the data no longer match the data the model learned from. Labels arrive slowly in operations, so drift is usually detected first from the inputs. A common statistic is the **population stability index (PSI)**. For each bin $i$ of a variable, it compares the share $p_i$ of training data with the share $q_i$ of current data.

$$\mathrm{PSI} = \sum_i (q_i - p_i)\,\ln\frac{q_i}{p_i}.$$

Values below 0.1 are usually taken as stable, and values above 0.25 signal a large shift.

In Figure 45.5 a deployed classifier watches a detector that loses 0.1–0.6 % of its gain per month. At month 33, a ground-processing change raises two short-wave infrared bands by 20 %. Without retraining, accuracy slides from 93.7 % to about 90–91 % by month 32, then drops to 80.3 %. With a PSI alarm at 0.1, the monitor fires at months 22 and 33. Each time the model is retrained on 2 000 newly labelled pixels and recovers to about 92–93 %.




Random forest (80 trees) trained on 4 000 synthetic 8-band pixels at launch; each month 3 000 new pixels are classified. The detector loses 0.1-0.6 % of its gain per month (fastest in the blue band); at month 33 a processing change raises the two short-wave infrared bands by 20 %. PSI: population stability index of each band against the training data (10 quantile bins); after an alarm the model is retrained on 2 000 newly labelled pixels, which become the new reference.

**FIGURE 45.5** Drift monitoring, computed with a random forest on synthetic pixels over 48 months. (a) A model never retrained (red) loses accuracy as the detector ages and after a processing change; one retrained at each alarm (green) recovers. (b) The largest population stability index across the bands, with the threshold and alarms.

### 45.3.4 Cloud Platforms and Ground Compute

Most training and much inference happens on the ground, close to the archives (Table 45.3). Users send their code to the data instead of downloading petabytes. Chapter 13 introduces these platforms; their status in October 2026 is summarised here.

**TABLE 45.3** Cloud platforms for Earth-observation AI.

| Platform | Operator | What it offers | Status, October 2026 |
|---|---|---|---|
| Google Earth Engine | Google | Catalogue of 80–90 PB and more than 1 000 data sets; analysis on Google's servers | Free for research; commercial use through Google Cloud |
| Copernicus Data Space Ecosystem | European Commission, ESA | About 91 PB online; all Sentinel data; processing services | Opened January 2023; replaced the old Open Access Hub (closed November 2023); 542 622 users at end of 2025 |
| NASA Earthdata Cloud | NASA, on Amazon Web Services | 116.2 PB of the 178.7 PB EOSDIS archive | Operational; serves NISAR products |
| AWS Open Data | Amazon | Free hosting of public data sets, including Landsat, Sentinel-2 and SpaceNet | Operational |
| Microsoft Planetary Computer | Microsoft | Open-data catalogue with a STAC search interface; paid "Pro" service for private data | Free computing hub retired June 2024; catalogue continues |

Training large models also needs clusters of GPUs or tensor processing units (TPUs), in the cloud or in agency supercomputers (Figure 45.6). A foundation model may occupy hundreds of accelerators for weeks, so compute time is budgeted like ground-station time.

**FIGURE 45.6** The Discover supercomputer at the NASA Center for Climate Simulation, Goddard Space Flight Center. Ground clusters and cloud services do most of the training for space AI. Image: NASA/GSFC.

## 45.4 Radiation and AI Hardware

The key idea of this section is that a neural network is a large block of numbers held in memory, and radiation changes numbers. Some changes are harmless; a few are catastrophic. Good design finds and protects the few.

### 45.4.1 How Radiation Disturbs Computers

Chapter 2 explains radiation and its effects; here is a short reminder. **Total ionising dose (TID)** slowly degrades transistors as charge builds up in their oxides; it is measured in krad(Si). **Displacement damage** knocks atoms out of the crystal lattice and mainly harms detectors and solar cells. **Single-event effects** are caused by one energetic particle. A **single-event upset (SEU)** flips a stored bit. A **single-event functional interrupt (SEFI)** hangs a processor until it is reset. A **single-event latch-up (SEL)** creates a short circuit that can destroy the chip unless the power is cut quickly.

The upset rate of a memory comes from folding its measured **cross-section** $\sigma(L)$, the sensitive area per bit, with the flux of particles at each linear energy transfer $L$.

$$R = \int \sigma(L)\,\frac{d\Phi}{dL}\,dL \quad \text{upsets per bit per second.}$$

Measured rates give a feel for the result. Static memory (SRAM) on two Chinese internet satellites at 1 050–1 425 km recorded about $7.6 \times 10^{-7}$ upsets per bit per day, mostly over the South Atlantic Anomaly. At that rate, a model of 3.4 million 8-bit weights (27.2 Mbit) would collect about 21 upsets a day; stored as 32-bit floating-point numbers, about 83.

### 45.4.2 Why Neural Networks Are Vulnerable

A network's **weights** are its learned parameters, stored in memory for the whole mission. Its **activations** are intermediate results, recomputed for every input. An upset in an activation spoils one result. An upset in a weight spoils every result until the memory is repaired.

Not all bits are equal. A 32-bit floating-point number has 1 sign bit, 8 exponent bits and 23 fraction bits. Flipping a low fraction bit changes a weight slightly. Flipping the highest exponent bit can multiply it by about $2^{128}$, or $10^{38}$, and one such weight turns the network's output into garbage. One flip changes an 8-bit integer weight by at most 128 quantisation steps, so its errors are bounded.

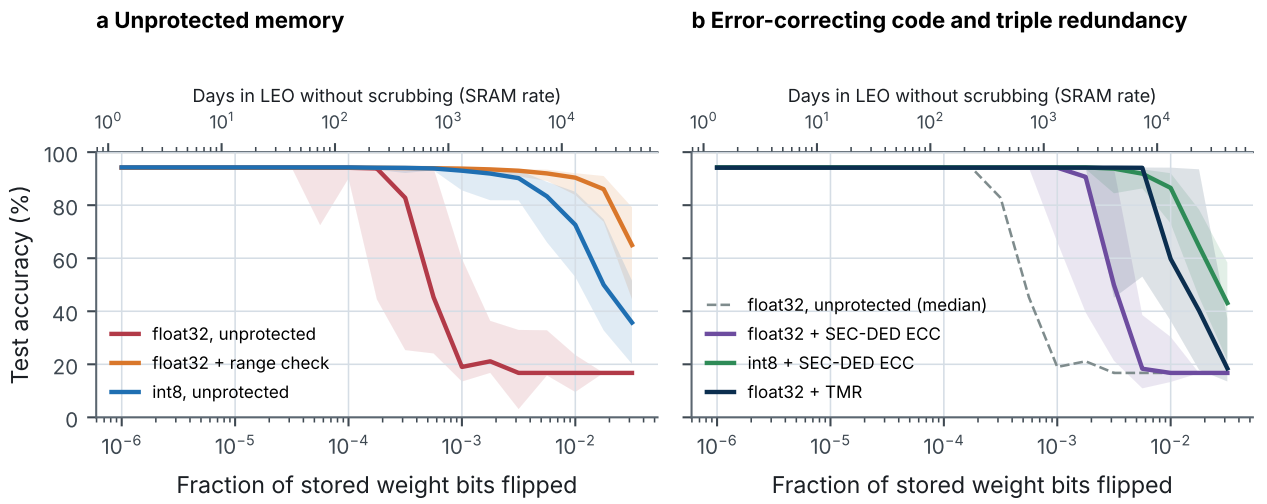
A flip in control logic, program code or a register can hang the processor or corrupt a whole layer. Errors that go unnoticed are called **silent data corruption (SDC)**. They are more dangerous than crashes, which at least announce themselves.

### 45.4.3 Mitigation

Engineers combine several defences, from hardware to algorithms.

- **Triple modular redundancy (TMR)** keeps three copies and takes a majority vote. If each copy works with probability $R$, the voted system works with probability $R_{\mathrm{TMR}} = 3R^2 - 2R^3$, better than $R$ whenever $R > 0.5$.
- **Error-correcting codes (ECC)** add check bits to each memory word. The common **SEC-DED** Hamming code adds 8 bits to each 64-bit word (12.5 % overhead). It corrects any single-bit error and detects any double error.
- **Scrubbing** reads and rewrites memory regularly, correcting single errors before a second one lands in the same word. With $n$ bits per word and upset probability $p$ per bit between scrubs, the chance of two errors in a word is about $\binom{n}{2} p^2$. Halving the scrub interval quarters it.
- **Watchdogs** restart a processor that stops responding, and current limiters cut power within microseconds of a latch-up.
- **Selective hardening** protects only the most sensitive parts: the exponent bits of weights, the first and last layers, or the control code. A cheap software version is a **range check** that clamps any weight larger than its known maximum.
- **Algorithm-based fault tolerance (ABFT)** checks the arithmetic. For a matrix product $C = AB$, the column sums must satisfy $\mathbf{1}^{\top} C = (\mathbf{1}^{\top} A)\, B$; checking this is cheap.
- **Redundant inference** runs the model twice, or on two processors, and compares the answers.

The fault-injection experiment in Figure 45.7 tests these defences on a small network of 4 992 weights. Random bit flips are injected into its stored weights. Unprotected float32 weights are the weakest. At a bit-error rate of $10^{-4}$, the median accuracy is still 94.1 %, but one trial in ten falls to 90.0 % or less. At $10^{-3}$ the median collapses to 19.0 %, because almost every trial hits a high exponent bit. A simple range check keeps 93.8 % at $10^{-3}$, and int8 storage keeps 93.0 % with no protection at all. SEC-DED ECC without scrubbing keeps the float32 median at full accuracy up to $10^{-3}$, but fails at $10^{-2}$, when many words collect two errors. Int8 with ECC keeps a median of 86.5 % at $10^{-2}$, and TMR keeps nine trials in ten at full accuracy up to $10^{-3}$. On the top axis, $10^{-4}$ equals about 130 days in LEO without scrubbing.




Numpy network 8-64-64-6 (4 992 weights) trained on 20 000 synthetic 8-band pixels of six classes; 2 000 test pixels; 40 random trials per point (line: median; band: 10th-90th percentile). Flips land uniformly on all stored weight bits (biases protected). Range check: weights non-finite or > 2x the largest original weight are set to zero. ECC: SEC-DED on 64-bit words, multi-bit words left uncorrected. Top axis: 7.6e-7 upsets per bit per day, measured in SRAM on LEO satellites.

**FIGURE 45.7** Fault injection into a numpy network (8–64–64–6) trained on synthetic pixels; 40 trials per point. (a) Unprotected float32 weights fail once a high exponent bit flips; a range check or int8 storage removes most of the damage. (b) SEC-DED ECC on 64-bit words (no scrubbing) and triple modular redundancy; grey dashes repeat unprotected float32.

The experiment points to a practical recipe. Store weights as integers. Protect them with ECC and scrub often. Add a range check. Keep a protected golden copy of the model to reload when the ECC reports an uncorrectable error.

### 45.4.4 Radiation Testing

No calculation replaces a test. **TID tests** expose parts to a cobalt-60 gamma source (ESCC 22900, MIL-STD-883 Method 1019). **Single-event tests** fire heavy ions or protons from an accelerator at a working device and count upsets, interrupts and latch-ups (ESCC 25100, JESD57). For AI processors, the best test runs the real model during exposure and checks every output, which measures silent data corruption directly.

Several AI processors have been tested this way. Intel's Myriad 2 vision chip passed a 36-hour beam test at CERN in late 2018 and two later tests without modification, which cleared it for Φsat-1. Heavy-ion tests at GANIL in France on NVIDIA Jetson Orin NX and Xavier NX modules found that about 80 % of events were correctable upsets, with no destructive latch-up. The HiMassSEE experiment on the ISS measured the environment that causes such events (Figure 45.8). Tested commercial chips have even flown on Mars (Figure 45.9).

**DATA BOX — RADIATION TESTING OF TPUS FOR PROJECT SUNCATCHER**

Google's Suncatcher study (November 2025) tested Trillium (v6e) TPUs with 67 MeV protons for an 81-satellite cluster at about 650 km.

- Dose behind 10 mm of aluminium: about 150 rad(Si) a year, 750 rad(Si) in five years.
- Memory irregularities began after 2 krad(Si); one chip had no hard failure up to 15 krad(Si).
- Silent data corruption: about one event per 14.4–20 rad, or one bad result in three million at one inference per second.
- Uncorrectable memory errors: about one per 44 rad; functional interrupts about one per 5 krad per chip.

**FIGURE 45.8** The HiMassSEE detector kit in the ISS's US Laboratory in August 2012. It measured how heavy shielding changes the radiation that causes single-event effects. Image: NASA/JSC.

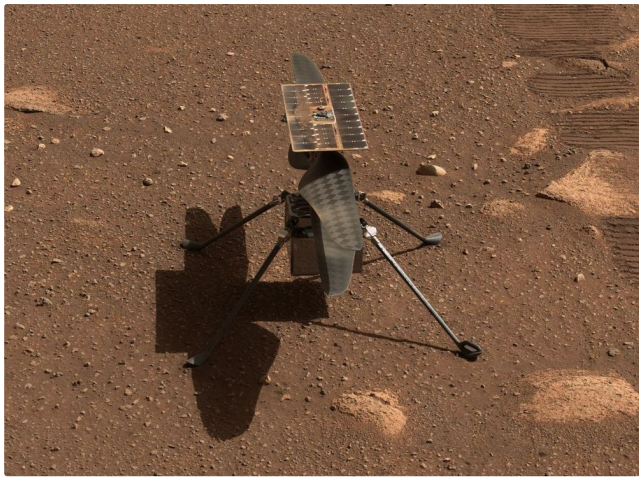


**FIGURE 45.9** Ingenuity on Mars, photographed by Perseverance's Mastcam-Z on 5 April 2021. Its navigation ran on a Snapdragon 801 smartphone processor. Image: NASA/JPL (NASA/JPL-Caltech/ASU).

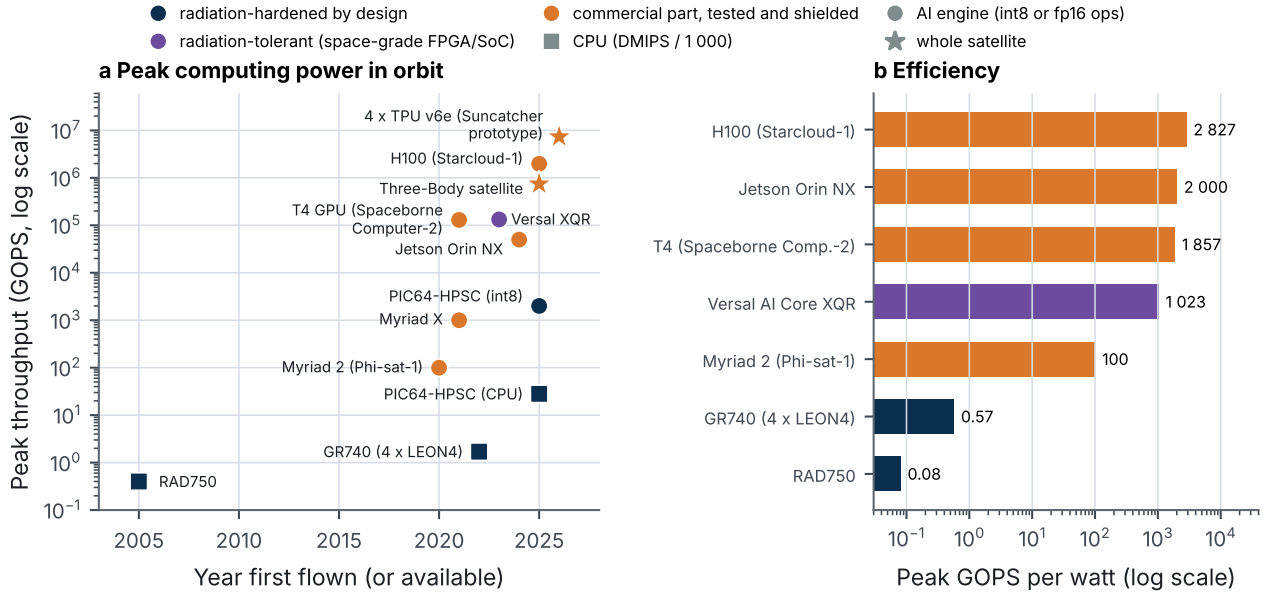
## 45.5 Processors for AI in Orbit

Space processors fall into three families. **Radiation-hardened** processors are designed and built to survive large doses and to be immune to latch-up. They are reliable but lag commercial chips by one to two decades. **Radiation-tolerant** parts, such as space-grade field-programmable gate arrays (FPGAs) and adaptive systems-on-chip, use commercial technology with careful design and testing. **Commercial** processors — vision chips, GPUs and TPUs — are screened, tested, shielded and supervised. Table 45.4 lists examples, and Figure 45.10 plots their peak throughput and efficiency.

**TABLE 45.4** Processors for AI in space (vendor peak figures). DMIPS: millions of Dhrystone instructions per second, a CPU benchmark; TOPS: trillions of int8 operations per second (dense).

| Processor | In space | Peak throughput | Power | Radiation approach | Example use |
|---|---|---|---|---|---|
| RAD750 (BAE Systems) | 2005 | 400 DMIPS | 5 W (CPU) | Hardened, 0.2–1 Mrad | Perseverance, JWST |
| GR740 (Gaisler), 4 × LEON4 | 2022 | 1 700 DMIPS | < 3 W (core) | Hardened, 300 krad | Rashid lunar rover |
| PIC64-HPSC (Microchip) | 2025 (samples) | 28 130 DMIPS; 2 TOPS | Not published | Hardened, 100 krad | In test; not flown |
| Versal XQRVC1902 (AMD) | 2023 (shipping) | 133 TOPS | Up to ~130 W | Tolerant; scrubbing | Payload processing |
| Myriad 2 (Intel) | 2020 | > 100 GFLOPS | About 1 W | Commercial; tested | Φsat-1 |
| Myriad X (Intel) | 2021 | 1 TOPS | As Myriad 2 | Commercial | SpaceCloud, CogniSAT-6 |
| Snapdragon 801 (Qualcomm) | 2021 | 4 × 2.26 GHz | Not published | Commercial; tested | Ingenuity |
| T4 GPU (NVIDIA) | 2021 | 130 TOPS | 70 W | Commercial; inside ISS | Spaceborne Computer-2 |

| Processor | In space | Peak throughput | Power | Radiation approach | Example use |
|---|---|---|---|---|---|
| SpaceCloud iX5, iX10 (Unibap) | 2021 (iX5) | iX10: CPU, GPU, Myriad X, FPGA | Tens of watts (iX10) | Commercial parts | D-Orbit ION carriers |
| Jetson Orin NX (NVIDIA) | 2024 | 50 TOPS (100 sparse) | 10–25 W | Commercial; tested | Aethero cubesat |
| H100 GPU (NVIDIA) | 2025 | ~2 000 TOPS | Up to 700 W | Commercial | Starcloud-1 |
| TPU v6e (Google) | 2026 | 1 836 TOPS | Not published | Commercial; tested | Suncatcher (4 chips) |
| Three-Body satellite (China) | 2025 | 744 TOPS | Not published | Not published | 12 satellites |




Vendor peak figures from the processor table: CPUs in millions of Dhrystone instructions per second (DMIPS / 1 000); AI engines in int8 operations (Myriad 2: fp16), dense, without sparsity. H100: SXM version (700 W); the variant flown was not disclosed. Versal power: about 130 W when fully used (commercial part). Parts without published power are omitted from (b). Real applications reach 10-50 % of peak.

**FIGURE 45.10** Processors for AI in orbit, from Table 45.4. (a) Peak throughput rose about seven orders of magnitude from the RAD750 to the H100. (b) Peak operations per watt: the best commercial AI engines reach 2 000–2 800 GOPS/W, the RAD750 0.08 — though CPU instructions and int8 operations are not directly comparable.

The comparison needs care. A CPU instruction does more varied work than an 8-bit multiply–add, vendors quote peaks, and real networks reach perhaps 10–50 % of them. Even so, the gap is real. The RAD750 delivers about 0.08 GOPS per watt; Myriad 2 about 100; a Jetson Orin NX about 2 000; an H100 about 2 800.

NASA's **High-Performance Spaceflight Computing (HPSC)** project aims to narrow the gap for hardened parts. Microchip's PIC64-HPSC, announced in July 2024, has eight 64-bit RISC-V cores with vector units. NASA's figures give it about 1 343 times the speed of the GR740 on its SpaceBench benchmark at 1 GHz. As of mid-2026, its chips were in radiation, thermal and shock testing; none had flown.

**FPGAs** are chips whose logic can be rewired after manufacture, which suits high-rate payload processing. AMD's space-grade Versal AI Core adds 400 small AI engines to an FPGA. It completed Class B qualification in December 2022 and has shipped since early 2023. A lidless version aiming at the stricter Class Y is sampling in 2026, with flight units planned for 2027.

Commercial chips dominate new demonstrations because they are thousands of times more efficient. Planet's Pelican-4, for example, used a Jetson Orin module to detect aircraft in its own images on 25 March 2026. Unibap's SpaceCloud computers have hosted customers' applications on D-Orbit's ION carriers since June 2021. The price is risk. The Ingenuity helicopter flew 72 times on Mars between April 2021 and January 2024 on a Snapdragon 801 phone processor (Figure 45.9). Its team accepted that the chip was less robust than flight parts, and two small microcontrollers ran the time-critical flight control.

> **ΦSAT-1: DEEP LEARNING SCREENS IMAGES IN ORBIT**
>
> Φsat-1 was an ESA cubesat. It was launched on 2/3 September 2020 on Vega flight VV16 into a sun-synchronous orbit at about 530 km. Its HyperScout-2 hyperspectral camera fed an Intel Myriad 2 vision chip of about 1 W. On it ran CloudScout, a small convolutional network developed with the University of Pisa. It kept images less than 70 % cloudy for downlink. Because no real images yet existed, it was trained on Sentinel-2 images processed to imitate the new camera. On test images it was 92 % accurate with 1 % false positives, taking about a third of a second per image at about 2 W. ESA estimated that discarding cloudy images could save about 30 % of the downlink. Φsat-1 showed that a commercial AI chip could survive and work in orbit.

## 45.6 Making Models Fit: Compression

The key idea of compression is that most networks are far larger and more precise than they need to be. Removing the excess saves memory, uplink time, energy and radiation exposure, often with almost no loss of accuracy.

### 45.6.1 Quantisation

**Quantisation** stores weights and activations as integers with fewer bits. In symmetric quantisation with $b$ bits, a scale factor

$$s = \frac{\max |w|}{2^{\,b-1} - 1}$$

maps each weight $w$ to the integer $q = \mathrm{round}(w/s)$, clipped to $\pm(2^{b-1} - 1)$. The network then uses $\hat{w} = s\,q$. The rounding error is roughly uniform between $\pm s/2$, so its variance is $s^2/12$. Each extra bit halves $s$ and improves the signal-to-noise ratio by about 6 dB. One scale per tensor is simplest; one scale per output neuron or channel fits uneven weights better.

**Post-training quantisation (PTQ)** converts a trained network directly. **Quantisation-aware training (QAT)** fine-tunes the network with quantised weights in the forward pass, so that it learns to tolerate rounding. Rounding has no useful gradient. QAT therefore uses the **straight-through estimator**, which passes the gradient through the rounding step as if it were absent.

These methods were applied to the 5 126-parameter network of Figure 45.7 (Figure 45.11). At 8 and 6 bits, PTQ loses nothing measurable: 94.2–94.3 % in every case. At 4 bits it loses about one point (93.2 % per tensor, 93.7 % per neuron). At 2 bits — weights of −1, 0 or +1 times a scale — PTQ collapses to 16.7 % per tensor and 51.5 % per neuron, while QAT still reaches 93.2 %. Model size falls from 20.0 kB to 5.9 kB at 8 bits and 2.3 kB at 2 bits; the 32-bit biases and scales set a floor.

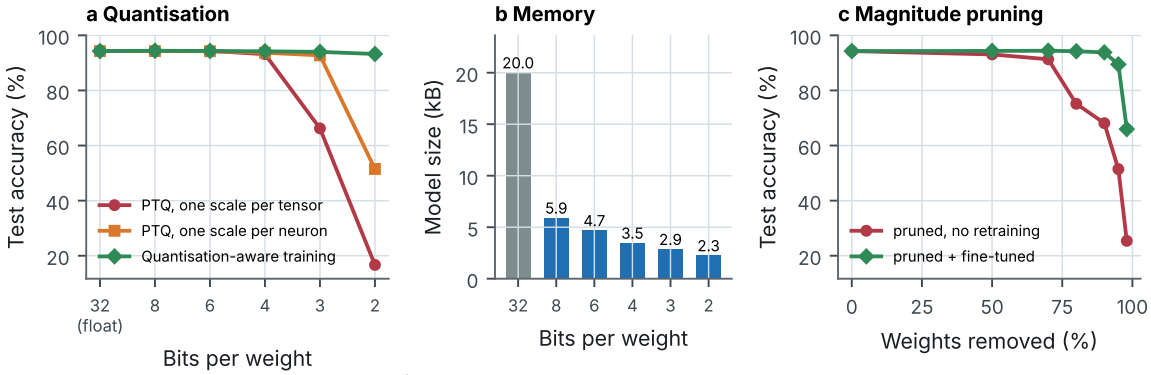
**EXAMPLE: SIZING AN ONBOARD CLOUD SCREEN**

A cloud screen the size of MobileNetV2 has 3.4 million parameters and needs about 0.30 billion multiply–accumulates (MACs) per 224 × 224-pixel tile. The processor draws 5 W with a peak of 1 TOPS in int8; assume 30 % of peak.

1. **Model size.** In float32: $3.4 \times 10^6 \times 4$ bytes = 13.6 MB. In int8: 3.4 MB, four times smaller.
2. **Uplink time.** At 256 kbit/s: $3.4 \times 8 \times 10^6 / 256\,000 \approx 106$ s for int8 (one pass), 425 s for float32.
3. **Work per tile.** Two operations per MAC give $0.6 \times 10^9$ operations; at $0.3 \times 10^{12}$ per second, a tile takes 2.0 ms.
4. **Energy per inference.** E = P t = 5 W × 2.0 ms = 10 mJ per tile.
5. **Per image.** A 10 000 × 10 000-pixel image holds about 2 000 tiles: about 4 s and 20 J. Screening 300 images a day uses about 1.7 Wh. (CloudScout on Φsat-1 took about 0.3 s at about 2 W, roughly 0.6 J per image.)
6. **Radiation.** At $7.6 \times 10^{-7}$ upsets per bit per day, the 27.2 Mbit of int8 weights collect about 21 upsets a day. With SEC-DED ECC and daily scrubbing, a double error in one word occurs only about once in four years.

## 45.6.2 Pruning, Distillation and Efficient Architectures

**Pruning** removes weights that matter little, usually the smallest in magnitude. Panel (c) of Figure 45.11 shows that removing 80 % of the weights without retraining drops accuracy to 75.2 %. After ten epochs of fine-tuning with the remaining weights, it recovers to 94.2 %. At 95 % pruning the fine-tuned network keeps 89.5 %. Pruning saves memory only if the hardware can skip zeros, either through sparse formats or by removing whole neurons or channels.




Same network and data as the bit-flip figure (5 126 parameters). Weights quantised symmetrically, biases and activations kept in 32-bit floating point. QAT: 12 epochs of fine-tuning with quantised weights in the forward pass (straight-through estimator). Pruning: global magnitude threshold, then 10 epochs of fine-tuning with the mask fixed. Sizes include biases and one scale per neuron.

**FIGURE 45.11** Compressing the network of Figure 45.7, tested on 8 000 synthetic pixels. (a) Post-training quantisation (PTQ) is harmless down to 6 bits; below 4 bits only quantisation-aware training (QAT) holds accuracy. (b) Model size. (c) Magnitude pruning works only with fine-tuning.

**Knowledge distillation** trains a small "student" network to imitate a large "teacher". The student learns from the teacher's soft probabilities, which carry more information than hard labels. Its loss has two terms:

$$L = (1-\alpha)\,\mathrm{CE}(y, p_s) + \alpha\, T^2\, \mathrm{KL}\big(p_t^{(T)} \,\|\, p_s^{(T)}\big),$$

Here CE is the cross-entropy with the true labels and KL the Kullback–Leibler divergence between the two networks' outputs. The "temperature" $T$ softens both distributions, and $\alpha$ weights the two terms.

**Efficient architectures** are designed small from the start. Mobile networks replace a standard convolution with a **depthwise separable** one: a per-channel spatial filter followed by a 1 × 1 convolution that mixes channels. For a $k \times k$ kernel and $N$ output channels, this cuts the work by a factor of about $1/N + 1/k^2$ — nearly nine times for 3 × 3 kernels. Chapter 44 describes these architectures.

## 45.7 On Board or on the Ground?

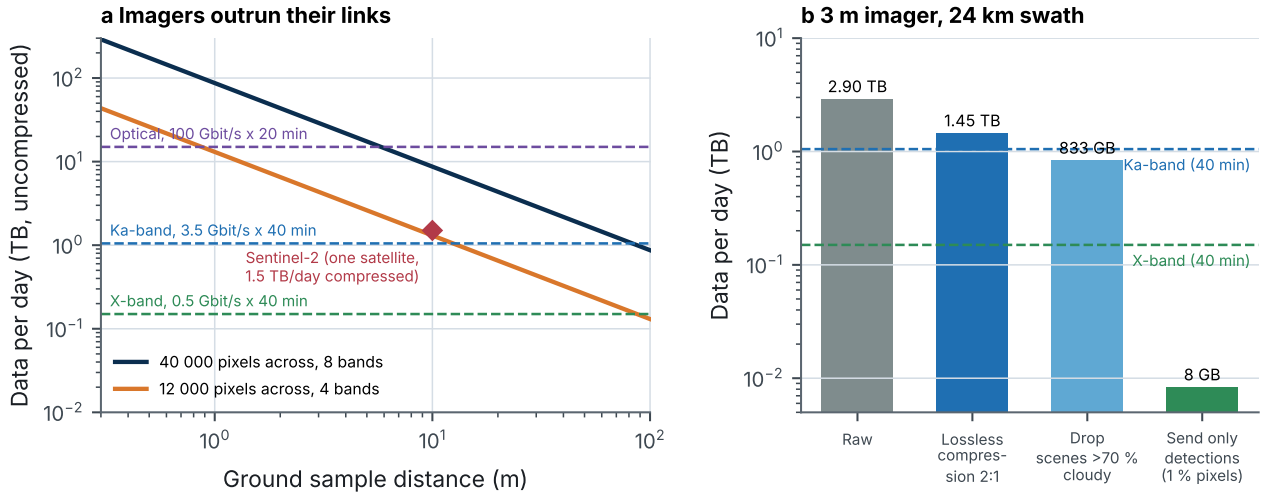
Every mission must decide where its models run. The key trade is between the cost of sending data down and the cost of processing them in orbit.

### 45.7.1 The Downlink Bottleneck

A pushbroom imager with $N$ pixels across its track, ground sample distance $g$ and ground speed $v_g$ scans $v_g/g$ lines per second. With $N_b$ bands of $b$ bits each, its data rate is

$$R = N \frac{v_g}{g} N_b b.$$

At 1 m resolution, 20 000 pixels across, 4 bands, 12 bits and a ground speed of 7 km/s, that is 6.7 Gbit/s. A link's daily capacity is its rate times the daily contact time with ground stations. The two are compared in Figure 45.12. An X-band link of 0.5 Gbit/s with 40 minutes of contact a day carries 0.15 TB. A Ka-band link of 3.5 Gbit/s carries 1.05 TB, and an optical link of 100 Gbit/s for 20 minutes carries 15 TB in clear weather. Applied to Sentinel-2's bands, the same model gives about 4.4 TB a day. That is consistent with the 1.5 TB a day each satellite sends after onboard compression of about 3:1.




Pushbroom model: data rate = pixels across track x lines per second (ground speed 7.0 km/s / GSD) x bands x 12 bits, imaging 30 % of the time. Link capacity = rate x daily contact time (assumed). Sentinel-2 check: the same model with its 10/20/60 m bands gives about 4.4 TB/day before compression. (b) Scene cloud fractions follow a U-shaped beta distribution with a mean of 55 % (land); scenes more than 70 % cloudy are discarded.

**FIGURE 45.12** Imager data versus link capacity. (a) Daily data from pushbroom imagers imaging 30 % of the time, against assumed daily capacities of X-band, Ka-band and optical links; Sentinel-2's reported 1.5 TB a day for comparison. (b) For a 3 m imager, compression, cloud screening and sending only detections cut 2.9 TB a day to 8 GB.

Optical links change the picture (Chapter 7). NASA's TBIRD cubesat sent 4.8 TB in a single pass of under five minutes in April 2023, at 200 Gbit/s. But clouds block lasers, so optical ground stations must be spread across regions with clear skies (Figure 45.14).

### 45.7.2 Cloud Screening

About two-thirds of Earth is covered by cloud at any moment. More than a decade of MODIS observations gives a global cloud fraction of about 67 %: about 55 % over land and 72 % over ocean (Figure 45.13). For an optical imager, most cloudy pixels are useless, so a model that discards them on board saves link capacity and ground storage. The risk is a **false negative** — a clear image wrongly discarded is lost forever.

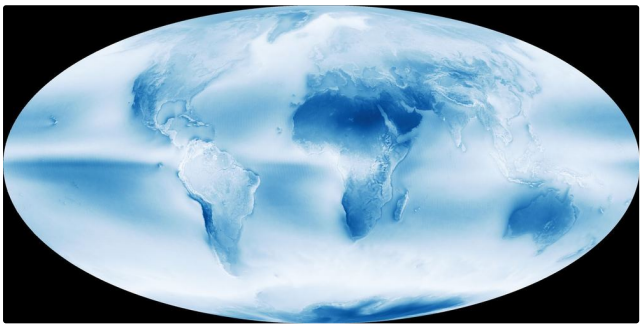


**FIGURE 45.13** Average cloud cover from MODIS. About 67 % of Earth is typically cloud-covered, most of all over the oceans. Image: NASA/GSFC.

**FIGURE 45.14** The 34 m antenna DSS-13 at Goldstone, California, with an optical terminal (below the centre of the dish) that receives laser signals from NASA's Psyche spacecraft. Clouds limit optical links. Image: NASA/JPL (NASA/JPL-Caltech).

**EXAMPLE: DOWNLINK SAVED BY ONBOARD CLOUD SCREENING**

A small satellite carries a 3 m imager with 8 000 pixels across (a 24 km swath). It has 4 bands of 12 bits, images 30 % of the time over land, and downlinks on X-band at 0.5 Gbit/s.

1. **Raw data.** 8 000 × (7 000 / 3) lines/s × 4 × 12 bits = 0.90 Gbit/s; over 30 % of a day this is 2.90 TB.
2. **Compression.** Lossless compression of 2:1 gives 1.45 TB a day. That needs 6.5 hours of X-band contact — far more than a small network of stations provides.
3. **Cloud statistics.** Assume scene cloud fractions follow a U-shaped beta distribution with a mean of 55 %, typical of land: many scenes are nearly clear and many nearly overcast. Then 42.6 % of scenes are more than 70 % cloudy.
4. **Screening.** Discarding those scenes leaves 0.83 TB a day and cuts the contact time to 3.7 hours. The discarded scenes held about 70 % of all cloudy pixels but only about 9 % of the clear ones.
5. **Detections only.** If the mission needs only objects such as ships, sending small image chips around detections (1 % of pixels) reduces the volume to about 8 GB a day — under three minutes of contact.

The screen saves about 40 % of the link but costs 9 % of the clear pixels. Whether that trade is right depends on the mission's users.

### 45.7.3 Latency, Power, Thermal and Verification Costs

Downlink is not the only factor (Table 45.5). Onboard processing can cut the delay from observation to alert from hours to minutes. It also lets a satellite retask itself, as JPL's dynamic-targeting experiment on CogniSAT-6 did in 2025 (Chapter 43). But every watt of computing must be generated, stored for eclipse and rejected as heat, and flight software must be verified to a higher standard.

**TABLE 45.5** Trade-offs between onboard and ground processing.

| Factor | Processing on board | Processing on the ground |
|---|---|---|
| Downlink | Products and alerts, megabytes to gigabytes | All raw data, terabytes |
| Latency | Seconds to minutes (with relays) | Hours, waiting for passes and processing |
| Computing | Limited; tolerant parts lag | Practically unlimited GPUs and TPUs |
| Power and heat | Adds 1–700 W to the bus | Cheap mains power |
| Updating models | Limited by uplink; must be re-verified | Any time |
| Re-analysis | Discarded data are lost | Full archive kept for future methods |
| Assurance | Flight-software standards; hard to fix | Normal software practice |

The usual answer is a mix: simple, well-tested models on board to screen, compress and alert, and heavier models on the ground.

## 45.8 Updating AI in Orbit

A model frozen at launch will age (see Figure 45.5). The key idea of this section is that space AI must be designed to be updated safely, like any other flight software.

### 45.8.1 Uploading New Models

Uploading a model needs four things. The flight software must accept new applications without a full reload. The uplink must carry the model, which 8-bit storage makes easier. Each upload must be authenticated. And a rollback path must restore the previous version if the new one fails.

Missions now demonstrate this routinely. ESA's **Φsat-2**, a 6U cubesat built by Open Cosmos and launched on 16 August 2024, runs its AI on Ubotica's CogniSAT processing board. Its applications run on the NanoSat MO Framework and can be installed and updated from the ground. At launch it carried four: image-to-map conversion, cloud detection, vessel detection and image compression. Two more — marine anomaly detection and wildfire detection — were uploaded after launch. Its operational phase from April 2025 to March 2026 was used to fine-tune the applications in orbit.

**OPS-SAT: A FLYING LABORATORY FOR SOFTWARE**

ESA's OPS-SAT, a 3U cubesat launched on 18 December 2019, was built to let outside teams test new software on a real spacecraft. ESA described its experimental computer as about ten times more powerful than those on its other spacecraft of the time. By its re-entry on 22–23 May 2024, 134 teams from 26 countries had run more than 284 experiments. Firsts included training machine-learning models on board and retraining them with live data. Its 2022 Kelvins competition asked teams to build small, quantised image classifiers designed to run on the satellite. OPS-SAT showed that frequent, safe software changes are possible when the spacecraft is designed for them.

### 45.8.2 Software-Defined Payloads and Split Computing

A **software-defined payload** is one whose function is set largely by software: a radio whose waveforms can change, or an imager whose processing chain can be replaced. Packaged as containers, its applications can serve several customers on one satellite.

**Split computing** divides a network between satellite and ground. The first layers run on board and turn the image into compact features, which are sent down to the rest of the network. Splitting where the features are smallest saves downlink without fitting the whole model on board.

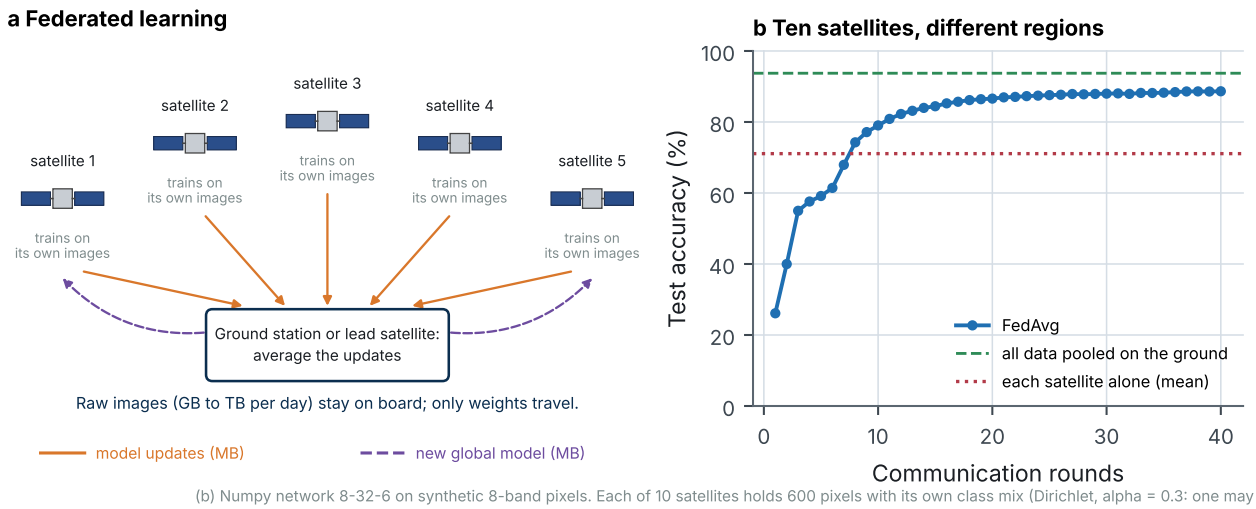
### 45.8.3 Federated Learning across Constellations

**Federated learning** trains one model across many devices without collecting their raw data. In a constellation, each satellite trains on its own images and sends only model updates, megabytes instead of terabytes. A ground station or lead satellite averages them. In the standard **FedAvg** algorithm, the new global weights are a weighted mean:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} \, w_{t+1}^{(k)},$$

Here $w_{t+1}^{(k)}$ are the weights trained by satellite $k$ in round $t$ on its $n_k$ samples, and $n = \sum_k n_k$.

The simulation in Figure 45.15 has ten satellites, each seeing a different mix of surfaces, as if one flew mostly over ocean and another over desert. A satellite training alone reaches only 71.1 % on average on a balanced test set, and pooling all data on the ground gives 93.7 %. FedAvg reaches 79.0 % after 10 rounds and 88.6 % after 40 without moving any raw data. The remaining gap comes from the satellites' very different data, a known weakness of simple averaging.




(b) Numpy network 8-32-6 on synthetic 8-band pixels. Each of 10 satellites holds 600 pixels with its own class mix (Dirichlet, alpha = 0.3: one may see mostly ocean, another desert). Each round every satellite trains 2 epochs from the global model; updates are averaged by sample count. Test: 6 000 pixels with all classes equally likely.

**FIGURE 45.15** Federated learning across a constellation. (a) Satellites train on their own images and send only model updates, which are averaged and returned. (b) Computed with numpy networks on synthetic pixels: FedAvg (blue) approaches pooling all data on the ground (green) and far exceeds satellites training alone (red).

As of October 2026, federated learning across satellites remains **research**. Simulation studies have adapted it to intermittent contacts and to about 100 satellites, but no in-orbit demonstration across a constellation had been reported.

## 45.9 Computing in Orbit: Orbital Data Centres

The boldest idea in this chapter turns edge AI inside out: put data-centre computers into orbit, powered by sunlight and cooled by radiating heat to space. The physics allows it, but the engineering and economics are unproven. Table 45.6 lists the main projects as of October 2026.

### 45.9.1 Demonstrations and Proposals

**TABLE 45.6** Computing in orbit: demonstrations and proposals, October 2026.

| Project | Who | When | What | Status |
|---|---|---|---|---|
| Spaceborne Computer-2 | HPE, NASA | 2021; new unit 2024 | Servers and a T4 GPU on the ISS | Demonstration |
| Kepler optical relay | Kepler | January 2026 | 10 relay satellites with GPU modules | In service since August 2026 |
| Three-Body constellation | ADA Space, Zhejiang Lab | May 2025 | 12 satellites, laser links to 100 Gbit/s | Demonstration; 2 800 planned |
| Starcloud-1 | Starcloud | November 2025 | One H100 on a 60 kg satellite | Demonstration; next satellite 2027 |
| Suncatcher prototype | Google, Planet | October 2026 | Four TPUs; one-year test | Demonstration |
| Orbital data centres | SpaceX | FCC filing, January 2026 | Up to one million satellites | Proposal |

**Spaceborne Computer-2** showed the first benefit: less data to send. In a 2021 genomics test it reduced 182 GB of raw data to 13 MB of results, over a link limited to about two hours a week at up to 250 KB/s.

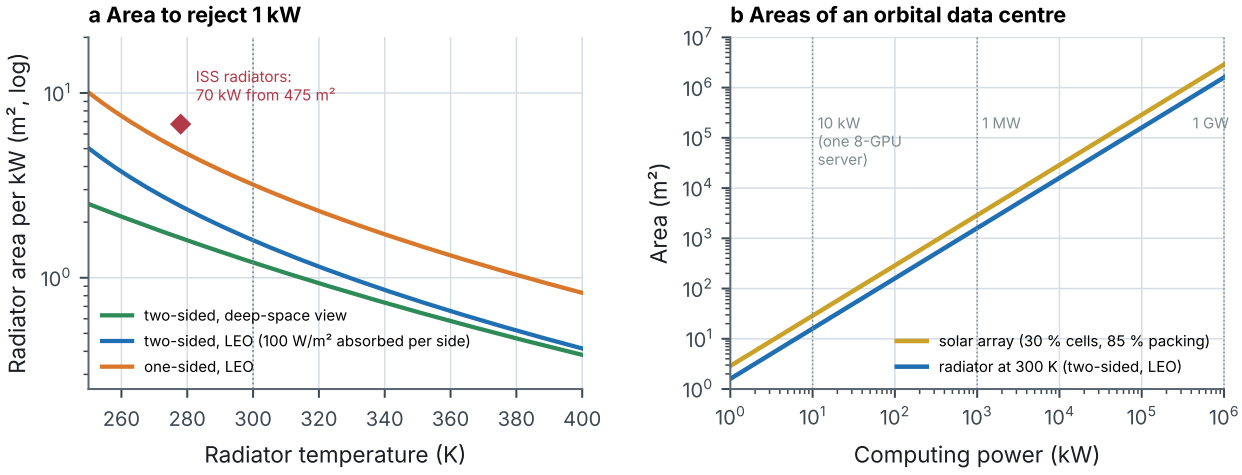
China's **Three-Body Computing Constellation** is the largest effort so far. Its first 12 satellites, launched on 14 May 2025, offer about 5 POPS (peta-operations per second) combined, linked by lasers. In February 2026 Chinese state media reported ten AI models deployed, two with 8 billion parameters. **Starcloud-1**, launched on 2 November 2025, carried one H100 GPU. In December 2025 the company reported training a small language model on the works of Shakespeare in orbit. Both are demonstrations; neither yet sells computing at scale.

## 45.9.2 Cooling: the Radiator Problem

In orbit, the only way to lose heat is to radiate it, and almost all the power a processor uses becomes heat. A radiator panel of area $A$, emissivity $\varepsilon$ and temperature $T$ rejects

$$P = \varepsilon \sigma A \left(T^4 - T_{\text{sink}}^4\right) - \alpha A q_{\text{abs}},$$

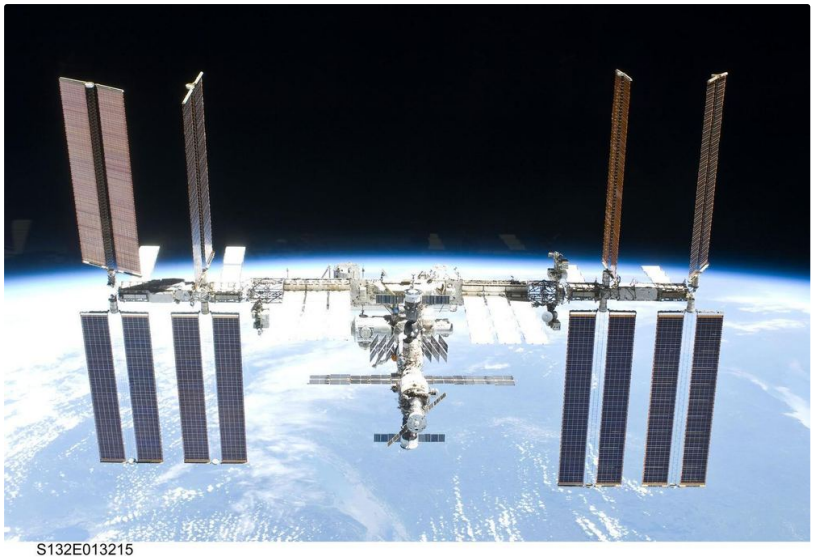
Here $\sigma$ is the Stefan–Boltzmann constant, $5.67 \times 10^{-8}$ W m$^{-2}$ K$^{-4}$. The flux $q_{\text{abs}}$ is the sunlight and Earth infrared that the panel absorbs (Chapter 5). At 300 K with $\varepsilon$ = 0.9, each side radiates 413 W/m$^2$. Because of the fourth power, area grows steeply as temperature falls (Figure 45.16). Hotter chips mean smaller radiators, but electronics age faster when hot.




Net rejection per side = emissivity (0.9) x Stefan-Boltzmann constant x T^4 minus absorbed environmental flux; fin efficiency, temperature drop from chip to radiator and design margin are ignored, which is why the real ISS radiators need about four times the ideal area. Solar array in continuous sunlight (dawn-dusk orbit), 1 361 W/m².

**FIGURE 45.16** Radiator physics for computing in orbit. (a) Area to reject 1 kW: 1.6 m² for a two-sided panel at 300 K in LEO; the ISS's radiators need about four times the ideal. (b) Radiator and solar-array areas grow with computing power; a gigawatt would need about 1.6 km² of radiator and 2.9 km² of solar array.

The ISS gives a reality check (Figure 45.17). Its external active thermal control system rejects up to 70 kW. It uses two radiator wings of three units each; every unit is 23.3 m × 3.4 m, for 475 m² of panel in all. That is 6.8 m² per kW, against an ideal of about 1.6 m² per kW for two-sided panels near 280 K.



S132E013215

**FIGURE 45.17** The ISS in May 2010 (STS-132). Between the solar-array wings, white radiator panels on the truss reject up to 70 kW — the heat of about seven eight-GPU AI servers. Image: NASA/JSC.

**EXAMPLE: A RADIATOR FOR 10 KW OF COMPUTING**

A 10 kW AI server — about one machine with eight data-centre GPUs — must reject its heat at 300 K in low Earth orbit. Assume a two-sided panel with emissivity 0.9, absorbing 100 W/m² from Earth and the Sun on each side.

1. **Ideal flux.** $\varepsilon\sigma T^4 = 0.9 \times 5.67 \times 10^{-8} \times 300^4 = 413$ W/m² per side.
2. **Ideal area.** With no absorbed flux, A = 10 000 / (2 × 413) = 12.1 m².
3. **With environment.** Net flux 413 − 100 = 313 W/m² per side, so A = 10 000 / (2 × 313) = 16.0 m².
4. **With margin.** Allow 30 % for fin efficiency, temperature drops and degradation: about 21 m².
5. **Mass.** ISS radiator units weigh about 14 kg/m² (1 123 kg for 79 m²), giving about 300 kg.
6. **Launch cost.** At about $3 600/kg, the radiator alone costs about $1.1 million to launch. The solar array for 10 kW needs another 29 m².

At 350 K the area before margin would fall to 7.5 m², if the chips tolerate the heat.

### 45.9.3 Economics

The economic case rests on three claims. Sunlight in a dawn–dusk orbit is almost continuous; Google estimates that panels in some orbits collect up to eight times the yearly energy of a panel at mid-latitudes. No land, water or grid connection is needed. And launch costs are falling.

Google's Suncatcher study puts numbers on the last point. At about $200/kg, possible by the mid-2030s, launched power would cost about $810 per kilowatt per year. Terrestrial data centres pay $570–3 000 per kilowatt per year for electricity. At a current price of about $3 600/kg, the estimate is $14 700 per kilowatt per year. The **ASCEND** study for the EU found orbital data centres feasible only if launchers became ten times less polluting over their life cycle.

The counter-arguments are strong. Radiators and solar arrays are large and heavy. Failed chips cannot be swapped. Radiation causes silent errors, as the Suncatcher tests showed. Data must still cross limited links to Earth, and crowded orbits add debris risk (Chapter 21). For now, orbital data centres are serious demonstrations and bold proposals, not an industry.

### PRACTICE: QUALIFYING AN AI PAYLOAD FOR A SMALL SATELLITE

Small satellites are often the first home of a new AI payload (Figure 45.18). A typical workflow:

1. **Define the function and its criticality.** State what the model decides and what happens if it is wrong; keep it out of safety-critical loops. The European handbook ECSS-E-HB-40-02A (November 2024) gives guidance.
2. **Budget resources.** Set limits for power, energy per orbit, heat, memory, mass and data volume. Estimate throughput at 20–30 % of the processor's peak.
3. **Choose hardware and protection.** Predict the dose and upset rate for the orbit (Chapter 2) with a design margin of at least two. Add a latch-up current limiter, a watchdog and a radiation-tolerant supervisor for any commercial chip.
4. **Test for radiation.** Run total-dose tests on parts from the flight lot. Run proton or heavy-ion tests with the real model executing, logging every upset, interrupt and wrong output.
5. **Freeze and quantise the model.** Compare the flight version bit for bit with the ground reference on the locked test set. Measure accuracy, time and energy on the flight hardware.
6. **Inject faults.** Flip bits in weights, activations and code. Choose ECC, scrub interval and range checks that hold accuracy at the predicted upset rate with margin.
7. **Test the environment.** Run vibration, thermal-vacuum and electromagnetic tests with the processor at full load in the hot case.
8. **Secure updates.** Package the model as an application with signed uploads, version records and a rollback path. Keep safe mode independent of the AI.
9. **Plan operations.** Commission in shadow mode — deciding but not acting — and downlink samples to check the decisions. Then monitor for drift.

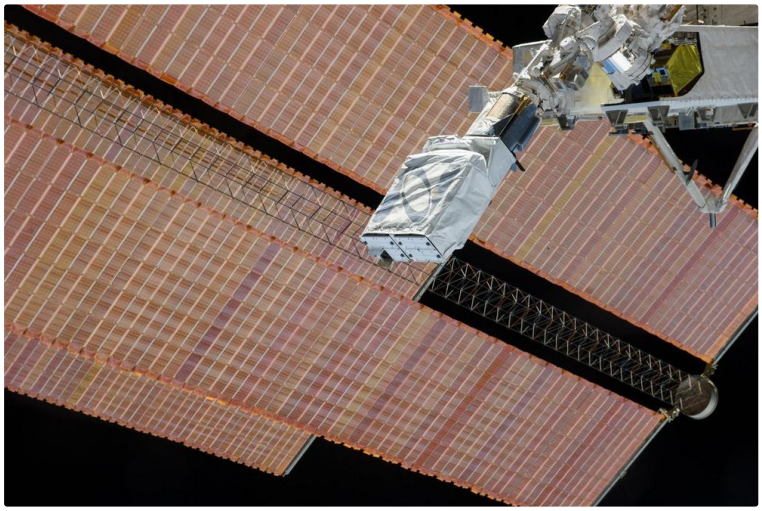


**FIGURE 45.18** Japan's Small Satellite Orbital Deployer, held by the Kibo robotic arm on 11 February 2014, shortly before it released NanoRacks cubesats from the ISS. Many AI payloads are first proven on small satellites like these. Image: NASA/JSC.

### KEY TAKEAWAYS

- Archives hold hundreds of petabytes, but labelled, AI-ready data are scarce. Active learning can cut labelling about threefold; systematic label errors harm far more than random ones.
- Benchmarks enable fair comparison but leak, saturate and contain label errors. Synthetic data need domain randomisation and adaptation to cross the sim-to-real gap.
- MLOps — versioned data, tracked experiments, model registries, validation gates and drift monitoring — keeps a model trustworthy for the life of a mission.
- Radiation flips stored bits. Float32 weights fail when an exponent bit flips; int8 storage, range checks, ECC with scrubbing and TMR restore robustness.
- Commercial AI chips are thousands of times more efficient than hardened CPUs but need testing, shielding and supervision; hardened parts such as PIC64-HPSC are closing the gap slowly.
- Quantisation to 8 bits costs almost nothing, and QAT, pruning and distillation go further. Onboard screening saves downlink but discards data for ever.
- Models in orbit must be updatable, as Φsat-2 and OPS-SAT showed. Federated learning and orbital data centres remain research and demonstration, limited by radiators, radiation and economics.

### REVIEW QUESTIONS

1. Compare expert, crowdsourced and weak labels. How does margin sampling choose what to label next?
2. In Figure 45.2, why does random label noise of 50 % reduce accuracy only slightly, while 50 % of urban pixels labelled as soil cuts urban recall to about 31 %?
3. What is the sim-to-real gap? Use the SPEED+ numbers to describe its size, and explain how domain randomisation and domain adaptation reduce it.
4. List the stages of an MLOps pipeline. What should a validation gate check before a model is uplinked?
5. Compute the PSI for a band where the training data fall into four bins in proportions 0.25 each and current data in proportions 0.10, 0.20, 0.30 and 0.40. Is retraining advised?
6. Why does a single flipped exponent bit in a float32 weight do so much more damage than a flipped bit in an int8 weight? How do ECC, scrubbing and range checks work together?
7. A model has 10 million int8 weights in SRAM with $7.6 \times 10^{-7}$ upsets per bit per day. How many upsets occur per day, and how often must memory be scrubbed so that a 72-bit ECC word rarely holds two errors?
8. Explain post-training quantisation and quantisation-aware training. Why did 2-bit PTQ fail in Figure 45.11 while 2-bit QAT kept 93.2 %?
9. A 0.5 m imager with 30 000 pixels across, 4 bands and 12 bits images 20 % of the time. Estimate its daily data volume and compare it with X-band and optical links. What onboard processing would you propose?
10. Estimate the radiator area needed to reject 100 kW at 330 K from a two-sided panel absorbing 100 W/m$^2$ per side. Discuss two technical and two economic obstacles to orbital data centres.

# 46 CHAPTER AI for Earth Observation

Image: NASA/GSFC

> *Earth-observation satellites now collect far more images than people could ever look at, so most pixels are first seen by a machine. This chapter explains, from first principles and with small models trained from scratch, how AI turns pixels into information. It covers land-cover maps, objects, change, crop calendars, radar speckle, ships, floods and methane plumes. It then follows verified deployments in agriculture, forests, cities, oceans, disasters, greenhouse gases and development, as of October 2026. It describes AI on board satellites and smart tasking, the commercial analytics business and geospatial foundation models in practice. It ends with the traps — domain shift, label noise, biased samples, wrong area estimates and privacy — and how good practice avoids them.*

**IN THIS CHAPTER**

46.1 From Pixels to Information
46.2 Land: Crops, Forests, Cities and Water
46.3 Oceans and Maritime Awareness
46.4 Disasters
46.5 Greenhouse Gases and Human Development
46.6 AI on Board and Smart Tasking
46.7 The EO Analytics Business
46.8 Pitfalls and Good Practice

Every day, Earth-observation (EO) satellites image the whole land surface several times over. Planet's fleet alone can collect about 200 million km$^2$ a day. Sentinel-1's radars see through cloud, day and night. Thermal, hyperspectral and very-high-resolution sensors add more. No team of people can inspect all of it. The images are only useful if software finds the floods, ships, fires, new buildings and methane leaks in them, quickly and reliably.

That software is increasingly **machine learning**: models whose behaviour is learned from examples rather than written as rules. This chapter shows how such models work on EO data, where they are used today, and where they fail. The hero image above shows centre-pivot fields

in Kansas in a Landsat false-colour composite. Crop type, health and harvest date can all be read from such images by a trained model.

The physics of remote sensing is covered in Chapters 13–16. The learning methods themselves are explained in Chapters 43 and 44, and data and onboard hardware in Chapter 45. Weather and climate models are left to Chapter 47, and military uses to Chapter 50. Here we join the two: what a learning model needs to know about EO data, and what EO users need to know about learning models.

## 46.1 From Pixels to Information

The key idea is that EO analysis is a short list of tasks — classify, segment, detect, compare, estimate — applied to images that are unlike everyday photographs. They have many spectral bands, a time dimension and an exact location on Earth. Each task has a matching family of models and a matching way to measure success (Table 46.1).

The pipeline in Figure 46.1 shows where the models sit. Raw data are calibrated and corrected into **analysis-ready data** (ARD) — pixels in physical units on a map grid (Chapter 13). Models turn ARD into maps, alerts and estimates. Users send back corrections and new labels, and some detections trigger new images of the same place. A growing share of the work also happens on the satellite itself.

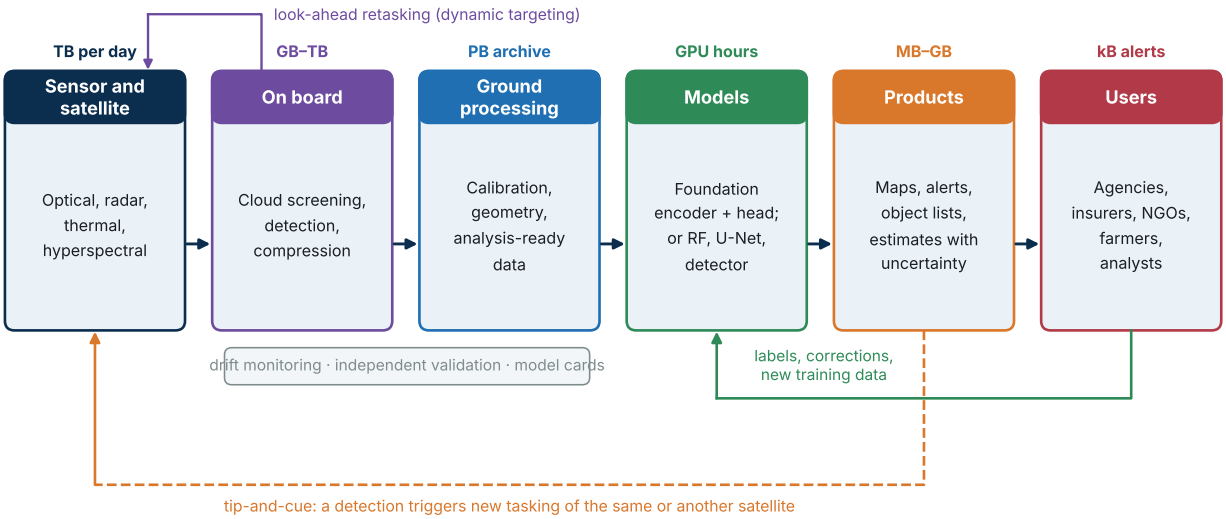



**FIGURE 46.1** The EO data pipeline with AI. Each stage shrinks the data: terabytes of pixels a day become megabytes of maps and kilobytes of alerts. Three loops close the system: users supply new labels, detections cue new collections (tip-and-cue), and an onboard look-ahead camera can retask the satellite in under 90 seconds.

### 46.1.1 What Makes EO Data Different

Four things set EO images apart from the photographs on which computer vision grew up. First, they have **many bands**: Sentinel-2 records 13, from blue to short-wave infrared, and imaging spectrometers record over 200. Second, they come in **time series**: the same field is seen every few days. Third, every pixel has a **location**, so neighbouring pixels are alike and nearby samples are not independent. Fourth, the **physics changes with the sensor**: radar measures surface roughness and moisture, not colour (Chapter 15).

Labels are scarce. A satellite can image a country in a day, but surveying land cover on the ground takes months. Many EO models are therefore trained on a few thousand labelled pixels, on labels made by other models, or on simulations (Chapter 45).

**TABLE 46.1** Common EO tasks, the models used for them and how they are scored. IoU is intersection over union, the overlap between mapped and true areas divided by their union.

| Task | Output | Typical models | Usual metrics | EO examples |
|---|---|---|---|---|
| Scene classification | One label per image chip | CNN, vision transformer | Accuracy, F1 | Cloudy or clear; land-use type |
| Semantic segmentation | A class for every pixel | Random forest, U-Net, segmentation transformer | Overall accuracy, IoU | Land cover, floods, burn scars |
| Object detection | Boxes with labels | Faster R-CNN, YOLO, CFAR + classifier | Precision, recall, mean average precision | Ships, aircraft, vehicles |
| Instance segmentation | An outline per object | Mask R-CNN | Mask average precision | Building footprints, fields |
| Change detection | Changed pixels or objects | Siamese CNN, bi-temporal classifier | F1 of the change class | Deforestation, damage |
| Time-series classification | A label per pixel or field | Random forest, temporal CNN, attention | Accuracy, F1 | Crop type, phenology |
| Regression and retrieval | A physical quantity | Gradient boosting, CNN, physics + learning | RMSE, bias | Biomass, methane emission rate |
| Image-to-image | A new image | U-Net, GAN, diffusion model | PSNR, SSIM, task skill | Super-resolution, despeckling, SAR to optical |

### 46.1.2 Land Cover: Pixels and Neighbourhoods

**Land-cover mapping** gives every pixel a class such as water, forest, cropland or built-up land. The classic approach treats each pixel on its own. Its features are the reflectances in each band, plus indices such as the **normalised difference vegetation index**,

$$\mathrm{NDVI} = \frac{\rho_{\mathrm{NIR}} - \rho_{\mathrm{red}}}{\rho_{\mathrm{NIR}} + \rho_{\mathrm{red}}},$$

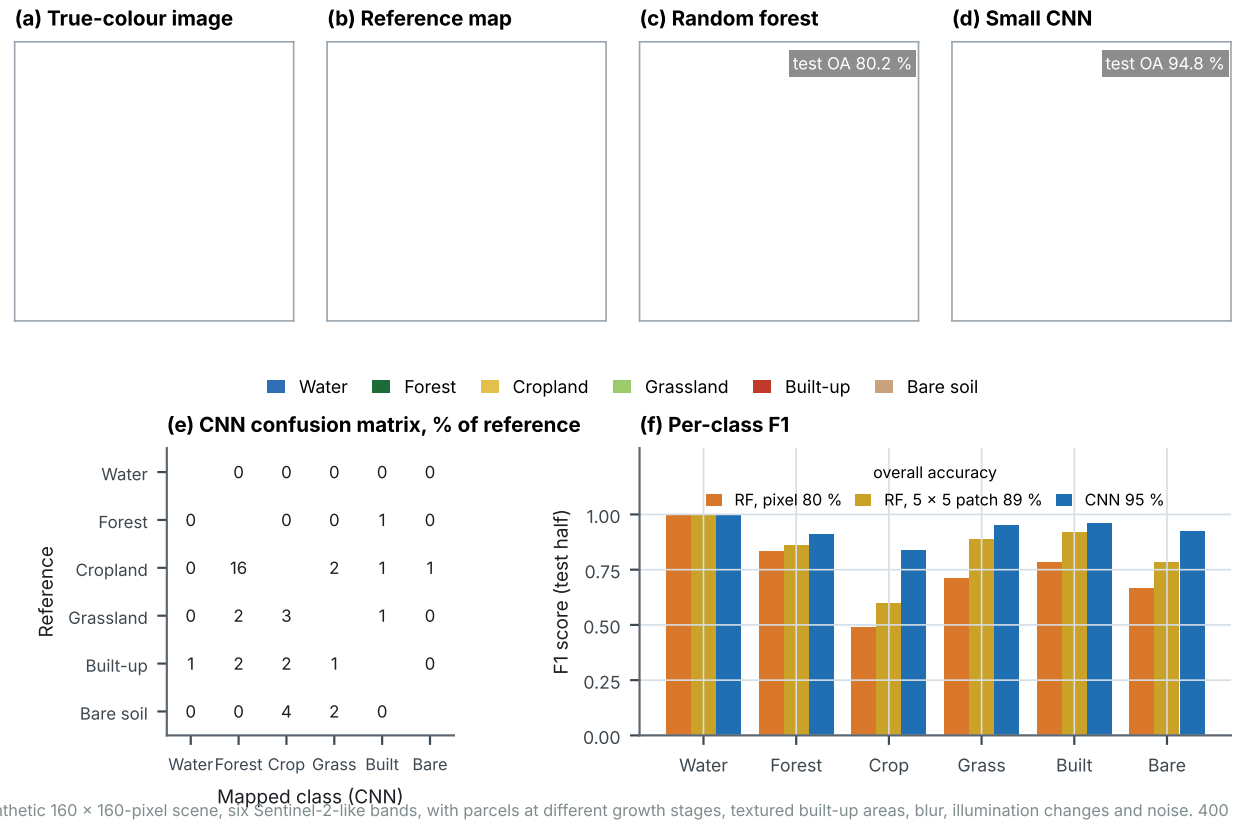
where $\rho$ is reflectance. Healthy plants reflect strongly in the near infrared (NIR) and absorb red light, so NDVI is high for vegetation and near zero for soil and water. A **random forest** — hundreds of decision trees, each trained on a random subset of the data, voting together (Chapter 43) — is still the workhorse for such pixel-based maps.

A pixel alone is often ambiguous. A dry field looks like bare soil, and a suburb is a mixture of roofs, roads and gardens. A **convolutional neural network** (CNN) looks at a neighbourhood instead. Each layer slides small filters over the image and sums weighted pixel values,

$$y_{ij} = \sigma\Big(\sum_{c}\sum_{u,v} w_{c,uv}\, x_{c,\,i+u,\,j+v} + b\Big),$$

where $\sigma$ is a simple non-linear function. The filters learn textures and shapes as well as colours (Chapter 44). For whole maps, **encoder–decoder** networks such as the U-Net segment large tiles in one pass.

In Figure 46.2 the two are compared on a synthetic six-class scene with Sentinel-2-like bands. Both models learned from 400 labelled pixels per class in the western half and were tested on all 12 800 pixels of the eastern half. The random forest on single pixels reached an overall accuracy of 80.2 %. Cropland was its weakest class, with an F1 score — the balance of precision and recall — of only 0.49, because fields at different growth stages look like grass or bare soil. A random forest given the same 5 × 5-pixel neighbourhoods reached 89.0 %. A small CNN with about 10 000 weights, written in numpy, reached 94.8 % and raised cropland F1 to 0.84. Context, not the network as such, did most of the work.




Synthetic 160 × 160-pixel scene, six Sentinel-2-like bands, with parcels at different growth stages, textured built-up areas, blur, illumination changes and noise. 400 labelled pixels per class from the western half train each model; all 12 800 pixels of the eastern half test it. CNN: one 5 × 5 × 64 convolution, ReLU, 1 × 1 output (10 054 weights), written in numpy.

**FIGURE 46.2** Land-cover mapping of a synthetic scene. (a)–(d) Image, reference and two maps; models learned from the western half and were tested on the eastern half. (e) The CNN's remaining confusion is mostly cropland labelled as forest. (f) Neighbourhood information raises F1 for every mixed or textured class.

The synthetic test is easier than reality: global maps reach overall accuracies of about 74–77 % (Table 46.2). But the ranking is typical. Pixel-based random forests remain strong baselines, and convolutional and transformer networks win where texture and shape matter.

### 46.1.3 Objects: Ships, Aircraft and Buildings

**Object detection** finds each object and draws a box around it. Two-stage detectors first propose candidate regions and then classify them; one-stage detectors such as the YOLO family do both in one pass, which suits fast processing (Chapter 44). **Instance segmentation** outlines each object, which is what building-footprint maps need.

EO objects are small and rotated. A 50 m ship is five pixels long at 10 m resolution, and a car is a few pixels at 0.3 m. Detectors for EO therefore use rotated boxes, image pyramids and tiles that overlap so that objects on tile edges are not cut in half. Large public sets such as xView (over a

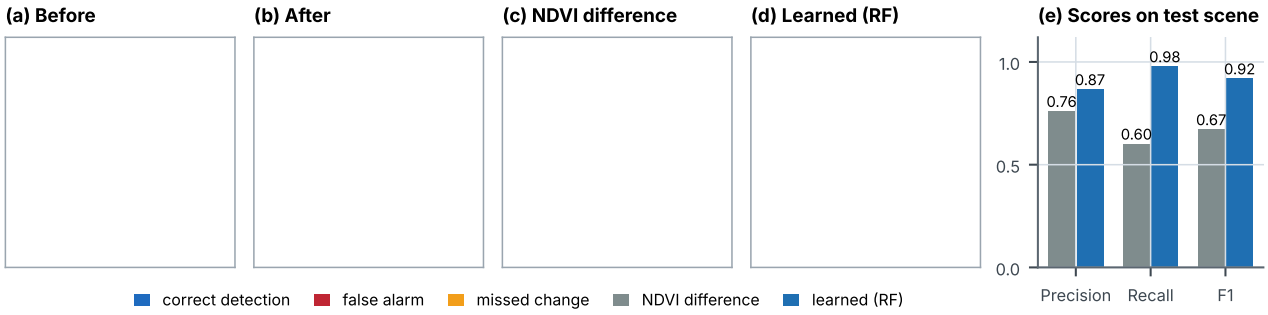
million objects in 0.3 m images) and xBD (850 736 buildings before and after 19 disasters) made this progress possible (Chapter 45).

### 46.1.4 Change Detection

Most EO questions are about **change**: what was cleared, flooded, built or burned. The simplest method subtracts an index between two dates and applies a threshold, for example flagging forest loss where $\Delta\mathrm{NDVI} = \mathrm{NDVI}_2 - \mathrm{NDVI}_1$ falls below −0.5. It is fast and easy to explain. But it reacts to every change in the index: harvests, seasons, haze, shadows and small misalignments between the images.

A **learned change detector** is trained on examples of the changes that matter and of those that do not. It sees both images, all bands and the local neighbourhood. Deep versions use **Siamese networks**, two copies of one encoder that process the two dates and compare their features.

The test in Figure 46.3 makes the difference concrete. The task is to find forest clearings in a synthetic scene that also contains harvested fields, drying grass, haze, a cloud shadow and a one-pixel misregistration. Both methods were tuned on four other scenes. The best NDVI threshold (−0.55) found only 60 % of the cleared pixels, with a precision of 0.76 and an F1 of 0.67. A random forest on 16 bi-temporal features found 98 %, with a precision of 0.87 and an F1 of 0.92. It had learned that a drop in NDVI over a field is a harvest, not deforestation.




Synthetic 128 × 128-pixel scene (false colour: NIR, red, green). Forest clearings are the only true change; harvests, drying grass, haze, a cloud shadow and 1-pixel misregistration are not. The NDVI threshold (-0.55) and the random forest (200 trees, 16 bi-temporal features) were both tuned on four other scenes.

**FIGURE 46.3** Change detection on a synthetic scene where only forest clearings count as change. (a, b) False-colour images before and after. (c) An NDVI-difference threshold misses partly cleared patches and flags harvests. (d) A random forest trained on other scenes finds almost all clearings. (e) Precision, recall and F1 on the test scene.

Dense time series go further. Algorithms such as CCDC and BFAST fit a seasonal curve to every pixel's history and flag a break when new data stop fitting it (Chapter 13). Operational forest alerts, described below, combine such statistics with machine learning.

### 46.1.5 Time Series and Crop Types

Crops are best recognised by their **calendar**. Winter wheat greens up in spring and is harvested in summer. Maize and soybean start later. Grassland drops after each mowing (Figure 46.4 a). A model that sees the whole season can tell them apart even when single images cannot.

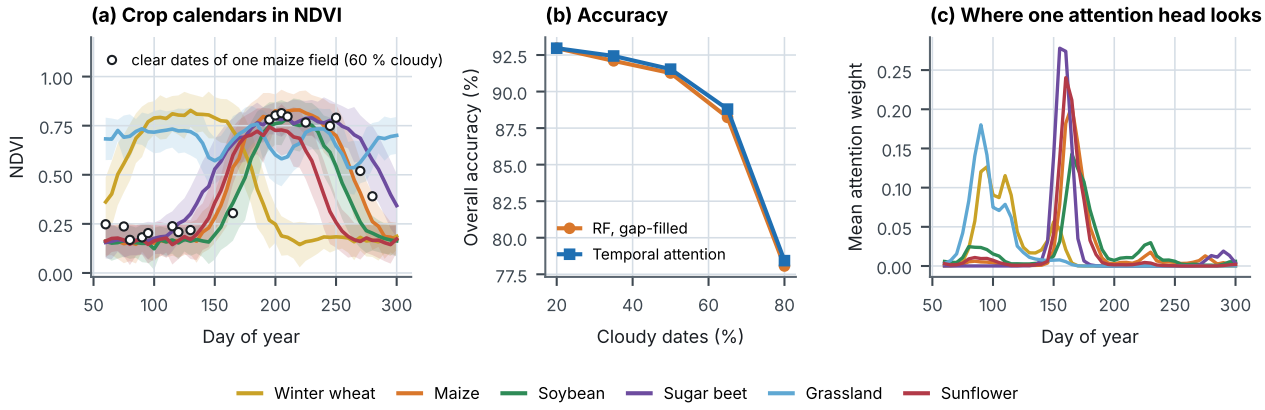
Clouds make the series irregular. One option is to fill the gaps by interpolation and give the filled series to a random forest. Another is **temporal attention**, which works directly on the clear dates. Each observation $t$ becomes a feature vector $\boldsymbol{h}_t$ that encodes its value and its day of year. A

learned query $q$ scores each date, and the model averages the dates with weights

$$a_t = \frac{\exp(q \cdot k_t / \sqrt{d})}{\sum_{s \in \text{clear}} \exp(q \cdot k_s / \sqrt{d})}, \qquad z = \sum_t a_t \, h_t,$$

where $k_t$ is a "key" computed from $h_t$ and $d$ its length. Cloudy dates simply drop out of the sum. The pooled vector $z$ is then classified. Garnot and colleagues' pixel-set encoder with temporal attention (2020) reached 94.2 % overall accuracy on the S2-Agri benchmark of French fields.

In Figure 46.4 both approaches are tested on synthetic NDVI series for six crops, with timing shifts and undetected thin cloud. A numpy attention network with six heads and about 5 000 weights matched a random forest on gap-filled series. Both reached 93.0 % with 20 % of dates cloudy and about 78 % with 80 % cloudy. Panel (c) shows what one head learned: it looks at early spring, which separates wheat and grass, and at early June, which separates the summer crops. On small, clean problems attention gives no free gain. Its advantages appear with large and varied training sets, many bands and irregular sampling across regions.




Synthetic fields: double-logistic NDVI curves (grass: three mowing cuts) with ±10-day timing shifts, amplitude spread and noise; 49 dates every 5 days; clouds remove dates at random and 8 % of 'clear' dates carry undetected thin cloud. Both models trained on 5 000 fields, tested on 3 000 per point. Attention network: six heads, 4 998 weights, numpy.

**FIGURE 46.4** Crop types from NDVI time series (synthetic data). (a) Mean seasonal curves ± one standard deviation, and the clear observations of one field. (b) Accuracy against the share of cloudy dates for a random forest on gap-filled series and a temporal-attention network. (c) Mean attention weights of one head for each crop.

### 46.1.6 Clouds, Sharper Images and Fusion

**Cloud and shadow masking** is the first step for any optical model, because cloud covers about two-thirds of Earth at any time. Rule-based masks such as Fmask test brightness, temperature and band ratios. Learned masks now do better. The s2cloudless detector uses gradient-boosted trees on ten Sentinel-2 bands, and CNNs trained on sets such as CloudSEN12 (49 400 patches) also find thin cloud and shadows. Missed thin cloud is a quiet source of error in every downstream product.

**Pan-sharpening** merges a sharp panchromatic band with coarser colour bands (Chapter 14). **Super-resolution** goes further and predicts detail finer than any band, for example 10 m Sentinel-2 images sharpened to 2.5 m. Networks trained on pairs of images from fine and coarse sensors learn plausible detail. But they can **hallucinate**: they may invent edges, cars or field boundaries that look real. Super-resolved images are useful for display and as model inputs, but they are not new measurements.

**Data fusion** combines sensors. Radar sees through cloud, optical data are easier to interpret, and the two together beat either alone for crops, floods and forests. Networks can even **translate** radar into optical-looking images. Such images help analysts read radar, but they must never be used as if they were observations.

### 46.1.7 Radar: Speckle and Ships

Radar images carry **speckle**, the grainy pattern made when echoes from many small scatterers in a pixel add with random phases (Chapter 15). For a uniform surface with true reflectivity $R$, the measured intensity is

$$I = R\,n, \qquad n \sim \Gamma(L, 1/L),$$

where $L$ is the number of independent looks averaged. A single-look image ($L = 1$) has a standard deviation equal to its mean. Smoothness is measured by the **equivalent number of looks**, $\mathrm{ENL} = \mu^2/\sigma^2$, over a uniform area.

Averaging neighbours raises ENL but blurs edges. The **Lee filter** (1980) adapts. It smooths where the image is uniform and keeps the original pixel where it is not:

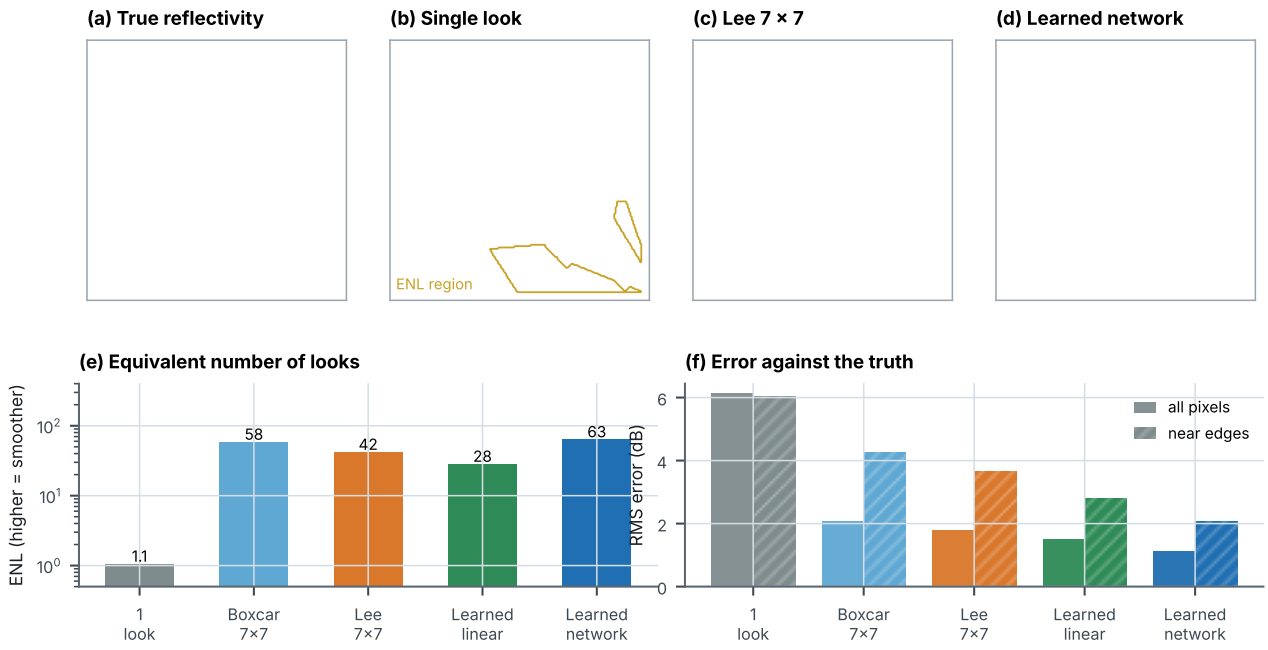
$$\hat{R} = \bar{I} + k\,(I - \bar{I}), \qquad k = \frac{1 - C_u^2/C_I^2}{1 + C_u^2}.$$

Here $\bar{I}$ is the local mean. $C_I$ and $C_u$ are the local and speckle-only coefficients of variation, the standard deviation divided by the mean. Learned filters instead train on pairs of speckled and clean images. Since real clean images do not exist, they use simulated speckle or several dates averaged together. Self-supervised methods train on speckled data alone.

In Figure 46.5 four filters are compared on a synthetic single-look scene. A 7 × 7 boxcar reached an ENL of 58 but blurred edges, with an error of 4.25 dB near them. The Lee filter reached an ENL of 42 and an overall error of 1.81 dB. A least-squares 9 × 9 linear filter, learned from data, cut the error to 1.49 dB. A small numpy network on the same 9 × 9 patches reached an ENL of 63 and an error of only 1.14 dB, and 2.08 dB near edges. The lesson: ENL alone rewards blurring, so always measure error near edges and small targets too.

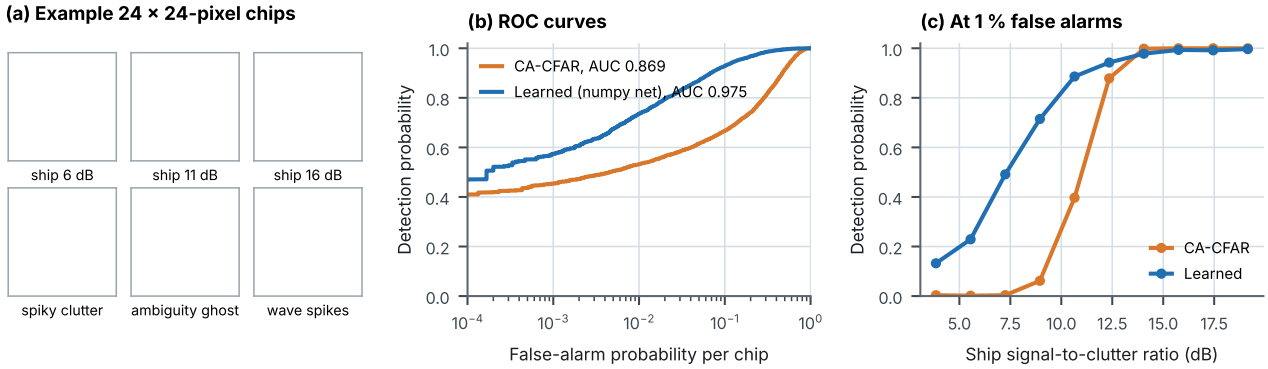
**Ship detection** in radar is the classic EO detection task. Ships are bright metal on a darker sea. The standard detector is the **constant false alarm rate** (CFAR) test. It estimates the local sea brightness $\hat{\mu}$ from a ring of pixels around each test cell and declares a target when the cell exceeds $T = \alpha\hat{\mu}$. For exponential clutter with a known mean, the false-alarm probability is $P_{fa} = e^{-\alpha}$, so $\alpha = -\ln P_{fa}$. Real sea clutter is spikier, so $\alpha$ must be larger.

CFAR flags anything bright. Breaking waves, azimuth “ghosts” of bright objects and small islands all cause false alarms. A learned classifier can look at shape. In Figure 46.6, chips of synthetic K-distributed sea clutter contain ships 2–7 pixels long or look-alikes. At a false-alarm rate of 1 % per chip, cell-averaging CFAR found 53 % of ships, and 286 of its 300 false alarms were ghosts or wave spikes. A small numpy network found 74 %. Its area under the receiver operating characteristic (ROC) curve, which sums up performance at all thresholds, rose from 0.869 to 0.975. Its advantage was largest for weak ships, 5–11 dB above the clutter. In practice the two are chained: CFAR proposes candidates and a network confirms them.




Synthetic 192 × 192-pixel scene: 34 fields between −17 and −5 dB, a −22 dB road and ten +8 dB point targets, multiplied by single-look exponential speckle. Learned filters see 9 × 9 log-intensity patches and were trained on 120 000 patches from three other scenes; the network (two hidden layers, 18 945 weights) is written in numpy.

**FIGURE 46.5** Speckle filtering of a synthetic single-look SAR scene. (a)–(d) Truth, speckled image, Lee filter and learned network (dB scale). (e) Equivalent number of looks in the outlined uniform region. (f) RMS error against the truth for all pixels and near edges.




Synthetic Sentinel-1-like chips: K-distributed sea clutter (shape 1.5), ships 2–7 pixels long at 3–20 dB above the clutter, and look-alikes (azimuth ghosts, breaking-wave spikes). Training: 12 000 chips with flips and rotations; test: 6 000 ships and 30 000 negatives. CA-CFAR: brightest 2 × 2 mean over the background ring mean.

**FIGURE 46.6** Ship detection in synthetic SAR chips. (a) Ships at three signal-to-clutter ratios and three look-alikes. (b) ROC curves: detection probability against false alarms per chip. (c) Detection probability against ship brightness at 1 % false alarms.

### 46.1.8 Spectra and Heights

**Hyperspectral images** record a full spectrum in each pixel (Chapter 14). Their pixels are often mixtures. The **linear mixing model** writes a pixel spectrum $\boldsymbol{x}$ as

$$\boldsymbol{x} = \sum_k a_k \boldsymbol{e}_k + \boldsymbol{\varepsilon}, \qquad a_k \geq 0, \quad \sum_k a_k = 1,$$

where $\boldsymbol{e}_k$ are pure "endmember" spectra, such as soil, grass and roof, and $a_k$ their area fractions. **Unmixing** solves for the fractions. Learned methods add non-linear mixing and use the spatial context. Classifiers for hyperspectral data must also cope with hundreds of correlated, noisy bands and few labels. Dimension reduction and foundation models pre-trained on unlabelled cubes help (Chapter 44).

**3-D reconstruction** turns stereo pairs, radar interferometry or lidar into heights. Learned stereo matching fills gaps in shadows and on uniform roofs. Networks also estimate building heights from single images. Microsoft's building data set includes height estimates for more than 170 million buildings.

## 46.2 Land: Crops, Forests, Cities and Water

On land, AI's main job is to make **wall-to-wall maps** that are updated often. Many are now free and global (Table 46.2).

**TABLE 46.2** Global land products made with machine learning (status October 2026).

| Product | Producer, first release | Input, resolution | Method | Reported quality or size |
|---|---|---|---|---|
| WorldCover | ESA, 2021 (2020 map) | Sentinel-1 and -2, 10 m, 11 classes | Supervised classifier on yearly band statistics | Overall accuracy 74.4 % (2020), 76.7 % (2021) |
| Dynamic World | Google, World Resources Institute, 2022 | Every Sentinel-2 image ≤ 35 % cloud, 10 m, 9 classes | Fully convolutional network; class probabilities | 73.8 % agreement with expert labels |
| WorldCereal | ESA consortium, 2023 (2021 maps) | Sentinel-1, -2, Landsat, 10 m | Gradient-boosted trees | Crop extent: user's 88.5 %, producer's 92.1 % |
| Global ML Building Footprints | Microsoft, 2022– | Bing Maps and Vexcel imagery, 2014–2025 | Segmentation network + polygon step | About 1.4 billion buildings; precision 92–97 % |
| Open Buildings v3 | Google, 2023 | 50 cm imagery; 58 million km² | Segmentation network | 1.8 billion buildings |
| Tree cover loss and alerts | Univ. of Maryland, Wageningen, Global Forest Watch | Landsat, Sentinel-2, Sentinel-1 | Decision trees, statistical change tests | Annual since 2000; weekly alerts |

### 46.2.1 Agriculture

**Crop-type maps** feed food-security outlooks, subsidy checks and insurance. In Europe, paying agencies use Sentinel time series to check farmers' declarations field by field. ESA's **WorldCereal** produced the first global, seasonal 10 m maps of temporary crops, maize, cereals and irrigation for 2021. Its crop-extent map had a user's accuracy of 88.5 % and a producer's accuracy of 92.1 %; crop types agreed with field data at least 80 % of the time. The maps were released in 2023 as open data and open code.

**Yield forecasting** is harder. Models combine vegetation indices through the season with weather, soil and past yields. Such models work well where long yield records exist and poorly where they do not, which is often where forecasts matter most. Field patterns like those in Figure 46.7 are easy for people to read but differ greatly between regions.

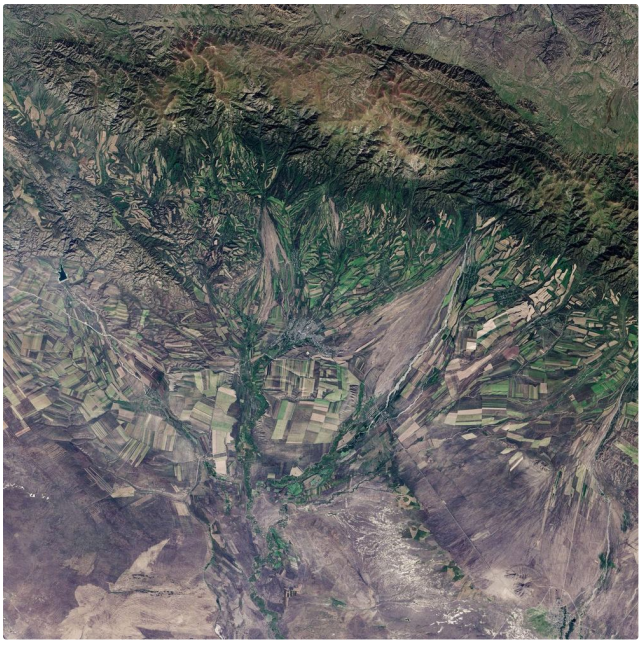


**FIGURE 46.7** Fields in eastern Kazakhstan seen by Landsat 8 on 9 September 2013. Some are harvested and bare, others still green — the kind of pattern a crop-type model learns from a season of images.
Image: NASA/GSFC.

**FIGURE 46.8** Forest loss in the Xingu basin, Brazil, from L-band radar: areas cleared before 1996 (black) and between 1996 and 2007 (red), from JERS-1 and ALOS-1 data. Radar alerts see through tropical cloud.
Image: NASA/JPL.

### 46.2.2 Forests and Deforestation Alerts

The University of Maryland's global forest-change maps processed about 654 000 Landsat scenes with decision trees in 2013. Updated every year, they are the basis of **Global Forest Watch**. Its 2025 data, released in April 2026, showed 4.3 million hectares of tropical primary forest lost. That was 36 % less than the record 6.7 million hectares of 2024, a year of extreme fires.

Annual maps are too slow for enforcement, so **alerts** were added. GLAD alerts flag new loss in each Landsat image at 30 m and later in Sentinel-2 at 10 m. The **RADD** alerts from Wageningen University use Sentinel-1 radar, which sees through the clouds that hide tropical forests for months (Figure 46.8). They cover about 50 tropical countries at 10 m, with a new radar image every 6–12 days. An alert is raised from a single new radar image. It is confirmed when the disturbance probability exceeds 97.5 % within 90 days. Global Forest Watch merges these streams into "integrated alerts" used by governments, companies checking supply chains and journalists.

### 46.2.3 Cities, Mines and Water

**Building footprints** are the base layer for population estimates, disaster planning and electrification. Microsoft has extracted about 1.4 billion footprints from aerial and satellite imagery, with regional precision of 92–97 % but recall as low as 71 % in Africa. Google's Open Buildings holds 1.8 billion detections across Africa, South and South-East Asia, Latin America and the Caribbean. Both are open and often fill gaps that community mapping has not reached.

**Mining** is a growing target. Amazon Mining Watch, from Earth Genome and partners, runs an ensemble of CNNs over Sentinel-2 to map gold mining in nine Amazon countries. Its August 2026 update reported about 99 % specificity and 90 % sensitivity, with maps updated every quarter

since 2025. **Water and wetlands** are mapped from long Landsat records and from radar, which separates open water, flooded vegetation and dry land.

## 46.3 Oceans and Maritime Awareness

At sea, the key question is "who is out there?" Ships are supposed to broadcast their identity and position through the **Automatic Identification System** (AIS). Many do not: their transponders are off, broken or not required. Satellite imagery finds these **"dark" vessels**, and machine learning makes it possible to do so for whole oceans.

> **MAPPING THE DARK FLEET (GLOBAL FISHING WATCH, 2024)**
>
> In January 2024, Paolo and colleagues at Global Fishing Watch published the first global map of industrial vessels and offshore structures in *Nature*. They processed about 2 PB of imagery from 2017–2021: 753 030 Sentinel-1 radar scenes for vessels and about 2.5 million Sentinel-2 scenes for infrastructure. A CFAR detector proposed objects. Three deep CNNs then confirmed vessels (about 97.5 % accurate), estimated their length and separated fishing from non-fishing vessels (90.5 % accurate). Detections were matched against 53 billion AIS messages.
>
> The study found 23.1 million vessel detections, about 63 300 vessels at sea at any moment. It concluded that 72–76 % of the world's industrial fishing vessels and 21–30 % of transport and energy vessels are not publicly tracked. It also counted about 28 000 offshore structures, with wind turbines more than doubling since 2017. Detection was above 70 % for 25 m vessels and above 90 % for vessels of 50 m or more. Smaller boats remain largely invisible to 10 m radar.

Before this, the **xView3** challenge (2021–22) had released close to 1 000 Sentinel-1 scenes labelled for dark-vessel detection. Models had to find vessels, estimate their length and say whether they were fishing. Commercial services now combine radar, optical, radio-frequency and AIS data. One example is Vantor's (formerly Maxar's) Maritime Sentry, which uses AIS-to-optical and radar-to-optical tipping. Fully automatic cueing between different companies' satellites is still mostly at the research stage.

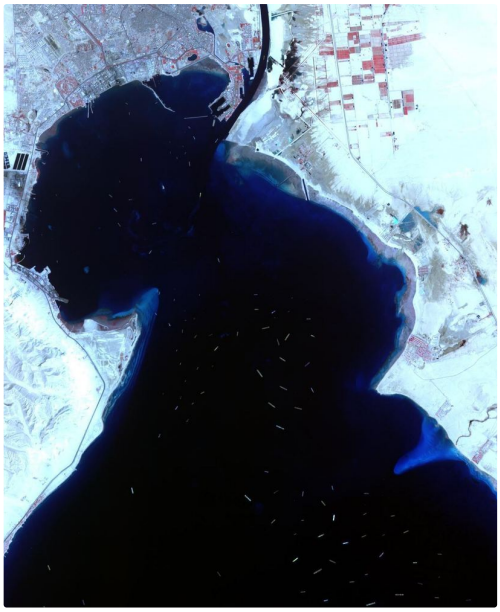


**FIGURE 46.9** More than 100 freighters waiting at the southern entrance of the Suez Canal on 4 April 2021, after the grounded Ever Given was freed, seen by ASTER on NASA's Terra satellite. Each bright dot is a ship — easy to count by eye here, but across whole oceans it is a job for detectors. Image: NASA/JPL.

Other maritime tasks follow the same pattern. **Oil spills** damp the small waves that scatter radar, so they appear as dark slicks. The European Maritime Safety Agency's CleanSeaNet service has screened radar images for spills since April 2007. It alerts national authorities within 30 minutes of the image. The difficulty is look-alikes: calm water, algae and rain cells also appear dark, so classifiers use shape, context and wind. **Sea ice** charts are still drawn largely by expert analysts from radar. ESA's AutoICE challenge (2023) showed that a U-Net combining Sentinel-1 with passive-microwave data could map ice concentration, type and floe size together (Chapter 47).

## 46.4 Disasters

In a disaster, speed matters more than perfection. The key idea is to deliver a good-enough map within hours of the first image, then improve it.

### 46.4.1 Flood Mapping

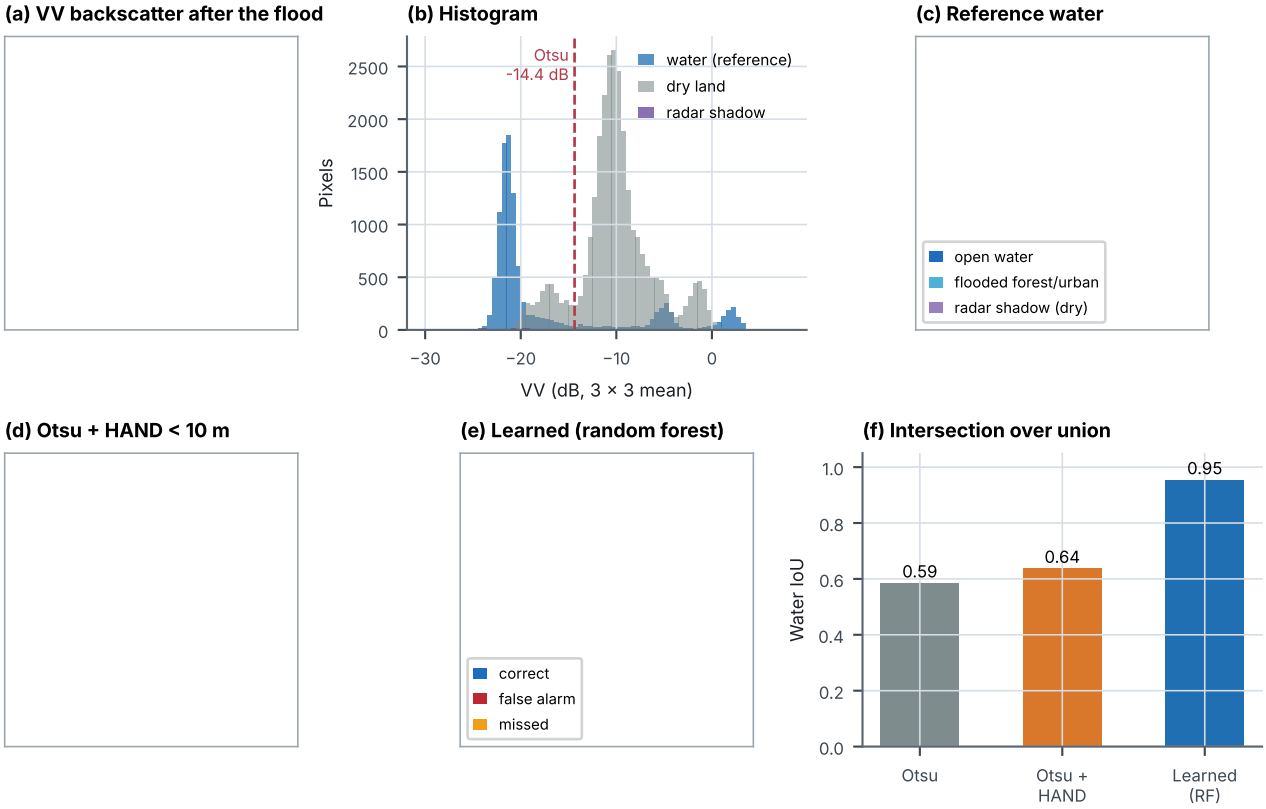
Radar is the main tool for floods because rain clouds hide the ground from optical sensors. Calm open water reflects radar away from the satellite, so it appears dark (Chapter 15). The traditional method picks a backscatter threshold, often with **Otsu's method**. It chooses the threshold $t$ that maximises the between-class variance of the histogram.

$$\sigma_B^2(t) = w_0(t)\, w_1(t)\, \left[\mu_0(t) - \mu_1(t)\right]^2$$

Here $w_0$ and $w_1$ are the fractions of pixels below and above $t$, and $\mu_0$ and $\mu_1$ their mean values. Errors come from dark land (smooth sand, tarmac, radar shadow), from flooded forests and towns that look *brighter* because of double bounce between water and trunks or walls, and from wind-roughened water.

The synthetic Sentinel-1-like scene of Figure 46.10 shows these effects. Otsu's threshold of −14.4 dB on VV backscatter (vertical polarisation sent and received) gave a water IoU of 0.59. Adding a **height above nearest drainage** (HAND) mask, which removes land more than 10 m

above the river, raised it to 0.64. It removed most radar-shadow false alarms but also some real ponds. A random forest that used both polarisations before and after the flood, plus HAND, reached 0.95. It found 1 836 of the 2 202 bright flooded-forest and flooded-urban pixels; the thresholds found almost none. Synthetic scenes flatter every method. Real scenes add wind, wet snow and errors in the labels themselves.




Synthetic 200 × 200-pixel Sentinel-1-like scene (VV and VH, before and after, about 4.4 looks) over a valley; floodwater fills land up to 2–5 m above the river (HAND, uneven), plus a few ponds; the HAND map given to the models carries ±1.5 m DEM errors. Look-alikes: smooth sand or tarmac, radar shadow on hills; flooded forest and buildings appear bright. Random forest: 150 trees, 8 features, trained on three other scenes.

**FIGURE 46.10** Flood mapping from a synthetic Sentinel-1-like scene. (a) Backscatter after the flood. (b) Histogram with Otsu's threshold; radar shadow is as dark as water. (c) Reference, including bright flooded forest and buildings. (d) Threshold plus HAND mask. (e) A random forest using both dates and polarisations. (f) Water intersection over union.

Operational systems already work this way. The Copernicus **Global Flood Monitoring** (GFM) service maps every new Sentinel-1 interferometric-wide-swath image automatically. It runs three independently developed algorithms, from LIST, DLR and TU Wien, and combines them by majority vote, with a likelihood layer for uncertainty. Since Sentinel-1A ended operations on 30 June 2026 after 12 years, GFM has used Sentinel-1C and -1D. The **Sen1Floods11** data set (2020) of 4 831 chips from 11 flood events showed a useful result: models trained on many automatically generated labels beat models trained on 446 hand-labelled chips.

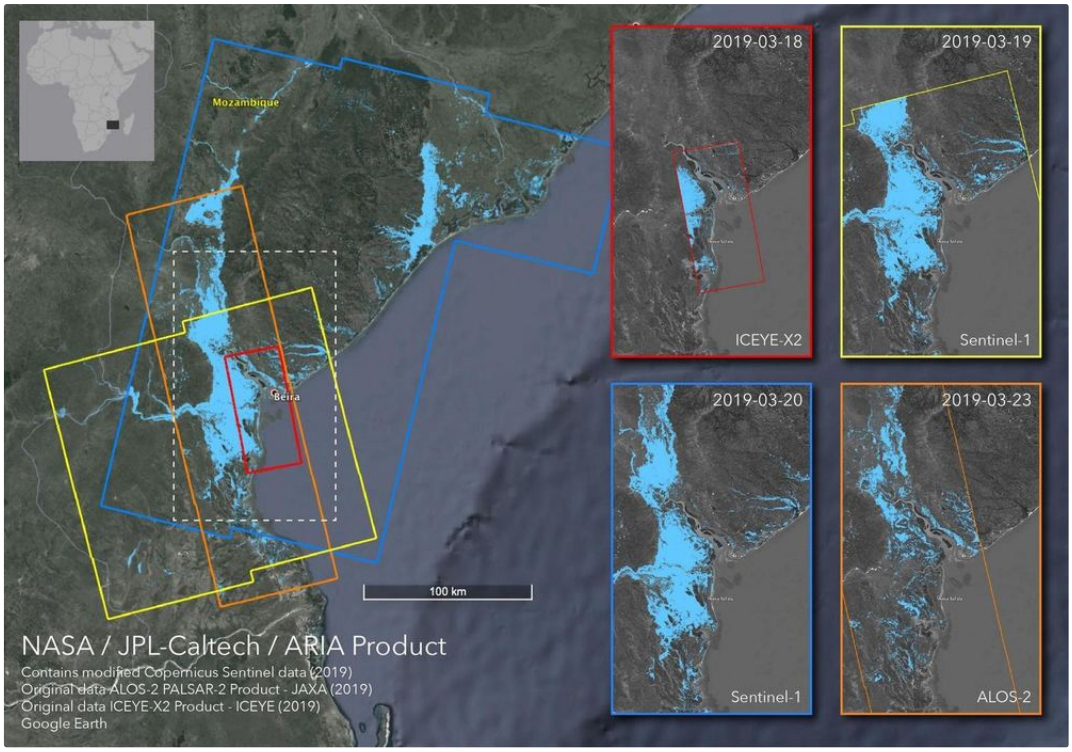



**FIGURE 46.11** Flood proxy maps of Mozambique after Cyclone Idai in March 2019, made by NASA JPL's ARIA team from ICEYE, Sentinel-1 and ALOS-2 radar images. Light blue marks likely floodwater; the panels track the flood from 18 to 23 March. Image: NASA/JPL (NASA/JPL-Caltech).

**EXAMPLE: HOW MUCH DOES ONE SENTINEL-1 PASS MAP?**

Sentinel-1 orbits at 693 km. In interferometric wide-swath mode its swath is 250 km. How much area does a 10-minute acquisition cover, and can a model keep up?

1. **Orbital speed.** $v = \sqrt{\mu/(R_E + h)} = \sqrt{398\,600/7\,064} = 7.51$ km/s.
2. **Ground speed.** $v_g = v\,R_E/(R_E + h) = 7.51 \times 6\,371/7\,064 = 6.77$ km/s.
3. **Area.** In 600 s the ground track advances 6.77 × 600 = 4 065 km. Times 250 km, that is about 1.02 million km$^2$ — roughly the area of Egypt.
4. **Pixels.** At 10 m spacing, $1.02 \times 10^{12}$ m$^2$ / 100 m$^2$ = $1.0 \times 10^{10}$ pixels per polarisation. Two polarisations at 2 bytes each are about 40 GB.
5. **Computing.** Cut into 512 × 512 tiles, that is about 38 800 tiles. At an assumed 50 billion operations per tile for a U-Net, the total is $1.9 \times 10^{15}$ operations. A graphics processor sustaining $30 \times 10^{12}$ operations a second needs about 65 s.

Inference is not the bottleneck. Downloading, calibrating and terrain-correcting the data take longer, and the satellite returns only every 6 days (12 days for each of the two satellites).

## 46.4.2 Rapid Mapping and Damage Assessment

Two coordination systems turn images into maps for responders. The **International Charter "Space and Major Disasters"**, in operation since 2000, lets disaster agencies request free imagery from member space agencies. It reached activation number 1 052 in August 2026; 2024 set a record of 85 activations. The **Copernicus Emergency Management Service** (CEMS) adds on-demand mapping. For the Myanmar earthquake of March 2025 it delivered 93 maps from 106 post-event images. Most rapid maps are still drawn or checked by analysts, with AI speeding up the steps.

**Building damage** is assessed by comparing images from before and after. Models trained on xBD grade each building on a four-level scale, from no damage to destroyed. After the February 2023 earthquakes in Türkiye, Microsoft's AI for Good Lab, Planet and the Turkish disaster agency AFAD used such a model on imagery from the first three days. They estimated that 3 849 buildings were damaged or destroyed in four cities. The World Food Programme and Google Research built the open **SKAI** tool. A UN evaluation over nine events found that it covered about seven times more area and gave first results about six times faster, in under a day. A later study comparing two companies' damage maps for the same disaster found that they disagreed in many places. Maps from different models should be treated as estimates, not facts.

Radar change detection gives a complementary view (Figure 46.12). It needs no cloud-free image and no labels, but it marks any change in the radar echo, not damage as such.

**FIGURE 46.12** Damage proxy map of eastern Puerto Rico after Hurricane Maria (September 2017), from Sentinel-1 images before and after landfall. Red and yellow pixels in the San Juan inset mark likely damage. Image: NASA/JPL (NASA/JPL-Caltech/ESA/Copernicus/Google).

**FIGURE 46.13** Thick smoke from the Grasshopper wildfire near Mount Hood, Oregon, photographed from the International Space Station on 31 July 2026. Constellations such as FireSat aim to detect fires far smaller than this one. Image: NASA/JSC.

## 46.4.3 Wildfires

Active fires are found by their heat at 3.9 µm (Chapter 14). New constellations add machine learning and speed (Figure 46.13). The German company **OroraTech** had 20 thermal satellites of its FOREST series in orbit by October 2026. Its satellites carry a commercial AI module that detects fires on board and sends compact alerts rather than images. The nonprofit Earth Fire Alliance's **FireSat**, built by Muon Space with Google, launched a prototype in March 2025. Its first three operational satellites followed in July 2026, on SpaceX's Transporter-17. They aim to detect fires as small as 5 m × 5 m, with at least twice-daily revisits of fire-prone regions. The stated goal is hourly global revisit by 2029. In both systems, learning helps separate real fires from hot roofs, sun glint and volcanoes.

These services are summarised in Table 46.3.

**TABLE 46.3** AI in disaster services.

| Service | Data | What AI does | Status (Oct 2026) |
|---|---|---|---|
| Copernicus GFM | Every Sentinel-1 IW image | Three-algorithm ensemble flood maps with likelihood | Operational, global |
| International Charter | Member agencies' satellites | Mostly analyst mapping; AI tools in some workflows | Operational; > 1 050 activations |

| Service | Data | What AI does | Status (Oct 2026) |
|---|---|---|---|
| Copernicus EMS rapid mapping | Commercial and Copernicus images | Analyst maps, partly automated | Operational, on demand |
| xBD-trained damage models (e.g. Microsoft, SKAI) | Sub-metre optical, before and after | Building-by-building damage grades | Used in many disasters; accuracy varies |
| FireSat | Six-channel infrared constellation | Fire detection from small hot spots | First 3 operational satellites launched 2026 |
| OroraTech FOREST | Thermal smallsats | Fire detection on board | Commercial service; 20 satellites |

## 46.5 Greenhouse Gases and Human Development

The key idea of this section is that machine learning now finds individual sources of methane from space, and estimates how much they emit. The same tools of image regression also map wealth and poverty.

### 46.5.1 Finding Methane Plumes

Methane traps about 80 times more heat than carbon dioxide over 20 years, but lasts only about a decade in the air. Stopping large leaks — "super-emitters" — is one of the quickest ways to slow warming. Methane absorbs sunlight near 1.65 and 2.3 µm. Imaging spectrometers detect it with a **matched filter**, which tests each pixel spectrum $\boldsymbol{x}$ against the expected absorption signature $\boldsymbol{t}$:

$$\hat{\alpha} = \frac{(\boldsymbol{x} - \boldsymbol{\mu})^{\top} \boldsymbol{\Sigma}^{-1} \boldsymbol{t}}{\boldsymbol{t}^{\top} \boldsymbol{\Sigma}^{-1} \boldsymbol{t}},$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are the mean and covariance of the background spectra. The result $\hat{\alpha}$ is a map of methane enhancement. Bright surfaces, roads and some minerals produce false plumes, and checking every map by eye is slow.

Machine learning helps in two ways. CNNs separate real plumes from artefacts, and regression models estimate emission rates. Schuit and colleagues (2023) combined a CNN with a second classifier on TROPOMI data from Sentinel-5P. They found 2 974 plumes in 2021, with a mean source rate of about 44 t/h. In September 2026 Google and NASA JPL published MAPL-EMIT, a model for the EMIT spectrometer on the International Space Station. It was trained on 3.6 million physics-simulated plumes. It found about 50 % more plumes than human experts and more than 23 000 additional plumes worldwide.

**TABLE 46.4** Satellites used to find methane sources.

| Mission | Pixel size | Role | Status (Oct 2026) |
|---|---|---|---|
| TROPOMI (Sentinel-5P) | About 5.5 × 7 km | Daily global mapping; finds the largest plumes | Operating |
| GHGSat constellation | About 25 m | Commercial point-source monitoring | Operating; more than a dozen satellites |
| EMIT (ISS) | 60 m | Imaging spectrometer; mineral dust and methane | Operating |
| Tanager-1 (Carbon Mapper, Planet) | About 30 m | Methane and $CO_2$ point sources | Operating since 2024 |
| Sentinel-2, Landsat | 20–30 m | Multispectral; detect only very large plumes | Operating |

| Mission | Pixel size | Role | Status (Oct 2026) |
|---|---|---|---|
| MethaneSAT (EDF) | About 100 × 400 m | Area-wide emissions | Lost on 20 June 2025 |

DATA BOX — METHANE ALERTS IN NUMBERS

- **UNEP's Methane Alert and Response System (MARS)**, launched in November 2022, combines satellite data with AI-assisted analysis and notifies governments and operators. By July 2026 it had sent more than 5 000 alerts to 33 countries.
- **Response.** Only 13 % of alerts receive a reply from the operator or government, up from about 1 % in 2024. More than 40 verified mitigations have been recorded in 10 countries.
- **Tanager-1's first year** (September 2024 – September 2025): about 5 392 published methane plumes and 1 234 $CO_2$ plumes. Of the methane plumes, 2 661 came from oil and gas, 1 327 from waste and 1 211 from coal.
- **Loss of MethaneSAT.** Contact with MethaneSAT was lost on 20 June 2025, about 15 months after launch. It had been designed to map emissions across whole oil and gas basins.

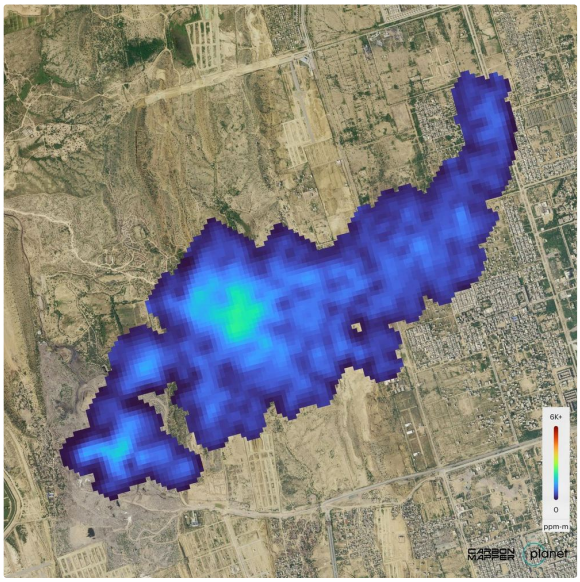



**FIGURE 46.14** The first methane plume seen by Tanager-1, on 19 September 2024 over Karachi, Pakistan: a plume about 4 km long from a landfill. Carbon Mapper's preliminary estimate was about 1 200 kg of methane an hour. Image: NASA/JPL (Carbon Mapper/Planet Labs PBC).

## 46.5.2 Estimating the Emission Rate

A plume shows how much methane is in the air, not how fast it is being released. The **integrated mass enhancement** (IME) method links the two. The IME is the total extra methane mass in the plume mask:

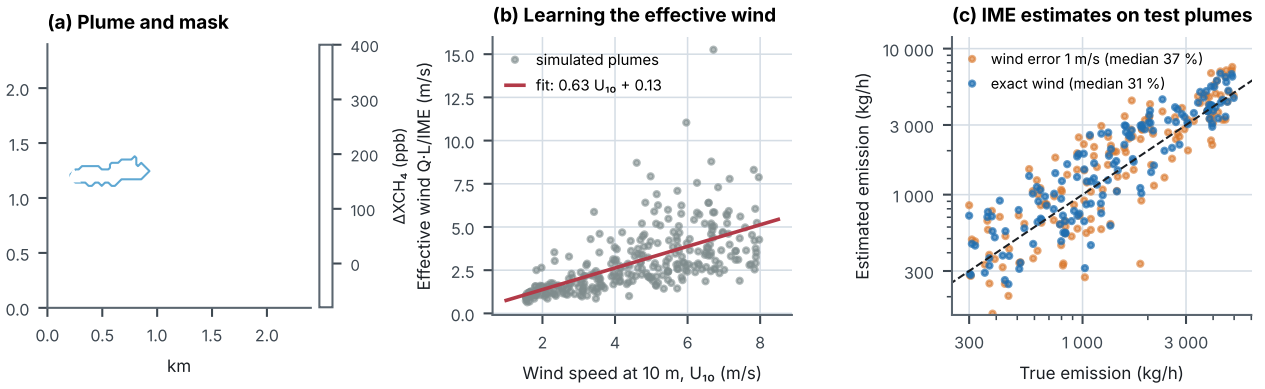
$$\mathrm{IME} = \sum_{i \in \mathrm{mask}} \Delta\Omega_i \, A_i ,$$

where $\Delta\Omega_i$ is the column enhancement in kg/m² and $A_i$ the pixel area. If the plume has length scale $L = \sqrt{A_{\mathrm{mask}}}$ and is carried by an effective wind $U_{\mathrm{eff}}$, the methane is renewed every $L/U_{\mathrm{eff}}$ seconds. The emission rate is then

$$Q = \frac{U_{\mathrm{eff}} \, \mathrm{IME}}{L} .$$

The effective wind is not the wind at 10 m, $U_{10}$. It is **learned** from simulated plumes, usually as a straight line $U_{\text{eff}} = \alpha U_{10} + \beta$. For Sentinel-2, published fits use $\alpha = 0.33$ and $\beta = 0.45$ m/s.

In Figure 46.15 this calibration is repeated with a simple plume model at 30 m pixels. A fit on 300 simulated plumes gave $U_{\text{eff}} = 0.63\,U_{10} + 0.13$ m/s; the coefficients depend on the sensor, the mask rule and the plume model. On 146 test plumes the median error was 31 % with the true wind, and 37 % when the wind had a 1 m/s error. A single estimate can be wrong by a factor of two, so repeated passes matter more than any one number.




Synthetic 30 m imagery (2.4 km × 2.4 km): puffy Gaussian plumes, 300–5 000 kg/h, winds 1.5–8 m/s, 50 ppb pixel noise plus correlated surface artefacts. Mask: smoothed image above 2σ, connected to the source. U_eff fitted on 300 plumes, tested on 146. Q = U_eff · IME / L, L = √(mask area).

**FIGURE 46.15** The IME method on synthetic 30 m methane images. (a) A 1 500 kg/h plume with its detection mask. (b) Effective wind needed to recover the true emission, and a straight-line fit. (c) Estimated against true emission for test plumes, with exact wind and with a 1 m/s wind error.

**EXAMPLE: A METHANE EMISSION RATE FROM A PLUME**

A Sentinel-2 image shows a plume from a gas well. The plume mask holds 1 250 pixels of 20 m. The mean column enhancement in the mask is 100 ppb (parts per billion of the total air column). The 10 m wind from a weather model is 3.5 m/s. Assume a dry air column of 101 325 Pa / 9.81 m/s² = 10 329 kg/m².

1. **Unit conversion.** The air column holds 10 329 / 0.028 97 = 356 500 mol/m². A 1 ppb enhancement is 356 500 × $10^{-9}$ × 0.016 04 kg/mol = 5.72 × $10^{-6}$ kg/m².
2. **Mask area and length.** A = 1 250 × 400 m² = 0.5 km², so L = √(5 × $10^5$) = 707 m.
3. **IME.** 100 × 5.72 × $10^{-6}$ kg/m² × 5 × $10^5$ m² = 286 kg.
4. **Effective wind.** $U_{\text{eff}}$ = 0.33 × 3.5 + 0.45 = 1.61 m/s.
5. **Emission rate.** Q = 1.61 × 286 / 707 = 0.649 kg/s ≈ 2 340 kg/h.
6. **Wind uncertainty.** With 2.5 or 4.5 m/s instead, Q is 1 860 or 2 820 kg/h, about ±20 %.

Sustained for a year, 2.3 t/h is about 20 500 t of methane. With a 20-year warming potential of about 80, that is about 1.6 million t of $CO_2$-equivalent — from one source.

## 46.5.3 Mapping Poverty

Surveys of household income are expensive and rare in many low-income countries. In 2016, Jean and colleagues showed in *Science* that a CNN could predict village wealth in five African countries from daytime satellite images. They trained it first to predict night-time lights, a cheap proxy for economic activity, and explained up to 75 % of the variation in surveyed wealth. In 2022 Chi and colleagues mapped relative wealth for all 135 low- and middle-income countries at 2.4 km resolution. They used satellite imagery with phone-network, connectivity and terrain

data. Trained on surveys from 56 countries, the estimates explained 56–70 % of the variation in held-out survey wealth. In 2020 the Government of Togo used such maps, with phone data, to direct emergency cash transfers. The maps are estimates with known errors. Used for targeting, they can wrongly exclude poor households, so they should be checked against local knowledge.

## 46.6 AI on Board and Smart Tasking

The key idea is to decide in orbit what is worth collecting and sending down, because downlink and imaging time are the scarce resources (Chapter 45). Two uses stand out: **screening** images on board, and **retasking** the satellite before it reaches a target.

### 46.6.1 Demonstrations so far

ESA's **Φsat-1** (2020) showed that a deep network on a vision chip drawing only a watt or two could discard cloudy images in orbit. Its successor, **Φsat-2** (2024), carried several uploadable apps, including vessel and wildfire detection (Chapter 45). The main EO demonstrations are listed in Table 46.5. Most are still demonstrations; few run as part of a paying service.

**TABLE 46.5** Onboard AI for Earth observation.

| Mission or system | Year | What the AI does on board | Status (Oct 2026) |
|---|---|---|---|
| Φsat-1 (ESA) | 2020 | Discards images more than 70 % cloudy | Completed demonstration |
| Satellogic NewSat with Palantir Edge AI | 2022 | Runs third-party detection models | Demonstration |
| Φsat-2 (ESA, Open Cosmos) | 2024 | Cloud, vessel, fire and anomaly apps; map generation; compression | Demonstration; apps updated in orbit |
| CogniSAT-6 with JPL dynamic targeting | 2025 | Looks ahead, skips cloudy targets | First flight test, July 2025 |
| OroraTech FOREST | 2022– | Fire detection and alert compression | Commercial service |
| Planet Pelican-4 | 2026 | Aircraft detection on raw images (about 80 % accurate) | Demonstration, March 2026 |
| Compressed Prithvi model on Kanyini and IMAGIN-e | 2026 | Geospatial foundation model run in orbit | Research demonstration |

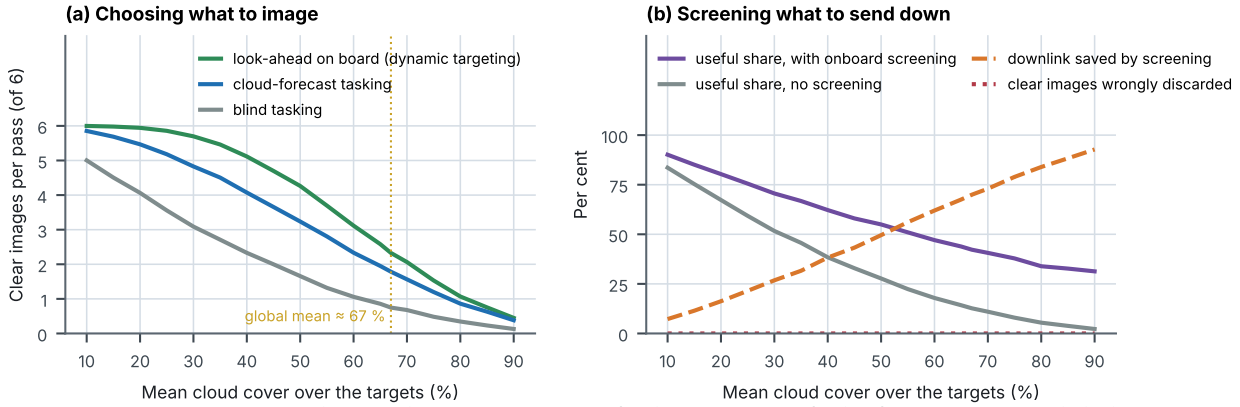
**DYNAMIC TARGETING ON COGNISAT-6 (2025)**

CogniSAT-6 is a 6U cubesat designed, built and operated by Open Cosmos. Its payload, from Ubotica, pairs a commercial AI processor with a hyperspectral camera from Simera Sense. It was launched in March 2024. In mid-July 2025, NASA JPL's **Dynamic Targeting** software flew on it for the first time. The satellite tilted 40–50° forward to image the ground about 500 km ahead. It analysed that image on board, judged whether the target was clear, and decided to image it or skip it — all in under 90 seconds and without human help. JPL's next steps are to hunt short-lived events such as storms, fires and eruptions, and to let a leading satellite pass targets to the ones behind it.

### 46.6.2 What Smart Tasking Is Worth

A satellite passing over a region can usually image only some of the possible targets. A Monte Carlo model in Figure 46.16 estimates the value of choosing well. A pass has 20 candidate targets and room for six images. An image is useful if it is less than 20 % cloudy. At a mean cloud cover of 67 %, blind tasking yields 0.75 clear images per pass. Tasking with a cloud forecast yields 1.78. A look-ahead camera on board yields 2.33, three times as many. At 30 % cloud the gains are smaller: 3.1, 4.8 and 5.7.

Screening after imaging is a different trade. It does not create clear images, but it stops cloudy ones from using the link. At 67 % cloud, keeping only images estimated under 50 % cloudy freed 70 % of the downlink. It raised the useful share of what was sent from 13 % to 42 %, and almost no clear image was lost.




Monte Carlo, 4 000 passes per point: 20 candidate targets per pass, room for 6 images; scene cloud fractions follow a U-shaped beta distribution, correlated along track (ρ = 0.6). An image is useful if under 20 % cloudy. Forecast skill: correlation 0.7 with the truth; look-ahead camera: 0.97. Screening keeps images estimated under 50 % cloudy (error 0.08).

**FIGURE 46.16** Clouds and onboard intelligence (Monte Carlo). (a) Clear images per pass from blind tasking, forecast-based tasking and an onboard look-ahead camera, against mean cloud cover. (b) Effect of onboard screening on the share of useful images sent down and on the downlink used.

**Tip-and-cue** extends the idea across a constellation. A wide-area sensor — a radar, a radio-frequency receiver or AIS — detects something, and a sharper sensor is tasked to look. A dark vessel found by radar can be imaged by an optical satellite on its next pass. Such chains are automated within single companies today. Between companies, and from satellite to satellite without a ground loop, they are mostly at the research stage.

## 46.7 The EO Analytics Business

Most commercial value lies not in pixels but in answers. Novaspace valued the EO data and services market at about $5 billion in 2023 and expected it to pass $8 billion by 2033, with services rising from $3.1 billion to $4.9 billion. Defence is one of the main drivers of growth. Planet, one of the largest commercial imaging operators, reported revenue of $307.7 million for the year to January 2026, up 26 %, with a backlog of over $900 million.

Three changes define the business in 2026 (Chapter 52). First, operators sell **subscriptions and analytics** rather than single images: alerts for new construction, vessel lists, crop statistics. Second, data are sold as **AI-ready products**: calibrated, co-registered time series with cloud masks and standard tiling. EarthDaily, for example, launched six satellites in May 2026 for a

constellation designed for broad-area change detection, with 22 spectral bands. Third, **foundation models** are moving into practice. Google's AlphaEarth embeddings summarise each 10 m pixel and year since 2017 as 64 numbers in Earth Engine. A small classifier on these embeddings can map a new class from relatively few labels. NASA and IBM's Prithvi models are fine-tuned for floods, burn scars and crops (Chapter 44).

**EXAMPLE: IMAGES PER DAY VERSUS ANALYSTS**

A constellation collects about 200 million km$^2$ a day in scenes of about 32.5 km × 19.6 km, as PlanetScope does. Suppose an analyst needs 5 minutes to check one scene for new buildings and works 7 hours a day.

1. **Scenes per day.** 32.5 × 19.6 = 637 km$^2$ per scene; 200 × 10$^6$ / 637 ≈ 314 000 scenes a day.
2. **Analyst capacity.** 7 × 60 / 5 = 84 scenes per analyst per day.
3. **Analysts needed.** 314 000 / 84 ≈ 3 740 analysts, every day, for one constellation and one question.
4. **With AI triage.** A change detector flags 2 % of scenes (6 280), and analysts also check a random 1 % of the rest (about 3 080) to watch for misses. That is about 9 360 scenes, or about 111 analysts.
5. **Pixels.** At 3 m, each scene holds about 71 million pixels per band, and the day about 2.2 × 10$^{13}$.

The model does not replace the analysts. It points their attention and lets them spend time on what matters. Its miss rate, measured on the random checks, sets how far its output can be trusted.

## 46.8 Pitfalls and Good Practice

EO models fail in predictable ways. The key idea is that an accuracy figure means little unless you know where, when and how it was measured (Table 46.6).

### 46.8.1 Domain Shift

A model trained in one region, season or sensor often fails in another. Soils, crops, building styles, sun angles and atmospheric haze all differ. In Figure 46.17 (a), a random forest trained on one synthetic region lost accuracy as the test region became drier and hazier: from 77.8 % to 53.2 % at the largest shift. Adding random spectral changes to the training data helped only slightly and cost accuracy at home. Adding 240 labelled pixels from the new region kept 67.5 %. A few local labels are usually worth more than clever tricks.

Shifts also come from the data chain. A change in processing, such as a new radiometric baseline, alters every pixel slightly (Chapter 45). Models should be monitored after deployment and checked whenever the data chain changes.

### 46.8.2 Labels, Imbalance and Validation

**Label noise** is common. Reference maps are out of date, photo-interpreters disagree, and labels from other models carry their errors along. **Class imbalance** is the rule: forest loss, ships and methane plumes are rare, so a model can look accurate while missing most of them (Chapter 43).

**Spatial autocorrelation** inflates accuracy. If training and test pixels come from the same fields, the test measures memory, not skill. Chapter 43 shows how random splits overstate accuracy compared with splits by region. Hold out whole regions, scenes or events.

**Geographic bias** follows from where labels exist. Many large labelled sets were built mainly from North America and Europe, so models often perform worst where data are needed most. Report accuracy by region, not just globally.

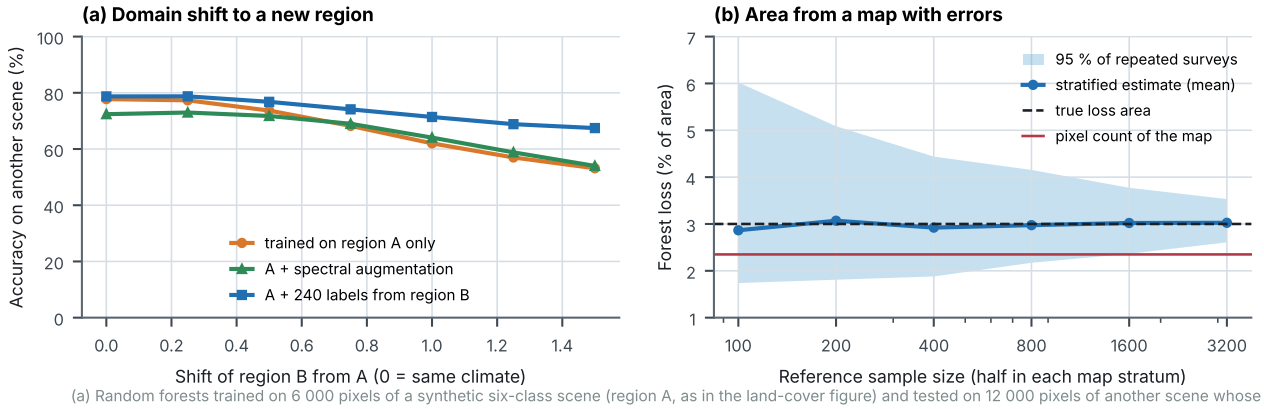
### 46.8.3 Getting the Area Right

A map is often used to report an **area**: hectares of forest lost, or square kilometres flooded. Counting mapped pixels is biased whenever omission and commission errors do not cancel. Good practice, set out by Olofsson and colleagues (2014), uses a **stratified random sample** of reference points, checked independently, inside each map class. The estimated proportion of true class $k$ is

$$\hat{p}_k = \sum_h W_h \frac{n_{hk}}{n_h}, \qquad \widehat{\mathrm{SE}}(\hat{p}_k) = \sqrt{\sum_h W_h^2 \frac{\hat{p}_{hk}(1-\hat{p}_{hk})}{n_h - 1}},$$

where $W_h$ is the mapped share of stratum $h$, $n_h$ the number of samples in it, $n_{hk}$ those found to be class $k$, and $\hat{p}_{hk} = n_{hk}/n_h$.

In Figure 46.17 (b), a map of a million pixels finds 65 % of 30 000 true loss pixels and adds 4 000 false ones. Counting gives 2.35 % loss, while the truth is 3.00 %. The stratified estimate is unbiased. With 3 200 reference points, 95 % of repeated surveys fell between 2.61 % and 3.53 %. With only 100 points, the reported confidence interval contained the truth in only 37 % of surveys. The few omission errors hidden in the large “no loss” stratum were usually missed by the sample. Small samples give confident-looking but wrong intervals.




(a) Random forests trained on 6 000 pixels of a synthetic six-class scene (region A, as in the land-cover figure) and tested on 12 000 pixels of another scene whose soils, vegetation and haze shift towards a drier climate (shift 1: soils up to 35 % brighter, plant NIR 25 % lower, more haze). (b) A $10^6$-pixel map finds 65 % of 30 000 loss pixels and adds 4 000 false ones; 400 simulated surveys per sample size, estimator of Olofsson et al. (2014).

**FIGURE 46.17** Two pitfalls, computed. (a) Accuracy of a land-cover random forest as the test region drifts away from the training region, with two remedies. (b) Forest-loss area from pixel counting and from stratified sampling (Olofsson et al. 2014), against reference sample size.

**TABLE 46.6** Common pitfalls of AI for Earth observation and how to avoid them.

| Pitfall | What goes wrong | Remedy |
|---|---|---|
| Domain shift | Accuracy falls in new regions, seasons or sensors | Train on varied data; add local labels; monitor drift |
| Label noise | Model learns the errors of its labels | Second interpreter on a sample; robust losses; clean the test set |
| Class imbalance | Rare events missed while accuracy looks high | Report recall, precision and PR curves; set thresholds for real prevalence |

| Pitfall | What goes wrong | Remedy |
|---|---|---|
| Spatial autocorrelation | Test scores overstated | Split by region, scene or event; use buffers |
| Geographic bias | Worst performance where data are scarcest | Collect labels in under-served regions; report by region |
| Biased area estimates | Pixel counts misreport change | Stratified reference sample; area-adjusted estimates with confidence intervals |
| Hallucinated detail | Generated images mistaken for data | Label synthetic outputs; never measure from them |

### 46.8.4 Explainability, Ethics and Privacy

**Explanations** help to catch shortcuts, such as a ship detector that keys on harbour walls rather than ships (Chapter 44). For maps, the most useful explanation is often simple: show the input images beside the output, with a per-pixel confidence.

**Privacy** matters more as resolution improves. At 30 cm, people cannot be recognised, but vehicles, crowds and the routines of a household can be. Combining imagery with other data can identify individuals. Licensing rules limit some collection; for example, the US rule on imagery of Israel was relaxed from 2 m to 0.4 m in 2020. Other ethical questions concern who controls alerts about people's land, and whether maps are used to help or to punish. Assurance and governance are treated in Chapter 53.

#### PRACTICE: BUILDING A FLOOD-MAPPING MODEL FROM SENTINEL-1

1. **Define the product.** Example: water extent at 20 m within 6 hours of each pass, scored by water-class IoU against hand-labelled chips, with a separate score for flooded vegetation.
2. **Get the data.** Use Sentinel-1 Ground Range Detected (GRD) images in VV and VH. Calibrate them to the backscatter coefficient $\sigma^0$ and terrain-correct them with the Copernicus digital elevation model (DEM). Add a pre-flood reference image from the same orbit, a permanent-water map and a HAND layer.
3. **Get labels.** Start from Sen1Floods11 and similar open sets. Add hand labels from your region, including urban and forest floods, and have a second person relabel 10 % to measure label noise.
4. **Set a baseline.** Run Otsu's threshold on VV (in dB) with the HAND and permanent-water masks. Record its IoU; every model must beat it.
5. **Train.** Fit a random forest on per-pixel features (VV, VH, their changes, HAND), then a small U-Net on 256 × 256 tiles. Use flips and rotations for augmentation, never mixing tiles from one event across splits.
6. **Validate by event.** Hold out whole floods from other continents and seasons. Report IoU, precision and recall per event, and for open water and flooded vegetation separately.
7. **Check failure cases.** Inspect wind-roughened lakes, wet snow, sand, radar shadow and dense cities, and add labels where errors cluster.
8. **Deploy and monitor.** Run on each new image automatically, publish a confidence layer, and compare with GFM and analyst maps after every major event. Retrain when the sensor or processing changes.

### KEY TAKEAWAYS

- EO analysis reduces to a few tasks — segmentation, detection, change, time series and retrieval — on images with many bands, a time axis and a location. Neighbourhood and time information often matter more than the choice of network.
- Simple baselines stay strong. Random forests, NDVI differences, Lee filters, CFAR and Otsu thresholds should be beaten, by a measured margin, before a deep model is adopted.
- Operational AI products now map land cover, buildings, crops and forest loss globally, track dark vessels, map floods from every Sentinel-1 image and find methane plumes from orbit.
- Emission rates, flood areas and loss areas are estimates with real uncertainty: the IME method's single-plume errors are often 30–40 %, and pixel counts of maps are biased.
- Onboard AI is moving from demonstration to service. Look-ahead tasking can triple the clear images collected in cloudy regions, and screening frees much of the downlink.
- Most failures come from the data: domain shift, label noise, imbalance, spatially correlated validation and geographic bias. Split by region, sample properly and report uncertainty.

### REVIEW QUESTIONS

1. Name the task type, a suitable model and a metric for: counting aircraft at airports; mapping burned area; classifying crop types from a season of images; estimating forest biomass.
2. In Figure 46.2, why did a random forest given 5 × 5 neighbourhoods gain almost nine points over the same forest on single pixels? Which classes gained most, and why?
3. Explain why an NDVI-difference threshold flags harvested fields as deforestation. What extra information did the learned detector of Figure 46.3 use?
4. Write the temporal-attention weights for a time series in which two of five dates are cloudy. Why does attention not need gap filling?
5. Derive the CFAR multiplier $\alpha$ for exponential clutter and $P_{fa} = 10^{-6}$. Why do azimuth ghosts and breaking waves still cause false alarms?
6. Why can the equivalent number of looks reward a bad speckle filter? Which other measures would you report?
7. A flooded forest appears brighter in radar than the same forest when dry. Explain why, and how a flood-mapping model can still classify it as water.
8. A methane plume has an IME of 500 kg over a mask of 1 km$^2$. The 10 m wind is 5 m/s. Using $U_{\text{eff}} = 0.33\,U_{10} + 0.45$ m/s, estimate the emission rate in kg/h, and its range if the wind is uncertain by ±1.5 m/s.
9. Using Figure 46.16, estimate the number of clear images per pass at 55 % mean cloud cover for each tasking strategy. When would onboard screening be a poor choice?
10. A forest-loss map shows 2 000 km$^2$ of loss. A stratified sample shows that 20 % of mapped loss is false and that 1 % of the mapped "no loss" area of 98 000 km$^2$ is real loss. Estimate the true loss area and explain why pixel counting failed.

CHAPTER

# 47 AI for Weather, Climate and Space Weather

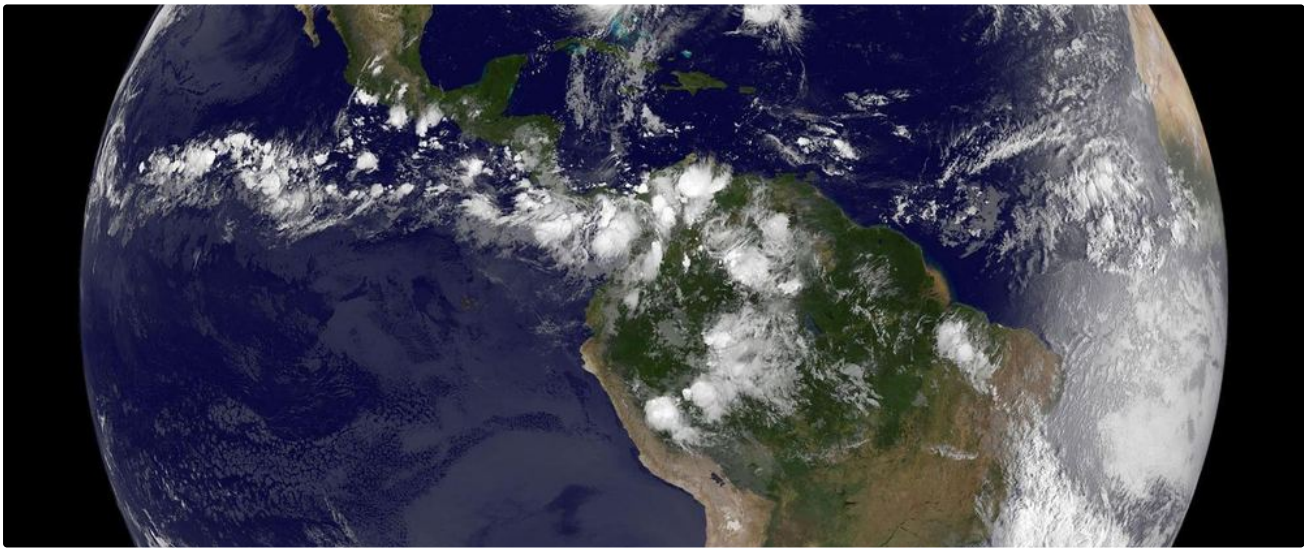


Image: NASA/GSFC

*Between 2022 and 2026 machine learning changed weather forecasting faster than any idea since the first computer forecasts. This chapter explains how. It recalls how physics models and data assimilation turn satellite data into forecasts, then follows the learned models from the first research papers to operational service at ECMWF, NOAA and other centres. It shows how they are built and trained, how their skill is measured fairly, and where they still fall short. It covers machine learning inside the observing chain, nowcasting, climate emulators, downscaling and digital twins. It ends with space weather: flares, coronal mass ejections, geomagnetic storms and the slow road to operations. Small models trained in Python make each idea concrete.*

IN THIS CHAPTER

47.1 From Physics to Learning
47.2 The Machine-Learning Weather Revolution, 2022–26
47.3 How Learned Weather Models Work
47.4 Measuring Skill Fairly
47.5 Machine Learning in the Observing Chain
47.6 Climate: Emulators, Downscaling and Digital Twins
47.7 Space Weather

Weather forecasting is one of the largest users of satellite data. It is also where machine learning has changed operational practice fastest. In 2022 a learned weather model was a research curiosity. By October 2026 learned models were running every day at the European Centre for Medium-Range Weather Forecasts (ECMWF), the US National Oceanic and Atmospheric Administration (NOAA), Germany's weather service and China's. A learned model used at the US National Hurricane Center slightly beat the official track forecasts of 2025 at some lead times.

These models did not appear from nowhere. They learned from decades of **reanalysis**, the best reconstruction of past weather, made by physics models that absorbed billions of satellite observations. They still start from analyses made the same way. The hero image above shows Hurricane Matthew nearing Florida on 7 October 2016, seen by NOAA's GOES-East satellite. Images like this one are the raw material of every learned forecast.

This chapter builds on Chapter 16 (weather satellites and forecasting), Chapters 22 and 23 (the space environment) and Chapters 43 and 44 (learning methods). Data and computing are covered in Chapter 45, and AI for Earth observation in Chapter 46.

## 47.1 From Physics to Learning

The key idea of this section is that a forecast needs two things: a good picture of the atmosphere now, and a good rule for how it changes. For 70 years both came from physics. Machine learning first replaced the rule, and is now starting on the picture.

### 47.1.1 The Forecasting Chain

A modern weather service is a pipeline (Figure 47.1). Observations come from satellites, weather balloons (radiosondes), aircraft, ships, buoys, radars and ground stations. They are checked, corrected for instrument biases and thinned. **Data assimilation** then combines them with a short-range forecast, the **background**, to make the **analysis**: the best estimate of the whole atmosphere at one moment. A **numerical weather prediction (NWP)** model then steps the analysis forward in time by solving the equations of fluid flow, heat and moisture. Finally, the output is corrected statistically and turned into products and warnings.

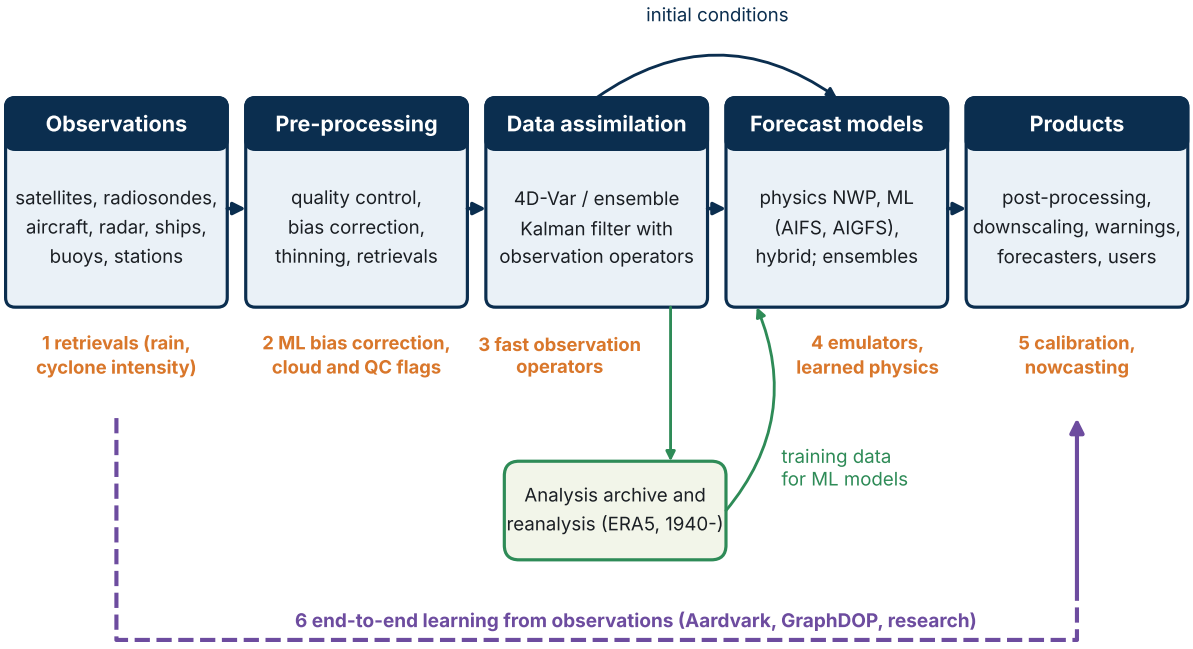



**FIGURE 47.1** The weather forecasting chain and the places where machine learning enters it. Learned models trained on reanalysis now run alongside physics models (4); end-to-end systems that learn directly from observations (6) are still research.

Variational data assimilation finds the atmospheric state $\mathbf{x}$ that best fits both the background $\mathbf{x}_b$ and the observations $\mathbf{y}$. It does this by minimising a cost function:

$$J(\mathbf{x}) = \tfrac{1}{2}(\mathbf{x}-\mathbf{x}_b)^{\mathsf{T}}\mathbf{B}^{-1}(\mathbf{x}-\mathbf{x}_b) + \tfrac{1}{2}\big(\mathbf{y}-H(\mathbf{x})\big)^{\mathsf{T}}\mathbf{R}^{-1}\big(\mathbf{y}-H(\mathbf{x})\big).$$

Here $\mathbf{B}$ and $\mathbf{R}$ are the error covariances of the background and of the observations, and $H$ is the **observation operator**. For a satellite sounder, $H$ is a radiative-transfer model that computes the radiances the instrument would see through the model atmosphere (Chapter 16). The four-dimensional version, **4D-Var**, fits observations spread over a 6–12-hour window. Every term in this equation is a place where learning can help, as the section on the observing chain shows.

### 47.1.2 Reanalysis: the Training Set

A **reanalysis** runs one fixed assimilation system over decades of archived observations. Because the system never changes, the record is consistent in time. ECMWF's **ERA5**, produced for the European Union's Copernicus Climate Change Service, covers 1940 to the present. It gives hourly fields on a 0.25° grid (about 31 km) with 137 levels from the surface to about 80 km, and many products on 37 pressure levels. Its successor, ERA6, is now in production at about 14 km. Its first decades are due for release towards the end of 2027.

Almost every learned global model has been trained on ERA5. A typical training set holds 1979–2017 or 1979–2018 at 6-hour intervals: about 57 000 global states, each with about a million grid points and tens to hundreds of variables. Later years are kept for testing. A learned model can only be as good as the reanalysis it imitates: it inherits ERA5's errors, its smoothing of small scales and its weak points, such as rainfall.

### 47.1.3 Chaos Sets the Limit

The atmosphere is **chaotic**: small errors grow until the forecast is no better than a guess from climate statistics. Errors in the large-scale flow double in about one to two days, so medium-range forecasts lose most of their skill after 10–15 days. A better model or analysis can push that limit outwards, but cannot remove it.

## 47.2 The Machine-Learning Weather Revolution, 2022–26

The key idea of this section is that within three years learned models went from first papers to running beside physics models in operations. They matched or beat the best physics models on most standard scores, at a tiny fraction of the computing cost.

### 47.2.1 Landmark Models

Table 47.1 lists the models that set the pace. **FourCastNet** (NVIDIA and partners, February 2022) produced a week-long global forecast at 0.25° in under two seconds. It matched ECMWF's physics model at short lead times for large-scale variables, but its skill fell behind at longer range. Still, it showed what was possible. **Pangu-Weather** (Huawei, published in *Nature* in July 2023) was the first learned model reported to beat ECMWF's deterministic forecast on its main scores. It used a transformer that treats pressure levels and surface fields as one three-dimensional volume.

**GraphCast** (Google DeepMind, *Science*, November 2023) was more accurate than ECMWF's high-resolution forecast (HRES) on 90 % of 1 380 verification targets. It made a 10-day forecast in under one minute on a single Google TPU v4 chip.

**TABLE 47.1** Landmark machine-learning global weather models. Ranges and grids are those of the first published versions.

| Model | Developer, year | Architecture | Grid and step | Range | Status, October 2026 |
|---|---|---|---|---|---|
| FourCastNet | NVIDIA and partners, 2022 | Fourier neural operator | 0.25°, 6 h | About 1 week | Research; later versions in NVIDIA Earth-2 |
| Pangu-Weather | Huawei, 2023 | 3D transformer | 0.25°, 13 levels | 7 days and more | Research; run experimentally at ECMWF until May 2026 |
| GraphCast | Google DeepMind, 2023 | Graph network on a multi-mesh | 0.25°, 37 levels, 6 h | 10 days | Open; basis of NOAA's AIGFS |
| FengWu | Shanghai AI Laboratory and partners, 2023 | Multi-modal transformer | 0.25°, 37 levels | Useful to 10.75 days | Research |
| FuXi | Fudan University, 2023 | Cascade of three transformers | 0.25°, 6 h | 15 days | Research |
| NeuralGCM | Google, 2024 | Physics core plus learned physics | 2.8°–0.7° | Days to decades | Research; open |
| GenCast | Google DeepMind, 2024 | Diffusion ensemble | 0.25°, 12 h | 15 days, 50 members | Research; followed by WeatherNext 2 (2025) |
| Aurora | Microsoft, 2025 | 1.3-billion-parameter foundation model | 0.25° and 0.1° | 10 days; air quality, waves | Open; run experimentally at ECMWF until May 2026 |
| AIFS Single and ENS | ECMWF, 2024–25 | Graph encoder and decoder, transformer processor | 28–31 km, 6 h | 15 days; ENS 51 members | Operational since February and July 2025 |
| Aardvark Weather | Cambridge, Turing Institute, Microsoft, ECMWF, 2025 | End to end from observations | 1.41°, 5 levels | 10 days | Research |

Two further steps followed. **GenCast** (Google DeepMind, *Nature*, December 2024) produced ensembles instead of single forecasts. It was more skilful than ECMWF's physics ensemble (ENS) on 97.2 % of 1 320 targets, and on 99.8 % beyond 36 hours. Each 15-day member took about 8 minutes on one TPU v5 chip. **NeuralGCM** (Google, 2024) kept a differentiable physics core for the large-scale flow and learned only small-scale processes such as clouds. It stays stable over decades, so it can also simulate climate.

Foundation models followed (Chapter 44). **Aurora** (Microsoft, *Nature*, May 2025) was pre-trained on over a million hours of weather and climate data, then fine-tuned for forecasts at 0.1°, air pollution, ocean waves and cyclone tracks. In November 2025 Google DeepMind released

**WeatherNext 2**. Its **functional generative network** injects random noise inside the network, so each run gives a different but realistic forecast. It makes hundreds of scenarios in under a minute on one TPU and drives the forecasts in Google's consumer products.

### 47.2.2 Into Operations

The decisive change came when weather centres put learned models into daily service (Table 47.2). "Operational" means run every day on a fixed schedule, monitored, and delivered to users with a service commitment. In every case the learned model runs **beside** the physics model, not instead of it.

ECMWF built its own model (see the box at the end of this section); NOAA started from an outside one. Through its Project EAGLE, it fine-tuned GraphCast on analyses from its own assimilation system. Of its three models of December 2025, **AIGFS** is a single forecast with better cyclone tracks but weaker intensity than the physics GFS. **AIGEFS** is a 31-member learned ensemble that needs about 9 % of the computing of the physics ensemble. **HGEFS** joins 31 physics and 31 learned members into one 62-member "grand ensemble" — the first such hybrid in operations, NOAA believes.

**TABLE 47.2** Machine-learning global models at national and international centres, October 2026.

| Centre | System | In service | How it was built | Notes |
|---|---|---|---|---|
| ECMWF (Europe) | AIFS Single | 25 Feb 2025 | Own graph–transformer model, Anemoi software | 28 km, 4 runs a day to 15 days; open data; v2 on 12 May 2026 added waves |
| ECMWF | AIFS ENS | 1 Jul 2025 | Trained on CRPS | 50 members plus control, 31 km |
| NOAA (USA) | AIGFS, AIGEFS, HGEFS | Dec 2025 | GraphCast fine-tuned on NOAA analyses | 16-day forecast in about 40 min with 0.3 % of GFS computing; 31-member AI ensemble; 62-member hybrid |
| DWD (Germany) | AICON | Runs since Sep 2025; fully operational 2 Mar 2026 | Graph network built with Anemoi | About 13 km, to 180 h |
| CMA (China) | Fengqing | 2025 | With Tsinghua University | Rain skill comparable to physics models to 10 days |
| Met Office (UK) | FastNet | Not yet | With the Alan Turing Institute | Research; similar RMSE to the Met Office global model (2026) |

### 47.2.3 Learning Directly from Observations

All the models above start from an analysis made by physics-based assimilation. The next step is to learn the whole chain. **Aardvark Weather** (Cambridge, the Alan Turing Institute, Microsoft Research and ECMWF; *Nature*, 2025) takes in raw data from scatterometers, sounders, geostationary imagers, stations, ships and radiosondes. It outputs forecasts on a 1.41° grid and for individual stations in about one second on four GPUs; ECMWF's physics system needs about 1 000

node-hours for assimilation and forecast. Using about 8 % of the observations that operational systems ingest, it beat NOAA's GFS on many variables but trailed ECMWF's HRES in the upper atmosphere and at short range.

ECMWF's **GraphDOP** (2024–25) learns from satellite and in-situ observations alone and was skilful to about five days in its first version. A Google DeepMind preprint of September 2026, **WeatherNext 3**, ingests low-latency geostationary images and predicts station temperatures anywhere. These systems are research, but they could free learned forecasting from its dependence on physics-based analyses.

LANDMARK: ECMWF'S AIFS GOES OPERATIONAL (2025–26)

- **25 February 2025.** ECMWF put its **Artificial Intelligence Forecasting System (AIFS)** into operations, the first data-driven global model in service at a major international centre. It runs at 28 km beside the 9 km physics model (IFS). ECMWF reported gains of up to 20 % on some scores, including cyclone tracks, and about 1 000 times less energy per forecast.
- **Design.** A graph network maps the input grid onto an internal grid of about 1°, a transformer steps it forward 6 hours, and a decoder maps it back. Pre-trained on four decades of ERA5 and fine-tuned on operational analyses, it trained in about a week on 64 GPUs. A 10-day forecast takes about 2.5 minutes on one GPU.
- **1 July 2025. AIFS ENS** followed: a 51-member ensemble trained on the continuous ranked probability score. ECMWF reported gains over the physics ensemble of up to 20 % for surface temperature.
- **12 May 2026.** Version 2 of both arrived with the physics model's new cycle, 50r1, and added ECMWF's first operational data-driven wave forecasts. Both were re-tuned on analyses from the new cycle. Started from the new analyses, version 1 had lost skill, with 2 m temperature errors up to 20 % larger.
- **Open by design.** AIFS forecasts are free open data (CC-BY-4.0). Its training software, Anemoi, is shared with national services; Germany's AICON is built on it.

## 47.3 How Learned Weather Models Work

The key idea of this section is that a learned weather model is a function that maps today's state to the state a few hours later. It is trained by comparing millions of its predictions with reanalysis, then applied again and again to reach the desired lead time.

### 47.3.1 The Forecast as a Learned Time Step

Let $\mathbf{X}_t$ be the global state at time $t$: temperature, wind, humidity and geopotential on every level, plus surface fields. A learned model $f_\theta$ with parameters $\theta$ predicts the change over one step $\Delta t$, usually 6 hours:

$$\hat{\mathbf{X}}_{t+\Delta t} = \mathbf{X}_t + f_\theta(\mathbf{X}_t, \mathbf{X}_{t-\Delta t}).$$

A 10-day forecast applies the model 40 times, each time feeding its own output back in. This is called an **autoregressive roll-out**. Predicting the change rather than the full state makes learning easier, because most of the atmosphere changes little in 6 hours.

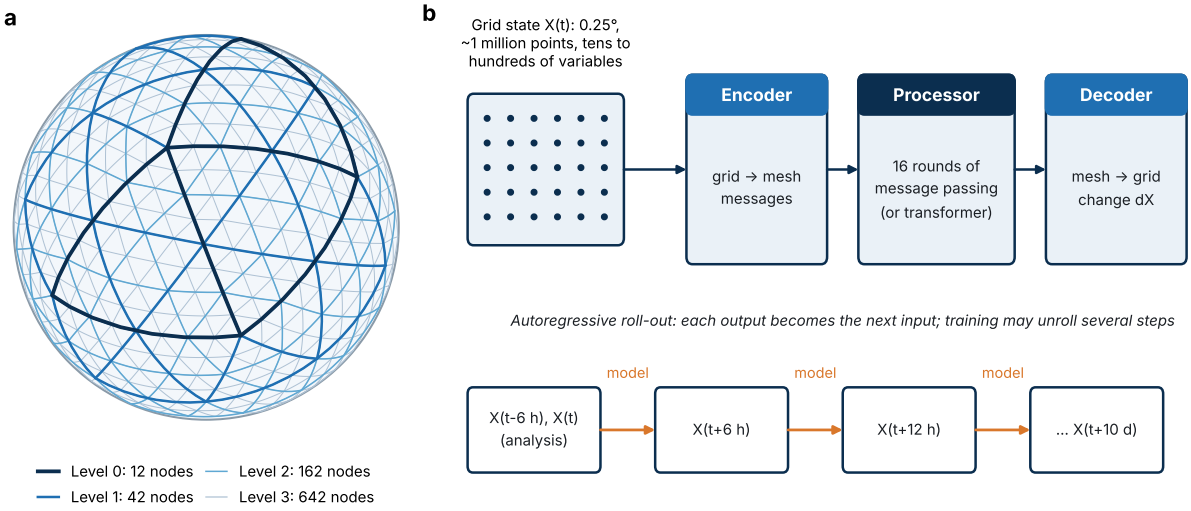
Training minimises a weighted mean squared error between the model output and reanalysis over many past dates:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{\text{dates}} \sum_{j=1}^{J} \sum_{i,v,\ell} a_i \, w_v \, w_\ell \left( \hat{x}^{(j)}_{i,v,\ell} - x^{(j)}_{i,v,\ell} \right)^2 .$$

Here $i$ labels grid points, $v$ variables and $\ell$ levels. The **area weight** $a_i \propto \cos(\text{latitude})$ stops the crowded grid points near the poles from dominating. The weights $w_v$ and $w_\ell$ set how much each variable and level matter. The sum over $j$ covers several roll-out steps: fine-tuning on 2–12 steps teaches the model to correct its own errors, but also makes its forecasts smoother.

### 47.3.2 Graphs, Transformers and Meshes

The network inside $f_\theta$ must let information travel across the globe. **Graph neural networks** (Chapter 44) pass messages between the nodes of a mesh. GraphCast and AIFS use an **encode–process–decode** structure (Figure 47.2). An encoder maps the latitude–longitude grid onto an internal mesh, a processor exchanges messages along its edges many times, and a decoder maps the result back as the predicted change. GraphCast's **multi-mesh** keeps the edges of every refinement level of an icosahedron, from 12 nodes up to 40 962. Long edges carry information quickly across the globe; short edges resolve local detail.




(a) Computed by repeated subdivision of an icosahedron; a multi-mesh keeps the edges of every level, so messages travel both short and long distances (GraphCast uses levels 0-6, 40 962 nodes).

**FIGURE 47.2** (a) An icosahedral multi-mesh computed by repeated subdivision: each level splits every triangle into four, and the multi-mesh keeps the edges of all levels. (b) The encode–process–decode structure of a graph weather model and its autoregressive roll-out.

**Transformers** (Chapter 44), used by Pangu-Weather, FuXi and Aurora, let patches of the globe attend to each other, usually within local windows to save memory. **Neural operators**, such as FourCastNet's, learn in Fourier or spherical-harmonic space. Once trained on enough data, all three designs reach similar skill; they differ mainly in memory and speed.

### 47.3.3 Ensembles: Diffusion and CRPS Training

A single forecast trained on mean squared error has a known flaw. When the future is uncertain, the forecast that minimises squared error is the **average of all possible futures**. An average is smooth: a storm that may go left or right is forecast as a weak storm in the middle. The fix is to forecast a probability distribution, as physics ensembles do.

Two learned approaches are in use. **Diffusion models** (Chapter 44), as in GenCast, learn to turn random noise into a realistic weather state, step by step, conditioned on the current state. Each fresh noise sample gives another ensemble member. **CRPS training**, used by AIFS ENS and WeatherNext 2, adds noise inside the network and directly optimises the **continuous ranked probability score** of the ensemble.

$$\mathrm{CRPS}(F, y) = \mathbb{E}\,|X - y| - \tfrac{1}{2}\,\mathbb{E}\,|X - X'|$$

Here $X$ and $X'$ are independent members drawn from the forecast distribution $F$ and $y$ is what happened. The first term rewards members close to the truth. The second rewards spread between members, so the ensemble does not collapse to one forecast. For one deterministic forecast the CRPS reduces to the absolute error.

### 47.3.4 A Learned Weather Model in Miniature

The ideas above can be tried on a small chaotic system. The **Lorenz-96 model** is a ring of 40 variables that behave like a weather quantity around a latitude circle.

$$\frac{dx_k}{dt} = (x_{k+1} - x_{k-2})\,x_{k-1} - x_k + F, \qquad k = 1, \ldots, 40$$

The quadratic term moves "weather" around the ring, $-x_k$ damps it and $F = 8$ drives it, giving chaos. By a common convention, 0.05 time units count as 6 hours.

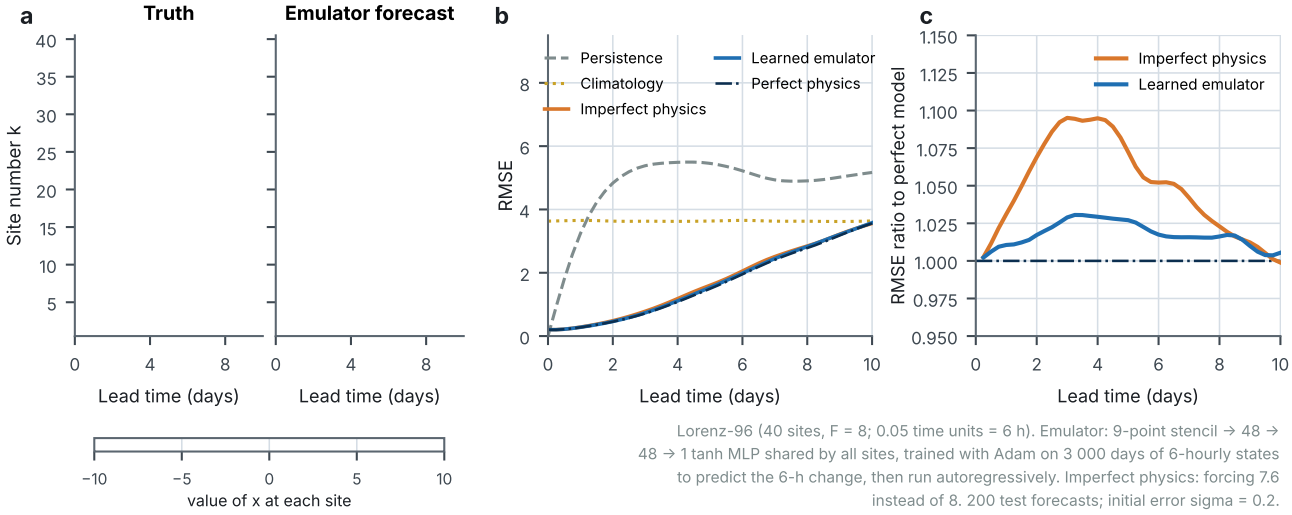
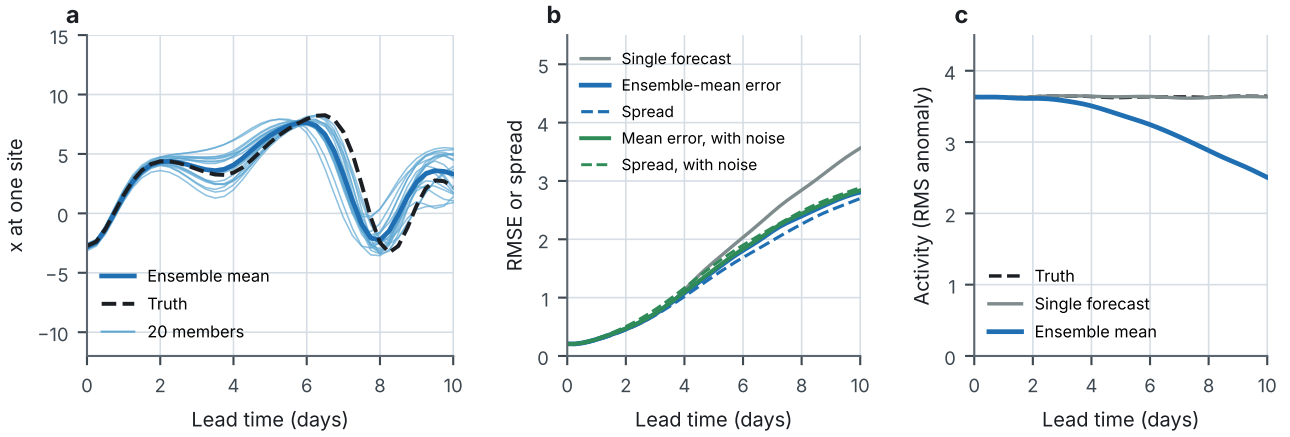



**FIGURE 47.3** A learned emulator of the chaotic Lorenz-96 system. (a) Truth and a 10-day emulator forecast as space–time diagrams. (b) Error growth for 200 forecasts: the emulator (blue) tracks the perfect physics model, beats a physics model with a 5 % forcing error (orange) and far outperforms persistence. (c) Error relative to the perfect model.

We generated 3 000 days of 6-hourly "reanalysis" and trained a small neural network to predict the 6-hour change at each site from the values at that site and its eight nearest neighbours. Like a graph model, the same network serves all sites. It has 2 881 parameters. We then ran 200 forecasts from initial states with small errors, as if from an imperfect analysis (Figure 47.3). The learned emulator's error grew almost exactly like that of the perfect physics model: 0.74 against 0.71 after 3 days and 1.53 against 1.49 after 5 days. It beat a physics model with a 5 % error in its forcing, which reached 0.78 and 1.60. All three reached half the climatological spread after 5.5–5.7 days. Persistence — forecasting no change — was worse than climatology after just over a day. Chaos, not the model, set the limit.

The same emulator shows how ensembles help (Figure 47.4). Twenty members started from perturbed analyses spread out as errors grew. After 10 days the ensemble-mean error was 2.81, against 3.56 for a single forecast. The **spread** — the scatter of the members — was 2.70. A spread close to the error of the ensemble mean means the ensemble is **well calibrated**: its uncertainty is honest. With perturbations only at the start, the spread was 4–7 % too small. Adding a little noise at each step, a crude stand-in for model error, brought spread and error to within 2–7 % of each other.



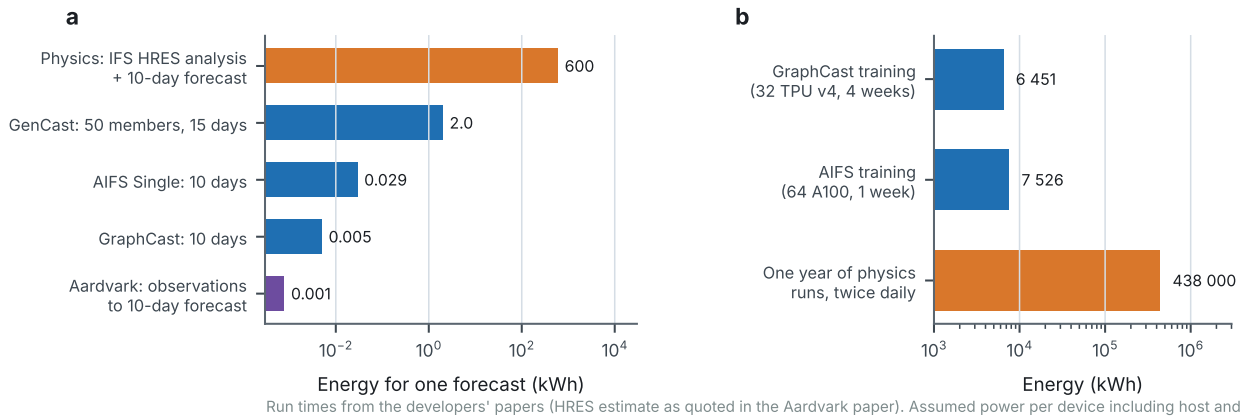

Lorenz-96 learned emulator of the previous figure; 150 start dates, 20 members from analysis perturbations (sigma = 0.2). Green: extra Gaussian noise (sigma = 0.06) added at every 6-h step to represent model error.

**FIGURE 47.4** An ensemble of the learned Lorenz-96 emulator. (a) Twenty members, their mean and the truth at one site. (b) The ensemble mean beats a single forecast, and its spread matches its error once a little model-error noise is added (green). (c) The ensemble mean loses "activity": it smooths towards climatology, as any forecast trained on squared error does.

Panel (c) shows the price of the mean. The root-mean-square anomaly of a single forecast, its **activity**, stays at 3.6, like the truth. The ensemble mean's activity falls to 2.5 by day 10. A deterministic model trained on squared error over long roll-outs drifts towards the same smooth mean. This is why learned forecasts look blurred at long range, and why the best systems now forecast ensembles.

### 47.3.5 Computing and Energy

Learned models are cheap to run and expensive to train (Figure 47.5). The physics system behind ECMWF's HRES needs roughly 1 000 node-hours of a supercomputer for one assimilation and 10-day forecast. AIFS needs 2.5 minutes of one GPU, and GraphCast under a minute of one TPU. Even a 50-member GenCast ensemble needs only about 7 TPU-hours. Training is the expensive part: about 21 500 TPU-hours for GraphCast and 10 750 GPU-hours for AIFS. Physics models need supercomputers (Figure 47.6); a trained learned model does not.




Run times from the developers' papers (HRES estimate as quoted in the Aardvark paper). Assumed power per device including host and cooling share: CPU node 0.6 kW, A100 GPU 0.7 kW, TPU 0.3 kW. ML forecasts except Aardvark still need an analysis from data assimilation.

**FIGURE 47.5** Energy for one forecast (a) and for training (b), on a logarithmic scale, from published run times and assumed power draws. A learned forecast uses thousands of times less energy, but it still needs an analysis made by the physics system.

### EXAMPLE: ENERGY OF ONE LEARNED AND ONE PHYSICS FORECAST

Compare one 10-day forecast from AIFS with one from a physics model. The physics run time is not published, so it is an assumption: 100 nodes for 1 hour, at 0.6 kW per node including cooling. Assume 0.7 kW for one GPU with its share of host and cooling.

1. **Physics forecast.** E = 100 nodes × 1 h × 0.6 kW = 60 kWh.
2. **Learned forecast.** AIFS takes 2.5 min = 0.0417 h on one GPU: E = 0.0417 h × 0.7 kW ≈ 0.029 kWh, or about 105 kJ.
3. **Ratio.** 60 / 0.029 ≈ 2 000. ECMWF quotes about 1 000; the exact factor depends on the assumptions, but three orders of magnitude is robust.
4. **Training.** AIFS used about 64 GPUs for one week: 64 × 168 h × 0.7 kW ≈ 7 500 kWh. That equals about 125 physics forecasts, or two months of twice-daily runs. Training pays for itself quickly, even if repeated every year.
5. **What is not saved.** The learned forecast still starts from an analysis made by 4D-Var with the physics model; in the Aardvark paper's estimate, assimilation plus forecast take about 1 000 node-hours. Until assimilation is also learned, the saving for the whole system is far smaller than in step 3.

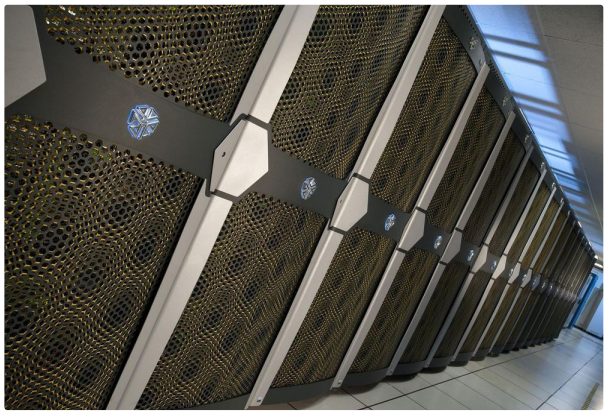


**FIGURE 47.6** The Pleiades supercomputer at NASA's Ames Research Center in 2008. Physics-based weather and climate models need machines like this; learned models run on a single graphics processor once trained. Image: NASA/ARC (Dominic Hart).

## 47.4 Measuring Skill Fairly

The key idea of this section is that a forecast is only as good as the test it passes. Learned models win most standard scores, but scores reward different things, and a smooth forecast can win a score while missing the storm.

### 47.4.1 Scores

Table 47.3 lists the main scores. **RMSE** penalises large errors and rewards smooth forecasts. The **anomaly correlation coefficient (ACC)** measures how well the pattern of departures from climate is forecast:

$$\mathrm{ACC} = \frac{\sum_i a_i (f_i - c_i)(o_i - c_i)}{\sqrt{\sum_i a_i (f_i - c_i)^2 \, \sum_i a_i (o_i - c_i)^2}},$$

where $f$, $o$ and $c$ are forecast, observation and climatology, and $a_i$ are area weights. By convention, a 500 hPa height forecast with ACC above 0.6 still has useful skill.

**TABLE 47.3** Common scores for weather forecasts.

| Score | What it measures | Typical use |
|---|---|---|
| RMSE | Size of errors; favours smooth forecasts | All variables and lead times |
| Anomaly correlation (ACC) | Pattern of departures from climate | 500 hPa height; useful above 0.6 |
| CRPS | Quality of a probability forecast | Ensembles |
| Spread–error ratio | Honesty of the ensemble spread | Ensembles; ideal value 1 |
| Track and intensity error | Distance to the centre; wind error | Tropical cyclones |
| Critical success index | Hits ÷ (hits + misses + false alarms) | Rain, nowcasting |
| Power spectrum | Energy at each spatial scale | Detecting blurring |

### 47.4.2 Where Learned Models Win

On the scores of the WeatherBench 2 benchmark and the centres' own verification, the best learned models beat the best physics models for most upper-air variables from day 1 to day 10. The gains are largest for large-scale patterns, such as 500 hPa height, and for tropical-cyclone tracks, which depend mainly on the steering flow. Learned forecasts also jump around less from run to run, as DWD found for AICON.

> **DATA BOX — AN AI MODEL AT THE NATIONAL HURRICANE CENTER, 2025**
>
> - In June 2025 Google opened Weather Lab, with an experimental cyclone model giving 50 scenarios out to 15 days. NHC forecasters saw it live all season, including Category 5 Melissa (Figure 47.7).
> - NHC's verification report (spring 2026) found that its ensemble mean "slightly outperformed" the official Atlantic track forecasts at 12–72 hours, with comparable intensity skill.
> - In the eastern Pacific it beat all other models, the official forecast and the consensus aids at 48–120 hours.
> - It was one input to human forecasters, whose own track forecasts beat their 2020–24 average.

**FIGURE 47.7** Category 5 Hurricane Melissa about 80 km south of Jamaica, photographed from the International Space Station on 28 October 2025. Learned models forecast the tracks of such storms well; their peak intensity is harder. Image: NASA/JSC.

### 47.4.3 Where They Fall Short

**Smoothing.** Models trained on squared error lose small-scale detail with lead time (Figure 47.4). Their power spectra fall below the real atmosphere's at scales under a few hundred kilometres, so fronts broaden and rain spreads into drizzle.

**Extremes.** In a study of Storm Ciarán (November 2023), four learned models placed the storm well but all underestimated its peak winds, which gusted to 111 knots (about 205 km/h) in Brittany. A 2025–26 study of record-breaking heat, cold and wind found that ECMWF's HRES beat GraphCast, Pangu-Weather and FuXi at nearly all lead times. The learned models underestimated how often and how far records were broken. A model trained on the past struggles with states it has never seen — a growing problem as the climate warms.

**Intensity.** Learned models tend to underestimate the peak winds of tropical cyclones. A 0.25° grid is too coarse to resolve an eyewall, and the training data are themselves smooth. NOAA's AIGFS 1.0 and its hybrid ensemble were both weaker on intensity than their physics counterparts.

**Physical consistency.** Nothing forces a learned model to conserve mass or energy, or to keep winds balanced with pressure. Outputs are mostly close to balance because the training data are, but small errors matter when the output drives other models. Learned models also lack many of the hundreds of fields, such as cloud ice or radiation fluxes, that physics models provide.

**Dependence on analyses.** A learned model inherits every error of its starting analysis; ECMWF's AIFS had to be re-tuned when the physics assimilation changed in 2026. A system of learned models alone would still need the satellites, the assimilation and the physics model that create its inputs.

**PRACTICE: EVALUATING A LEARNED WEATHER MODEL FAIRLY AGAINST A PHYSICS BASELINE**

1. **Fix the test period.** Use at least one full year, preferably two, that the model never saw in training or validation.
2. **Use the same starting point.** Start both models from the same analysis, or verify each against its own analysis and against observations. Verifying a model against the analysis it was trained to imitate flatters it.
3. **Match the resolution.** Compare on a common grid. Coarsening a high-resolution physics forecast removes its small-scale "double penalty" and makes the comparison fair.
4. **Score probabilistically.** Use CRPS and spread–error ratio for ensembles, RMSE and ACC for single forecasts. Report by region, season and lead time, with confidence intervals.
5. **Check the physics.** Plot power spectra, check mass and energy budgets, and look at balance between wind and pressure.
6. **Test the extremes.** Score storms, heatwaves and records separately, against station observations. Use event-based scores (CSI, hit rate) and tropical-cyclone track and intensity errors.
7. **Read real cases.** Have forecasters examine high-impact cases side by side. Track run-to-run consistency.
8. **Count the whole cost.** Include the assimilation that makes the analysis, the training and the retraining, not only the run time.

## 47.5 Machine Learning in the Observing Chain

The key idea of this section is that learning is useful long before the forecast model. It helps turn raw satellite signals into usable quantities, correct biases, speed up the physics inside assimilation, and forecast the next few hours.

### 47.5.1 Retrievals from Satellites

A **retrieval** estimates a geophysical quantity, such as rain rate, from what an instrument measures. Geostationary imagers see cloud tops every 5–15 minutes but not the rain beneath. Microwave instruments in low orbit see the rain but pass only a few times a day (Chapter 16). Neural networks have linked the two since the 1990s. PERSIANN ("precipitation estimation from remotely sensed information using artificial neural networks") maps infrared cloud-top temperatures to rain rate, trained on microwave and radar estimates. Multi-satellite products such as NASA's IMERG have used such estimates to fill gaps between microwave passes.

Deep networks now use several channels and the pattern of clouds. GREMLIN, from Colorado State University's CIRA, turns GOES imagery and lightning data into synthetic radar maps. In a one-week test, assimilating them gave better forecasts out to 12 hours over the mountainous western United States, where radar coverage is poor. It remained research in 2025.

**Tropical-cyclone intensity** from satellites still relies largely on the **Dvorak technique**: a trained analyst matches cloud patterns to an intensity scale. When only satellite estimates exist, the National Hurricane Center puts their uncertainty at 10–20 %. Convolutional networks trained on infrared and microwave images, with aircraft reconnaissance as ground truth, can make the estimate objective and repeatable. Most such networks are still research tools.

### 47.5.2 Bias Correction, Quality Control and Fast Observation Operators

Every satellite channel has biases that drift with orbit, temperature and age. Operational 4D-Var corrects them with **variational bias correction**, using a few predictors per channel. Neural networks can learn more complex, state-dependent biases and flag radiances contaminated by cloud, a task now done with hand-tuned thresholds. Both remain mostly research.

The observation operator $\boldsymbol{H}$ is costly: a radiative-transfer model runs for every observation in every iteration of 4D-Var. A network trained on that model can reproduce it many times faster, especially for cloudy and rainy scenes. It must also give accurate derivatives, because 4D-Var follows the gradient of $\boldsymbol{J}$. Such emulators are a step towards learned assimilation (Table 47.4).

**TABLE 47.4** Machine learning in the observing and assimilation chain.

| Task | Classical method | Learned approach | Status, 2026 |
|---|---|---|---|
| Rain from geostationary infrared | Cloud-top temperature thresholds | Neural networks since PERSIANN (1997); deep networks on many channels | Used in products; deep versions in research |
| Synthetic radar where radar is missing | None | Convolutional network on imager and lightning data (GREMLIN) | Research and demonstration |
| Tropical-cyclone intensity | Dvorak pattern matching | Convolutional networks on infrared and microwave images | Research |
| Radiance bias correction | Variational bias correction | Neural networks with state-dependent predictors | Research |
| Observation operator | Radiative-transfer models | Network emulators with accurate gradients | Research |
| Nowcasting rain, 0–2 h | Optical-flow extrapolation | Generative networks (DGMR, NowcastNet), MetNet-3 | MetNet-3 in Google products; others research |

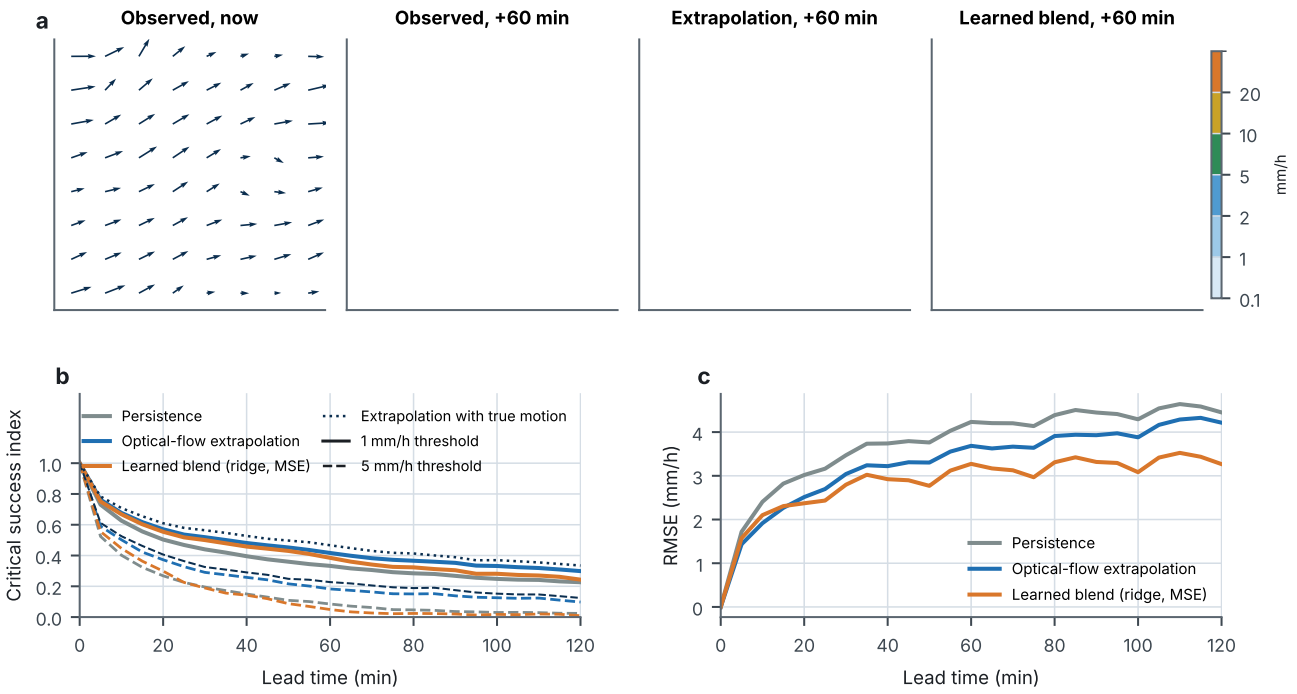
### 47.5.3 Nowcasting

A **nowcast** forecasts the next 0–6 hours, mainly for rain and storms. Physics models need time to "spin up" storms, so for the first hours the best tool has long been **extrapolation**: estimate how radar echoes move, then move the latest image along. The motion is found by **optical flow**, which assumes that rain is carried along unchanged.

$$\frac{\partial R}{\partial t} + \mathbf{u}\cdot\nabla R = 0, \qquad \hat{R}(\mathbf{r}, t + n\Delta t) = R(\mathbf{r} - n\,\mathbf{u}\,\Delta t,\; t)$$

The first equation defines the motion field $\mathbf{u}$, found by optimisation as the shift that best matches successive images. The second moves the latest image along $\mathbf{u}$. Systems based on this idea, such as STEPS and the open-source pySTEPS, run at many weather services.

Figure 47.8 tests it on a synthetic radar field. Rain cells of many sizes grow and decay, small cells faster than large ones, while the whole field moves at 39 km/h. We estimated the motion by block matching across three pairs of frames. At 60 minutes, extrapolation reached a critical success index (CSI) of 0.42 for rain above 1 mm/h, against 0.33 for persistence. Extrapolation with the true motion reached 0.47, so the motion estimate cost a little skill. For heavy rain above 5 mm/h, the CSI fell to 0.18, because small intense cells die quickly.




Synthetic radar: 384 km x 384 km, 2 km pixels, 5-min frames; power-law log-rain field whose small scales decorrelate faster, moving uniformly at 39 km/h. Motion from block matching of three frame pairs; learned blend = ridge regression of log rain on the extrapolated field smoothed at 4 scales, trained on earlier frames. 15 test times.

**FIGURE 47.8** Nowcasting a synthetic advecting rain field. (a) The field now with estimated motion vectors, the truth 60 minutes later, the optical-flow extrapolation and a learned blend. (b) Critical success index for rain above 1 mm/h (solid) and 5 mm/h (dashed). (c) RMSE: the learned blend has the lowest error but loses heavy rain.

We then trained a simple learned correction. A regression predicted future rain from the extrapolated field smoothed at four scales, by minimising squared error on earlier frames. It had the lowest RMSE at every lead time: 3.27 mm/h at 60 minutes against 3.69 for extrapolation. Yet it scored worse on the CSI: at 60 minutes only 0.05 for heavy rain, against 0.18 for extrapolation. Squared-error training spreads rain into a smooth, light field, so intense cores vanish. This is the nowcasting version of the blurring seen in Figure 47.4.

Deep-learning nowcasts face the same trap; the leading ones avoid it with generative training. **DGMR**, from DeepMind and the Met Office (*Nature*, 2021), makes sharp 90-minute forecasts from 20 minutes of radar. More than 50 Met Office meteorologists ranked it first in 89 % of cases. **NowcastNet** (Tsinghua University, *Nature*, 2023) joins a physical advection scheme to a generative network; 62 Chinese meteorologists ranked its 3-hour forecasts first in 71 % of cases. Google's **MetNet-3** (2023) forecasts rain, temperature and wind up to 24 hours ahead at 1–4 km from radar, satellite and station data. It runs in Google products for the United States and 27 European countries. NVIDIA's **StormScope** (January 2026) forecasts storms 0–6 hours ahead directly from geostationary images.

## 47.6 Climate: Emulators, Downscaling and Digital Twins

The key idea of this section is that climate questions differ from weather questions. They ask about statistics over decades under conditions that have never occurred. Learning helps where it can be checked against physics, and is dangerous where it must extrapolate.

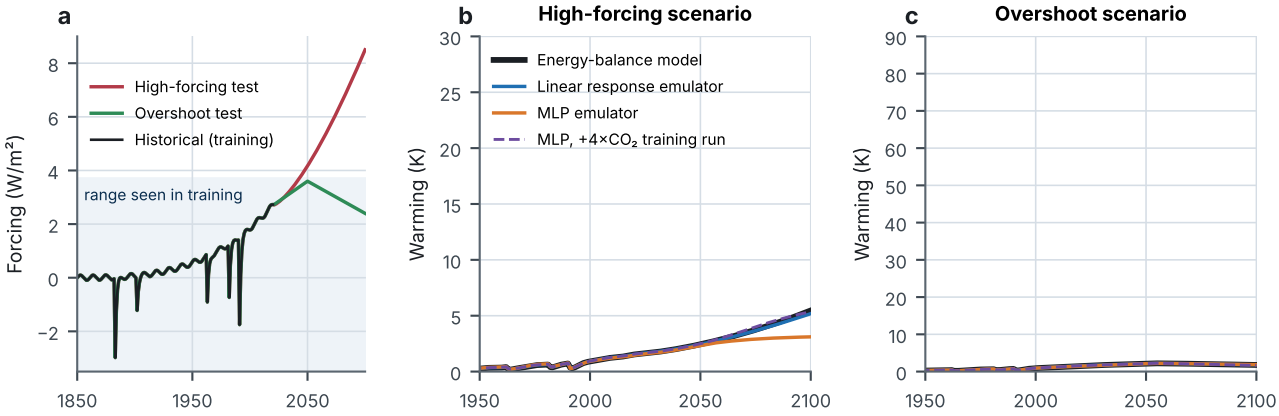
### 47.6.1 Climate Emulators

A **climate emulator** is a fast learned model that imitates an expensive climate model. **Full emulators** step the atmosphere forward like a weather model but stay stable for centuries. ACE2, from the Allen Institute for AI, was trained on ERA5 and physics-model output and runs about 1 500 simulated years per day on one GPU at 1°. A coupled atmosphere–ocean version, SamudrACE, followed in 2025. **Reduced emulators** map emission scenarios to global or regional warming, so thousands of scenarios can be explored.

The danger is **extrapolation**. A weather model sees the same kinds of state in training and in use. A climate model is asked about a world warmer than anything in its training data. Figure 47.9 shows what can happen. The "truth" is a two-layer **energy-balance model**: an upper ocean and atmosphere of heat capacity $C$, coupled to a deep ocean of heat capacity $C_d$.

$$C\frac{dT}{dt} = F - \lambda(T)\,T - \gamma(T - T_d), \qquad C_d\frac{dT_d}{dt} = \gamma(T - T_d)$$

Here $F$ is the radiative forcing, $\lambda$ the climate feedback, which weakens slightly as the planet warms, and $\gamma$ the exchange with the deep ocean. Its equilibrium warming for doubled $CO_2$ is 3.7 K. We trained two emulators on three runs with forcing up to 3.7 W/m$^2$: a historical run, a 1 % per year rise in $CO_2$ to doubling, and an abrupt doubling. Both emulators see the forcing smoothed over 1–400 years. One is linear, like a physical response function. The other is a small neural network (MLP).




Two-layer energy-balance model (C = 7.3 and 106 W yr m$^{-2}$ K$^{-1}$, feedback 1.13 - 0.035 T W m$^{-2}$ K$^{-1}$, exchange 0.73 W m$^{-2}$ K$^{-1}$) with red-noise variability. Emulators map forcing smoothed over 1-400 years to warming; trained on historical, 1 %/yr to 2×$CO_2$ and abrupt 2×$CO_2$ runs; MLP = mean of 5 networks.

**FIGURE 47.9** Emulating a two-layer energy-balance climate model. (a) Training forcing (black and shaded range) and two unseen test scenarios. (b) Under high forcing, outside the training range, the neural-network emulator (orange) fails, while the linear response emulator (blue) stays close. Adding an abrupt 4×$CO_2$ run to training (violet) fixes the network. (c) In an overshoot scenario inside the training range, all emulators work.

In an overshoot scenario that stayed within the training range, all emulators reproduced the truth, with RMSE of 0.06 K or less over 2021–2100. In a high-forcing scenario reaching 8.5 W/m$^2$ by 2100, the true warming was 5.47 K. The linear emulator gave 5.19 K: close, but low, because it cannot know that the feedback weakens. The neural network gave only 3.11 K. Its tanh units had saturated outside the range they were trained on. When an abrupt 4×$CO_2$ run was added to training, the network gave 5.36 K. The lesson applies to every learned climate model: **train on the conditions you will predict**, prefer structures that respect physics, and test outside the training range on purpose.

### 47.6.2 Downscaling

Global models run at 10–100 km; impacts happen at 1 km. **Downscaling** fills the gap. Statistical downscaling regresses local weather on large-scale predictors. **Generative downscaling** adds realistic small-scale detail, giving many possible fine fields for one coarse input. NVIDIA's CorrDiff, trained with data from Taiwan's Central Weather Administration, maps 25 km ERA5 fields to about 2 km: a network predicts the mean field and a diffusion model adds the detail. The output must be checked against stations, because a network can invent convincing but false detail.

### 47.6.3 Satellite Climate Records and Extremes

Learning from observations needs long, stable records. ESA's **Climate Change Initiative (CCI)** reprocesses decades of satellite data into over 100 consistent data sets, covering many of the 55 **essential climate variables** of the Global Climate Observing System. They are used to train and test learned models and to detect change. Statistical learning can even find the fingerprint of global warming in a single day's weather pattern. For **attribution** of extremes, learned models can generate thousands of plausible seasons to estimate how much more likely a heatwave has become. Their answers are only as good as their behaviour in the tails, which must be tested separately.

### 47.6.4 Digital Twins of the Earth

A **digital twin** is a model kept in step with observations that users can question. For example, how would this heatwave look in a world 2 °C warmer? The EU's **Destination Earth**, run by ECMWF, ESA and EUMETSAT, has a Climate Change Adaptation twin and a Weather-Induced Extremes twin, both at kilometre scale. Phase 2 added learned components for land, ocean, sea ice, waves and hydrology. Phase 3 (June 2026–June 2028) will couple them into a European learned Earth-system model. NVIDIA's **Earth-2** is a platform of open models (Table 47.5); in January 2026 it added a 15-day model and a storm nowcaster.

**TABLE 47.5** Learning in climate science: selected systems.

| System | Developer, year | What it does | Status, October 2026 |
|---|---|---|---|
| ClimaX | Microsoft and partners, 2023 | Foundation model pre-trained on climate-model output | Research |
| ACE2, SamudrACE | Allen Institute for AI and partners, 2024–25 | Emulates atmosphere (and ocean) at 1°, about 1 500 years a day on one GPU | Research; open |
| NeuralGCM | Google, 2024 | Hybrid physics–learned model, stable for decades | Research; open |
| CorrDiff | NVIDIA, 2024 | Generative downscaling, 25 km to about 2 km | Open; developed with Taiwan's weather agency |
| Destination Earth | EU: ECMWF, ESA, EUMETSAT, 2022– | Kilometre-scale climate and extremes twins with learned parts | Phase 3, June 2026–June 2028 |
| Earth-2 | NVIDIA, 2024– | Platform of open weather and climate models | New models released January 2026 |
| ESA CCI | ESA, 2010– | Over 100 satellite climate data sets | Operational archive |

## 47.7 Space Weather

The key idea of this section is that space weather has the same forecasting needs as the weather, with far fewer observations. That makes machine learning attractive and also risky. The Sun is watched by a handful of spacecraft, and big storms are rare.

### 47.7.1 Why Space Weather Is Hard to Learn

Space weather (Chapter 22) runs from the Sun to the ground. A flare or coronal mass ejection (CME) leaves the Sun. The solar wind carries it to the Earth in one to four days. It reaches the L1 monitors about an hour before the Earth. It then drives currents in the magnetosphere and ionosphere (Chapter 23). Each link has its own data and forecasting problem (Table 47.6).

**TABLE 47.6** Machine learning for space weather (selected tasks). TSS: true skill statistic.

| Task | Main inputs | Typical learned method | Reported skill | Status |
|---|---|---|---|---|
| Solar flares (24 h) | Magnetogram features or images | SVM, CNN, foundation models | TSS about 0.5–0.8 in research | Research; human-led forecasts in operations |
| CME arrival time | Coronagraph images, CME speed | Regression, CNN | Mean absolute error 6–12 h on research samples | Research |
| Solar-wind speed at L1 | Solar EUV images | CNN with LSTM | RMSE 72–82 km/s, 4 days ahead | Research |
| Ground magnetic disturbance | L1 solar wind | Recurrent and graph networks (DAGGER) | 30 min ahead, worldwide | Research demonstration |
| Kp and Dst indices | L1 solar wind, past index | Neural networks, LSTM, Gaussian processes | Kp ensemble up to 72 h (GFZ) | Some products |
| Ionosphere (TEC) | GNSS maps, solar indices | LSTM, ConvLSTM | Better than climatological models in studies | Research |
| Radiation belts | Particle data, solar wind | Neural networks | Electron flux forecasts in studies | Research |

Three things make learning hard. Data are few: the best magnetograms, from the Solar Dynamics Observatory (SDO), cover only since 2010, about one and a half solar cycles. Events are rare: there are only a few dozen great storms in the whole space age. And the key quantity for geomagnetic storms, the north–south component of the magnetic field in a CME ($B_z$), cannot be measured until the CME reaches L1.

### 47.7.2 Flare Forecasting from Magnetograms

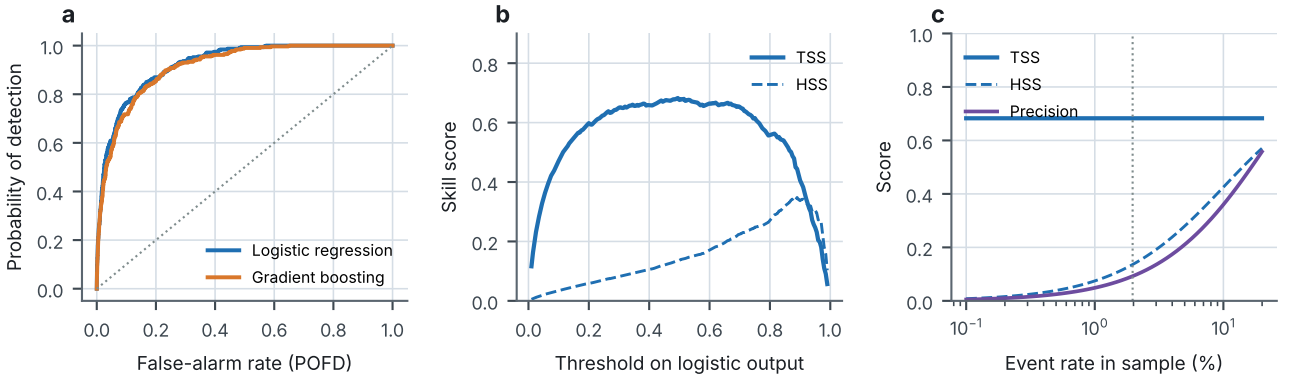
Large flares come from active regions with strong, twisted magnetic fields (Figure 47.12), mapped by SDO's Helioseismic and Magnetic Imager (HMI). Features computed from each map include the total unsigned flux, the flux near the **polarity inversion line** where the field changes sign, electric currents and shear. These features, or the images themselves, feed a classifier that gives the probability of an M- or X-class flare in the next 24 hours.

Flares are rare, so accuracy is useless as a score. A model that always says "no flare" can be 98 % accurate. Flare forecasts are compared with the **true skill statistic** (Chapter 43):

$$\mathrm{TSS} = \frac{\text{hits}}{\text{hits} + \text{misses}} - \frac{\text{false alarms}}{\text{false alarms} + \text{correct negatives}},$$

the probability of detection minus the probability of false detection. It ranges from −1 to 1 and does not depend on how common flares are in the sample. In 2015 Bobra and Couvidat reached TSS 0.76 with a support vector machine on HMI features, though only about 42 % of its flare warnings came true. A 2019 comparison of operational systems over 2016–17 found that most scored above "no skill", with TSS values roughly from 0 to 0.5. No method was best on every score.

Figure 47.10 shows the method on synthetic data. A hidden "free energy" drives five magnetogram-like features and the chance of a flare, with an unobserved trigger added. The flare rate is 2 %. A logistic regression reached a maximum TSS of 0.68, and gradient-boosted trees 0.66. Panel (c) shows why TSS is used. Keep the detection and false-alarm rates fixed: TSS stays constant, but precision rises from 2.5 % at a 0.5 % event rate to 56 % at 20 %. Scores like precision and the Heidke skill score (HSS) cannot be compared between solar maximum and solar minimum.




Synthetic region-days: five magnetogram-like features driven by a hidden free-energy variable; flare (M class or above within 24 h) drawn from a logistic law with an unobserved trigger term. First 60 % for training, last 40 % for testing. (c) Fixed detection and false-alarm rates of the logistic model.

**FIGURE 47.10** Flare forecasting on synthetic magnetogram features. (a) ROC curves for logistic regression and gradient boosting. (b) TSS and HSS against the decision threshold. (c) At fixed detection and false-alarm rates, TSS does not depend on the event rate; precision and HSS do.

### EXAMPLE: THE TRUE SKILL STATISTIC FROM A FLARE CONTINGENCY TABLE

The logistic model of Figure 47.10 was tested on 16 000 region-days, of which 315 had an M-class or larger flare. At its best threshold it gave 268 hits, 47 misses, 2 635 false alarms and 13 050 correct negatives.

1. **Probability of detection.** POD = 268 / (268 + 47) = 0.851.
2. **Probability of false detection.** POFD = 2 635 / (2 635 + 13 050) = 0.168.
3. **True skill statistic.** TSS = 0.851 − 0.168 = 0.683.
4. **Precision.** 268 / (268 + 2 635) = 0.092: only about 1 warning in 11 is followed by a flare.
5. **Accuracy.** (268 + 13 050) / 16 000 = 0.832. A forecast of "never" scores 0.980 but has TSS 0.
6. **Heidke skill score.** HSS = 2(ad − bc) / [(a + c)(c + d) + (a + b)(b + d)] = 0.136, with a–d the hits, false alarms, misses and correct negatives. False alarms keep it low.
7. **Meaning.** A high TSS shows that the model separates flaring from quiet regions, not that its warnings are usually right. Users who pay for false alarms should choose a higher threshold.

Since 2018 deep networks have learned directly from magnetogram and extreme-ultraviolet images, with results similar to feature-based methods on small samples. The NASA–IBM **Surya** foundation model (August 2025) was pre-trained on more than 200 TB of SDO data. Fine-tuned for flares, it reportedly beat existing benchmarks by about 16 % in preliminary tests (Chapter 44).

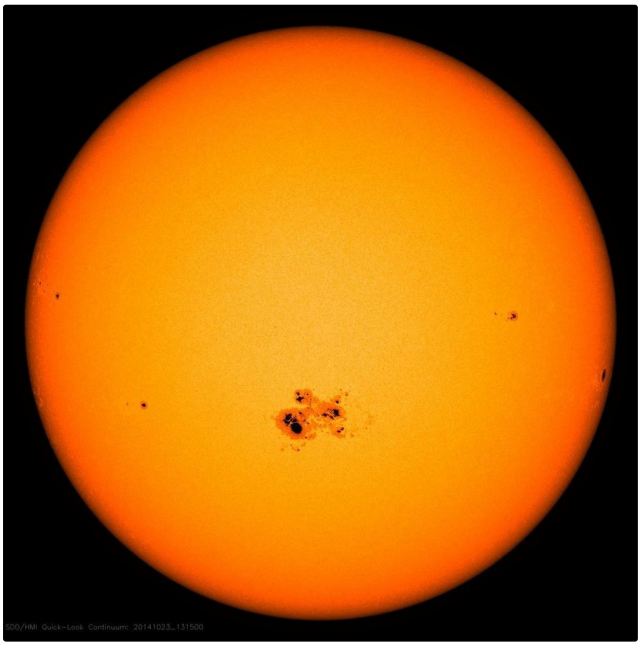


**FIGURE 47.11** The largest sunspot group of solar cycle 24, active region 12192, seen by SDO in October 2014. It produced six X-class flares, none of which launched a coronal mass ejection. Image: NASA/GSFC.

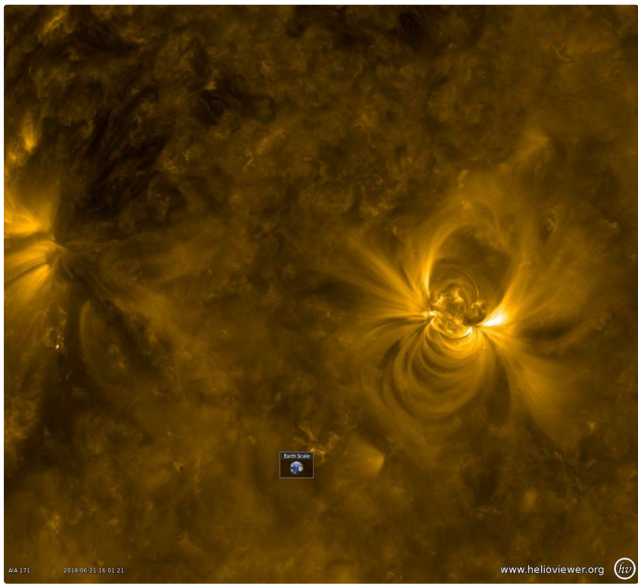



**FIGURE 47.12** Magnetic loops above a fast-growing active region in extreme-ultraviolet light, imaged by SDO in June 2018. Flare forecasts learn from the magnetic fields beneath such loops. Image: NASA/JPL (NASA/GSFC/Solar Dynamics Observatory).

Region 12192 (Figure 47.11) is a warning about labels. It produced 6 X-class, 22 M-class and 53 C-class flares in one disk passage, yet none of its X-class flares launched a CME. A model trained to predict flares would rate it highly. A model trained to predict geomagnetic storms should not.

### 47.7.3 CMEs and the Solar Wind

Forecasting when a CME will arrive is still hard. NASA's Community Coordinated Modeling Center collects real-time forecasts from many teams in its CME Scoreboard. A 2024 re-analysis found a mean absolute arrival-time error of 13.2 hours, a standard deviation of 17.4 hours and a bias of −2.5 hours (slightly early). That is only slightly better than in the Scoreboard's first five years, from 2013. Learned models report 6–12 hours on their own samples, such as 5.9 hours for a support vector machine on 182 CMEs and 9.75 hours for an ensemble on 363 CMEs. The samples differ, so the numbers cannot be compared directly.

Networks can also forecast the background solar wind. Coronal holes in extreme-ultraviolet images release fast wind that reaches the Earth 3–4 days later. An image network combined with a recurrent network forecast the wind speed at L1 four days ahead with an RMSE of 82 km/s; a later study reached 72 km/s. The physics-based WSA–Enlil model remains the operational standard.

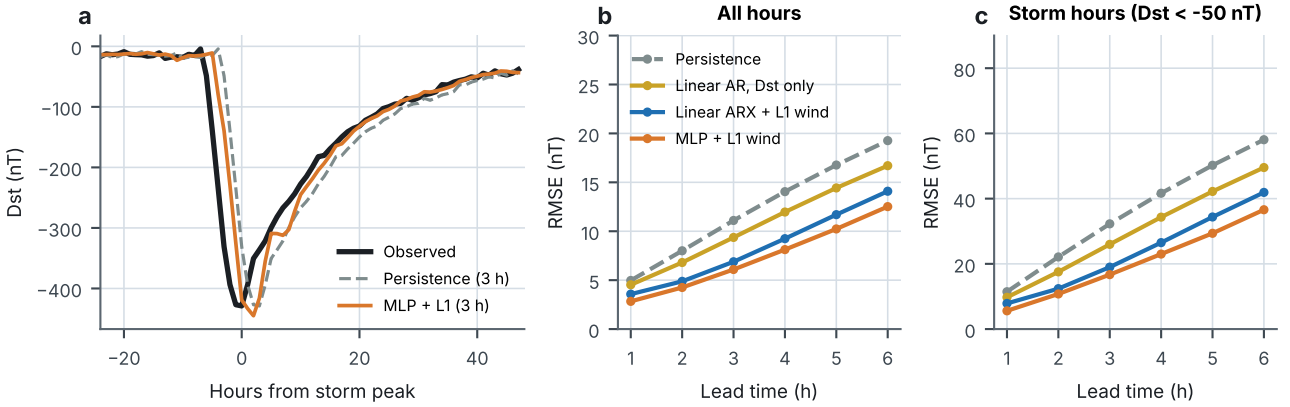
### 47.7.4 Geomagnetic Indices

Once the solar wind reaches L1, forecasts become much better, because the driver is measured. The **Dst index** measures the ring current that weakens the field at low latitudes during storms. **Kp** measures disturbance at mid-latitudes on a 0–9 scale (Chapter 22). A classic physics-based model links Dst to the solar-wind electric field $VB_s$, the solar-wind speed times the southward magnetic field:

$$\frac{d\,\mathrm{Dst}^*}{dt} = Q(VB_s) - \frac{\mathrm{Dst}^*}{\tau(VB_s)},$$

where $Q$ is the energy injection and $\tau$ a recovery time of several hours. Storms push the aurora towards the equator (Figure 47.14). Neural networks have forecast Dst and Kp since the early 1990s. Today's models include recurrent networks and Gaussian processes that also give uncertainty. GFZ in Potsdam runs a machine-learning Kp ensemble every hour, up to 72 hours ahead.

We built a synthetic test (Figure 47.13). Four years of hourly solar wind, with recurring fast streams and random CMEs, drove the Dst model above. Three years were used for training and the fourth for testing; the test year held a −429 nT superstorm, stronger than any in training. With only past Dst, a linear model reduced the 3-hour-ahead RMSE from 11.1 nT for persistence to 9.4 nT. Adding L1 solar-wind data cut it to 6.9 nT, and a neural network with the same inputs to 6.1 nT. In storm hours the network's skill score against persistence, $1 - \mathrm{MSE}/\mathrm{MSE}_{\mathrm{pers}}$, was 0.73. Most of the gain came from the L1 data; the network added a smaller step.




Four years of hourly synthetic solar wind (recurrent streams, CMEs, random Bz) drive the O'Brien-McPherron Burton-type Dst model with pressure correction and 2 nT noise. Years 1-3 train, year 4 tests; inputs: 6 hours of past Dst, and for ARX/MLP 7 hours of L1 VBs (one hour ahead of Earth) and dynamic pressure.

**FIGURE 47.13** Forecasting a Dst-like index from synthetic solar wind. (a) The largest test storm, beyond anything in training, with 3-hour forecasts. (b) RMSE over all test hours against lead time. (c) RMSE in storm hours. Upstream L1 data give most of the skill; a neural network adds more.

NASA's **DAGGER** model (2023), from the Frontier Development Lab, used solar-wind data from upstream spacecraft such as ACE and Wind to forecast the ground magnetic disturbance worldwide, 30 minutes ahead. It runs in under a second and updates every minute. Its authors noted that its average errors were close to persistence and that it underestimated the strongest disturbances, near 1 000 nT. Like most work in this field, it is a research demonstration.

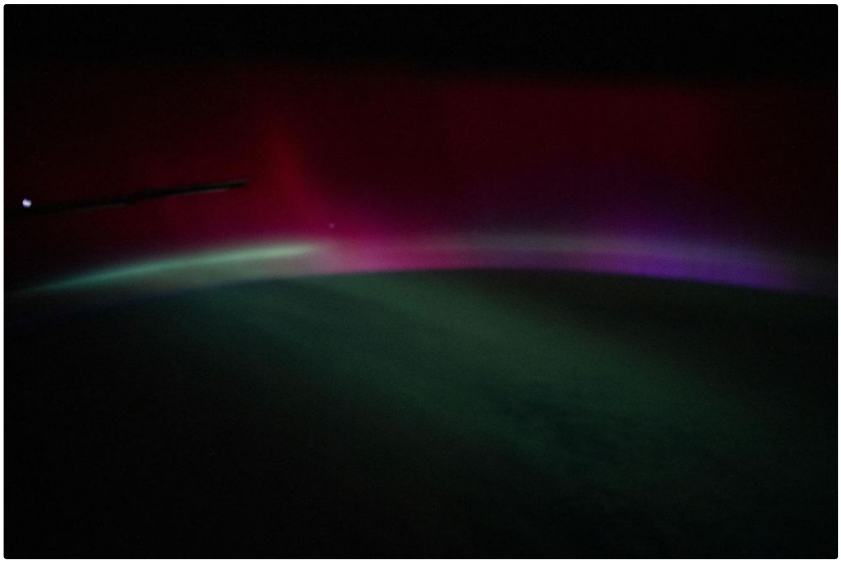


**FIGURE 47.14** An aurora with rare deep-red and purple colours over the Atlantic off Portugal, photographed from the International Space Station on 19 January 2026, during a geomagnetic storm that reached Kp 9−. Image: NASA/JSC.

**EXAMPLE: HOW MUCH WARNING CAN AN L1 MONITOR PLUS A LEARNED MODEL GIVE?**

NOAA's SOLAR-1 observatory sits near L1, about 1.5 million km sunward of the Earth. Assume 4 minutes to downlink, process the data, run a learned model (under 1 s) and issue an alert.

1. **Slow wind, 400 km/s.** Travel time $t = 1.5 \times 10^9$ m / $4 \times 10^5$ m/s = 3 750 s = 62.5 min. Warning ≈ 62.5 − 4 = 58.5 min.
2. **Fast CME, 750 km/s.** t = 2 000 s = 33.3 min; warning ≈ 29 min.
3. **Extreme CME, 1 500 km/s.** t = 1 000 s = 16.7 min; warning ≈ 13 min.
4. **What learning adds.** It cannot make the solar wind travel more slowly. It can turn the measured wind into a local forecast of ground disturbance or Kp within seconds, as DAGGER does for 30 minutes ahead at moderate speeds.
5. **Longer warnings.** A CME seen leaving the Sun gives one to four days of notice, but arrival errors of about 13 hours and the unknown $B_z$ make this a "watch", not a warning. ESA's planned Vigil spacecraft at L5 would see CMEs from the side and improve arrival times.

### 47.7.5 Ionosphere and Radiation Belts

The ionosphere delays GNSS signals in proportion to its **total electron content (TEC)** (Chapters 18 and 23). Global TEC maps from thousands of receivers form long image sequences, well suited to recurrent and convolutional networks. Studies report 1–24-hour forecasts that beat empirical models in quiet conditions but are weaker in storms, which are rare in training data. Neural networks driven by solar-wind and particle data also forecast radiation-belt electrons (Chapter 23) between the few spacecraft that measure them. Both remain mostly research.

### 47.7.6 From Research to Operations

Operational space-weather centres have been slower than weather centres to adopt learned models. NOAA's Space Weather Prediction Center (SWPC), the Met Office Space Weather Operations Centre and ESA's Space Weather Service Network rely mainly on human forecasters. Their tools are physics models, such as WSA–Enlil and NOAA's Geospace model, and empirical

rules. Learning appears in a few products, such as neural-network Kp forecasts. Three obstacles stand out: few events to validate against, the high cost of a missed storm, and the long, formal path from research to operations.

**LANDMARK: THREE DECADES OF NEURAL NETWORKS IN SPACE WEATHER**

- **Early 1990s.** A group at the Swedish Institute of Space Physics in Lund began using neural networks to predict geomagnetic indices from solar-wind data. By the early 2000s it ran real-time forecasts of Dst; its Kp and Dst forecasts are now distributed through ESA's space-weather service.
- **2010s.** SDO magnetograms enabled image-based flare forecasting. Research classifiers reached TSS near 0.8; operational comparisons showed far lower skill.
- **2023.** NASA's DAGGER forecast ground magnetic disturbances worldwide 30 minutes ahead in under a second.
- **2025.** NASA and IBM released the Surya foundation model openly. NOAA's SWFO-L1 launched with NASA's IMAP on 24 September 2025, reached L1 in January 2026 and was renamed SOLAR-1 (Figure 47.15).
- **2026.** SOLAR-1 was declared operational in mid-2026, after its CCOR-2 coronagraph in June. Operational use of learning was still limited to a few products; the bottleneck had moved from algorithms to validation and trust (Chapter 53).

**FIGURE 47.15** The Falcon 9 carrying NASA's IMAP, the Carruthers Geocorona Observatory and NOAA's SWFO-L1 (now SOLAR-1) at Launch Complex 39A on 22 September 2025, two days before launch. SOLAR-1 carries a coronagraph and solar-wind instruments for operational forecasts. Image: NASA/KSC (SpaceX).

Space-weather forecasting will probably follow the weather path, more slowly. New monitors at L1, and later L5, will supply data, and long archives with agreed benchmarks will allow fair tests. Hybrid models that keep the physics of the solar wind and learn the rest are the most likely to earn forecasters' trust.

### KEY TAKEAWAYS

- Learned weather models step the atmosphere forward 6 hours at a time and learn from ERA5 reanalysis. Since 2023 the best have matched or beaten physics models on most scores, using about a thousandth of the energy per forecast.
- ECMWF's AIFS (2025), NOAA's AIGFS, AIGEFS and HGEFS (December 2025), Germany's AICON and China's Fengqing run operationally — always beside physics models.
- Squared-error training smooths forecasts and weakens extremes; diffusion and CRPS-trained ensembles restore detail and honest uncertainty. Record extremes and cyclone intensity remain weak points.
- Learned models depend on physics-based analyses. End-to-end systems that learn from raw observations, such as Aardvark Weather and GraphDOP, are promising research.
- In the observing chain, learning helps with retrievals, bias correction, fast observation operators and nowcasting; nowcasts need generative training to keep heavy rain sharp.
- Climate emulators and downscaling are fast and useful, but learned models fail outside their training range. Digital twins such as Destination Earth combine physics and learning.
- Space weather has far less data and fewer events. Flare forecasts are compared by the true skill statistic, CME arrival errors are still about 13 hours, and L1 data plus learned models give tens of minutes of warning. Operational use is still limited.

### REVIEW QUESTIONS

1. Explain why almost all learned weather models still depend on data assimilation and satellites, even though they replace the physics model.
2. Write the autoregressive update of a learned weather model. Why does it predict the change over 6 hours rather than the full state, and why is the loss weighted by the cosine of latitude?
3. In Figure 47.3 the learned emulator beats a physics model with a 5 % forcing error, yet all forecasts reach half the climatological spread after about 5.5–5.7 days. What limits the forecast range?
4. Why does a deterministic model trained on squared error produce smooth forecasts at long lead times? Use Figure 47.4 to explain how ensembles and CRPS training address this.
5. Give two reasons why the energy saving of a learned forecast is smaller for the whole forecasting system than for the forecast step alone.
6. List four weaknesses of learned weather models and a test from the Practice box that would reveal each.
7. In Figure 47.8 the learned blend has the lowest RMSE but the worst CSI for heavy rain. Explain the difference and how DGMR avoided it.
8. Why did the neural-network climate emulator in Figure 47.9 fail in the high-forcing scenario while the linear emulator did not? What would you change in its training?
9. A flare model gives 40 hits, 10 misses, 300 false alarms and 9 650 correct negatives. Compute its POD, POFD, TSS and precision. Would a satellite operator be satisfied?
10. A CME travels from L1 to the Earth at 900 km/s. With 5 minutes of processing latency, how much warning does an L1 monitor give? Why can a CME seen leaving the Sun not give a reliable warning days ahead?

CHAPTER

# 48 AI for Autonomy, Navigation and Robotics

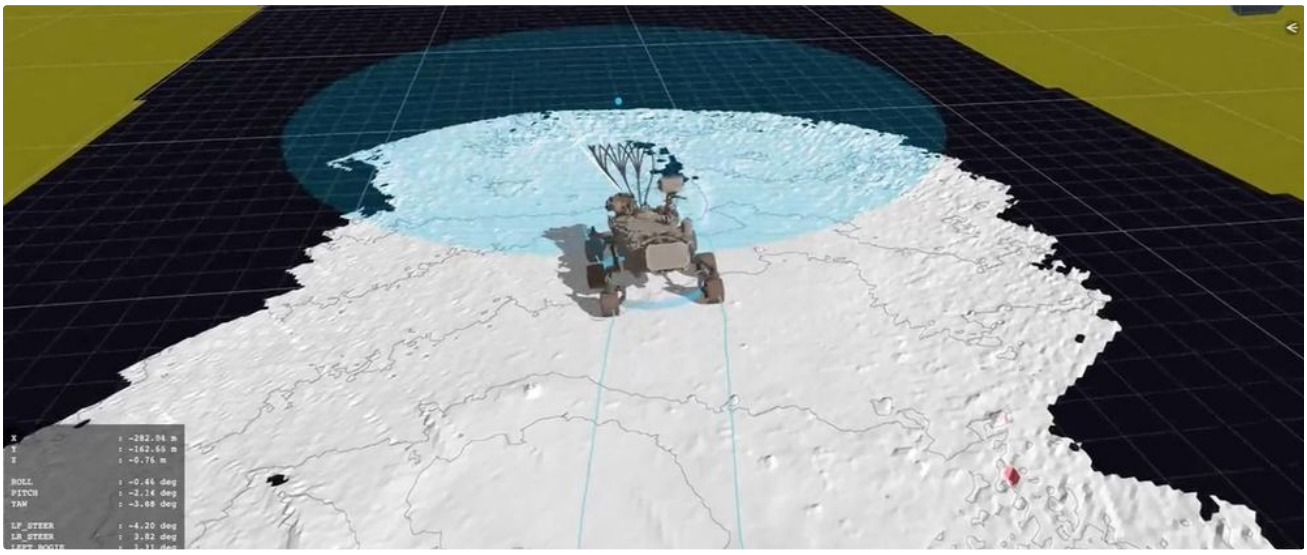


Image: NASA/JPL (NASA/JPL-Caltech)

*A spacecraft that waits for instructions from Earth cannot land on Mars, dodge debris every few minutes or grab a tumbling satellite. It must sense, decide and act on its own. This chapter explains how. It starts with why autonomy is needed and the ECSS autonomy levels, then looks inside an autonomous spacecraft: executives, fault protection, model-based diagnosis and onboard planners. It covers navigation by stars, craters and terrain maps, pose estimation for rendezvous, convex guidance and reinforcement learning, docking, rovers, robot arms, free-flyers, swarms and constellations. Small optimisation and learning experiments, run from scratch, show what each method does well and where it fails. Status is given as of October 2026.*

IN THIS CHAPTER

48.1 Why Spacecraft Need Autonomy
48.2 Inside an Autonomous Spacecraft
48.3 Onboard Planning and Scheduling
48.4 Navigation Without the Ground
48.5 Pose Estimation for Rendezvous and Servicing
48.6 Guidance and Control: Optimisation or Learning?
48.7 Rendezvous, Docking and Proximity Operations
48.8 Robots and Swarms

Every spacecraft is a robot. Most of the time it follows a script written on the ground. But some decisions cannot wait for the ground. A Mars lander has about seven minutes from the top of the atmosphere to the surface, while a radio signal takes up to 22 minutes to arrive. A rover that stops after every metre to ask for advice will never get anywhere. A fleet of ten thousand satellites meets debris many times an hour. In each case the spacecraft must sense its situation, decide what to do and act — on its own.

This chapter is about the software that makes those decisions. Much of it is not machine learning. Planners search through possible schedules, filters fuse sensor data, and optimisers compute fuel-optimal trajectories with mathematical guarantees. Learning enters where hand-written rules struggle: recognising craters, finding a satellite's corners in a camera image, or choosing a path across rough ground. The hero image above shows the idea. It is a replay of telemetry from Perseverance's first self-driving (AutoNav) drive: the rover's own three-dimensional map of the ground ahead and the path it chose.

Background is covered elsewhere: control and estimation in Chapter 6, deep-space navigation in Chapter 24, landing in Chapter 27, rovers in Chapter 28 and servicing in Chapter 40. Machine-learning basics are in Chapters 43 and 44, and onboard computers in Chapter 45.

## 48.1 Why Spacecraft Need Autonomy

The key idea is simple. A spacecraft needs autonomy whenever the ground cannot respond fast enough, often enough or cheaply enough. Four things set the limit: light-time, gaps in contact, the number of spacecraft and the speed of events.

### 48.1.1 Light-Time, Gaps and Scale

**Light-time** is the time a radio signal takes to travel one way. It is 0.12 s to geostationary orbit and 1.3 s to the Moon. To Mars it ranges from about 3 to 22 minutes, and to Jupiter from 33 to 54 minutes. Voyager 1 will be one light-day from Earth in November 2026. A question and its answer take twice as long.

**Contact gaps** often matter more. A low-orbit satellite passes over a ground station for about ten minutes at a time, a few times a day. A Mars rover talks to Earth mostly through orbiters that fly over a few times per sol (a Martian day of 24 h 39 min). Deep-space missions share NASA's Deep Space Network with dozens of others and may get a few hours of contact a day.

**Scale** is the newest driver. Over ten thousand Starlink satellites are in orbit, and other constellations are following (Chapter 17). No team of people can plan every pointing change, data dump and collision-avoidance manoeuvre for such a fleet by hand.

**Speed of events** sets the final limit. A failed thruster valve, a boulder in a landing zone or a closing piece of debris must be dealt with in seconds to hours. Autonomy also saves money, because fewer people are needed to watch routine operations.

### 48.1.2 Levels of Autonomy

The European standard on space segment operability, ECSS-E-ST-70-11C (2008, revised in October 2025), defines levels on three separate scales (Table 48.1). The **E** scale covers the execution of nominal operations. The **D** scale covers onboard storage of mission data, and the **F** scale covers what the spacecraft does after a failure. A mission states a level on each scale. A typical Earth-observation satellite might be E2, D2 and F1; a Mars rover is E3 or E4 for driving, with F2 for many faults.

**TABLE 48.1** Autonomy levels of ECSS-E-ST-70-11C for execution (E), data management (D) and failure management (F). The standard defines two data levels and two failure levels, not four.

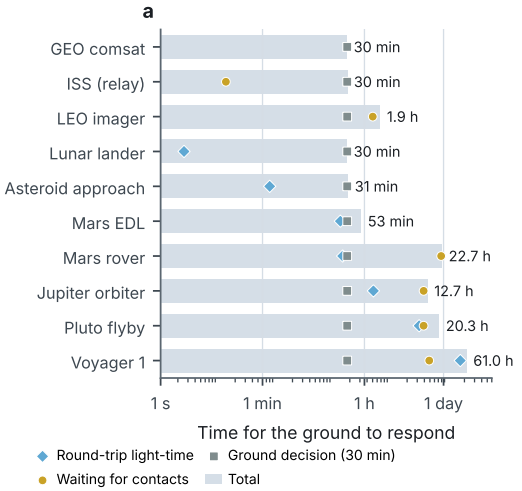
| Level | The spacecraft can... | Typical example |
|---|---|---|
| E1 | Run ground commands in real time; time-tagged safety commands | Early satellites |
| E2 | Run a pre-planned, ground-written timeline | Most Earth-observation satellites |
| E3 | React to events with onboard rules and procedures | Rover hazard avoidance |
| E4 | Re-plan on board from goals | EO-1; Perseverance's planner |
| D1 | Store essential data during a ground outage or failure | Event logs kept for the next pass |
| D2 | Store all mission data, independent of the ground | Memory sized for the longest gap |
| F1 | Detect and isolate a failure; go to a safe state | Sun-pointing safe mode |
| F2 | Recover and resume nominal operations | Deep-space probes using spare units |

### 48.1.3 How Much Autonomy Is Enough?

A simple model shows where each level is needed (Figure 48.1). Suppose an event happens on board. The ground learns of it, decides and replies. The time this takes is

$$T_{\text{ground}} = 2\tau + 2W + T_{\text{decide}}, \qquad W = \tfrac{1}{2}(1-f)^2 P,$$

where $\tau$ is the one-way light-time and $T_{\text{decide}}$ the time the team needs to decide. $W$ is the mean wait for a contact window, for a spacecraft in contact a fraction $f$ of a cycle of length $P$. The factor $(1-f)^2$ appears because an event falls outside contact with probability $1-f$ and then waits, on average, half the gap.




b

| Equipment fault | Landing guidance | Driving hazard | Science event | Plan change | Collision warning |
|---|---|---|---|---|---|
| F1 | – | – | – | E2 | E2 |
| F1 | – | – | – | E2 | E2 |
| F1 | – | – | E3 | E3 | E2 |
| **F2** | **E4** | – | – | – | – |
| **F2** | **E4** | – | E2 | – | – |
| **F2** | **E4** | – | – | – | – |
| **F2** | – | **E4** | **E4** | E3 | – |
| **F2** | – | – | E3 | E3 | E3 |
| **F2** | – | – | **E4** | E3 | – |
| **F2** | – | – | – | **E4** | – |

Model: response = 2 × one-way light-time + 2 × mean wait for a contact, (1 − f)²P/2 for contact fraction f and cycle P, + 30 min for a ground decision. (b) compares it with each event's time constant (fault, guidance 1 min; driving hazard 10 s; science event 2 h; plan change 6 h; conjunction 1 day): ratio < 0.3 ground in the loop (E2); 0.3–10 onboard reactions (E3, F1); ≥ 10 onboard re-planning (E4); F2 where ground recovery takes ≥ 6 h or in critical phases. Illustrative heuristic, not an ECSS rule.

**FIGURE 48.1** How much autonomy different missions need, from a simple model. (a) Time for the ground to respond to an event on board: round-trip light-time, waiting for contact windows and a 30-minute decision. (b) The autonomy level implied when that time is compared with how fast typical events unfold. Grey cells are events that do not apply.

The figure applies the model with a 30-minute decision time. A geostationary satellite or the space station gets an answer in about half an hour. A low-orbit imager with two polar ground stations waits about 1.9 hours, mostly for the next pass. A Mars rover waits about 23 hours, and

Voyager 1 about 61 hours. Where the ground is more than about ten times too slow, the spacecraft must re-plan for itself (E4). Equipment faults need at least a safe mode (F1) everywhere. Far from Earth, or during landing, they need full recovery on board (F2).

## 48.2 Inside an Autonomous Spacecraft

An autonomous spacecraft is not one clever program. It is a stack of simple, well-tested parts, each working on its own time scale. Learning is usually confined to the parts that interpret sensor data.

### 48.2.1 Sense, Plan, Act

Most designs follow a **three-layer architecture** (Figure 48.2). The **functional layer** runs fast control loops: attitude control, engine firing, camera exposure, and the perception software that turns images into estimates of position or hazards. The **executive layer** runs the plan. It starts activities, checks that resources and timing stay within limits, and switches modes. The **deliberative layer** thinks ahead. A planner turns goals into a timeline, science software decides what is worth observing, and navigation software chooses paths and manoeuvres.

Two functions cut across the stack. **Fault detection, isolation and recovery (FDIR)** watches for anything that goes wrong. A **runtime monitor** checks commands against simple safety rules before they reach the hardware. The ground does not disappear. It sends goals, models and rules instead of individual commands, and receives summaries and selected data instead of everything.

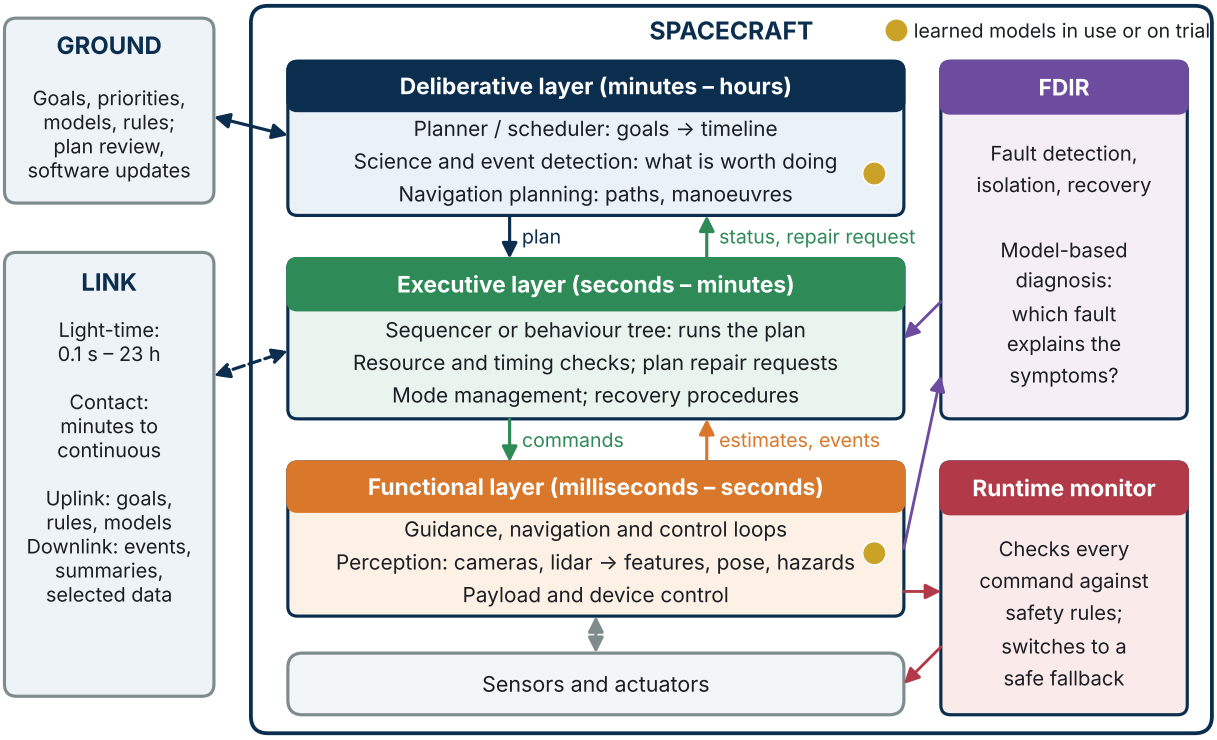



**FIGURE 48.2** Block diagram of an autonomous spacecraft. Faster layers sit lower in the stack; the ground sends goals and rules rather than commands. Gold dots mark where learned models are used or tested today: event detection and perception.

### 48.2.2 Executives, Behaviour Trees and Fault Protection

The **executive** turns a plan into action. Classic flight executives are timeline sequencers: time-tagged commands plus rules such as "if the battery falls below 30 %, stop the payload". **Behaviour trees**, taken from computer games and ground robotics, organise behaviours as a tree. Each node is "ticked" in turn and reports success, failure or running. A *sequence* node runs its children in order until one fails; a *fallback* node tries alternatives until one succeeds. Trees are easy to read and test, which is why they are increasingly proposed for space robots.

**FDIR** is the oldest form of autonomy. Monitors compare telemetry with limits. When a limit is crossed, a recovery action switches to a redundant unit or puts the spacecraft in **safe mode**: Sun-pointing, power-positive, listening for the ground. That is level F1. Level F2 goes further and resumes the mission, as deep-space probes must.

Fault protection can also cause failures, when the rules meet a situation nobody imagined. In October 2016 ESA's Schiaparelli lander saw its inertial unit saturate for about a second under the parachute. The navigation software then computed a negative altitude, released the parachute and fired the thrusters for only a few seconds. The lander fell from about 3.7 km. In April 2023 ispace's Hakuto-R lander crossed a crater rim during descent. The sudden jump in measured altitude made its software reject the good altimeter data as faulty. The lander kept descending slowly from about 5 km until its fuel ran out. Both lessons are the same: autonomy must be tested against the real terrain and sensors, not only the expected ones.

### 48.2.3 Model-Based Diagnosis

Rules say "if symptom A, then do B". They become unmanageable when faults combine. **Model-based diagnosis** takes another approach. Engineers write a model of each component and its possible modes — a valve can be *open*, *closed*, *stuck open* or *stuck closed* — and how the modes connect. The software then searches for the most probable set of modes that explains what the sensors see:

$$\hat{m} = \arg\max_{m} \; P(m \mid \text{observations}) \;\propto\; P(\text{observations} \mid m)\, P(m).$$

Faults are rare, so the prior $P(m)$ favours explanations with few failed parts. NASA's **Livingstone** engine, built at Ames Research Center in the 1990s, did this with logical models and fast search. It then planned the reconfiguration needed to work around the fault. Learned anomaly detectors on the ground (Chapter 49) now complement such models.

### 48.2.4 Twenty-Five Years of Flight Experience

Landmark flights of onboard autonomy are listed in Table 48.2. Three patterns stand out. Autonomy has grown from one-off experiments into everyday operations. Most of it is search, estimation and optimisation, not learning. And learned components have entered mainly through perception and science, such as EO-1's classifiers and rover targeting. The first in-orbit test of a learned attitude controller was reported only in 2025.

**TABLE 48.2** Landmark flights of onboard autonomy, from experiments to daily operations. "Operational" means used in normal mission operations.

| Year | Mission | Autonomous function | Status | Result |
|---|---|---|---|---|
| 1999 | Deep Space 1 | Remote Agent planning, execution, diagnosis | Flight experiment | 100 % of objectives |
| 1999–2001 | Deep Space 1 | AutoNav optical navigation | Flight demonstration | Comet Borrelly flyby, 2001 |
| 2004–17 | EO-1 | Onboard planning and event detection (ASE) | Operational | About $1 M a year saved |
| 2010– | Opportunity, Curiosity, Perseverance | AEGIS science targeting | Operational | 889 targets 2016–May 2026 |
| 2019– | Starlink | Autonomous collision avoidance | Operational | 207 152 manoeuvres in 6 months |
| 2020 | OSIRIS-REx | Natural Feature Tracking to sample site | Operational | Within about 1 m |
| 2021 | Perseverance | Lander Vision System | Operational | Touchdown 5 m from aim |
| 2021– | Perseverance | Enhanced AutoNav | Operational | > 90 % of 45 km |
| 2022 | DART | SMART Nav terminal guidance | Operational | Hit 160 m Dimorphos |
| 2023– | Perseverance | Onboard planner (from 5 Oct 2023) | Operational | Hundreds of plans |
| 2024 | SLIM | Crater-matching pinpoint landing | Flight demonstration | Under 10 m before final avoidance |
| 2025 | InnoCube | Learned attitude controller (LeLaR) | Flight experiment | First full manoeuvre 30 Oct 2025 |

#### REMOTE AGENT: AN AI IN CHARGE OF A SPACECRAFT (DEEP SPACE 1, MAY 1999)

NASA's Deep Space 1 tested twelve new technologies. One was the **Remote Agent**, the first AI software to control a spacecraft without ground commanding. It had three parts: a constraint-based planner and scheduler, a multi-threaded "Smart Executive", and Livingstone for diagnosis and recovery. The experiment ran during 17–21 May 1999, for about 35 hours in all.

A 2-day scenario started at 11:04 PDT on 17 May. Remote Agent planned and ran ion-engine thrusting and imaging. When a simulated fault left a camera stuck on, it tried to switch it off, then re-planned around the extra power use. Early on 18 May the part of the executive that issues commands stopped, so the ion engine was not turned off as planned; a backup command shut it down later. The cause was a race condition between two executive threads that had not appeared in extensive ground testing. About 70 % of the objectives had been met by then. There was no time to fix and re-test the software that week. The team instead wrote a new 6-hour scenario and ran it with the bug still on board. On 21 May that scenario completed the remaining objectives, so the experiment met 100 % of them. Its deadlock became a classic argument for formal verification of flight software.

## 48.3 Onboard Planning and Scheduling

A planner decides *what* to do; a scheduler decides *when*. In space the two are usually combined. The key idea is that a plan is a set of activities that must satisfy hard constraints — time windows, power, data volume and instrument states — while earning as much value as possible.

### 48.3.1 Plans, Constraints and Search

Written as mathematics, choosing observations is an integer programme. Each candidate observation $i$ has a value $v_i$, and $x_i$ is 1 if it is chosen and 0 if not.

$$\max_{x} \sum_{i} v_i x_i \quad \text{subject to} \quad Ax \le b, \qquad x_i \in \{0, 1\}.$$

Each row of $Ax \le b$ is one constraint. Two observations that overlap in time cannot both be chosen. The energy used in an orbit must stay within the battery budget, and data taken between downlinks must fit in memory. Such problems are **NP-hard**: no known algorithm solves every case quickly, and the worst-case effort grows exponentially with size. Flight planners therefore use three families of methods.

- **Greedy heuristics** sort candidates by a rule, such as value per unit of resource, and add each one that still fits. They are fast and predictable but can miss better combinations.
- **Local search** starts from a plan and keeps making small changes — add an observation, remove a clashing one — accepting changes that help, and sometimes ones that do not, to escape dead ends. JPL's **CASPER** planner uses a related idea called *iterative repair*: when reality departs from the plan, it fixes only the parts that conflict.
- **Exact optimisation**, such as mixed-integer linear programming (MILP) or constraint programming, proves that a plan is the best possible. It needs more computing and its run time is harder to bound.

### 48.3.2 A Toy Scheduling Problem

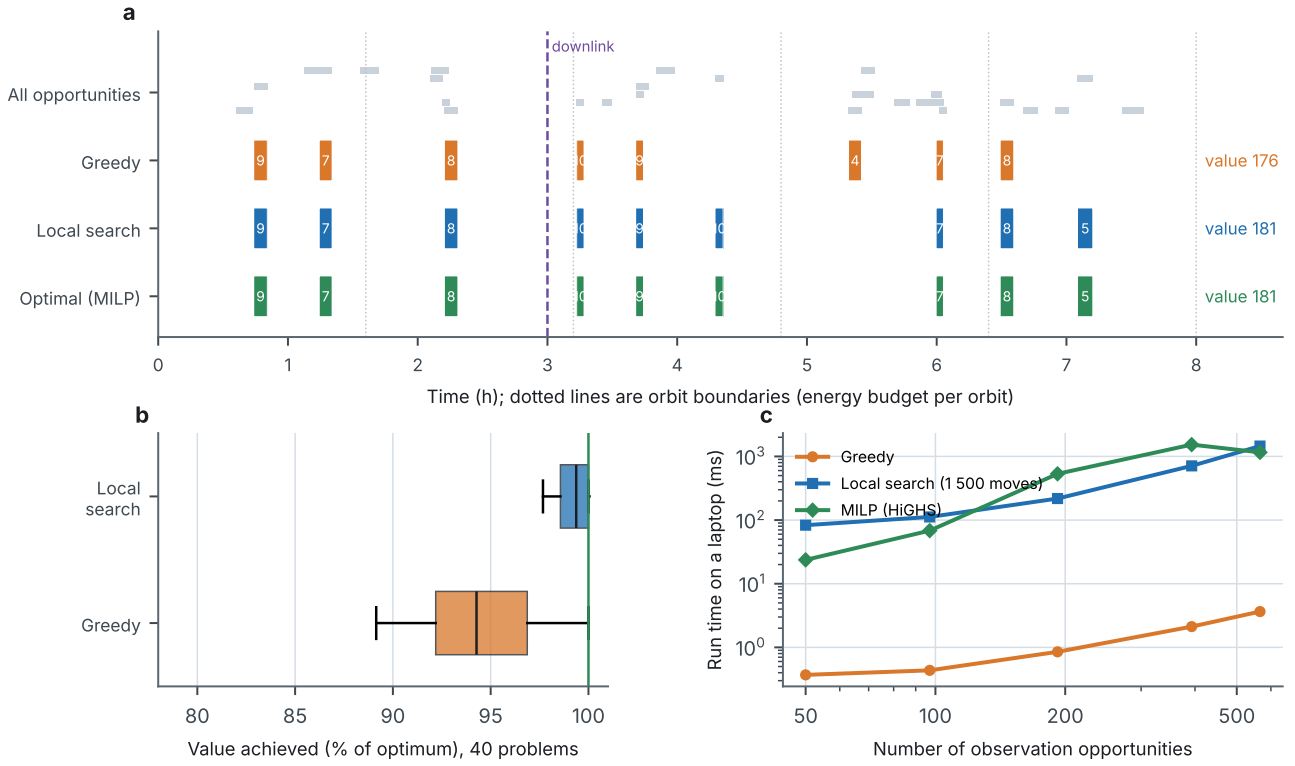
Figure 48.3 compares the three on a synthetic day for an imaging satellite. There are 45 targets, each visible one to three times, with values from 1 to 10. The satellite has one instrument with 2-minute slews, an energy budget per 96-minute orbit and a memory emptied at four downlink passes. A typical day has about 90 opportunities and just over 100 constraints.

Over 40 random days, the greedy rule reached 94.4 % of the optimal value on average and 89 % in the worst case, in under a millisecond on a laptop. Local search by **simulated annealing** reached 99.2 % on average in about a tenth of a second, and found the exact optimum on 15 of the 40 days. The exact MILP solver also took about a tenth of a second. As problems grew to about 570 opportunities, MILP took over a second and the greedy rule's shortfall stayed near 9 %. Flight computers are a hundred to a thousand times slower than a laptop. Flight planners therefore pair a fast heuristic, which always yields a safe plan, with improvement when time allows.

### 48.3.3 Planners in Flight Today

The most visible planner today runs on Perseverance. Its **onboard planner** (also called Simple Planner) was developed from 2016 and verified from October 2021, and entered operations on 5 October 2023. It is a priority-first, non-backtracking scheduler: it places activities in priority order and never undoes a placement, which keeps its run time bounded. It re-schedules when an activity ends early or late, or when energy differs from predictions, so the rover can use margin that ground planners had to hold back. In its first two years it ran hundreds of plans with very few anomalies.

Europe has tested autonomy mainly through small missions. ESA's OPS-SAT CubeSat (2019–2024) was an open flying laboratory for experiments in onboard planning and machine learning, among others. France's space agency CNES flew autonomous orbit control on the DEMETER microsatellite from 2005 and an autonomous guidance kit on PICARD in 2014. Its ASTERIA orbit-control software was later demonstrated on OPS-SAT. These steps matter because operators accept autonomy gradually, function by function, once each one has proved itself.




Synthetic day of 45 targets with 1–3 visibility windows each (3–10 min), values 1–10, one instrument with 2 min slews, payload energy ≤ 14 units per 96-min orbit, memory ≤ 40 units between four downlink passes. Exact solution by mixed-integer linear programming (scipy/HiGHS). Flight processors are 100–1 000 times slower than a laptop.

**FIGURE 48.3** Scheduling one day of observations for a synthetic imaging satellite. (a) Part of one day: all opportunities (top) and the plans chosen by a greedy rule, local search and an exact mixed-integer solver, with values. (b) Value achieved relative to the optimum over 40 random days. (c) Run time against problem size on a laptop.

THE AUTONOMOUS SCIENCECRAFT ON EO-1 (2004–2017)

NASA's Earth Observing-1 satellite, launched in 2000 to test new imagers, became the longest-running test of an onboard planner. Its **Autonomous Sciencecraft Experiment** (ASE) began operating on board in January 2004 and was declared validated in May 2004. In November 2004 it became EO-1's primary operations system. It had three parts. Onboard classifiers analysed Hyperion images for events such as eruptions, floods, fires and ice break-up. The CASPER planner re-planned observations around new goals. A robust executive ran the result.

The computer was modest: a 12 MHz Mongoose V processor of about 8 million instructions per second (MIPS), separate from the main flight computer for safety. ASE had only about 4 MIPS and 128 MB of memory to itself. CASPER's model was kept small so that it could build a plan in tens of minutes. By 2007 ASE had made over 11 000 autonomous data acquisitions and over 1 500 closed-loop re-tasks. Alerts from other satellites, such as volcano warnings, could trigger Hyperion images through ground "sensorwebs". EO-1's operations costs fell from $3.6 million in fiscal year 2005 to $1.6 million in 2006, and JPL attributed about $1 million a year of the saving to ASE. Science events captured per megabyte of downlink rose over a hundredfold. ASE remained the primary planning system until the end of the mission in 2017, after more than 60 000 autonomous observations.

## 48.4 Navigation Without the Ground

Navigation means knowing where you are and how fast you are moving. Most spacecraft are navigated from the ground by radio tracking (Chapter 24), which is accurate but slow. The key idea of autonomous navigation is to measure position against something the spacecraft can see — stars, asteroids, craters or terrain — and fuse those measurements in an onboard filter.

### 48.4.1 Optical Navigation in Deep Space

A camera measures a direction. If it sees a known asteroid against background stars, it measures the unit vector $\hat{\boldsymbol{u}} = (\mathbf{r}_a - \mathbf{r})/\|\mathbf{r}_a - \mathbf{r}\|$ from the spacecraft at $\mathbf{r}$ to the asteroid at $\mathbf{r}_a$. A Kalman filter (Chapter 6) combines such directions, taken over days, with the equations of motion to fix the orbit.

Deep Space 1's **AutoNav** was the first system to do this on its own, in 1999. It also steered the spacecraft through its flyby of comet Borrelly in September 2001. The same software kept Stardust's camera locked on comet Wild 2 during its 2004 flyby. In July 2005 it flew Deep Impact's impactor into comet Tempel 1, making the final targeting manoeuvres in the last two hours. DART's **SMART Nav** went further in September 2022. It took over about four hours before impact, picked out the 160 m moonlet Dimorphos from its larger partner only about an hour before impact, and steered into it at about 6 km/s. None of these systems used machine learning: they relied on image centroiding, geometry and filtering.

### 48.4.2 Terrain-Relative Navigation

Near a surface, the spacecraft can compare what it sees with a map. This is **terrain-relative navigation (TRN)**. It solves a problem that radio tracking cannot: an inertial unit alone may drift by kilometres during entry, while the safe spots in a rugged landing area may be only tens of metres across.

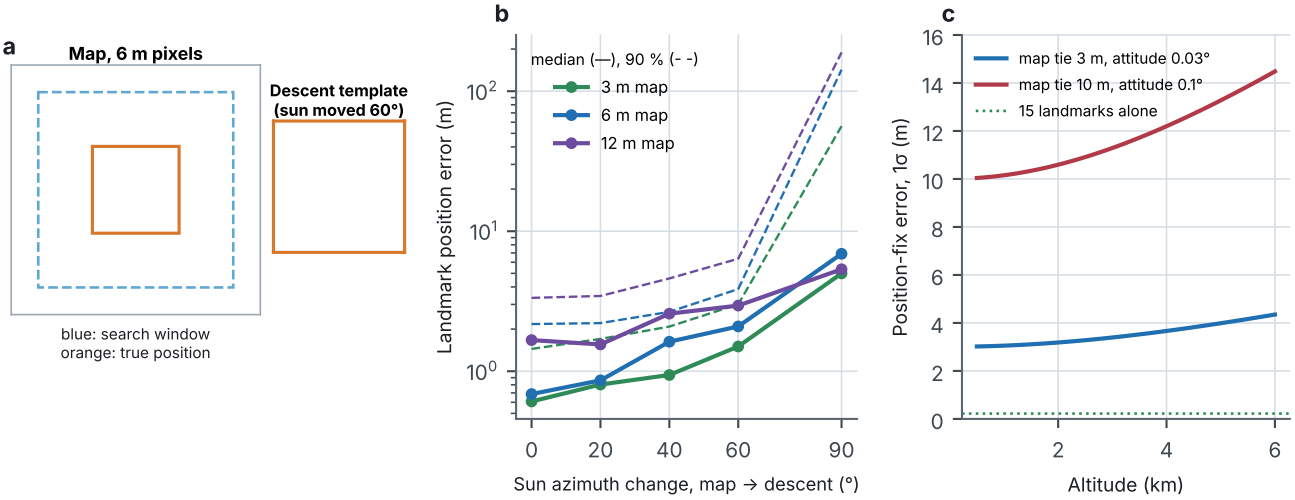
Perseverance's **Lander Vision System** (LVS) set the standard in February 2021. Its maps came from HiRISE orbital images, resampled to 12 m pixels for a coarse search and 6 m for a fine one. During descent, LVS first matched 15 landmarks across three images, correlating 128 × 128-pixel patches against large map windows in the frequency domain. It then matched up to 150 small landmarks per image and fused them with inertial data. This cut the position error from up to 3.2 km to the required 40 m or less. LVS then chose the safest reachable spot from an onboard hazard map. Perseverance touched down within 5 m of the point LVS aimed for.

The matching step uses **normalised cross-correlation**. For a template $t$ from the descent image and a map patch $m$ at offset $(i, j)$,

$$\rho(i,j) = \frac{\sum_k (t_k - \bar{t})(m_{k+ij} - \bar{m}_{ij})}{N\,\sigma_t\,\sigma_{m,ij}},$$

where the sums run over the $N$ pixels of the template. The offset with the highest $\rho$ is the match. Because $\rho$ removes the mean and scales by the spread, it tolerates changes in brightness and contrast. It does not tolerate changes in shadow direction, because shadows move when the Sun moves.

Figure 48.4 tests this on synthetic terrain. With the Sun in the same place for the map and the descent image, 32 × 32-pixel templates matched a 6 m map well. The median error was 0.7 m, about a tenth of a pixel. When the Sun had moved 60° in azimuth, the median grew to 2.1 m. At 90° a third of matches failed outright. Real systems therefore plan landing times to match the lighting of the map, and reject outliers. Panel (c) shows the bigger lesson: with many landmarks, the matching error is not the limit. Errors in tying the map to the inertial frame, and in knowing the camera's attitude, dominate the final fix.




Synthetic 3 km terrain (fractal relief, 1 200 craters, albedo noise) rendered at 25° sun elevation; 32 × 32-pixel descent templates matched by normalised cross-correlation in a ±20-pixel window, 40 trials per point. (c) Root-sum-square of landmark error/√N (6 m map, 20° sun change), map registration and attitude-knowledge error projected over the altitude.

**FIGURE 48.4** Terrain-relative navigation on synthetic terrain. (a) A 6 m map with a 32 × 32-pixel descent template and its search window. (b) Error of single landmark matches as the Sun's azimuth changes between map and descent; dashed lines show the 90th percentile. (c) A position-fix error budget against altitude: the map tie and attitude knowledge dominate.

**EXAMPLE: HOW ACCURATE CAN TERRAIN-RELATIVE NAVIGATION BE?**

A lander carries a 6 m-pixel map tied to the inertial frame with 3 m (1σ) error. Its attitude is known to 0.03°. At 4 km altitude it matches $N = 15$ landmarks, each with the median error of Figure 48.4 for a modest lighting change: 0.86 m, or 0.14 pixel.

1. **Landmarks.** Averaging $N$ independent matches gives $0.86/\sqrt{15} = \mathbf{0.22\ m}$.
2. **Attitude.** A pointing error $\delta\theta$ shifts the projected position by $h\,\delta\theta = 4\,000\ \text{m} \times 5.2 \times 10^{-4}\ \text{rad} = \mathbf{2.09\ m}$.
3. **Map tie.** The map itself may sit 3 m from where the navigation frame thinks it is: **3.0 m**.
4. **Total.** Independent errors add in root-sum-square: $\sqrt{0.22^2 + 2.09^2 + 3.0^2} = \mathbf{3.7\ m}$.

The answer is about ten times better than the 40 m requirement LVS had to meet, and matches its 5 m touchdown. Doubling the number of landmarks hardly helps. Halving the map-tie error helps most. With a 10 m map tie and 0.1° attitude knowledge, the same matcher gives 12 m.

Other landers use similar ideas. China's Chang'e landers have hovered about 100 m above the Moon since 2013 to choose a safe spot with lidar and cameras (Chapter 27). Japan's SLIM matched craters in its descent images with an onboard crater map in January 2024. All 14 of its image-matching fixes were judged accurate, and its position error before the final hazard avoidance was under 10 m. NASA's **SPLICE** project packages TRN, hazard-detection lidar, a Navigation Doppler Lidar and a descent computer for commercial landers. On Intuitive Machines' IM-1 in February 2024, a safety switch left the lander's own laser rangefinders disabled. A software patch to use NASA's Doppler lidar instead reportedly failed for want of a data flag, and Odysseus landed on its side. In June 2025 ispace's second lander crashed after its laser rangefinder reported late. In these recent failures the trouble lay in sensors and in the logic that judges their data, not in the navigation algorithms.

### 48.4.3 Crater Detection with Learning

Craters are the natural landmarks of airless worlds. A crater-based navigator detects craters in the descent image, matches the pattern with a crater catalogue and solves for position. Classical detectors look for the bright-and-dark pair that a bowl makes in low sunlight, or correlate the image with crater templates. Learned detectors instead train a network on many labelled examples (Chapter 44). The best-known example, **DeepMoon** (2019), trained a U-Net on lunar elevation maps. It recovered 92 % of the craters in a human-made catalogue, roughly doubled the number of detections, and had an estimated 11 % false-positive rate among new finds.

Figure 48.5 compares the two approaches on synthetic scenes. The template matcher was given an ideal crater rendered at the *true* Sun angle. The learned detector was a small numpy neural network with about 16 500 weights, trained in seconds on 29 689 patches from ten other scenes under random lighting. At a low Sun of 10°, both found craters equally well: average precision (the area under the precision–recall curve) was 0.56 and 0.62. As the Sun rose, shadows shrank and eroded craters faded. At 25° the template matcher dropped to 0.28, and at 45° to 0.09, while the network kept 0.57 and 0.34. At 65° both failed.

What matters for navigation is the **position fix**. Matching the 40 best detections to the catalogue with RANSAC gave errors of 0.2–0.8 m with templates at 10°. The network's fixes were slightly less precise, 0.5–1.7 m, because its sliding window is coarser. Template-based fixes locked onto wrong

matches, more than 100 m off, in one of three scenes at 25° and in all scenes from 45° upwards. The network's fixes stayed within 3 m to 45° and mostly within 10 m at 65°. The learned detector was not more accurate, but it was more *robust*, because it had seen many lighting conditions. Synthetic tests flatter both methods; real images add blur and craters missing from the catalogue.

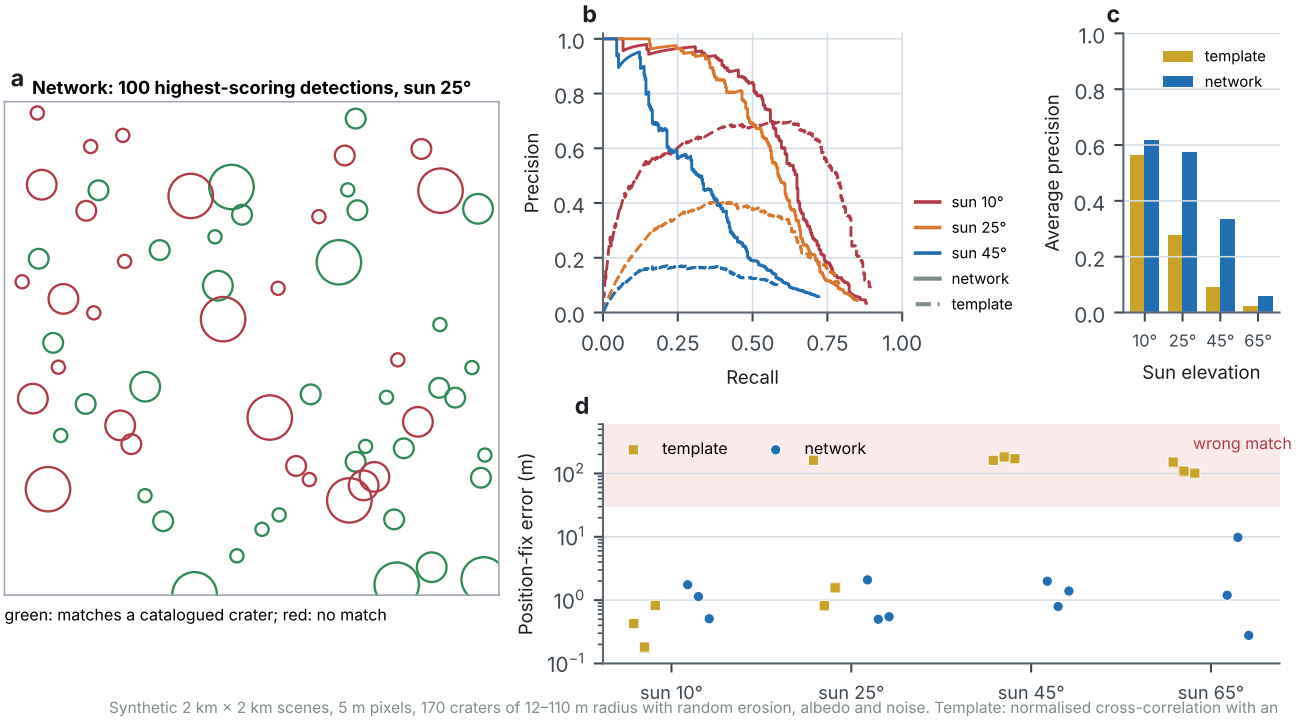
### 48.4.4 Small Bodies

Asteroids and comets are hard targets. Their gravity is weak and irregular, and maps must be built on arrival. **OSIRIS-REx** navigated to the Nightingale site on Bennu in October 2020 with **Natural Feature Tracking**. The spacecraft rendered the expected terrain from an onboard shape model and correlated it with its camera images. It touched down within about 1 m of its target. Japan's **Hayabusa2** dropped reflective target markers on Ryugu and homed in on them. Its second touchdown, in July 2019, was 60 cm from the aim point. ESA's Hera, due at Didymos in November 2026, relies on an autonomous "self-driving" navigation system around the pair.

Table 48.3 summarises the main onboard navigation systems.

**TABLE 48.3** Onboard navigation systems that have flown. None used machine learning in its flight loop.

| System | Mission | Sensor | Method | Reported accuracy |
|---|---|---|---|---|
| AutoNav | DS1, Stardust, Deep Impact | Camera | Asteroid and comet directions, filter | Comet impact, 2005 |
| SMART Nav | DART, 2022 | DRACO camera | Target centroiding, onboard guidance | Hit 160 m moonlet |
| Natural Feature Tracking | OSIRIS-REx, 2020 | Navigation camera | Rendered-template correlation | About 1 m |
| Target markers | Hayabusa2, 2019 | Camera, flash, rangefinder | Marker tracking | 60 cm |
| Lander Vision System | Perseverance, 2021 | Camera, inertial unit | Landmark correlation with maps | Within 5 m of aim |
| Hover and avoid | Chang'e 3–6, 2013–24 | Lidar, cameras | Hazard map at about 100 m | Safe landings |
| Crater matching | SLIM, 2024 | Camera | Crater detection and catalogue match | Under 10 m before avoidance |




Synthetic 2 km × 2 km scenes, 5 m pixels, 170 craters of 12–110 m radius with random erosion, albedo and noise. Template: normalised cross-correlation with an ideal crater rendered at the true sun angle. Network: 256–64–1 perceptron trained on patches from ten other scenes under random lighting. (d) Position fix by RANSAC matching of the 40 best detections to the crater catalogue, three test scenes per sun angle.

**FIGURE 48.5** Crater detection on synthetic lunar scenes with 5 m pixels. (a) The 100 highest-scoring detections of a small neural network at 25° Sun elevation; green circles match a catalogued crater. (b) Precision–recall curves for the network (solid) and template matching at the true Sun angle (dashed). (c) Average precision against Sun elevation. (d) Error of the position fix obtained by matching detections to the crater catalogue; the shaded band marks wrong matches.

## 48.5 Pose Estimation for Rendezvous and Servicing

To dock with, inspect or capture another spacecraft, a chaser must know the target's **pose**: its position and its attitude relative to the camera (Chapter 40). A cooperative target carries markers, reflectors or radio beacons. A non-cooperative one — a dead satellite or a rocket stage — does not.

Its pose must be estimated from images and the known shape alone, under harsh lighting that changes every few minutes.

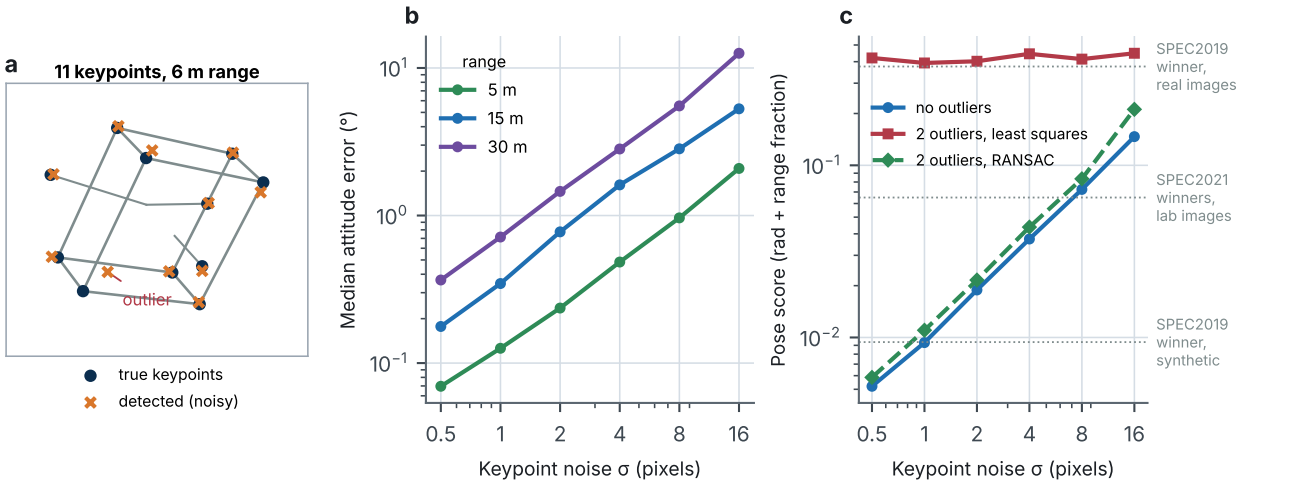
### 48.5.1 Keypoints and Perspective-n-Point

The most successful learned approach splits the task in two. First, a convolutional network finds the target in the image and predicts the pixel positions of a few **keypoints** — corners, antenna tips, nozzle centres — whose 3-D positions on the target are known. Second, a geometric solver recovers the pose. A point $\mathbf{X}_i$ on the target appears in the image at

$$\mathbf{u}_i = \pi\big(K(R\,\mathbf{X}_i + \mathbf{t})\big),$$

where $R$ and $\mathbf{t}$ are the rotation and translation, $K$ holds the camera's focal length and centre, and $\pi$ divides by depth. The **Perspective-n-Point (PnP)** solver finds the $R$ and $\mathbf{t}$ that minimise the squared reprojection error $\sum_i \|\mathbf{u}_i - \hat{\mathbf{u}}_i\|^2$. **RANSAC** (random sample consensus) protects it from wrong keypoints. It fits poses to many small random subsets, keeps the pose that most points agree with, and refits on those "inliers". This split is popular because the network does what networks are good at — recognising image patterns — and the geometry stays exact and checkable.

Keypoint errors become pose errors as shown in Figure 48.6, for a camera like the one used in the Stanford–ESA challenges and an 11-keypoint target. At 15 m range, keypoint noise of 1 pixel gave a median attitude error of 0.35° and a position error of 0.2 % of range. At 8 pixels the attitude error grew to about 3°. Doubling the range roughly doubled the error, because the target covers fewer pixels. Two badly misplaced keypoints out of 11 ruined a plain least-squares fit, giving a pose score of about 0.4. RANSAC restored almost the full accuracy.




Monte Carlo, 120 random attitudes per point; SPEED-like camera (focal length 3 003 pixels); 11 keypoints on a 0.8 m body with antennas; DLT and POSIT initialisation, Levenberg–Marquardt refinement, RANSAC on 5-point POSIT samples. (c) at 15 m; mean score as in the ESA–Stanford challenges, position error capped at 100 %.

**FIGURE 48.6** Pose from keypoints by Perspective-n-Point. (a) True and noisy keypoints on a satellite-like target, with one outlier. (b) Median attitude error against keypoint noise at three ranges. (c) Mean pose score at 15 m, with and without two outliers and with RANSAC; dotted lines mark the winning scores of the 2019 and 2021 challenges.

### 48.5.2 The ESA–Stanford Challenges and the Domain Gap

Two open competitions shaped this field. In the **Satellite Pose Estimation Challenge** of 2019, run by ESA's Advanced Concepts Team and Stanford's Space Rendezvous Laboratory, 48 teams estimated the pose of the PRISMA mission's Tango satellite. The data set held about 15 000

rendered images and 305 laboratory photographs of a mock-up. The score adds the attitude error in radians to the position error divided by range. The winner, from the University of Adelaide, used keypoints and PnP. It scored 0.0094 on rendered test images, about 0.4° and 3 cm. On real photographs its score was 0.375, forty times worse.

That jump is the **domain gap**, also called the sim-to-real gap: networks trained only on rendered images meet real images with different noise, reflections and lighting (Chapter 45). The 2021 challenge attacked it directly. Teams trained on 59 960 rendered images and were tested on 9 531 hardware-in-the-loop images taken with two kinds of lighting: a “lightbox” imitating Earth glow and a “sunlamp” imitating direct sunlight. With heavy data augmentation, style randomisation and adaptation methods, the winning scores were 0.073 (lightbox) and 0.059 (sunlamp). By Figure 48.6, such scores correspond to six to eight pixels of keypoint noise at 15 m, against about one pixel for the 2019 winner on rendered images.

### 48.5.3 Lidar and Sensor Fusion

Cameras fail in darkness and glare. **Lidar** measures range directly by timing laser pulses. Its point cloud is matched to the target’s 3-D model, often with the **iterative closest point** algorithm, which pairs each measured point with the nearest model point and solves for the best rigid motion. Lidar works at any Sun angle but has fewer pixels and costs more power. Crew Dragon, ATV and the commercial servicers combine laser sensors with cameras.

## 48.6 Guidance and Control: Optimisation or Learning?

Guidance decides the path; control makes the vehicle follow it. The key message of this section is that the most impressive autonomous guidance in flight today is **optimisation**, not learning. Learning is promising but much harder to verify.

### 48.6.1 Convex Guidance

A powered landing must reach a target with zero velocity, using as little propellant as possible, without exceeding thrust limits or flying into the ground. The thrust magnitude has a lower bound as well as an upper one, because rocket engines cannot be throttled to zero. That lower bound makes the problem non-convex — it can have many local optima — and hard to solve reliably on board.

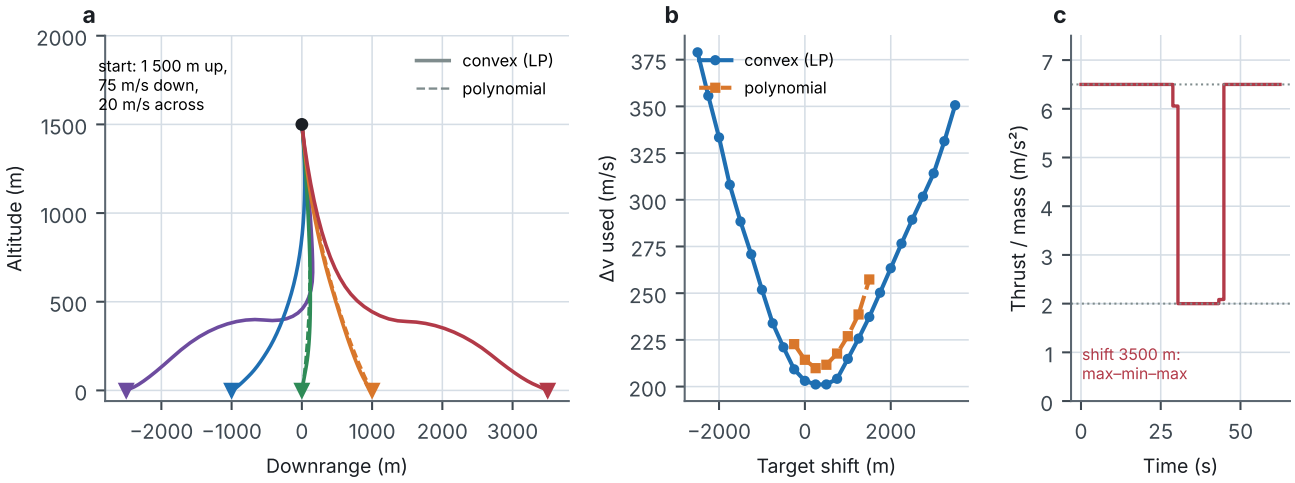
In 2007 Behçet Açıkmeşe and Scott Ploen at JPL showed a way out, called **lossless convexification**. They introduced a slack variable $\sigma$ for the thrust acceleration magnitude and wrote the constraints as

$$\|\mathbf{u}(t)\| \le \sigma(t), \qquad \rho_{\min} \le \sigma(t) \le \rho_{\max}, \qquad \min \int_0^{t_f} \sigma(t)\, dt.$$

This relaxed problem is convex, so a solver finds its global optimum reliably and quickly. They proved that at the optimum $\|\mathbf{u}\| = \sigma$, so the relaxation loses nothing. The resulting **G-FOLD** algorithm (guidance for fuel-optimal large diverts) was flown on Masten’s Xombie rocket in 2012–13. In July 2013 it diverted 750 m sideways at about 25 m/s. Its custom solver could compute a

trajectory in about 100 ms on a 1.4 GHz processor. SpaceX engineers have described using convex optimisation to land Falcon 9 boosters. This is not machine learning: nothing is learned from data, and every answer can be checked against the constraints.

A 2-D version for a Mars landing can be solved by linear programming, approximating the circle $\|\mathbf{u}\| \le \sigma$ by a 16-sided polygon (Figure 48.7). From 1 500 m altitude and 75 m/s descent, the convex method could reach targets from 2.5 km behind to 3.5 km ahead. A classical polynomial guidance law, with acceleration linear in time, could reach only from 250 m behind to 1.5 km ahead. Where both worked, the polynomial law used 4–8 % more Δv. The optimal thrust profile is "max–min–max": brake hard, coast at minimum thrust, then brake hard again.




2-D Mars landing (g = 3.71 m/s²), thrust acceleration 2–6.5 m/s², within 60° of vertical, 10° glide slope. Convex: 40-step discretisation, ‖u‖ ≤ σ by a 16-sided polygon, minimise Σσ Δt by linear programming (HiGHS) with a search over flight time. Polynomial: acceleration linear in time, best feasible flight time.

**FIGURE 48.7** Fuel-optimal powered descent solved by linear programming (lossless convexification) compared with polynomial guidance. (a) Trajectories to targets at different distances; dashed lines show polynomial guidance where it is feasible. (b) Δv against target shift: convex guidance reaches a three times wider span of targets. (c) The optimal thrust profile for the largest shift.

## 48.6.2 Reinforcement Learning

**Reinforcement learning (RL)** learns a control **policy** $\pi(a \mid s)$ — what action $a$ to take in state $s$ — by trial and error in a simulator. Each trial earns rewards $r_t$, and learning maximises the expected **return**

$$J(\pi) = \mathbb{E}\Big[\sum_{t=0}^{T} \gamma^t r_t\Big],$$

where $\gamma \le 1$ discounts distant rewards. Policy-gradient methods nudge the policy's parameters in the direction that raises $J$. Simpler **policy-search** methods such as the cross-entropy method try many random parameter sets, keep the best few and sample around them. RL's appeal is that it needs no explicit model of the optimal solution, only a simulator and a reward.

In Figure 48.8 a small neural-network policy, with 12 hidden units and 49 weights, learns to land on the Moon in one dimension. Training used the cross-entropy method: 60 generations, 64 candidate policies each and 24 simulated landings per candidate — 92 160 landings in about 7 seconds. From generation 8 every landing in a validation set was soft (below 1.5 m/s). The policy

then kept improving its fuel use, from 58 m/s of Δv to 31 m/s. A simple proportional–derivative (PD) controller that tracks a speed profile used 48 m/s. The learned policy had discovered a near-optimal "suicide burn": fall fast, then brake at full thrust at the last moment.

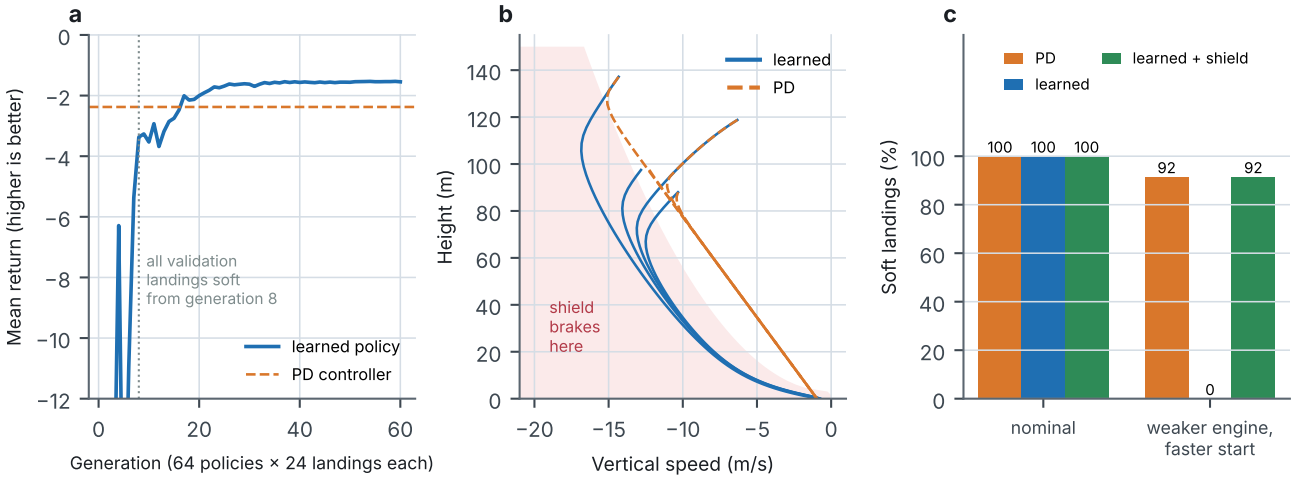
That efficiency is also its weakness. With 20 % less thrust than expected and faster starting speeds, the PD controller still landed softly 92 % of the time. The learned policy never did: its late braking left no margin. Research on landing, docking and attitude control shows the same pattern. RL policies match or beat classical controllers inside their training conditions but can fail abruptly outside them.

Real flights are beginning. In October 2025 the University of Würzburg's **InnoCube** CubeSat performed a full attitude manoeuvre with a controller trained by deep RL in simulation. Its team reported this LeLaR experiment as the first in-orbit demonstration of an AI-based attitude controller. On the ISS in February 2025, Stanford researchers used a neural network to give the Astrobee robot's trajectory optimiser a starting guess. Solver iterations fell by 50–60 %, and the optimiser still enforced every constraint. Learning that speeds up an optimiser, rather than replacing it, is the most likely route into flight.

### 48.6.3 Verifying Learned Controllers

Classical controllers come with proofs such as stability margins and worst-case bounds. A neural-network policy has none by default. It may exploit flaws in the simulator and behave strangely in states it never saw. Testing can show the presence of failures but not their absence.

**Runtime assurance** offers a practical answer. In the *simplex* architecture, an untrusted high-performance controller runs inside a "safety envelope". A simple monitor checks every command. If the state approaches the edge of the envelope, a verified backup controller takes over. In Figure 48.8 the monitor computes the stopping distance at 70 % of rated thrust and orders full braking when it approaches the height. With this **shield**, the learned policy landed softly 92 % of the time in the shifted test, as often as the PD controller. The price was fuel: the shield acted in every landing, raising the nominal Δv from 31 to 42 m/s. Assurance methods, from formal verification of networks to simplex designs, are covered in Chapter 53.




1-D lunar landing (g = 1.62 m/s², thrust acceleration 0–4 m/s², 0.2 s steps) from 80–150 m at 5–15 m/s descent; soft landing ≤ 1.5 m/s. Policy: 2–12–1 network trained by the cross-entropy method on return = −Δv/20 − 2 × excess touchdown speed. Shifted test: 20 % less thrust, start up to 20 m/s. Shield: full braking when the stopping distance at 70 % of rated thrust nears the height.

**FIGURE 48.8** Learning to land, in one dimension on the Moon. (a) Return of a neural-network policy trained by the cross-entropy method, with a PD controller for comparison. (b) Height against speed for four landings; in the shaded region a runtime monitor would order full braking. (c) Soft landings in nominal conditions and with a weaker engine and faster starts, for the PD controller, the learned policy and the learned policy with a shield.

Table 48.4 compares the three approaches.

**TABLE 48.4** Ways to compute guidance and control on board, from fixed laws to learned policies.

| Approach | How it decides | Guarantees | Flight status (2026) | Main risk |
|---|---|---|---|---|
| Classical (PD, LQR) | Fixed feedback law | Stability margins | Universal | Wastes fuel and envelope |
| Convex optimisation | Solves an optimisation on board | Global optimum; checkable | Rocket landing; G-FOLD tests | Solver time; model errors |
| Learned policy (RL) | Network trained in simulation | Statistical testing only | First flight tests (2025) | Fails outside training |
| Learned + runtime assurance | Network inside a monitored envelope | From verified monitor and backup | Research | Conservative envelope |
| Learned warm start | Network guesses, optimiser solves | Optimiser's guarantees kept | ISS test (2025) | A bad guess slows the solver |

## 48.7 Rendezvous, Docking and Proximity Operations

Automated docking is the oldest everyday autonomy in human spaceflight. The key idea is a strict sequence of hold points and approach corridors (Chapter 40), with the chaser flying itself and humans or the ground able to stop it at any time.

Soviet spacecraft have docked automatically since 1967, with the Igla system and, from 1986, the Kurs radio system still used by Soyuz and Progress. Europe's five **Automated Transfer Vehicles** (2008–2014) docked with the ISS using relative GPS, then laser rangefinders and a camera-based "videometer" reading retroreflectors (Figure 48.9). **Crew Dragon** docked itself for the first time in March 2019, using lidar and visible and thermal cameras. China has made automated rendezvous fast. Tianzhou-5 docked with the Tiangong station 2 hours 7 minutes after launch in November 2022, a world record. Shenzhou-21 brought its crew in about 3.5 hours in 2025.

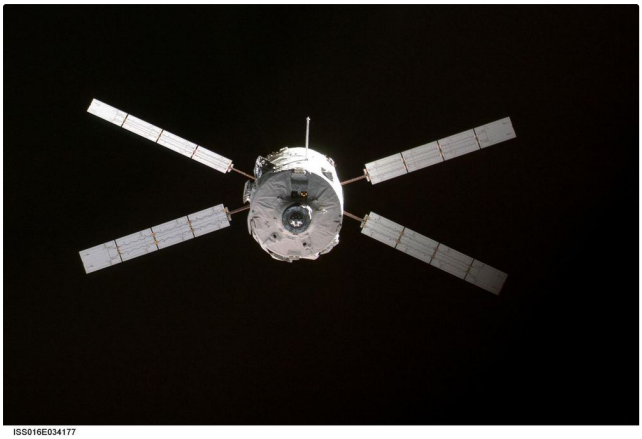



**FIGURE 48.9** ESA's first Automated Transfer Vehicle, Jules Verne, approaching the ISS during a rehearsal of its automated approach on 31 March 2008. Image: NASA/JSC.

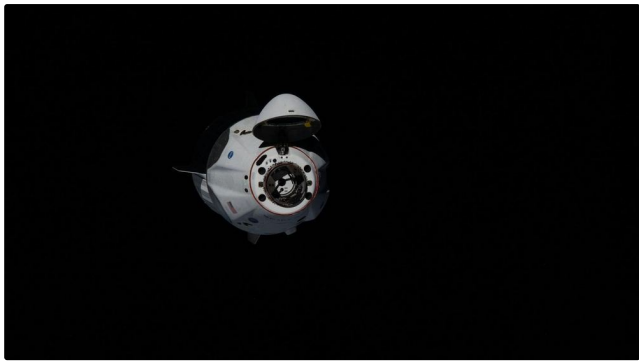


**FIGURE 48.10** Crew Dragon on its first crewed approach to the ISS on 31 May 2020, nose cone open to show the docking mechanism. Image: NASA/JSC (Chris Cassidy).

Servicing adds non-cooperative targets. DARPA's Orbital Express captured its partner autonomously in 2007, and Northrop Grumman's MEV-1 docked with Intelsat 901 in 2020. Astroscale's ADRAS-J approached a spent rocket stage to about 15 m in 2024. Learned pose estimation is the main entry point for AI here. The systems are compared in Table 48.5.

**TABLE 48.5** Automated rendezvous and docking systems. Most combine onboard control with human go/no-go decisions at hold points.

| Vehicle | First automated docking | Relative sensors | Who decides | Typical time to dock |
|---|---|---|---|---|
| Soyuz, Progress | 1967 (Igla); Kurs from 1986 | Radio (Kurs) | Onboard; crew backup (TORU) | 3 h – 2 days |
| ATV | 2008 | Relative GPS, laser, videometer | Onboard; ground can abort | About 10 days (ATV-4) |
| Crew Dragon | 2019 | Lidar, visible and thermal cameras | Onboard; crew monitors | About 1 day |
| Shenzhou, Tianzhou | 2011 | Radar, laser, cameras | Onboard; manual backup | 2 h 7 min (record) to 2 days |
| MEV-1, MEV-2 | 2020 | Cameras, infrared, lidar | Onboard; ground go/no-go | Months (slow orbit raising) |
| ADRAS-J | 2024 (approach only) | Cameras, lidar | Onboard; ground holds | Months |

## 48.8 Robots and Swarms

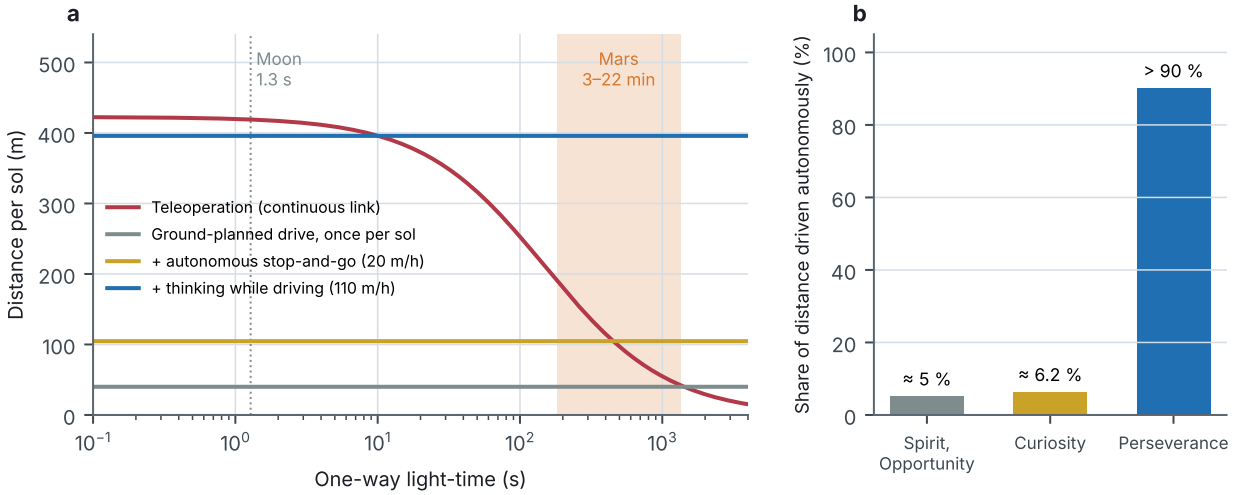
Robots are where autonomy is most visible. A robot's usefulness grows with the distance it can safely go, or the work it can safely do, between two contacts with its operators.

### 48.8.1 Rover Autonomy

A Mars rover drives in three ways (Chapter 28). In a **blind drive**, operators plan a path through terrain they have seen in stereo images, and the rover follows it with **visual odometry** — tracking features between images to measure its true motion. In **AutoNav**, the rover builds its own 3-D map, scores each patch for slope, rocks and steps, and chooses paths towards a goal.

Perseverance's **Enhanced AutoNav** adds "thinking while driving". A dedicated vision processor makes the maps while the wheels turn. An algorithm called Approximate Clearance Evaluation checks whether the rover's body can clear the rocks along each candidate path.

The benefit is large (Figure 48.11). Spirit and Opportunity used their early AutoNav for about 5 % of their driving, and Curiosity for about 6 %. Perseverance used AutoNav for more than 90 %. On 11 October 2026 it passed Opportunity's off-world driving record, reaching 45.22 km in about five and a half years. Opportunity needed more than fourteen years for the same distance. Perseverance can cover up to 412 m in a sol.




(a) Model: 3.5 h drive window per sol; top speed 4.2 cm/s; teleoperation in 10 m steps, each waiting a round trip plus 60 s of operator time; ground plans 40 m blind per sol; autonomous average speeds assumed. (b) NASA and IEEE Spectrum figures to 2024–26.

**FIGURE 48.11** Why rovers drive themselves. (a) Modelled distance per sol against one-way light-time for teleoperation with a continuous link, a single ground-planned drive per sol, and two kinds of onboard autonomy. (b) Share of distance driven autonomously by NASA's Mars rovers.

### EXAMPLE: HOW MUCH AUTONOMY DOES A MARS ROVER NEED?

Assume a one-way light-time of 20 minutes and one uplink of commands per sol. The rover can drive for 3.5 hours a day at a top speed of 4.2 cm/s.

1. **Teleoperation.** Even with a continuous link, each 10 m step takes 238 s of driving and 2 400 s of round-trip light-time. Add a minute to decide, and a step takes 2 698 s. In 3.5 hours that is 4.7 steps, or **47 m**.
2. **Ground planning.** With one uplink per sol, operators can plan only as far as their images show, about **40 m** of blind driving.
3. **Onboard autonomy.** At an average of 110 m/h while thinking while driving, the rest of the window adds about 356 m, giving about **400 m** per sol.
4. **Decisions.** If AutoNav assesses the terrain about once per metre, a 400 m drive needs about 400 hazard decisions. The ground can make one set of decisions per sol.
5. **Faults.** If a fault happens just after the morning uplink, the team may not hear of it for several hours, until the next relay pass. The rover must make itself safe (F1) and often continue (F2).

So driving needs event-driven autonomy (E3), resource-aware re-planning helps (E4), and fault handling must be F1–F2. To cross 10 km, ground planning alone would need about 250 driving sols; autonomy needs about 25.

Autonomy also chooses what to study. **AEGIS** (Autonomous Exploration for Gathering Increased Science) finds rocks in navigation-camera images after a drive and points a laser spectrometer at the best one, without waiting for the ground (Figure 48.12). It was first demonstrated on Opportunity in 2010. On Curiosity from 2016 it selected the scientists' preferred kind of target over 93 % of the time, against about 24 % expected without it. By May 2026 it had picked 596 targets on Curiosity and 293 on Perseverance.

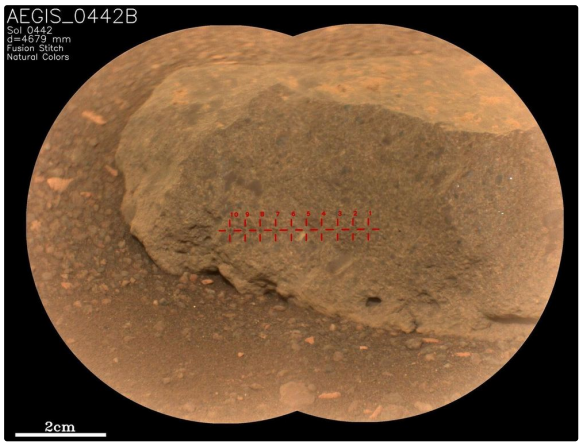



**FIGURE 48.12** A rock targeted by AEGIS on Perseverance on 18 May 2022, one of the first two chosen by the software for SuperCam without input from Earth. The red marks show the laser spots. Image: NASA/JPL (NASA/JPL-Caltech/LANL/CNES/IRAP).

The newest step moves AI into route planning on the ground. On 8 and 10 December 2025, Perseverance drove 210 m and 246 m along waypoints chosen by generative vision-language models. The models worked from orbital images and terrain-slope maps. Before upload, engineers checked the commands in JPL's digital twin of the rover, verifying more than 500 000 telemetry variables.

### 48.8.2 Arms, Free-Flyers and Humanoids

Robot arms in space are mostly supervised rather than autonomous. **Canadarm2** (2001) and **Dextre** (2008) on the ISS are driven by ground controllers or crew. Ground teams run pre-planned, automatic motion sequences for routine jobs, while astronauts usually fly the capture of cargo ships. Canada's **Canadarm3** was designed for the Gateway station. After NASA paused Gateway in March 2026, its builder MDA Space continued under its Canadian Space Agency contract, refocused on lunar-surface logistics, with launch no earlier than 2029.

Inside the ISS, three cube-shaped **Astrobee** free-flyers have operated since 2019. They navigate by matching camera images to a map of the station, move with electric fans and serve as a test bed for autonomy research (Figure 48.13). **Robonaut 2**, the first humanoid robot in space, worked on the ISS from 2011 to 2018. It was mostly teleoperated, testing whether a dexterous robot could use tools and switches made for people (Figure 48.14). The Japanese start-up **GITAI** tested an arm inside the commercial Bishop airlock in 2021. In March 2024 it mounted a second arm outside and performed assembly and servicing tasks, reaching technology readiness level 7.

### 48.8.3 Swarms, Formations and Constellations

Spacecraft working together must know where the others are and agree on who does what. The Swedish-led **PRISMA** mission (2010) demonstrated autonomous formation flying with two satellites, Mango and Tango — the Tango of the 2019 pose challenge. ESA's **Proba-3**, launched in December 2024, flies two satellites about 150 m apart. One casts a precise shadow on the other's coronagraph to create artificial solar eclipses. Holding the formation to about a millimetre needs onboard control, using GPS, star trackers, lasers and radio links. Its first eclipse images were taken in May 2025, and it was commissioned for autonomous formation flying in June 2025.

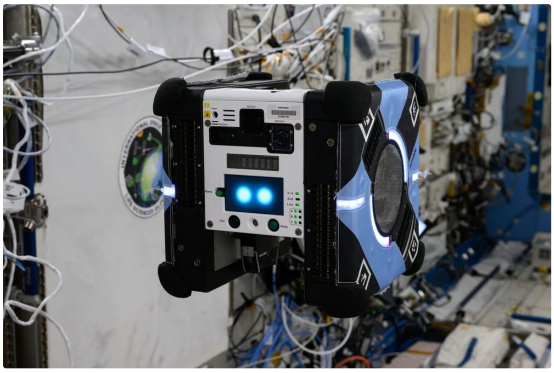


**FIGURE 48.13** An Astrobee free-flying robot during a Zero Robotics student programming demonstration on the ISS on 1 May 2024. Image: NASA/JSC.

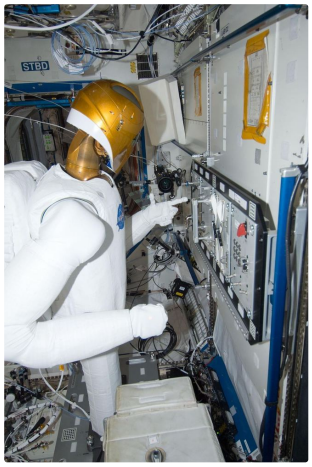


**FIGURE 48.14** Robonaut 2 in the ISS Destiny laboratory on 2 January 2013, operating valves on a task board under remote command from the ground. Image: NASA/JSC.

NASA's **Starling** mission (July 2023) flew four CubeSats to test swarm autonomy. Its Distributed Spacecraft Autonomy software let the satellites share observations of the ionosphere over crosslinks and plan follow-up measurements together. Its StarFOX experiment tracked swarm members and other objects with commercial star trackers. In a test with SpaceX reported in March 2025, a Starling satellite accepted responsibility for avoiding a close approach with a Starlink satellite through a shared screening service. It then planned and executed the avoidance manoeuvre on its own. On the Moon, the **CADRE** project will land three suitcase-sized rovers that explore and map as a team without moment-to-moment commands (Figure 48.15). It is due to fly on Intuitive Machines' IM-3, no earlier than early 2027.

**FIGURE 48.15** A development model of a CADRE rover on its first autonomous drive in JPL's Mars Yard in June 2023. Three such rovers are to map the Moon together as a team. Image: NASA/JPL (NASA/JPL-Caltech).

Table 48.6 gives the status of the robotic systems discussed.

**TABLE 48.6** Robots, swarms and formations with onboard autonomy. NET means no earlier than.

| System | Type | Autonomy | Status, October 2026 |
|---|---|---|---|
| Perseverance | Mars rover | Enhanced AutoNav, onboard planner, AEGIS | Operating; record 45.22 km; extended to about 2028 |
| Canadarm2, Dextre | ISS arms | Supervised; automated routine motions | Operating |
| Canadarm3 | Lunar arm system | Planned supervised autonomy | Refocused on lunar surface; NET 2029 |
| Astrobee | ISS free-flyers | Map-based navigation; research autonomy | Operating since 2019 |
| GITAI S2 | Arm on Bishop airlock | Autonomous and teleoperated tasks | Demonstrated 2024 |
| Starling | 4-CubeSat swarm | Distributed planning, star-tracker navigation | Primary mission done; extended tests 2024–25 |
| CADRE | 3 lunar rovers | Cooperative mapping | Awaiting IM-3, NET early 2027 |
| Proba-3 | 2-satellite formation | Millimetre formation control | Operating since 2025 |

At constellation scale, autonomy is already routine. Starlink satellites receive conjunction warnings and plan and execute avoidance manoeuvres themselves.

### STARLINK'S AUTONOMOUS COLLISION AVOIDANCE IN NUMBERS

- **Volume.** From 1 December 2025 to 31 May 2026, Starlink satellites made 207 152 propulsive collision-avoidance manoeuvres: 65 137 by first-generation and 142 015 by second-generation satellites. The previous six months (June–November 2025) saw 148 696.
- **Rate.** That is about 36 manoeuvres per satellite per year for the first generation and 46 for the second.
- **Threshold.** SpaceX manoeuvres when the probability of collision exceeds 3 in 10 million ($3 \times 10^{-7}$), plus miss-distance criteria. The common industry threshold is 1 in 10 000.
- **Misses.** The onboard system ran without system-wide outages. In 22 individual cases the targeted risk was not reduced, mostly because predictions of the other object were poor or arrived too late. Newly launched satellites not yet tracked and short Space-Track.org outages were among the causes.
- **New sensing.** In January 2026 SpaceX announced Stargaze, which uses about 30 000 Starlink star trackers to spot nearby objects. It aims to screen conjunctions in minutes rather than hours.

Source: SpaceX semi-annual report to the US Federal Communications Commission, mid-2026; SpaceX announcements.

### EXAMPLE: COLLISION-AVOIDANCE DECISIONS FOR A 10 000-SATELLITE CONSTELLATION

Assume Starlink's second-generation rate of 46 manoeuvres per satellite per year.

1. **Per year.** 10 000 × 46 = **460 000** manoeuvres.
2. **Per day and hour.** 460 000 / 365.25 = 1 259 a day, or 52 an hour — one every **69 s**, day and night.
3. **Human review.** If each decision took an analyst 10 minutes, that would be 210 person-hours a day. At about 1 800 working hours per person per year, it would need about **43 full-time analysts** for this task alone.

This is why the decision is automated: humans set the thresholds and handle the exceptions, while the satellites plan and execute routine manoeuvres. The weak point moves from deciding to *knowing*: most failures come from poor tracking data about the other object (Chapter 50).

### PRACTICE: DESIGNING AND VERIFYING AN AUTONOMY FUNCTION WITH A RUNTIME MONITOR

1. **State the decision.** Write down what the function decides, how often, and what a wrong decision costs. Example: "choose a safe landing point within 10 s, with at most one hazardous choice in 10 000 landings".
2. **Define the operating domain.** List the conditions it must handle — Sun angles, terrain, sensor noise, speeds — and which of them the training data cover.
3. **Build the simplest baseline.** Start with a classical method, such as a template, a PD law or a greedy rule. A learned method must beat it by a measured margin.
4. **Design the safety envelope.** Write simple, checkable rules the system must never break, such as stopping distance or keep-out zones, and a backup that always keeps it inside them.
5. **Add the monitor.** Make it independent of the learned component, and simple enough to verify exhaustively or formally.
6. **Test in layers.** Run millions of randomised simulations, then hardware-in-the-loop tests. Report failure rates with confidence intervals, and test outside the domain too.
7. **Measure the cost of safety.** Record how often the monitor intervenes and what that costs. Frequent interventions mean the learned part or the envelope needs work.
8. **Fly in shadow mode first.** Run the function on board without letting it act, compare its choices with the existing system, then give it authority step by step.

### KEY TAKEAWAYS

- Spacecraft need autonomy when the ground cannot respond fast enough, often enough or cheaply enough. Light-time, contact gaps, fleet size and event speed set the level; ECSS defines E1–E4 for execution, D1–D2 for data and F1–F2 for failures.
- Most autonomy in flight is search, estimation and optimisation, not learning: executives, fault protection, diagnosis, planners, optical navigation and convex guidance.
- Learning has entered through perception and science: crater detection, keypoint networks for pose and rover science targeting. Learned detectors are often more robust to lighting than templates, but not more precise.
- Terrain-relative navigation can reach metre-level accuracy. Map registration and attitude knowledge, not image matching, usually limit it. Recent landing failures came from sensors and the logic that judges their data.
- Reinforcement learning can find efficient policies but may fail abruptly outside its training conditions. Runtime monitors with verified backups, and learning that speeds up an optimiser, are the practical routes to flight.
- Autonomy is now routine at scale: Perseverance drives itself for over 90 % of its distance, and Starlink satellites made over 200 000 autonomous avoidance manoeuvres in six months.

### REVIEW QUESTIONS

1. Using $T_{\text{ground}} = 2\tau + 2W + T_{\text{decide}}$, estimate the ground response time for a Jupiter orbiter with 43 minutes of one-way light-time and 8 hours of contact a day. Which ECSS execution level does a 2-hour science event require?
2. Explain the difference between levels F1 and F2. Give one mission for which F1 is enough and one that needs F2, with reasons.
3. Describe the three layers of an autonomous spacecraft and their time scales. Where are learned models most often used, and why?
4. How does model-based diagnosis choose between explanations of a fault? Why does the prior probability favour explanations with few failed components?
5. In Figure 48.3, why does a greedy rule fall short of the optimum, and why might a flight planner still use one? What property of Perseverance's onboard planner keeps its run time bounded?
6. Why does normalised cross-correlation tolerate changes in brightness but not in Sun direction? Give two ways a landing system can cope.
7. Repeat the terrain-relative navigation example for a 12 m map, 150 landmarks, a 5 m map tie and 0.05° attitude knowledge at 3 km. Which term dominates?
8. In the 2019 pose-estimation challenge, the winning score was forty times worse on real images than on rendered ones. What is this gap called, and what methods narrowed it in 2021?
9. Why is G-FOLD not machine learning? What does lossless convexification achieve?
10. In Figure 48.8, why did the learned policy fail with a weaker engine, and how did a runtime monitor restore safety? What did the monitor cost?

CHAPTER

# 49 AI for Mission Operations, Communications and Networks

Image: NASA/JSC (ROBERT MARKOWITZ NASA-JSC)

*Every spacecraft is flown from the ground by people and software that watch its telemetry, plan its activities, schedule its contacts and send its commands. This chapter shows where machine learning and optimisation now help with that work, and where they do not yet. It covers anomaly detection in telemetry, from fixed limits to learned models, false-alarm budgets, prognostics of batteries and wheels, automation, digital twins and language-model assistants, and the human factors that decide whether they are trusted. It then turns to scheduling ground contacts and to communications — cognitive radio, signal recognition, adaptive coding, beam and spectrum management, interference, laser-mesh routing and 6G — and ends with the security of ground segments. Small models run in Python make each method concrete.*

IN THIS CHAPTER

49.1 Mission Operations Today
49.2 Watching Telemetry: Anomaly Detection
49.3 Prognostics and Health Management
49.4 Automating Operations
49.5 Scheduling Contacts and Ground Networks
49.6 AI in Space Communications
49.7 Networks in Orbit
49.8 Securing Ground Segments and Links

Spacecraft are flown from the ground. A flight control team watches the stream of **telemetry** (TM) — measurements that a spacecraft sends about its own health. It plans activities, books contacts with ground stations and sends **telecommands** (TC). The hero image above shows NASA's Artemis II flight control team in Houston on 10 April 2026, during the splashdown of the first crewed flight to the Moon since 1972. Rooms like this one have worked in much the same way since Apollo: skilled people, many screens and fixed alarm limits.

Two trends are changing this. The first is scale: one observatory produces tens of thousands of telemetry channels, and one operator flies more than ten thousand satellites. The second is practical machine learning, optimisation and language models. This chapter explains how they help to watch telemetry, predict failures, automate routine work, schedule ground networks, run radio links and route data. It separates what is operational, demonstrated or still research, as of October 2026.

The chapter builds on Chapters 7, 11 and 17 (ground segment, constellations and communication satellites) and on the learning methods of Chapters 43–45. Onboard autonomy is covered in Chapter 48, space situational awareness and defence in Chapter 50, and assurance of AI in Chapter 53.

## 49.1 Mission Operations Today

The key idea of this section is that mission operations are a loop: plan, command, monitor and analyse. Machine learning helps at every step, but the loop and the people in it remain.

### 49.1.1 What a Control Centre Does

A **mission operations centre** holds the people and software that fly a spacecraft (Chapter 7). Its core, the **mission control system** (MCS), turns raw telemetry frames into named, calibrated parameters, checks limits, raises alarms and sends commands. Four other functions surround it. **Flight dynamics** determines the orbit and plans manoeuvres. **Mission planning** turns requests into a timeline that respects power, thermal, data and visibility limits. The **ground network** provides antennas and contact time. **Engineering support** — subsystem experts on call — investigates anything unusual.

Work follows a daily cycle. Planners build tomorrow's timeline, and command sequences are checked on a simulator before uplink. During each contact, controllers watch live telemetry and alarms; afterwards, engineers study trends. When something goes wrong, an **anomaly report** records the symptoms, the cause and the fix. Years of such reports, telemetry and command logs are the raw material for the learning methods in this chapter.

### 49.1.2 How Much Telemetry?

Spacecraft telemetry is small in bytes but rich in structure. NASA's James Webb Space Telescope records telemetry at about 100 kbit/s. On the ground it is unpacked into about 36 000 named parameters, called **mnemonics**, with 4 000 more derived on the ground. Some are sampled every 0.064 s, others every 32 s or more. A 28-minute file holds about 10 million samples. EUMETSAT routinely reports about 46 000 housekeeping parameters from Sentinel-3. Table 49.1 gives more examples.

**TABLE 49.1** Examples of the scale of modern operations. Sources: SpaceOps 2025 papers (JWST, Sentinel-3), ESA-ADB paper, Planet (IAC 2018) and SpaceX's report to the US Federal Communications Commission.

| System | Scale | What it means for operations |
|---|---|---|
| James Webb Space Telescope | 36 000 telemetered + 4 000 derived points; ~100 kbit/s | Too many for people to watch; trending software is essential |
| Sentinel-3 (EUMETSAT) | ~46 000 housekeeping parameters routinely reported | Learned outlier detection is trialled on a subset |

| System | Scale | What it means for operations |
|---|---|---|
| ESA anomaly benchmark (2024) | 1.55 billion data points, 844 annotated events | Real anomalies are rare: 31–118 per mission |
| Planet Dove fleet (2018) | >200 satellites, >2 000 passes a day, 8 engineers | Operations run by exception; nominal passes automated |
| Starlink (June–November 2025) | 148 696 collision-avoidance manoeuvres in six months | Decisions too frequent for people to make one by one |

### 49.1.3 From One Satellite to Thousands

A traditional mission has a team for each spacecraft, with people on console around the clock. That model does not scale to constellations. In 2018 Planet flew more than 200 Dove imaging satellites with eight engineers in Berlin and San Francisco. By 2022 Planet reported that commanding each pass, scheduling contacts, reporting and delivering images were fully automated. Periodically run Python scripts handled the most common anomalies. This cut daily Dove monitoring from about four hours to 30 minutes and ended night shifts for the larger SkySats.

This style is called **management by exception**. Software flies the nominal mission; people handle what the software cannot. At the largest scale, decisions are taken by machines because there are too many for people. Starlink satellites made 148 696 collision-avoidance manoeuvres between June and November 2025, decided autonomously, without a person in the loop, when the predicted collision probability exceeded about 3 in 10 million (Chapters 21 and 48).

As Figure 49.1 shows, decision support now grows beside the data path from spacecraft to engineers. It includes monitoring models trained on the archive, planning solvers, automation scripts, assistants and digital twins. Monitoring and scheduling tools are operational in some centres. Language-model assistants are pilots, and natural-language commanding is research.

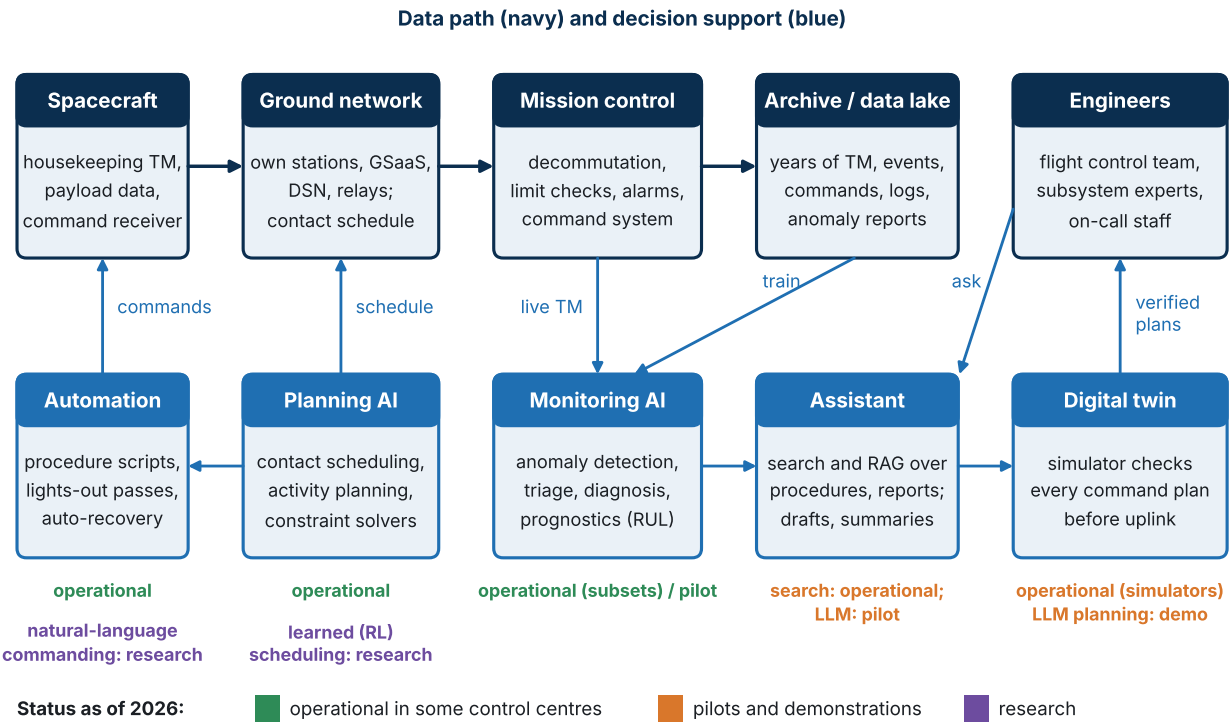



**FIGURE 49.1** Where learning and automation enter the ground segment. The data path (navy) runs from the spacecraft through the ground network and mission control system to the archive and the engineers. Decision-support functions (blue) are trained on the archive or fed by live telemetry; their status in 2026 is colour-coded.

## 49.2 Watching Telemetry: Anomaly Detection

The key idea of this section is that a monitor must find rare, varied faults among millions of normal samples, while raising few false alarms. Learned models of normal behaviour find faults that fixed limits miss, but only if their false alarms are controlled.

### 49.2.1 Limits and Their Limits

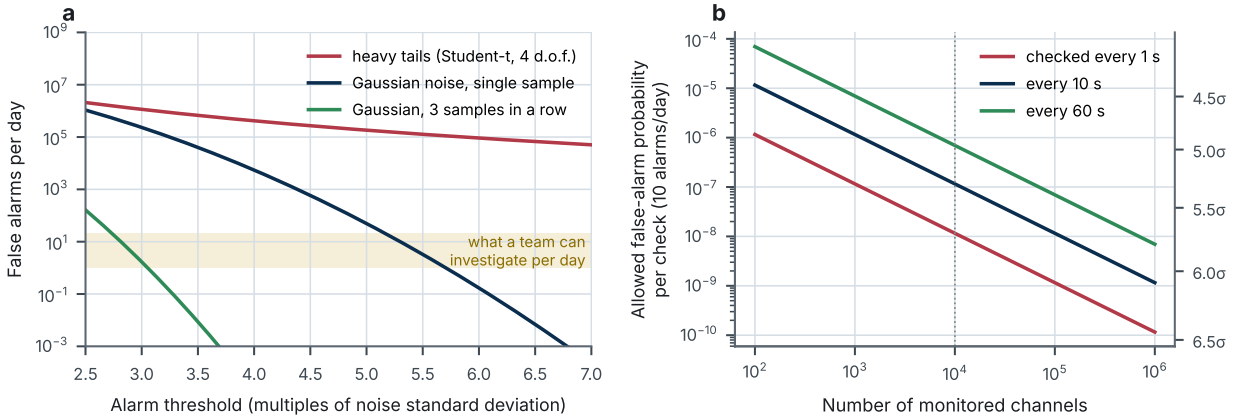
The classic monitor is the **limit check**. Each parameter has a lower and an upper limit, often a soft "warning" pair and a hard "alarm" pair. A value outside them raises an alarm. Limits can depend on the mode: a heater current is checked against one range when the heater is commanded on and another when it is off. Limit checks are simple, fast and easy to explain. They also catch most serious failures, because a failed unit usually drives some parameter far out of range.

Their weakness is everything else. A temperature can be normal for daylight but wrong in eclipse. A slow drift stays inside the limits for months, then crosses them too late. And limits set too tight produce false alarms, which cost attention and teach people to ignore alarms.

The false-alarm budget is simple arithmetic. Suppose $N$ channels are each checked $f$ times a second, and each check has a false-alarm probability $p$. The expected number of false alarms per day is then given below.

$$n_{\mathrm{FA}} = N\, f \cdot 86\,400 \cdot p.$$

For a threshold at $k$ standard deviations of Gaussian noise, the two-sided probability is $p = 2Q(k)$, where $Q$ is the tail probability of the standard normal distribution. In Figure 49.2 this is applied to 10 000 channels checked every 10 s. A 3σ threshold, common in textbook statistics, gives about 233 000 false alarms a day. Fewer than 10 a day needs about 5.3σ for perfectly Gaussian noise. Real telemetry noise has heavier tails, so empirical thresholds must be set from the data. Requiring several consecutive exceedances helps greatly when noise is independent, but much less when it is correlated in time.




Model: 10 000 channels, each checked against its threshold every 10 s (8.64 × 10⁷ checks a day), two-sided thresholds, independent noise. Real telemetry noise is often heavy-tailed and correlated in time, so the green curve is a best case.

**FIGURE 49.2** The false-alarm budget of a large telemetry set. (a) False alarms per day for 10 000 channels checked every 10 s, against the threshold in multiples of the noise standard deviation. (b) The per-check false-alarm probability allowed for 10 alarms a day, against the number of channels; the right axis gives the equivalent Gaussian threshold.

### EXAMPLE: FALSE ALARMS PER DAY FOR 10 000 CHANNELS

A science satellite has 10 000 monitored channels, each checked every 10 s. The team can investigate about 10 alarms a day.

1. **Checks per day.** 10 000 × 86 400 s / 10 s = $8.64 \times 10^7$ checks.
2. **At $p = 10^{-6}$ per check.** $8.64 \times 10^7 \times 10^{-6} \approx$ **86 false alarms a day** — about nine times the budget. Checking every second instead would give 864.
3. **Allowed probability.** $10 / 8.64 \times 10^7 =$ **$1.16 \times 10^{-7}$** per check. For Gaussian noise this is a two-sided threshold of **5.3σ**.
4. **At a 5σ threshold.** $p = 5.7 \times 10^{-7}$, giving about 50 false alarms a day.
5. **Persistence.** Requiring three exceedances in a row at 3σ gives $(2.7 \times 10^{-3})^3 \approx 2 \times 10^{-8}$ per check, or about 1.7 a day — if the noise is independent. Correlated noise raises this, so the rule must be tested on archived data.
6. **Lesson.** With thousands of channels, the per-channel false-alarm rate must be tiny. Any learned detector needs the same budget, set on real data.

## 49.2.2 Kinds of Anomaly

An **anomaly** is behaviour that differs from what is expected. Not every anomaly is a fault: a commanded manoeuvre or a calibration looks unusual the first time but is normal for the operators. ESA's benchmark calls these **rare nominal events**. Table 49.2 lists the main kinds of anomaly and the detectors that suit them.

**TABLE 49.2** Kinds of telemetry anomaly and the detectors that suit them.

| Kind | What it looks like | Example | Detectors that suit it |
|---|---|---|---|
| Point | One sample far from its neighbours | Bit flip, sensor glitch | Limits, prediction residuals |
| Contextual | Normal value at the wrong time or mode | Daylight temperature in eclipse | Models that use orbit phase and mode |
| Collective | Normal values in an abnormal sequence | Oscillating control loop | Sequence models, windowed features |
| Drift | Slow change of level or trend | Rising friction, sensor ageing | Trend tests, residuals against other channels |
| Multivariate | Each channel normal, their combination not | Current rises while load is unchanged | Cross-channel models, isolation forests |

### 49.2.3 Learning What Normal Looks Like

Learned monitors share one idea: model normal behaviour from history, then score how far new data departs from it. There are three main families.

**Prediction-based** detectors forecast each channel and watch the error. A model $f$ predicts the value $x_t$ from recent history and from context $\mathbf{c}_t$, such as orbit phase and commanded mode. The standardised residual is

$$z_t = \frac{\lvert x_t - f(x_{t-L}, \dots, x_{t-1}, \mathbf{c}_t) \rvert}{\sigma_{\text{res}}},$$

where $\sigma_{\text{res}}$ is the residual spread on normal training data. An alarm is raised when $z_t$ exceeds a threshold. JPL's **Telemanom** system, described in 2018, used a **long short-term memory** (LSTM) network for each of 82 channels of the SMAP satellite and the Curiosity rover (Chapter 44). It smoothed the errors and set thresholds automatically from their recent distribution. On 105 labelled anomalies it reached 87.5 % precision and 80 % recall (Chapter 43).

**Reconstruction-based** detectors squeeze the data through a narrow layer and rebuild it. An **autoencoder** with encoder $h$ and decoder $g$ is trained so that $g(h(\mathbf{x})) \approx \mathbf{x}$ for normal data. The anomaly score is the reconstruction error $\lVert \mathbf{x} - g(h(\mathbf{x})) \rVert^2$. Patterns the network never saw are rebuilt badly.

**Distance- and isolation-based** detectors look at where a sample sits among the training data. The k-nearest-neighbour score is the distance to the $k$-th closest normal sample. A one-class support vector machine draws a boundary around normal data. An **isolation forest** (2008) builds random trees that split the data at random values of random features. Unusual points are isolated after few splits. The score is

$$s(\mathbf{x}) = 2^{-\bar{h}(\mathbf{x})/c(n)},$$

where $\bar{h}$ is the average path length over the trees and $c(n)$ the average for $n$ normal points. Scores near 1 mean anomalous.

### 49.2.4 A Test on Synthetic Telemetry

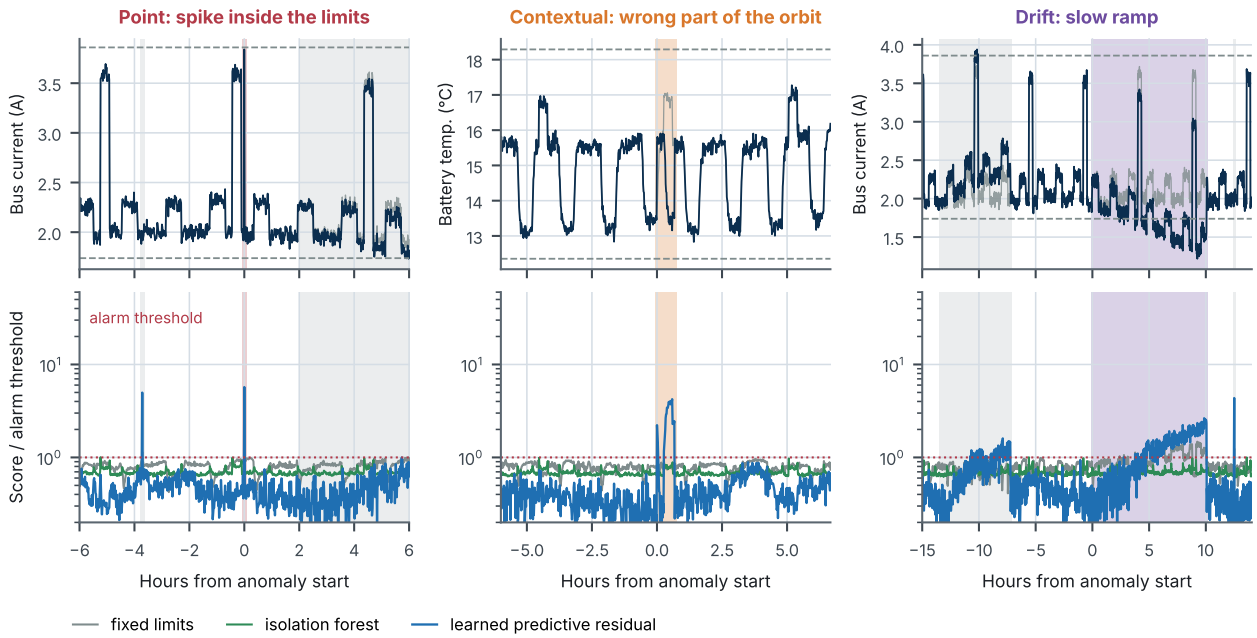
To see the strengths of each family, this chapter's script builds six housekeeping channels for a satellite in a 96-minute orbit with 36 % eclipse. Battery voltage, battery temperature, bus current, panel temperature, wheel speed and heater duty all follow the orbit, a commanded payload mode

and a 14-day seasonal cycle. Slow unmodelled variations are added. Fourteen days train each detector. Fourteen more days test it, with 14 point, 12 contextual and 8 drift anomalies injected (Figure 49.3).

Three detectors are compared. The **fixed limits** are the training minimum and maximum, widened by 5 % of the range. The **learned predictive detector** is a small neural network for each channel, written in numpy. It predicts the channel from the other five channels, the orbit phase and the mode, and scores the largest standardised residual. The **isolation forest** from scikit-learn sees the channel values, orbit phase and their changes. Each detector's threshold lets 0.1 % of normal samples alarm — about 1.4 false alarms a day.

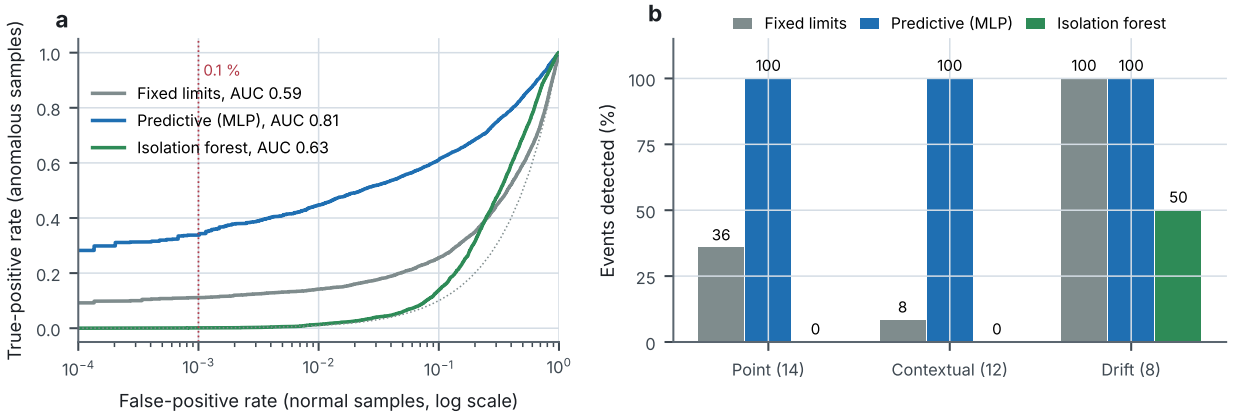
The results appear in Figure 49.4. The predictive network found all 34 injected events, and its sample-by-sample area under the ROC curve (AUC; Chapter 43) was 0.81. The fixed limits found all 8 drifts, but only after 45 % of each ramp, on average. They caught 5 of the 14 spikes — those that left the normal range — and 1 of the 12 contextual events. The isolation forest found only half the drifts and none of the spikes or contextual events.

Two lessons follow. First, context makes learned detectors useful: the network knew the orbit phase and the other channels, so a value copied from daylight into eclipse stood out. Second, a generic detector is not automatically better than limits. The isolation forest's scores saturate outside the training range, so a far-out spike looks no stranger than the many normal points near the edge. Real systems should keep limit checks and add learned detectors beside them. Real telemetry is also much harder than these synthetic data, as the next section shows.




Synthetic telemetry: six channels sampled once a minute, driven by a 96-min orbit (36 % eclipse), a commanded payload mode, a 14-day season and slow unmodelled variations. Grey: signal without the anomaly; dashed: fixed limits (training minimum and maximum). Thresholds are set so that 0.1 % of normal test samples alarm.

**FIGURE 49.3** Three injected anomalies and the detectors' responses. Top: a spike that stays inside the limits, a segment copied from the wrong part of the orbit, and a slow drift (navy; grey shows the signal without the anomaly; dashed lines are the fixed limits). Bottom: each detector's score divided by its alarm threshold.




Same data as the previous figure. (a) Sample-by-sample ROC curves. (b) Share of injected events with at least one alarm when each detector's threshold lets 0.1 % of normal samples (about 1.4 per day) through. Neural network: 2 hidden layers of 24 tanh units per channel, trained 12 epochs with Adam; isolation forest: 300 trees (scikit-learn).

**FIGURE 49.4** Detector performance on the synthetic test set. (a) Sample-by-sample receiver operating characteristic (ROC) curves on a logarithmic false-positive axis. (b) Share of injected events caught at a false-alarm rate of 0.1 % of normal samples.

## 49.2.5 Benchmarks and Real Results

Comparing detectors fairly needs real, labelled data. Until 2024 the main public sets were the SMAP and Curiosity channels released with Telemanom. ESA then published a much larger benchmark, built with operations engineers at its European Space Operations Centre (ESOC) in Darmstadt.

### DATA BOX — ESA'S ANOMALY-DETECTION BENCHMARK (ESA-ADB), 2024

- **Data.** Real telemetry from three ESA missions, two used in the benchmark; 1.55 billion data points, 7.32 GB compressed. Developed by ESA with KP Labs (Poland); first released in June 2024.
- **Mission 1.** 76 channels (58 monitored), 14 years, 200 annotated events: 118 anomalies, 78 rare nominal events and 4 data gaps.
- **Mission 2.** 100 channels (47 monitored), 3.5 years, 644 events: only 31 anomalies, but 613 rare nominal events such as manoeuvres and resets.
- **Scoring.** Event-wise precision first, because operators care about false alarms; then which channels and subsystems were flagged, and timing.
- **Results.** An adapted Telemanom was best for Mission 1, with an event-wise F0.5 score of 0.968 for anomalies alone. A simple windowed isolation forest was best on a small channel subset of Mission 2 (0.949). No algorithm found the nine real anomalies hidden among Mission 2's rare nominal events, and adding channels added false detections.
- **Conclusion of the authors.** Typical algorithms are not yet good enough for operators; new methods are needed. A July 2026 follow-up found supervised graph networks more accurate, and simple unsupervised methods much cheaper to run.

## 49.2.6 Operational Tools

Learned monitors have been used in real control centres for about two decades, usually beside limit checks and for selected parameters. Table 49.3 summarises the best-documented ones.

EUMETSAT's experience is typical. Its Sentinel-3 flight control team ran a k-nearest-neighbour tool and a neural network in parallel. The choice of training data mattered more than the algorithm. Novel-but-harmless detections were the main burden, and adding the timeline of

commanded operations cut them. Engineers also needed to see *why* a parameter was flagged, which favoured simple methods. The tools did find real anomalies, including micrometeoroid or debris impacts.

**TABLE 49.3** Learned telemetry monitors with published operational use or trials.

| Tool | Organisation | Method | Status |
|---|---|---|---|
| Inductive Monitoring System (IMS) | NASA Ames | Clusters of nominal data; distance to nearest cluster | Used for ISS gyroscopes from about 2005; about a dozen Mission Control systems by 2012 |
| Novelty Detection | ESA/ESOC | k-nearest-neighbour outlier score on orbit statistics | Developed at ESOC in the early 2010s; trialled by CNES in 2013 |
| NOSTRADAMUS | CNES with TeSA | One-class support vector machine | Prototype (2014), operational assessment |
| ATHMoS | DLR German Space Operations Center | Outlier detection; later neural prediction about 4.5 h ahead | Developed 2016–19 |
| Telemanom | NASA JPL | LSTM prediction and dynamic thresholds | Piloted on SMAP; false positives limited its use |
| CHART outlier tools | EUMETSAT | k-nearest neighbours; neural network | Routine on a subset of parameters; Sentinel-3 assessment (2025) |

LANDMARK: TWENTY YEARS OF LEARNED TELEMETRY MONITORING

- **1988.** Houston's Mission Control tested expert systems beside conventional Space Shuttle displays (Chapter 43).
- **About 2005.** ISS engineers adopted NASA Ames' Inductive Monitoring System, learned from months of normal data, to watch a failing gyroscope.
- **2010s.** ESA, CNES and DLR brought outlier detection into European control centres.
- **2018.** JPL's Telemanom showed that LSTM prediction with automatic thresholds could catch most labelled anomalies. In its SMAP pilot, false positives were the main obstacle to central use.
- **2024.** ESA released ESA-ADB, the first large, operator-labelled benchmark. It showed how far typical algorithms remain from operators' needs.
- **2025–26.** EUMETSAT reported its Sentinel-3 trial. ESOC's search assistant was in use on 14 missions, and the Aerospace Corporation and Google showed a proof-of-concept agent for triage in large constellations.

### 49.2.7 From Detection to Diagnosis

An alarm says that something is wrong. **Diagnosis** says what, and **root-cause analysis** says why. In a large constellation one fault often triggers dozens of alarms across subsystems. The first step is therefore to group alarms that share a time, a satellite and a likely cause. The diagnosis itself is still mostly human work (Figure 49.5).

Classical diagnosis uses fault trees, failure-mode tables and model-based reasoners that compare telemetry with a model of the system (Chapter 6). Learning adds three things. It can match a new alarm pattern to similar past anomalies. It can learn which alarms occur together and which comes first, a hint of cause. And it can gather the context — recent commands, space weather, ground-station problems — that an engineer would otherwise collect by hand.

In April 2026 the Aerospace Corporation and Google's public-sector arm showed a proof-of-concept tool for large constellations in low Earth orbit. It combines telemetry, payload and ground-station data in one view. It links an anomaly to recent events, such as payload activity and environmental conditions, to help engineers find the cause. The aim is to save the minutes on-call staff now spend correlating data by hand. It is decision support, not a replacement for engineers. A 2025 JPL study of similar "agentic" detection for Deep Space Network data systems reported preliminary results, not yet robust to false positives.

**FIGURE 49.5** Nzinga Tull, Hubble systems anomaly response manager, in the control room at NASA's Goddard Space Flight Center on 15 July 2021. A month earlier Hubble's payload computer had stopped, and the team switched to backup hardware to restore science. Diagnosis of this kind is still human work, supported by data tools. Image: NASA/GSFC (Rebecca Roth).

## 49.3 Prognostics and Health Management

The key idea of this section is that predicting *when* a part will fail is worth more than detecting that it has. Prognosis combines a physical degradation model with data, and must report its uncertainty honestly.

### 49.3.1 Remaining Useful Life

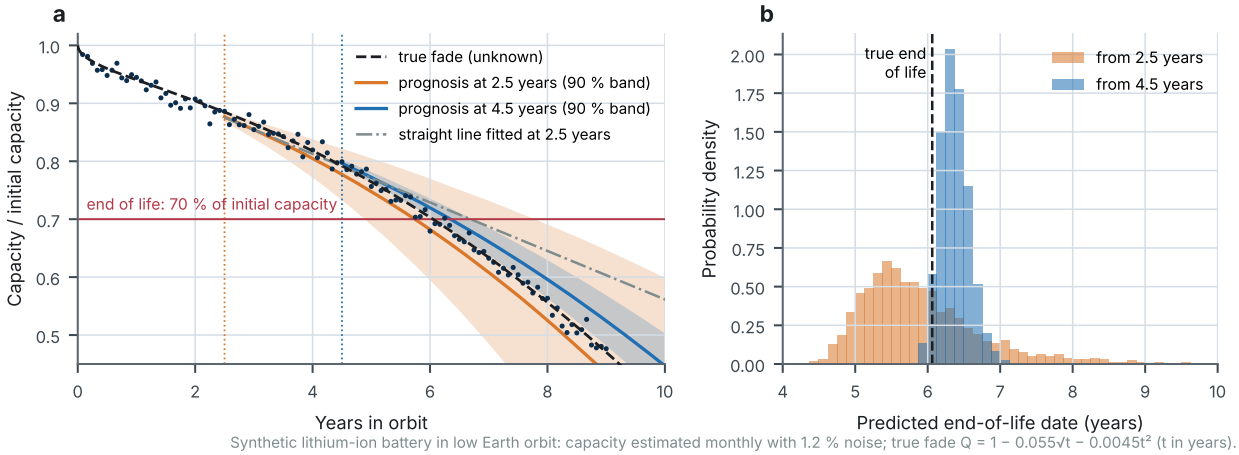
**Prognostics and health management** (PHM) estimates the **remaining useful life** (RUL) of a component: the time until it can no longer meet its requirement. Batteries, reaction wheels, gyroscopes, thrusters and solar arrays all wear out in ways that can be trended.

A lithium-ion battery in low Earth orbit charges and discharges about 15 times a day, some 5 500 cycles a year. Its capacity fades. Early fade is often proportional to the square root of time, as a surface layer grows on the electrode. Later, fade can accelerate. A simple model of the capacity fraction $Q$ is

$$Q(t) = 1 - a\sqrt{t} - b\,t^2,$$

with the coefficients $a$ and $b$ learned from the satellite's own data. Because $Q$ is linear in $a$ and $b$, **Bayesian linear regression** gives their joint uncertainty in closed form. Thousands of sampled curves then give a distribution for the date when $Q$ falls below the end-of-life threshold.

In Figure 49.6 this is applied to a synthetic battery with monthly capacity estimates and 1.2 % noise. The true end of life, at 70 % capacity, is at 6.06 years. A prognosis made after 2.5 years puts the end of life at 5.75 years (median), with a 90 % interval of 4.9–7.8 years. After 4.5 years of data the median is 6.35 years and the interval shrinks to 6.08–6.71 years. A straight line fitted at 2.5 years says 6.7 years, with no hint of its uncertainty. The model form matters: in tests with an extra free linear term, the early intervals often missed the true date.




Synthetic lithium-ion battery in low Earth orbit: capacity estimated monthly with 1.2 % noise; true fade $Q = 1 - 0.055\sqrt{t} - 0.0045t^2$ (t in years). Prognosis: Bayesian linear regression on $\sqrt{t}$ and $t^2$ (form assumed known from ground life tests) with a Gaussian prior; 4 000 posterior samples give the end-of-life distribution.

**FIGURE 49.6** Battery capacity-fade prognosis. (a) Monthly capacity estimates (dots), the true fade (dashed), prognoses made at 2.5 and 4.5 years with 90 % bands, and a straight-line fit. (b) The predicted end-of-life date from each prognosis.

Operators use exactly this kind of tool. Planet's PROTOS, built for the five RapidEye satellites, used a **Kalman filter** (Chapter 43) to estimate each battery's depth of discharge from charge counting and voltage. A fade model with measured temperatures predicted when each 15 Ah battery would reach 50 % capacity. A genetic algorithm then re-planned imaging, downlinks and heater use to slow the fade. Planet estimated a 15 % longer mission life for satellites already past their 7.25-year design life. On the ISS, trending told engineers when to replace ageing batteries, a job done by astronauts (Figure 49.7).

**FIGURE 49.7** NASA astronaut Andrew Morgan on the International Space Station's port truss on 11 October 2019, during a spacewalk to replace older nickel–hydrogen batteries with lithium-ion ones. Trending of battery capacity tells operators when such work is needed. Image: NASA/JSC.

### 49.3.2 Wheels, Gyroscopes and Other Wear-Out Parts

Reaction wheels and gyroscopes have ended or reshaped several missions. Kepler lost its second wheel in 2013, which ended its original mission. Dawn lost wheels in 2010 and 2012 and finished its mission on thrusters. Hubble has flown with a single gyroscope since June 2024, which costs about 12 % of its observing efficiency.

Wheel health shows in friction torque, motor current, temperature and speed noise; rising drag can precede failure by months. Learned models remove the normal dependence on speed and temperature, so that the remaining trend shows wear, and compare identical wheels across a fleet. Most published work still uses ground life tests or single satellites.

## 49.4 Automating Operations

The key idea of this section is that automation should take over routine, well-understood work, while people keep the judgement and the authority. The newest tools — digital twins and language-model assistants — are useful when their output is checked.

### 49.4.1 From Scripts to Lights-Out Operations

Automation in a control centre grows in steps (Table 49.4). At the first level, people type every command. At the highest, software runs the mission and people are called only when something unusual happens. Most modern constellations work at the third or fourth level. Most flagship science missions work at the second, with automation for routine passes and people on console for critical events.

**Lights-out operations** means that nobody is on console for routine passes. The ground system schedules contacts, uplinks commands, downlinks data and checks health, and pages an on-call engineer if a check fails. The key technology is not AI but reliable procedure automation: scripts tested on a simulator and kept under configuration control. Learning enters where rules are hard to write, such as deciding whether an unusual pattern is safe to ignore. The example below shows what automated triage can save.

**TABLE 49.4** Levels of automation in spacecraft operations.

| Level | What software does | What people do | Typical use |
|---|---|---|---|
| 1 Manual | Displays data, checks limits | Send every command, watch every pass | Early operations, critical events |
| 2 Assisted | Runs approved procedures step by step | Start procedures, confirm critical steps | Science missions, GEO satellites |
| 3 Automated passes | Runs nominal contacts and planning | Review reports in working hours | Small constellations, mature missions |
| 4 Management by exception | Detects, triages and fixes known anomalies | Handle new problems, on call | Large constellations (e.g. Planet) |
| 5 Autonomy | Decides on board, including safety actions | Set goals and policies, audit | Collision avoidance, deep space (Chapter 48) |

**EXAMPLE: OPERATOR WORKLOAD SAVED BY AUTOMATED TRIAGE**

A 300-satellite constellation raises about 1 200 alarms a day, mostly known, harmless patterns such as eclipse transitions. Manual triage takes about 3 minutes per alarm.

1. **Manual workload.** 1 200 × 3 min = 3 600 min = **60 hours a day**, or 7.5 eight-hour shifts.
2. **Grouping.** Merging correlated alarms, 3 per group on average, leaves 400 incidents.
3. **Classification.** A learned classifier closes 80 % as known patterns, leaving **80** for people.
4. **Review.** With more context, each takes 6 minutes: 80 × 6 = 480 min = **8.0 hours**.
5. **Audit and result.** Spot-checking 5 % of the 320 closed incidents at 2 minutes adds 32 min. The total is 8.5 hours instead of 60: a saving of **86 %**, from 7.5 shifts to about 1.1.
6. **Price.** If 5 real anomalies a day pass through and 1 % are wrongly closed, one is missed every 20 days. The audit and limit checks that bypass the classifier are there to catch it.

### 49.4.2 Ground Digital Twins

A **digital twin** is a model kept in step with one real system, using its own telemetry. In operations, the oldest form is the **operational simulator**. It runs the real flight software on emulated processors, with models of the rest of the spacecraft and its environment. Every command plan for a major mission is checked on such a simulator before uplink, and teams train on it for failures. Rover missions also keep physical twins on Earth (Figure 49.8).

Learning makes twins faster and closer to the real vehicle. Learned surrogates can replace slow thermal or power models, and parameters can be refitted to flight data as the spacecraft ages. Twins can then answer “what if” questions in seconds. In December 2025 JPL checked rover drives proposed by vision–language models on a digital twin of Perseverance before sending them to Mars. The twin simulates more than 500 000 telemetry variables (Chapters 44 and 48).

**FIGURE 49.8** OPTIMISM, the full-scale engineering twin of the Perseverance rover, at the Mars Yard garage at NASA’s Jet Propulsion Laboratory on 29 October 2021. Physical test beds like this one and software twins are used to check commands before they are sent to Mars. Image: NASA/JPL (NASA/JPL-Caltech).

### 49.4.3 Language-Model Assistants

Operations knowledge is scattered across procedures, design documents, test reports, anomaly reports, logs and the telemetry database. Finding the right page during an anomaly can take longer than fixing the fault. **Large language models** (LLMs; Chapter 44) can search, summarise

and draft from this material. The safest pattern is **retrieval-augmented generation** (RAG): the system retrieves the relevant passages and the model answers from them, citing its sources.

ESOC's **Operations Companion (OCAI)** is the best-documented example. A 2025 paper reports it in use on 14 ESOC missions, for search and for training new flight control teams. It combines keyword search, semantic search on embeddings and a knowledge graph of operations terms. It covers command and telemetry archives, procedures, anomaly reports, manuals and logs, and language-model answers with references were being added. Telespazio Germany planned to build it into its commercial mission control system, with constellr's HiVE constellation as pilot.

Start-ups are also building language-model features into mission control software. One, Quindar, raised 18 million US dollars in November 2025 for software that covers planning, flight dynamics and command and control. Earlier that year it described an "AI autopilot" with manual override. Published evidence of how well such assistants work in real operations is still thin. Table 49.5 lists the main risks and the usual defences.

**TABLE 49.5** Risks of language-model assistants in mission operations and common defences.

| Risk | Example in operations | Defence |
|---|---|---|
| Hallucination | Invented limit value or procedure step | Answers only from retrieved sources, with citations; people check |
| Stale or wrong sources | Superseded procedure retrieved | Configuration-controlled document store; version tags |
| Over-trust | Operator accepts a plausible diagnosis | Show evidence and confidence; training; audits |
| Unsafe action | Assistant drafts a hazardous command | No direct command access; validation and approval gates |
| Data leakage | Export-controlled data sent to an outside service | Models hosted on site or in accredited clouds; access control |
| Prompt injection | Malicious text in a log or document changes behaviour | Treat retrieved text as data; restrict tools (Chapter 53) |

### 49.4.4 Natural-Language Commanding

Telling a spacecraft what to do in plain words is still research. In one 2024 study, a language model flew spacecraft in a simulated pursuit–evasion game and ranked second in the challenge. Others propose models that turn goals into plans for an onboard planner.

The risks are clear. A spoken request is ambiguous, while a telecommand must be exact. A model can produce a command that is valid in form but wrong in intent: the right command sent to the wrong unit, or at the wrong time. Any practical system therefore needs strict layers. The model translates the request into a structured command. The command is checked against the command database and flight rules. It is simulated on the digital twin. A person approves it, and critical commands keep their usual two-step protection.

### 49.4.5 People and Automation

Automation changes the human job; it does not remove it. Decades of research in aviation and process control name three effects. **Automation bias** is the tendency to accept a machine's output without checking, or to miss what it fails to flag. **Trust calibration** is the match between how

much people trust a tool and how reliable it is. Both over-trust and under-trust are harmful. **Skill fade** is the loss of manual skill and system knowledge when people rarely practise them.

Together they create an irony first described in 1983: the more reliable the automation, the less prepared people are for the rare day it fails. Good practice follows. Show the evidence behind each alert, not just a score. Report each detector's measured precision. Keep people practising failures on the simulator. And hand over from software to person with enough context to act.

## 49.5 Scheduling Contacts and Ground Networks

The key idea of this section is that antenna time is a scarce resource shared by many spacecraft. Optimisation, not learning, does most of the work, but learning helps to predict demand and to speed up solvers.

### 49.5.1 The Deep Space Network

NASA's **Deep Space Network** (DSN) serves about 40 missions with antennas at Goldstone (Figure 49.9), Madrid and Canberra (Chapter 7). Demand routinely exceeds capacity, often by up to 40 %. JPL reported that the gap between requested and schedulable time grew from about 300 hours a week in 2011–12 to about 500 hours by late 2015.

Since July 2011 the DSN has been scheduled with **Service Scheduling Software** (S3). Each mission states its needs: hours, antennas and time windows. A scheduling engine turns them into tracks, using systematic search and then local repair of conflicts, and the missions negotiate the draft peer to peer. JPL has studied deep reinforcement learning and mixed-integer programming for the same problem (2021); both are research. The hardest part is agreeing what "fair" means.

### 49.5.2 Ground Stations as a Service

Small-satellite operators often buy antenna time instead of building stations. **Ground-station-as-a-service** (GSaaS) providers sell passes or minutes on shared networks through web interfaces (Chapter 7). Amazon Web Services opened AWS Ground Station in 2019. Microsoft retired its Azure Orbital Ground Station on 18 December 2024 and sold its ten 6 m antennas. KSAT lists more than 300 antennas at 28 locations, and in March 2022 its small-satellite network handled more than 52 000 passes.

On a shared network, each operator wants enough contacts with short gaps at low cost, and the provider wants busy antennas without conflicts. Automated interfaces with priorities and pre-emption rules make this work. Learning helps to forecast demand and to predict which passes will fail. Contacts through relay satellites are booked in the same way (Figure 49.10).

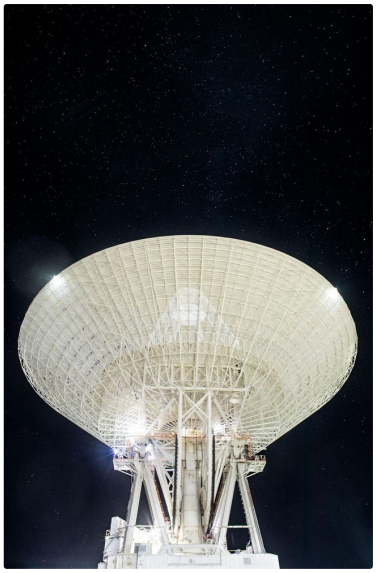


**FIGURE 49.9** The 70 m antenna DSS-14 at Goldstone, California, in September 2025. Its time is shared among dozens of missions through automated scheduling and negotiation. Image: NASA/JPL (NASA/JPL-Caltech).

**FIGURE 49.10** The relay satellite TDRS-L being prepared in January 2014, shortly before launch. Relay contacts are booked like ground-station passes. Image: NASA/KSC (NASA/KSC - Dimitri Gerondidakis ).

### 49.5.3 Scheduling a Constellation's Contacts

The problem can be stated exactly. A **pass** $p$ is a window when satellite $s(p)$ is visible from station $g(p)$, from $t_p^{\text{start}}$ to $t_p^{\text{end}}$. A binary variable $x_p$ is 1 if the pass is used. Each antenna can track one satellite at a time, with a set-up gap between passes. Each satellite can talk to one station at a time and has $D_s$ gigabytes waiting. The delivered data $y_s$ is limited by both:

$$\max \sum_{s} y_s + \lambda\, z \quad \text{subject to} \quad y_s \le D_s, \quad y_s \le \sum_{p:\, s(p)=s} R\,(t_p^{\text{end}} - t_p^{\text{start}})\, x_p, \quad z \le y_s / D_s \ \text{for all } s,$$

with the antenna and satellite conflict constraints. Here $R$ is the downlink rate and $z$ is the share of data delivered for the worst-served satellite, so $\lambda$ buys fairness. This is a **mixed-integer linear programme** (MILP). Standard solvers handle problems with thousands of passes in seconds.

In Figure 49.11 it is compared with two greedy rules for 36 imaging satellites in six polar planes and three single-antenna stations: Svalbard, Inuvik and Awarua. Over 24 hours there are 827 visible passes. Each satellite has 100–300 GB waiting, 7 040 GB in all, at 3 GB a minute. At Svalbard more than one satellite is in view 54 % of the time, so passes compete. Taking passes first-come, first-served delivers 69.8 % of the data and leaves nine satellites below half. Taking the longest passes first delivers 76.9 %. The MILP, solved in under a second, delivers 81.6 %, and every satellite gets at least 66 %.

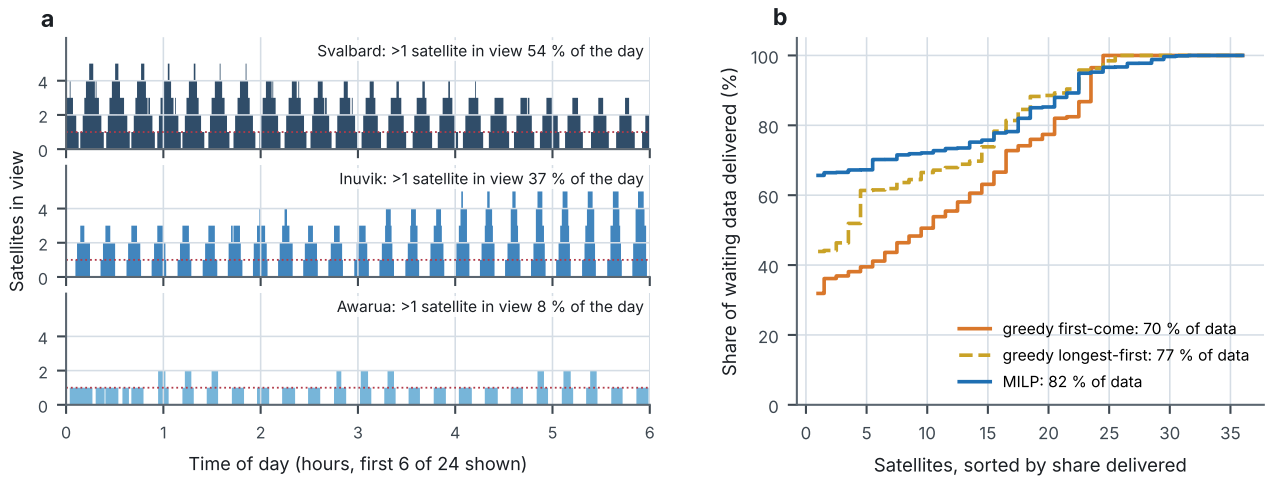



**FIGURE 49.11** Scheduling 36 satellites over three single-antenna stations for one day. (a) Number of satellites in view of each station during the first 6 hours; above the red line passes compete for one antenna. (b) Share of each satellite's waiting data delivered, sorted, for two greedy rules and the mixed-integer programme.

Real schedulers add slew times, uplinks, data generated through the day, station costs and weather, and re-plan as passes fail. Learning can predict inputs, such as data volumes and pass reliability, or learn heuristics that speed up a solver. The optimisation model still provides the constraints and the yardstick.

## 49.6 AI in Space Communications

The key idea of this section is that radios increasingly sense their environment and adapt to it. Learning helps to recognise signals, choose waveforms, share beams and spectrum, and find interference. Most of it is research or early deployment, but some is already inside commercial systems.

### 49.6.1 Cognitive Radio and Spectrum Sensing

A **cognitive radio** senses its radio environment and changes its frequency, power, waveform or timing to work better. The simplest sensing method is **energy detection**: compare the power in a band with the noise floor. More advanced methods recognise the type of signal present, which is where learning enters.

NASA's **SCaN Testbed** on the International Space Station (Figure 49.12), operated from 2012, was the main flight laboratory for such ideas. It carried three software-defined radios that could be reprogrammed from the ground. In a 2018 experiment, a cognitive engine on the ground used reinforcement learning to choose channels. When jammed, it told the satellite transmitter to switch to the channel expected to stay clear longest. The closed loop worked against several types of jammer, but the published summary gives no throughput figures.

**FIGURE 49.12** The SCaN Testbed, then called CoNNeCT, during a test at NASA's Glenn Research Center before its launch to the International Space Station. Its three software-defined radios were used from 2012 to try reconfigurable and cognitive communication in orbit. Image: NASA/GRC.

## 49.6.2 Recognising Signals

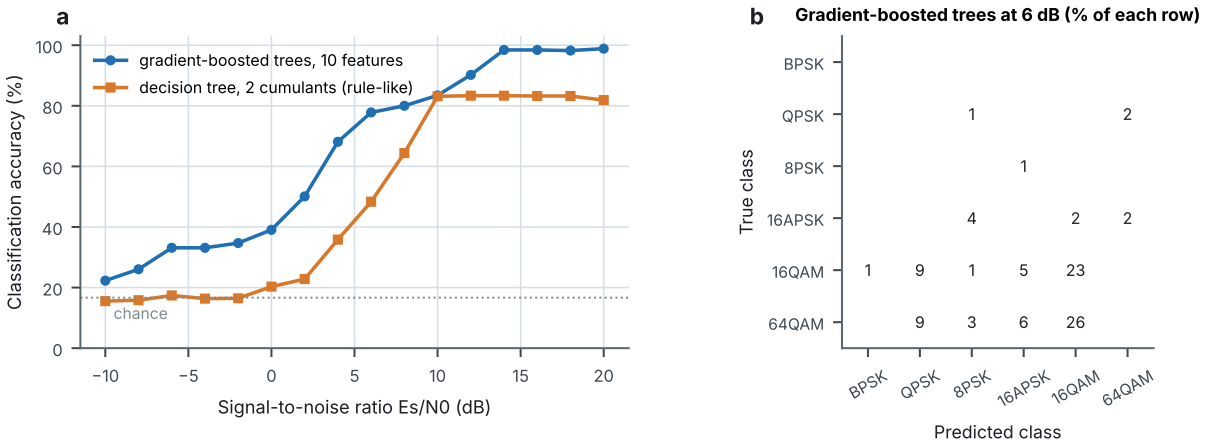
**Automatic modulation classification** identifies how a received signal is modulated — BPSK, QPSK, 16QAM and so on (Chapter 7). It is used in spectrum monitoring, interference hunting and signals intelligence (Chapter 19). The classical method uses **higher-order cumulants**: statistics of the complex baseband samples $y$ that differ between constellations but not with carrier phase. For example, with $M_{pq} = \mathbf{E}[y^{p-q}(y^*)^q]$,

$$C_{40} = M_{40} - 3M_{20}^2, \qquad C_{42} = M_{42} - |M_{20}|^2 - 2M_{21}^2.$$

Normalised by $M_{21}^2$, noise-free BPSK has $C_{42} = -2$, QPSK $-1$ and 16QAM $-0.68$. Noise pushes all of them toward zero.

Deep learning entered this field in 2016, when researchers fed raw I/Q samples to convolutional networks. Their open RadioML data set of 2018 holds 24 modulations and 2.56 million examples at signal-to-noise ratios (SNR) from −20 to +30 dB. Residual networks reached about 95 % accuracy at high SNR, and much less at low SNR.

The script behind Figure 49.13 repeats the classical approach on synthetic signals. Six modulations used in satellite links are generated, 512 symbols each, with random phase, a small frequency offset, unknown gain and noise. Ten cumulant and amplitude features feed two classifiers. A depth-3 decision tree on only $|C_{40}|$ and $C_{42}$, a rule-like expert classifier, levels off at 83 % because it cannot separate 16QAM from 64QAM. Gradient-boosted trees on all ten features reach 98–99 % above 14 dB, 78 % at 6 dB and 39 % at 0 dB, against 17 % for chance. At 6 dB the dense QAM constellations are still confused with each other.




Synthetic signals: 6 modulations, 512 symbols per example at one sample per symbol, random carrier phase, small frequency offset and unknown gain, white Gaussian noise; 160 training and 160 test examples per class and SNR. Features: normalised cumulants (C20, C40, C41, C42, C60, C63, M80) and amplitude statistics.

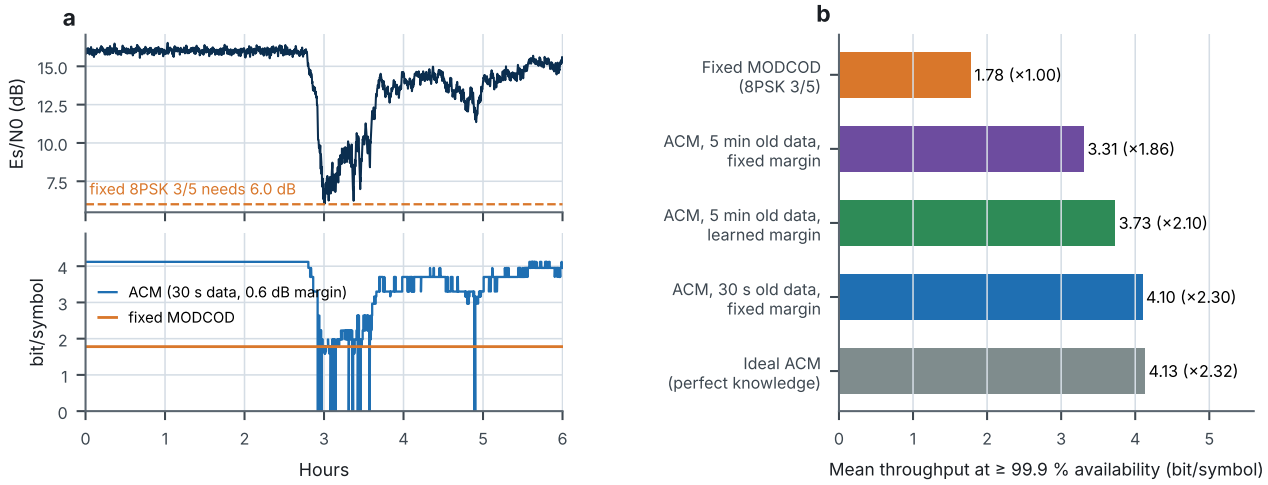
**FIGURE 49.13** Modulation classification of synthetic I/Q signals. (a) Accuracy against signal-to-noise ratio for a two-feature decision tree and for gradient-boosted trees on ten features. (b) Confusion matrix of the boosted trees at 6 dB.

### 49.6.3 Adaptive Coding and Modulation

Satellite links above 10 GHz fade in rain (Chapter 17). A link with **constant coding and modulation** (CCM) must use a robust waveform that works through the worst fade it is designed for. In clear sky, which is most of the time, it wastes capacity. **Adaptive coding and modulation** (ACM), standardised in DVB-S2 (2005) and DVB-S2X (2014), changes the modulation and code rate — the **MODCOD** — as the measured signal-to-noise ratio changes. The receiver reports its $E_s/N_0$ (energy per symbol over noise density). The transmitter then picks the most efficient MODCOD whose threshold is below the measured value minus a safety margin.

The simulation in Figure 49.14 covers one year of a Ka-band downlink with a clear-sky $E_s/N_0$ of 16 dB. Rain falls 5.4 % of the time, with fades from a standard log-normal model: 2.3 dB is exceeded 1 % of the time and 9.4 dB 0.1 % of the time. For 99.9 % availability, a fixed link must use 8PSK 3/5 at 1.78 bit/symbol. ACM driven by measurements 30 s old needs only a 0.6 dB margin and averages 4.10 bit/symbol — 2.3 times more — within 1 % of ideal ACM.

Learning does not help a fast loop, which already works almost ideally. It helps when decisions must be made well ahead, such as planning a pass or choosing beams and gateways minutes early. If the measurement is 5 minutes old, a fixed margin of 3.0 dB is needed and throughput falls to 3.31 bit/symbol. The script then learns a state-dependent margin from the first 90 days. It is a quantile of how much the fade can change, given the current fade and its trend. This raises throughput to 3.73 bit/symbol — 13 % more at the same availability — because the margin is small in clear sky and large in rain. Published research uses neural networks and weather radar or forecasts in the same role.




Synthetic Ka-band downlink, one year at 10 s steps: rain 5 % of the time with log-normal fades (Maseng–Bakken model; exceeded 1 % of the time: 2.3 dB, 0.1 %: 9.4 dB) plus scintillation; clear-sky Es/N0 16 dB; DVB-S2 thresholds (ETSI EN 302 307-1) plus 0.5 dB implementation margin. (b) Each ACM scheme uses the smallest safety margin that keeps unavailability at or below 0.1 %; the learned margin is a quantile of the fade change predicted from the current fade and its trend (trained on the first 90 days, scored on the rest).

**FIGURE 49.14** Adaptive coding and modulation over a synthetic Ka-band rain-fade year. (a) Six hours around the year's deepest fade: Es/N0 (top) and spectral efficiency of fixed and adaptive MODCODs (bottom). (b) Mean throughput at 99.9 % availability for fixed coding, three ACM schemes and ideal ACM.

### EXAMPLE: THROUGHPUT GAIN OF ADAPTIVE CODING AND MODULATION

Use the results of Figure 49.14 for a carrier of 250 million symbols per second (Mbaud). Ignore framing overhead.

1. **Fixed coding.** 8PSK 3/5 needs 5.5 dB plus a 0.5 dB implementation margin; it works 99.912 % of the time. Throughput: $250 \times 10^6 \times 1.78 \times 0.99912 \approx$ **445 Mbit/s**.
2. **ACM, 30 s loop.** Mean 4.096 bit/symbol: $250 \times 10^6 \times 4.096 \approx$ **1 024 Mbit/s**, a factor of **2.30**.
3. **Ideal ACM.** 4.130 bit/symbol, 1 033 Mbit/s: the 0.6 dB safety margin costs less than 1 %.
4. **Why so large?** In clear sky the link uses 32APSK 5/6 at 4.12 bit/symbol, which needs 14.3 dB plus margin. The fixed link must keep about 10 dB in reserve (16 dB available, 6.0 dB needed) for rare fades.
5. **Five-minute-old data.** A fixed 3.0 dB margin gives 3.305 bit/symbol (826 Mbit/s). The learned, fade-dependent margin gives 3.727 bit/symbol (932 Mbit/s): **13 % more** at the same 99.9 % availability.
6. **Caveat.** The gain depends on climate, frequency and clear-sky margin. Systems with a smaller clear-sky margin gain less.

## 49.6.4 Beams, Power and Spectrum in High-Throughput and LEO Systems

High-throughput satellites have hundreds of spot beams, and LEO broadband satellites steer many beams with phased arrays (Chapter 17). Demand is uneven in space and time. **Beam hopping** lights each beam only part of the time, in a pattern that follows demand. The first over-the-air demonstration, in ESA's BEHOP project in June 2018, used DVB-S2X "superframes" through a conventional Eutelsat satellite. Digital payloads, such as those on Eutelsat Quantum (launched 2021) and many newer satellites, make such flexibility routine.

Choosing which beams to light, with how much power and bandwidth, is a large combinatorial problem that changes from milliseconds to minutes. Operators use optimisation and heuristics. Since about 2018, many papers have trained **deep reinforcement learning** agents for beam

hopping, power control and channel allocation; in simulation they often beat simple heuristics. Hybrids, in which a network proposes a pattern and an optimiser repairs it, look most promising. Operators rarely publish what their resource managers use, so operational use is hard to confirm.

### 49.6.5 Interference: Detection and Geolocation

Interference is a daily problem for satellite operators. Causes include badly pointed terminals, cross-polarised carriers, terrestrial 4G and 5G signals and, increasingly, deliberate jamming (Chapter 18 for navigation signals). Monitoring systems scan the spectrum around the clock. Learning helps in three steps, as in ESA's SkyMon project, completed in 2021. A model first recognises known carriers from their spectral shape. A detector then flags spectra that depart from the expected mask, and a classifier names the type of interference. Researchers at KTH and the Swedish Space Corporation showed in 2019 that LSTM models trained on real spectrum data could find short and long interference events in time and frequency.

Finding *where* an interferer is uses physics. The uplink is received through the wanted satellite and a neighbour; the differences in arrival time and Doppler shift define curves on the Earth that cross at the transmitter (Chapter 19).

## 49.7 Networks in Orbit

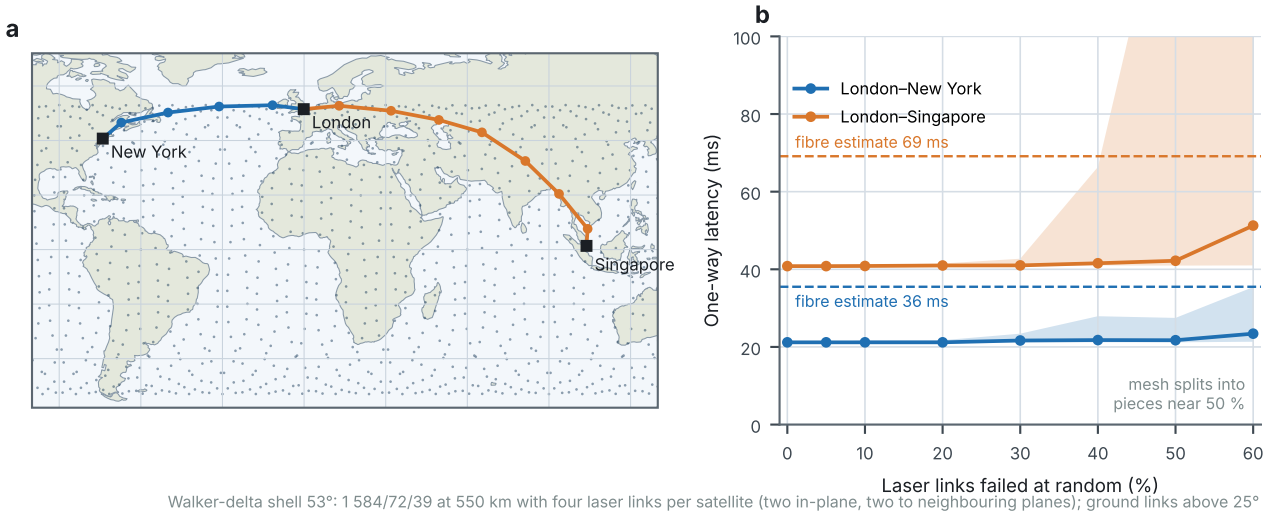
The key idea of this section is that a constellation with inter-satellite links is a network whose shape changes every second. Classical routing works well. Learning is studied for traffic prediction, failures and the joint control of space and ground networks.

### 49.7.1 Routing in Laser Meshes

Iridium has routed calls through radio crosslinks since its service began in 1998, with up to four links per satellite. Starlink uses lasers. In January 2024 SpaceX reported more than 9 000 lasers in orbit carrying more than 42 petabytes a day. Links ran at up to 100 Gbit/s over distances up to 5 400 km. The constellation has grown much larger since. The US Space Development Agency's Transport Layer also links its satellites optically (Chapter 19).

In a **Walker-delta** shell (Chapter 11), each satellite can link to two neighbours in its own plane and one in each neighbouring plane. This gives a grid that wraps around the Earth. Routing then becomes a **shortest-path** problem on a graph: satellites are nodes, links are edges weighted by distance (delay), and Dijkstra's algorithm finds the fastest path. Because the satellites move, the graph is recomputed every few seconds. Light in vacuum travels about 47 % faster than in optical fibre, so long space paths can beat fibre.

The script behind Figure 49.15 builds a 1 584-satellite shell at 550 km and 53°, with four laser links per satellite, using the networkx library. From London to New York the best path uses four satellites and three laser hops. The one-way delay is 21.2 ms, against about 35.5 ms for fibre that is 1.3 times longer than the great circle. London to Singapore takes 40.8 ms, against about 69 ms. When links fail at random, routing simply goes around them. With 30 % of links failed, the median delay hardly changes. Near 50 %, the grid starts to break into pieces, as theory predicts for a square grid, and some routes need long detours.




Walker-delta shell 53°: 1 584/72/39 at 550 km with four laser links per satellite (two in-plane, two to neighbouring planes); ground links above 25° elevation; snapshot geometry. Shortest paths by distance (Dijkstra, networkx); 30 random failure draws per point, band 5–95 %. Fibre: 1.3 × great-circle distance at c/1.468.

**FIGURE 49.15** Routing in a laser-linked Walker-delta constellation. (a) Snapshot of the 1 584 satellites (dots) and the shortest paths from London to New York and Singapore. (b) One-way delay against the share of laser links failed at random (median and 5–95 % band of 30 draws), with fibre estimates.

Shortest paths are easy; managing load is not. Traffic moves with the time of day, and gateways and laser terminals fail or lose sight of each other. Research uses learned traffic forecasts, reinforcement-learning routing and graph neural networks to spread load and pre-compute routes. The bar is high: a learned router must beat classical algorithms that are fast, predictable and easy to verify.

### 49.7.2 Network Digital Twins and Self-Driving Networks

A **network digital twin** is a running model of a whole network: satellites, links, gateways, traffic and the control software. Operators use it to test changes before applying them — a new routing policy, a software update, a gateway closure. Learned models make twins fast enough to run in a loop. One example is a graph neural network that predicts delay and loss for a traffic pattern far faster than a packet-level simulator.

The long-term aim is a **self-driving network**: operators state goals, such as “keep latency below 40 ms”, and software configures, watches and corrects the network. How much of today’s automation uses learning, rather than rules and optimisation, is rarely published.

### 49.7.3 Toward AI-Native 6G Non-Terrestrial Networks

Satellites are joining the mobile standards. The 3GPP body added **non-terrestrial networks** (NTN) in Release 17 (2022), so that phones and IoT devices can use satellites. Release 19 (2025) added **regenerative payloads**, which carry a full base station on the satellite. 6G studies started in Release 20, including learning-based features of the radio interface such as compressed channel-state feedback. First 6G specifications are expected around 2028–29. The ITU’s framework for 6G (IMT-2030), adopted in 2023, lists AI and ubiquitous connectivity among its aims.

“AI-native” means that learned models are built into the radio and network from the start, with standard ways to train, test and update them. For satellites this could cover beam management, handover prediction for fast-moving cells, Doppler and timing correction, and energy saving. All

are research or pre-standard work in 2026. Direct-to-device services from Starlink, AST SpaceMobile and others (Chapter 17) give the commercial pull. Table 49.6 summarises the status of AI across communications and networks.

**TABLE 49.6** Status of AI in space communications and networks, October 2026.

| Problem | Typical method | Status (2026) |
|---|---|---|
| Spectrum monitoring, carrier recognition | Learned classifiers on spectra | Commercial products and ESA projects |
| Modulation classification | Cumulant features; convolutional and residual networks | Mature research; used in monitoring and signals intelligence |
| Interference detection | LSTM and anomaly detection on spectra | Demonstrated on real data; early products |
| ACM and power control | Feedback loops; learned fade prediction | ACM operational; learning research |
| Beam hopping, resource allocation | Optimisation; deep reinforcement learning | Optimisation operational; RL research |
| Anti-jamming channel selection | Reinforcement learning | Flight experiment (SCaN Testbed, 2018) |
| Laser-mesh routing | Shortest paths; learned traffic forecasts, RL | Classical operational; learning research |
| 6G NTN air interface | AI/ML features under study in 3GPP | Standards work |

## 49.8 Securing Ground Segments and Links

The key idea of this section is that the ground segment is the easiest way into a space system. Machine learning helps defenders spot unusual behaviour, but attackers can target the learning itself (Chapter 53).

Ground systems are ordinary computer networks with antennas, modems and cloud links attached, and they face the same attacks as any business network. The best-known case came on 24 February 2022, the first day of Russia's full-scale invasion of Ukraine. Attackers abused the management network of Viasat's KA-SAT system to push a "wiper" program, later named AcidRain, to user modems. Tens of thousands of modems in Ukraine and across Europe stopped working. Spill-over cut remote monitoring of 5 800 Enercon wind turbines in central Europe. The European Union attributed the attack to Russia in May 2022.

Learned defences work at several layers:

- **Network intrusion detection.** Models of normal traffic between ground-segment computers flag unusual connections, volumes or timing. Like telemetry monitors, they must meet a strict false-alarm budget.
- **Command anomaly detection.** A model of normal command sequences flags commands that are valid but unusual: wrong time, wrong order, wrong source. This adds to the encryption and authentication of telecommands (CCSDS Space Data Link Security; Chapter 7).
- **Telemetry for security.** Telemetry anomaly detectors can also reveal tampering, such as an unexpected mode change.

- **Jamming and spoofing detection.** Learned classifiers on signal power, spectra and receiver statistics detect jamming of command links and spoofing of navigation signals (Chapter 18). Research has trained such detectors for intermittent jamming of telemetry and telecommand links.

The Aerospace Corporation's SPARTA framework (2022) catalogues attack techniques against spacecraft, including some shown only in laboratories, and helps defenders plan detections. Two warnings apply to learned defences. Attackers can evade them by mimicking normal behaviour or slowly poisoning training data. And the models, pipelines and assistants become new targets (Chapter 53).

### PRACTICE: DEPLOYING AN ANOMALY-DETECTION SYSTEM IN A CONTROL CENTRE

1. **Start from the operators' needs.** Agree which subsystems matter most, what a useful alert looks like and how many alerts a day the team can handle — for example, 10. This budget sets every threshold.
2. **Assemble the data.** Collect several years of telemetry, command logs, timelines and anomaly reports. Label known anomalies and rare nominal events, such as manoeuvres and calibrations, with the engineers who saw them.
3. **Keep the baseline.** Keep limit checks running unchanged. The learned system adds alerts; it does not remove existing protection.
4. **Choose simple models first.** Start with per-channel prediction residuals and a nearest-neighbour or isolation-forest score, using orbit phase and commanded modes as inputs. Try deep networks only if they clearly do better on held-out data.
5. **Test like an operator.** Use a time-ordered split with a full year of test data. Score detected events, alerts per day and warning time, not just sample accuracy (ESA-ADB is a good template).
6. **Run in shadow mode.** Run the system beside normal operations for several months without paging anyone. Review every alert with the engineers and record whether it was useful.
7. **Explain each alert.** Show the channels involved, the expected and measured values, and similar past cases. Engineers will not act on a bare score.
8. **Plan for change.** Mark planned events so that they are not flagged, retrain after configuration changes and season shifts, and track alert precision every month.
9. **Go live gradually.** Start with one subsystem during working hours, with a named owner and a way to suppress or tune noisy detectors. Write down what the system does not cover.

### KEY TAKEAWAYS

- Modern spacecraft produce tens of thousands of telemetry parameters (JWST about 40 000, Sentinel-3 about 46 000), and constellations fly hundreds to thousands of satellites. Operations are moving to automation and management by exception.
- The false-alarm budget drives every monitor: 10 000 channels checked every 10 s need fewer than about $1.2 \times 10^{-7}$ false alarms per check to stay below 10 a day.
- Learned detectors that use context — orbit phase, mode and other channels — catch contextual anomalies and drifts that fixed limits miss. Generic detectors can do worse than limits, and real benchmarks such as ESA-ADB remain hard.
- Prognosis needs a physical degradation model, honest uncertainty and data from the vehicle itself. Predictions tighten as data accumulate.
- Language-model assistants are useful for search and drafting when grounded in controlled documents. Natural-language commanding is research and needs strict validation, simulation and human approval.
- Contact scheduling is an optimisation problem: in this chapter's example a mixed-integer programme delivered 81.6 % of waiting data, against 69.8 % for first-come scheduling, and served every satellite fairly.
- In communications, ACM roughly doubles Ka-band throughput over fixed coding. Learning helps most where decisions are made well ahead. Laser meshes route around many failures with classical shortest-path algorithms. Ground segments remain the main cyber target.

### REVIEW QUESTIONS

1. A satellite has 4 000 channels sampled every 2 s. How many checks are made per day? What per-check false-alarm probability allows 5 false alarms a day, and what Gaussian threshold does that correspond to?
2. Explain the difference between point, contextual and drift anomalies, with one spacecraft example of each. Which kind can fixed limits never detect, and why?
3. Write the standardised residual used by a prediction-based detector. Why did the predictive network in Figure 49.3 detect a value copied from another part of the orbit, while the fixed limits did not?
4. Why did the isolation forest in Figure 49.4 miss spikes that left the normal range? What would you change in its inputs or in the system around it?
5. ESA-ADB scores detectors event by event and puts precision first. Why is this closer to operators' needs than sample-by-sample accuracy? What made Mission 2 hard?
6. In Figure 49.6, why is the 90 % interval for the end-of-life date much wider at 2.5 years than at 4.5 years? What would happen to the intervals if the model form were wrong?
7. List four risks of a language-model assistant in a control centre, and a defence against each. Why should such an assistant not send commands directly?
8. State the decision variables, constraints and objective of the contact-scheduling MILP. Why did first-come, first-served scheduling leave nine satellites below half their data?
9. A Ka-band link has a clear-sky Es/N0 of 16 dB and must tolerate a 9.4 dB fade. Using the DVB-S2 thresholds in this chapter, compare the spectral efficiency of fixed coding with that of ACM in clear sky. Why does a learned margin help more when decisions use 5-minute-old data than 30-second-old data?
10. In a laser mesh, why does the delay between London and New York change little when 30 % of links fail, yet rise sharply near 50 %? Give two ways in which learning could improve routing beyond shortest paths.

CHAPTER

# 50 AI for Space Situational Awareness, Defence and Security

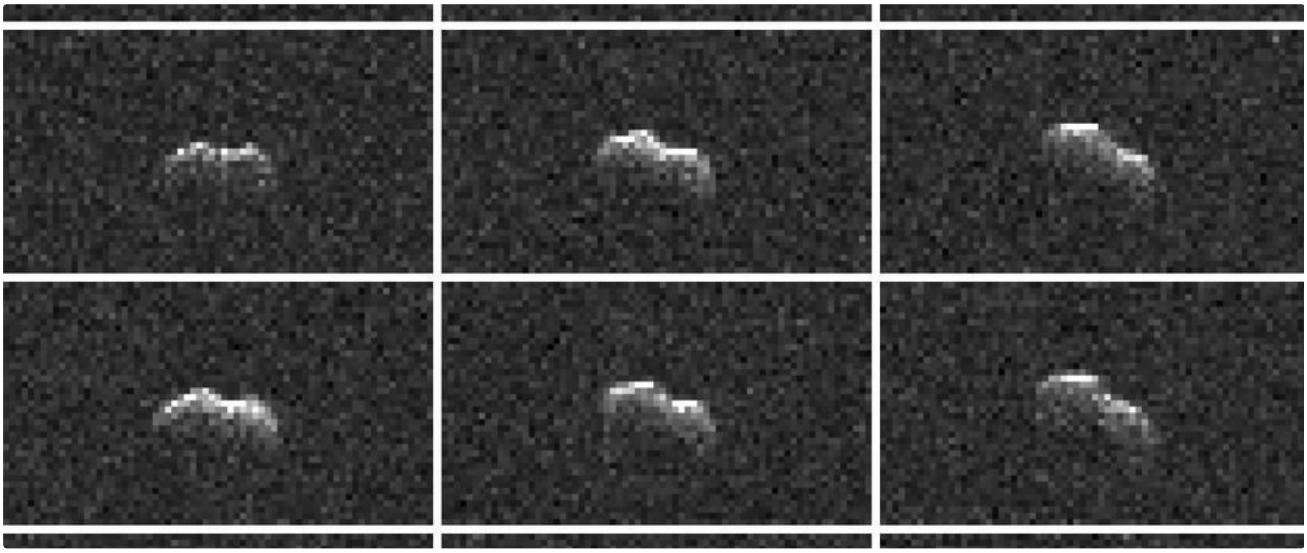


Image: NASA/JPL (NASA/JPL-Caltech and NSF/AUI/GBO)

*More than 35 000 objects are catalogued in Earth orbit, and the number of close approaches grows with the square of that number. This chapter shows where machine learning and optimisation help to keep track of it all, and where they do not. It covers streak detection, data association, orbit prediction with learned corrections, storm-time drag, conjunction screening and triage, manoeuvre detection, light-curve characterisation and re-entry. It then turns, from open sources only, to defence and intelligence uses — imagery exploitation, tip-and-cue, emitter recognition, missile tracking, battle management, proximity operations, deception and human–machine teaming — and ends with the law, policy and stability questions they raise. Small models run in Python make each method concrete.*

IN THIS CHAPTER

50.1 Space Situational Awareness at Machine Scale
50.2 Conjunction Assessment and Space Traffic
50.3 Watching Behaviour: Manoeuvres, Characterisation and Re-entry
50.4 AI in Defence and Intelligence Space Systems
50.5 Law, Ethics and Policy
50.6 Lessons from the Experiments

Space situational awareness (SSA) means knowing what is in orbit, where it is going and what it is doing. The hero image above shows an older side of the job. Engineers of NASA's Orbital Debris Program Office are recording impact craters on a Hubble camera that astronauts brought home

in 2009. Every crater is a small piece of evidence about the debris population. Today the evidence arrives as millions of radar and telescope observations a day. Software must turn them into orbits, warnings and decisions for more than fifteen thousand active satellites.

That software increasingly includes machine learning (ML) — models that learn patterns from examples rather than following hand-written rules. The same methods also serve defence and intelligence. They sift satellite images for change, fuse signals from many sensors and point the next satellite at the right place. This chapter explains both uses. It separates what is operational, what has been demonstrated and what is still research, as of October 2026. Defence topics are covered from open sources only and without operational detail.

The chapter builds on the engineering background of Chapter 21 (SSA and debris), Chapters 19 and 20 (national-security space and missile warning) and Chapter 22 (space weather). Learning methods are introduced in Chapters 43 and 44, data and onboard computing in Chapter 45 and imagery analysis in Chapter 46. Autonomy for rendezvous is covered in Chapter 48 and the assurance of AI in Chapter 53.

## 50.1 Space Situational Awareness at Machine Scale

The key idea of this section is that SSA is a pipeline, from raw sensor data to decisions. Physics does most of the work. Learning helps at a few points where the data are messy, the volume is large or the physics is uncertain.

### 50.1.1 From Sensors to Decisions

Figure 50.1 shows the chain. **Sensors** — radars, telescopes, space-based cameras and commercial networks (Chapter 21) — produce raw data. **Detection** finds objects in that data. **Association** decides which known object each detection belongs to. Detections that match nothing are called **uncorrelated tracks** (UCTs). **Orbit determination** fits an orbit to each object's observations and predicts where it will go. The results form the **catalogue**. In October 2026 the public catalogue held about 35 000 objects in orbit, of which about 17 000 were working satellites.

From the catalogue, **screening** finds every close approach in the next few days. **Risk triage** decides which of them matter. Operators then **decide** whether to manoeuvre and **coordinate** with each other. Alongside this runs **behaviour analysis**: spotting manoeuvres and break-ups and learning what each object is.

Most of this pipeline is classical. Orbit determination uses least-squares fits and Kalman filters (Chapter 43), and screening uses geometry. The US Space Force declared its new catalogue software, ATLAS, operational in September 2025. It is replacing a system that dates from the 1980s. Its Unified Data Library pools government and commercial sensor data for space domain awareness. The service's 2025 Data and AI Strategic Action Plan made the library a formal programme and set goals for adopting AI. Table 50.1 summarises where learning fits.

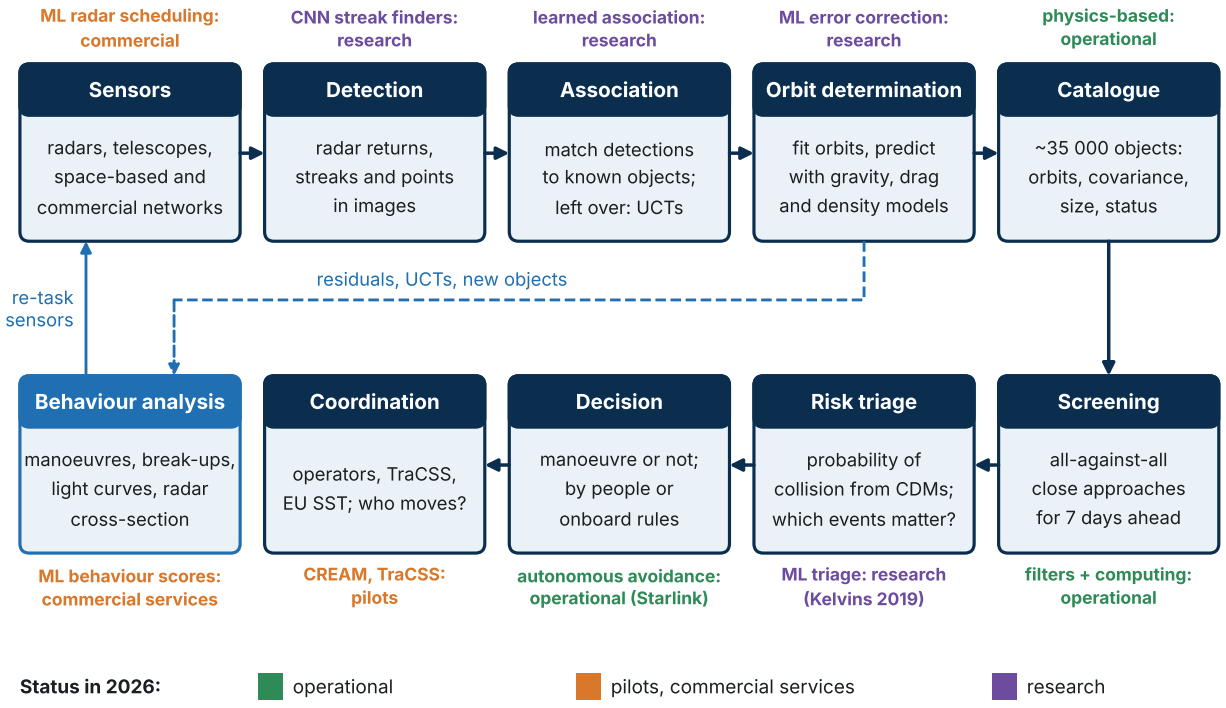



**FIGURE 50.1** The space situational awareness pipeline. The data path (top) turns sensor data into a catalogue; the services path (bottom) turns the catalogue into warnings and decisions. Tags give the status in 2026 of machine learning at each step.

**TABLE 50.1** Space situational awareness tasks, their classical methods and the places where machine learning helps.

| Task | Classical method | Where learning helps | Status (2026) |
|---|---|---|---|
| Detecting objects in images | Matched filters, stacking | CNN detectors for streaks and faint objects | Research; first pipelines |
| Associating detections | Gating and global assignment | Learned similarity after break-ups | Research |
| Predicting orbits | Numerical propagation, fitted drag | Learned corrections of prediction errors | Research |
| Thermospheric density | Empirical and data-assimilating models | Fast surrogates with uncertainty | Research |
| Conjunction triage | Probability of collision thresholds | Ranking events from CDM histories | Research; pilots |
| Manoeuvre detection | Residual tests, CUSUM | Classifiers on orbit histories | Commercial services |
| Characterisation | Expert reading of light curves | Classifiers of light curves | Research; commercial |
| Radar tasking | Fixed schedules and priorities | Learned detection probability | Commercial (vendor-reported) |

## 50.1.2 Finding Faint Streaks

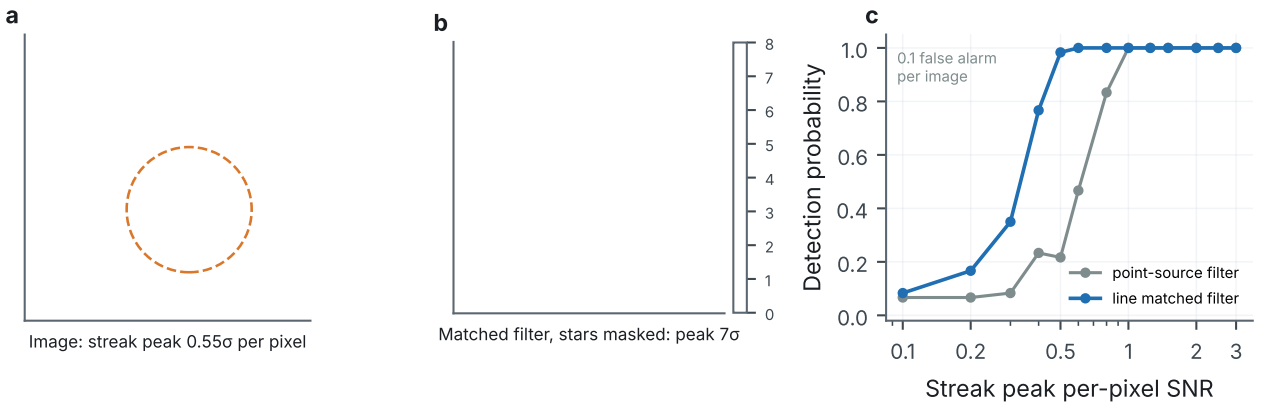
A satellite moves across the sky while a telescope tracks the stars. In a long exposure it therefore appears as a **streak**, not a point. A faint streak spreads its light over many pixels, so no single pixel stands out from the noise. The classic answer is the **matched filter**: correlate the image with a template shaped like the expected streak. For white noise this is the best linear detector.

Suppose the template $s_i$ matches the signal and the noise has standard deviation $\sigma$ per pixel. The filter output then has a signal-to-noise ratio of

$$\mathrm{SNR}_{\mathrm{MF}} = \frac{\sqrt{\sum_i s_i^2}}{\sigma} \approx \frac{A}{\sigma}\sqrt{\sqrt{\pi}\,\sigma_{\mathrm{psf}}\,L},$$

Here $A$ is the peak brightness along the streak, $L$ its length and $\sigma_{\mathrm{psf}}$ the width of the blur. A long streak therefore collects signal from many pixels. The catch is that the direction and length are unknown. The filter must be applied at many angles and positions, and each extra trial adds chances for a false alarm.

Figure 50.2 shows this on a synthetic star field. A 64-pixel streak whose peak is only 0.55 times the noise is invisible to the eye. After bright stars are masked, the matched filter finds it at 7.3σ. In the Monte Carlo test, at the same rate of 0.1 false alarm per frame, the matched filter detected half of the streaks at a peak signal of 0.34σ. A point-source detector needed 0.62σ, so the streak filter reached objects about 1.8 times fainter.




Synthetic 192 × 192 px frame, unit Gaussian noise, 90 stars (PSF σ = 1.3 px) and a 64 px streak. Stars masked above 6σ after PSF smoothing. (c) 60 trials per point on 128 px frames with stars masked; thresholds set from 240 empty frames for 0.1 false alarm per frame.

**FIGURE 50.2** Streak detection by matched filtering. (a) Synthetic frame with stars and a faint streak (circle). (b) Matched-filter output, maximum over 36 angles, after masking stars. (c) Detection probability against the streak's peak signal-to-noise ratio per pixel, for a point-source filter and a line matched filter at 0.1 false alarm per frame.

Learning enters in two ways. Convolutional networks (CNNs, Chapter 44) can be trained on real frames to find streaks of any shape while ignoring satellite trails, cosmic rays and stars. A 2026 study trained an object detector on 2 335 telescope images and reported 94 % precision and 97 % recall. Networks also help with **synthetic tracking**: shifting and adding many short frames along trial velocities, so that faint objects build up while noise averages out. Both are in research use and in some commercial pipelines. Matched filtering and stacking remain the operational workhorses.

Moving-object detection is older than satellites. Asteroid surveys find objects by their motion against the stars, as in Figure 50.3. The same software problems — faint signals, many trials and false alarms — run through both fields.

**FIGURE 50.3** Discovery images of asteroid 2018 LA, taken by the Catalina Sky Survey on 2 June 2018. The small object (circled) moves between frames against fixed stars. It entered the atmosphere over Botswana about eight hours later. Image: NASA/JPL (NASA/JPL-Caltech/CSS-Univ. of Arizona).

### 50.1.3 Association and Uncorrelated Tracks

Each new detection must be linked to a catalogue object. The usual measure is the **Mahalanobis distance** between the detection $\mathbf{z}$ and the predicted position $\hat{\mathbf{z}}$:

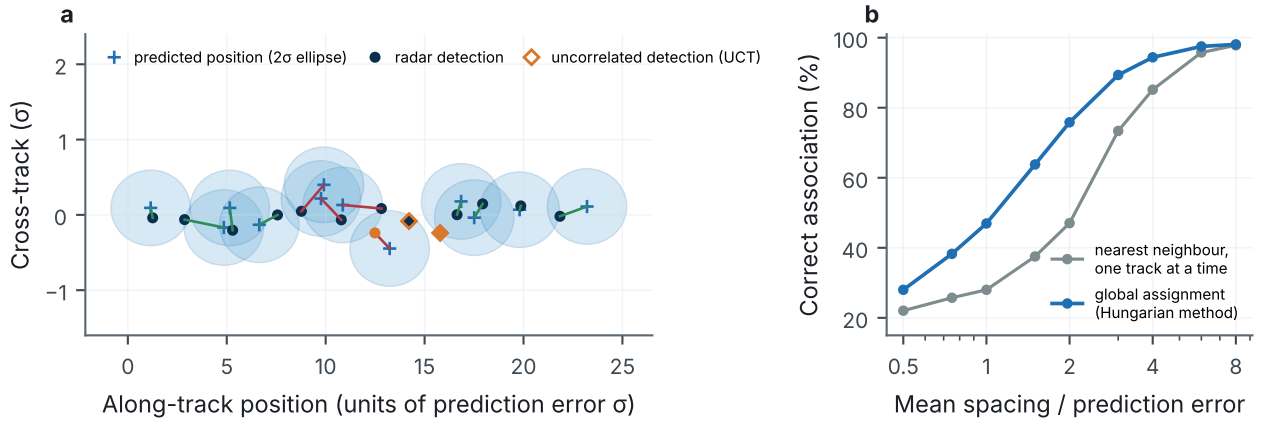
$$d^2 = (\mathbf{z} - \hat{\mathbf{z}})^{\mathsf{T}}\, \mathbf{S}^{-1}\, (\mathbf{z} - \hat{\mathbf{z}}),$$

where $\mathbf{S}$ is the combined covariance of prediction and measurement. It measures distance in units of uncertainty. A **gate** rejects pairs with $d^2$ above a threshold, such as 13.8 for 99.9 % confidence in two dimensions.

Association is easy when objects are far apart. It is hard after a launch that releases dozens of satellites together, or after a break-up. Then many predictions overlap and a greedy rule — give each object its nearest detection — makes chains of mistakes. **Global assignment** instead chooses all links at once to minimise the total cost $\sum_{ij} c_{ij} x_{ij}$, where $x_{ij} = 1$ links object $i$ to detection $j$. The Hungarian method solves this exactly.

Figure 50.4 tests both rules on a synthetic radar pass over a cluster of 24 objects. When the spacing was twice the prediction error, global assignment linked 76 % of objects correctly and the greedy rule 47 %. Both reach about 98 % when the objects are eight errors apart. Detections that match nothing become UCTs. They may be new fragments, unknown objects or simply wrong associations — so each UCT costs analyst time.

Learning helps by improving the inputs. Better predictions shrink the ellipses, and learned similarity — from brightness, radar cross-section or radio signatures — separates objects that geometry alone cannot. Research also uses multi-target trackers that keep several hypotheses alive until new data resolve them.




Synthetic radar pass over objects released together and spread along-track; prediction error σ along-track and 0.25σ cross-track; 92 % detection probability; two unknown objects per pass. Gate $\chi^2 < 13.8$ (99.9 %). (b) 24 objects, 300 trials per point.

**FIGURE 50.4** Data association after a cluster deployment. (a) One synthetic pass: predicted positions with 2σ ellipses, detections, global-assignment links (green right, red wrong) and unmatched detections (UCTs). (b) Share of objects correctly associated against the ratio of spacing to prediction error.

### 50.1.4 Orbit Prediction: Physics Plus Learning

In low Earth orbit (LEO), the largest error in an orbit prediction is usually drag. Drag lowers the semi-major axis $a$ at a rate $\dot{a} = -\rho B \sqrt{\mu a}$, where $\rho$ is air density, $B$ the ballistic coefficient ( $C_D A/m$) and $\mu$ Earth's gravitational parameter. A lower orbit is faster, so a density error makes the object run ahead of or behind its prediction. For a fractional density error $\varepsilon$, the along-track error grows with the square of time:

$$\Delta s(t) \approx \tfrac{3}{4}\, n\, \varepsilon\, |\dot{a}|\, t^2,$$

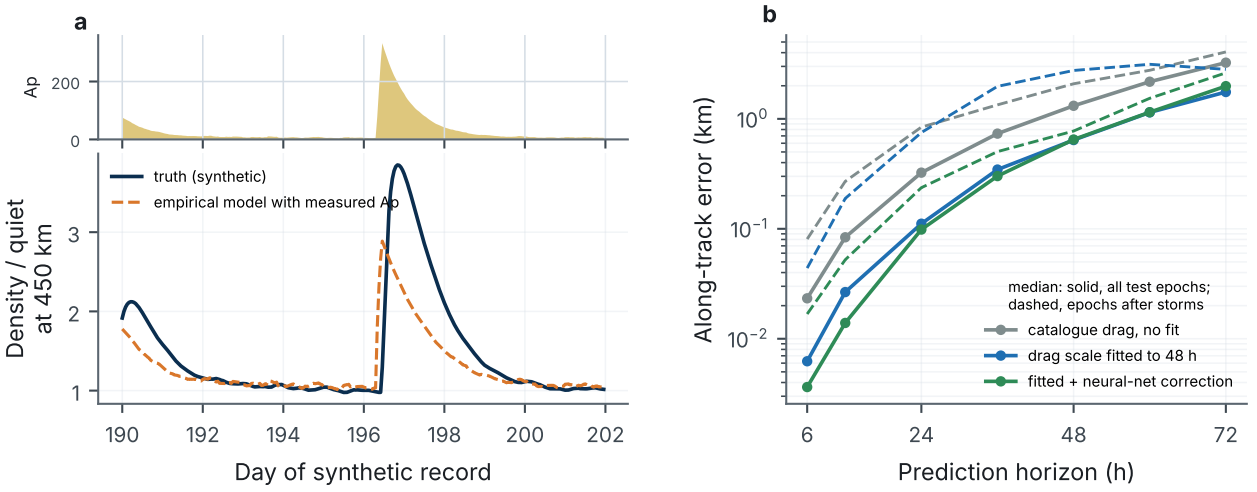
where $n$ is the mean motion. At 450 km, with $|\dot{a}|$ = 50 m/day and $n \approx 96.6$ rad/day, a 20 % density error gives about 0.7 km after one day and 6.5 km after three. This quadratic growth is why conjunction warnings beyond a few days are unreliable.

Operators fight it by fitting a drag scale factor to recent tracking data. Learning adds a **hybrid** step. A physics propagator makes the prediction. A learned model $g_\theta$ predicts the propagator's typical error, which is then removed.

$$\hat{\mathbf{x}}(t) = \mathbf{f}_{\text{phys}}(\mathbf{x}_0, t) + g_\theta(\text{features}).$$

The features include the forecast horizon, altitude, space-weather indices and the errors the propagator made in the recent past. Peng and Bai (2018) showed the idea with neural networks, support vector machines and Gaussian processes on a simulated catalogue. The physics keeps the prediction sensible; the learned part corrects patterns that the physics misses.

Figure 50.5 runs this on 160 synthetic objects over a 240-day record with storms. Fitting the drag scale to 48 hours of data nearly halved the median three-day error, from 3.2 km to 1.75 km. A small neural network on top did not help on average: its median three-day error was 1.98 km. But in the nine test epochs just after storms it cut the median one-day error from 0.75 km to 0.24 km. That sample is small, so the gain is only indicative. The lesson is general. A learned correction must be tested separately in the conditions that matter, because a gain in one regime can hide a loss in another.




160 synthetic objects at 350–650 km over a 240-day record with random storms. Truth: density with a lagged storm response; empirical model: weaker, unlagged response. Forecasts assume Ap persists. Network (2 × 32 tanh, numpy) trained on days 3–150, tested on days 160–236.

**FIGURE 50.5** Storm-time drag and a hybrid correction. (a) Synthetic Ap index (top) and density at 450 km relative to quiet conditions: truth with a lagged storm response and an empirical model with a weaker, unlagged response. (b) Median along-track error against forecast horizon for three predictors, for all test epochs (solid) and for epochs just after storms (dashed).

### 50.1.5 Density and Drag During Storms

Geomagnetic storms heat the thermosphere and can multiply density at satellite heights several times (Chapter 22). Empirical models such as NRLMSIS and JB2008 capture average behaviour. The US Space Force's High Accuracy Satellite Drag Model (HASDM) goes further: it adjusts density every few hours from the observed drag of dozens of calibration objects. Learning offers fast **surrogates** of such models. Licata, Mehta and colleagues compressed 20 years of HASDM density maps with principal component analysis and trained a network on the result. Their model reproduced the database with about 10 % error and gave calibrated uncertainty, even during storms. Such models are in research use; operational density still comes from physics and data assimilation.

#### THE MAY 2024 STORM AND THE CATALOGUE

- **Strength.** The storm of 10–11 May 2024 was the strongest in two decades, with the planetary ap index reaching 400.
- **Density.** Model density at 400 km rose to about six times its value 12 hours before.
- **Forecasts.** The 0–3-day ap forecasts underestimated the rise by 100–300.
- **Decay.** One Earth-observation satellite's decay rate jumped from 38 m to 180 m per day.
- **Operators.** Before the storm about 300 of nearly 10 000 active LEO satellites were manoeuvring. Afterwards thousands manoeuvred together — nearly half of all active LEO satellites, mostly Starlink.
- **Consequence.** Mass manoeuvres invalidate the conjunctions predicted for the coming week, so screening has to restart from new orbits.

Source: Parker and Linares, Journal of Spacecraft and Rockets (2024), from public two-line element data.

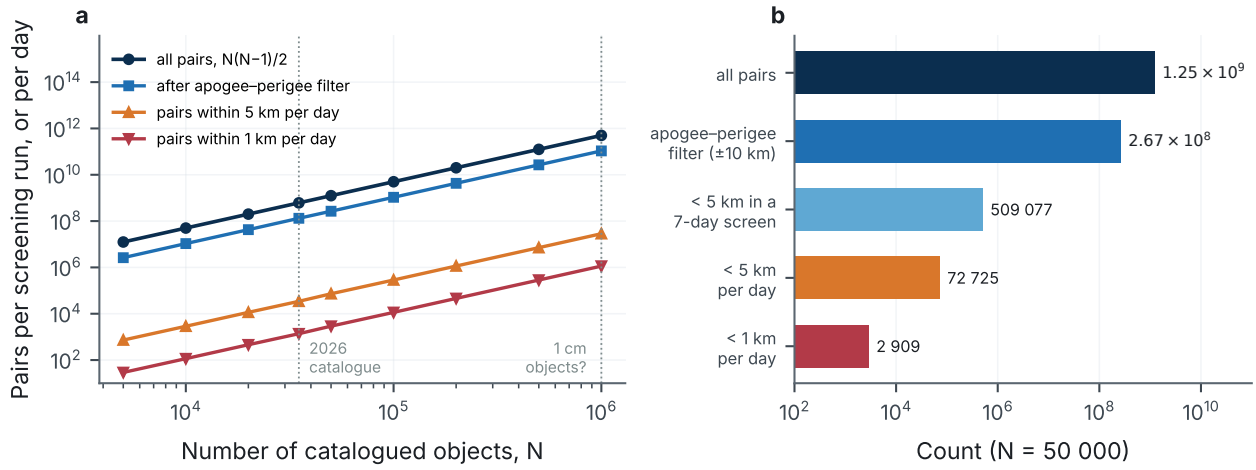
## 50.2 Conjunction Assessment and Space Traffic

The key idea of this section is a funnel. Billions of possible pairs shrink to thousands of close approaches a day and then to a handful of decisions. Computing handles the first steps easily. The hard part is deciding which few events deserve a manoeuvre.

### 50.2.1 Screening at Scale

A catalogue of $N$ objects has $N(N-1)/2$ pairs. Screening programs do not test every pair at every moment. They first apply cheap **filters**. The **apogee–perigee filter** drops pairs whose altitude ranges do not overlap. Orbit-path and time filters then drop pairs whose orbits never come close, or whose closest points are never reached at the same time.

Figure 50.6 shows the result for a synthetic catalogue shaped like today's. The apogee–perigee filter keeps about 21 % of pairs, because most objects crowd into LEO. The number of real close approaches is far smaller. A kinetic-gas estimate — treating objects as molecules in shells of the atmosphere — gives about 1 400 pairs within 1 km per day for 35 000 objects. Pairs inside one constellation are left out, because their orbits are designed to keep apart. Every count grows as $N^2$. Doubling the catalogue, for example by tracking smaller debris, quadruples the work.




Synthetic catalogue shaped like the 2026 public catalogue (78 % near-circular LEO with dense shells at 480–560 km, eccentric debris, MEO, GTO/HEO and GEO). Close approaches: kinetic-gas estimate with objects spread over their radial range in 5 km shells and a mean relative speed of 10 km/s.

**FIGURE 50.6** Conjunction-screening workload. (a) Pairs per screening run and close approaches per day against catalogue size, for a synthetic catalogue shaped like the 2026 public one. (b) The screening funnel for 50 000 objects.

**EXAMPLE: DAILY SCREENING WORKLOAD FOR A 50 000-OBJECT CATALOGUE**

The numbers come from the synthetic catalogue of Figure 50.6.

1. **Pairs.** $N(N-1)/2 = 50\,000 \times 49\,999/2 = 1.25 \times 10^9$ pairs.
2. **Filter.** The apogee–perigee filter (±10 km) keeps 21.3 %, or $2.67 \times 10^8$ pairs.
3. **Brute-force check.** Testing each surviving pair every minute for seven days takes $2.67 \times 10^8 \times 10\,080 = 2.7 \times 10^{12}$ distance checks. At $10^9$ checks per second, that is 45 minutes on one processor core, or 27 seconds on 100 cores. Computing is not the bottleneck.
4. **Close approaches.** The kinetic-gas estimate gives about 72 700 pairs within 5 km and 2 900 within 1 km each day — about a million a year within 1 km.
5. **People.** Suppose 1 % of the 1 km events needed an analyst for two hours. That would be 29 reviews and 58 hours a day — about 12 full-time analysts at 1 800 working hours a year.

This is why triage — choosing which events deserve people — matters more than raw computing speed.

### 50.2.2 Triage: Which Events Matter?

For each close approach, a **conjunction data message** (CDM) gives the predicted miss and the uncertainty of both objects (Chapter 21). The **probability of collision** $P_c$ is the chance that the true miss is smaller than the combined hard-body radius $R$. For a round uncertainty of standard deviation $\sigma$ in the encounter plane and $R \ll \sigma$,

$$P_c \approx \frac{R^2}{2\sigma^2} \exp\!\left(-\frac{d^2}{2\sigma^2}\right),$$

where $d$ is the predicted miss distance. For a fixed miss distance, $P_c$ peaks at $R^2/(e\,d^2)$ when $\sigma = d/\sqrt{2}$. With larger uncertainty, $P_c$ falls again. This is **dilution**: a poorly tracked object looks safe because nobody knows where it is.

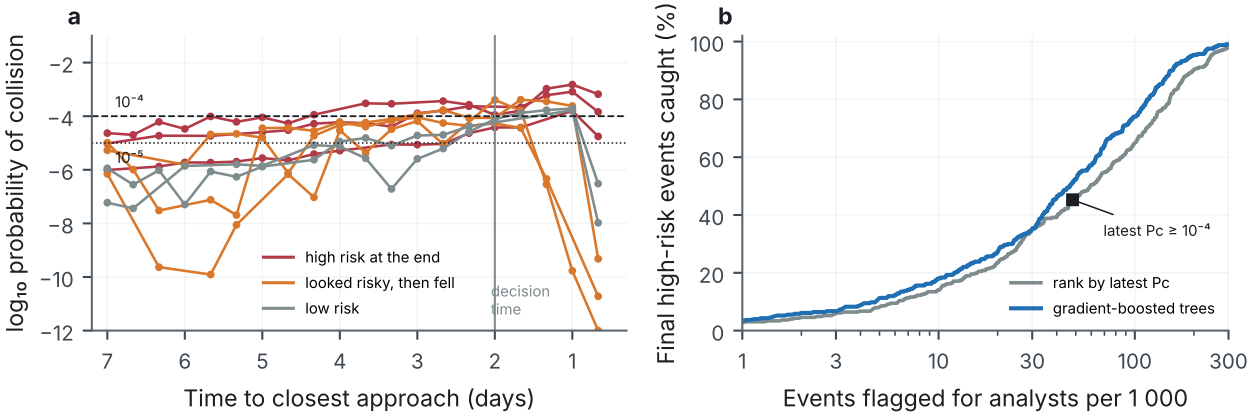
Each event produces a series of CDMs, typically one to three a day for about a week. The uncertainty shrinks as tracking accumulates, so $P_c$ jumps around (Figure 50.7). Operators usually decide about two days before the closest approach, the last moment for a cheap manoeuvre. The question for learning is whether the history of an event's CDMs predicts its final risk better than the latest value alone.

**THE KELVINS COLLISION-AVOIDANCE CHALLENGE (2019)**

In 2019 ESA ran a two-month machine-learning competition on its Kelvins platform. ESA's Space Debris Office released a database of 15 321 conjunction events and 199 082 CDMs, collected in 2015–2019 for its satellites. The data became public in October 2019. Each CDM had 103 attributes. Only 30 events had a final risk of $10^{-4}$ or more. Teams had to predict the final risk from CDMs available two days before closest approach.

Ninety-six teams made 862 submissions. The main result surprised many: the **naive forecast** — the latest known risk — was very hard to beat. Later tests with gradient-boosted trees beat it by about 1.3 % in the F2 score, a small but statistically clear gain. The organisers concluded that naive forecasts are an "unavoidable benchmark" for any future method (Uriot et al., 2022).

Figure 50.7 repeats the experiment on 40 000 synthetic events. Each has a true miss within 3 km, a hard-body radius of 15 m, shrinking and partly wrong covariances, and missing CDMs. An event counts as high risk if its final $P_c$ is at least $10^{-5}$. Ranking events by their latest $P_c$ at two days and flagging the top 40 per 1 000 caught 39 % of the high-risk ones. Gradient-boosted trees using the whole CDM history caught 46 %. The common rule "latest $P_c \geq 10^{-4}$" flagged 48 events per 1 000 and caught 45 %; the trees caught 51 % with the same number of alerts. The gain is real but modest, as in the Kelvins challenge. No method can recover information that the tracking data do not contain.




40000 synthetic events: true miss uniform within 3 km, hard-body radius 15 m, reported σ shrinking from km to tens of metres, covariance realism error 30 %, correlated estimation errors, 15 % of CDMs missing. High risk: final Pc ≥ $10^{-5}$. Trees trained on half, tested on the other half using CDMs up to 2 days before closest approach.

**FIGURE 50.7** Conjunction triage on synthetic CDM sequences. (a) Probability of collision against time to closest approach for example events. (b) Share of final high-risk events caught against the number of events flagged per 1 000, for ranking by the latest Pc and for gradient-boosted trees.

The last resort, when warning comes too late to manoeuvre, is to protect the people. On 24 March 2012 the six crew members of the International Space Station took shelter in their two Soyuz spacecraft as a debris fragment passed close by (Figure 50.8).

**FIGURE 50.8** Expedition 30 crew members in the ISS Zvezda module on 24 March 2012, preparing to shelter in their Soyuz spacecraft because a piece of debris could pass close to the station. Image: NASA/JSC (Andre Kuipers).

**EXAMPLE: PROBABILITY-OF-COLLISION THRESHOLDS AND MANOEUVRE NUMBERS**

A fleet of 1 000 satellites each has 300 close approaches a year with a miss distance inside $D$ = 5 km. Assume the true miss is spread evenly over that disc, the hard-body radius is $R$ = 5 m and the final uncertainty is $\sigma$ = 100 m.

1. **Risk without avoidance.** Each event has a collision chance of $R^2/D^2 = 10^{-6}$. The fleet expects $1\,000 \times 300 \times 10^{-6} = 0.3$ collisions a year.
2. **Trigger distance.** Setting $P_c$ equal to the threshold $P_{\text{th}}$ gives $d_{\text{th}} = \sigma\sqrt{2\ln(R^2/(2\sigma^2 P_{\text{th}}))}$. For $10^{-4}$ this is 225 m.
3. **Manoeuvres.** The share of events inside $d_{\text{th}}$ is $(d_{\text{th}}/D)^2 = 2.0 \times 10^{-3}$. That is 0.61 manoeuvres per satellite per year, or 606 for the fleet.
4. **Residual risk.** The risk left in events below the threshold is $2\sigma^2 P_{\text{th}}/R^2$ = 8 % of the total. A threshold of $10^{-4}$ thus removes 92 % of the risk; $10^{-5}$ removes 99.2 % for 1.16 manoeuvres per satellite per year.
5. **Poor tracking.** With $\sigma$ = 500 m the largest possible $P_c$ is $5 \times 10^{-5}$. A $10^{-4}$ threshold then never triggers, although the true risk is unchanged. A threshold of $3 \times 10^{-7}$ removes 99.4 % of the risk but needs 31 manoeuvres per satellite per year.

Better tracking reduces manoeuvres more than any threshold rule can (Table 50.2).

**TABLE 50.2** Manoeuvre numbers and risk removed for different collision-probability thresholds, from the worked example (300 events a year within 5 km, hard-body radius 5 m).

| Threshold $P_{\text{th}}$ | Trigger miss, σ = 100 m | Manoeuvres per satellite-year | Risk removed | Manoeuvres per satellite-year, σ = 500 m | Risk removed, σ = 500 m |
|---|---|---|---|---|---|
| $10^{-3}$ | 67 m | 0.05 | 20 % | never triggers | 0 % |
| $10^{-4}$ | 225 m | 0.61 | 92 % | never triggers | 0 % |
| $10^{-5}$ | 311 m | 1.16 | 99.2 % | 9.7 | 80 % |
| $3 \times 10^{-7}$ | 408 m | 2.0 | 99.98 % | 30.7 | 99.4 % |

## 50.2.3 Automated Decisions and Coordination

At the largest scale, decisions are already automatic. Starlink satellites receive warnings and plan and execute their own avoidance manoeuvres. They made more than 200 000 such manoeuvres in the six months to May 2026, at a threshold of $3 \times 10^{-7}$ (Chapter 48). This is rule-based autonomy rather than learning, and it works because the operator controls both the rules and the satellites. The weak point is data. In the few cases where a manoeuvre did not reduce the risk as planned, the cause was mostly a poor or late prediction of the other object.

Coordination between operators is the next step. ESA's **CREAM** project (Collision Risk Estimation and Automated Mitigation), begun in 2020, aims to cut operator workload, false alerts and response time. In August 2025 its ground prototype, built by GMV and Guardtime, could issue alerts, propose manoeuvres and support coordination. ESA was preparing a pilot phase and in-orbit demonstrations, first as a software payload on other satellites. Table 50.3 lists the main systems and their status.

In the USA, civil conjunction services are moving from the military to the Office of Space Commerce's **TraCSS**. In July 2026 it was still a pilot, with 62 users covering more than 11 000 satellites. Congress funded the office at $52.5 million for 2026, but the 2027 budget request did not

mention TraCSS. Europe's EU SST protected more than 700 satellites in 2026. It reported 15 639 close approaches in 2023, of which about 1 000 were high risk.

**TABLE 50.3** Selected conjunction, coordination and behaviour-analysis systems. Sources: operators' and agencies' public statements.

| System | Operator | What it does | Status (October 2026) |
|---|---|---|---|
| ATLAS and Space-Track | US Space Force | Catalogue processing; public data and CDMs | Operational (ATLAS from Sept. 2025) |
| TraCSS | US Office of Space Commerce | Civil conjunction service | Pilot: 62 users, >11 000 satellites (July 2026); funding contested |
| EU SST | EU Space Surveillance and Tracking | Collision avoidance, re-entry, fragmentation services | Operational; >700 satellites protected |
| CREAM | ESA with GMV and Guardtime | Automated risk assessment, manoeuvre design, coordination | Ground prototype; pilots and in-orbit demonstration in preparation |
| Starlink onboard avoidance | SpaceX | Autonomous screening response and manoeuvres | Operational |
| LeoLabs services | LeoLabs | Radar catalogue; ML manoeuvre, break-up and behaviour detection | Commercial (vendor-reported) |
| Agatha | Slingshot Aerospace for DARPA | Flags outlier satellites in constellations | Research and development (2023–24) |

## 50.3 Watching Behaviour: Manoeuvres, Characterisation and Re-entry

The key idea of this section is that a catalogue must know not only where objects are but how they behave. Has a satellite moved? Is it tumbling? Will it re-enter soon? Learning is most useful here, because behaviour leaves subtle patterns in long data histories.

### 50.3.1 Manoeuvre Detection

A manoeuvre is a sudden change in velocity. A burn of 1 cm/s along the track of a LEO satellite raises its semi-major axis by about 18 m. That is less than the noise in a single orbit estimate, and drag changes keep blurring the picture. Yet missed manoeuvres corrupt predictions, and unexplained ones may signal intent.

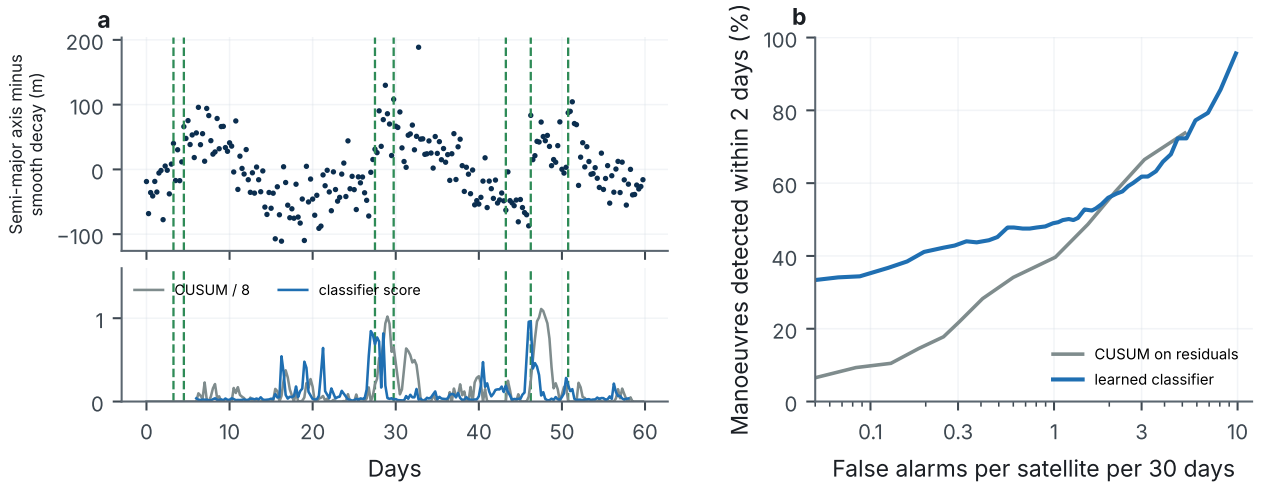
The classical tool is the **cumulative sum** (CUSUM) test. A short-term model predicts the next estimate. The standardised residual $r_k$ feeds two running sums,

$$S_k^{\pm} = \max\bigl(0,\; S_{k-1}^{\pm} \pm r_k - \kappa\bigr),$$

which raise an alarm when either exceeds a threshold $h$. The allowance $\kappa$ absorbs normal noise. A step change pushes one sum steadily upward, so CUSUM detects small persistent shifts that a single-point test would miss.

A learned classifier instead looks at a whole window of data around each moment. It learns what a burn looks like compared with a storm or an outlier. Figure 50.9 compares the two on 240 synthetic satellites. Each has 60 days of orbit estimates every six hours, drag that rises in storms

and burns of 15–150 m. At one false alarm per satellite per month, a tuned CUSUM found 39 % of burns within two days and gradient-boosted trees found 49 %. At 0.3 false alarms the gap widened to 22 % against 43 %. Both found more than 80 % of the largest burns (100–150 m). Burns below 50 m — two noise widths — stayed hard: CUSUM found 19 % of them and the trees 33 %.




240 synthetic LEO satellites, 60 days of semi-major-axis estimates every 6 h (noise 25 m, 2 % outliers ×5), drag decay 15–60 m/day rising in storms, burns of 15–150 m (about 1–8 cm/s; green dashed lines). Classifier: gradient-boosted trees on 8-day windows (6 days before, 2 after), trained on 120 satellites, tested on the other 120.

**FIGURE 50.9** Manoeuvre detection in noisy orbit estimates. (a) One synthetic satellite: semi-major axis minus its smooth decay, with burns (green dashed lines), the CUSUM statistic and the classifier score. (b) Share of burns detected within two days against false alarms per satellite per month.

Commercial providers now sell such detection. LeoLabs says it uses machine learning to detect manoeuvres in real time, score unusual “patterns of life” across its catalogue and spot new objects from deployments or break-ups. It also reports that learned radar scheduling lets it track objects 40 % more often. These are vendor claims without independent evaluation. Slingshot Aerospace’s **Agatha**, built for the US Defense Advanced Research Projects Agency (DARPA) from 2023, takes a different route. It was trained on simulated constellations and uses **inverse reinforcement learning** — inferring goals from observed behaviour — to flag satellites that act unlike their neighbours.

Behaviour analysis matters most in geostationary orbit, where inspector satellites manoeuvre near others. In mid-2025 China’s Shijian-21 and Shijian-25 appear to have docked in GEO, probably to test refuelling, and separated in late 2025. China did not confirm the operation. Commercial trackers such as COMSPOC and S2a followed it from the ground, and the US Space Force reportedly watched with its GSSAP inspector satellites.

**PRACTICE: BUILDING A MANOEUVRE-DETECTION PIPELINE**

1. **Define the need.** Decide which satellites to watch, the smallest burn that matters (for example 2 cm/s) and how quickly it must be found. Agree how many false alarms per day the analysts can handle.
2. **Gather data.** Collect orbit histories — two-line elements, ephemerides or your own orbit fits — with their uncertainties. Add space-weather indices, because storms mimic burns.
3. **Get labels.** Use operators' published manoeuvre histories, such as constellation ephemerides, or ask cooperating operators. Mark storms, deployments and orbit-raising phases separately.
4. **Build a baseline.** Fit a smooth decay model and run a CUSUM test on the residuals. Tune its window and allowance on training satellites only.
5. **Add a learned model.** Train a gradient-boosted classifier on windows of residuals and space-weather features. Split by satellite and by time, never at random, to avoid leakage (Chapter 43).
6. **Score like an operator.** Report the share of burns found within the required time at your false-alarm budget. Report results for small burns and storm periods separately.
7. **Estimate each burn.** For every alarm, fit the size, direction and time of the burn and its uncertainty. A flag without an estimate is of little use.
8. **Run in shadow mode.** Run beside existing tools for months. Review every alarm and missed burn with analysts.
9. **Monitor and retrain.** Track precision monthly. Retrain after new satellite types, new sensors or a change in solar activity.
10. **Share what you find.** Publish detected manoeuvres in a standard format, with their uncertainty, so that screening services and other operators can use them at once.

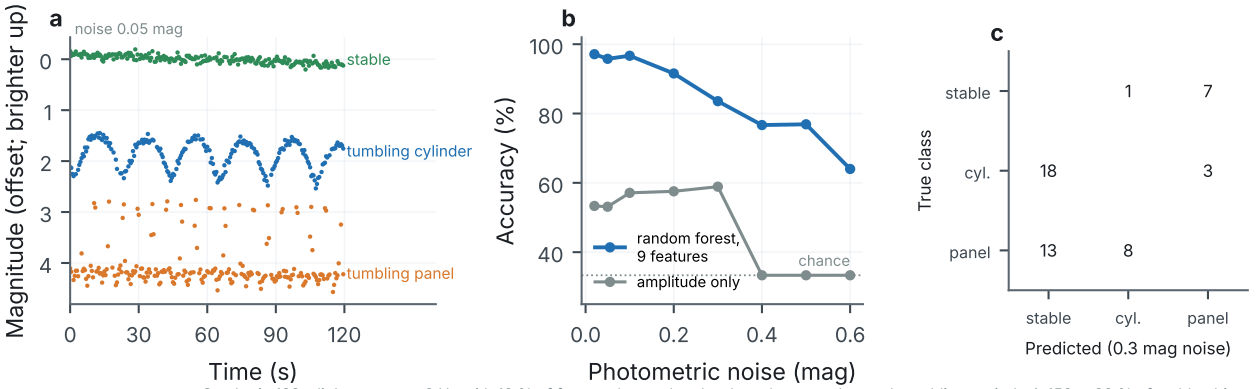
The same steps work for break-up detection and for change detection in imagery.

### 50.3.2 Characterising Objects from Light Curves

A **light curve** is an object's brightness over time. A stable, pointed satellite shines steadily. A tumbling rocket body brightens twice per turn as its long side faces the observer. A tumbling panel flashes when its flat surface briefly acts as a mirror. Period, amplitude and shape thus reveal what an object is and how it moves.

A good starting point is a set of physical **features**. The **Lomb–Scargle periodogram** finds the strongest period in unevenly sampled data. Skewness and the share of bright outliers measure flashes. The autocorrelation at the best period tests whether the pattern repeats. A classifier then learns from labelled examples.

Figure 50.10 tests this on synthetic two-minute light curves of three classes. A random forest on nine features was about 96–97 % accurate up to 0.1 magnitude of noise and 84 % at 0.3 magnitude. The most useful features were periodogram power, skewness and autocorrelation. A rule using amplitude alone managed only 53–59 %. Real studies report similar trends. Classifiers trained on light curves from Russia's Mini-MegaTORTORA wide-field camera reach about 75 % for three broad classes and 85–92 % for some finer tasks. Results differ with the class definitions, so the 2024 RoBo6 data set was released as a common benchmark.




Synthetic 120 s light curves at 2 Hz with 10 % of frames dropped and a slow phase-angle trend; tumbling periods 4–150 s; 30 % of stable objects show one glint. Features: spread, amplitude, Lomb–Scargle peak power and period, skewness, kurtosis, flash fraction, autocorrelation at the period. Trained on 2 400 curves (noise 0.02–0.6 mag); 450 test curves per noise level.

**FIGURE 50.10** Light-curve classification. (a) Synthetic light curves of a stable satellite, a tumbling cylinder and a tumbling panel. (b) Accuracy against photometric noise for a random forest on nine features and for a rule using amplitude alone. (c) Confusion matrix of the forest at 0.3 magnitude noise.

Radar adds a second view. **Radar cross-section** — how strongly an object reflects radio waves — varies with size, shape and attitude. Imaging radars such as Germany's TIRA can make pictures of satellites, which networks can help interpret. Characterisation feeds association, because two objects with different light curves are unlikely to be the same. It also feeds risk, because a tumbling derelict cannot manoeuvre.

### 50.3.3 Re-entry Prediction

About 2 000 catalogued objects re-enter each year (Chapter 21). Predicting when an uncontrolled object will re-enter is hard because the final days are dominated by drag at low altitude, where density is least certain. A common rule of thumb is an uncertainty of about 10–20 % of the remaining lifetime. A prediction made a day ahead is therefore uncertain by a few hours. That is more than a full orbit, or tens of thousands of kilometres along the track.

Learning has been tried here too. Salmaso, Trisolini and Colombo (2023) trained a sequence-to-sequence network on the altitude histories of more than 400 re-entered objects, adding the drag term, solar activity and area-to-mass ratio as inputs. It worked best for objects similar to its training set, a familiar limit of learned models. Controlled re-entries avoid the problem, as for ESA's ATV-3 in 2012 (Figure 50.11).

**FIGURE 50.11** The destructive re-entry of ESA's Automated Transfer Vehicle-3 (Edoardo Amaldi) over the South Pacific on 3 October 2012, photographed from the International Space Station. The re-entry was controlled, so the time and place were known in advance. Image: NASA/JSC.

## 50.4 AI in Defence and Intelligence Space Systems

The key idea of this section is that defence uses of AI in space are mostly about **attention**. Sensors now collect far more than people can look at. Machines filter, flag and cue; people check, judge and decide. This section uses open sources only. It describes what governments and companies have said publicly, without operational detail.

### 50.4.1 Imagery Exploitation at National Scale

Satellite imagery has shaped wars since the first reconnaissance satellites (Chapter 19). In 1991 Space Shuttle crews photographed the Kuwaiti oil fires set during the Gulf War (Figure 50.12). Today hundreds of commercial and government imaging satellites revisit most places several times a day. No analyst workforce could inspect all those images, so automated **object detection** and **change detection** (Chapter 46) now do the first pass.

**FIGURE 50.12** Smoke from burning oil wells north of Kuwait City, photographed from the Space Shuttle in April 1991 after the Gulf War. Space imagery has been part of conflict monitoring ever since. Image: NASA/JSC.

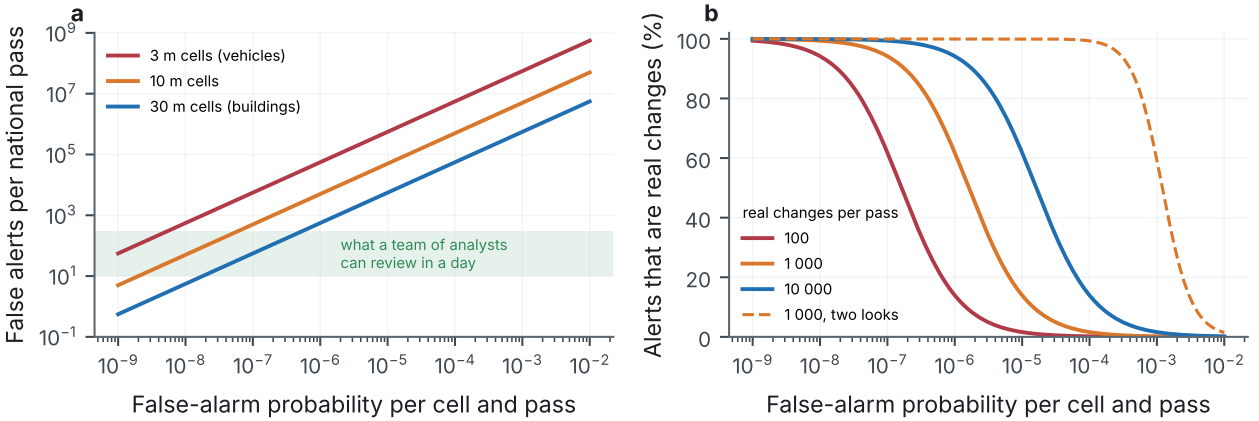
**FROM PROJECT MAVEN TO NGA MAVEN**

- **2017.** The US Department of Defense set up an Algorithmic Warfare team, known as Project Maven, to apply computer vision to drone video and other imagery. Its first algorithms were in use by December 2017.
- **2022.** Parts of Maven moved to the National Geospatial-Intelligence Agency (NGA), which handles imagery and maps.
- **November 2023.** NGA made NGA Maven a formal programme of record.
- **May 2025.** NGA's director reported more than 20 000 active users across more than 35 military tools, available to all services and combatant commands. Targeting timelines in one exercise fell "from hours to minutes". The ceiling of the related Maven Smart System contract rose to about $1.3 billion through 2029.
- **June 2025.** NGA began labelling intelligence products that are "100 percent machine-generated", with no human hand in the template or its release — the first of the 18 US intelligence agencies to do so routinely.

Sources: Breaking Defense and DefenseScoop reporting of NGA statements, 2025.

NGA also buys analytics from industry. Its Luno A contract (September 2024, up to $290 million, ten companies) buys computer vision and analytic services based on commercial imagery. Luno B (January 2025, up to $200 million, 13 companies) has funded orders such as change and movement detection and maritime analytics.

At national scale the main enemy is the false alarm. Clouds, shadows, seasons and small misregistrations all look like change. Figure 50.13 shows the arithmetic. A country of 500 000 km² divided into 30 m cells has $5.6 \times 10^8$ cells per look. Even a tiny per-cell false-alarm rate produces more alerts than any team can read.




Area 500 000 km² imaged once. (b) 30 m cells ($5.6 \times 10^8$ cells) and 90 % detection probability per look. Assumes false alarms independent between cells and looks; real clutter (clouds, shadows, seasons, misregistration) is correlated, so measured rates are needed.

**FIGURE 50.13** False alarms of automated change detection at national scale. (a) False alerts per pass over 500 000 km² against the per-cell false-alarm probability, for three cell sizes, with the range a team can review in a day. (b) Share of alerts that are real changes for 30 m cells, for different numbers of real changes per pass, and with confirmation by a second independent look.

**EXAMPLE: FALSE ALARMS IN AUTOMATED CHANGE DETECTION OVER A COUNTRY**

A detector screens 500 000 km$^2$ in 30 m cells after each pass. It finds 90 % of real changes and has a false-alarm probability of $10^{-4}$ per cell. There are 1 000 real changes per pass.

1. **Cells.** $500\,000\ \text{km}^2/(30\ \text{m})^2 = 5.56 \times 10^8$ cells.
2. **False alerts.** $5.56 \times 10^8 \times 10^{-4} = 55\,556$ per pass.
3. **Real alerts.** $0.9 \times 1\,000 = 900$.
4. **Precision.** Only $900/(900 + 55\,556) = 1.6\ \%$ of alerts are real.
5. **Target.** For half of the alerts to be real, the false-alarm rate must fall to $900/5.56 \times 10^8 = 1.6 \times 10^{-6}$ per cell.
6. **Second look.** Confirming each alert with an independent second image squares the false-alarm rate to $10^{-8}$. False alerts drop to about 6, while detection falls to 81 %. Precision becomes 99.3 %.

Step 6 assumes independent errors. Real clutter repeats from image to image, so the gain is smaller in practice. This is why measured false-alarm rates, not textbook ones, must drive the design.

## 50.4.2 Activity-Based Intelligence and Tip-and-Cue

**Activity-based intelligence** looks for patterns of activity rather than single objects: ships that meet at sea, vehicles that gather before an exercise, a radar that switches on at unusual times. It needs data from many sources over long periods. Machine learning helps to build a picture of normal behaviour and flag departures from it.

**Tip-and-cue** turns such flags into new collection (Figure 50.14). A wide-area sensor, such as a radar satellite or a radio-frequency receiver, notices something. Software then tasks a high-resolution satellite or another sensor to look again. The US National Reconnaissance Office (NRO) has described its **Sentient** research programme as an automated tipping-and-cueing system. In 2025 it told Congress that its new constellations and ground systems will "fuse data across collection platforms" to automate tipping and cueing. By mid-2026 the NRO had made 14 launches of its proliferated LEO constellation and said it had hundreds of satellites in orbit.

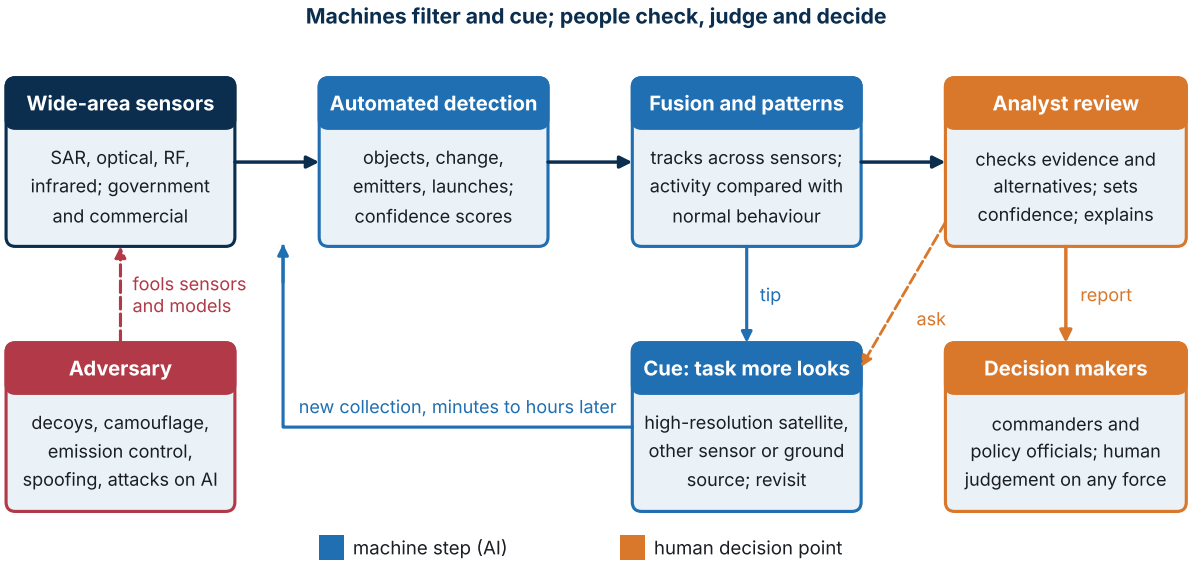



**FIGURE 50.14** Tip-and-cue across constellations. Machine steps (blue) detect, fuse and cue new collection; human decision points (orange) review evidence and make decisions. An adversary (red) tries to deceive both sensors and models.

Radar satellites are central to maritime monitoring because they see through cloud and darkness. In an early example from 1994, the Space Shuttle's imaging radar recorded a ship and its 28 km wake in the Atlantic (Figure 50.15).

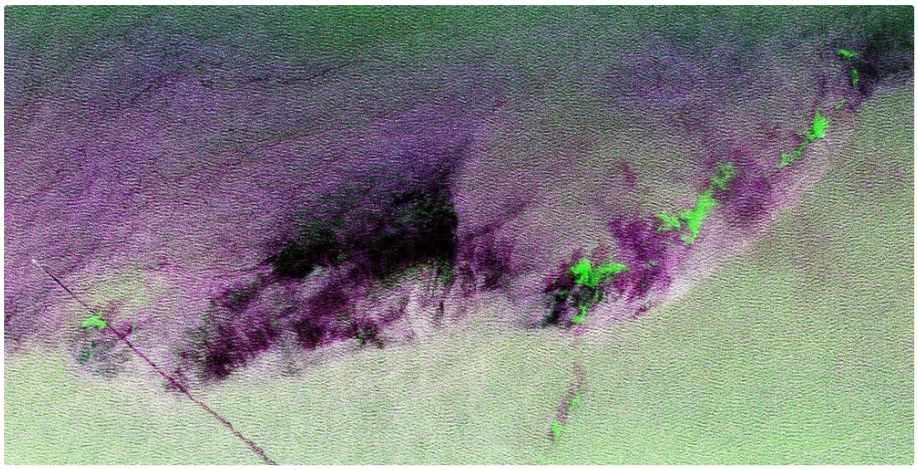


**FIGURE 50.15** A Spaceborne Imaging Radar-C/X-SAR image of the north-east Atlantic taken from Space Shuttle Endeavour on 11 April 1994. The bright spot at left is a ship and the line at lower left its wake, about 28 km long. Investigators thought that discharged oil made the wake last longer. The area shown is about 35 km × 65 km. Image: NASA/JPL (NASA/JPL).

### 50.4.3 Signals and Emitters

Radio receivers in orbit locate and classify emitters, such as radars, satellite phones and ship transponders (Chapter 19). Learning helps in two ways. **Signal classification** recognises the type of signal from its modulation (Chapter 49). **Specific emitter identification** goes further: it tells apart individual transmitters of the same type from tiny hardware imperfections, such as oscillator drift and amplifier distortion — a radio fingerprint.

Open research shows what is possible. In 2023 researchers at Oxford fingerprinted Iridium satellite transmitters from about 1.7 million recorded messages, using an autoencoder and a twin ("Siamese") network. Against messages replayed with a software-defined radio — a spoofing attack — the system had an equal error rate of 12 %. Commercial operators such as HawkEye 360 use clusters of satellites to geolocate signals and flag ships that switch off their identification beacons.

### 50.4.4 Missile Warning and Tracking

Missile-warning satellites stare at the Earth in the infrared and must find a rocket plume among clouds, sun glints and fires (Chapter 20). Classical processing subtracts the background, applies thresholds and confirms detections by tracking them over several frames. Learning is being studied for clutter suppression, event classification and **track fusion** — combining detections from several satellites and radars into one three-dimensional track. In September 2025 the US Space Force accepted the second delivery of its new FORGE missile-warning ground system for operations. Public material says little about the use of learning in operational chains. Errors here could have grave consequences, so changes demand long testing.

### 50.4.5 Battle Management and Golden Dome

Missile defence must turn tracks into decisions in minutes. **Battle management, command and control** (BMC2) software links sensors to interceptors and proposes which interceptor should engage which threat. The US **Golden Dome** programme, launched by executive order in January 2025, aims at a layered homeland defence that includes space sensors and possibly space-based interceptors.

Public facts are limited and changing. In April 2026 the Space Force gave 12 companies agreements worth up to $3.2 billion for space-based interceptor prototypes. Golden Dome's director, General Michael Guetlein, calls the command-and-control layer the programme's "glue layer". An industry consortium of nine firms ran a live demonstration of part of it in early 2026, and initial operational capability is planned for 2028. He put the cost at $185 billion through 2035. A Congressional Budget Office estimate in May 2026, for an illustrative architecture, put it at about $1.2 trillion over 20 years. In September 2026 Guetlein said that the military must make decisions "at machine speed". Senior leaders described a role for AI in helping human operators handle the flood of sensor data. How far humans stay in each decision has not been described in public detail.

### 50.4.6 Autonomy for Rendezvous and Proximity Operations

Inspector satellites, servicers and possible weapons all fly close to other spacecraft. Their guidance and navigation methods are covered in Chapter 48. The defence angle is speed and secrecy. In June 2026 the Space Force's **Victus Haze** exercise launched a Rocket Lab target spacecraft 16 hours and 42 minutes after notice. True Anomaly's Jackal, already in orbit, then approached it and imaged it from several sides. Its mission software planned the sortie and commanded the manoeuvres, and it finished in 61 hours of a 72-hour window. Such missions rely partly on onboard autonomy, because at close range there is little time to wait for commands from the ground. They also stress SSA, because observers must decide quickly whether an approach is inspection, servicing or a threat (Table 50.4).

**TABLE 50.4** Defence and intelligence uses of AI in space systems, from open sources.

| Application | Example (public) | What the machine does | Status (October 2026) |
|---|---|---|---|
| Imagery exploitation | NGA Maven; NGA Luno contracts | Detects objects and change; drafts reports | Operational at scale |
| Tip-and-cue | NRO Sentient; proliferated constellation | Flags events; tasks other sensors | Described as operational in part; details classified |
| Emitter identification | SatIQ research; commercial RF constellations | Recognises signal types and individual transmitters | Research; commercial geolocation operational |
| Missile-warning processing | FORGE; SDA Tracking Layer | Rejects clutter; builds and fuses tracks | Operational processing; learning mostly research |
| Battle management | Golden Dome C2 consortium | Pairs threats with interceptors; presents options | In development; demonstrations 2026 |
| Proximity operations | Victus Haze; GSSAP; Shijian-21/25 | Guides approach; classifies others' approaches | Demonstrations and operations |

### 50.4.7 Deception and Counter-AI

Every detector invites an attempt to fool it. Classic **deception** uses decoys, camouflage and emission control. In 2023–24 analysts of commercial imagery found aircraft outlines painted on Russian airfields, including Tu-95 bomber shapes at Engels. The UK Ministry of Defence noted in 2024 that helicopters were still landing on the painted fighter outlines, which undermined the deception. Commercial imagery separated the paintings from real aircraft easily.

AI adds new targets. An **adversarial patch** is a printed pattern designed to make a neural network miss an object. A 2020 study showed that a patch covering about 10 % of an aircraft hid it from a common detector in aerial images, although only digitally. Training data can be poisoned, and models can be probed until they reveal their blind spots. Defences include training on adversarial examples, combining different sensors — a painted shape has no radar or heat signature — and keeping people in the review loop. Chapter 53 covers the security of AI in more depth.

### 50.4.8 Human–Machine Teaming

Automation changes how people think. **Automation bias** is the tendency to accept a machine's suggestion without checking. Its opposite, **disuse**, follows when a system cries wolf too often. Good teaming gives each side what it does best. Machines sift volume and keep track of many objects. People bring context, scepticism and responsibility.

Three practices help. Systems should show their evidence and confidence, not just a verdict. Operators should train with systems that sometimes fail, so that they learn when to doubt them. And the design should state clearly who decides what. NGA's label for machine-generated products is one example of making the machine's role visible. Flight dynamics officers at NASA, who own trajectory decisions for crewed missions, offer another model of human authority supported by software (Figure 50.16).

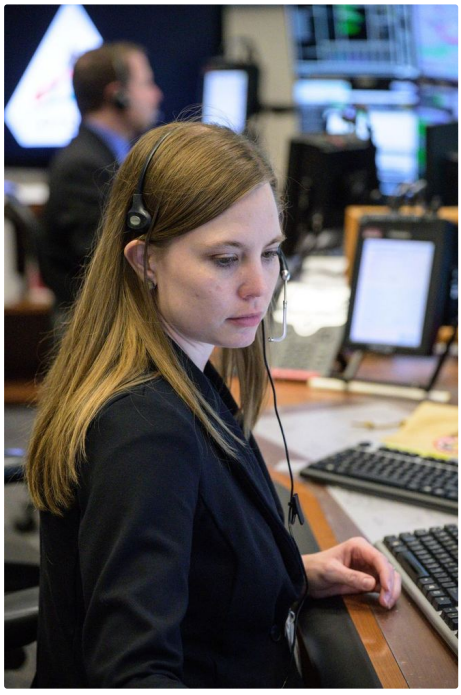


**FIGURE 50.16** Flight controller Julie Reed at the flight dynamics officer console in Mission Control, Houston, on 1 December 2022, as Orion left its distant retrograde orbit around the Moon during Artemis I. Software computes the trajectory options; a named person owns the decision. Image: NASA/JSC (ROBERT MARKOWITZ NASA-JSC).

## 50.5 Law, Ethics and Policy

The key idea of this section is that existing law applies to AI, but states disagree on whether new rules are needed. Space adds special worries about stability, because satellites are fragile and hard to defend.

### 50.5.1 Rules for Autonomy in Weapons

The main US policy is **Department of Defense Directive 3000.09**, "Autonomy in Weapon Systems", updated on 25 January 2023. It requires that systems let commanders and operators exercise "appropriate levels of human judgment over the use of force". Autonomous weapon systems need a senior-level review before formal development and again before fielding. The directive does not ban such weapons, and it does not require a human to approve every individual engagement. In June 2026 a presidential memorandum, NSPM-11, ordered an update of the directive within 90 days and a review every year. No revised text had been published by late October 2026.

### 50.5.2 International Norms

The USA launched the **Political Declaration on Responsible Military Use of Artificial Intelligence and Autonomy** in February 2023 and revised it in November 2023. It is a non-binding set of commitments: legal reviews, human accountability, testing, and safeguards such as the ability to disengage systems that behave unexpectedly. Almost 60 states had endorsed it by late 2024. Other forums run in parallel (Table 50.5).

**TABLE 50.5** Policy and legal instruments on military AI and autonomy.

| Instrument | Year | Type | Main content | Status (October 2026) |
|---|---|---|---|---|
| DoD Directive 3000.09 | 2012; updated 2023 | US policy | Human judgement over force; senior reviews | In force; update ordered June 2026 |
| Political Declaration | 2023 | Non-binding, ~60 states | Responsible military AI practices | Endorsements open |
| NATO AI strategy | 2021; revised 2024 | Alliance policy | Six principles of responsible use; testing | In force |
| REAIM summits | 2023, 2024, 2026 | Multi-stakeholder | Blueprint (Seoul); "Pathways to Action" (A Coruña) | About 35 states endorsed 2026 text (reported) |
| UN General Assembly resolutions | 2023, 2024, 2025 | Non-binding | Concern about lethal autonomous weapons | Resolution 80/57: 164 for, 6 against |
| CCW expert group (GGE) | 2016–2026 | Treaty forum, consensus | Elements of a possible instrument | Consensus report Sept. 2026; Review Conference Nov. 2026 |

Under the Convention on Certain Conventional Weapons (CCW), with 128 states parties, a group of governmental experts set up in 2016 has discussed **lethal autonomous weapon systems**. Its mandate ended in 2026. At its final session (31 August–4 September 2026) it agreed by consensus on elements that could form the basis of an instrument. The Seventh Review Conference, on 16–20 November 2026, will decide whether to open negotiations. The USA and Russia have opposed

new legally binding rules. Many other states and the International Committee of the Red Cross want a treaty. NATO's revised 2024 strategy keeps its six principles: lawfulness, responsibility and accountability, explainability and traceability, reliability, governability and bias mitigation.

### 50.5.3 Stability and Escalation in Space

AI could make space more stable or less. Better SSA helps: knowing quickly that a close approach is an accident, not an attack, reduces the risk of overreaction. Shared data and automated coordination make accidents less likely. But speed cuts both ways. If machines classify a manoeuvre as hostile within seconds, decision makers may feel pressure to respond before checking. Errors in one system can trigger responses in another, as in financial "flash crashes". And satellites that support nuclear command and early warning (Chapter 20) are especially sensitive. Misreading an attack on them could escalate a conflict quickly.

Common proposals include keeping human decision points for any action against a satellite, exchanging data on manoeuvres, testing systems against realistic deception, and agreeing norms against destructive anti-satellite tests (Chapter 19).

### 50.5.4 Export Controls and Dual Use

Almost everything in this chapter is **dual-use**: the same tracker serves a commercial operator and an air force. Space hardware falls under export controls such as the US International Traffic in Arms Regulations and Export Administration Regulations, and the multilateral Wassenaar Arrangement. Advanced AI chips have been controlled since 2022. A US rule of January 2025 that would also have limited exports of AI chips and model weights to most countries was rescinded in May 2025, and no replacement had been issued by mid-2026. Engineers should expect rules on chips, models and SSA data to keep changing and should check them early in any international project.

## 50.6 Lessons from the Experiments

Table 50.6 collects the results of this chapter's experiments. All used synthetic data with fixed random seeds, so they illustrate methods rather than measure real systems.

**TABLE 50.6** Summary of this chapter's synthetic experiments.

| Task and condition | Baseline | Learned or optimised method | Result |
|---|---|---|---|
| Streak detection | Point-source filter | Line matched filter | 1.8× fainter |
| Association, spacing 2σ | Greedy nearest neighbour | Global assignment | 76 % vs 47 % |
| Drag after storms, one day | Fitted drag scale | Plus neural network | 0.24 vs 0.75 km |
| Triage, 40 alerts per 1 000 | Latest Pc | Boosted trees | 46 % vs 39 % |
| Manoeuvres, 1 false alarm a month | CUSUM | Boosted trees | 49 % vs 39 % |
| Light curves, 0.3 mag noise | Amplitude rule | Random forest | 84 % vs 59 % |

Four lessons stand out. Physics and geometry carry most of the load. Learning gives real but moderate gains, mostly where data are noisy or patterns are subtle. Gains must be measured in the conditions that matter, such as storms or low false-alarm rates. And no model can create

information that the sensors did not collect.

### KEY TAKEAWAYS

- About 35 000 objects are catalogued in orbit (October 2026). The number of close approaches grows with the square of that number: 50 000 objects make about $1.25 \times 10^9$ pairs and thousands of approaches within 1 km a day.
- Most of the SSA pipeline is physics and geometry. Learning helps with faint detections, association, prediction errors, density during storms, triage, manoeuvre detection and characterisation — mostly in research or commercial services so far.
- Along-track errors grow with the square of time, so density errors dominate LEO predictions. Hybrid models that learn a correction to physics can help after storms but must be tested regime by regime.
- In conjunction triage the latest probability of collision is a strong baseline, as ESA's 2019 Kelvins challenge showed. Better tracking reduces manoeuvres more than any threshold rule, because poor tracking dilutes the probability.
- In defence, AI's main job is managing attention: NGA Maven had more than 20 000 users in 2025, and tip-and-cue links constellations. At national scale, false alarms set the limit — $10^{-4}$ per cell gives over 55 000 false alerts per pass over 500 000 km².
- Adversaries can deceive both sensors and models with decoys, camouflage and adversarial patterns. Fusing different sensors and keeping people in the review loop are the main defences.
- US policy requires "appropriate levels of human judgment over the use of force". International talks on autonomous weapons reached a consensus text in 2026, with the decision on negotiations due in November 2026.

### REVIEW QUESTIONS

1. A 40-pixel streak has a peak brightness of 0.5σ per pixel and a blur of $\sigma_{\mathrm{psf}}$ = 1.5 px. Estimate its matched-filter signal-to-noise ratio. Why does searching over many angles reduce the real gain?
2. Explain the Mahalanobis distance and the role of a gate. Why does global assignment outperform the greedy rule in Figure 50.4 when objects are close together?
3. Using $\Delta s \approx \frac{3}{4} n \varepsilon |\dot{a}| t^2$, estimate the along-track error after two days for a satellite at 450 km with $|\dot{a}|$ = 50 m/day and a 30 % density error.
4. In Figure 50.5 the neural-network correction helped after storms but not on average. Give two reasons, and say how you would test such a model before using it.
5. For a catalogue of 100 000 objects, how many pairs are there? By what factor does the number of daily close approaches change compared with 50 000 objects?
6. What is dilution in the probability of collision? Using the formula in this chapter, find the largest possible $P_c$ for a miss of 300 m and a hard-body radius of 10 m.
7. What did ESA's Kelvins challenge teach about machine learning for conjunction triage? Why is the latest risk such a strong baseline?
8. Compare CUSUM and a learned classifier for manoeuvre detection. Why do storms cause false alarms, and how can space-weather data reduce them?
9. A change detector screens 200 000 km² in 10 m cells with a false-alarm probability of $10^{-6}$ per cell. How many false alerts per pass does it make? What would confirmation by a second look achieve, and why is the real gain smaller?
10. Describe two ways AI in space systems could make a crisis more stable and two ways it could make it less stable. What human decision points would you keep?

CHAPTER

# 51 AI for Space Science, Planetary Exploration and Human Spaceflight

Image: NASA/JSC

*Space science now produces far more data than people can inspect. A survey telescope sends millions of alerts a night, and orbiters have imaged whole planets at metre scale. A crew on the way to Mars will be twenty minutes from help. This chapter shows how machine learning finds planets in light curves, classifies transients, maps galaxies and lenses, estimates redshifts and speeds up gravitational-wave analysis. It then turns to craters, boulders and minerals on other worlds, to science autonomy on rovers, and to AI for crew health, assistants, robots, smart habitats and Earth-independent operations. Small experiments, run from scratch, show what each method does and where it fails. Status is given as of October 2026.*

IN THIS CHAPTER

51.1 Why Science Needs Machine Learning
51.2 Exoplanets: Finding Small Signals in Big Data
51.3 The Time-Domain Sky
51.4 Galaxies, Lenses and Cosmology
51.5 The Sun: Features and Event Catalogues
51.6 Planetary Surfaces and Science Autonomy
51.7 AI for Human Spaceflight
51.8 Validating an AI Discovery

Science missions are built to collect data. For most of the space age, people could look at all of it. Astronomers inspected every photographic plate, and planetary geologists mapped every image by hand. That is no longer possible. Kepler watched about 150 000 stars every 30 minutes for four

years. The Vera C. Rubin Observatory now sends up to several million alerts each night. Orbiters around the Moon and Mars have returned millions of images sharp enough to show single boulders.

Machine learning (ML) helps in two ways. As a **filter and ranker**, it sorts a flood of data so that people look first at what most likely matters. As a **fast approximation** to slow physics, it gives in a second an answer that a full calculation needs hours to produce. Either way, scientists make the final decision. The hero image shows a third role: a robot that works beside a crew on the International Space Station (ISS).

This chapter covers astronomy, the Sun, planetary science and human spaceflight. Telescopes are described in Chapters 38 and 39, planetary missions in Chapters 24–29 and human spaceflight in Chapters 30–33. Learning methods are explained in Chapters 43 and 44, onboard computing in Chapter 45, and rover autonomy in Chapter 48.

## 51.1 Why Science Needs Machine Learning

The key idea is that data now grow faster than the number of people who can look at them. Learning methods turn huge archives into short lists that people can check.

### 51.1.1 The Data Deluge

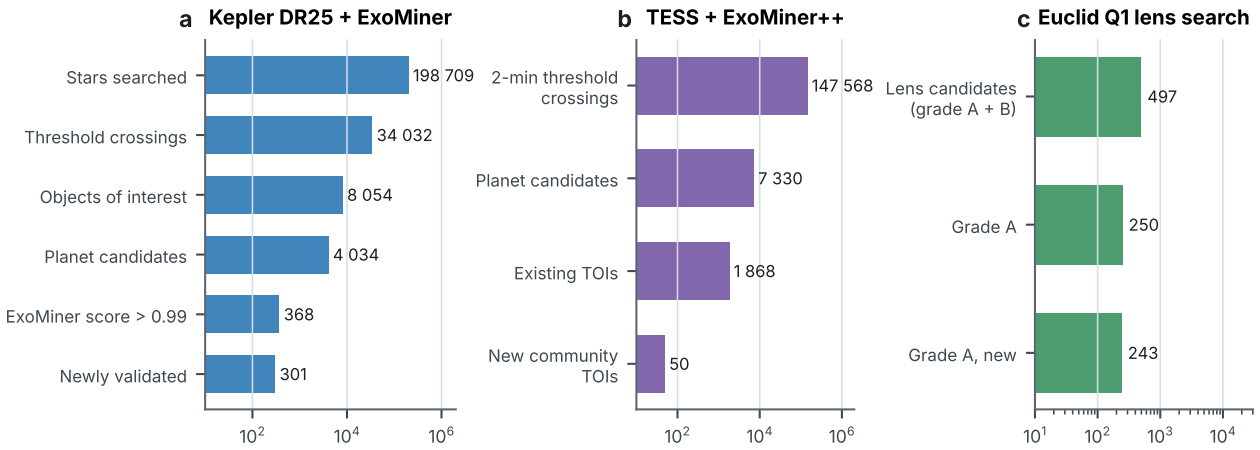
The data rates of some major science facilities are compared in Table 51.1. Light curves call for signal detection, images for object detection and segmentation. Alert streams need decisions within minutes, while archives can be mined for years.

**TABLE 51.1** Data rates of some major science facilities and the learning tasks they drive (status October 2026).

| Facility | Data rate or volume | Main learning tasks |
|---|---|---|
| Kepler (2009–18) | ≈ 150 000 stars every 30 min | Planet search and vetting |
| TESS (2018–) | Most of the sky in 27-day sectors | Vetting thousands of candidates |
| Gaia (2013–25) | > 500 TB for its fourth release (Dec 2026) | Classifying stars and their motions |
| Euclid (2023–) | ≈ 100 GB per day | Shapes, lenses, morphology |
| SDO (2010–) | About 1.5 TB per day | Solar features and event catalogues |
| Rubin LSST (2026–) | ≈ 10 TB per night; up to ~7 million alerts | Real/bogus, transient classes, redshifts |
| LIGO–Virgo–KAGRA | About 250 candidates in 2023–25 | Glitches, rapid parameter estimation |
| SKA (2030s) | ≈ 700 PB of products per year (estimate) | Imaging, radio transients |

### 51.1.2 Funnels, Not Oracles

A discovery usually passes through a **funnel**: a search finds many candidates, filters remove most of them, and people examine the rest. Three real funnels are shown in Figure 51.1. Kepler's final search of 198 709 stars produced 34 032 "threshold-crossing events" (TCEs): signals strong enough to pass the detection limit. Only 4 034 survived as planet candidates. A neural network then scored 1 922 unlabelled candidates, and 301 were validated as new planets.




Sources: Kepler DR25 catalogue (Thompson et al. 2018; 198 709 stars searched, 34 032 TCEs); ExoMiner (Valizadegan et al. 2022, 1 922 unlabelled KOIs scored); ExoMiner++ (Valizadegan et al. 2025); Euclid Q1 strong-lensing discovery engine (Euclid Collaboration 2025/26, 63 deg$^2$).

**FIGURE 51.1** Discovery funnels from published numbers. (a) Kepler's final search and the ExoMiner validation of 301 new planets. (b) The ExoMiner++ catalogue for two-minute TESS data. (c) The Euclid Q1 search for strong gravitational lenses. Each step removes most of the objects left by the step before.

Machine learning sits at the narrow middle of each funnel. It must keep nearly all real objects (high **recall**) while passing few false ones (high **precision**; Chapter 43). A model that ranks well saves years of human effort. A model that hides a class of objects can bias a whole field.

### 51.1.3 From Pixels to Alerts

A typical pipeline for a survey telescope is shown in Figure 51.2. The observatory side is automatic. Images are calibrated and a deep "template" image of the same field is subtracted. Anything that has changed leaves a residual. A first network rejects artefacts — cosmic rays, satellite trails and bad subtractions — that are far more common than real changes. Alerts then go to **brokers**: software services that add information from catalogues, classify each object and pass it to the science teams that asked for it.

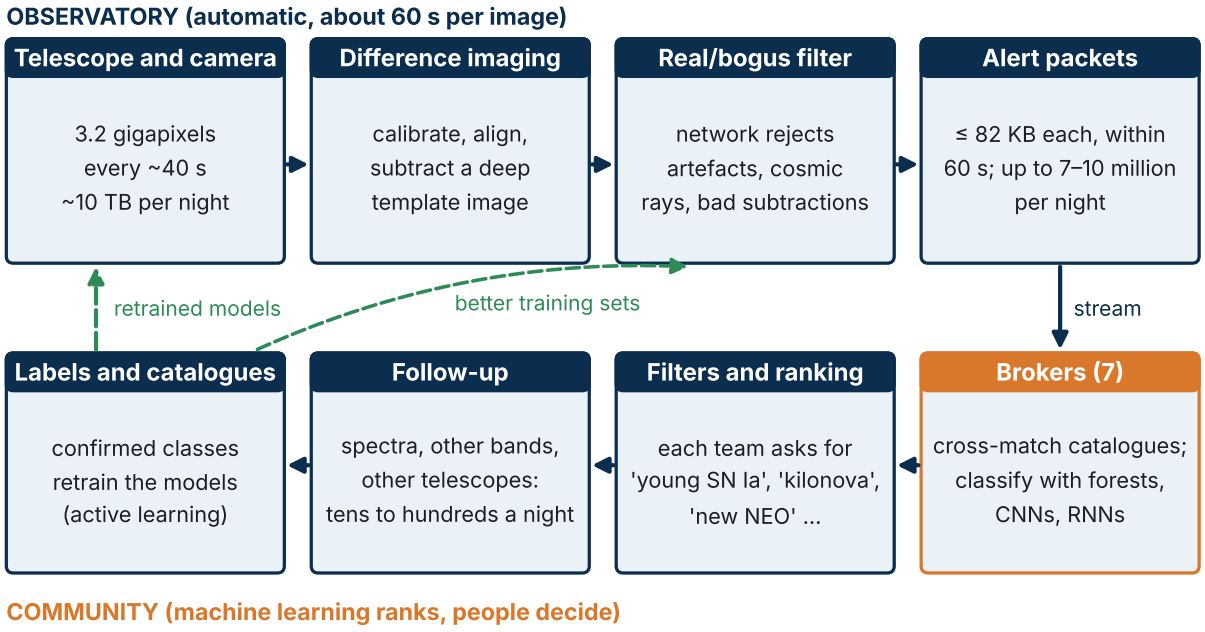



**FIGURE 51.2** A survey alert pipeline, modelled on the Vera C. Rubin Observatory. Learning appears at three points: rejecting artefacts, classifying alerts in the brokers, and ranking targets for follow-up. Confirmed classes return as labels and improve the next models.

The loop at the bottom matters as much as the arrows at the top. Every spectrum taken for follow-up becomes a new label, so the models learn from their own mistakes.

## 51.2 Exoplanets: Finding Small Signals in Big Data

Planet searches find far more false signals than planets. Learning is most useful in **vetting**: deciding which signals are real.

### 51.2.1 Searching Light Curves

When a planet crosses its star, the star dims by the transit depth $\delta = (R_p/R_\star)^2$ (Chapter 39). An Earth-sized planet in front of a Sun-like star gives 84 parts per million (ppm). Noise from the detector, from the star and from the spacecraft is usually larger than that for a single transit. The signal becomes visible only when many transits are added together.

The standard search is **box least squares (BLS)**. It tries every period $P$, every phase and a few transit durations. For each trial it folds the light curve and compares the mean flux inside a box with the mean outside. The result is a signal-to-noise ratio:

$$\mathrm{SNR} = \frac{\hat{\delta}}{\sigma}\sqrt{n_{\mathrm{in}}}.$$

Here $\hat{\delta}$ is the measured depth, $\sigma$ the noise per point and $n_{\mathrm{in}}$ the number of points in transit. A signal with SNR above a threshold, 7.1 for Kepler, becomes a TCE.

A search on simulated data is shown in Figure 51.3. Two years of photometry at 30-minute cadence contain starspots, gaps and white noise of 250 ppm per point. A planet 1.3 times Earth's radius, with a depth of 142 ppm, is hidden in it; one transit has an SNR of only 1.7. The light curve is first **detrended**: a two-day running median removes the starspot signal. The BLS search over 40 000 trial periods then finds a peak at 17.632 days (true value 17.63) with SNR 8.3 from 42 transits. The next-highest unrelated peak reaches 6.4.

The recovered box depth is 130 ppm, a little less than the 142 ppm injected. Limb darkening makes a real transit rounded rather than box-shaped, and detrending removes a little depth. Such losses are why pipelines fit a physical transit model after the search.

### 51.2.2 Vetting with Neural Networks

Most TCEs are not planets. In Kepler's final catalogue, 88 % of them did not become planet candidates. Some are **eclipsing binaries**: pairs of stars that eclipse each other. Their eclipses are often V-shaped, may show a shallow secondary eclipse half an orbit later, and may differ in depth between odd and even events. Others are instrumental artefacts, such as sudden drops in pixel sensitivity, or stellar variability that the search mistook for a dip.

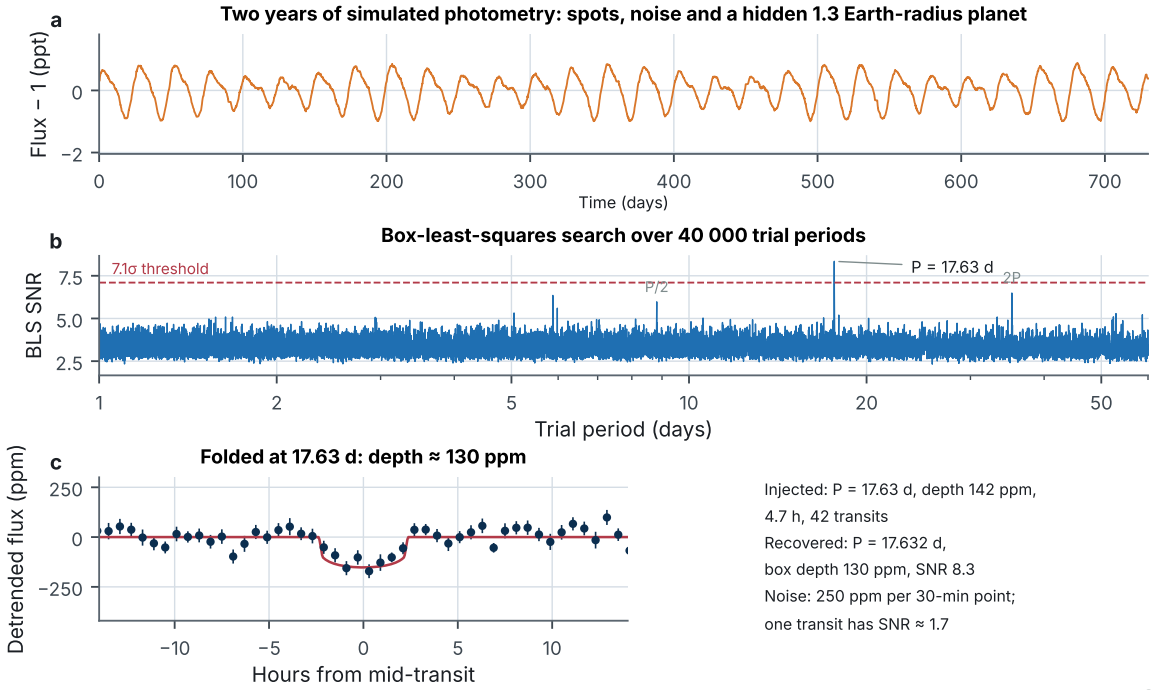



**FIGURE 51.3** A transit search on simulated data. (a) Two years of photometry with starspot modulation; the orange line is the trend removed before the search. (b) Box-least-squares signal-to-noise against trial period; the peak lies at the planet's period, with weaker peaks at half and twice the period. (c) Data folded at the recovered period, binned (dark points) and compared with the injected transit (red).

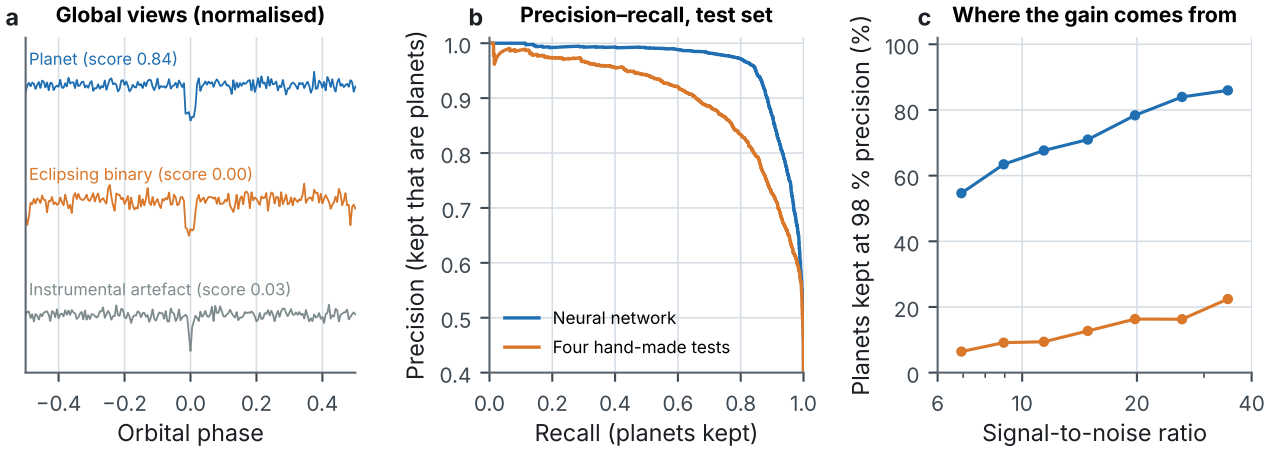
Kepler's team first vetted signals by eye, then with the **Robovetter**, a set of expert rules. Each rule tests one property, such as a secondary eclipse or a shift of the star's image during the dip. In 2017 a Google engineer and an astronomer at the University of Texas at Austin trained a **convolutional neural network** (Chapter 44) on about 15 000 labelled Kepler signals. Their network, AstroNet, looked at two "views" of each folded light curve: the whole orbit and a zoom on the transit. It reached about 96 % accuracy on test data.

### LANDMARK: KEPLER-90I, THE FIRST PLANET FOUND BY A NEURAL NETWORK (DECEMBER 2017)

- AstroNet was run on 670 stars that already had several known planet candidates. Weak signals around such stars are more likely to be real.
- It found Kepler-90i, a planet about 30 % larger than Earth with a 14.4-day orbit, and Kepler-80g. Kepler-90 became the first known star with eight planets, tying the Solar System.
- The team said openly that the network also produced many false positives. Each new planet was confirmed by further statistical tests and by checking the folded data by hand.
- The code was released openly in 2018, and learned vetting has since become a standard step in planet searches.

NASA's **ExoMiner** (2021) went further. Its network has separate branches for transit shape, secondary eclipse, odd–even depths and image centroids, so it can explain why a signal looks false. On Kepler test data it kept 93.6 % of planets at a precision of 99 %, against 76.3 % for the best earlier classifier. Applied to 1 922 unlabelled candidates, it gave 368 a score above 0.99, and 301 of these were validated as new planets. **ExoMiner++** (2025) adapted the method to TESS. It was trained on Kepler's well-labelled signals together with TESS data, whose labels are noisier. It flagged 7 330 of 147 568 two-minute-cadence TESS signals as planet candidates.

In Figure 51.4 the idea is repeated on 36 000 synthetic folded signals. They include planets, several kinds of eclipsing binary and artefacts, with SNR from 6 to 40. A small numpy neural network sees the raw global and local views. A logistic-regression vetter sees only four hand-made tests: secondary eclipse, V shape, asymmetry and depth just outside the transit. Both were trained on 24 000 signals and tested on 8 000.




Synthetic data: 36 000 folded signals (40 % planets, 35 % eclipsing binaries with V shapes, secondary eclipses or odd–even differences, 25 % artefacts), SNR 6–40. Network: 262 inputs (201-bin global and 61-bin local views), two ReLU layers (64, 32), dropout, Adam; trained on 24 000, tested on 8 000. Seed 510.

**FIGURE 51.4** Vetting synthetic transit signals. (a) Examples of a planet, an eclipsing binary and an artefact, with the network's planet score. (b) Precision against recall: the network keeps far more planets at high precision than a classifier built on four hand-made tests. (c) The fraction of planets kept at 98 % precision, by signal-to-noise ratio.

The network reached 92.0 % accuracy. At 95 % precision it kept 84.8 % of planets, against 43.7 % for the hand-made tests. At 99 % precision the gap widened: 58.7 % against 6.5 %. The network gained most at low SNR, where single tests are noisy but the whole shape still carries information. Diluted background binaries, with weak secondaries and U-shaped eclipses, stayed hard for both. In real data they are rejected by other evidence, such as shifts of the star's image or spectra from the ground.

**FIGURE 51.5** The Transiting Exoplanet Survey Satellite (TESS) being lifted onto a work stand at Kennedy Space Center in February 2018. Its four cameras produce thousands of candidate signals that learned vetting tools help to sort. Image: NASA/KSC (NASA/Ben Smegelsky).

### 51.2.3 Radial Velocities and Stellar Noise

Transits give a planet's size. Its mass comes from the **radial-velocity** method: the star's spectrum shifts as the planet pulls it back and forth. Earth makes the Sun wobble by only about 9 cm/s. Starspots and convection on the star's surface change the shapes of its spectral lines. They produce false velocity signals of around 1 m/s, ten times larger than the planet.

**Gaussian processes** (Chapter 43) are the usual tool: they model the star's activity as correlated noise, fitted together with the planet's orbit. In 2022 a team trained a convolutional network on the line shapes in more than 600 solar spectra from the HARPS-N solar telescope, taken over three years. It reduced the scatter of the solar velocities from 1.75 m/s to 1.04 m/s — a large step, but still ten times too much to find an Earth twin.

> **EXAMPLE: AN EARTH TWIN IN KEPLER DATA, AND THE FALSE-ALARM BUDGET**
>
> Kepler's noise for a bright, quiet Sun-like star was about 30 ppm over 6.5 hours. An Earth-sized planet in a one-year orbit gives a depth of 84 ppm and a central transit of about 13 hours.
>
> 1. **Noise over one transit.** Noise falls as the square root of time: $30 \times \sqrt{(6.5/13)} = \mathbf{21.2\ ppm}$. One transit gives SNR = 84/21.2 = **4.0**.
> 2. **Several transits.** Four transits give $4.0 \times \sqrt{4} = \mathbf{7.9}$, just above the 7.1 threshold. Three transits give **6.8**, which is below it. About 3.2 transits are needed, so an Earth twin is at the edge of a four-year mission.
> 3. **Why 7.1?** For pure Gaussian noise, the chance of a value above $7.1\sigma$ is $6.2 \times 10^{-13}$. Suppose each of the 198 709 stars needs about five million independent trials of period, phase and duration. That is $10^{12}$ tests, which gives about **0.6** false alarms. The threshold was set for exactly this reason.
> 4. **Reality.** Real noise is not Gaussian. The search produced 34 032 TCEs, and 88 % were not planet candidates.
> 5. **Human cost.** At five minutes per signal, vetting all TCEs by eye would take about 2 840 hours, or about one and a half working years. A neural network scores them all in minutes.
>
> The threshold controls false alarms from random noise; vetting controls false alarms from everything else. Learned vetting does not lower the threshold. It makes the work above the threshold affordable.

## 51.3 The Time-Domain Sky

The key idea of this section is speed. Transients fade in hours to weeks, so they must be classified while they can still be observed.

### 51.3.1 Rubin and the Alert Brokers

The **Vera C. Rubin Observatory** in Chile has an 8.4 m telescope and the world's largest astronomical camera, with 3 200 megapixels. Its Legacy Survey of Space and Time (LSST) images the whole southern sky every few nights for ten years. Each image is compared with a template within about a minute. Every change becomes an **alert**: a small packet with the measurement, image cut-outs and the object's history.

DATA BOX — RUBIN'S FIRST YEAR OF ALERTS

- **First look.** First images were released in June 2025. The first world-public alerts reached the brokers on the night of 24 February 2026, about 800 000 of them.
- **Survey.** The ten-year survey formally began in mid-2026. The camera takes an image about every 40 s and the observatory produces about 10 TB of data a night.
- **Alerts.** The design allows 10 000 alerts per image and about 1 000 images a night, or 10 million alerts. The current estimate is up to 7 million a night and about 20 billion over the survey.
- **Speed and size.** 98 % of alerts must leave within 60 s of readout; each packet is up to 82 KB.
- **Brokers.** Seven "full-stream" brokers receive every alert: ALeRCE, AMPEL, ANTARES, Babamul, Fink, Lasair and Pitt-Google. Two downstream services take subsets, one for Solar System objects.
- **Early science.** In about six weeks of early test surveys, Rubin found more than 11 000 new asteroids, including 33 near-Earth objects.

Brokers were rehearsed for years. In the 2018 **PLAsTiCC** challenge, more than 1 000 teams classified simulated Rubin light curves, with only about 7 850 labelled for training. The winner degraded bright training objects to look faint and distant, then used gradient-boosted trees. The lesson — make the training set look like the data — applies far beyond astronomy.

### 51.3.2 Classifying Transients

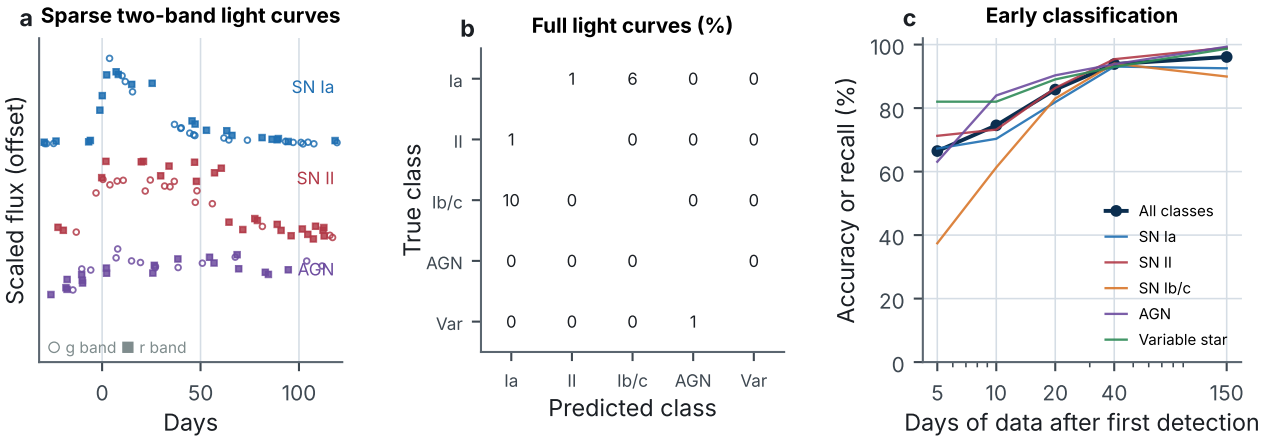
A survey light curve is sparse: each object is seen every few days, in different filters, with noisy fluxes. Classifiers work on the raw sequence or on features such as rise time, decline rate, peak colour and periodicity. A **random forest** (Chapter 43) takes a vote among many decision trees $T_b$, each trained on a random sample of the data.

$$\hat{y}(\mathbf{x}) = \operatorname{mode}\{T_1(\mathbf{x}), \dots, T_B(\mathbf{x})\}.$$

ALeRCE, one of the Rubin brokers, has long used such a forest for light curves, alongside a convolutional network that classifies image cut-outs.

A small version is shown in Figure 51.6. Synthetic two-band light curves of five classes were generated: type Ia supernovae, type II and type Ib/c supernovae, active galactic nuclei (AGN) and periodic variable stars. Each had 28–45 irregular epochs with noise. A random forest of 200 trees used 15 features and was trained on 6 000 objects.

With full light curves the forest classified 96.1 % of 3 000 test objects correctly. Most errors confused type Ia with type Ib/c supernovae, which also happens in real data. The useful number is the early one. Five days after the first detection, accuracy was only 66.5 %, and type Ib/c supernovae were recognised just 37 % of the time. After 20 days it reached 85.8 %. Astronomers want spectra *before* the peak, so brokers must rank objects while the classification is still uncertain. They therefore report probabilities, not labels.




Synthetic two-band light curves: SN Ia, II, Ib/c (Bazin-type profiles with class-dependent rise, decline, colour and secondary peak), AGN (damped random walk) and periodic variables; 28–45 irregular epochs with noise. Random forest (200 trees, 15 features); 6 000 training and 3 000 test objects. Seed 5152.

**FIGURE 51.6** Classifying synthetic transient light curves. (a) Examples of a type Ia supernova, a type II supernova and an active galactic nucleus. (b) Confusion matrix for full light curves. (c) Accuracy, and recall for each class, as more days of data arrive after the first detection.

**EXAMPLE: RUBIN'S ALERT STREAM AGAINST HUMAN ATTENTION**

Take 7 million alerts in a 10-hour night.

1. **Rate.** 7 000 000 / 36 000 s = **194 alerts per second**.
2. **People.** If a trained person needs 3 s to glance at one alert, a night's stream needs 5 830 hours of attention. That is about **730 people** working 8-hour shifts, every night for ten years.
3. **Data.** At up to 82 KB per alert, the stream is about 0.6 TB a night. Most alerts concern known variable stars and asteroids.
4. **Follow-up.** Suppose the world's telescopes can take 100 spectra of new transients a night. That is one alert in **70 000**.
5. **What machine learning must do.** It must remove artefacts, recognise known variables and asteroids, and rank the rest. A 1 % error in a filter applied to 7 million alerts is still 70 000 mistakes a night. Errors must be counted, not just accuracy.

No human team can read the stream. People decide which questions to ask the brokers, check the top of each ranked list, and take the spectra that create new labels.

### 51.3.3 Radio Bursts and Gravitational Waves

**Fast radio bursts (FRBs)** are millisecond flashes of radio waves from distant galaxies. In 2018 a convolutional network re-analysed five hours of Green Bank Telescope data from Breakthrough Listen. It found 72 new bursts from the repeating source FRB 121102, on top of the 21 found by the original search. The second CHIME/FRB catalogue (2026) lists 4 539 bursts from 3 641 sources. At this rate, sorting real bursts from radio interference must be automatic.

**Gravitational-wave** detectors measure changes in arm length smaller than a proton. The standard search is a **matched filter**. It correlates the data with a bank of theoretical waveforms $h$, weighting each frequency $f$ by the detector noise $S_n(f)$.

$$\rho^2 = 4 \int_0^\infty \frac{|\tilde{h}(f)|^2}{S_n(f)}\, df.$$

The fourth observing run of LIGO, Virgo and KAGRA ran from May 2023 to 18 November 2025 and produced about 250 candidate signals. Learning helps at three points. **Gravity Spy** combines volunteers and a network to classify detector "glitches", short noise bursts that can mimic signals. Research pipelines such as Aframe search for signals with neural networks in milliseconds, though they do not yet issue official alerts.

The biggest gain is in **parameter estimation**. Standard Bayesian sampling of a merger's masses, spins and sky position can take hours to days. **Simulation-based inference** instead trains a network on millions of simulated signals to output the posterior directly. In March 2025 a team reported DINGO-BNS, which infers all 17 parameters of a neutron-star merger in about one second. Its 90 % sky areas were typically about 30 % smaller than those of the standard rapid method. Such speed lets telescopes point at a kilonova before it fades.

Learning can also improve the instrument itself. In September 2025 researchers from Google DeepMind and LIGO reported **Deep Loop Shaping**, a reinforcement-learning controller (Chapter 48) for one mirror-alignment loop at LIGO Livingston. It cut control noise between 10 and 30 Hz by more than 30 times, and by up to 100 times in narrow bands. It was a demonstration on one loop, not yet part of routine observing.

## 51.4 Galaxies, Lenses and Cosmology

Learning turns images of billions of galaxies into measurements: shapes, distances and the rare objects among them.

### 51.4.1 Galaxy Shapes

**Galaxy Zoo**, launched in 2007, asked volunteers to classify galaxy images (Chapter 43). Its labels trained **Zoobot**, a family of networks that predict how volunteers would answer each question about a galaxy. In 2023 Zoobot measured detailed shapes for 8.67 million galaxies in the DESI Legacy Imaging Surveys. For Euclid, tests on simulated images suggest that about 800 million galaxies in the wide survey could be measured reliably.

In 2024 a network trained on labelled Hubble images was adapted to the sharper, redder images of the James Webb Space Telescope (JWST). It classified about 20 000 distant galaxies in the CEERS survey and agreed with visual classifications 80–90 % of the time. Moving a model between instruments with few new labels is called **domain adaptation** (Chapter 45).

### 51.4.2 Strong Lenses in Euclid

A massive galaxy bends the light of a galaxy behind it into arcs or rings. Such **strong lenses** weigh the dark matter in the lensing galaxy and test cosmology, but only roughly one galaxy in ten thousand shows one. Euclid's first quick release (March 2025) covered 63 square degrees. An ensemble of networks, including Zoobot, ranked the galaxies. Volunteers on the Space Warps project then inspected the best candidates, and experts graded and modelled them.

The result was a catalogue of 497 galaxy–galaxy lenses, 250 of them "grade A"; 243 of those were new. The same engine is expected to yield about 7 000 high-confidence (grade A or B) lenses in the first major data release, DR1. Its first part is due in November 2026 and the full release in mid-2027. The full survey should yield more than 100 000.

### 51.4.3 Photometric Redshifts

Most galaxies are too faint for a spectrum, so their distance must be estimated from colours in a few broad filters. This is a **photometric redshift**. Spectral features, such as the 4000 Å break and the Lyman break, move through the filters as redshift grows. Learning methods map colours directly to redshift, trained on galaxies whose spectroscopic redshifts are known. Quality is measured by the normalised scatter of $\Delta z = (z_{\mathrm{phot}} - z_{\mathrm{spec}})/(1 + z_{\mathrm{spec}})$:

$$\sigma_{\mathrm{NMAD}} = 1.4826 \times \mathrm{median}\,|\Delta z - \mathrm{median}(\Delta z)|,$$

and by the fraction of **outliers** with $|\Delta z| > 0.15$. Rubin's goals for its "gold" sample, brighter than magnitude 25.3 in the i band, are a scatter below 0.02 (requirement 0.05) and fewer than 10 % large (3σ) outliers.

In Figure 51.7, a random forest is tested on 60 000 synthetic galaxies observed in six Rubin-like filters. Trained on a representative sample of 30 000, it reached a scatter of 0.017 with 12.0 % outliers. Errors grew steeply for faint galaxies, from 0.007 at i ≈ 20 to 0.13 at i ≈ 25. Then the forest was trained only on galaxies brighter than i = 22.5 and nearer than z = 1.2, like a typical spectroscopic sample. The outlier fraction rose to 19.4 %, and to 61 % for the faintest galaxies. A model cannot learn what its training set never shows.

Synthetic galaxies: continuum, 4000 Å break and emission lines varying with type, dust; Lyman-α forest absorption; six LSST-like Gaussian filters with 10-year depths; i < 25.3. Random forest (120 trees) on i magnitude and five colours; 30 000 training (b: only the 13 929 with i < 22.5 and z < 1.2), 30 000 test galaxies; outliers |Δz|/(1+z) > 0.15. Seed 5151.

**FIGURE 51.7** Photometric redshifts from a random forest. (a) Photometric against true redshift when the training set represents the whole population. (b) The same when training uses only bright, nearby galaxies: faint and distant galaxies are pulled to the wrong redshifts. (c) Outlier fraction against brightness for both cases.

### 51.4.4 Simulation-Based Inference in Cosmology

Cosmology usually compresses a survey into a few summary statistics, such as the two-point correlation of galaxy positions, and fits them with a formula. Much information lies in the non-linear structure that such summaries miss. **Simulation-based inference (SBI)** uses Bayes' theorem,

$$p(\boldsymbol{\theta} \mid \mathbf{x}) \propto p(\mathbf{x} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}),$$

without writing down the likelihood $p(\mathbf{x} \mid \boldsymbol{\theta})$. Instead, thousands of simulated universes with known parameters $\boldsymbol{\theta}$ train a network to estimate the posterior. The SimBIG project applied this to part of the BOSS galaxy survey. Using the full three-dimensional galaxy field, it constrained the

clustering amplitude $\sigma_8$ 2.65 times more tightly than a standard power-spectrum analysis of the same data. The method is only as good as its simulations: if they miss a physical effect, the answer is precise but wrong.

Learning also works inside telescope pipelines (Chapter 38), flagging cosmic rays and artefacts and separating overlapping objects. Landmark results in astronomy are listed in Table 51.2.

**TABLE 51.2** Landmark uses of machine learning in astronomy. CNN: convolutional neural network; RNN: recurrent neural network.

| Result | Year | Data | Method | Outcome | Status (Oct 2026) |
|---|---|---|---|---|---|
| AstroNet | 2017 | Kepler | 1-D CNN | Kepler-90i, Kepler-80g | Research, open code |
| ExoMiner | 2021 | Kepler | Explainable CNN | 301 validated planets | Planets in NASA archive |
| ExoMiner++ | 2025 | TESS | Kepler + TESS training | 7 330 candidates | Public catalogue |
| FRB 121102 search | 2018 | Green Bank | CNN | 72 new bursts | Research |
| Galaxy Zoo DESI | 2023 | DESI imaging | Zoobot CNN | 8.67 million galaxies | Public catalogue |
| Euclid lens engine | 2025 | Euclid Q1 | CNN ensemble + volunteers | 497 lenses | In Euclid pipeline |
| SimBIG | 2023–24 | BOSS | Simulation-based inference | $\sigma_8$ 2.65× tighter | Research |
| DINGO-BNS | 2025 | LIGO–Virgo | Neural posterior | 17 parameters in 1 s | Research |
| Deep Loop Shaping | 2025 | LIGO Livingston | Reinforcement learning | Control noise ÷ 30–100 | Demonstration |
| Rubin brokers | 2026 | LSST alerts | Forests, CNNs, RNNs | Real-time classes | Operational |

## 51.5 The Sun: Features and Event Catalogues

The key idea is that heliophysics needs *catalogues*: lists of every flare, sunspot group, filament and eruption, with times and positions. Forecasting space weather is covered in Chapter 47. Here the aim is to describe what happened.

The Solar Dynamics Observatory (SDO) sends about 1.5 TB of images a day. Around its launch in 2010, NASA funded a **Feature Finding Team** to build automatic detection modules. Their reports feed the **Heliophysics Event Knowledgebase** (HEK), which scientists search for, say, every filament that erupted near an active region. A 2016 study counted more than 280 000 event reports for seven kinds of solar phenomenon. Coronagraph catalogues such as CACTus detect coronal mass ejections automatically, and are compared with catalogues made by people.

The early modules used thresholds, edge detection and tracking. Since about 2018, convolutional networks have segmented coronal holes, filaments and active regions in several wavelengths at once; the Surya foundation model is described in Chapter 44. The challenge is consistency. A catalogue that changes its detector can create false trends over the 11-year solar cycle, so versions must be recorded and old data reprocessed.

# 51.6 Planetary Surfaces and Science Autonomy

Planetary images and spectra become maps — of craters, rocks and minerals — and those maps drive landing, driving and sampling decisions. Learning makes global maps possible. Physics decides what they mean.

## 51.6.1 Craters and Surface Ages

On airless bodies, craters accumulate at a known rate. Counting them dates a surface: more craters per square kilometre means an older surface. The method rests on two functions calibrated with Apollo and Luna samples (Chapter 29). The **production function** gives the shape of the size distribution, and the **chronology function** gives the density of craters larger than 1 km, $N_1$, after $T$ billion years. For the Moon, Neukum and colleagues (2001) give

$$N_1(T) = 5.44 \times 10^{-14}\left(e^{6.93\,T} - 1\right) + 8.38 \times 10^{-4}\,T \quad \text{per km}^2.$$

The first term describes the heavy bombardment of the early Moon; the second, the nearly constant rate since about 3 billion years ago.

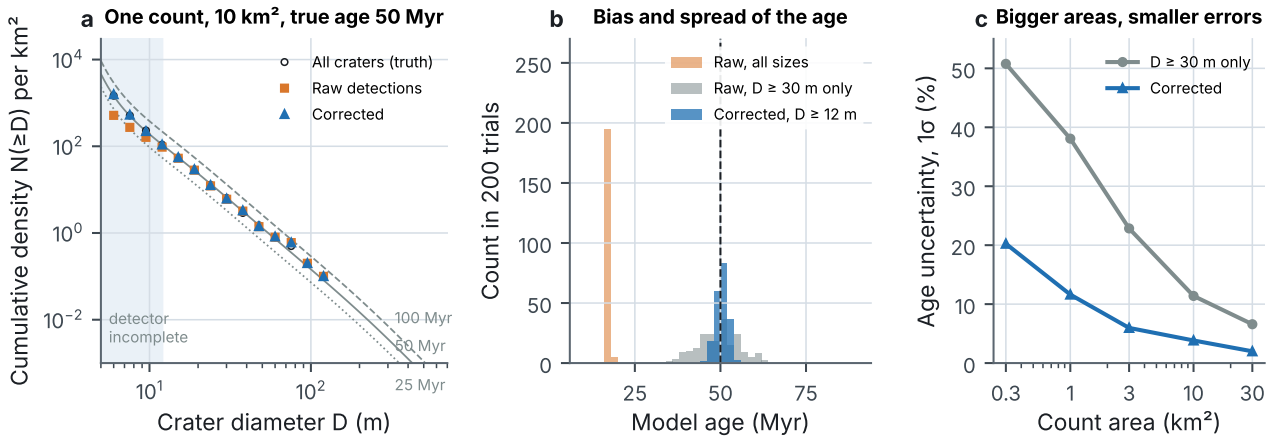
Counting small craters by hand is slow and inconsistent between people, so trained detectors now do it at planetary scale. In 2020 a team used transfer learning on Chang'e-1 and Chang'e-2 data to find 109 956 new lunar craters. In 2021 a detector trained on HiRISE images found about 90 million craters in the Context Camera mosaic of Mars. Rays of small secondary craters then traced a group of Martian meteorites to their source craters in the Tharsis region. Crater detection for navigation is covered in Chapter 48.

A detector does not see every crater. Small craters, only a few pixels across, are often missed, and shadows or boulders can be mistaken for craters. The fix is to measure **completeness** $c(D)$ and the false-positive density $f(D)$ by testing the detector on images with known craters. A model age is then fitted by **maximum likelihood**. Suppose $k_i$ craters are counted in size bin $i$ of an area $A$. The expected count $\lambda_i$ and the log-likelihood are

$$\lambda_i(T) = A\,\Delta N_i(T)\,c_i + A f_i, \qquad \ln L(T) = \sum_i \left[k_i \ln \lambda_i - \lambda_i\right],$$

Here $\Delta N_i$ is the number per unit area in bin $i$ from the production and chronology functions.

In Figure 51.8 this is applied to a synthetic 10 km$^2$ lunar surface aged 50 million years, imaged at 1.5 m per pixel. The synthetic detector finds half the craters 7 pixels across and 95 % of large ones. About 40 % of its detections between 6 and 10 m are false. Fitting the raw counts at all sizes gives an age of 17 million years, three times too young. Counting only craters of 30 m and larger gives an unbiased median of 49 million years, but with a spread of ±13 %. Correcting for completeness and false positives, and fitting from 12 m upwards, gives 51 million years with a spread of only ±3 %.




Lunar production and chronology functions of Neukum et al. (2001). Synthetic detector on 1.5 m pixels: recall 50 % at 7 px rising to 95 %, 7 % diameter error, false positives near the resolution limit. Completeness and false-positive rates measured by injection on a separate 30 km² labelled set; ages by Poisson maximum likelihood in 10 bins per decade. Seed 5153.

**FIGURE 51.8** Crater-count dating with an imperfect detector. (a) Cumulative crater densities for one synthetic count, with model curves for three ages; raw detections fall below the truth at small sizes. (b) Ages from 200 repeated counts: raw counts at all sizes are far too young; corrected counts are accurate and precise. (c) Age uncertainty against count area.

The lesson applies to every automated catalogue: a detector's errors must be measured and modelled, not just reduced. Small craters also reach **equilibrium** on old surfaces, where new craters erase old ones as fast as they form. Below that size, no detector can date the surface.

### 51.6.2 Boulders, Rockfalls and Landslides

Boulders tell geologists how deep the regolith is and how surfaces erode. They are also hazards for landers and rovers (Chapter 27). In 2020 a convolutional network searched images from the Narrow Angle Camera (NAC) of NASA's Lunar Reconnaissance Orbiter for boulders that had rolled downhill. It mapped 136 610 such rockfalls and showed that impacts trigger most of them. In 2025 a team mapped about 94 million boulders larger than about 4.5 m, using 635 000 NAC images between 60° S and 60° N.

Shadows hide some of the most valuable terrain. Permanently shadowed craters near the lunar poles may hold water ice (Chapter 41). In 2021 a deep-learning tool called HORUS removed noise from NAC images of these craters, lit only by faint light scattered from nearby slopes. It revealed features a few metres across at about 1.5 m per pixel. It was later used to examine candidate target regions for NASA's VIPER rover.

Such mapping projects are summarised in Table 51.3. Fresh impacts found by a classifier in Mars orbiter images are described in Chapter 44.

**TABLE 51.3** Planetary mapping with machine learning.

| Project | Body, data | Method | Result | Use |
|---|---|---|---|---|
| Lunar rockfalls (2020) | Moon, LRO NAC | CNN detector | 136 610 rockfalls | Erosion; hazards |
| Chang'e craters (2020) | Moon, Chang'e-1/2 | Transfer learning | 109 956 new craters | Crater ages |
| HORUS (2021) | Moon, NAC in shadow | Physics-guided denoising | ≈ 1.5 m images of shadowed craters | Ice prospecting, VIPER |
| Mars craters (2021) | Mars, CTX mosaic | CNN detector | ≈ 90 million craters | Meteorite source craters |

| Project | Body, data | Method | Result | Use |
|---|---|---|---|---|
| AI4Mars (2021) | Mars, rover images | Crowd labels + segmentation | 326 000 labels | Terrain classes for driving |
| Lunar boulders (2025) | Moon, 635 000 NAC images | CNN detector | ≈ 94 million boulders | Regolith depth; landing |

### 51.6.3 Minerals from Spectra

Minerals absorb light at characteristic wavelengths. Olivine has a broad band near 1.05 µm, pyroxenes absorb near 0.9 and 1.9 µm, and clays and carbonates have narrow bands between 2.2 and 2.5 µm (Chapter 13). Imaging spectrometers record hundreds of wavelengths for every pixel. CRISM, on NASA's Mars Reconnaissance Orbiter, did this at about 18 m per pixel from 2006 until it was retired in April 2023. Its maps of carbonates in Jezero crater helped select the landing site of Perseverance (Figure 51.9).

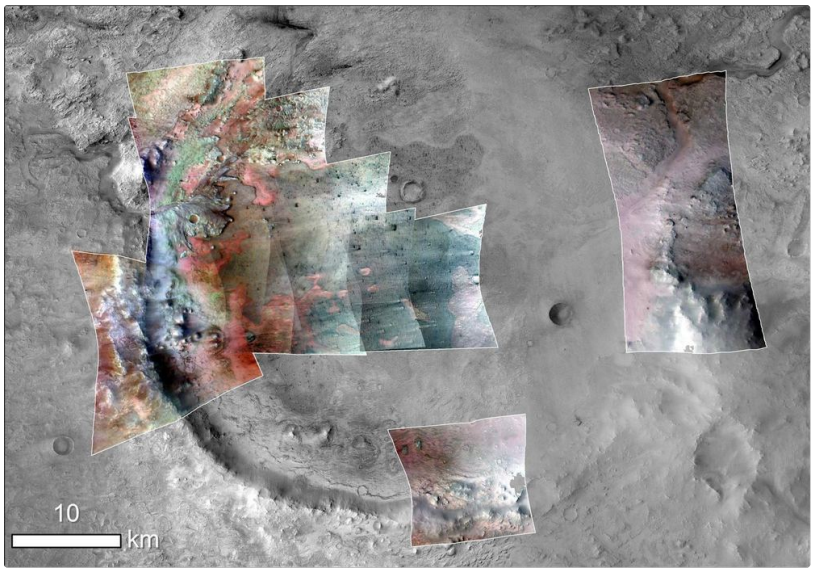



**FIGURE 51.9** Minerals in Jezero crater, mapped by the CRISM imaging spectrometer and the Context Camera on Mars Reconnaissance Orbiter. Green marks carbonates; red marks olivine-rich sand. Maps like this helped NASA choose Perseverance's landing site. Image: NASA/JPL (NASA/JPL-Caltech/MSSS/JHU-APL/Purdue/USGS).

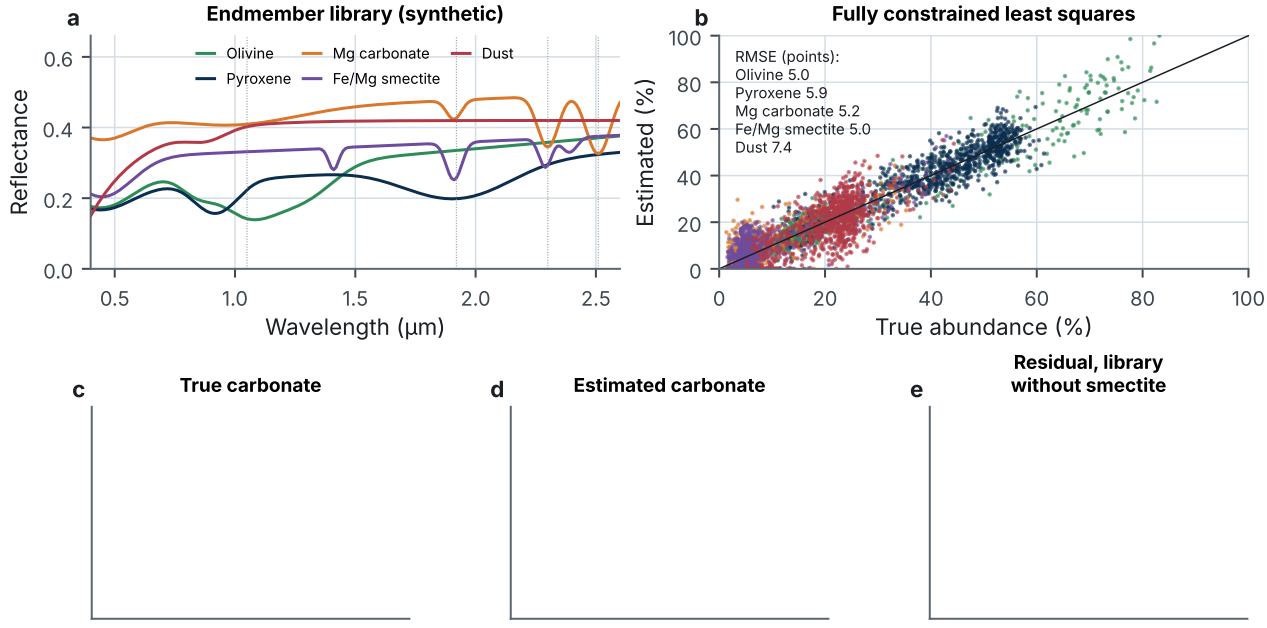
A pixel usually contains several minerals. In **linear spectral unmixing**, the spectrum $\mathbf{x}$ of a pixel is modelled as a weighted sum of pure "endmember" spectra, the columns of a matrix $\mathbf{E}$.

$$\mathbf{x} = \mathbf{E}\,\mathbf{a} + \mathbf{n}, \qquad a_k \geq 0, \quad \sum_k a_k = 1.$$

The abundances $\mathbf{a}$ are found by least squares with both constraints. This is called **fully constrained least squares (FCLS)**. When no spectral library is available, **non-negative matrix factorisation (NMF)** can estimate endmembers and abundances together, but its endmembers need not be pure minerals.

In Figure 51.10 both methods are applied to a synthetic scene of 3 456 pixels with 221 bands. It contains a carbonate- and clay-bearing fan, an olivine-rich channel, pyroxene and dust. The endmembers vary from pixel to pixel in band depth and brightness, and topographic shading changes the overall brightness. With the right library, FCLS recovered the abundances with

errors of 5.0–7.4 percentage points. Unconstrained least squares gave similar average errors, but 5.3 % of its abundances were negative, which is physically impossible. NMF found endmembers that differed from the true minerals by 8–19° in spectral angle.




Synthetic 48 × 72-pixel scene, 221 bands (0.40–2.60 µm), five endmembers with absorption bands at the positions of real minerals, linear (areal) mixing, noise SNR 150. FCLS = non-negative least squares with a sum-to-one row. Without the smectite spectrum, the residual map outlines where the library is incomplete. Seed 5154.

**FIGURE 51.10** Linear unmixing of a synthetic hyperspectral scene. (a) The five endmember spectra; dotted lines mark key absorption bands. (b) True against estimated abundances from fully constrained least squares. (c–d) True and estimated carbonate abundance. (e) Fit residual when the clay (smectite) spectrum is left out of the library: the error outlines where the clay is.

Panel (e) shows the most useful diagnostic. With smectite removed from the library, the fit residual rose sharply where smectite was present; its 99th percentile was 2.8 times the median. The residual map points to minerals the library lacks. In fine-grained real surfaces, light passes through several grains and mixing is non-linear. Networks trained on laboratory mixtures are an active research topic, but their results are still checked against band positions a spectroscopist can explain.

### 51.6.4 Landing Sites, Traverses and Resources

Before a landing, engineers must know where the ground is safe. JPL's **SPOC** (Soil Property and Object Classification) network classifies terrain in HiRISE images into types such as sand, bedrock and ridges. It was used to judge how easily a rover could drive at candidate landing sites for Perseverance. The **AI4Mars** project (2021) collected about 326 000 terrain labels on 35 000 rover images from volunteers, ten per image. A segmentation network trained on them reached more than 96 % accuracy on an expert-labelled test set (the labelling tool is shown in Chapter 44).

Resource prospecting combines these tools (Chapter 41). Maps of shadowed craters, boulders, slopes and temperature are merged to find ice that a rover could reach. The NASA–IBM Lunar Foundation Model (2026) is one example; Chapter 44 gives its results and limits.

### 51.6.5 Science Autonomy on Rovers and Probes

A rover or probe can also decide what to measure. The first steps were small. In 2006 the Mars Exploration Rovers received software that looked for dust devils and clouds in sequences of images. It first detected a dust devil on board Spirit in April 2007. On dust-devil sequences it cut the data volume by about 75 %, because only frames with moving features were sent home.

**AEGIS** (Chapter 48) picks rock targets for the laser spectrometers on Curiosity and Perseverance; by May 2026 it had chosen nearly 900. Perseverance's X-ray spectrometer **PIXL** goes further. Its hexapod mount places it close to a rock, and it scans a postage-stamp-sized patch with thousands of X-ray shots. Its **adaptive sampling** software watches the results. When it sees minerals of interest, such as carbonates or phosphates, it stops and collects data for longer (Figure 51.11). Each decision is made on Mars, without waiting for the ground.

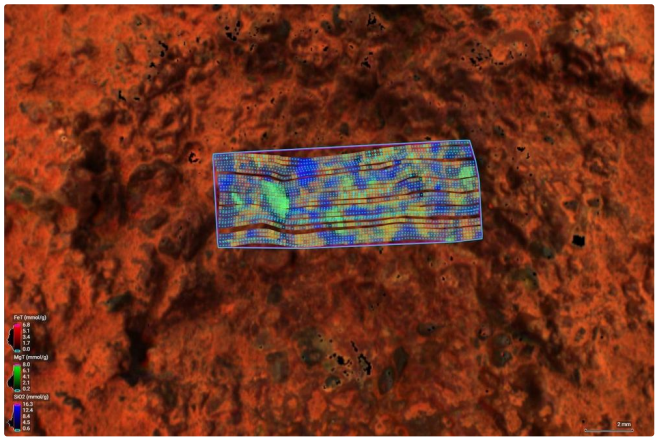


**FIGURE 51.11** A PIXL scan of the rock target "Thunderbolt Peak", recorded by Perseverance on 8 July 2024. Each dot marks a point hit by the X-ray beam; colours show the amounts of iron (red), magnesium (green) and silica (blue). PIXL's software can lengthen its measurements on board when it finds minerals of interest. Image: NASA/JPL (NASA/JPL-Caltech/DTU/QUT).

Future probes to icy moons may have to detect a plume or outburst and re-plan within hours. In sample curation (Chapter 29), learning could help find rare grains in images and keep records consistent between laboratories, without ever risking irreplaceable material. Onboard science autonomy is summarised in Table 51.4.

**TABLE 51.4** Onboard science autonomy on planetary missions.

| System | Mission | What it decides | Since | Status (Oct 2026) |
|---|---|---|---|---|
| Dust-devil and cloud detection | Spirit, Opportunity | Which frames to send home | 2006–07 | Missions ended |
| Autonomous Sciencecraft | EO-1 | Re-plans imaging after events (Chapter 48) | 2004 | Mission ended 2017 |
| AEGIS | Opportunity, Curiosity, Perseverance | Laser targets after a drive | 2010 | Operational |
| PIXL adaptive sampling | Perseverance | Where to dwell longer in a scan | 2021 | Operational |
| Onboard plume or outburst detection | Icy moons, comets | When to re-target | — | Research |

## 51.7 AI for Human Spaceflight

A crew far from Earth must become its own doctor, engineer and mission control. AI can help, but only if it works without a link to Earth and its advice can be checked.

### 51.7.1 Watching Crew Health

Wearable sensors measure the body continuously. Canada's **Bio-Monitor** has been used on the ISS since early 2019 (Figure 51.12). It is a stretchable shirt with a headband and a small recorder. It tracks heart activity, breathing, skin temperature, blood pressure and blood oxygen during work, exercise and sleep. On Artemis II in April 2026, all four astronauts wore wrist devices for the ARCHeR study. These logged movement, sleep and exposure to light around the Moon.

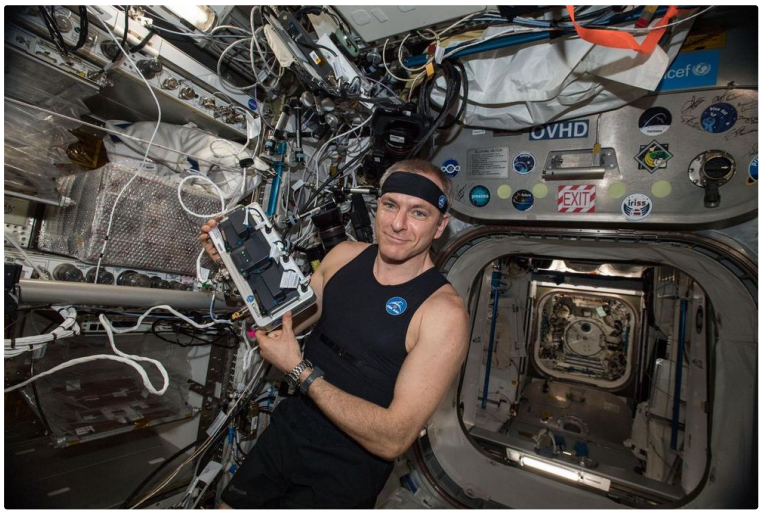



**FIGURE 51.12** Canadian astronaut David Saint-Jacques demonstrates the Bio-Monitor smart shirt on the ISS in January 2019. Its sensors record heart activity, breathing, blood pressure, blood oxygen and skin temperature. Image: NASA/JSC (David Saint-Jacques).

Such data are only useful if someone looks at them. Learning methods can build a **personal baseline** for each astronaut, the normal range of heart rate, temperature and sleep, and flag departures from it. A rising resting heart rate and temperature can precede the symptoms of an infection. The statistics are those of anomaly detection (Chapter 49), but a miss can cost a life.

### 51.7.2 Exploration Medicine

On the ISS, a sick astronaut can come home within hours. In January 2026, NASA's Crew-11 returned about a month early, the first time an ISS crew was brought home for medical reasons (Chapter 30). A Mars crew will have no such option for many months. Its medical kit, training and software must cover almost everything that can go wrong.

NASA plans medical kits with simulation. Its **Integrated Medical Model** runs Monte Carlo missions over about 100 medical conditions. Each simulated case uses drugs and supplies from the kit, and untreated conditions count as risk, so planners can choose the kit that lowers risk most for its mass. A newer tool, IMPACT, adds conditions that matter for exploration.

Decision support is the next step. In 2025 NASA and Google tested a prototype **Crew Medical Officer Digital Assistant**. It takes speech, text and images, works with little or no link to Earth and records each recommendation. Three physicians, one of them an astronaut, graded it with

the structured exams used for medical students. They rated its assessment and treatment plan 88 % likely to be correct for an ankle injury, 80 % for ear pain and 74 % for flank pain. NASA called these early results; the tool is not approved for use.

**Ultrasound** is the main imaging tool in space, but good images need skill. On the ISS, experts on the ground guide astronauts in real time, which will not work with a 40-minute round trip. ESA's ALISSE prototype (2024) uses a network trained on hundreds of thousands of labelled scans. It tells the user where to move the probe and how likely it is that the image shows the right organ. In tests on kidneys and bladders, nine out of ten images taken by untrained students were clinically acceptable.

**Behavioural health** may be the hardest part (Chapter 30). ARCHeR also studied stress, cognition and teamwork, and researchers are testing whether voice, sleep and activity patterns give early warning of fatigue or conflict. Such monitoring raises questions of privacy and trust, so crews must agree to it and control what is shared.

### 51.7.3 Assistants and Robots in the Cabin

Voice assistants that can read procedures, log work and answer questions have been tested for several years (Table 51.5). Language models now make them far more capable. Their main risk is the confident wrong answer, or **hallucination** (Chapter 44). For crew use, an assistant must quote its sources, such as the procedure, the step and the telemetry, so that the astronaut can check them.

LANDMARK: FROM CIMON TO LANGUAGE MODELS IN ORBIT (2018–2025)

- **CIMON** (Crew Interactive Mobile Companion), a floating sphere built by Airbus and IBM for the German space agency DLR, reached the ISS in July 2018. On 15 November 2018, in a 90-minute session, it recognised Alexander Gerst and talked with him. It returned to Earth in August 2019.
- **CIMON-2** arrived in December 2019, adding tone analysis to judge a speaker's mood. Its speech processing ran on ground servers.
- **Callisto** flew inside Orion on the uncrewed Artemis I mission in late 2022, testing an offline voice assistant and video calls (Figure 51.14).
- In **August 2024** a generative language model ran on HPE's Spaceborne Computer-2 on the ISS. The Space Llama project followed in April 2025, with an open-source model tuned for station science.
- In **July 2025** China delivered the **Wukong AI** model to its Tiangong station. Its onboard part helped crew members prepare for a spacewalk.
- Also in **July 2025**, in the ICHIBAN test, JAXA astronaut Takuya Onishi spoke to CIMON in the Columbus module. CIMON passed his commands to JAXA's Int-Ball2 camera drone in Kibo and showed its video (Figure 51.13). JAXA called it the first time that robots built by different organisations had worked together in orbit.

**FIGURE 51.13** JAXA astronaut Takuya Onishi sets up CIMON on the ISS on 18 July 2025, during the ICHIBAN tests in which it relayed his voice commands to the free-flying camera robot Int-Ball2. Image: NASA/JSC.

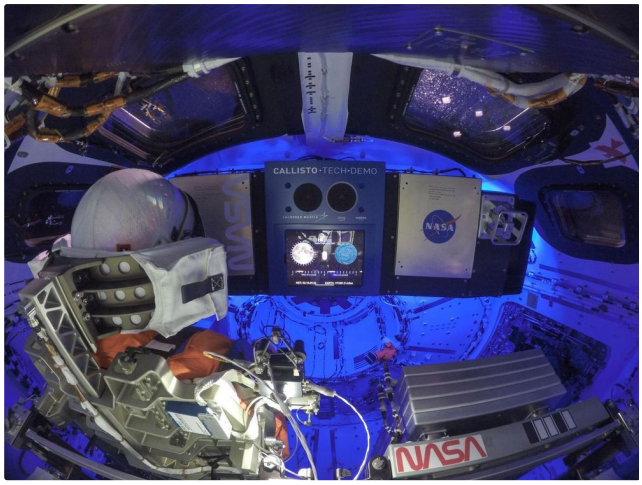



**FIGURE 51.14** The Callisto technology demonstration inside the Orion crew module on Artemis I, 18 November 2022. The payload tested an offline voice assistant and video calls for future crews. Image: NASA/JSC.

Robots are the physical side of the same idea. **Astrobee**, the free-flyer in the hero image, has tested autonomous inventory and inspection since 2019 (Chapter 48). JAXA's **Int-Ball2** films crew activities in Kibo. For a Mars habitat, the goal is a robot that keeps the station running while the crew sleeps or works outside, and before any crew arrives.

**TABLE 51.5** AI systems for crew support (status October 2026).

| System | Who, where | Function | Years | Status (Oct 2026) |
|---|---|---|---|---|
| Bio-Monitor | CSA, ISS | Wearable vital signs | 2019– | In use for research |
| ARCHeR wrist devices | NASA, Artemis II | Sleep, activity, light exposure | 2026 | Flown |
| CIMON, CIMON-2 | DLR, Airbus, IBM; ISS | Voice assistant, robot control | 2018– | Demonstrations; used in 2025 |
| Callisto | Lockheed Martin; Orion | Offline voice assistant | 2022 | Flown once (uncrewed) |
| Space Llama | Booz Allen, Meta, HPE; ISS | Language model on board | 2025– | Demonstration |
| Wukong AI | China; Tiangong | Language model, EVA preparation | 2025– | In use (reported) |
| Medical Officer Digital Assistant | NASA, Google | Diagnosis support | 2025 | Prototype, ground tests |
| ALISSE | ESA, GMV | Ultrasound guidance | 2024 | Prototype |
| Columbus ECLSS toolchain | Airbus, ESA | Anomaly detection, diagnosis | 2020– | Ground pilot |
| Astrobee, Int-Ball2 | NASA; JAXA | Free-flying robots | 2019–; 2023– | Operational test beds |

### 51.7.4 Smart Habitats and Predictive Maintenance

A habitat is a machine that keeps people alive (Chapters 30 and 32). Its fans, pumps, valves and filters wear out, and on a Mars mission every failed part must be fixed with a spare carried from Earth. Detecting wear early lets the crew plan repairs and can reduce the spares carried.

The European Columbus module on the ISS sends about 17 000 telemetry parameters. Its life-support system alone produces more than 1 000 signals sampled once a second. Since about 2020, Airbus has trained autoencoders on these data to learn normal behaviour. In 2025 researchers described a ground toolchain, integrated at Airbus, that detects anomalies, diagnoses their cause and recommends actions. In ESA's ORBIT-STAR project, Airbus built a demonstrator that reads Columbus sensor data and sends event messages to ground and crew. It was tested on a ground copy of the Columbus software; running it on board is the next step.

A synthetic test shows why learned models help (Figure 51.15). A fleet of 60 cabin fans ran for 1 100 days. Each wore out at a random time, and its vibration, current and temperature rose in its last weeks. But the fans also changed speed, and the cabin warmed and cooled. A fixed vibration limit therefore either alarmed whenever a fan ran fast, or alarmed late. **Gradient-boosted trees** — many small trees, each correcting the errors of the ones before — learned to predict "failure within 30 days" from the raw signals and their trends. They were trained on 40 fans and tested on 20 others. Allowing at most 0.5 false alarms per fan-year, the model gave a median warning of 29 days and warned of every failure. The fixed limit gave 15 days and missed 15 % of failures.

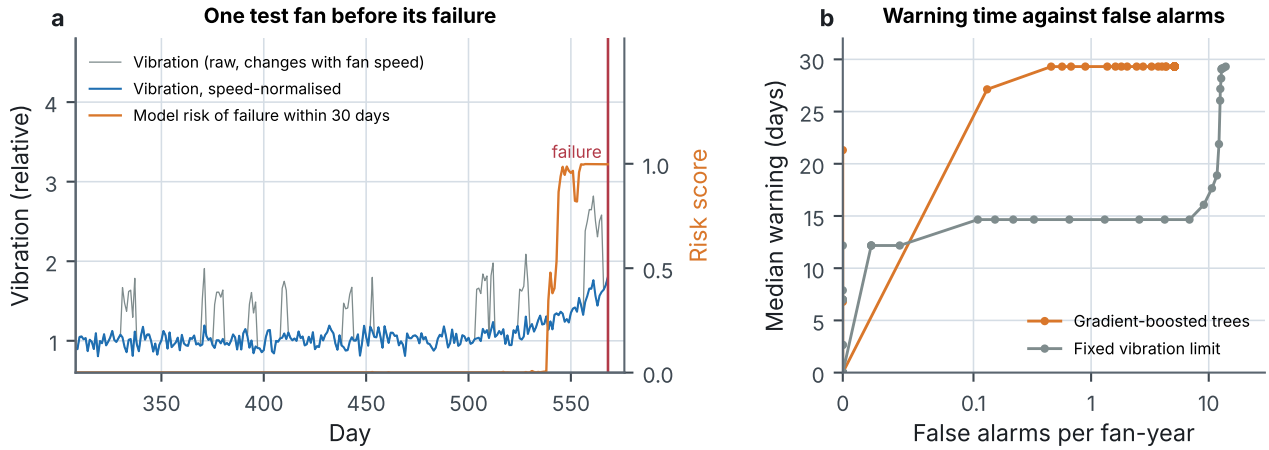
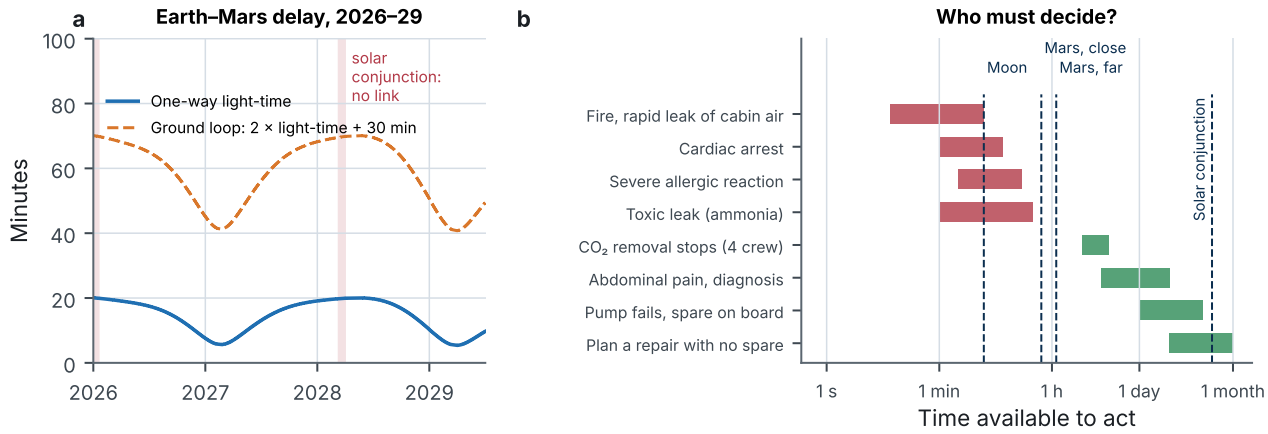



**FIGURE 51.15** Predictive maintenance of synthetic cabin fans. (a) One test fan before its failure: raw vibration rises whenever the fan runs fast; the speed-normalised signal and the model's risk score rise only as wear grows. (b) Median warning time against false alarms per fan-year, for gradient-boosted trees and a fixed vibration limit.

Real fleets are smaller and failures rarer, so models must share data across similar units and stations (Chapter 49).

### 51.7.5 Earth-Independent Operations

On the ISS, mission control answers within seconds. At Mars the one-way **light-time** — the time a signal takes to cross the distance $d$, $\tau = d/c$ — is 3 to 22 minutes. A ground answer takes at least $2\tau$ plus the time experts need to respond. For about two weeks near **solar conjunction**, when Mars is behind the Sun, there is no reliable link at all. In Figure 51.16 this is computed from the orbits of Earth and Mars for 2026–29. Mars passed behind the Sun on 9 January 2026 and will do so again around 20 March 2028, staying within 2° of it for about two and a half weeks (11–30 March 2028).




(a) Heliocentric Keplerian orbits of Earth and Mars (J2000 elements, coplanar approximation); shading where Mars is within 3° of the Sun. (b) Approximate times available before harm, compared with a ground loop of two light-times plus 30 minutes for experts to respond. Red: crew must act alone; green: ground advice can arrive in time, except during solar conjunction.

**FIGURE 51.16** Why a Mars crew must decide for itself. (a) One-way light-time between Earth and Mars in 2026–29 and the resulting ground loop, with solar conjunctions shaded. (b) Approximate time available to act in several emergencies and failures, compared with the ground loop for the Moon, for Mars near and far, and for solar conjunction.

NASA's analogue missions test this. The second one-year CHAPEA mission, with four volunteers in a 3-D-printed Mars habitat in Houston, ran from 19 October 2025 to its planned end on 31 October 2026, with simulated communication delays.

**EXAMPLE: AUTONOMY REQUIREMENTS FOR A MARS CREW WITH A 20-MINUTE DELAY**

Assume a one-way light-time of 20 minutes and 30 minutes for experts on Earth to respond.

1. **Ground loop.** $2 \times 20 + 30 =$ **70 minutes** for one question and answer. A diagnosis that needs three exchanges takes **3.5 hours**.
2. **Emergencies.** A fire, a fast air leak, cardiac arrest, a severe allergic reaction or a toxic leak must be handled within seconds to tens of minutes. The ground cannot help in time. Procedures, alarms and decision support must all be on board and work offline.
3. **Slower failures.** Four crew members exhale about 1.0 kg of $CO_2$ each per day. In 100 $m^3$ of air at 22 °C, the partial pressure rises by $\dot{m}RT/(MV) \approx$ **93 Pa per hour**. From 0.4 to 1.0 kPa takes **6.5 hours**, about 5.5 ground loops. The ground can advise, but onboard diagnosis should find the cause in the first hour.
4. **Conjunction.** For about 16–18 days no link is possible. A 900-day mission includes at least one conjunction. Any illness or failure in that window must be handled alone.
5. **Requirements.** The crew needs offline medical decision support, guided imaging, health monitoring with personal baselines, prognostics for critical spares, and a searchable record of every procedure. All must keep working if a computer fails, so simple manual procedures remain the backup.

The ground's role shifts from control to support. It reviews logs, updates models and plans the next days, while the crew and its software handle the present.

## 51.7.6 Augmented Reality for EVA and Maintenance

**Augmented reality (AR)** overlays instructions on what a person sees. In NASA's Project Sidekick, Microsoft HoloLens headsets reached the ISS in December 2015 (Figure 51.17). In "remote expert" mode, engineers on the ground saw what the astronaut saw and drew guidance into the view. In

"procedure" mode, the headset showed animated steps. With Mars delays, only the procedure mode is practical.

**FIGURE 51.17** NASA astronaut Scott Kelly wears a Microsoft HoloLens headset on the ISS on 20 February 2016, during Project Sidekick, a test of augmented reality for crew procedures. Image: NASA/JSC (Mikhail Kornienko).

Learning adds recognition: a headset that recognises a part, checks that the right bolt is being turned and records each step. For spacewalks (Chapter 33), helmet displays could show suit data, routes and procedures. Such systems must be verified like flight software, because a wrong overlay can be worse than none.

## 51.8 Validating an AI Discovery

A discovery made with machine learning must pass the same tests as any other, and some extra ones. A network's score is a ranking, not a probability that an object is real, until it is calibrated.

Three kinds of evidence matter. **Completeness** says how many real objects the method finds, measured by injecting fake signals into real data, as in the crater example. **Reliability** says how many of the claimed objects are real, measured with known false positives and with independent data. **Physical plausibility** asks whether the object makes sense, such as a transit shape that fits its star. Learned models can also exploit quirks of a data set, such as a camera artefact that happens to correlate with a class. This **leakage** (Chapter 43) is found only by careful checks.

Status needs the same care (Table 51.2, Table 51.4, Table 51.5). Most science results so far come from archives on the ground, with people checking the final list. Onboard science autonomy is operational only in narrow, well-tested forms, such as AEGIS and PIXL. AI for crew health and assistance is still at the stage of demonstrations and prototypes.

### PRACTICE: VALIDATING AN AI DISCOVERY BEFORE PUBLICATION

1. **Freeze the test set.** Set aside labelled data that the model never sees during development, split by object, region or time so that neighbours cannot leak between sets.
2. **Inject and recover.** Insert synthetic signals — transits, craters, lenses — into real data and run the full pipeline. Report completeness against signal strength and size.
3. **Beat a baseline.** Compare with the simplest established method, such as BLS with hand-made vetting, template matching or a $\chi^2$ fit. Report both on the same data.
4. **List the false-positive scenarios.** For each candidate, check every known impostor: eclipsing binaries, background stars, artefacts, shadows, unmodelled minerals. Use evidence the model did not see, such as centroids, spectra or other instruments.
5. **Calibrate the score.** Convert scores to probabilities with held-out data, then combine them with prior rates to estimate the chance that each object is a false positive.
6. **Explain and ablate.** Check which inputs drive a decision, and remove them one at a time. A planet score that depends on a detector edge is a warning.
7. **Confirm independently.** Seek follow-up with a different method or instrument, or ask an independent team to repeat the analysis.
8. **Publish everything needed to repeat it.** Release code, model weights, data versions, thresholds and the selection function, so that later studies can correct for what the method missed.

### KEY TAKEAWAYS

- Science data now outgrow human attention: Rubin can send up to about 7 million alerts a night, and Kepler's final search produced 34 032 signals. Machine learning acts as a filter and ranker in a discovery funnel; people make the final decision.
- Learned vetting finds planets that rules miss. AstroNet found Kepler-90i in 2017, and ExoMiner validated 301 Kepler planets. In this chapter's test, a small network kept 84.8 % of planets at 95 % precision, against 43.7 % for hand-made tests.
- Training sets must represent the data. Random-forest photometric redshifts trained only on bright, nearby galaxies had 19.4 % outliers instead of 12.0 %.
- Speed creates science: transients must be classified within days, and neutron-star mergers can now be characterised in about one second.
- Automated catalogues of craters, boulders and minerals now cover whole planets, but detectors must be calibrated. Ignoring completeness made a 50-million-year surface look 17 million years old.
- Onboard science autonomy is operational in narrow forms: AEGIS selects laser targets and PIXL lengthens its scans on Mars without waiting for Earth.
- A Mars crew faces a ground loop of over an hour and two weeks without contact at conjunction. Medical decision support, guided ultrasound, assistants and predictive maintenance are promising, but mostly prototypes in October 2026.

### REVIEW QUESTIONS

1. Explain the discovery funnel with Kepler's numbers. Why does a 7.1σ threshold still leave tens of thousands of false signals?
2. A planet gives a depth of 200 ppm and 3-hour transits. The noise is 300 ppm per 30-minute point. How many transits are needed for SNR 7.1?
3. In Figure 51.4, why does the neural network gain most at low signal-to-noise ratio? Which false positives remain hard, and how are they rejected in practice?

4. Rubin sends 7 million alerts in a 10-hour night. If a broker's filter has a 0.5 % false-positive rate for "young supernova", how many false candidates does it pass? Why should brokers report probabilities rather than labels?
5. Define $\sigma_{\mathrm{NMAD}}$ and the outlier fraction for photometric redshifts. Why did training on bright, nearby galaxies hurt faint galaxies most?
6. What does simulation-based inference avoid writing down? What is its main weakness?
7. Using the lunar chronology function, estimate $N_1$ for surfaces aged 0.05, 1 and 3.5 billion years. Why does the exponential term matter only for old surfaces?
8. Explain how completeness and false positives bias crater-count ages. How are they measured?
9. Write the linear mixing model with its constraints. How can the fit residual reveal a mineral missing from the library?
10. List the conditions that decide whether a Mars crew must act without help from Earth. Which AI tools would you fly first, and how would you verify them?

CHAPTER

# 52 AI in Spacecraft Design, Manufacturing, Launch and the Space Business

Image: NASA/MSFC (Fred Deaton)

*Long before a spacecraft flies, AI is at work in the engineering behind it. Algorithms now propose structures, search large design spaces with few simulations, check requirements, watch metal printers layer by layer and screen CT scans for flaws. On the launch range, learning helps with engine health, weather and scheduling. Flight-critical decisions stay with optimisation and fixed rules. In the business, models turn satellite data into market signals, price launch risk and size constellations. This chapter explains each method from first principles. Small experiments, run from scratch, measure what each method gains and where it fails. It closes with a safe way to bring AI tools into an engineering team. Status is given as of October 2026.*

IN THIS CHAPTER

52.1 AI Across the Life Cycle
52.2 Design: Letting Algorithms Propose Shapes
52.3 Exploring the Trade Space
52.4 AI for Systems Engineers
52.5 Manufacturing and Test
52.6 Launch: Where Learning Stays on the Ground
52.7 The Space Business
52.8 Bringing AI into an Engineering Team

A spacecraft is shaped by thousands of decisions taken years before launch. Each is made with incomplete information and a fixed budget. Artificial intelligence (AI) now helps with many of them. It does not replace the engineer. It proposes options, searches faster and checks more data than people can.

The hero image shows the starting point of much of this work: an engineer at NASA's Marshall Space Flight Center explaining rocket-engine hardware made by additive manufacturing (3-D printing). Printing frees the designer from the limits of machining. An algorithm can then draw shapes that no person would draw. Printing also creates new risks, such as hidden pores in the metal, which learned models help to find.

This chapter follows a spacecraft from design to the market. Systems engineering is covered in Chapter 4, structures in Chapter 5, launch vehicles in Chapters 34–36 and spaceports in Chapter 37. Learning methods are explained in Chapters 43 and 44, and computing hardware in Chapter 45. Trust, assurance and governance are the subject of Chapter 53.

## 52.1 AI Across the Life Cycle

The key idea of this chapter is simple. **AI proposes; physics, test and people verify.** A learned model can be wrong in ways that are hard to see. So nothing it produces enters the controlled design without passing the same checks as human work.

Figure 52.1 shows where AI enters the life cycle. Each stage has its own tools, but they share two features. First, every output passes a **verification gate**: a high-fidelity analysis, a test, an inspection or a review by a responsible engineer. Second, every stage writes data into a **digital thread** — one linked record of requirements, models, analyses, build records and test results.

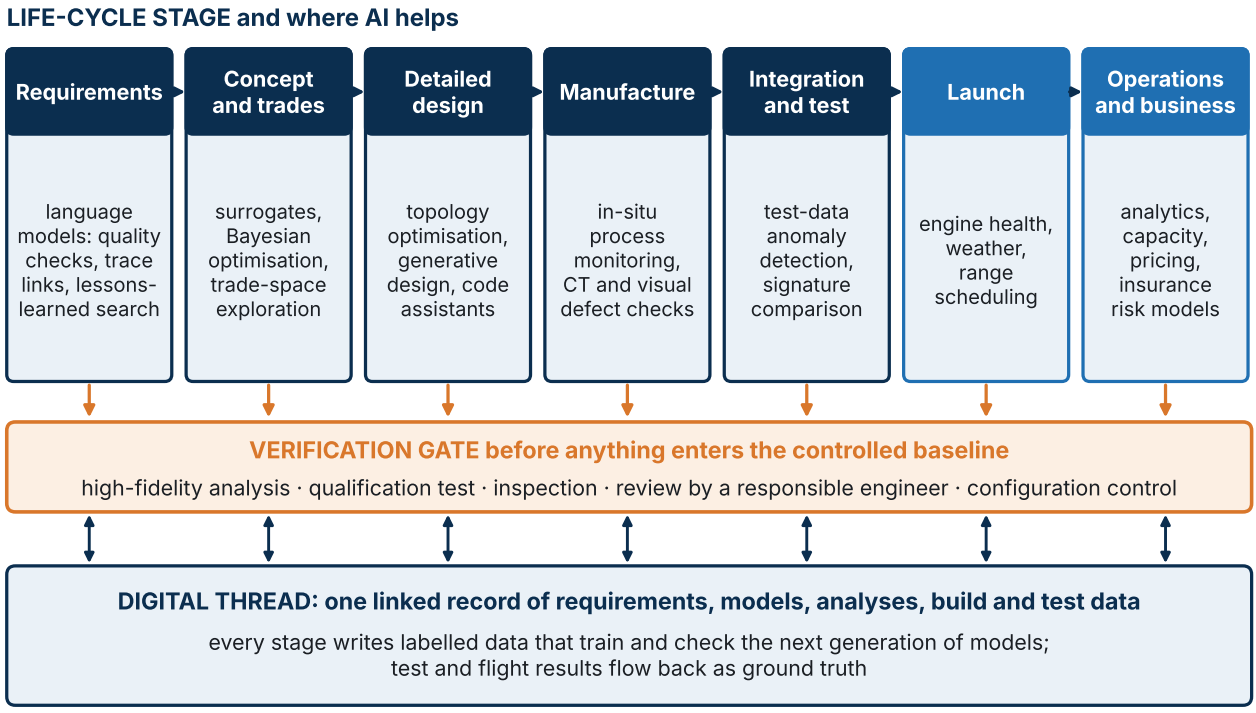



**FIGURE 52.1** Where AI helps in the life cycle of a spacecraft. Each stage has its own tools, but every output passes a verification gate before it enters the controlled design. A digital thread links all stages, so that test and flight results return as labelled data.

### 52.1.1 Where the Value Lies

Early choices fix most of the cost of a programme (Chapter 4), so improving the concept saves more than speeding up drawing. Yet early stages have the least data, and models there must work from physics and small samples. Later stages, such as inspection, have plenty of data but little freedom to change the design.

### 52.1.2 Operational, Demonstrated or Research?

Claims about AI range from routine use to press releases about prototypes. Table 52.1 sorts the main uses by status as of October 2026. "Operational" means used routinely on real programmes. "Demonstrated" means shown on real hardware or data but not yet routine. "Research" means published studies and prototypes.

**TABLE 52.1** Main uses of AI in design, manufacturing, launch and business, with an example and their status as of October 2026.

| Use | Main method | Example | Status (Oct 2026) |
|---|---|---|---|
| Structural brackets and mounts | Topology optimisation, generative design | NASA Goddard "evolved structures"; RUAG Sentinel-1 bracket | Operational |
| Antennas | Evolutionary search | NASA ST5 antennas (2006) | Flown |
| Whole engines from requirements | Rule-based computational models | LEAP 71 engines, 5–20 kN | Demonstrated (hot fire) |
| Fast simulation | Neural surrogates and operators | Commercial tools (PhysicsX, Neural Concept) | Early operational |
| Requirements and documents | Language models, retrieval | ESA SpaceQA, NASA drafts | Pilots and research |
| Printer monitoring | Image segmentation | ORNL Peregrine | Operational in labs and industry |
| CT and visual inspection | Classifiers | Factory inspection | Operational in industry |
| Engine health | Redlines; learned detectors on the ground | Shuttle AHMS (2006–11) | Redlines flown; learning on ground |
| Satellite-data analytics | Detection, regression | Retail, shipping, commodities | Operational |
| Insurance and pricing | Statistical risk models | Launch and in-orbit risk, parametric cover | Operational |

## 52.2 Design: Letting Algorithms Propose Shapes

The key idea of this section is that a computer can search shapes, not just check them. The engineer states the loads, the interfaces and the forbidden zones. The algorithm finds where material is needed.

### 52.2.1 Evolutionary Design

The first computer-designed hardware in space was a small wire antenna. NASA's Space Technology 5 (ST5) mission needed an X-band antenna with a wide beam for three small satellites. Around 2004 a team at NASA Ames used **evolutionary algorithms** to design it (Figure 52.2).

An evolutionary algorithm copies natural selection. It starts with a population of random designs, here wire shapes with a few bends. Each design is scored by a simulation; the score is called its **fitness**. The best designs are copied with small random changes (**mutation**) and mixed together (**crossover**). After many generations the population converges on good designs. The method needs no gradients, so it works with any simulator, but it may need tens of thousands of evaluations.

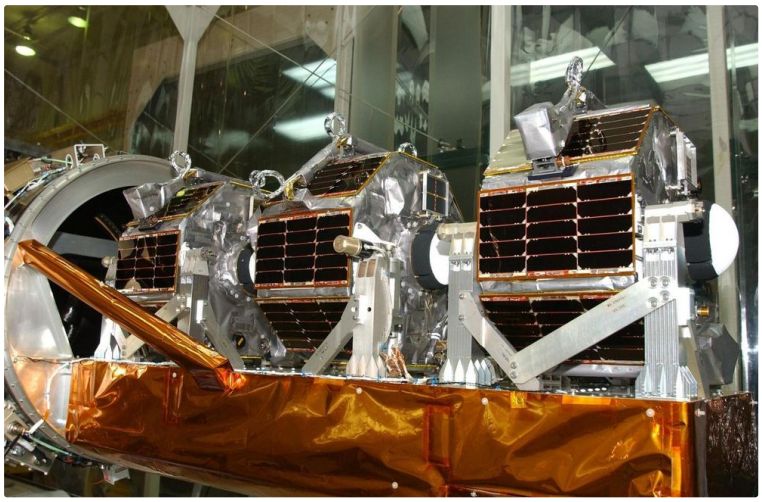


**FIGURE 52.2** The three ST5 microsatellites on their payload structure at Vandenberg Air Force Base in February 2006, a month before launch. Each carried an X-band antenna whose shape was found by an evolutionary algorithm. Image: NASA/KSC.

LANDMARK: THE EVOLVED ANTENNAS OF ST5 (2006)

- Jason Lohn of NASA Ames, Gregory Hornby of the University of California, Santa Cruz, and Derek Linden of JEM Engineering ran two evolutionary algorithms. One bent single wires; the other grew branching shapes.
- The winning antennas looked like bent paper clips. No human designer would have drawn them.
- Mid-project, a new launch vehicle put the mission into a much lower orbit, and the requirements changed. The team changed mainly the fitness function. Within a month of the change, a new antenna had been evolved and prototyped.
- On a spacecraft mock-up, two evolved antennas reached 93 % efficiency, against 38 % for two conventional quadrifilar helix antennas designed by a contractor. The evolved design also took about three person-months to design and build, against about five.
- The three satellites were launched on 22 March 2006 on a Pegasus XL. Each carried an evolved antenna, the first computer-evolved hardware in space.

### 52.2.2 Topology Optimisation

For structures, the most widely used method is **topology optimisation**. It decides where material should go inside a fixed design space. The space is divided into small finite elements. Each element $e$ gets a density $\rho_e$ between 0 (empty) and 1 (solid).

The popular **SIMP** method (solid isotropic material with penalisation) links the stiffness of an element to its density. The rule is

$$E(\rho_e) = E_{\min} + \rho_e^{\,p}\,(E_0 - E_{\min}), \qquad p \approx 3.$$

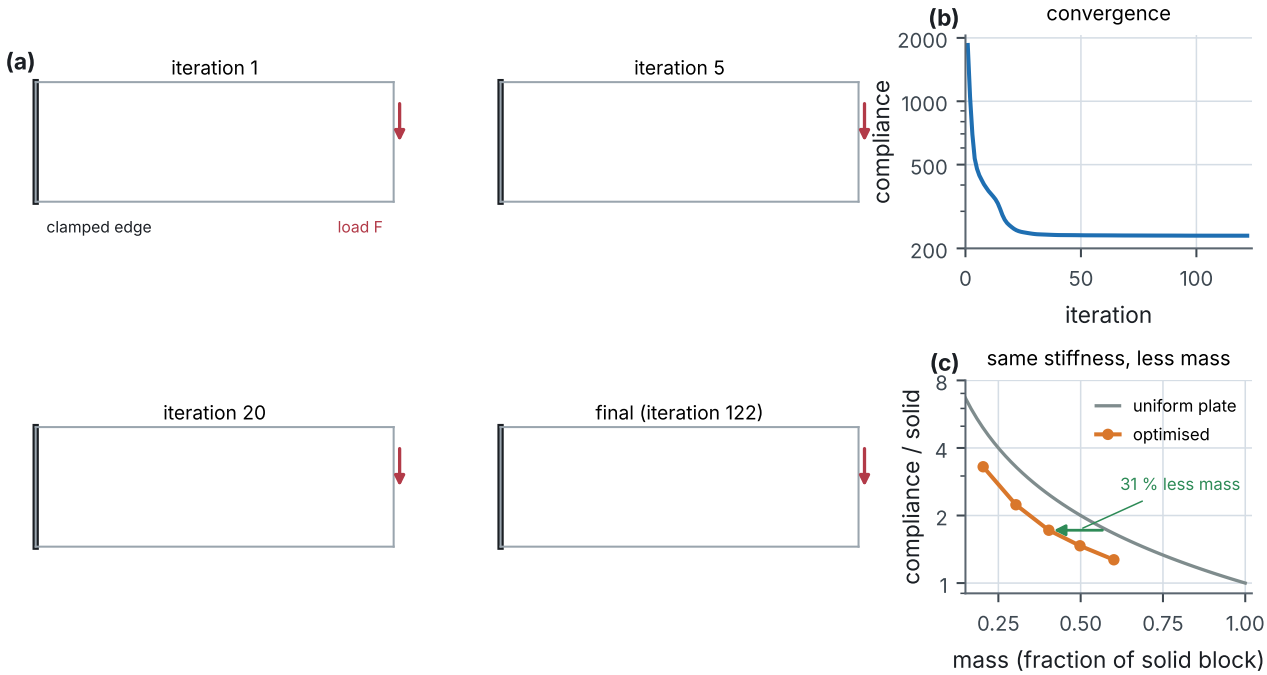
Here $E_0$ is the stiffness of solid material and $E_{\min}$ a tiny value that keeps the maths stable. The penalty $p$ makes half-dense elements poor value: they cost half the mass but give only an eighth of the stiffness. The optimiser therefore pushes elements towards 0 or 1.

The usual goal is the stiffest structure for a given mass. Stiffness is measured by its inverse, the **compliance** $c = \mathbf{F}^{\top}\mathbf{u}$: the work done by the load $\mathbf{F}$ through the displacements $\mathbf{u}$. The problem is

$$\min_{\rho} \; c(\rho) \quad \text{subject to} \quad \mathbf{K}(\rho)\,\mathbf{u} = \mathbf{F}, \qquad \sum_{e} \rho_e v_e \le V^{*},$$

where $\mathbf{K}$ is the stiffness matrix, $v_e$ the volume of an element and $V^{*}$ the allowed volume. The gradient has a simple form. For each element, $\partial c / \partial \rho_e = -p\,\rho_e^{p-1}(E_0 - E_{\min})\,\mathbf{u}_e^{\top}\mathbf{k}_0\mathbf{u}_e$, where $\mathbf{k}_0$ is the stiffness matrix of a solid element. Elements that store much strain energy gain material; the others lose it. A **filter** averages the densities over a small radius, which stops chequerboard patterns and sets a minimum member size.

The method is classical optimisation, not machine learning. It is still the engine under most "AI-designed" parts, so it belongs at the start of this chapter. Figure 52.3 shows it at work on a cantilever, clamped on the left and loaded at the right. The code is about a hundred lines of numpy. It follows the well-known "88-line" program published by Andreassen and colleagues in 2011.




SIMP method (penalty p = 3, density filter radius 1.6 elements, optimality-criteria update) on 120 × 40 four-node plane-stress elements, written in numpy/scipy after Andreassen et al. (2011). Compliance in (c) is recomputed for the black-and-white design (densities > 0.5 set to solid). The uniform plate is a plate of reduced thickness, whose stiffness scales with mass.

**FIGURE 52.3** Topology optimisation of a cantilever with 40 % of the material, run from scratch. (a) The density field after 1, 5, 20 and 122 iterations; a truss-like structure emerges. (b) The compliance falls by a factor of eight in the first 30 iterations. (c) For the same stiffness, optimised designs need 24–33 % less mass than a plate of uniform thickness.

The run took 122 iterations. Compliance fell from 1 851 to 230 units, and most of the gain came in the first 30 steps. The final design is a set of struts and ties, like a bridge truss.

How much mass does this save? Compare a plate of the same outline, thinned until it is as stiff as the optimised part. Rounded to solid or empty, the optimised design uses 40.3 % of the material of a solid block and is 1.72 times as compliant. An equally stiff uniform plate needs 58.2 %, so the saving is 30.6 % of the mass; across volume fractions from 0.2 to 0.6 it was 24–33 %.

**EXAMPLE: MASS SAVING FROM TOPOLOGY OPTIMISATION**

A small satellite carries 25 aluminium brackets machined as plates of uniform thickness, 12.0 kg in total. Each bracket is replaced by a topology-optimised part with the same stiffness, using the ratio from Figure 52.3.

1. **Mass ratio at equal stiffness.** The optimised design uses 40.33 % of the solid material; the equally stiff plate uses 58.15 %. The ratio is 0.4033 / 0.5815 = **0.694**.
2. **New mass.** 12.0 kg × 0.694 = **8.32 kg**. The saving is **3.68 kg** per satellite.
3. **Natural frequency.** The stiffness $k$ and the carried mass are unchanged, so $f = \frac{1}{2\pi}\sqrt{k/m}$ stays the same or rises slightly.
4. **Launch value.** At a reported rideshare price of about $7 000 per kilogram in 2026, 3.68 kg is worth about **$25 800** per satellite. For a batch of 100 satellites it is 368 kg.

The mass saving is real but modest per part. Its main value is often margin: kilograms freed for propellant, payload or growth late in the programme.

### 52.2.3 Generative Design in Practice

Commercial **generative design** tools wrap topology optimisation in a friendlier workflow. The engineer marks surfaces to keep, such as bolt holes, and zones to leave empty, such as an optical path. The tool returns many shapes, each checked for stress, frequency and how it can be made.

Industry uses such tools widely. RUAG Space, now Beyond Gravity, topology-optimised an antenna support bracket for Sentinel-1 and printed it in aluminium. It was half the mass of the original part and exceeded its stiffness requirement by more than 30 %; it was qualified for flight around 2016.

At NASA's Goddard Space Flight Center, research engineer Ryan McClelland has used such tools since about 2020 to design what he calls **evolved structures**. NASA reports that they can be up to two-thirds lighter than conventional parts, and that designs take an hour or two. They have been used on balloon telescopes, instruments and a mount for the Mars Sample Return programme, which lost its funding in 2026 (Chapter 29).

The best-documented case is a bracket for the **EXCITE** balloon telescope (Table 52.2 and Figure 52.4). The bracket holds a 1.35 kg tip/tilt mirror stage to the back of the telescope. It must survive 10 g vertically and 3 g sideways. Its first natural frequency must be above 100 Hz, and its target mass was 0.2 kg.

**TABLE 52.2** The EXCITE tip/tilt bracket: four human design iterations against generative designs that met the 0.2 kg target and the 100 Hz requirement. Data: R. McClelland, NASA Goddard (2023).

| Design | Time | Mass (kg) | 1st mode (Hz) | Stress (MPa) | Cost, lead time |
|---|---|---|---|---|---|
| Human, best of 4 | 2 days | 0.27 | 65 | 103 | No quote |
| Generative, milled | ≈ 1 hour | 0.20 | 147 | 14.8 | $1 000, 3 days |
| Generative, printed | ≈ 1 hour | 0.20 | 177 | 11.2 | $2 000, 3 weeks |

In four iterations, no human design that could be manufactured met the mass target. The one design that met both targets could be neither machined nor printed. The generative designs met both, with seven to nine times less peak stress. McClelland stresses that people must still review every result. Left alone, the algorithm can make members too thin to build or inspect.

**FIGURE 52.4** The EXCITE balloon-borne telescope hanging in a hangar at NASA's Columbia Scientific Balloon Facility in Fort Sumner, New Mexico, before its test flight on 31 August 2024. Several of its mounts were evolved structures designed with generative software. Image: NASA/GSFC.

### 52.2.4 Computational Engineering of Whole Systems

A rocket engine couples flow, combustion, heat transfer and structure. The Dubai company **LEAP 71** calls its approach **computational engineering**. Its software, Noyron, encodes engineering rules, physics and manufacturing limits as code, and generates a whole engine from a short specification. It is closer to an expert system than to a neural network: "AI-designed" can mean many things.

The results are measured by hot fire, not by claims. In June 2024 LEAP 71 fired a 5 kN kerosene–oxygen engine at Airborne Engineering in Westcott, UK. In December 2024 it fired a printed aerospike of the same thrust, and the measured pressures and temperatures were used to tune the model. In December 2025 the company reported two 20 kN methane–oxygen engines printed in a copper alloy. They went from specification to first firing in under three weeks. The bell-nozzle engine reached more than 93 % combustion efficiency. The aerospike reached its design chamber pressure of 50 bar, but start-up problems limited it to one firing. A 200 kN aerospike was printed in March 2026 for hot-fire tests later that year, and a 2 000 kN design has been shown.

### 52.2.5 Designing for the Printer

Printing changes what is cheap. In laser powder-bed fusion, a laser melts metal powder in layers about 30–60 µm thick. Complex internal channels cost little more than plain walls. But steep overhangs need supports, thin walls warp, and trapped powder and internal flaws must be found. Good design tools include these rules, so the optimiser proposes shapes that can be printed, cleaned and inspected.

## 52.3 Exploring the Trade Space

The key idea of this section is sample efficiency. A detailed simulation of a spacecraft may take hours. Learning methods help by choosing which simulations to run.

### 52.3.1 Multidisciplinary Design Optimisation

A spacecraft is a coupled system. A larger solar array gives more power but adds mass, drag and inertia. That changes the attitude control system, which changes power needs again. **Multidisciplinary design optimisation (MDO)** treats the whole design as one problem:

$$\min_{\mathbf{x}} f(\mathbf{x}, \mathbf{y}) \quad \text{subject to} \quad g_j(\mathbf{x}, \mathbf{y}) \le 0, \qquad \mathbf{y}_i = A_i(\mathbf{x}, \mathbf{y}_{j \neq i}).$$

Here $\mathbf{x}$ holds the design variables, such as orbit altitude. Each discipline $A_i$ — structures, thermal, power, orbit — computes outputs $\mathbf{y}_i$ that others need, and the constraints $g_j$ hold the requirements. Frameworks such as NASA's open-source OpenMDAO link the discipline codes and compute gradients. ESA's Concurrent Design Facility, open since 1998, does the same job with people and linked models.

When each evaluation is cheap, any optimiser works. When each takes hours, the number of runs becomes the limit. A surrogate model then stands in for the expensive analysis.

### 52.3.2 Surrogate Models

A **surrogate** (or emulator) is a fast approximation fitted to a modest number of expensive runs. Engineers have used polynomial "response surfaces" for decades. **Gaussian processes (GPs)** are now the standard choice for small data sets (Chapter 43). A GP predicts both a mean $\mu(\mathbf{x})$ and an uncertainty $\sigma(\mathbf{x})$. The uncertainty grows away from the runs already made. That is exactly what is needed to decide where to look next.

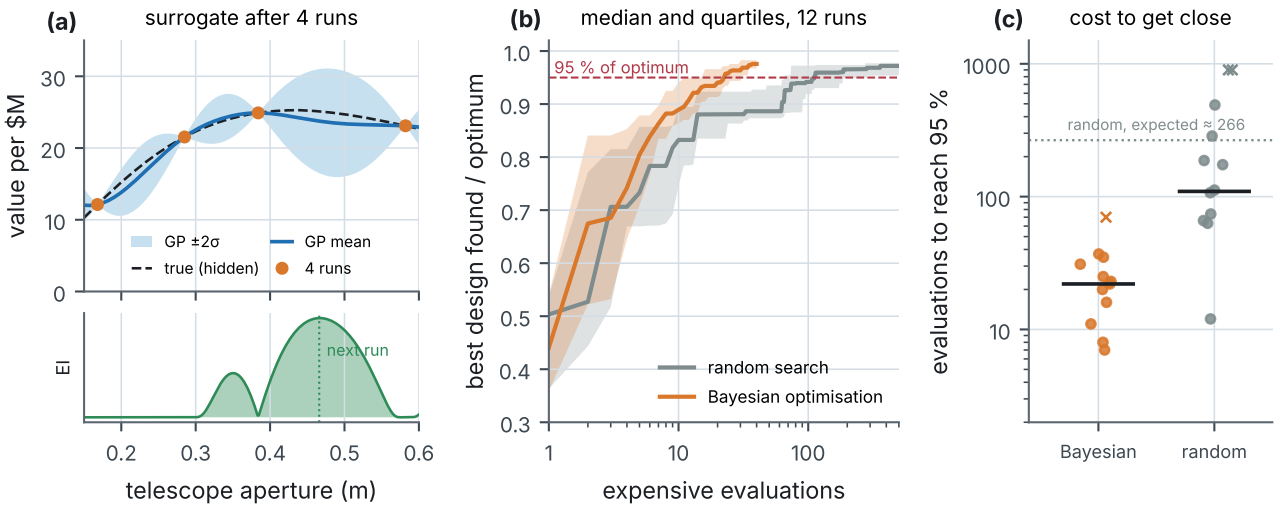
### 52.3.3 Bayesian Optimisation

**Bayesian optimisation (BO)** uses the GP to choose each new run. It scores every candidate design by an **acquisition function**. The most common is the **expected improvement** over the best value so far, $f^{*}$:

$$\mathrm{EI}(\mathbf{x}) = \left(\mu - f^{*}\right)\Phi(z) + \sigma\,\phi(z), \qquad z = \frac{\mu - f^{*}}{\sigma},$$

where $\Phi$ and $\phi$ are the cumulative and density functions of the standard normal distribution. The first term rewards designs that are predicted to be good. The second rewards designs that are uncertain and might be good. The run with the highest EI is made, the GP is refitted, and the loop repeats.

Figure 52.5 tests the method on a toy model of an Earth-observation smallsat. It stands in for an expensive simulation. There are four design variables: orbit altitude, telescope aperture, solar-array area and propellant mass. Lower orbits give sharper images but decay sooner unless propellant is carried. A larger aperture sharpens images but adds mass and heater power. The goal is image value per million dollars over the satellite's life. The best design lies at 428 km with a 0.44 m aperture, a 4.9 m$^2$ array and 16 kg of propellant.




Toy model of an Earth-observation smallsat standing in for an expensive simulation: four variables (altitude 400–800 km, aperture 0.15–0.6 m, array 1–6 m², propellant 0–60 kg); value = lifetime × images per day × value per image / cost. Gaussian process with a Matérn kernel (scikit-learn), expected-improvement (EI) acquisition, 6 Latin-hypercube starts, 40 evaluations; crosses = runs that did not reach 95 %. In (a) the other three variables are held at their optimum values.

**FIGURE 52.5** Bayesian optimisation against random search on a toy smallsat design model. (a) The Gaussian-process surrogate after four runs, along one variable, with its uncertainty band; expected improvement (EI) picks the next run where the model is both promising and uncertain. (b) Best design found against the number of expensive evaluations, over 12 independent runs. (c) Evaluations needed to come within 5 % of the optimum.

Only 0.38 % of random designs come within 5 % of the optimum. Random search therefore needs about 266 evaluations on average to find one. Bayesian optimisation needed a median of 22, and 11 of its 12 runs got there within 40. After 40 evaluations its median result was 97.6 % of the optimum, against 88.6 % for random search. Random search needed about 500 evaluations to reach 97 %.

### EXAMPLE: SIMULATIONS SAVED BY BAYESIAN OPTIMISATION

Suppose each evaluation in Figure 52.5 is a coupled thermal, structural and orbit simulation taking 8 hours on 256 processor cores.

1. **Runs to get within 5 % of the optimum.** Random search: about **266** on average. Bayesian optimisation: median **22**. The saving is 244 runs.
2. **Computer time.** 244 runs × 8 h = **1 952 hours**, or 81 days if run one after another.
3. **Core-hours.** 1 952 h × 256 cores ≈ **500 000 core-hours**. At an assumed $0.03 per core-hour, that is about **$15 000** of computing.
4. **Calendar time.** BO is sequential: 22 runs take 7.3 days. Random runs can be spread over many machines; with 10 in parallel, 266 runs take 8.9 days. Batch versions of BO choose several runs at once to close this gap.
5. **The real gain.** The computing cost is small. The calendar time and the engineer's attention are what matter, and BO gives a better design in the same time.

BO works best with up to about 20 variables and a smooth objective. With hundreds of variables, gradient-based MDO is usually better.

## 52.3.4 Neural Surrogates and Neural Operators

A GP becomes slow beyond a few thousand runs, because its cost grows with the cube of the number of points. For large simulation archives, **neural surrogates** take over. A neural network is trained on thousands of past simulations. It then predicts a new case in milliseconds. **Neural**

**operators**, such as the Fourier neural operator (Chapter 44), learn a mapping from one field to another. For example, they can map a wing shape and flow conditions to the whole pressure field around it.

Uses include aerodynamic databases for launchers and capsules, thermal maps of electronics boxes and fast stress checks inside optimisation loops. Investors have noticed: PhysicsX (London) raised $135 million in June 2025 and Neural Concept (Switzerland) $100 million in December 2025. Both sell learned simulation tools to aerospace and other industries.

Two cautions apply. First, a surrogate is only trustworthy inside the range of its training data; Chapter 44 shows how errors explode outside it. Second, the final design must be checked with the full, validated simulation and by test. A surrogate is a tool for search, not for certification.

## 52.4 AI for Systems Engineers

The key idea of this section is that language models are good at reading and drafting text, and poor at guaranteeing truth. They help most where a person checks every output.

### 52.4.1 Checking Requirements

A good requirement is necessary, clear, verifiable and states one thing (Chapter 4). Bad ones contain "weak" words such as *adequate*, *as appropriate* or *user-friendly*, open values such as "TBD", or two requirements joined by "and". Rule-based checkers have flagged such patterns since the 1990s. NASA's Automated Requirement Measurement tool, for example, counted weak phrases and options in specifications.

Language models go further. They can judge whether a requirement is testable, suggest a measurable rewrite and find conflicts between documents. Researchers at the University of Strathclyde trained **SpaceTransformers** (2021). These language models were further trained on space-systems text and tested on recognising concepts in requirements from the European ECSS standards. Domain training improved results over general models. Still, any rewrite is a suggestion. A wrong unit or a lost tolerance in a requirement can survive into hardware.

### 52.4.2 Searching Lessons Learned

Agencies hold decades of lessons in reports that few people read. **Retrieval-augmented generation (RAG)** combines a search engine with a language model (Chapter 44). The question is used to find relevant passages in a trusted archive, and the model answers from those passages with citations. Researchers working with ESA built **SpaceQA**, a question-answering system over reports from ESA's Concurrent Design Facility. NASA teams have proposed and prototyped similar tools for the NASA Lessons Learned system and for tracing requirements. One team building a NASA retrieval tool heard that some of its answers were wrong. On checking the sources, it found that the official guidance had changed, and the tool was right. The reverse happens too: a RAG system is only as current and as correct as its archive.

### 52.4.3 Models from Text

**Model-based systems engineering** keeps the design in a connected model rather than in documents (Chapter 4). Its language, SysML, gained a textual form in **SysML v2**, formally adopted by the Object Management Group in 2025. A textual language is something language models can write. Research teams have built systems that draft SysML v2 models from natural-language descriptions. They report that a language model alone often produces invalid syntax. Adding a grammar checker and retrieval of correct examples makes the drafts much better. These are research tools in 2026.

### 52.4.4 Code and Tests

Code assistants draft functions, unit tests and scripts. The evidence on productivity is mixed. In a controlled trial published by METR in July 2025, 16 experienced open-source developers completed 246 tasks. When AI tools were allowed, they took 19 % longer, although they believed they had been 20 % faster. Quality is also a concern. A 2022 study had a code assistant complete 89 security-relevant scenarios. About 40 % of the 1 689 programs it produced contained a vulnerability.

For flight software the rules do not change: NASA's NPR 7150.2 and ECSS-E-ST-40 still demand reviews, tests and traceability, whoever wrote the code. Assistants are most useful for ground tools, documentation and test cases. A person decides which tests are meaningful, and the tests themselves check the code.

### 52.4.5 Data Security and Export Control

Engineering data are often export-controlled under ITAR or EAR (US export-control rules) or commercially sensitive. Sending them to a public online service can break the law or a contract, so agencies and primes use approved, access-controlled deployments. NASA's digital-engineering team used commercial language models to draft internal procedures. This saved months of work, and every output was treated as a draft to verify.

Table 52.3 summarises the tools for systems engineers and the check that each one needs.

**TABLE 52.3** AI tools for systems engineers, the main risk of each and the check it needs.

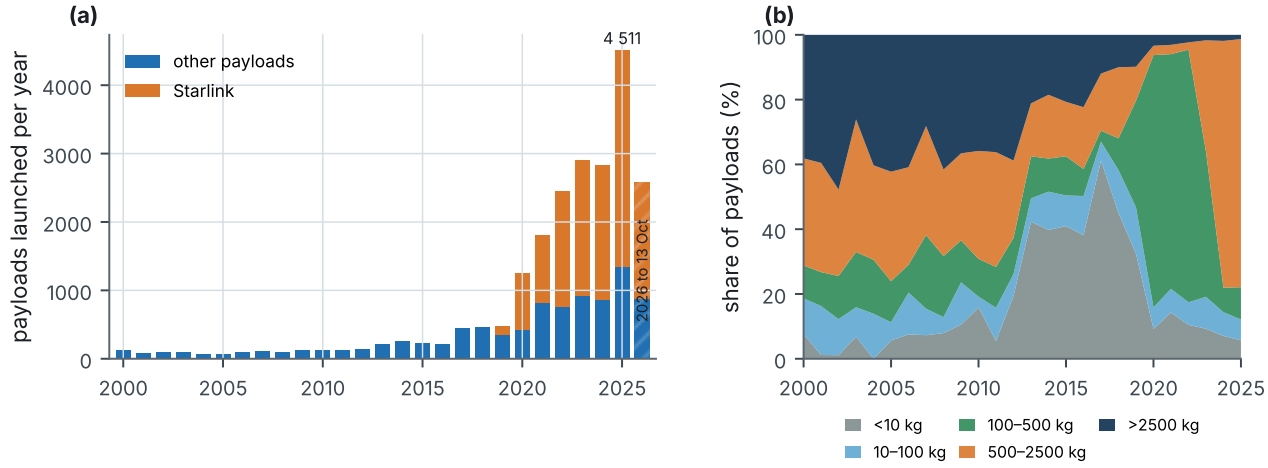
| Task | How AI helps | Main risk | Required check |
|---|---|---|---|
| Requirement quality | Flags vague, compound or untestable text; suggests rewrites | Changed meaning, lost tolerances | Owner approves every change |
| Lessons-learned search | Retrieves and summarises past reports with citations | Out-of-date or invented answers | Read the cited source |
| Traceability | Proposes links between requirements, design and tests | Missing or false links | Review by the systems engineer |
| SysML v2 drafting | Writes model text from descriptions | Invalid syntax, wrong semantics | Grammar check and model review |
| Code and tests | Drafts functions, test cases and scripts | Subtle bugs, security flaws | Standard review, static analysis, test |
| Report drafting | Turns data and notes into a draft | Plausible but wrong numbers | Check every number against its source |

# 52.5 Manufacturing and Test

The key idea of this section is that factories produce data at a scale people cannot inspect by eye. Learning sorts the data so that people look first at what most likely matters. The cost of a missed flaw sets how the sorting is tuned.

## 52.5.1 From One-Off to Mass Production

For most of the space age, satellites were built one at a time. That has changed (Figure 52.6). In 2025, 4 511 payloads were launched, nineteen times as many as in 2015. Starlink satellites made up 70 % of them. Most payloads launched in 2025 weighed 500–2 500 kg, because Starlink's current satellites fall in that class.




Data: GCAT (J. McDowell), payloads by launch year and mass class, updated 13 October 2026.

**FIGURE 52.6** The shift to mass production. (a) Payloads launched each year, with Starlink shown separately; 2026 is to 13 October. (b) The share of payloads by mass class: small satellites rose from 2013, then mass-produced constellation satellites of 500–2 500 kg took over.

Factories now build satellites at rates once reserved for aircraft parts (Table 52.4). SpaceX's filing for its 2026 share sale reported that its plant in Redmond, Washington, averaged about 70 Starlink satellites per week from December 2025 to April 2026. Amazon's factory in Kirkland, Washington, is designed to build up to five satellites per day at peak. Amazon says its test process there cut test time per satellite from months to days.

**TABLE 52.4** Satellite production rates reported for 2025–26.

| Producer | Satellites | Rate | Source and status |
|---|---|---|---|
| SpaceX, Redmond (USA) | Starlink | ≈ 70 per week (Dec 2025–Apr 2026) | SpaceX share-sale filing, 2026 |
| Amazon, Kirkland (USA) | Amazon Leo (formerly Kuiper) | Up to 5 per day at peak | Company, design capacity |
| ICEYE (Finland) | Radar satellites | ≈ 50 per year, 100 planned from 2027 | Company |
| Typical science mission | One spacecraft | One in 5–10 years | — |

At 70 satellites a week, no team can examine every weld, scan and test trace in detail. Statistical process control, automated inspection and anomaly detection become part of the production line. The same tools serve one-off science missions, where each part is precious and a missed flaw may end the mission.

### 52.5.2 Watching the Printer

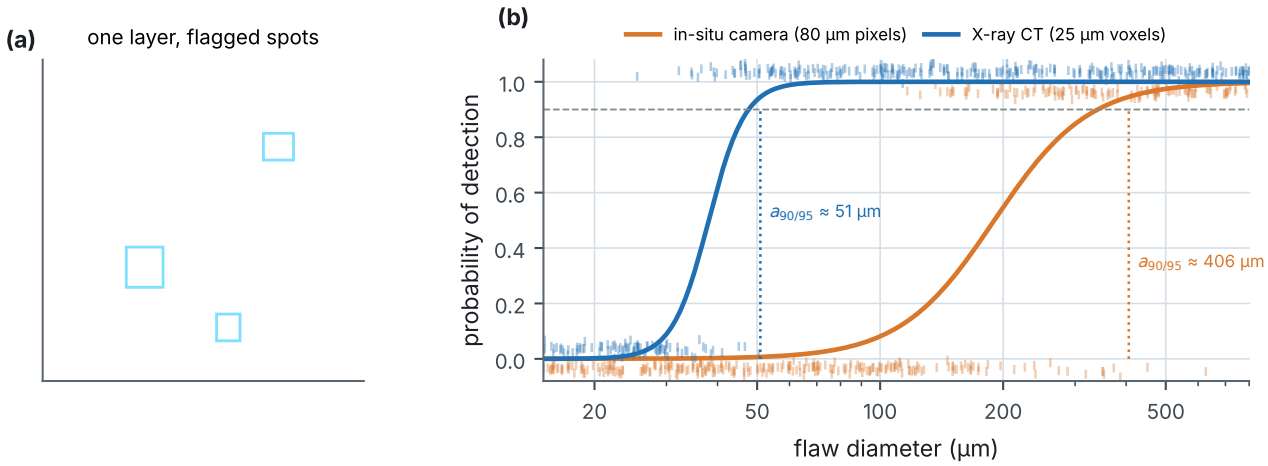
A metal printer builds a part from thousands of layers. Each layer is a chance for a flaw. **Lack-of-fusion** voids form where the laser did not melt enough powder. **Keyhole** pores form where it melted too deeply and trapped gas. Spatter, recoater streaks and warping cause other defects. Cameras, photodiodes and thermal sensors can watch every layer and every melt track. A build may produce tens of gigabytes of images.

Oak Ridge National Laboratory's **Peregrine** software is a leading example. It segments layer images with a convolutional network (Chapter 44) into classes such as powder, part, spatter and streaks, and raises alerts during the build. It was trained using X-ray computed tomography (CT) of finished parts as ground truth, and works with many makes of printer. By 2023, 15 US government laboratories used it, and 14 research licences had been granted to industry.

How small a flaw can such monitoring find? Non-destructive evaluation answers this with a **probability-of-detection (POD) curve**. Many flaws of known size are inspected, and each is recorded as a hit or a miss. A logistic curve in the logarithm of flaw size $a$ is fitted:

$$\mathrm{POD}(a) = \frac{1}{1 + \exp\!\left(-(\ln a - \mu)/\sigma\right)}.$$

The key number is $a_{90/95}$: the flaw size found 90 % of the time, with 95 % confidence. For fracture-critical parts, the structure must tolerate any flaw smaller than this size (Chapter 5).




Synthetic data. (a) Simulated melt-pool emission image of one 7.7 mm × 7.7 mm layer with scan-track texture and three dark flaws; boxes mark pixels more than 4σ below the local median. (b) 400 simulated flaws per sensor (ticks: hits at 1, misses at 0), contrast reduced by blur of one pixel, detection at 3.5σ; logistic hit/miss POD fit in log size (MIL-HDBK-1823A style), a90/95 = 95 % bootstrap bound on the size found 90 % of the time.

**FIGURE 52.7** Simulated in-situ monitoring of powder-bed fusion. (a) A melt-pool image of one layer; boxes mark spots darker than the local background. (b) Hit/miss probability-of-detection curves for an in-situ camera and for X-ray CT, with the 90/95 flaw size of each.

Figure 52.7 shows the idea with simulated data. A camera with 80 µm pixels blurs small flaws into the background. Its $a_{90/95}$ came out at about 406 µm. A CT scan with 25 µm voxels found flaws down to about 51 µm. These numbers are illustrative. Real values depend on the machine, the metal and the sensor, and must be measured for each process. The lesson holds, though. In-situ monitoring is a fast first screen and a record of the build. It does not replace CT for critical parts.

### 52.5.3 Seeing Inside: CT and Visual Inspection

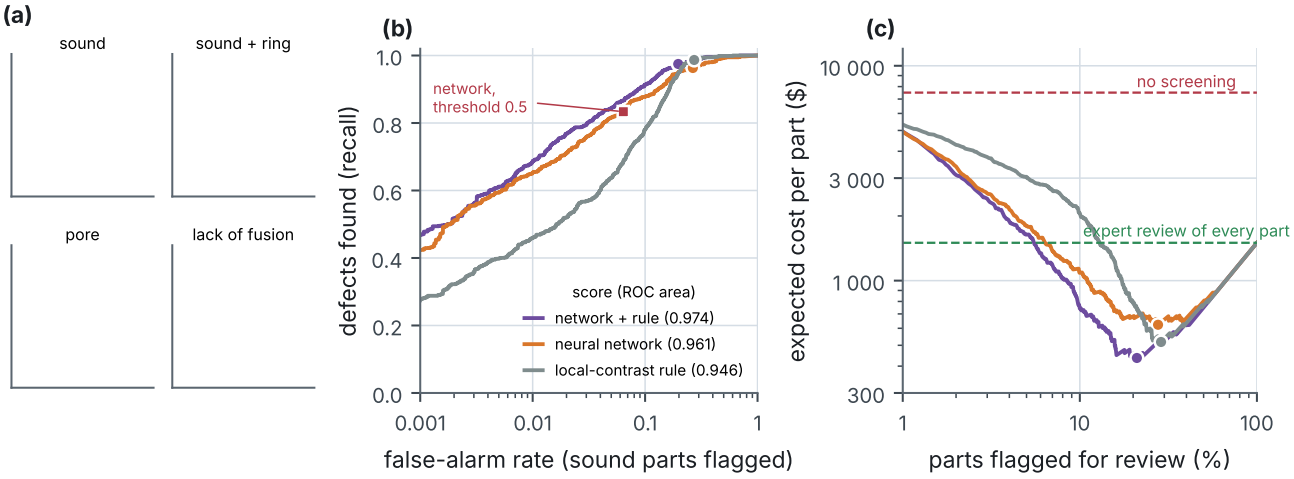
X-ray CT gives a 3-D density map of a part. A single scan holds billions of voxels, and an expert may need an hour to review it. A **classifier** can screen the scan and flag regions that look like pores or lack-of-fusion flaws. The expert then reviews only those regions.

Every screening model must choose a **threshold**. Above it a region is flagged; below it the part passes. The best threshold depends on what errors cost. Let a review cost $C_{\text{rev}}$ and a missed defect cost $C_{\text{esc}}$ (an "escape"). For a defect prevalence $\pi$, the expected cost per part is

$$\text{cost} = (\text{fraction flagged}) \times C_{\text{rev}} + \pi\,(1 - \text{recall})\,C_{\text{esc}},$$

where recall is the fraction of defects caught. If the model's probabilities are well calibrated, decision theory gives a simple rule: flag a part when $P(\text{defect}) > C_{\text{rev}}/(C_{\text{rev}} + C_{\text{esc}})$. With a review costing \$1 500 and an escape \$250 000, the optimal threshold is about 0.6 %, not the 50 % that software uses by default.

Figure 52.8 tests this on simulated CT patches. The patches contain pores and lack-of-fusion flaws. They also contain things that look like flaws but are not: ring and streak artefacts of the scanner, part edges and dense inclusions. Three screens were compared. The first is a classic **local-contrast rule**: smooth the image and flag the darkest spot relative to its surroundings. The second is a small neural network that sees the raw pixels. The third combines the two scores with a logistic regression.




Synthetic 24 × 24-pixel CT patches with build texture, beam hardening, part edges, ring and streak artefacts and noise; defects are pores and lack-of-fusion flaws. Network: two hidden layers (128, 32), scikit-learn, trained on 20 000 patches (30 % defective); tests on 24 000 patches at 3 % prevalence; the combined score is a logistic regression on both outputs. Costs: review \$1 500 per flagged part, escape \$250 000 per missed defect; dots = minimum-cost threshold chosen on a separate validation set.

**FIGURE 52.8** Screening synthetic X-ray CT patches. (a) Examples: sound material, a ring artefact, a pore and a lack-of-fusion flaw. (b) ROC curves of three screens; dots mark the minimum-cost operating points, and the square the network at its default threshold of 0.5. (c) Expected cost per part against the fraction of parts flagged; both baselines are far more expensive.

The combined screen had the largest area under the ROC curve, 0.974, against 0.961 for the network and 0.946 for the rule. But the ranking by cost was closer than the ranking by ROC area. At the lowest-cost threshold, chosen on a separate validation set, the combined screen cost \$521 per part. The rule cost \$543 and the network alone \$709. The network at its default threshold of 0.5 missed one defect in six and cost \$1 381 per part. A well-designed physical rule is a strong baseline; a learned model must beat it where it matters.

**EXAMPLE: COST–BENEFIT OF AN INSPECTION MODEL**

A factory prints 2 000 flight parts a year. About 3 % contain a defect. Reviewing a part's CT scan costs $1 500; an escaped defect costs $250 000 on average, through later test failures, rework and mission risk. Use the results in Figure 52.8.

1. **No screening.** 2 000 × 0.03 × $250 000 = **$15.0 million** a year of expected escape cost.
2. **Expert review of every part.** 2 000 × $1 500 = **$3.0 million** a year, assuming the experts miss nothing.
3. **Combined screen at its best threshold.** It flags 22.2 % of parts (444 reviews) and catches 97.5 % of defects. The cost is 2 000 × $521 = **$1.04 million**, a saving of **$1.96 million** a year against full review.
4. **Escapes.** 2 000 × 0.03 × (1 − 0.975) = **1.5 defective parts** a year still escape. With the network at its default threshold, about **10** would escape.
5. **Sensitivity.** If escapes cost ten times more, the best threshold moves lower and more parts are reviewed. If the model is retrained or the printer changes, the threshold must be checked again.

The saving comes from choosing the threshold with real costs, not from a higher ROC area. For fracture-critical parts, the residual escapes must also be covered by design margins and proof tests.

### 52.5.4 Composites and Electronics

Carbon-fibre parts are checked by **ultrasonic C-scans**, which map echoes from inside the laminate, and by **thermography**, which watches heat spread after a flash. Classifiers learn to flag the patterns of delaminations, porosity and wrinkles. Electronics boards pass through **automated optical inspection (AOI)**, and X-ray machines check hidden solder joints. Deep learning has cut the false calls that used to send many good boards for manual review. Throughout, the threshold is tuned to the cost of escapes, and each model is rechecked on new production lots.

### 52.5.5 Test Data: Vibration and Thermal Vacuum

Every flight unit goes through environmental tests (Chapter 4): vibration, acoustic, shock and thermal vacuum (Figure 52.9). A vibration test of a spacecraft may record hundreds of accelerometer channels. Before and after each full-level run, a **low-level sine sweep** measures the natural frequencies. If a main frequency shifts by more than a few per cent, something may have loosened or cracked. Engineers compare these "signatures" by eye, channel by channel.

Anomaly detection can make this comparison for every channel and flag the few that changed (Chapter 49 explains the methods). In thermal vacuum tests, models of expected temperatures flag a heater that switches too often or a sensor that drifts. Real test failures are rare, so most methods learn what "normal" looks like and flag departures. The test conductor makes the final call.

### 52.5.6 Supply-Chain Risk

A spacecraft contains parts from hundreds of suppliers, some with a single source. **Supply-chain risk analytics** combine purchase records, supplier finances, news, export rules and shipping data. They flag parts at risk of delay, counterfeiting or obsolescence, as a ranked list for buyers and engineers to act on.

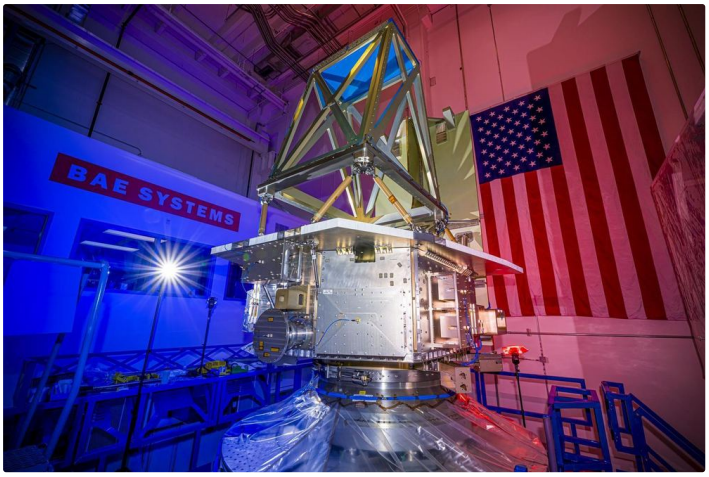



**FIGURE 52.9** The bus structure of NASA's NEO Surveyor on a shaker table at BAE Systems in Boulder, Colorado, during vibration testing in August 2025, with mass simulators standing in for the telescope and sunshade. Image: NASA/JPL (BAE Systems Space & Mission Systems).

## 52.6 Launch: Where Learning Stays on the Ground

The key idea of this section is that the decisions made on board a rocket in flight use optimisation and fixed rules that can be verified. Learning helps on the ground: in tests, forecasts, schedules and analysis.

### 52.6.1 Why Landing Guidance Is Optimisation

A booster must land on a small target at zero speed, within thrust limits. Chapter 48 shows how **convex optimisation** solves this problem. A convex problem has a single optimum, and modern solvers find it in a bounded number of steps. That gives three guarantees that a learned controller lacks. If a solution exists, it will be found. Every constraint will hold. The time needed can be bounded. Masten's tests of the G-FOLD algorithm in 2012–13 used this approach, and SpaceX engineers have described using it to land Falcon 9 boosters (Figure 52.10).

A neural-network policy might land well in simulation, but it offers no such guarantees outside its training conditions (Chapter 48). Learning can still help by giving the solver a good starting guess, which speeds it up while the solver keeps every constraint.

**FIGURE 52.10** A Falcon 9 first stage landing at Landing Zone 40, Cape Canaveral Space Force Station, on 1 October 2026, after launching NASA's SpaceX Crew-13 mission. SpaceX engineers have described its landing guidance as convex optimisation solved in real time. Image: NASA/KSC (NASA/John Kraus).

### 52.6.2 Engine and Vehicle Health

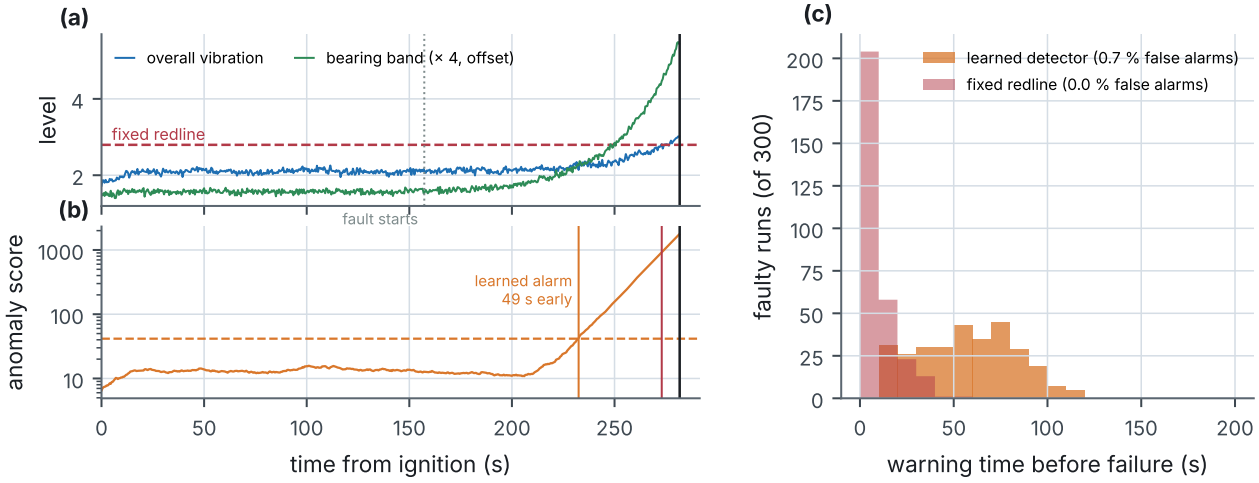
A turbopump that fails can destroy an engine in less than a second. **Redlines** — fixed limits on pressure, temperature or vibration — shut the engine down if any limit is crossed. They are simple, fast and verifiable. But a fixed limit must allow for every normal operating condition, so it is often set high and trips late.

A learned detector can find faults earlier by modelling how each sensor *should* behave at the current operating point. The residual — measured minus predicted — is small in normal running. Several residuals $\mathbf{e}$ are combined by the **Mahalanobis distance**,

$$d^2 = \mathbf{e}^\top \boldsymbol{\Sigma}^{-1} \mathbf{e},$$

where $\boldsymbol{\Sigma}$ is the covariance of the residuals in normal runs. This distance weights each residual by its usual spread and allows for correlations between channels.

Figure 52.11 compares the two approaches on 300 simulated hot fires with a growing bearing fault. The engine throttles between 80 % and 109 % of rated power. The fixed redline, set just above the highest normal vibration, gave a median warning of only 5 seconds; 19 % of failures came with no warning at all. The learned detector gave a median warning of 58 seconds, with false alarms in 0.7 % of normal runs. Earlier warning in test means less damage to expensive test hardware.




Synthetic 500 s hot fires with throttle changes between 80 % and 109 % power level; six channels (two synchronous vibration levels, bearing-band and broadband vibration, temperature, pressure). Learned detector: quadratic model of each channel against power level fitted on 40 normal runs, Mahalanobis distance of the residuals, smoothed (EWMA), threshold set on 200 validation runs. Black line: failure. Faults: bearing wear growing exponentially (time constant 8–50 s); 8 % of runs contain a brief sensor glitch.

**FIGURE 52.11** Health monitoring of simulated turbopump hot fires. (a) One run with a growing bearing fault: overall vibration stays under the fixed redline almost until failure. (b) The learned anomaly score crosses its threshold much earlier. (c) Warning time before failure over 300 faulty runs: the learned detector warns about a minute early, the redline only seconds early or not at all.

Why not fly the learned detector? A false shutdown in flight can lose a mission, and a learned model can fail on conditions it never saw. Learned detectors suit ground tests, such as the RS-25 firings in Figure 52.12, and analysis after flight. On board they are at most advisory, unless they meet the assurance standards of Chapter 53.

**LANDMARK: THE SHUTTLE'S ADVANCED HEALTH MANAGEMENT SYSTEM (2006–11)**

- The Space Shuttle main engine (SSME, also called RS-25) ran its high-pressure turbopumps at tens of thousands of revolutions per minute. A failed bearing or blade could destroy the engine.
- The Advanced Health Management System upgraded the engine controller with digital signal processors and new software. It watched turbopump vibration in real time against a **vibration redline**, and could shut down an engine if vibration grew too high.
- It first flew in monitor-only mode on STS-116 in December 2006 and was then made active.
- The logic was deterministic: fixed signal processing and limits, verified like any flight software.

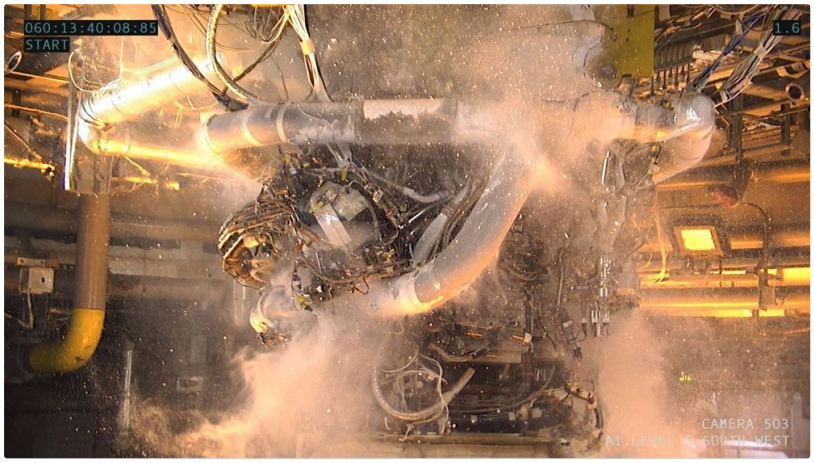



**FIGURE 52.12** A full-duration hot fire of an RS-25 engine on the Fred Haise Test Stand at NASA's Stennis Space Center on 29 February 2024, during certification of new production engines for SLS. Each test records hundreds of sensor channels. Image: NASA/SSC (NASA Stennis).

### 52.6.3 Weather, Lightning and the Range

Weather causes about half of all scrubs at Cape Canaveral (Chapter 37), and lightning rules are the hardest to meet. A model from NASA Marshall and the University of Alabama in Huntsville detected 84 % of lightning 15–30 minutes ahead, with a 22 % false-alarm rate. A study at the Air Force Institute of Technology trained models that could automate the 45th Weather Squadron's daily lightning forecast (Chapter 47). The launch-commit criteria themselves remain fixed rules.

Busy ranges also need scheduling. Florida's Space Coast hosted a record 109 launches in 2025 and planned for 100–120 in 2026. Each launch needs a pad, a window, hazard areas at sea and in the air, tracking assets and recovery ships. Choosing a conflict-free timetable is an optimisation problem. Learning helps predict the inputs, such as the chance of a scrub or of a ship entering a hazard area. Launch teams rehearse these decisions in simulations (Figure 52.13).

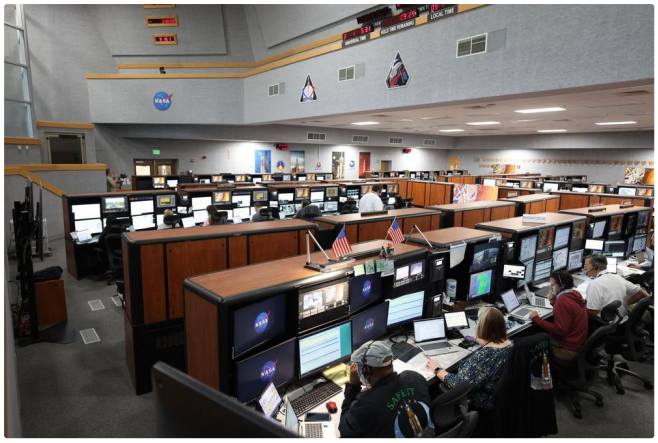


**FIGURE 52.13** The Artemis launch team in Firing Room 1 of the Launch Control Center at Kennedy Space Center during an Artemis II countdown simulation on 8 October 2025. Simulations like this test software, procedures and people against realistic failures. Image: NASA/KSC (NASA/Glenn Benson).

### 52.6.4 Flight Safety: Deterministic by Design

An **autonomous flight safety system (AFSS)** decides on board whether to end a flight that strays (Chapter 34). It uses the vehicle's own navigation and fixed rules, such as "terminate if the impact point crosses this line". Its first operational use was on Falcon 9 in February 2017. Here determinism is the point. Every rule can be reviewed, every path tested and every decision explained. Learning enters only upstream, for example in better models of falling debris used to draw the safety lines.

### 52.6.5 Trajectories and Mission Design

Trajectory optimisation relies on mature numerical methods (Chapter 24). Learning helps by screening: surrogates estimate the propellant for thousands of candidate transfers in seconds, and networks give the solver starting guesses. The final trajectory is always computed and checked with the full physics.

Table 52.5 summarises the division of labour.

**TABLE 52.5** Who decides what during a launch: verifiable logic on board, learning on the ground.

| Function | On board in flight | Role of learning |
|---|---|---|
| Landing and ascent guidance | Convex or classical optimisation | Starting guesses; offline design |
| Engine protection | Fixed redlines and logic | Ground-test anomaly detection; setting limits |
| Flight termination | Rule-based AFSS | Debris and risk models used to set rules |
| Launch weather | None (ground decision) | Lightning, wind and cloud forecasts |
| Range scheduling | None (ground decision) | Predicting scrubs, conflicts and delays |
| Trajectory design | Stored or optimised trajectory | Surrogates, warm starts, screening |

## 52.7 The Space Business

The key idea of this section is that the space business sells answers and manages risk. Learning turns satellite data into decisions. Statistics prices the risk of launch and operations. Optimisation sizes and prices the capacity of constellations.

### 52.7.1 Selling Answers from Orbit

Most commercial value lies not in pixels but in what they reveal (Chapter 46). Analytics companies count cars, ships, aircraft and containers. They estimate crop yields, measure oil in storage tanks from the shadows of their floating roofs, and track ships. Customers include traders, insurers, governments and supply-chain managers.

Satellite-tracked shipping became big business. In November 2024 the analytics company Kpler agreed to buy Spire's maritime business, which tracks ships from orbit, for about $241 million. The deal closed in 2025. The best-studied use is in finance (see the box).

> **DATA BOX: SATELLITE CAR-PARK COUNTS AND SHARE PRICES**
>
> A study published in 2025 in the *Journal of Financial and Quantitative Analysis* tested whether satellite data give investors an edge.
>
> - **Data:** about 4.7 million daily counts of cars in the car parks of 67 078 stores of 44 major US retailers, 2011–2017. The counts came from RS Metrics and Orbital Insight, which used image analysis to count cars.
> - **Signal:** an unusual change in how full the car parks were during a quarter predicted the retailer's earnings.
> - **Returns:** in the three days around earnings announcements, retailers with fuller car parks beat the market by 1.66 %. Those with emptier car parks fell behind it by 3.10 %. The gap was 4.76 %.
> - **Who gained:** mostly sophisticated investors who bought the data, above all short sellers betting on bad news. Buying by individual investors became less well informed, and shares became less liquid around the reports.
> - **Lesson:** satellite analytics carry real information. But unequal access to data can widen the gap between large and small investors rather than making prices fairer.

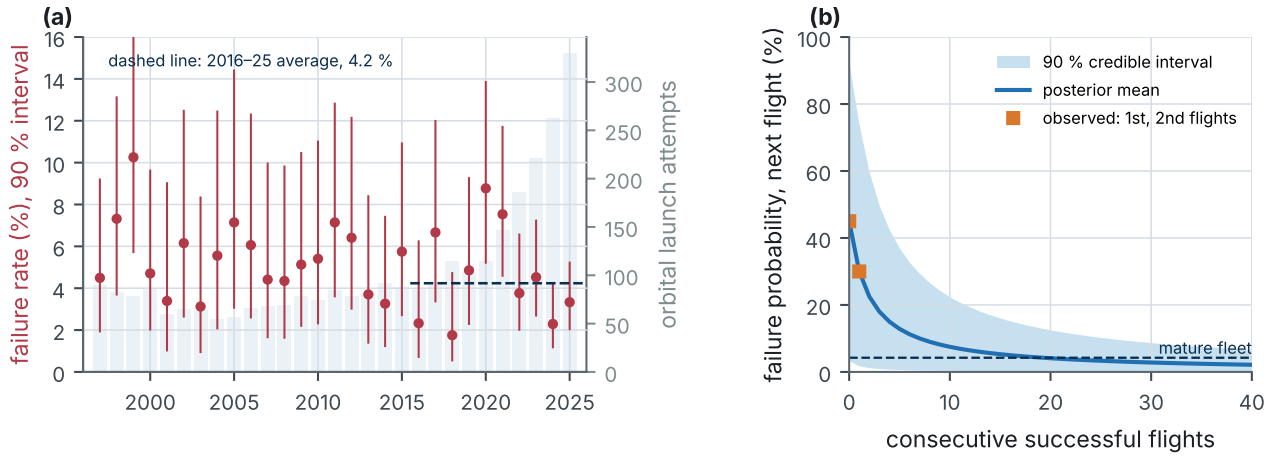
### 52.7.2 Insurance and Risk Models

Insurers cover launch failure and loss of a satellite in orbit. The market is small, with annual premiums of roughly $550–700 million in 2023–25, so one bad year can wipe out several good ones. On Earth, satellites also enable **parametric insurance**. It pays a fixed amount when a measured index crosses a trigger, such as wind speed or flood extent mapped by radar, with no claim assessment. Descartes Underwriting, for example, uses satellite imagery and machine learning to set triggers and pay quickly.

How does an insurer price a rocket that has flown only twice? The simplest honest model treats each flight as a coin toss with an unknown failure probability $p$. Prior knowledge is expressed as a **beta distribution**, $\mathrm{Beta}(\alpha, \beta)$. After $n$ flights with $k$ failures, the updated (posterior) estimate has mean

$$\hat{p} = \frac{\alpha + k}{\alpha + \beta + n}.$$

This is the **beta–binomial** model. It lets each flight shift the estimate by an amount that depends on how much was already known.




(a) Orbital launch attempts and failures by year from GCAT (J. McDowell, updated 13 October 2026); intervals from Jeffreys priors. (b) Beta(0.9, 1.1) prior with a mean of 45 %, the reported maiden-flight failure rate (Seradata); orange squares are the reported first- and second-flight failure rates since 1990.

**FIGURE 52.14** Launch risk from data. (a) Orbital launch failure rates by year from 1997 to 2025, with 90 % intervals, over the number of launch attempts. (b) A Bayesian estimate of the failure probability of a new rocket as it builds a record of successes, starting from the reported 45 % failure rate of maiden flights; one success brings it to 30 %, the reported rate for second flights.

Figure 52.14 applies the model. Across the world, 70 of 1 652 orbital launch attempts failed in 2016–25, a rate of 4.2 %. Maiden flights fail far more often: about 45 % according to the analysts Seradata, and second flights about 30 % since 1990. A prior of $\mathbf{Beta(0.9, 1.1)}$ has a mean of 45 %. After one success the model gives 30 %, matching the reported second-flight rate. After ten straight successes it gives 7.5 %, and after 20 it gives 4.1 % — the rate of the mature fleet. One failure in the first three flights keeps the estimate at 12.7 % even after ten more successes. Real underwriters add engineering judgement, the record of the company and the cost of the payload.

### DATA BOX: SPACE INSURANCE, 2023–25

- **2023:** one of the worst years on record. Claims were about $995 million against premiums of about $557 million. Two satellites caused most of the loss: ViaSat-3 Americas, whose large antenna failed to deploy (a claim of about $420 million), and Inmarsat-6 F2, lost to a power failure ($348 million). More than 85 % of losses came after the satellite had separated from the rocket, not from launch failures.
- **2024:** claims fell to about $211 million against premiums of about $580 million (Seradata). Insurers raised rates after 2023, and total working capacity shrank to about $550 million in 2024 and $502 million in 2025 (Gallagher).
- **2025:** a quiet year until late December. Then SpainSat NG-2, a Spanish military communications satellite insured for over $400 million, was damaged on its way to geostationary orbit. Its likely total loss raised the year's loss ratio from about 15 % to about 75 % (Gallagher).

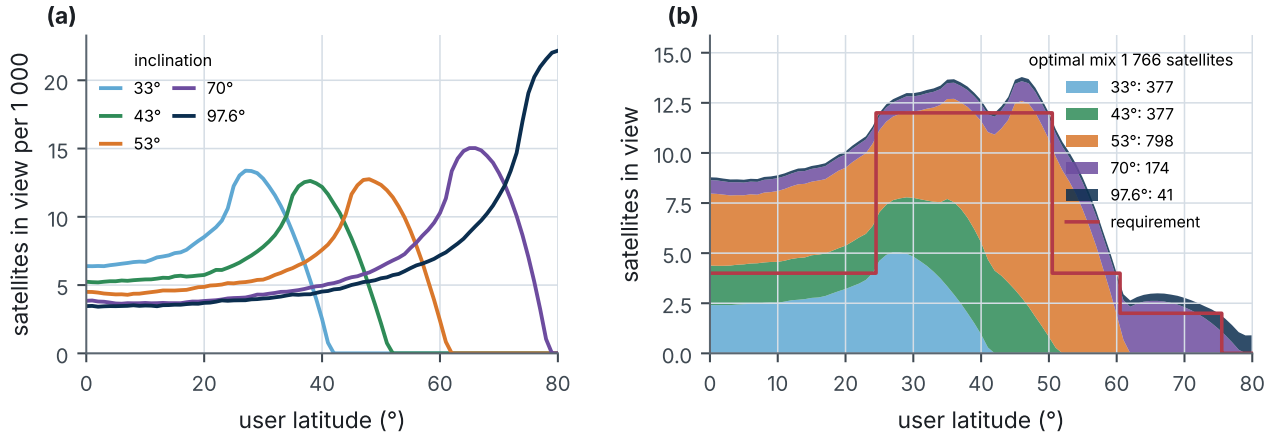
## 52.7.3 Constellation Design and Capacity

A broadband constellation sells capacity where its users are. Satellites in an inclined orbit spend most time near the edges of their latitude range. For a circular orbit of inclination $i$, the density of satellites per unit area at latitude $\varphi$ varies as

$$n(\varphi) \propto \frac{1}{\sqrt{\sin^2 i - \sin^2 \varphi}}, \qquad |\varphi| < i.$$

It is lowest at the equator and highest near latitude $\varphi = i$. Figure 52.15 (a) shows the number of satellites in view, above 25° elevation, for shells at 550 km. A shell at 53° inclination gives most coverage near 50° latitude, where much of the demand in North America and Europe lies. A polar shell is needed for high latitudes.

Choosing the mix of shells is an optimisation problem. In Figure 52.15 (b), a **linear programme** chooses how many satellites to put in each of five shells. It minimises the total while giving at least 12 satellites in view at 25–50° latitude, 4 elsewhere up to 60° and 2 up to 75°. The answer is 1 766 satellites in a mix of all five shells, with 798 at 53°. A single polar shell would need 3 095 satellites for the same requirement. Real operators solve much larger versions of this problem, with beams, spectrum, ground stations and launch costs. Learning helps with the input that matters most and is least certain: where demand will be.




Circular shells at 550 km with satellites spread evenly in orbit position and node (Monte Carlo); a satellite is in view above 25° elevation. (b) Linear programme: choose the number of satellites in each shell to minimise the total while meeting an illustrative requirement of 12 satellites in view at 25–50° latitude, 4 elsewhere up to 60° and 2 up to 75°.

**FIGURE 52.15** Constellation geometry and sizing. (a) Satellites in view above 25° elevation per 1 000 satellites, for shells at 550 km with different inclinations. (b) A linear programme chooses the number of satellites in each shell to meet an illustrative demand profile with the fewest satellites.

### 52.7.4 Customers, Pricing and Spectrum

Capacity in a constellation is fixed in each place, but demand is not. When a cell fills, the operator must refuse new users or let service slow for everyone. Starlink has done both. From 2024 it charged a one-off "congestion charge" of $100 in some US areas. By 2025–26 the charge was reported at $250 to more than $1 000 in busy areas, and some cities were shown as sold out. Demand models forecast sign-ups by cell, churn models flag users likely to leave, and pricing balances revenue against quality of service.

Spectrum is the other scarce resource. New systems must coordinate their frequencies through the ITU and national regulators and respect interference limits. Learning helps operators detect and locate interference and manage their beams (Chapter 49); the rules themselves are set by treaties.

### 52.7.5 AI Companies, Investment and New Demand

Money flowed into space in 2025–26, and much of it was linked to AI. The trackers differ, because they count different things, as Table 52.6 shows. Seraphim reported a record $12.4 billion of space-technology venture investment in 2025. Space Capital, which counts a wider range of

companies, reported a record $36 billion in the first quarter of 2026 alone. It named AI, geopolitics and computing in orbit as the drivers.

**TABLE 52.6** Money in the space business, 2025–26. Trackers use different definitions, so their figures cannot be compared directly.

| Item | Figure | Source and note |
|---|---|---|
| Global space economy, 2025 | $686 billion (+12 %) | Space Foundation, July 2026 |
| Space-tech venture investment, 2025 | $12.4 billion (record) | Seraphim |
| Investment, Q1 2026 | $36 billion, 148 companies (record) | Space Capital (wide definition) |
| Investment, Q1 2026 | $9.4 billion, 82 companies | Novaspace (narrower definition) |
| SpaceX IPO, June 2026 | ≈ $75 billion ($85.7 billion with extra shares) | Largest IPO on record |
| Space insurance premiums, 2025 | ≈ $675 million | Secondary sources |

The largest deal joined a space company and an AI company. In February 2026 SpaceX completed an all-share takeover of the AI company xAI. In June 2026 it sold shares to the public for about $75 billion at a valuation of about $1.77 trillion. It was the largest initial public offering (IPO) on record. Its filing reportedly pointed to orbital AI computing from as early as 2028. It also warned that the technology is unproven and may never be commercially viable. Google, with Planet, launched a prototype of its Suncatcher computing satellites in October 2026, and Starcloud flew a data-centre graphics processor in 2025. Chapter 45 discusses whether computing in orbit can compete with ground data centres.

AI creates demand for space in three ways. It needs data, such as images and weather records, to train models. It needs connectivity to reach ships, aircraft and remote sites. And, more speculatively, it may move computing itself into orbit. Each claim deserves the usual engineering test: what are the numbers, and has it flown?

## 52.8 Bringing AI into an Engineering Team

The key idea of this section is that AI tools are engineering tools. They need requirements, verification and configuration control like any other. Teams that treat them this way gain speed without losing safety.

Most failures of AI in engineering are not dramatic. A generated number is copied into a report unchecked. A surrogate is used outside its range. A model trained on last year's printer is used on this year's. Ordinary engineering discipline catches each of them. The practice box gives a short workflow; Chapter 53 covers formal assurance, security and governance.

### PRACTICE: INTRODUCING AN AI TOOL INTO A SPACECRAFT DESIGN TEAM SAFELY

1. **State the job and the error budget.** Write down what the tool will do and what a wrong answer would cost. Example: "suggest bracket shapes; a wrong shape costs one re-analysis." Avoid uses where errors would reach flight hardware unchecked.
2. **Check the data rules.** Confirm with your export-control and security officers which data the tool may see. Use an approved deployment, and never paste controlled data into public services.
3. **Find a baseline.** Measure how the task is done today, by people or by simple rules. A new tool must beat that baseline on your own data, not on the vendor's examples.
4. **Test on held-out cases.** Keep a set of past problems with known answers that the tool never saw during tuning. Measure errors, false alarms and misses. For classifiers, set the threshold from real costs.
5. **Keep a human gate.** Decide who reviews each output and how. Nothing enters the controlled baseline — drawings, requirements, code or test reports — without the usual analysis, test or review.
6. **Record provenance.** Log the tool, its version, its inputs and the reviewer for each output that is used. Store this in the digital thread, so that any result can be traced and repeated.
7. **Watch for drift.** Re-check performance when the process, the supplier, the software version or the design space changes. Retire tools that no longer pass.
8. **Share lessons.** Report both successes and failures to the team's lessons-learned system, so that the next project starts from evidence.

### KEY TAKEAWAYS

- AI proposes; physics, test and people verify. Every AI output must pass the same verification gate as human work before it enters a controlled design.
- Most "AI-designed" structures rest on topology optimisation. In a 2-D test, it saved about 31 % of mass at equal stiffness. Goddard's generative EXCITE bracket met mass and frequency targets that no buildable human design met in four iterations.
- Bayesian optimisation with a Gaussian-process surrogate found a near-optimal design in a median of 22 expensive runs, where random search needed about 266.
- Language models help with requirements, lessons learned, SysML v2 drafts and code. But they make plausible errors, and export-controlled data need approved tools.
- In manufacturing, the value of a screening model comes from choosing its threshold with real costs. In our test, the cost-optimal screen cost a third of full expert review, while the default threshold missed one defect in six.
- On rockets in flight, guidance, engine protection and flight safety use optimisation and fixed rules. Learning works on the ground: health monitoring in tests, weather, scheduling and design.
- In the business, AI turns satellite data into market signals, prices risk and sizes constellations. AI-linked investment and the SpaceX–xAI merger reshaped space finance in 2026.

### REVIEW QUESTIONS

1. Explain why the SIMP method raises density to a power $p \approx 3$. What would happen with $p = 1$?
2. A topology-optimised bracket uses 40 % of the material of a solid block and is 1.7 times as compliant. What fraction of the solid material would a uniform plate of equal stiffness need, and what is the mass saving?

3. How does an evolutionary algorithm differ from a gradient-based optimiser? Give one advantage and one disadvantage of each.
4. Write the expected-improvement formula and explain the role of each of its two terms. Why does Bayesian optimisation need fewer evaluations than random search?
5. Name three tasks in systems engineering where language models help. For each, describe the check that must follow.
6. What does $a_{90/95}$ mean on a probability-of-detection curve, and why is it needed for fracture-critical printed parts?
7. A review costs \$1 000 and an escaped defect \$400 000. With calibrated probabilities, what threshold minimises the expected cost? Why is the default of 0.5 a poor choice?
8. Give three reasons why landing guidance uses convex optimisation rather than a learned policy.
9. Using a $\mathrm{Beta}(0.9, 1.1)$ prior, estimate the failure probability of a new rocket after five consecutive successes. How does one early failure change the picture?
10. Explain why satellites in a 53° shell give more coverage at 45° latitude than at the equator. How would an operator use this in planning capacity?

CHAPTER

# 53 Trustworthy AI for Space: Assurance, Security, Governance and Outlook

Image: NASA/GSFC (NASA/IV&V)

*Machine learning now helps to forecast weather, screen images, plan operations and steer robots. Before a learned function may act on its own in space, engineers must show that it is safe, secure and lawful. That is hard: behaviour is learned from data rather than written down, and it can fail silently when conditions change. The chapter reviews the standards that appeared in 2024–26 and the main assurance methods — design domains, testing, formal verification, runtime monitors and assurance cases — with small experiments that show their strengths and limits. It covers attacks on learned systems and their defences, the fast-changing law of AI, the energy cost of computing and an outlook to 2035. It ends with a review of Part XIII.*

IN THIS CHAPTER

53.1 Why Learned Systems Are Hard to Assure
53.2 Standards and Guidance, 2024–26
53.3 Building the Evidence: Assurance Methods
53.4 Security: Attacks on Learned Systems
53.5 Governance, Law and Ethics
53.6 Outlook to 2035
53.7 Part XIII in Review

A spacecraft's flight software is among the most carefully checked code in the world. Every requirement is written down, every line is traced to a requirement, and every branch is tested. A neural network breaks this pattern. Its behaviour is not written by people. It is learned from examples and stored in millions of numbers that nobody can read. Yet Chapters 43–52 show

learned functions spreading into weather forecasting, image screening, operations, navigation and design. The question is no longer whether AI can help, but how far we can trust it and how we can show that trust to others.

This chapter is about that showing. **Assurance** means collecting evidence that a system is good enough for its job, and building an argument from that evidence that an independent reviewer can check. The hero image shows NASA's Katherine Johnson Independent Verification and Validation Facility in Fairmont, West Virginia. Since 1993, engineers there who are independent of each project have checked the software of NASA missions. Learned software needs the same scrutiny, with new tools.

The chapter explains why learned systems are hard to assure, surveys the standards and describes assurance methods, with small experiments run for this book. It then covers attacks, law and ethics as of October 2026, and an outlook to 2035. It closes Part XIII with a synthesis. Learning methods are explained in Chapters 43 and 44, and onboard hardware in Chapter 45.

## 53.1 Why Learned Systems Are Hard to Assure

The key idea of this section is that classical assurance checks code against a written specification. A learned model has neither readable code nor a complete specification, so the evidence must come from data, tests and protection around the model.

### 53.1.1 Behaviour Learned from Data

A learned model $f_\theta$ has parameters $\theta$ chosen to make the average loss small on $n$ training examples (Chapter 43). What matters in service is the loss on the inputs it will actually meet, drawn from the operational distribution $P_{\text{op}}$.

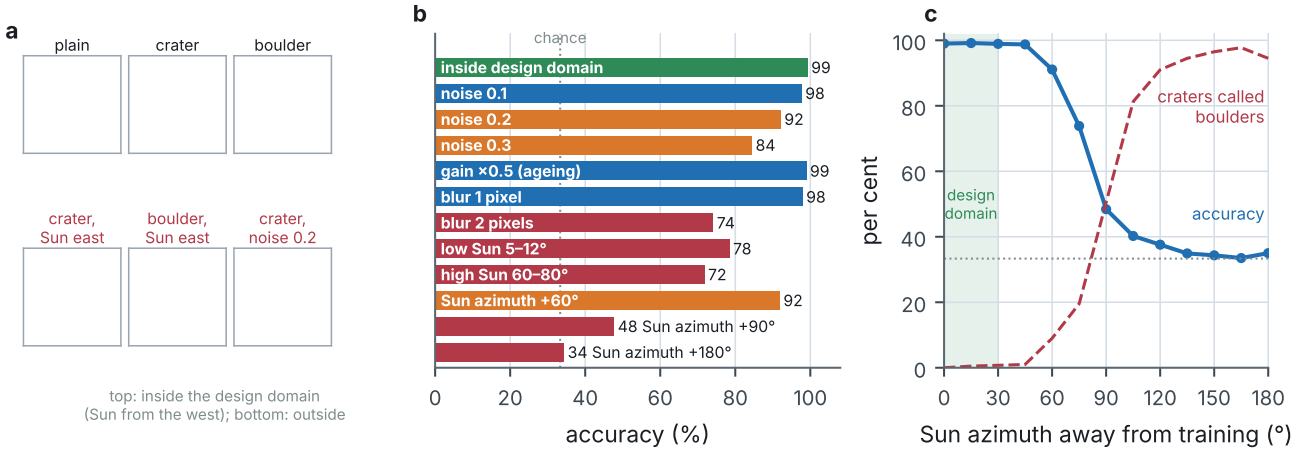
$$\hat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(\mathbf{x}_i), y_i\big), \qquad R_{\text{op}}(\theta) = \mathbb{E}_{(\mathbf{x},y)\sim P_{\text{op}}}\, \ell\big(f_\theta(\mathbf{x}), y\big).$$

If training, test and operational data all come from the same distribution, the test error is a fair estimate of $R_{\text{op}}$. If they do not — a **distribution shift** — nothing in training limits the error. Four problems follow:

- **The specification gap.** Nobody can define "a crater" or "a safe landing site" completely in words. The labelled examples become the real specification, so the data must be checked as carefully as requirements.
- **Opacity.** The model is a large set of numbers. No single number maps to a requirement, and code-coverage measures used for normal software say little about what a network has learned.
- **Shift.** Light, seasons, detectors and targets change. A model can be excellent on its test set and poor in orbit, often without any sign that it is wrong.
- **Fragility.** Tiny, carefully chosen changes to an input can flip the output (see the section on security). Models with the same test accuracy can also behave very differently on unfamiliar inputs.

### 53.1.2 A Small Experiment in Distribution Shift

Figure 53.1 makes these points concrete. A small neural network learns to label 16 × 16-pixel surface patches as plain ground, crater or boulder. The patches are synthetic. Each is drawn from a height map and lit by the Sun using a simple shading law. In training the Sun always comes from the west, between 20° and 45° above the horizon. That set of conditions is the model's **operational design domain (ODD)**: the conditions in which it is meant to work.




Synthetic 16 × 16 patches rendered from height maps with Lambertian shading; network 256–64–32–3 (ReLU, numpy, Adam) trained on 9 000 patches with the Sun from the west (azimuth 150–210°, elevation 20–45°) and noise 0.02. Each bar or point: 1 200 new test patches. Seeds fixed.

**FIGURE 53.1** Distribution shift in a learned crater and boulder classifier. (a) Example patches; the bottom row lies outside the training conditions. (b) Accuracy under noise, a halved detector gain, blur, and changes of Sun elevation and direction. (c) Accuracy and the share of craters called boulders as the Sun moves away from the training direction.

Inside its domain the network is 99.2 % accurate. It shrugs off some changes that look large to a person. With noise of 0.2 (half the brightness of an average pixel), accuracy is still 92.1 %, and halving the detector gain changes almost nothing. Other changes are fatal. With the Sun 90° from the training direction, accuracy falls to 47.7 %. With the Sun from the east, it falls to 34.3 %, no better than guessing among three classes. More than 90 % of craters are then called boulders, because a crater lit from one side looks like a boulder lit from the other. Very low or very high Sun and strong blur also cost 21–27 points.

Three lessons carry over to real systems. The ODD must list every factor that changes the input, not just the obvious ones. Each factor must be tested across its range, including just outside the domain. And the failures are silent. With the Sun from the east, 95 % of the wrong answers came with more than 90 % confidence.

### 53.1.3 How Classical Flight Software Is Assured

Space and aviation software standards share one idea: the worse a failure's consequences, the more evidence is required. The European standards ECSS-E-ST-40C (software engineering) and ECSS-Q-ST-80C (software product assurance), both revised in April 2025, assign each function a **criticality category** from A to D. The category follows from the severity of the worst hazard it can cause. NASA's NPR 7150.2D classifies software from Class A (human-rated) to Class F (general computing). Civil aviation uses DO-178C (also published as ED-12C), with design assurance levels (DAL) A–E. Table 53.1 compares the schemes.

At the top levels, every requirement is traced to code and tests, and structural coverage shows that tests exercised every decision in the code. **MC/DC**, for example, requires each condition in a decision to be shown to change the outcome on its own. A network has almost no branches, so such measures are trivially met and prove nothing. The new ML guidance therefore adds evidence about data, learning and runtime behaviour on top of the classical process.

**TABLE 53.1** How space and aviation standards rank software by criticality, and where machine learning stands in October 2026. MC/DC: modified condition/decision coverage.

| Scheme | Levels | Top level means | What the top level demands | Status of ML |
|---|---|---|---|---|
| ECSS-Q-ST-80C Rev.2 (2025) | Categories A–D | Failure can cause loss of life or of the system | Full verification, independent review, strict coverage | ML handbook covers B–D only |
| NASA NPR 7150.2D (2022) | Classes A–F | Human-rated software | Full process, IV&V where required | Handbook topics added 2025 |
| DO-178C / ED-12C (2011) | DAL A–E | Catastrophic failure condition | 71 objectives; MC/DC structural coverage | No ML provisions; ED-324 in work |
| EASA AI concept papers | AI levels 1–3 | Level 3: advanced automation | Learning assurance plus classical assurance | Level 3 in consultation (2026) |

## 53.2 Standards and Guidance, 2024–26

The key idea of this section is that the first ML assurance standards now exist, but they are mostly guidance for lower criticality. No standard yet allows a learned function to be the only protection against a catastrophic failure.

Table 53.2 lists the main documents. Space has one dedicated document so far. **ECSS-E-HB-40-02A**, the European machine-learning handbook, was published on 15 November 2024. It covers whether to use ML at all, data management, training, model verification, system-level evaluation and a **safety cage** architecture (described below). It is guidance, not a binding standard, and covers criticality categories B–D; category A is out of scope for now. In 2025 NASA added topics on AI to its Software Engineering Handbook, under the existing NPR 7150.2 classes.

**TABLE 53.2** Standards and guidance for assuring AI, as of October 2026.

| Document | Body and date | Scope | Status, October 2026 |
|---|---|---|---|
| ECSS-E-HB-40-02A | ECSS, Nov 2024 | ML handbook: data, training, verification, safety cage | Guidance; categories B–D |
| ECSS-E-ST-40C Rev.1, ECSS-Q-ST-80C Rev.2 | ECSS, Apr 2025 | Space software engineering and assurance | In force |
| NPR 7150.2D; handbook topics 7.25, 8.25 | NASA, 2022; 2025 | Software classes; AI in engineering and assurance | In force; guidance |
| AI Roadmap 2.0; Concept Paper Issue 2 | EASA, 2023; 2024 | Learning assurance, AI levels 1–2 | Guidance; Issue 3 consulted Jun–Aug 2026 |
| NPA 2025-07 | EASA, Nov 2025 | First AI rules for aviation | Proposal; consultation closed 2026 |
| ED-324 / ARP6983 | EUROCAE WG-114, SAE G-34 | Process standard for aeronautical AI | Drafting |
| ISO/IEC 22989, 23053 | ISO/IEC, 2022 | AI concepts, terms, ML framework | Published |

| Document | Body and date | Scope | Status, October 2026 |
|---|---|---|---|
| ISO/IEC 23894; 42001 | ISO/IEC, 2023 | AI risk management; AI management system | Published; 42001 certifiable |
| ISO/IEC TR 24029-1; ISO/IEC 24029-2 | ISO/IEC, 2021; 2023 | Robustness of neural networks; formal methods | Published (report; standard) |
| ISO/IEC TR 5469 | ISO/IEC, Jan 2024 | Functional safety and AI | Published (report) |
| AI RMF 1.0; AI 600-1 | NIST, 2023; 2024 | Voluntary risk framework; generative-AI profile | Revision ordered 2025; none by Oct 2026 |
| UL 4600 | UL, 2020; 3rd ed. 2023 | Safety cases for autonomous products | Published |

Aviation is further ahead, and space engineers follow its lead. The European Union Aviation Safety Agency (EASA) defines three **AI levels**. Level 1 assists a human. Level 2 is human–AI teaming. Level 3 is "advanced automation", with little or no human oversight. EASA's Concept Paper Issue 2 (2024) gave guidance for Levels 1 and 2. In November 2025 EASA proposed its first binding AI rules, in Notice of Proposed Amendment (NPA) 2025-07. A proposed Issue 3 of the concept paper, published on 3 June 2026, extends the guidance to reinforcement learning and to Level 3. A joint EUROCAE–SAE group, set up in 2019, is drafting the process standard ED-324 (ARP6983 in the USA); it was unpublished in October 2026.

General standards come from the joint ISO and IEC committee on AI. ISO/IEC 42001 (December 2023) defines an AI management system that organisations can be certified against, as ISO 9001 does for quality. ISO/IEC TR 5469 (January 2024) covers AI inside safety functions, conventional safety functions guarding AI, and AI used to design safety functions. The voluntary AI Risk Management Framework of the US National Institute of Standards and Technology (NIST) has four functions: govern, map, measure and manage. A revision ordered in July 2025 had not appeared by October 2026. **UL 4600** is the best model for the argument itself. Instead of prescribing a process, it asks for a complete, checkable **safety case** for an autonomous product.

### LANDMARK: THE FIRST DESIGN-ASSURANCE STUDY FOR NEURAL NETWORKS (2020–21)

- In 2019–20 EASA and the Swiss company Daedalean studied how a neural network could be certified for a safety-related avionics task. The case study was a camera-based system guiding an aircraft to a runway.
- Their March 2020 report, *Concepts of Design Assurance for Neural Networks* (CoDANN), introduced **learning assurance**: evidence about the data, the learning process and generalisation, added to the usual software assurance.
- It proposed the **W-shaped process** (Figure 53.2): the classical V of design and verification, with a second V for data and learning inserted in the middle.
- A second study (May 2021) applied the ideas to visual traffic detection, and the US Federal Aviation Administration assessed them in 2022. These studies shaped the EASA concept papers and the ECSS handbook.

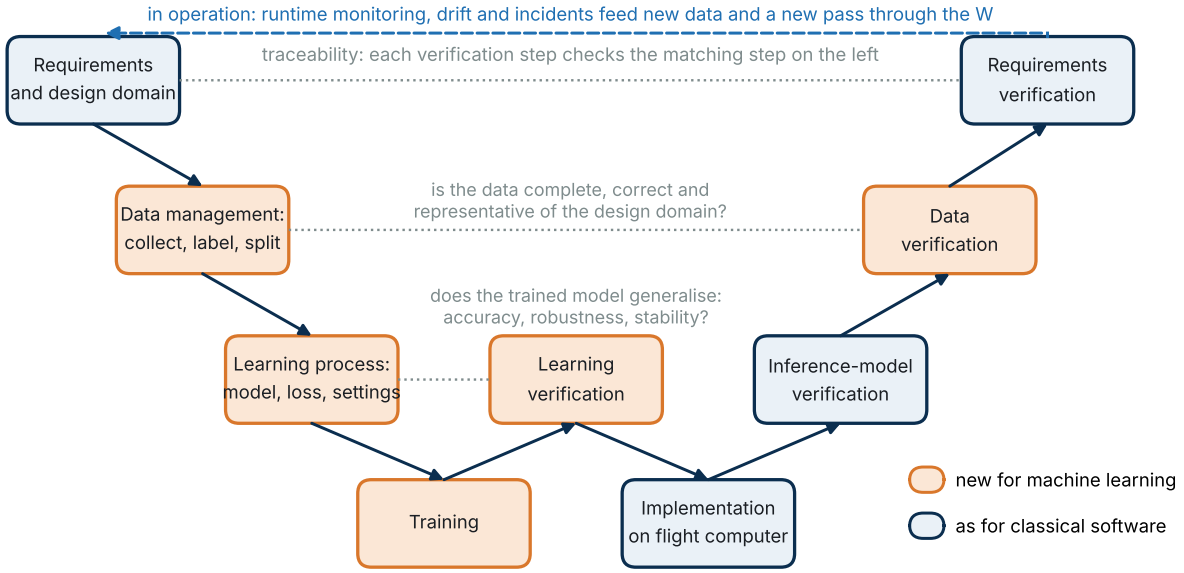



**FIGURE 53.2** A W-shaped learning-assurance life cycle, simplified after the EASA concept papers. Orange steps are new for machine learning; dotted lines link each verification step to the step it checks. In operation, monitoring and incidents feed new data and a new pass through the W.

## 53.3 Building the Evidence: Assurance Methods

The key idea of this section is that no single method is enough. Assurance combines a defined domain, good data, tests, small proofs and runtime protection, tied together in an argument.

### 53.3.1 Design Domain, Data and Requirements

Everything starts with the **ODD**. For a lunar hazard detector, it lists altitude, speed, Sun elevation and azimuth, terrain types, dust, camera settings and sensor health, each with a range. The data must then **cover** the domain: every combination that matters must appear often enough to train and test on. Coverage is never complete in many dimensions, so engineers sample key factors on grids, fill the rest randomly and record what is missing.

Data need their own paperwork. A **datasheet** for a dataset (proposed in 2018) records how the data were collected and labelled, who labelled them, how they were split and what is known to be missing. A **model card** (2019) records what a model is for, how it was tested, on which slices of the domain, and its known limits. Both are now part of the model registry in a good machine-learning operations (MLOps) pipeline (Chapter 45).

**Requirements for an ML component** go beyond “accuracy above 99 %”. They state performance for each slice of the domain, such as each Sun elevation band, and cover robustness, stability, calibrated confidence (Chapter 44), timing, memory and behaviour under radiation upsets (Chapter 45). Each must be testable and traced to evidence.

### 53.3.2 Robustness Testing

Robustness tests change inputs in controlled ways. **Corruption tests** add noise, blur, compression, stray light, dead pixels and changes of gain, as in Figure 53.1. **Metamorphic tests** use known relations instead of labels. For example, a crater shifted by two pixels must stay a

crater. **Adversarial tests** search for the worst input within a small budget (see the section on security below). Tests can show weaknesses but never prove their absence. Still, they are cheap and find most real problems early.

### 53.3.3 Formal Verification of Neural Networks

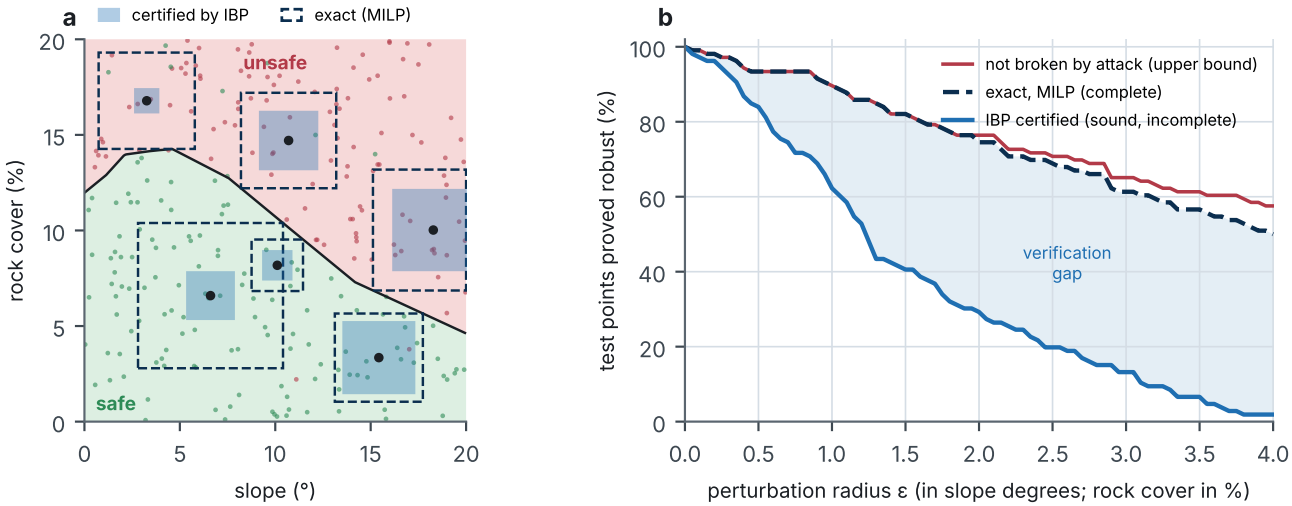
Formal verification aims to **prove** a property for every input in a region, not just for the inputs tested. The usual property is **local robustness**: for an input $\mathbf{x}_0$, no input within distance $\varepsilon$ changes the decision. In the $L_\infty$ norm the region is a box: each input value may move by up to $\varepsilon$.

The simplest sound method is **interval bound propagation (IBP)**. It pushes the whole box through the network layer by layer. Write the box as a centre $\mathbf{c}$ and a radius $\mathbf{r}$. A linear layer with weights $\mathbf{W}$ and bias $\mathbf{b}$ maps it to

$$\mathbf{c}' = \mathbf{W}\mathbf{c} + \mathbf{b}, \qquad \mathbf{r}' = |\mathbf{W}|\,\mathbf{r},$$

where $|\mathbf{W}|$ takes the absolute value of every weight. A ReLU unit, $\max(0, z)$, never decreases, so it simply clips both ends of each interval at zero. At the output, if the lowest possible score of the predicted class exceeds the highest possible score of every other class, the property is proved. IBP is fast, but its boxes grow at each layer. It can fail to prove a property that is true.

**Complete** methods give an exact answer: either a proof or a counter-example. They encode each ReLU exactly, either with logic in a satisfiability-modulo-theories (SMT) solver or with a binary variable in a **mixed-integer linear program (MILP)**. The Reluplex solver (2017) used this approach to verify properties of the neural networks proposed for the ACAS Xu aircraft collision-avoidance system. Today's leading tools combine fast bounds with branch-and-bound search. One of them, α,β-CROWN, won the annual international verification competition VNN-COMP every year from 2021 to 2025.




Synthetic landing-site data (1 500 training points, 3 % label noise); network 2–16–16–2 (ReLU, numpy). 106 correctly classified test points. Radii in the L∞ norm (a box). MILP: scipy milp with a big-M encoding of each ReLU; attack: projected gradient with 3 random starts.

**FIGURE 53.3** Proving robustness on a tiny network. (a) A learned "safe or unsafe" boundary for landing sites from slope and rock cover, with the largest box around six test points that interval bound propagation (IBP) can prove robust and the true largest box from an exact mixed-integer solver. (b) Share of test points proved robust against the box size.

Figure 53.3 compares the methods on a toy landing-site classifier with two inputs: slope and rock cover. The network has two hidden layers of 16 ReLU units and is 96.4 % accurate. For 106 correctly classified test points, IBP proved a median radius equal to 1.2° of slope. The exact MILP radius was 4.0°: IBP proved, on average, only 46 % of the true radius. A gradient attack found counter-examples only beyond the exact radius, as it must. At a radius of 1° (one degree of slope and one point of rock cover), IBP certified 62 % of points and the exact method 90 %.

The exact method took under a second per point here, but its cost can grow exponentially with network size. Complete verification works for small and medium networks, not for large vision or language models. And local robustness is not correctness: a network can be provably robust and still wrong. Formal methods therefore suit small, critical networks — controllers, monitors, surrogates — and specific properties such as "never command thrust above the limit".

### 53.3.4 Runtime Monitors and Safety Cages

If a model cannot be fully verified, it can be **contained**. The **simplex architecture** (Lui Sha, 2001) runs an unverified high-performance controller beside a simple verified one. A verified switch hands control to the simple one whenever the system nears the edge of a safe region. Chapter 48 shows a simplex "shield" on a learned lunar-landing controller. The ECSS handbook calls the general idea a **safety cage** (Figure 53.4). An input monitor checks that inputs lie inside the domain. An output monitor checks that outputs are plausible. A verified arbiter uses the learned result only if both checks pass.

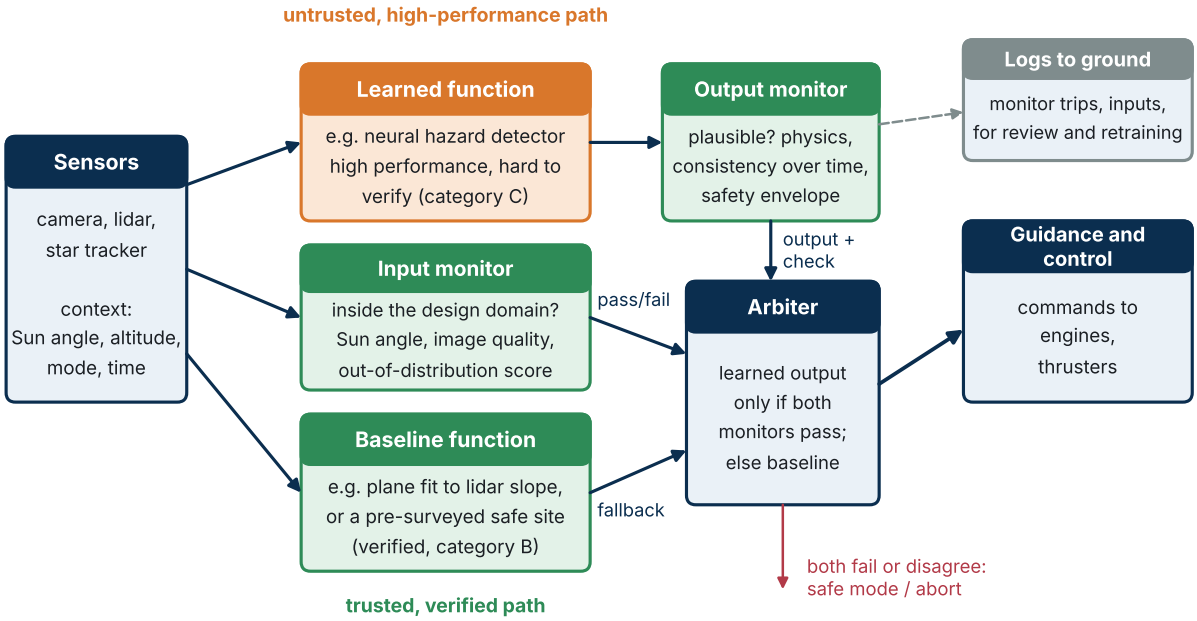


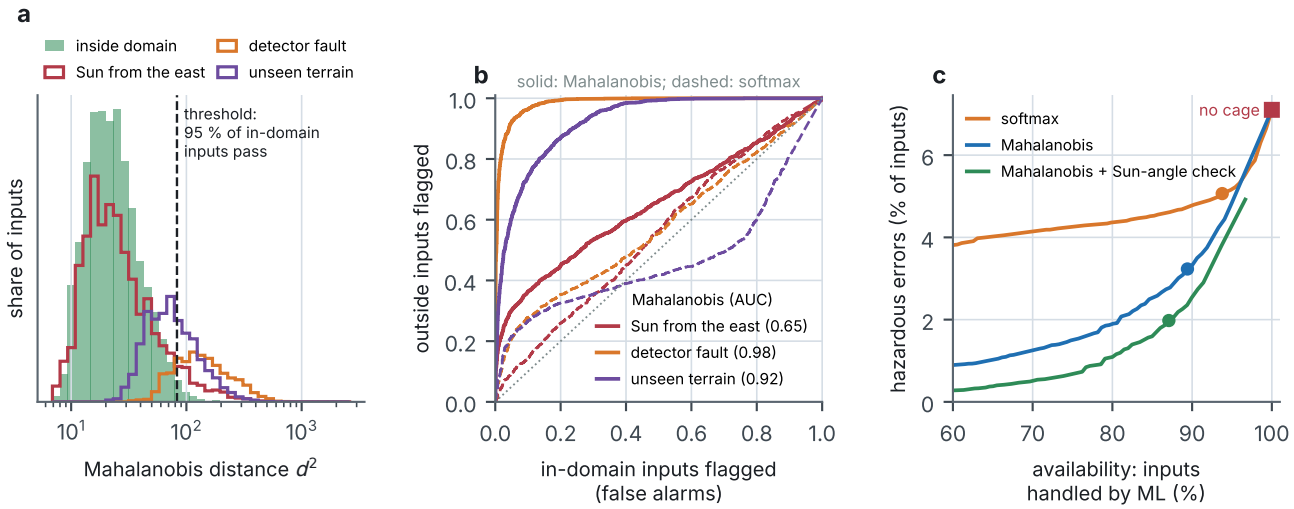

**FIGURE 53.4** A safety-cage (simplex) architecture for an onboard learned function. Only the arbiter, monitors and baseline need the highest assurance; the learned function can sit at a lower category.

The cage changes the assurance problem. The arbiter, monitors and baseline are small and conventional, so classical methods can verify them to a high category. The learned function then needs less assurance, because its failures are caught.

A common input monitor measures how far an input's internal features lie from the training data. With hidden features $\mathbf{h}$, and class means $\boldsymbol{\mu}_k$ and covariances $\boldsymbol{\Sigma}_k$ from training data, the score is the **Mahalanobis distance** to the nearest class.

$$d^2(\mathbf{x}) = \min_k (\mathbf{h} - \boldsymbol{\mu}_k)^\mathsf{T} \boldsymbol{\Sigma}_k^{-1} (\mathbf{h} - \boldsymbol{\mu}_k).$$

A large $d^2$ means the input looks unlike anything seen in training, so the monitor rejects it. The threshold sets the trade between false alarms and missed detections. A simpler baseline flags inputs for which the network's top **softmax** probability is low.




Same network and patches as the previous figure. Monitor: distance to the nearest class in the 96 hidden units, with class means and covariances from the training set. Each outside set: 1 800 patches. Stream in (c): 90 % inside the domain, 10 % outside (equal thirds); dots mark the threshold that passes 95 % of in-domain inputs; flagged inputs go to a safe fallback.

**FIGURE 53.5** Runtime monitoring of the patch classifier. (a) Mahalanobis distances for inputs inside the design domain and for three kinds of input outside it. (b) Receiver operating characteristic (ROC) curves for the Mahalanobis and softmax monitors. (c) Availability against hazardous errors for safety cages built from each monitor.

Figure 53.5 tests both monitors on the patch classifier. Three kinds of outside input were used: Sun from the east, a detector fault with bad columns and hot pixels, and an unseen terrain type (a long trough). With the threshold set so that only 5 % of in-domain inputs are rejected, the Mahalanobis monitor caught 92 % of faults and 60 % of unseen terrain. The softmax monitor caught 21 % and 22 %. Neither monitor saw the Sun problem well (30 % and 9 %). A crater lit from the east really does look like a familiar boulder, so the network's features look normal too. The cure is not a better image monitor but a **domain check on data the spacecraft already knows**: the Sun's direction comes from navigation. The design lesson: check the domain directly wherever possible, and use learned monitors only for what cannot be checked directly.

**Uncertainty-aware decisions** extend the idea. The model reports a calibrated probability or a conformal prediction set (Chapter 44), and the logic acts only when the uncertainty is small enough for what is at stake. Otherwise it asks the ground or falls back to a safe default.

**EXAMPLE: WHAT A SAFETY CAGE COSTS AND BUYS**

Take the patch classifier of Figure 53.5 and a stream of inputs. 90 % lie inside its domain and 10 % outside, split equally among Sun from the east, detector faults and unseen terrain. A **hazardous error** is an input on which the learned function acts and is wrong; a flagged input goes to a safe fallback.

1. **No cage.** In-domain errors: 0.90 × 1.1 % = 1.0 % of all inputs. Outside: 3.3 % × (65 % + 18 % + 100 %) = 6.1 %, since unseen terrain is always mislabelled. Total **7.1 %**, with 100 % availability.
2. **Mahalanobis cage.** Reject inputs with $d^2$ above the 95th percentile of in-domain scores. Availability falls to **89.4 %** and hazardous errors to **3.2 %**.
3. **Add the Sun-angle check.** All Sun-from-east inputs are now rejected. In-domain inputs keep 0.90 × 95 % = 85.5 % availability with 0.65 % errors. Faults that slip through add 0.3 % availability and almost no errors. Unseen terrain passes 40 % of the time, adding 1.3 % to both. Total: availability **87.1 %**, hazardous errors **2.0 %**.
4. **Trade.** The cage cut hazardous errors by a factor of 3.6 for a loss of 12.9 points of availability. Two-thirds of the remaining hazard is unseen terrain, so the next step is an output check, such as a slope from lidar, not a tighter threshold.

A cage turns an unbounded failure mode into a known rate of fallbacks. Whether the trade is worth it depends on what the fallback costs: a slower landing, a lost image, or an aborted manoeuvre.

**LANDMARK: A NEURAL NETWORK FLIES A SATELLITE UNDER A WATCHDOG (2026)**

- In July 2026 the US Air Force Research Laboratory (AFRL) reported a new flight test. A neural network had controlled a CubeSat's attitude for one orbit without human help.
- The network was trained by reinforcement learning (Chapter 48), then tested in simulation and in the laboratory before upload.
- Two runtime-assurance systems flew with it in **shadow mode**: an "AI watchdog" that detects unverified situations and would switch to backup systems, and a "guardrail" that would intervene near predefined safety limits. They observed and logged, but did not take control.
- AFRL gave no performance figures, and the flight was a demonstration. But it follows the pattern this chapter recommends: learned control, conventional limits, and evidence from shadow mode before monitors get authority.

### 53.3.5 Testing, Simulation and Independent Review

Statistical testing gives the most direct evidence: run the function on many independent inputs drawn from the domain and count failures. If $n$ tests show no failures, the failure probability $p$ is below a bound $p_U$ with confidence $C$ when $(1 - p_U)^n = 1 - C$. Rearranged,

$$n = \frac{\ln(1 - C)}{\ln(1 - p)} \approx \frac{-\ln(1 - C)}{p},$$

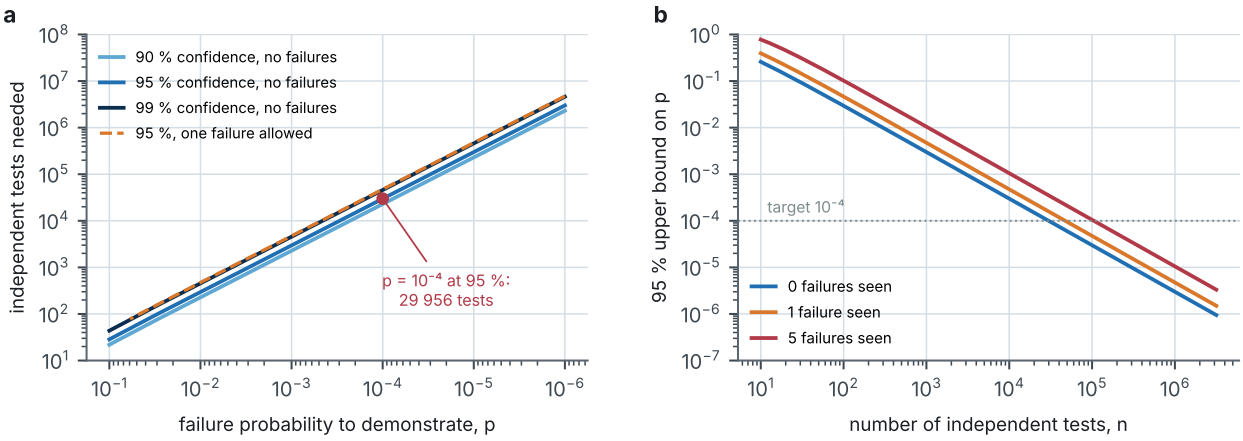
which at 95 % confidence gives the "rule of three": $n \approx 3/p$ (Figure 53.6).

### EXAMPLE: HOW MANY TESTS PROVE A FAILURE RATE BELOW $10^{-4}$?

A learned hazard detector must mislabel fewer than one landing-site image in 10 000, with 95 % confidence.

1. **No failures allowed.** $n = \ln(0.05)/\ln(1 - 10^{-4}) = 29\,956$ independent test images, close to $3/p = 30\,000$.
2. **Other confidence levels.** At 90 %, 23 025 images; at 99 %, 46 049.
3. **One failure seen.** If one test fails, the campaign must grow to **47 437** images to support the same claim (binomial distribution).
4. **A smaller campaign.** 10 000 images with no failure support only $p < 3.0 \times 10^{-4}$.
5. **The catch.** The images must be independent and drawn from the operational domain. A thousand frames from one descent are not a thousand independent tests. For aviation's catastrophic level of $10^{-9}$ per flight hour, the same rule needs $3 \times 10^{9}$ hours, about 342 000 years of testing.

Testing alone can support moderate failure rates. Rare-failure claims need architecture: a verified cage around the learned part, so that its failures are caught rather than merely counted.




Binomial statistics, assuming independent tests drawn from the operational distribution. (a) Smallest n with P(at most k failures | n, p) ≤ 1 − confidence. (b) One-sided Clopper–Pearson 95 % upper bound. For no failures, n ≈ 3/p at 95 % (the 'rule of three').

**FIGURE 53.6** What testing can show. (a) Independent tests needed to demonstrate a failure probability, for several confidence levels and with one failure allowed. (b) The 95 % upper bound on the failure probability after a test campaign, for 0, 1 or 5 failures seen.

**Simulation** supplies the volume. Landers, rovers and rendezvous systems are tested on millions of simulated scenarios, with **scenario coverage** measured over the ODD. But a learned model can exploit flaws of the simulator, and synthetic images differ from real ones (the sim-to-real gap; Chapter 45). The answer is a ladder of evidence: simulation, then hardware-in-the-loop tests with real cameras and computers, field and flight tests (Figure 53.7), and shadow-mode operation in space.

Three other kinds of evidence complete the picture:

- **Explainability as evidence.** Attribution maps (Chapter 44) show which inputs drove a decision and catch shortcuts, such as a ship detector that looks at the quay. They only support other evidence: methods can disagree, and a plausible map does not prove a correct model.

- **MLOps traceability.** Every deployed model must be traceable to its code, data version, settings and test results, so that it can be rebuilt and an anomaly traced to its cause (Chapter 45).
- **Independent verification and validation (IV&V).** A team that did not build the model repeats key tests, writes its own adversarial cases and reviews the argument. This matters even more for ML, because the developers' data choices are themselves a source of error.

**FIGURE 53.7** NASA's Morpheus prototype lander lifting off on free flight 15 at Kennedy Space Center in December 2014. During the 97-second flight its autonomous landing and hazard avoidance sensors surveyed a field of hazards, chose a safe site and guided the lander to it. Flight tests like this close the gap between simulation and reality. Image: NASA/KSC.

Table 53.3 summarises what each method can and cannot show.

**TABLE 53.3** Assurance methods for learned functions: what each shows and its limits.

| Method | What it shows | Main strength | Main limit | Use in space, 2026 |
|---|---|---|---|---|
| Design domain and data coverage | Data match the conditions of use | Basis for all other evidence | Never complete in many dimensions | ECSS and EASA guidance |
| Statistical testing | Bound on failure rate | Simple and convincing | About 3/p independent tests | Standard practice |
| Robustness tests | Stability to noise, blur, attacks | Finds weaknesses early | Cannot show their absence | Growing practice |
| Formal verification | Proof over an input region | A real guarantee | Small networks, local properties | Research; aviation benchmarks |
| Runtime monitors, safety cage | Catches outside inputs and bad outputs | Handles the unknown | Misses look-alikes; costs availability | Flight demos (shadow mode) |
| Explanations | Which inputs drove a decision | Reveals shortcuts | Not proof; methods disagree | Reviews and audits |
| Simulation, hardware in the loop | System behaviour in many scenarios | Volume and rare cases | Sim-to-real gap | Standard for autonomy |
| Traceability, IV&V | What was built, checked by others | Audit and repeatability | Discipline and cost | NASA IV&V since 1993 |

### 53.3.6 Putting It Together: the Assurance Case

The evidence is joined by an **assurance case**: a structured argument that a system is acceptably safe (or secure) for a given use in a given domain. **Goal Structuring Notation (GSN)** draws it as a tree (Figure 53.8). Each goal is a claim. A strategy explains how it is broken into sub-goals. Each

sub-goal ends in evidence. Context and assumptions are stated, so that a reviewer can attack them.

### PRACTICE: WRITING AN ASSURANCE CASE FOR AN ONBOARD ML FUNCTION

1. **State the claim and its limits.** Write the top goal in one sentence naming the function, the hazard and the domain. "The detector does not make landing less safe inside its domain" is checkable; "the AI is safe" is not.
2. **Define the domain.** List every factor that changes the input — light, geometry, terrain, sensor state, radiation — with ranges. Note which factors the spacecraft can measure directly.
3. **Fix the criticality.** Run a hazard analysis. Decide which parts must reach which category, and design a cage so that the learned part can sit at category C or D.
4. **Choose a strategy.** Argue over the life cycle (requirements, data, learning, implementation, operations) and over the runtime protection. Assign each sub-goal an owner.
5. **Collect evidence for each sub-goal.** Datasheet and coverage report; tests per domain slice with confidence bounds; robustness and adversarial tests; proofs for the monitor; bit-exact checks on the flight computer; radiation and fault-injection results.
6. **Attack your own case.** List "defeaters": ways each claim could be false. Unseen terrain, label errors, a monitor blind spot. Add evidence or change the design until each is answered.
7. **Get independent review.** A team that did not build the system reviews the case and data; record their findings and your answers.
8. **Keep it alive.** Every retraining, new domain or anomaly in orbit reopens the case. Version it with the model.

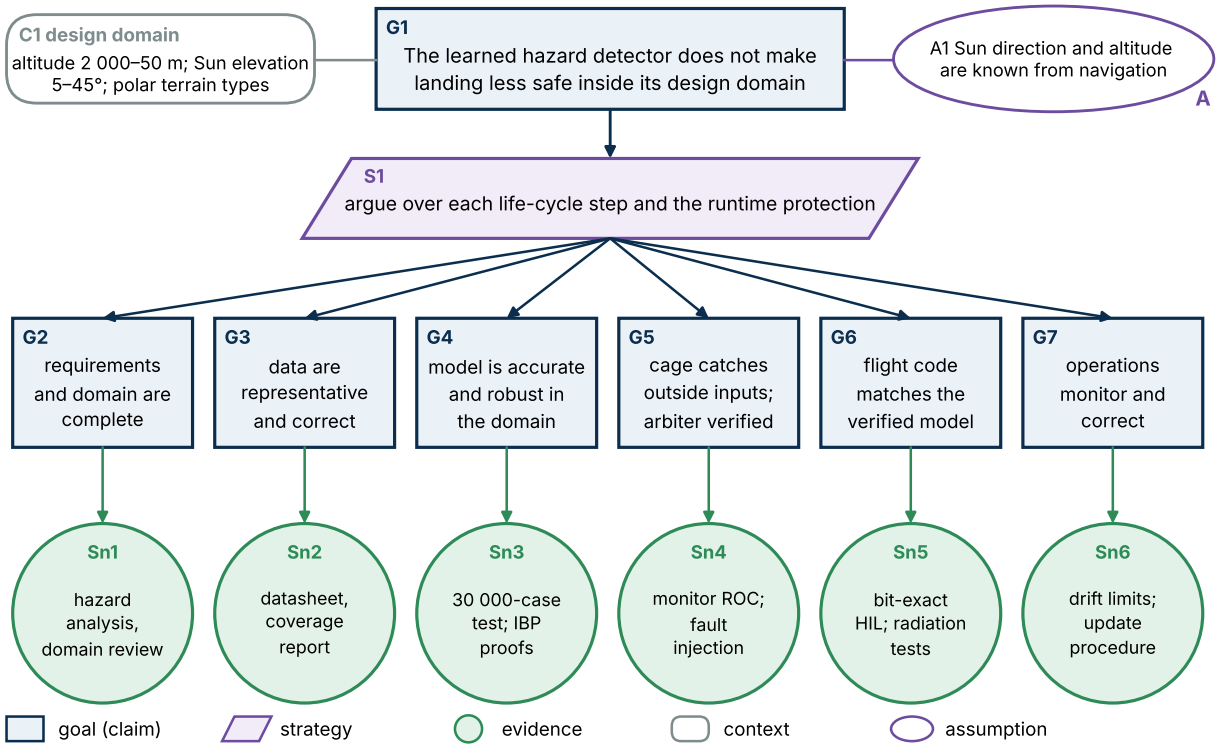



**FIGURE 53.8** An assurance case in Goal Structuring Notation for a learned hazard detector on a lunar lander (simplified). Each claim must end in evidence that a reviewer can inspect.

## 53.4 Security: Attacks on Learned Systems

The key idea of this section is that learning creates new ways to attack a system. An attacker can target the data, the model or the inputs, as well as the computers and links around them.

Learned systems add attacks at every stage of the life cycle (Table 53.4). NIST's taxonomy of adversarial machine learning (March 2025) groups them into evasion, poisoning and privacy attacks, plus supply-chain and prompt-injection attacks on generative AI.

**TABLE 53.4** Attacks on learned systems across the life cycle, with examples and defences.

| Attack | Stage | What the attacker needs | Example | Main defences |
|---|---|---|---|---|
| Evasion (adversarial examples) | Operation | Control of inputs | Patches hid aircraft from a detector (2020, Ch. 50) | Robust training, input checks, sensor fusion |
| Poisoning and backdoors | Training data | Write access to a data source | 0.01 % of a web-scale set for $60 (2023) | Provenance, hashes, data audits |
| Model extraction and inversion | Deployment | Query access | Copying a cloud analytics model | Rate limits, privacy-preserving training |
| Supply chain | Development | A shared model or library | About 100 malicious models on a public hub (2024) | Signed artefacts, safe file formats, scans |
| Prompt injection, leakage | Operation | Text the assistant will read | Instructions hidden in a log or document (Ch. 49) | Treat text as data; limit tools; human approval |
| Classic cyber attack | All | Network access | KA-SAT modems wiped (2022, Ch. 49) | Segmentation, encryption, NIST IR 8401 |

### 53.4.1 Adversarial Examples

An **adversarial example** is an input changed slightly, on purpose, so that a model gets it wrong. Such inputs were first described in 2013. The simplest way to make one is the **fast gradient sign method (FGSM)** of 2014. It moves every pixel by $\varepsilon$ in the direction that most increases the loss:

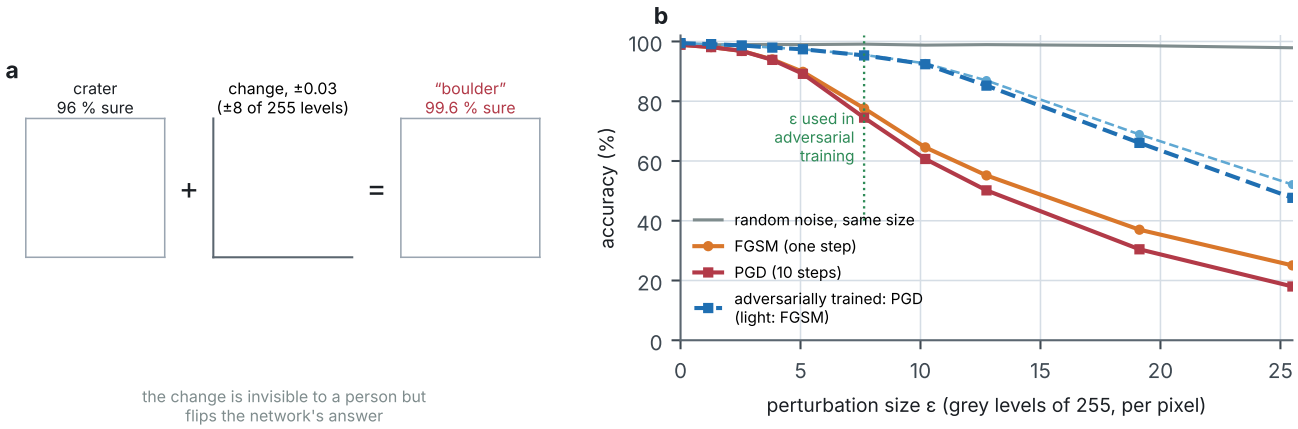
$$\mathbf{x}' = \mathrm{clip}_{[0,1]}\left(\mathbf{x} + \varepsilon\ \mathrm{sign}\,\nabla_{\mathbf{x}}\,\ell(f_\theta(\mathbf{x}), y)\right).$$

**Projected gradient descent (PGD)** repeats small steps of this kind and keeps the result within $\varepsilon$ of the original. It is the standard strong attack for testing.

Figure 53.9 attacks the patch classifier of Figure 53.1. Random noise of up to ±26 grey levels (of 255) hardly matters: accuracy never falls below 97.9 %. A crafted change of only ±8 levels cuts accuracy to 77.6 % with FGSM and 74.5 % with PGD. At ±13 levels PGD leaves 50.2 %, and at ±26 levels 18.0 %. The example crater, 96 % "crater" before the attack, becomes a 99.6 % "boulder", yet a person sees no difference. **Adversarial training**, which mixes attacked examples into training, helps: the retrained network keeps 95.3 % against PGD at ±8 levels and 85.2 % at ±13. It still falls to 47.7 % at ±26 levels.

How real is the threat in space? A digital attack needs access to the pixels — in the onboard computer, the ground pipeline or the data link — so normal cyber security is the first defence. **Physical** attacks need no such access. Printed patches, paint patterns and decoys can change

what a sensor sees (Chapter 50). Defences are layered: robust training, input checks, several sensors (optical and radar rarely share the same blind spot), and people reviewing important decisions.




Same network and patches as before (1 800 in-domain test patches). White-box attacks use the gradient of the loss with respect to the pixels; PGD = projected gradient descent, step ε/4, random start. The adversarially trained network saw FGSM examples with ε = 0.03 (about 8 grey levels) during training.

**FIGURE 53.9** Adversarial examples on the patch classifier. (a) A crater, an FGSM change of at most ±8 grey levels per pixel, and the result, which the network calls a boulder. (b) Accuracy against the size of the change for random noise, FGSM and PGD attacks, and for a network trained on adversarial examples.

### 53.4.2 Poisoning, Backdoors and the Supply Chain

**Data poisoning** inserts bad examples into training data. A **backdoor** is a poisoning that teaches the model a hidden trigger: the model behaves normally until a pattern appears, then outputs what the attacker wants. Large datasets built from the web are easy targets. In 2023 a team showed that, for about US$60, an attacker could have bought expired internet domains hosting images for two popular web-scale datasets. Through them, it could have controlled 0.01 % of a dataset of hundreds of millions of image–text pairs. Space datasets are smaller and better curated, but they are open and re-used, so labels, benchmarks and archives need provenance checks and cryptographic hashes.

**Pre-trained models** are a supply-chain risk like any third-party software. In February 2024 security researchers found about 100 models on a popular public model hub that ran hidden code when loaded, through an unsafe file format. Most were probably tests by other researchers, but one opened a remote connection to the user's computer. A downloaded model must be treated as untrusted code: loaded only in safe formats, scanned and signed.

A model offered as a service can also be **extracted** — copied by training on its answers — or probed to reveal its training data (**model inversion** and **membership inference**). Rate limits, coarse outputs and training that limits memorisation reduce the risk.

### 53.4.3 Language Models in Operations

Language-model assistants (Chapters 44 and 49) add two specific risks. In **prompt injection**, text that the assistant reads — a web page, an email, an anomaly report — contains instructions that it follows, even if the attacker never talks to the assistant. **Data leakage** happens when an assistant reveals confidential procedures, keys or personal data it has seen. The defences are architectural. Treat all retrieved text as data, never as commands. Give the assistant the fewest tools and permissions it needs. Keep a human approval step before any command reaches a spacecraft.

### 53.4.4 Securing AI-Enabled Space Systems

Learned components live inside ordinary space systems, so classical space cyber security comes first (Chapters 7 and 49). NIST's report IR 8401 (December 2022) applies its Cybersecurity Framework to the satellite ground segment. The Aerospace Corporation's **SPARTA** framework (first released in October 2022) catalogues attacks on spacecraft: in 2026 it listed 9 tactics and 85 techniques. In Europe, the NIS2 directive already covers operators of space ground infrastructure. The proposed EU Space Act of June 2025 would add space-specific cyber rules; it was still being negotiated in October 2026.

> DATA BOX — ATTACKING SATELLITES ON PURPOSE, 2023
>
> - **OPS-SAT (April 2023).** In an exercise run with ESA before the CYSAT conference in Paris, four researchers from Thales took control of the application environment of ESA's OPS-SAT test satellite. They inserted code that could, for example, mask areas in its images. ESA kept control of the satellite throughout.
> - **Moonlighter (June 2023).** The Aerospace Corporation and the US Space Systems Command built Moonlighter, a CubeSat designed to be hacked. It launched on 5 June 2023 on a cargo flight to the ISS.
> - **Hack-A-Sat 4 (August 2023).** At the DEF CON 31 conference in Las Vegas, five finalist teams attacked Moonlighter in orbit. An Italian team, mHACKeroni, won the US$50 000 first prize.
> - **Why it matters for AI.** The OPS-SAT attack could alter imagery at the source. A learned function downstream would analyse falsified data with full confidence, so integrity checks belong before the model.

A sound defence for an AI-enabled mission combines four layers:

- **Protect the pipeline**: signed data, models and code; separate networks for training, ground operations and commanding; secure model upload with rollback (Chapter 45).
- **Harden the model**: adversarial and corruption tests, robust training and monitoring of input statistics.
- **Contain the model** in a safety cage, so that a fooled model cannot command a hazardous action.
- **Keep people in the loop** for irreversible decisions, and train them to spot a model under attack.

## 53.5 Governance, Law and Ethics

The key idea of this section is that AI law is moving fast and differs between regions. In October 2026 it touches space mostly through data, ground services and products sold on Earth, and less through onboard functions.

### 53.5.1 The European Union AI Act

The **AI Act**, Regulation (EU) 2024/1689, is the first broad AI law. It entered into force on 1 August 2024 and sorts AI uses by risk. Unacceptable practices, such as social scoring, are banned. These bans, and a duty to build staff AI literacy, apply from 2 February 2025. **High-risk** systems — for example safety components of critical infrastructure, or AI in products under EU safety law such

as medical devices — need risk management, data governance, logging, human oversight, accuracy and robustness. They also need conformity assessment. Providers of **general-purpose AI models** have had transparency and copyright duties since 2 August 2025. Fines for banned practices reach €35 million or 7 % of world turnover. AI used only for military or national-security purposes is excluded.

In 2026 a "digital omnibus" amendment, agreed in May and published in July, delayed the rules for stand-alone high-risk systems to 2 December 2027, and for high-risk AI in regulated products to 2 August 2028. Transparency duties, such as labelling AI-generated content, still apply from 2 August 2026. Generative systems already on the market have until 2 December 2026 to add machine-readable marks to their output. For space engineers the main routes into the Act are data services used in critical infrastructure and, through EASA's rules, AI in aviation; NPA 2025-07 is partly a response to the Act. Most onboard spacecraft functions are not listed as high-risk, but customers increasingly ask for the same evidence.

### 53.5.2 The United States

US policy turned in 2025 from managing AI risk to accelerating adoption. Executive Order 14110 of 2023 was revoked on 20 January 2025. Executive Order 14179 (23 January 2025) and America's AI Action Plan (July 2025) stress innovation, infrastructure and exports. Memoranda of April 2025 tell federal agencies to appoint chief AI officers, list their AI uses and apply minimum practices to "high-impact" ones. Executive Order 14365 (December 2025) directs federal action against state AI laws judged "onerous". A national policy framework of March 2026 asked Congress to pre-empt conflicting state laws. As of October 2026 there is no general federal AI statute, and state AI laws continue to apply. Military AI rules are described in Chapter 50.

### 53.5.3 China, the United Kingdom and Others

**China** regulates specific services. Interim measures for generative AI services, with registration of algorithms, took effect in August 2023. Rules requiring visible and hidden labels on AI-generated content took effect on 1 September 2025. An amendment adding the first AI article to the Cybersecurity Law, adopted in October 2025, took effect on 1 January 2026. The **United Kingdom** has no general AI law. It relies on existing regulators applying shared principles. Its AI Safety Institute was renamed the AI Security Institute in February 2025.

### 53.5.4 International Agreements

The **Council of Europe Framework Convention on AI** is the first binding international treaty on AI and human rights. It opened for signature in Vilnius on 5 September 2024. The EU ratified it in May 2026, and by the latest treaty records no other party had. It needs five ratifications, including three by Council of Europe member states, to enter into force. At the United Nations, the **Global Digital Compact** (September 2024) led to resolution 79/325 of August 2025. It created an Independent International Scientific Panel on AI, whose 40 members were appointed in February 2026, and a **Global Dialogue on AI Governance**. The Dialogue held its first session in Geneva on 6–7 July 2026, where the Panel presented a preliminary report. A series of summits — Bletchley Park (2023), Seoul (2024), Paris (2025) and New Delhi (February 2026) — produced non-binding declarations. The New Delhi Declaration had 91 signatories, including the USA, the UK and China. Table 53.5 summarises the main instruments.

**TABLE 53.5** Main AI governance instruments as of October 2026. Military AI is covered in Chapter 50.

| Instrument | Jurisdiction | Type | Key dates | Status, October 2026 |
|---|---|---|---|---|
| AI Act, Reg. (EU) 2024/1689 | European Union | Binding, risk-based | In force Aug 2024; bans Feb 2025 | High-risk rules delayed to Dec 2027 and Aug 2028 |
| Executive orders, Action Plan, OMB memos | United States | Executive policy | Jan–Jul 2025; Dec 2025 | No general federal AI law; state laws apply |
| Generative-AI measures, labelling rules | China | Binding regulations | Aug 2023; Sep 2025 | In force; AI added to Cybersecurity Law Jan 2026 |
| Sector-based principles | United Kingdom | Regulators' guidance | 2023–25 | No general AI law |
| Framework Convention on AI | Council of Europe | Treaty | Opened Sep 2024 | Ratified by the EU; not yet in force |
| Global Digital Compact; Scientific Panel; Global Dialogue | United Nations | Non-binding | 2024; 2025; Jul 2026 | Panel report Jul 2026; next Dialogue May 2027 |
| New Delhi Declaration | 91 signatories | Non-binding | Feb 2026 | Signed |

### 53.5.5 Export Controls

Space hardware has long been export-controlled; the USA eased licensing of some spacecraft items to close allies in October 2024. Advanced AI chips have been controlled since 2022. A US rule of January 2025 that would have limited chip and model-weight exports worldwide was rescinded in May 2025. In January 2026 the USA moved some high-end chips to case-by-case licensing for China, combined with a 25 % import duty. Engineers buying processors for an AI payload, or sharing models and training data with foreign partners, should check controls early (Chapter 50).

### 53.5.6 Agency Principles and Human Oversight

NASA published a *Framework for the Ethical Use of Artificial Intelligence* in April 2021. It sets six principles: fair; human-centric and societally beneficial; explainable and transparent; accountable; secure and safe; and scientifically and technically robust. NASA appointed its first chief AI officer in May 2024. By October 2026 NASA's web pages referred to the same framework as one for "super intelligence". ESA works through the ECSS standards and EU law. Defence users follow principles such as the US Department of Defense's five of 2020: responsible, equitable, traceable, reliable and governable.

**FIGURE 53.10** The Artemis II flight control team in the White Flight Control Room at NASA's Johnson Space Center during the crew's lunar flyby on 6 April 2026. Every principle for trustworthy AI comes back to people like these, who must understand, question and, when necessary, overrule the tools they use. Image: NASA/JSC (ROBERT MARKOWITZ NASA-JSC).

All these documents call for **meaningful human oversight**. In practice that means operators who understand what a tool can and cannot do (Figure 53.10). They need the time and information to question it, and the authority to overrule it. It also means guarding against **automation bias**: the tendency to accept a machine's answer because it is usually right. Interfaces should show confidence and reasons, and training should include cases where the tool is wrong.

### 53.5.7 Privacy and Dual Use

Satellites now see the ground at 30 cm or better, and some revisit a site many times a day (Figure 53.11). One image reveals little about a person. But learned detectors can track vehicles, ships and crowds over months and link them to other data, building a "pattern of life" that can identify people. The results can then be personal data under laws such as the EU's General Data Protection Regulation. Good practice limits resolution and retention to what a task needs, aggregates results, and logs who queried what. Almost every tool in Part XIII is also **dual-use**: the same detector serves a port authority and an army (Chapter 50).

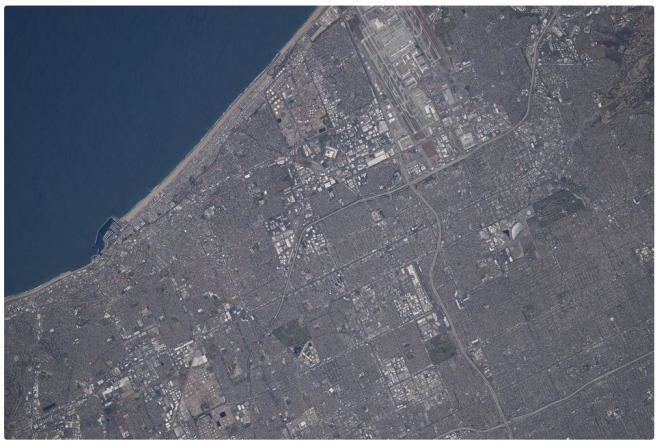


**FIGURE 53.11** Los Angeles International Airport (top) and the surrounding city, photographed from the International Space Station on 29 May 2026. Learned detectors can count aircraft, cars and ships in images like this; combined over time, such counts raise questions of privacy and dual use. Image: NASA/JSC.

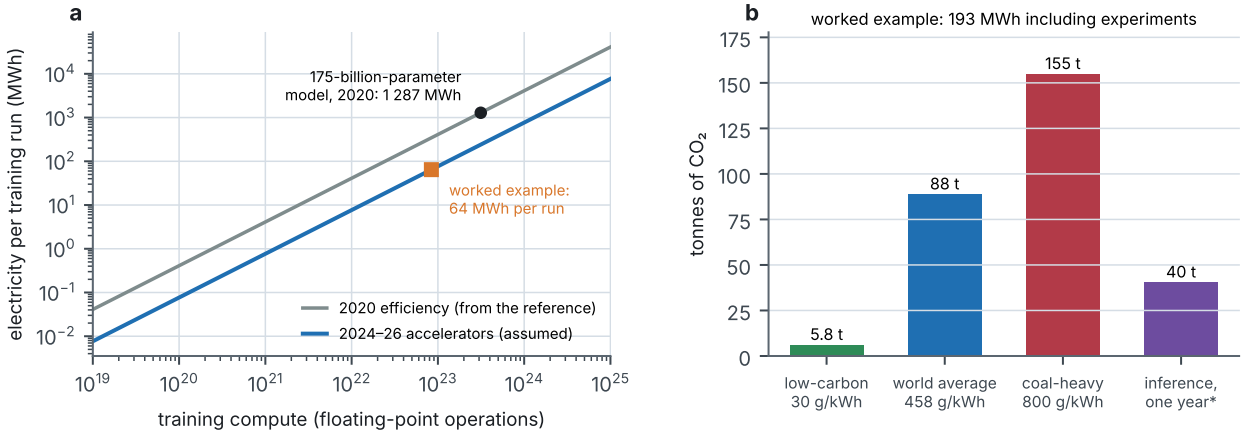
### 53.5.8 The Environmental Cost of Computing

The International Energy Agency estimated that data centres used about 415 TWh in 2024, about 1.5 % of the world's electricity. It expects this to roughly double, to about 945 TWh, by 2030, with AI the main driver (Figure 53.12). The energy for one training run can be estimated from the compute: a model with $N$ parameters trained on $D$ tokens needs about $C = 6ND$ floating-point operations (Chapter 44). Then

$$E \approx \frac{C}{F_{\text{peak}}\, u}\, P_{\text{acc}}\, k_{\text{host}}\, \text{PUE},$$

where $F_{\text{peak}}$ is the peak speed of one accelerator, $u$ the fraction of it actually used, and $P_{\text{acc}}$ its power. The factor $k_{\text{host}}$ adds host computers and networking, and the power usage effectiveness (PUE) adds cooling and other overheads (Figure 53.13).

**FIGURE 53.12** City lights from Tokyo to Osaka and Hiroshima, photographed from the International Space Station on 15 September 2025. Data centres for AI are a fast-growing part of the electricity demand that lights scenes like this. Image: NASA/JSC.




Compute C = 6 N D (N parameters, D training tokens). Assumed: 989 TFLOP/s peak per accelerator, 40 % utilisation, 0.7 kW board power × 1.3 for host and network × 1.2 data-centre overhead (PUE). Reference: Patterson et al. (2021). *1 million queries a day at 0.24 Wh each (a median reported by one large provider in 2025), world-average grid.

**FIGURE 53.13** Energy and emissions of training. (a) Electricity per training run against compute, for 2020-era efficiency (from a published estimate for a 175-billion-parameter model) and for assumed 2024–26 accelerators. (b) Emissions of the worked example on three electricity grids, and of one year of a popular inference service.

**EXAMPLE: ENERGY AND CARBON TO TRAIN A MID-SIZE MODEL**

Estimate the cost of training a 7-billion-parameter model on 2 trillion tokens — a mid-size language model, or a large Earth-observation foundation model (Chapter 44).

1. **Compute.** $C = 6 \times 7 \times 10^{9} \times 2 \times 10^{12} = 8.4 \times 10^{22}$ operations.
2. **Accelerator time.** Assume 989 × $10^{12}$ operations per second at peak and 40 % utilisation: $8.4 \times 10^{22} / (3.96 \times 10^{14})$ s = **59 000 accelerator-hours**, or 4.8 days on 512 accelerators.
3. **Power.** 0.7 kW per accelerator × 1.3 for hosts and network × PUE 1.2 = 1.09 kW. One run: 59 000 h × 1.09 kW = **64 MWh**.
4. **Real projects.** Failed runs, tuning and evaluation often triple the total: **193 MWh**.
5. **Carbon.** On a low-carbon grid (30 g $CO_2$/kWh): **5.8 t**. At the 2025 world average of 458 g/kWh: **88 t**. On a coal-heavy grid (800 g/kWh): **155 t**.
6. **Use.** One million queries a day at 0.24 Wh each — a median reported by one large provider in 2025 — need 88 MWh a year, or 40 t $CO_2$ at the world average. After about 2.2 years the answers have used more energy than the whole training effort. At ten million queries a day this takes under three months.

For comparison, a 175-billion-parameter model trained in 2020 was estimated at 1 287 MWh and 552 t $CO_2$; the assumed modern hardware uses about 5 times less energy per operation. The grid changes the carbon by a factor of 27.

Onboard inference uses only a few watts (Chapter 45); the large costs lie on the ground.

### 53.5.9 Workforce and Skills

Trustworthy AI needs people who understand both machine learning and safety-critical engineering, and they are rare. Data scientists seldom know hazard analysis, and many assurance engineers have never trained a model. Mixed teams and training on the new standards are closing the gap.

## 53.6 Outlook to 2035

The key idea of this section is to separate what has happened from what may happen. Projections are marked with a confidence level.

Figure 53.14 sets out one view of the next decade. First, the facts as of October 2026. Learned weather forecasts have been operational at the European Centre for Medium-Range Weather Forecasts (ECMWF) since February 2025 (Chapter 47). Foundation models for Earth observation and the Sun are openly available (Chapter 44). Onboard AI has flown on several small satellites (Chapters 45 and 46), and a neural network has flown a satellite's attitude for one orbit. Single operators already run autonomous collision avoidance across thousands of satellites (Chapter 48). No learned function has yet been certified as the sole protection against a catastrophic failure in space or aviation.

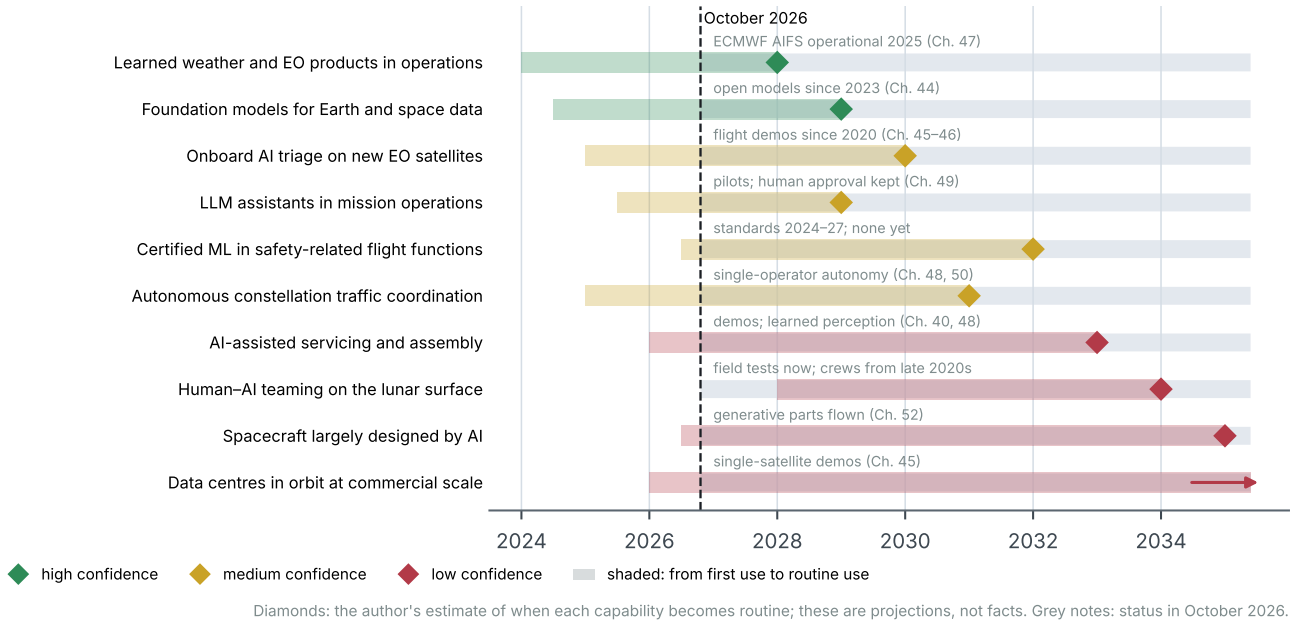



**FIGURE 53.14** An outlook to 2035. Shaded bars run from first use to the estimated year of routine use (diamonds), coloured by the author's confidence. Grey notes give the status in October 2026. These are projections, not facts.

The projections, with confidence:

- **Foundation models for Earth and space (high confidence).** Shared pre-trained models will be the normal starting point for EO, weather and heliophysics products by about 2030. Assuring a *base model* then becomes a shared task, like qualifying a processor.
- **Certified ML in safety-related functions (medium).** Aviation will probably certify Level 1–2 functions first, once ED-324 and EASA's rules are final. Space will follow at ECSS category B, inside safety cages, in the early 2030s.
- **Autonomous constellations (medium).** Coordination between operators — who moves when two satellites meet — needs shared rules and data more than better models (Chapters 21 and 50).
- **AI-designed spacecraft (low).** Algorithms already shape parts: evolved antennas flew on NASA's ST5 in 2006 (Chapter 52). A spacecraft designed largely by AI is unlikely before 2035, because every design must still be verified.
- **Human–AI teaming on the Moon (low).** Artemis astronauts are expected to land from about 2028 (Figure 53.15). Assistants for traverses, procedures and medical care will be tried first in analogue tests (Figure 53.16), and trusted only as far as they can explain themselves.
- **AI-assisted servicing and orbital computing (low).** Docking life-extension vehicles have served satellites commercially since 2020 (Chapter 40). Robotic servicing with learned perception and data centres in orbit are still at the demonstration stage (Chapters 40 and 45). Both depend more on hardware and economics than on AI.

**FIGURE 53.15** NASA astronauts Joe Acaba and Mark Vande Hei test Lunar Outpost's Pegasus lunar terrain vehicle prototype at Johnson Space Center in September 2026. NASA selected Lunar Outpost and Astrolab in May 2026 to build the first lunar terrain vehicles. Image: NASA/JSC (NASA/Bill Stafford).

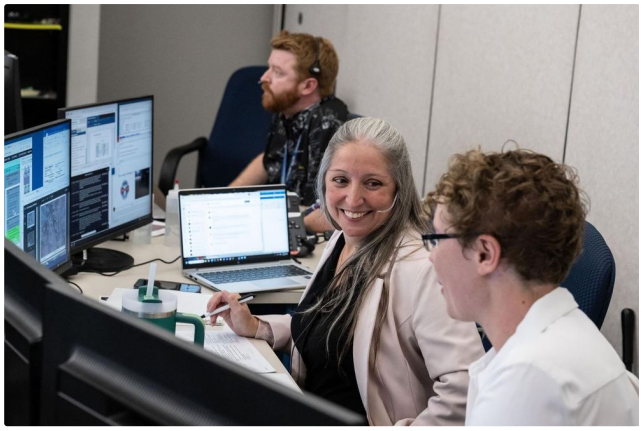


**FIGURE 53.16** Flight controllers at Johnson Space Center guiding simulated moonwalks in Arizona's San Francisco Volcanic Field during the JETT 5 field test in May 2024. Analogue tests like this are where decision-support tools for surface operations can be tried before flight. Image: NASA/JSC (James Blair - NASA - JSC).

Several problems remain open:

- **Assuring foundation models** trained by others on data nobody can fully inspect.
- **Monitors with guarantees**: detecting inputs outside the domain that look familiar, as in Figure 53.5.
- **Verification at scale** for convolutional and transformer networks, and for properties beyond local robustness.
- **Assurance of learning in orbit**, when a model updates itself from new data.
- **Agents that act**: language-model systems that use tools need limits that can be checked.
- **Common evidence formats**, so that a model qualified by one organisation can be re-used by another.

## 53.7 Part XIII in Review

Part XIII began with the foundations of machine learning and ends with trust. Table 53.6 gathers the main lesson of each chapter.

**TABLE 53.6** Part XIII in one table: the lesson of each chapter and the most mature use as of October 2026.

| Chapter | Topic | Central lesson | Most mature use, 2026 |
|---|---|---|---|
| 43 | AI foundations | Test on data split by place and time; report uncertainty | Classical ML in data products |
| 44 | Deep learning, foundation models | Pre-train once, adapt many times; check calibration | Open EO and weather models |
| 45 | Data, hardware, edge AI | Data and radiation, not algorithms, limit onboard AI | Cloud screening, triage demos |
| 46 | Earth observation | Domain shift and area estimation decide real accuracy | Land cover, floods, methane |
| 47 | Weather, climate, space weather | Learned forecasts rival physics but need its analyses | ECMWF AIFS operational |

| Chapter | Topic | Central lesson | Most mature use, 2026 |
|---|---|---|---|
| 48 | Autonomy and robotics | Learning helps perception; verified logic keeps control | Rover and avoidance autonomy |
| 49 | Operations and networks | Alert budgets, not accuracy, decide usefulness | Telemetry anomaly tools |
| 50 | SSA, defence, security | Physics carries most of the load; people keep decisions | Tracking, imagery triage |
| 51 | Science and human spaceflight | ML is a funnel; discoveries need independent checks | Planet vetting, alert brokers |
| 52 | Design, manufacturing, launch, business | AI proposes; physics, test and people verify | Generative parts, inspection |
| 53 | Trustworthy AI | Assure data, model and runtime; contain what cannot be proved | Handbooks; shadow-mode demos |

Three threads run through all eleven chapters. First, **physics and engineering still carry most of the load**: learning works best as a fast approximation, a filter or a perception layer inside a well-understood system. Second, **data decide everything** — how they were collected, labelled and split, and how far operation differs from training. Third, **trust is earned by evidence and architecture together**. A model is trusted when its domain is defined, its performance honestly measured, its failures contained and its behaviour visible to people. Space has always demanded this discipline; machine learning makes it more necessary, not less.

KEY TAKEAWAYS

- Learned behaviour comes from data, not a written specification, and fails silently outside its design domain. A crater classifier fell from 99.2 % to 34.3 % when the Sun came from the other side, while noise barely mattered.
- The first ML assurance documents exist — ECSS-E-HB-40-02A (2024), EASA's concept papers and NPA 2025-07, ISO/IEC 42001 and TR 5469 — but they cover lower criticality. No learned function is yet the sole protection against a catastrophic failure.
- Evidence combines a defined domain with data coverage, statistical testing (about 3/p failure-free tests: 29 956 for $10^{-4}$), robustness tests, formal proofs for small networks, traceability and independent review.
- A safety cage contains what cannot be proved. Here it cut hazardous errors from 7.1 % to 2.0 % for 12.9 points of availability; checking the domain directly (Sun angle) beat any learned monitor for that factor.
- Attacks target data, models and inputs: changes of ±8 grey levels cut accuracy to 74.5 %, and poisoning web-scale data can cost tens of dollars. Defences are layered: secure pipelines, robust training, cages and people.
- AI law diverges: the EU AI Act's high-risk rules now apply from December 2027 and August 2028; the USA has no general AI statute; China regulates services and labels; international agreements are mostly non-binding.
- Training a 7-billion-parameter model takes about 190 MWh including experiments; the grid changes its emissions 27-fold, and a popular service can use more energy answering queries than training.

### REVIEW QUESTIONS

1. Write the empirical and operational risks. Why does a low test error say nothing about performance after a distribution shift? Give two space examples.
2. In Figure 53.1, why is the classifier robust to noise and gain but not to the Sun's direction? What would you add to its design domain and its test plan?
3. Compare ECSS criticality categories, NASA software classes and DO-178C levels. Why do structural-coverage measures such as MC/DC say little about a neural network?
4. Describe the W-shaped learning-assurance process. Which steps are new compared with classical software, and what evidence does each produce?
5. Show how interval bound propagation passes a box through a linear layer and a ReLU. Why is it sound but incomplete? What did Figure 53.3 show about the size of the gap?
6. Draw a safety cage for a learned star-tracker or crater-navigation function. Which parts need the highest assurance category, and why?
7. Using the Mahalanobis monitor results, explain why some outside inputs are hard to detect. How does a direct domain check change the cage's trade between availability and hazard?
8. How many independent failure-free tests demonstrate a failure probability below $10^{-3}$ with 99 % confidence? Why can testing alone never support $10^{-9}$ per hour?
9. Explain FGSM and PGD. Why does random noise of the same size barely affect the classifier while a crafted change does? List four defences for a spacecraft image pipeline.
10. A company will train a 7-billion-parameter foundation model and sell flood maps from it to an EU water authority. Which legal, security and environmental questions should it answer first?